Weekly 엔터프라이즈 중장기 기술 로드맵 (Product Roadmap Plan)

문서 버전: v0.320.0 (현재) ~ v1.0-VISION

작성일자: 2026년 8월 9일

최종 정렬: 2026년 8월 21일 (v0.25.0 기준)

문서 분류: 비즈니스 및 아키텍처 중장기 로드맵 (Strategic Product Roadmap)

---

1. 현재 위치와 다음 방향

v0.10.0까지 **"주간보고를 잘 만드는 시스템"** 은 상당 부분 완성됐습니다. 작성·승인, 기간 집계, 경영 인사이트, PPTX 내보내기, Confluence 근거 기반 초안, 내용 검색, 관리자 분석이 모두 동작합니다.

다음 단계의 성격은 다릅니다. **"회사가 지금 무슨 일을 하고 있고, 무엇이 바뀌었고, 무엇이 막혀 있고, 왜 그렇게 결정했는지 아는 시스템"** 으로 전환합니다. 이를 위한 핵심 축은 네 개입니다.

```
================================================================================
                        [Weekly 다음 단계 4개 핵심 축]
================================================================================
 1. WorkItem         주차를 초월하는 업무 식별자. 나머지 세 축의 전제 조건
 2. 주간 변화 요약     지난주 대비 신규·진척·완료·지연·재개 자동 판별
 3. Decision & Ask   결정사항과 상위 조직 요청을 이슈와 분리해 기록
 4. Dependency       업무 간 선행·차단 관계로 부서 간 병목 조기 발견
================================================================================
```

1.1 왜 WorkItem이 먼저인가

현재 데이터 모델은 `Report → ReportItem`이며, 같은 업무가 매주 **새 행으로 다시 생성**됩니다. 주차를 넘는 업무 동일성은 기간 집계에서 제목 정규화와 토큰 포함 관계로 **추정**하고 있을 뿐, 저장된 사실이 아닙니다.

변화 요약, Aging, Dependency, Risk 예측은 모두 "이 업무가 지난주의 그 업무와 같은가"에 답할 수 있어야 성립합니다. WorkItem 없이 각 기능이 제각각 추정 로직을 다시 구현하면 기능마다 판정이 어긋납니다.

---

2. 완료된 마일스톤 (v0.1.0 ~ v0.25.0)

```
================================================================================
 [v0.1.0]  Single Container, PPTX Zip Engine, Keycloak OIDC, RBAC, MCP 1.0
 [v0.2.0]  자유 텍스트 AI 초안, 과거 PPTX Import, 사내 AI Gateway
 [v0.3.0]  Confluence 6.9.1 증분 수집과 사용자별 자동 초안
 [v0.4.0]  주간보고 생애주기와 연동 개선
 [v0.5.0]  보고서 복제, 업그레이드 안전 비밀 설정(WEEKLY_ENCRYPTION_KEY)
 [v0.6.0]  기간 업무보고(월·분기·반기·연간), 중복 제거, 경영 인사이트, 업무 시각화
 [v0.7.0]  AI 원자 사실 구조화, Confluence 근거 검증, PPTX Import 구조 보존
 [v0.7.1]  보고서 수정 장애 복구, 문자 수 기준 입력 검증
 [v0.7.2]  비밀 설정 유실 방지 기동 차단, PPTX 드래그 앤 드롭
 [v0.8.0]  화면 캡처 슬라이드, 기간 보고 전용 분면 레이아웃
 [v0.9.0]  빠른 이동 팔레트, 보고서 내용 검색, 화면 상태 딥링크
 [v0.9.1]  가져오기 검토 화면 반응형 정리
 [v0.10.0] 관리자 분석: 한국어 키워드 추출, 워드 클라우드, 조직·참여 분석
 [v0.11.0] WorkItem 도입, 업무 Aging 추적, 상위 조직 요청 필드
 [v0.12.0] 3단계 검색(정확·표기 유사·의미 유사), 표기 변형 합산 워드 클라우드
 [v0.13.0] 회의 모드와 발표 모드, 경영 요약, 조직 간 중복·협업 분석, 인수인계
 [v0.14.0] PPTX 내보내기가 있는 모든 화면의 발표 모드
 [v0.14.1] 발표 모드 화면 캡처 슬라이드
 [v0.14.2] 첨부 파일 유실 진단, 발표 모드 이미지 정렬 수정
 [v0.15.0] 지난주 계획 이어받기, 임베딩 신선도 판정, 백업·복구 도구
 [v0.16.0] 편집 내용 유실 방지, 업무 병합·분리, 로그인 시도 제한
 [v0.17.0] 주간 변화 요약, 보고 품질 점검, 재개 업무 판정 누락 수정
 [v0.18.0] 보고서 조회 색인, 제출 마감 설정과 시간대 오판 수정, 기간 보고의 업무 식별 일원화
 [v0.19.0] CSV 수식 주입 차단, 세션 만료 복구, PPTX 내보내기 메모리 축소
 [v0.20.0] 발표 모드 자동 맞춤, 발표자 화면, 슬라이드 목록·번호 이동
 [v0.21.0] 발표 테마 3종, 회사 템플릿 강조색 연동, 읽을 수 있는 최소 글자 크기
 [v0.22.0] 기동 차단 해소와 재개 가능한 백필, 휴대폰 메뉴 접근성
 [v0.23.0] 주차 격자 변경 안전장치, 조회 가능한 감사 이력, 오류 추적 ID 노출
 [v0.24.0] 작성 화면 우선순위 정리, 저장 상태 표시, 대화상자 동작 통일
 [v0.25.0] 업무 인사이트 응답 상한과 순위, 제목 토큰화 재사용
================================================================================
```

기존 로드맵의 다음 항목은 계속 유효하며 이 문서의 후반 단계에 배치합니다.

- **다중 PPTX 템플릿**: 사업부·부서별 서로 다른 양식 동적 선택
- **미제출자 자동 알림**: Slack·Teams·사내 메일 (v0.10.0의 미제출 현황 데이터를 그대로 활용)
- **Source-to-Report Agent**: 커밋·결재·Jira 근거 수집 후 후보 생성

---

3. 버전별 계획

3.1 v0.11 — WorkItem과 업무 Aging (완료)

- **WorkItem 도입**: `id`, `title`, `owner`, `organization`, `status`, `startDate`, `dueDate`, `progress`, `parentWorkItemId`. `ReportItem`은 `workItemId`로 연결되어 주차별 스냅샷 역할만 맡습니다.
- **과거 데이터 시딩**: 기간 집계의 제목 정규화·토큰 포함 병합 로직을 재사용해 기존 보고서에서 WorkItem 후보를 생성하고, 사용자가 확인·분리·병합합니다. 자동 판정을 확정으로 쓰지 않습니다.
- **업무 Aging**: WorkItem 단위로 지속 주차, 이월 횟수, 동일 이슈 지속 기간, 진척 정체 구간을 추적합니다. 현재 기간 집계가 계산하는 값을 업무 생애주기로 옮깁니다.
- **Management Ask**: 이슈와 분리된 `상위 조직 요청` 필드를 함께 도입합니다. 난이도가 가장 낮고 임원 보고 가치가 가장 큰 항목이므로 WorkItem과 같은 릴리즈에 싣습니다.

3.2 v0.12 — 검색 품질과 선택 확장 활용 (완료)

로드맵 후반의 의미 기반 검색을 앞당겨 기반만 먼저 놓았습니다. 4.3의 선행 결정이 실제 운영 환경에서 확인 가능해졌기 때문입니다.

- **3단계 검색**: 정확 일치 → `pg_trgm` 표기 유사 → `pgvector` 의미 유사. 앞 단계 결과가 5건 미만일 때만 다음 단계를 수행하고, 각 단계는 앞 단계가 찾은 보고서를 건너뜁니다.
- **확장은 선택**: 두 확장이 모두 없어도 정확 일치 검색이 그대로 동작합니다. 마이그레이션은 확장이 없으면 활성화를 건너뛰고 진행합니다.
- **표기 변형 합산 워드 클라우드**: 띄어쓰기만 다른 표기를 하나로 합산합니다. 유사도가 아니라 공백 제거 후 정확 일치로 판정합니다(4.3의 오판 금지 원칙 적용).
- **임계값은 설정으로**: 코사인 점수 범위는 임베딩 모델마다 다르므로 고정값으로 둘 수 없습니다. 관리자 화면의 임베딩 현황과 함께 조정합니다.

3.3 v0.13 — 회의·경영 관점과 조직 간 업무 관계 (완료)

WorkItem이 자리 잡으면서 로드맵 후반의 Meeting Mode(구 3.6)와 Executive Digest(구 3.9)를 앞당겨 구현했습니다. 두 기능 모두 WorkItem 스냅샷에서 결정적으로 파생되므로 선행 조건이 이미 충족돼 있었습니다.

- **회의 모드와 발표 모드**: 결정 필요·신규 이슈·지속 이슈·변경점·보고 누락만 남기고, 전체 화면 슬라이드로 회의에서 바로 진행합니다.

- **경영 요약**: 관측 사실의 가중 합으로 최대 10건을 선정하고 선정 근거를 항상 함께 제시합니다(4.3 원칙).
- **업무 인사이트**: 조직 간 중복 의심, 유사 업무, 협업 지도, 반복 운영 업무. 모두 사람이 확인할 후보로 제시합니다.
- **인수인계**: 진행 경과, 이슈 해소 여부, 미해결 사항을 기록에서 모읍니다.
- **과거 사례 검색**: 자연어로 유사 업무와 이슈 해결 경과를 찾습니다. v0.12의 임베딩을 선택 가속 경로로 사용합니다.

3.4 v0.14 — 발표 모드 일반화 (완료)

발표 모드를 회의 모드 전용에서 분리해, **PPTX를 내보낼 수 있는 모든 화면**에서 사용할 수 있게 했습니다. 내보내기와 발표는 같은 내용을 다른 매체로 전달하는 것이므로 한쪽만 제공할 이유가 없습니다.

- 대상: 내 주간보고, 과거 보고, 팀 주간보고, 기간 업무보고, 회의 모드
- 발표 컴포넌트는 슬라이드 배열만 받으며 보고서·기간 집계·회의 안건을 알지 못합니다. 덱 구성은 각 화면이 담당합니다.
- 내 주간보고는 **저장하지 않은 편집 내용도 그대로** 발표합니다. 작성 직후 그대로 설명하는 경우가 많기 때문입니다.
- v0.14.1에서 첨부한 화면 캡처도 슬라이드로 포함합니다. 배치는 첨부 패널의 `본문 앞`·`본문 뒤` 표시를 따르며, PPTX와 달리 표지를 항상 먼저 보여 줍니다.

3.5 v0.15 — 작성 보조와 데이터 신뢰성 (완료)

v0.15.0은 이 단계의 원래 주제인 주간 변화 요약보다, 그 앞에 있던 세 가지를 먼저 처리했습니다. 두 가지는 조용히 틀린 답을 내고 있었고, 하나는 매주 모든 사용자가 치르는 비용이었습니다.

- **지난주 계획 이어받기 (완료)**: 작성 화면이 직전 보고서의 차주 계획을 해당 업무 옆에 그대로 보여 주고, 아직 보고하지 않은 계획은 목록으로 모아 한 번에 업무 항목으로 옮깁니다. 짝짓기는 WorkItem 식별자 우선, 없으면 공백 제거 후 정확 일치입니다(4.5의 원칙 적용). 직전 *주차*가 아니라 직전 *보고서* 기준이므로 한 주를 건너뛰어도 이어집니다.
- **임베딩 신선도 판정 (완료)**: v0.11에서 보고 항목을 행 갱신 방식으로 바꾼 뒤, 임베딩 대상 판정이 "임베딩 행이 없는 항목"이어서 **수정된 내용이 영원히 재임베딩되지 않았습니다.** 판정을 본문 해시 비교로 바꾸고, 관리자 화면에 갱신 대기 건수를 노출합니다. 해시는 PostgreSQL이 계산합니다. 판정이 SQL 조건이어야 하기 때문입니다.
- **백업·복구 도구 (완료)**: 4.2의 두 저장소 문제를 문서가 아니라 도구로 다룹니다. `scripts/weekly-backup.sh`가 DB와 상태 볼륨을 같은 복구 지점으로 묶고, 참조된 첨부 파일이 실제로 보관됐는지 검증

합니다.

- **주간 변화 요약 (v0.17.0에서 완료)**: 아래 3.5.1을 보십시오.
- **보고 품질 점검 (v0.17.0에서 완료)**: 아래 3.5.1을 보십시오.
- **AI 표현 점검(선택, 남음)**: 모호한 표현과 근거 없는 완료 주장은 AI Gateway가 켜져 있을 때만 추가로 제시합니다. 규칙 기반 점검은 오프라인에서도 항상 동작합니다.

3.5.1 v0.17 — 주간 변화 요약과 보고 품질 점검 (완료)

- **판정은 하나만 둡니다.** 변화 분류를 회의 모드와 변화 요약이 **같은 함수**로 수행합니다. 1.1이 WorkItem을 먼저 도입한 이유와 같습니다. 화면마다 비교를 다시 구현하면 같은 업무가 한 곳에서는 완료로, 다른 곳에서는 진척으로 보이고 읽는 사람이 둘 다 믿지 않게 됩니다.
- **재개 판정 누락 수정**: v0.13의 회의 모드는 한 주 이상 쉬었다가 다시 보고된 업무를 **어느 구획에도 넣지 못하고 조용히 버리고 있었습니다.** 조용했다 돌아온 일은 회의가 가장 듣고 싶어 하는 소식입니다. `재개`를 정식 분류로 넣어 해소했습니다.
- **그대로 진행 중은 세되 보여 주지 않습니다.** 요약이 보고서에 이미 적힌 내용을 반복하면 요약이 아닙니다. 완료 후 보고에서 빠진 업무도 누락이 아니므로 제외합니다.
- **보고 품질 점검은 작성 중인 초안을 봅니다.** 저장본만 볼 수 있는 점검은 작성자가 이미 지나간 버전에 대해 답합니다. 규칙 네 가지(진척도 역행, 계획 반복, 실적 누락, 이슈 장기화)는 결정적이며 AI가 꺼진 환경에서도 동작합니다. 이미 상위 조직 요청으로 올린 이슈에는 다르게 말합니다. 조치한 사람에게 조치하라고 반복하면 점검 전체가 무시됩니다.

3.5.2 v0.18 — 성능, 지표 정확성, 판정 일원화 (완료)

- **보고서 조회 색인**: `report_items.report_id`에 색인이 없어 보고서를 열 때마다 테이블 전체를 훑고 있었습니다. PostgreSQL은 외래 키의 참조하는 쪽을 색인하지 않습니다. 8만 행 기준 5.07ms·1,144버퍼에서 0.074ms·11버퍼로 줄었습니다. `report_comments`, `report_status_history`도 같이 처리했습니다.
- **제출 마감 설정과 시간대 오판 수정**: 정시 판정이 `submitted_at::date <= week_start + 7`이었습니다. 조직 정책을 코드에 박아 둔 것이고, `::date`가 데이터베이스 세션 시간대(UTC)로 계산돼 **마감 다음 날 오전 9시 이전 제출이 전부 정시로 집계**됐습니다. 마감을 `주차 시작일 + N일 + H시`로 설정하게 하고 판정을 서비스 시간대로 명시했습니다. 화면은 기준을 함께 표시합니다.
- **기간 보고의 업무 식별 일원화**: 기간 집계만 여전히 제목 유사도 80%로 같은 업무를 다시 추측하고 있었습니다. **한 담당자 안에서는** v0.16.0의 병합·분리 결과를 그대로 쓰고 추측하지 않습니다. **담당자 사이에**

서는 사람을 가로지르는 저장된 식별자가 없으므로 제목 유사도를 유지합니다. 기간 보고는 조직의 한 업무를 한 줄로 보여 주기 위한 것이기 때문입니다.

3.5.17 v0.33 — 개인 API 키 경계 훑기 (완료)

v0.32의 경계 훑기를 세션 쿠키가 아니라 **개인 API 키**로 다시 했습니다.

- **키의 보안 자세는 견고했습니다.** 범위 이름이 전부 `:read`인 것을 보고 "저장되기만 하고 강제되지 않을 것"이라고 가정했는데 **틀렸습니다.** `apiKeyRequestAllowed`가 기본 거부에 GET 전용이고, 허용 경로도 네 갈래뿐입니다. 읽기 전용 키로 수정·제출·생성·키 발급·키 회전을 시도해 전부 403이었습니다.
- **키는 주인을 넘어서지 못합니다.** 일반 사용자의 키에 `analytics:read`를 넣어도 팀 분석은 볼 수 없습니다. 역할 검사가 그대로 뒤에 이어지기 때문입니다.
- **회전은 즉시 듣습니다.** 조회 질의가 키 버전을 대조하므로 회전 다음 요청부터 401입니다. 200 → 401을 실제로 확인했습니다.
- **조직 경계도 세션과 같습니다.** 조직 1 팀장의 키로 조직 7 보고서 403, 팀 보고서는 260건으로 자기 조직만.
- **다만 거부 이유가 하나뿐이었습니다.** 범위가 없어서인지, 경로 자체가 닫혀 있어서인지, 쓰기라서인지가 구분되지 않았습니다. 업무 추적·인수인계·회의 모드·경영 요약·업무 인사이트·담당자 목록은 **어떤 범위로도 열리지 않는데**, 메시지는 범위를 추가하면 될 것처럼 읽혔습니다. 셋을 구분해 말합니다.
- **이 규칙이 어디에도 적혀 있지 않았습니다.** README는 키를 MCP 토큰으로만 소개했고 openapi의 `/keys` post에는 설명이 없었습니다. 범위별로 무엇이 열리는지 표로 적었습니다.

3.5.16 v0.32 — 권한 경계 훑기 (완료)

경로 79개를 가드별로 나누면 `requireAuth`만 47개, `requireRole(ADMIN)` 25개, 팀장 이상 7개입니다. **47개는 핸들러가 스스로 범위를 좁혀야 한다**는 뜻이므로, 그쪽을 두드렸습니다.

- **일반 사용자 경계는 전부 막혀 있었습니다.** 다른 사람의 보고서 읽기·수정·삭제·제출·복제·댓글·승인·반려, 인수인계, 팀장 전용 5개, 관리자 전용 3개 — 19가지 전부 403이었습니다.
- **팀장의 조직 경계도 막혀 있었습니다.** 다른 조직 보고서 403, 업무 병합·분리 400, 팀 보고서·담당자 목록·업무 인사이트 모두 자기 조직으로 좁혀졌습니다.
- **한 곳이 통과했습니다.** 인수인계가 역할만 확인하고 대상이 범위 안인지는 보지 않았습니다. 업무는 `loadWorkItems`가 걸러 새지 않았지만, **이름은 사용자 표에서 그대로 채워 넣고 있었습니다.** 팀장이 식별자를 훑어 전사 이름을 읽을 수 있었고, 더 나쁘게는 범위 밖 담당자가 **평범한 빈 인수인계**로 돌아와 "열어 볼 수 없는 사람"과 "맡은 일이 없는 사람"이 구분되지 않았습니다.

- 허용 대상을 `/team/members`가 제안하는 사람과 **정확히 같게** 맞췄습니다. 목록이 제안한 사람은 전부 열리고, 목록 밖은 전부 403입니다. 목록이 제안해 놓고 화면이 거절하면 그 목록은 거짓말입니다.

3.5.15 v0.31 — 쓰기 표면 훑기, 화면에만 있던 규칙 (완료)

GET 42개에 이어 쓰기 37개를 훑었습니다. 각각 **빈 객체·깨진 JSON·배열·null** 네 가지 잘못된 본문으로 호출했고, 경로 변수는 존재하지 않는 값으로 채웠습니다.

- **표면 자체는 튼튼했습니다.** 500이 하나도 없었습니다. `AI_UNAVAILABLE` 503은 AI를 끈 배포에서 맞는 답이고, 개인 키 회전은 본문을 무시하지만 **호출자 본인 키에만** 걸리고 감사 로그도 남으므로 결함이 아닙니다.
- **다만 빈 본문으로 `POST /reports`를 부르면 항목 없는 보고서가 만들어졌고, 그대로 제출까지 됐습니다.** 화면은 "업무 항목을 하나 이상 입력하세요"로 막는데 그 규칙이 **브라우저에만** 있었습니다. API와 MCP는 문서화된 표면입니다.
- 초안은 비어 있어도 됩니다 — 초안이란 그런 것입니다. 제출이 한 주가 끝났다고 말하는 행위이므로 **제출 지점**에서 막습니다. 409 `EMPTY_REPORT`.
- 통합 테스트에 이 경우를 넣고, **가드를 되돌리면 실패하는 것**을 실제 데이터베이스에서 확인했습니다.

3.5.14 v0.30 — 내보내기가 앱 밖으로 나가 원시 JSON을 보여 주던 문제 (완료)

`asString` 나머지 호출부를 훑었고 **더 나올 것이 없었습니다.** 남은 네 곳은 모두 `nil` 가드 안이거나 정수 인자였습니다. 대신 v0.29에서 결함을 찾아낸 방식 — 기본 인자로 표면 전체를 훑기 — 을 API 전체에 적용했습니다. GET 42개를 전부 호출했고 5xx는 없었지만, 같은 조건에서 세 엔드포인트가 서로 다르게 답하는 것이 눈에 띄었습니다.

- **내보내기가 전부 `<a href>`로 API를 직접 가리키고 있었습니다.** 잘 될 때는 동작하지만 안 될 때가 문제입니다. 브라우저가 앱을 떠나 응답을 그대로 그립니다. **세션이 만료된 탭에서 내보내기를 누르면 `{"success":false,...,"UNAUTHORIZED"}`가 페이지로 표시되고 앱은 탭에서 사라집니다.** 돌아올 방법은 뒤로 가기뿐입니다.
- 내보내기를 `fetch`로 바꿔 실패가 다른 실패와 같은 토스트로 가고, 401이면 다른 곳과 똑같이 세션 만료 화면으로 갑니다. 서버가 정한 한글 파일명은 `Content-Disposition`에서 그대로 가져옵니다.
- **비어 있는 기간에서 세 엔드포인트가 서로 다르게 답하고 있었습니다.** 목록은 200에 `items: []`, CSV는 200에 머리글만, PPTX는 409 `EMPTY_ROLLUP`. 발표 모드 버튼만 잠겨 있고 CSV·PPTX는 누를 수 있었습니다. 세 버튼을 같은 조건으로 잠급니다.

3.5.13 v0.29 — 남은 상한을 끝까지, 그리고 MCP 도구가 첫 질문에 답하지 못하던 문제 (완료)

- **MCP 도구 셋이 인자 없이 부르면 실패하고 있었습니다.** 선택 인자를 `asString(arguments["x"])`로 읽는데, 없는 키에 `fmt.Sprint`를 걸면 빈 문자열이 아니라 **문자열** `"<nil>"`이 나옵니다. 모든 기본값 처리가 `if value == ""`라 그 네 글자가 실제 값처럼 통과했습니다. `weekly_reports_search`는 `"<nil>"`을 날짜로 PostgreSQL에 보냈고, `period_report_rollup`은 조회 범위가 잘못됐다고 답했습니다. **에이전트가 가장 먼저 던지는 질문 — "무슨 보고서가 있나" — 이 늘 실패하고 있었습니다.**
- **MCP 도구 실패 원인을 아무 데도 남기지 않고 있었습니다.** 호출자에게는 한 문장, 고칠 사람에게는 아무것도 없었습니다. 이 로그를 붙이자마자 위 결함이 드러났습니다.
- **남은 조용한 상한을 정리했습니다.** MCP 보고서 검색(`LIMIT 100`)은 전체 건수와 함께 반환하고, **일부만 보냈다는 사실을 문장으로도 적습니다.** 이 응답을 읽는 것은 사람이 아니라 언어 모델이고, 모델의 세계는 이 payload가 전부입니다. Import 이력(`LIMIT 100`)은 전체 건수와 쪽 넘기기를 갖습니다.
- **관리자 화면의 "미제출 인원" 숫자가 25에서 멈추고 있었습니다.** 목록은 누락이 많은 순 상위 25명인데 헤드라인 숫자가 그 목록의 길이였습니다. 41명이 밀려 있어도 25로 보였습니다. 순위는 순위대로 두고 전체 인원을 따로 셉니다.
- **AI 초안 경고가 20건에서 조용히 잘리고 있었습니다.** 몇 건을 버렸는지 적는 줄을 붙입니다.

3.5.12 v0.28 — 우리가 만든 규칙을 우리가 어기고 있던 자리 (완료)

v0.25에서 5.의 품질 관리에 **"목록을 반환하는 API는 상한과 전체 건수를 함께 갖습니다"**를 적었습니다. 정작 보고서 목록이 그 규칙을 어기고 있었습니다.

- `queryReports`가 `LIMIT 500`만 걸고 배열을 그대로 돌려주고 있었습니다. 담당자 120명·26주에서 보고서 3,114건 중 500건이 왔고, 화면은 최근 5주만 보여 주면서 그게 전부인 것처럼 보였습니다. `{items, total, limit, offset}`으로 바꾸고 화면은 `3,114건 중 200건`과 `더 보기`를 표시합니다.
- **그 잘림이 인수인계 화면에서 기능 결함이 되고 있었습니다.** 담당자 선택 목록을 팀 보고서 한 쪽에서 만들고 있어서, **최근 5주 안에 보고하지 않은 사람은 고를 수 없었습니다.** 6주 전에 조직을 떠난 사람으로 재현했습니다 — 인수인계가 필요한 사람이 정확히 그 사람입니다. 사람 목록은 사람 표에서 옵니다(`GET /api/v1/team/members`). 비활성 계정도 포함하고 마지막 보고 주차를 함께 보여 줍니다.
- **빠른 이동은 8건만 쓰면서 200건을 받아 오고 있었습니다.** `limit=8`을 붙였습니다.
- 상한 값은 잘라 맞추고(1~500) 읽을 수 없는 값은 기본값으로 봅니다. 400으로 돌려주는 것은 요청한 쪽이 이미 아는 것을 다시 말할 뿐입니다.

3.5.11 v0.27 — 메타포를 남은 화면으로, 색이 갈라진 나머지 자리 (완료)

- **대시보드의 "지난주 대비 변화"가 숫자 일곱 개였습니다.** `완료 0 신규 0 재개 3 진척 0 역행 0 정체 0 누락 3`은 데이터이지 답이 아닙니다. 한 줄 띠로 그려 넓은 주황이면 정체된 주, 넓은 회색이면 보고가 빠

진 주로 읽힙니다. 신규·재개·진척은 모두 "움직였다"는 뜻이라 같은 파랑을 쓰고 읽기 순서상 붙어 있어 한 덩어리로 보입니다.

- **업무 추적 상세도 같은 주차 기록을 갖고 있었습니다.** v0.26의 주차별 띠를 그대로 붙였습니다. 아래 목록은 "N주차에 무슨 일이 있었나"에 답하고, 위 띠는 "몇 주차를 읽어야 하나"에 답합니다. 상세를 여는 사람이 실제로 가진 질문은 후자입니다.
- **색이 갈라진 자리가 세 군데 더 있었습니다.** v0.26이 토큰으로 모은 것은 차트와 결재 배지였고, 대시보드 변화 숫자·업무 상태 칩·증감 표시는 여전히 각자 hex를 썼습니다. 대시보드는 완료를 청록으로, 보고 누락을 빨강(=문제)으로 칠하고 있어 바로 옆에 새로 붙인 띠와 어긋났습니다.
- **`.delta-up` / `.delta-down`이 각각 두 번 정의돼 있었습니다.** 용어 언급 증감(파랑/회색)과 진척 증감(녹색/빨강)이 같은 클래스 이름을 썼고, 뒤에 온 정의가 앞엣것을 전부 덮었습니다. **워드클라우드에서 늘어난 용어가 줄어든 용어의 색으로 그려지고 있었습니다.** 축이 다르므로 이름을 나눴습니다.
- **같은 선택자가 서로 다른 색으로 두 번 정의되면 테스트가 실패합니다.** 색 이름과 값의 표기 차이(`var(--blue)`와 `#2563eb`)는 충돌로 세지 않습니다.

3.5.10 v0.26 — 시각적 메타포 (완료)

화면 15개 중 시각 요소가 있는 것은 2개뿐이었고, 나머지는 표와 문장이었습니다. 표가 답을 담고 있어도 읽는 사람이 답을 조립해야 하면 그 화면은 답한 것이 아닙니다.

- **색이 두 곳에서 반대로 말하고 있었습니다.** 차트에서 주황은 정체, 파랑은 진행인데, 보고서 결재 배지에서는 주황이 제출됨, 파랑이 마감이었습니다. 기간 보고 화면은 둘을 같은 카드에 놓습니다. 한 색이 한 뜻을 갖도록 정리하고(회색 멈춤·파랑 진행·녹색 완료·주황 정체·빨강 문제) 값을 `:root` 한 곳에 모았습니다. 구성비 색은 어느 상태도 쓰지 않는 보라 계열로 분리했습니다. **다시 갈라지면 Go 테스트가 실패합니다.**
- **협업 "지도"가 66행짜리 표였습니다.** 본부 12개면 조합이 정확히 66개입니다. "우리 본부가 어디와 붙어 있나"를 알려면 66줄을 읽고 자기 이름이 나온 줄을 기억해야 했습니다. 조직 × 조직 행렬로 바꿔 가로줄 하나가 곧 그 조직의 이웃이 됩니다. 정확한 숫자가 필요하면 표를 펼칠 수 있습니다.
- **인수인계에 필요한 그림이 이미 만들어져 있었습니다.** 화면 클래스 이름은 `handover-timeline`인데 실제로는 목록이었습니다. 인수인계가 답할 질문은 "언제부터, 얼마나 자주, 어디서 멈췄나"인데, 진행 경과는 진척이 변한 주차만 남기므로 **빈 구간을 표현할 수 없습니다.** 주차별 기록을 응답에 담고 보고한 주는 채우고 누락된 주는 비워 그립니다. 26주 중 6주만 보고된 업무가 한눈에 보입니다.
- **경영 요약 점수가 왜 그 순서인지 안 보였습니다.** 점수는 원래 귀속 가능한 사실의 합인데(중복 +40, 이슈 주차 ×10 …) 응답은 총점 하나와 문장 목록만 보냈습니다. 근거마다 기여 점수를 함께 보내고 화면은 점수를 그 부분들로 이루어진 막대로 그립니다. 막대 길이는 화면 최고점 대비이므로 순위가 읽지 않아도 보입니다. **총점과 근거 합계가 어긋나면 테스트가 실패합니다.**

3.5.9 v0.25 — 업무 인사이트 규모 (완료)

- **응답에 상한이 없었습니다.** 업무 1,800건에서 조건에 맞는 쌍 234,900개를 전부 담아 **158MB·2.3초**였고, 화면은 그것을 `map`으로 전부 그리려 했습니다. 관련도 상위 200건씩만 담고 전체 건수를 함께 반환하도록 바꿔 **297KB·0.3초**가 됐습니다. 처음 발견 당시의 더 겹치는 데이터에서는 링크 1,606,500개에 911MB·8.8초였습니다.
- **같은 제목을 320만 번 토큰화하고 있었습니다.** 비교는 쌍마다 필요하지만 토큰화는 업무마다 한 번이면 됩니다. 미리 계산해 재사용하니 2.5초에서 0.5초가 됐습니다.
- **인수인계가 조직 전체의 비교 결과를 받아 쓰고 있었습니다.** 한 사람을 넘겨주는 화면인데 모든 쌍을 비교하고 그중 그 사람 것만 골랐습니다. 대상자에 닿는 쌍만 비교하고 업무당 5건까지 담아 **858KB·1.5초 → 29KB·0.03초**가 됐습니다.
- **잘라 냈다는 사실을 화면이 말합니다.** 로드맵의 "silent cap 금지"를 따라, 탭 숫자는 전체 건수를 보여 주고 목록 위에 몇 건 중 몇 건인지 적습니다.
- **집계는 상한 밖에서 합니다.** 협업 지도와 경영 요약의 중복 채점은 조건에 맞는 모든 쌍에서 만듭니다. 목록의 상한이 집계의 상한이 되면 화면이 없는 것을 없다고 말하게 됩니다. 커밋 전 리뷰에서 잡았습니다.
- 5.의 품질 관리에 한 줄이 붙습니다. **목록을 반환하는 API는 상한과 전체 건수를 함께 갖습니다.**

3.5.8 v0.24 — 작성 화면과 대화상자 (완료)

측정 결과 매주 여는 화면이 부차적인 것부터 보여 주고 있었습니다.

- **작성 화면 우선순위**: 1440×900에서 첫 업무 제목 입력칸이 y=921로 폴드 아래에 있었고, 1366×768에서는 업무 카드 전체가 화면 밖이었습니다. 그 위를 AI 카드(180px, 꺼져 있으면 안내만)와 캡처 카드(138px, 저장 전에는 사용 불가)가 차지했습니다. AI 카드는 실제로 쓸 수 있을 때만 띄우고 캡처는 업무 항목 아래로 옮겨 **y=567**로 올렸습니다.
- **포커스와 저장 상태**: 작성 화면이 커서를 아무 데도 놓지 않았고(로그인 화면은 제대로 놓습니다), 저장 여부를 알리는 표시도 없어 v0.16.0의 이탈 확인 대화상자를 만나야 알 수 있었습니다. 둘 다 넣었습니다.
- **대화상자 동작 통일**: 상세 창 여섯 개가 Esc로 닫히지 않고 포커스를 잡지 않으며 Tab이 뒤 화면으로 새어 나갔습니다. 팔레트와 발표 모드에는 Esc가 있어 같은 앱에서 창 닫는 방법이 화면마다 달랐습니다. 공용 `Modal`로 모았습니다.
- **빈 상태**: 절반은 원인과 해결을 말하고 절반은 사실만 말했습니다. 다음 행동이 있는 곳에는 버튼을 넣었습니다.

3.5.7 v0.23 — 설정 안전장치와 감사 추적 (완료)

- **주차 시작 요일 변경**: 서식 설정처럼 보이지만 실제로는 데이터 이관입니다. 실제 서버에서 MONDAY → WEDNESDAY로 바꾼 결과, 작성자의 이번 주 보고서가 사라져 같은 기간에 두 번째 보고서를 만들 수 있었고, 참여 분석의 과거 주차가 전부 0건이 됐으며, 기간 보고가 한 주를 두 주로 셌습니다. 이제 영향 범위를 알리고 확인을 받으며, 기간이 겹치는 보고서 생성 자체를 막습니다.
- **조회 가능한 감사 이력**: 최근 500건을 조건 없이 반환하는 것이 전부였고 보관 정책도 없었습니다. 저장은 하되 답할 수는 없는 통제였습니다. 작업·행위자·대상·기간 필터와 페이지, 전체 건수, 보관 기간 설정을 넣었습니다.
- **오류 추적 ID**: 서버가 요청마다 만들어 로그에 남기고 응답에도 담던 식별자를 화면이 버리고 있었습니다. 5xx 오류 문구에 함께 보여 줍니다.
- 4.3에 한 줄이 붙습니다. **과거 데이터의 의미를 바꾸는 설정은 확인 없이 적용하지 않습니다.**

3.5.6 v0.22 — 기동과 휴대폰 (완료)

- **기동이 백필을 기다리지 않습니다**: 업무 식별자 백필과 첨부 무결성 확인이 서버가 응답을 시작하기 전에 동기로 돌고 있었습니다. 5만 건에서 `/healthz` 응답까지 34.8초가 걸렸고, 쿠버네티스 기본 `livenessProbe`는 약 30초에 파드를 죽입니다. **재시작할 때마다 하나의 트랜잭션을 처음부터 다시 시작**했으므로 규모가 큰 배포는 영원히 끝내지 못합니다. 배경으로 옮기고 500건씩 나눠 커밋해 재시작에도 이어지게 했습니다. `startupProbe`도 매니페스트에 넣었습니다.
- **휴대폰에서 메뉴 4개에 닿지 못했습니다**: 하단 메뉴 13개 중 4개가 화면 밖에 그려지고 그 띠는 스크롤되지 않았습니다. 릴리즈를 거듭하며 메뉴를 늘린 결과입니다. 스크롤 가능하게 하고, 가장자리 페이드로 더 있다는 것을 알리고, 현재 화면의 항목이 보이도록 스크롤합니다.
- 4.2의 원칙에 한 줄이 붙습니다. **기동 경로에 말뭉치 전체를 도는 작업을 추가하면 안 됩니다.** 필요하면 배경에서, 이어서 할 수 있게 나눠서 합니다.

3.5.5 v0.21 — 발표 화면 테마 (완료)

- **읽을 수 있는 최소 글자 크기**: v0.20의 자동 축소는 배율만 줄여서, 1280×720에서 구획 이름이 9.3px까지 내려갔습니다. 이제 **여백을 먼저 줄이고**, 글자는 화면 높이의 1/50(프로젝션 경험칙)을 밑돌지 않는 선까지만 줄입니다. 그래도 넘치면 줄이는 대신 **잘렸다고 알립니다.** 구획 이름의 상한이 화면과 무관한 15px 고정이던 것도 화면에 비례하도록 바꿨습니다.
- **테마 3종**: 어두운 화면(기존), 밝은 화면, 고대비. 밝은 방과 약한 프로젝터에서는 어두운 그라데이션이 가장 불리하고, 캡처 슬라이드는 대개 밝은 스크린샷이라 어두운 판 위에서 눈부십니다. 고대비는 그라데이션 없이 검정 바탕에 단일 강조색만 씁니다. 약한 프로젝터가 여섯 색을 구분해 줄 것이라고 기대할 수 없기 때문입니다.

- **회사 템플릿 강조색 연동**: 내보낸 PPTX는 회사 템플릿을, 화면은 고정 파랑·청록을 써서 같은 보고가 매체에 따라 다른 회사 자료처럼 보였습니다. 템플릿 테마의 `accent1`을 읽어 화면 강조색으로 씁니다.
- 구획 색이 없던 `변경점`에 색을 넣고, 발표자 창이 테마를 따르게 하고, `prefers-reduced-motion`을 존중합니다.

3.5.4 v0.20 — 발표 모드 (완료)

v0.13에서 회의용으로 만든 발표 모드를 실제 회의실 화면 크기에서 다시 점검했습니다.

- **내용이 잘리던 문제**: 슬라이드 텍스트 상한이 화면 크기와 무관한 고정 글자 수였고, 슬라이드 상자는 `overflow:hidden`이었습니다. 1280×720에서 한 블록의 **39%가 아무 표시 없이 사라졌습니다.** 이제 상자를 실제로 재서 들어갈 만큼만 줄입니다. 읽을 수 있는 하한(0.62배)을 넘어서까지 줄이지는 않고, 그때만 안내를 띄웁니다.
- **발표자 화면**: 회의는 보통 한 화면을 방에 미러링하고 노트북은 발표자가 봅니다. 두 화면에 다른 것이 있어야 합니다. 별도 창에 경과 시간, 다음 슬라이드, 그리고 **슬라이드가 줄여서 보여 주는 그 내용의 전체 원문**을 띄웁니다. 측정값으로 881자 대 2292자였습니다.
- **슬라이드 목록과 번호 이동**: 40장짜리 덱에서 "그건 3번 안건이었죠"에 대응할 방법이 화살표뿐이었습니다.
- 화면 끄기(B), 태블릿 스와이프, 새로고침 후 위치 복원, 스크린리더 안내를 함께 넣었습니다.

3.5.3 v0.19 — 내보내기 안전성과 실패 복구 (완료)

- **CSV 수식 주입 차단**: 기간 보고 CSV가 보고 본문을 그대로 실어 보내고 있었습니다. 스프레드시트는 `=`, `+`, `-`, `@`로 시작하는 셀을 수식으로 실행하므로, 아무 직원이 쓴 `=HYPERLINK(...)`가 그 파일을 여는 상급자의 기계에서 실행됩니다. 셀 앞에 작은따옴표를 붙여 막았습니다. `-` 뒤에 공백이 오는 글머리표는 그대로 둡니다. 공격은 연산자 바로 뒤에 함수 이름이나 명령이 있어야 성립하고, 그렇지 않으면 보고서 본문 상당수에 따옴표가 붙습니다.
- **세션 만료 복구**: 세션이 끊겨도 화면이 그대로 돌아가고 있었습니다. `.catch`가 없는 화면은 스피너가 영원히 멈추지 않았고, 어디에도 401을 처리하는 곳이 없었습니다. API 계층이 401을 알리고 셸이 로그인 화면으로 되돌립니다. **단, 저장하지 않은 편집이 있으면 화면을 유지합니다.** v0.16.0이 지키기로 한 바로 그 내용을 로그인 폼을 띄우려고 버릴 수는 없습니다. 세션 쿠키는 탭 사이에 공유되므로 다른 탭에서 로그인하면 이 화면 그대로 저장됩니다.
- **화면 오류 격리**: 렌더 중 예외 하나가 앱 전체를 흰 화면으로 만들었습니다. 오류 경계를 넣어 해당 화면 안에서 끝나게 하고 돌아갈 길을 제시합니다.

- **PPTX 내보내기 메모리 축소**: 첨부 이미지를 전부 메모리에 올린 뒤, 본문 앞·뒤 두 묶음 때문에 패키지 전체를 두 번 다시 만들고 있었습니다. 한 번에 처리하고 이미지는 zip에 쓰는 순간 읽도록 바꿨습니다.

3.6 v0.16 — 편집 보호와 자동 판정 정정 (완료), Decision Log (남음)

v0.16.0은 Decision Log보다 먼저 처리해야 할 세 가지를 처리했습니다. 하나는 지금도 데이터를 잃고 있었고, 하나는 4.3의 원칙이 유일하게 지켜지지 않던 자리였으며, 하나는 보안 통제 목록에서 유일하게 비어 있던 항목이었습니다.

- **편집 내용 유실 방지 (완료)**: 저장하지 않은 편집이 있으면 화면 이동·뒤로 가기·탭 닫기·로그아웃 전에 확인합니다. 저장에 **실패했을 때 화면 내용을 서버 사본으로 덮어쓰던 동작도 제거**했습니다. 실패한 저장이 지키려던 바로 그 내용을 지우고 있었습니다.
- **업무 병합·분리 (완료)**: 3.1이 약속하고 4.3이 요구한 정정 경로입니다. 자동 판정을 되돌릴 수단이 없으면 그것은 제안이 아니라 확정이며, 오판은 Aging·인수인계·중복 탐지·협업 지도까지 전파됩니다.
- **로그인 시도 제한 (완료)**: 시도 횟수 제한이 전혀 없었습니다. 계정당 제한은 기본 활성, IP당 제한은 기본 비활성입니다. NAT 뒤에서는 의미 있는 임계값이 무관한 사용자를 함께 잠급니다.

정정이 다음 저장에도 살아남는 것이 이 기능의 전부입니다. 업무 식별은 저장할 때마다 제목에서 다시 유도되므로, 유도 결과를 바꾸지 않는 정정은 작성자의 다음 편집이 되돌립니다. 병합은 원본 행과 정규화 키를 남기고 대상을 가리켜 옛 제목이 대상으로 해석되게 하고, 분리는 옮긴 스냅샷을 고정해 제목이 바뀌기 전까지 유도를 이깁니다.

3.6.1 v0.34 — Decision Log 1단계: 명시적 기록 (완료)

로드맵이 순서를 못 박아 뒀습니다 — **명시적 기록이 먼저, AI 제안이 나중.** 감사·기억 시스템의 신뢰도를 모델 재현율에 걸 수 없기 때문입니다. 이번에는 앞의 절반을 만들었습니다.

- **주간보고가 담지 못하는 기록입니다.** 보고서는 무슨 일이 있었는지를 적지, 누가 무엇을 왜 정했고 무엇을 하기로 했는지는 적지 않습니다. 그래서 반년 뒤에 실제로 나오는 질문 "왜 이렇게 하기로 했더라"는 아직 기억하는 사람을 찾아 물어야 했고, **아무도 기억하지 못하는 순간을 위해 만든 인수인계 화면**에 보여 줄 것이 없었습니다.
- **결정자와 기록자를 구분합니다.** 팀장이 임원 결정을 대신 적는 경우가 흔합니다. 둘을 합치면 이 기록이 가장 필요한 상황에서 값이 틀립니다.
- **뒤집힌 결정을 지우지 않습니다.** 새 결정이 이전 결정을 대체하면 이전 것은 `SUPERSEDED`가 됩니다. 뒤집힌 결정은 없던 결정보다 정보가 많습니다.

- **기록은 볼 수 있는 사람이, 삭제는 기록한 사람이.** 업무 병합·분리와 규칙이 다른데, 병합은 남의 보고 이력을 고쳐 쓰는 일이고 기록은 덧붙이는 일이기 때문입니다. 반대로 다른 사람이 조용히 지울 수 있는 기록은 기록이 아닙니다.

남은 것은 3.6.2를 보십시오.

3.6.2 v0.35 — Decision Log 2단계: 인수인계의 결정 이력 (완료)

이 기록이 필요한 이유가 인수인계였습니다. 업무 추적에만 있으면 결정을 적은 사람만 볼 수 있고, 정작 아무것도 모르는 채 업무를 넘겨받는 사람에게는 닿지 않습니다.

- **잘라 내지 않습니다.** 인수인계가 존재하는 이유가 "아무도 기억하지 못하는 것"인데 그 기록을 잘라 내면 화면의 존재 이유가 사라집니다. 업무 15건·결정 17건에서 16.0KB → 22.9KB, 8ms → 5ms였습니다.
- **질의는 하나입니다.** 업무마다 조회하면 화면에 뜬 업무 수만큼 질의가 나갑니다. v0.25가 한 릴리즈를 들여 없앤 모양입니다.
- **기한 지난 후속 조치가 주의 문구를 이깁니다.** 다른 주의 문구는 기록이 무엇을 보여 주는지 말하지만, 이것은 누군가 약속하고 지키지 않은 것을 지목합니다. 머리줄에 미해결 결정 수와 기한 초과 수를 함께 셉니다.
- **인수인계에서는 읽기만 합니다.** 인수인계는 읽는 문서이지 일하는 화면이 아닙니다. 기록과 정정은 업무를 관리하는 업무 추적에 둡니다.

3.6.3 v0.36 — 회의 결과 반영 (완료)

회의는 안건을 만들어 주고 거기서 멈췄습니다. 회의에서 나온 결론은 누군가의 수첩으로 들어갔고, 다음 월요일에 그 업무의 담당자는 아무 표시도 없는 빈 편집기를 열었습니다.

- **회의 화면에서 그 자리에 적습니다.** 안건이 화면에 떠 있고 문구가 아직 생생할 때 적는 것이 가장 정확합니다. 나중에 옮겨 적으려면 대개 옮겨 적지 않습니다.
- **후속 조치가 주인에게 돌아옵니다.** 작성 화면이 지난주 계획을 돌려주는 바로 그 자리에 `결정에 딸린 후속 조치` 카드가 생깁니다. 기한이 지난 것은 그렇게 표시합니다.
- **차주 계획으로 들어갑니다.** 금주 실적이 아닙니다. 하기로 한 일이지 했다는 주장이 아닙니다.
- **이미 이번 주 보고에 쓴 업무는 빼고 보여 줍니다.** 작성자가 이미 적은 것을 다시 권하면 카드 전체가 무시됩니다.
- **본인 업무만입니다.** 후속 조치는 해야 할 일이고, 남의 의무 목록은 할 일 목록이 아닙니다.

3.6.4 v0.37 — 기간 보고의 결정 이력 (완료)

로드맵의 **결정 이력 조회**가 이걸로 끝납니다. 인수인계(v0.35)에 이어 기간 보고에도 붙였습니다.

- **무엇을 했는지 위에 있고, 왜 그렇게 하기로 했는지가 아래에 있습니다.** 분기 보고가 일한 목록만 담고 결정을 빼면, 읽는 사람은 왜 그렇게 갔는지 물을 수 없습니다.
- **사람 범위 조건을 집계와 공유합니다.** 조건을 두 벌 쓰면 언젠가 어긋나고, 한쪽 사람들을 집계하면서 다른 쪽 결정을 나열하는 보고서는 결정을 아예 안 넣느니만 못합니다. 조직 1 팀장 17건 / 관리자 18건으로 범위가 갈리는 것을 확인했습니다.
- **상한 50건에 전체 건수를 함께 줍니다.** 5.의 "목록을 반환하는 API는 상한과 전체 건수를 함께 갖는다"를 따릅니다.

3.6.5 v0.38 — 결정을 덱과 발표 모드로 (완료)

v0.37은 화면만 채웠습니다. **임원이 실제로 받는 것은 화면이 아니라 덱입니다.**

- **PPTX에 `기간 내 결정` 슬라이드가 들어갑니다.** 업무 상세 뒤, 이슈·리스크 앞입니다 — 기간이 스스로를 설명한 뒤 요청을 합니다. 끝을 요청으로 맺어야 회의가 움직입니다.
- **후속 조치가 남은 결정을 먼저 싣습니다.** 이미 이행된 결정은 이력이고, 아직 갚을 것이 남은 결정이 브리핑에 들어갈 이유입니다.
- **발표 모드도 같이 받습니다.** 3.4가 정한 원칙입니다 — 내보내기와 발표는 같은 내용을 다른 매체로 전달하므로 한쪽만 제공할 이유가 없습니다.
- **표 칸에 줄바꿈을 쓰지 않았습니다.** DrawingML에서 `<a:t>` 안의 줄바꿈은 줄바꿈이 아니라 `<a:br/>`이 필요합니다. 확인할 수 없는 렌더링 동작에 기대는 대신 구분자를 명시했습니다.

3.6.6 v0.39 — AI 후보 제안 (완료), Decision Log 완결

로드맵이 이 기능을 **마지막에** 둔 이유를 그대로 따랐습니다. 감사·기억 시스템의 신뢰도를 모델 재현율에 의존시킬 수 없으므로, 명시적 기록(v0.34)·인수인계(v0.35)·회의 반영(v0.36)·기간 보고(v0.37·v0.38)가 모두 선 뒤에야 그 위에 얹었습니다.

- **제안만 합니다. 아무것도 저장하지 않습니다.** 후보는 사람이 고쳐서 확정할 양식일 뿐이고, 실제로 0건이 저장되는 것을 확인했습니다.
- **"여기 없다고 해서 결정이 없었다는 뜻이 아니다"를 서버가 함께 보냅니다.** 화면이 빠뜨릴 수 없도록 payload에 담습니다. 전수인 것처럼 보이지만 아닌 목록은 목록이 없는 것보다 나쁩니다.
- **후보마다 보고 원문을 인용합니다.** 확정하는 사람이 모델이 아니라 기록에 대고 확인할 수 있어야 합니다.
- **모델이 지어낸 날짜는 서버가 비웁니다.** 없는 날짜가 그럴듯한 날짜보다 낫습니다 — 비어 있으면 양식이 오늘로 채우고, 확정하는 사람이 알아차립니다.

- **AI가 꺼진 배포에서는 버튼을 띄우지 않습니다.** 늘 "관리자가 설정해야 합니다"라고 답하는 버튼은 사람에게 버튼 누르기를 그만두라고 가르칩니다.

이로써 3.6.1의 Decision Log 네 항목이 모두 완료됐습니다.

- **명시적 기록 우선**: 결정사항을 항목 단위로 직접 기록하는 필드를 먼저 제공합니다. 결정자, 날짜, 근거, 후속 조치, 관련 WorkItem을 구조화합니다.
- **AI 후보 제안**: 기존 보고 텍스트에서 결정사항 후보를 **제안**하고 사용자가 확정합니다. 감사·기억 시스템의 신뢰도를 모델 재현율에 의존시키지 않습니다.
- **결정 이력 조회**: WorkItem 타임라인과 기간 보고에서 결정 흐름을 함께 확인합니다.
- **회의 결과 반영**: 회의 모드는 v0.13에서 완료됐고, 남은 것은 결과를 되돌려 쓰는 것입니다. 회의에서 정한 Decision과 Action Item을 기록하면 다음 주 보고서 초안에 반영합니다. 결정 기록 모델을 전제하므로 이 단계에 함께 둡니다.

3.7 v0.40 — Dependency / Blocker Graph 1단계: 관계 모델과 순환 방지 (완료)

- **간선은 하나입니다.** "A가 B를 기다린다"와 "B가 A를 막는다"는 같은 사실을 양끝에서 본 것입니다. 두 종류로 저장하면 둘이 어긋날 수 있고, 스스로 모순되는 그래프는 없느니만 못합니다.
- **선언은 한쪽이 합니다.** 기다리는 쪽 담당자가 자기 일이 무엇을 기다리는지 선언하고, 상대의 동의를 받지 않습니다. **상대가 동의해야 하는 의존은 아무도 기록하지 않는 의존입니다.** 대신 사유를 함께 남기고, 상대는 자기 화면에서 그것을 보고 이의를 제기하거나 관계를 삭제할 수 있습니다.
- **조직 경계를 넘습니다.** 선행 업무는 조회 범위 밖이어도 됩니다. 다른 조직의 업무를 기다리는 것이 이 기능이 존재하는 이유입니다.
- **순환은 막고 사슬을 알려 줍니다.** `순환 의존이 됩니다: A → B → C → (이 업무)`. 보이지 않는 그래프를 두고 추측하게 두지 않습니다.
- 순환 검사 질의를 실제 PostgreSQL에 `PREPARE`하다가 결함을 잡았습니다. 재귀 CTE의 비재귀항이 `varchar(240)[]`, 재귀항이 `varchar[]`여서 PostgreSQL이 질의 자체를 거부했습니다. 양쪽을 `text`로 맞췄습니다.

남은 것: 병목 집계와 회의 모드의 `타 조직 Dependency` 구획.

3.7.1 v0.41 — 병목 집계와 회의 구획 (완료), 3.7 완결

- **병목 탭**: 미완료 업무 2건 이상을 막고 있는 선행 업무를 셉니다. 로드맵의 "현재 N개 업무가 하나의 선행 항목 때문에 정체"가 이것입니다.

- **범위를 막는 쪽에 걸었다가 되돌렸습니다.** 처음에는 선행 업무의 주인을 기준으로 걸렀는데, 그러면 **팀장이 자기 팀을 막고 있는 다른 조직의 병목을 못 봅니다** — 정작 가장 필요한 것인데요. 막히는 쪽 기준으로 바꾸니 조직 1 팀장이 조직 7의 업무가 자기 팀 4건을 막고 있다는 것을 봅니다.
- **회의 모드에** `타 조직 대기` **구획**을 넣었습니다. 지속 이슈 뒤, 변경점 앞입니다 — 방이 들어야 할 것이 아니라 방이 처리해야 할 것이기 때문입니다. 한 팀 안의 의존은 그 두 사람이 정리하지만, 조직을 넘는 의존은 양쪽에 말할 수 있는 사람이 방에 있어야 풀립니다.
- 완료된 선행 업무는 제외합니다. 끝난 업무는 무엇도 막지 않으며, 남겨 두면 아무도 지우지 않은 관계로 목록이 채워집니다.

이로써 3.7이 완결됐습니다.

3.7.2 v0.42 — 선행 관계를 정체 판정에 반영 (완료)

- **원인이 등록된 정체는 다른 안건입니다.** 점수는 같습니다 — 일이 멈춘 정도는 같으니까요. 읽는 사람이 할 일이 다릅니다: `진척 정체`를 보면 담당자에게 묻고, `타 조직 대기`를 보면 두 조직을 연결합니다. 같은 멈춤에 다른 조치이므로 같게 읽혀서는 안 됩니다.
- 근거 문장이 무엇을 기다리는지 이름을 댑니다. `진척이 2주째 멈춰 있고, 본부 7의 '설비 자동화 구축'(담당자 7)이(가) 끝나기를 기다리고 있습니다.`
- **경영 요약의** `headline`**을 서버가 계산해 두고 어느 화면도 쓰지 않고 있었습니다.** 항목마다 `장기 이슈` · `진척 정체` · `중복 투자 의심`을 정해 놓고 화면은 넷을 전부 `위험`으로만 보여 줬습니다. v0.23의 추적 ID와 같은 부류입니다.

3.7.3 v0.43 — 보고 품질 점검에 선행 관계 반영 (완료), 3.7 전부 완결

`PLAN_REPEATED`는 **막힌 이유를 적으라고 요구하는 규칙**입니다. 선행 업무를 이미 등록해 둔 사람에게 그 말을 되풀이하는 것은, v0.17이 상위 조직 요청에 대해 세운 원칙을 어기는 것입니다 — **조치한 사람에게 조치하라고 반복하면 점검 전체가 무시됩니다.**

- 선행 관계가 등록된 업무는 WARN이 아니라 **INFO**가 되고, 문구가 "무엇을 기다리는지 이름을 대며 그 관계가 아직 유효한지" 묻는 것으로 바뀝니다.
- **소견 자체는 남깁니다.** 계획을 몇 주째 그대로 적은 사실은 여전히 볼 만하고, 등록해 둔 의존이 이미 해소됐는데 관계만 남아 있는 경우가 실제로 있습니다.

이로써 3.7이 전부 끝났습니다.

- **관계 모델**: WorkItem 간 `dependsOn`, `blockedBy`를 조직 경계를 넘어 관리합니다.
- **병목 집계**: "현재 N개 업무가 하나의 선행 항목 때문에 정체" 형태로 경영 화면에 제시합니다.

- **순환 방지**: 관계 등록 시 순환 의존을 차단하고 경로를 함께 보여 줍니다.
- 회의 모드의 `타 조직 Dependency` 안건 구획은 이 단계에서 추가합니다.

3.8 v0.44 — Evidence Lineage 1단계: 공통 모델과 PPTX 계보 (완료)

근거는 있었는데, 쓸모 있어지는 순간에 버려지고 있었습니다. Confluence 후보는 `candidate_sources`에 페이지를, PPTX Import는 분석 결과에 슬라이드 번호를 들고 있었지만, 후보를 수락하거나 Import를 확정하면 `report_items` 행만 남고 둘 다 사라졌습니다. `report_items`를 참조하는 테이블이 임베딩 하나뿐이었으므로 답이 숨어 있을 곳도 없었습니다.

- **출처 종류마다 표를 만들지 않았습니다.** `report_item_sources` 하나에 MANUAL·CONFLUENCE·PPTX·AI_TEXT·JIRA·GIT·CI·ITSM·API를 담습니다. 커넥터마다 표를 두면 "근거가 몇 건인가"가 종류마다 다른 질의가 되고, 비교할 수 있게 하는 것이 이 모델의 목적입니다.
- **확정 화면이 슬라이드 번호를 되돌려 보냅니다.** 분석은 알고 있었고 확정이 버리고 있었습니다.
- **비어 있음은 근거 없음이 아닙니다.** 직접 타이핑했거나 이 모델 이전에 작성됐다는 뜻입니다.
- 통합 테스트가 실제 PostgreSQL에서 근거 행을 확인하고, **기록을 빼면 실패**합니다.

남은 것: Confluence 수락 경로, 문장 단위 근거, AI 요약의 근거 수 표기.

3.8.1 v0.45 — Confluence 계보 (완료)

- **수락 처리가 항목을 만들지 않고 있었습니다.** `acceptConfluenceCandidates`는 후보에 표시만 하고, 실제 항목은 편집기 저장이 만듭니다. 그리고 편집기가 실어 보내던 `candidateId`를 **서버가 무시**하고 있었습니다 — 그래서 근거를 옮길 지점 자체가 없었습니다.
- 서버가 `candidateId`를 받아 저장 시점에 그 초안의 페이지·버전·활동 종류를 계보 모델로 복사합니다. **저장할 때만** 하고 수정할 때는 하지 않습니다 — 이미 저장된 줄을 고친다고 출처가 바뀌지는 않고, 매번 기록하면 한 페이지가 타이핑 횟수만큼 늘어납니다.
- 식별자는 클라이언트가 보내므로 **본인 초안인지 서버가 확인**합니다.
- 통합 테스트가 실제 PostgreSQL에서 `CONFLUENCE · 설계 검토 회의록 · v7 · 작성 후 수정`을 확인하고, 복사를 빼면 실패합니다.

3.8.2 v0.46 — 역방향 계보 (완료), 문장 단위는 보류

문장 단위 근거를 만들려다 전제가 없다는 것을 확인하고 멈췄습니다. AI는 내용을 원자 단위 줄 배열(`currentResults`, `nextPlans`, `issues`)로 나누지만 `sourceSlides`는 **항목 단위**입니다. 문장마다 근거를 표시하려면 그 귀속을 모델에게 새로 요구해야 하고, 여기서는 그 귀속의 품질을 측정할 방법이 없습니

다. **측정할 수 없는 기능을 로드맵 항목을 지우기 위해 만드는 것은 이 프로젝트가 지켜 온 방식이 아닙니다.** 보류하고 사유를 남깁니다.

대신 계보 모델이 이미 가능하게 만든 것을 만들었습니다.

- **역방향 계보**: `이 페이지가 어느 보고에 근거로 쓰였나`. 정방향은 보고서를 읽는 사람에게 답하고, 역방향은 **페이지·덱·커밋의 주인**에게 답합니다 — 무엇이 자기 것 위에 세워졌는지 가장 늦게 알고, 바꾸기 전에 가장 알아야 하는 사람입니다. v0.44에서 이 질문을 위한 색인을 만들어 놓고 물어볼 방법이 없었습니다.
- 보고서 상세의 근거 칩을 누르면 같은 근거를 쓴 다른 보고가 펼쳐집니다. **조회 범위 안의 인용만** 나열합니다.
- 범위 술어에 별칭을 대려고 `work_items`를 조인했다가, `work_item_id`가 NULL이면 그 사람의 **모든 업무와 조인돼 건수가 부풀려지는** 것을 발견하고 보고서 자체를 별칭으로 바꿨습니다.

3.8.3 남음 — 문장 단위 근거와 AI 요약 근거 수

- Confluence 후보의 `candidate_sources` 근거 모델을 모든 Source로 일반화합니다. Manual, Confluence, Jira, Git Commit, CI, ITSM, PPTX Import를 같은 provenance 테이블로 관리합니다.
- 문장 단위로 근거 수와 출처 종류를 표시하고 AI 요약에도 근거 수를 함께 표기합니다.

3.9 v0.19 — 다중 PPTX 템플릿과 미제출 알림

- 조직별 PPTX 템플릿 매핑과 선택. 기간 보고 전용 레이아웃과 병행합니다.
- v0.10.0의 미제출 현황을 Slack·Teams·메일 알림으로 연결합니다. 오프라인망 제약을 고려해 발송 채널은 관리자 설정으로 선택합니다.

3.9.1 v0.47.0에서 완료 — 조직별 PPTX 템플릿

배포 하나에 표지 하나였습니다. 본부 두 곳이 서로 다른 보고 서식을 쓰기 시작하면, 그때부터 모두가 남의 표지로 보고서를 내보냅니다.

- `pptx_templates`에 `organization_id`를 추가했습니다(마이그레이션 017). 조직당 하나, 전사 기본 하나를 부분 유니크 인덱스 두 개로 보장합니다. 기존 단일 행은 `organization_id IS NULL`인 전사 기본이 됩니다.
- 내보내기는 **작성자가 속한 조직**의 서식을 찾습니다. 없으면 가장 가까운 상위 조직으로 올라가고, 끝까지 없으면 전사 기본을 씁니다. 팀마다 같은 파일을 올리게 하면 그중 절반은 작년 판을 쓰게 됩니다.
- 관리자 화면은 적용 대상을 고르고, 그 조직에 **실제로 적용되는** 서식을 이름까지 보여줍니다. 물려받은 경우 어느 조직에서 왔는지 함께 표시합니다.

발견한 것:

- 전사 기본 조회가 `WHERE id=1`로 남아 있었습니다. 식별자가 시퀀스가 되면서 새로 올린 기본 서식은 id가 1이 아니게 됐고, 업로드는 성공하는데 화면은 계속 참조 서식을 보여줬습니다. 조회를 `organization_id IS NULL`로 바꿨습니다.
- 화면이 하위 팀에 대해 "물려받음"이라고만 답하고 무엇을 물려받는지는 비워 뒀습니다. 관리자가 그 화면을 여는 이유가 바로 그것이므로, 내보내기가 하는 것과 같은 조상 탐색을 화면도 하게 했습니다.
- 통합 테스트의 정리를 `t.Cleanup`으로 걸었더니 `defer db.Close()` 뒤에 실행돼 닫힌 풀에 대고 삭제했습니다. 행이 그대로 남아 다음 실행이 유니크 제약에 걸렸습니다. `defer`로 바꿨습니다.

3.9.2 남음 — 미제출 알림

Slack·Teams·메일 발송은 이 오프라인 환경에서 **보냈다는 사실을 확인할 방법이 없습니다**. 발송 코드를 쓰는 것은 가능하지만 검증되지 않은 기능을 완료로 표시하지 않기 위해 남겨 둡니다.

3.10 v0.20 — Risk Forecast

- Executive Digest는 v0.13에서 완료됐습니다. 남은 것은 **예측**입니다. 진척 추이, 이월 횟수, 이슈 지속 기간으로 일정 위험을 예측합니다.
- 현재 경영 요약은 이미 관측된 사실만 제시합니다. 예측은 성격이 다르므로 근거를 항상 함께 제시하고 근거 없는 점수는 표시하지 않습니다.

3.10.1 v0.48.0에서 완료 — 완료 시점 추정

먼저 확인한 사실이 계획을 바꿨습니다. **예측할 일정이 없습니다.** `work_items.due_date`는 스키마에 있지만 이를 쓰는 API도 화면도 한 군데도 없어서, 전부 NULL입니다. 그래서 마감일 대비 위험을 계산하는 대신, 보고된 진척이 실제로 말하는 것만 계산합니다. "이 속도면 얼마나 더 걸리는가"입니다.

점쟁이가 되지 않도록 두 가지 규칙을 두었습니다.

- **점이 아니라 구간을 냅니다.** 전체 기간 평균 속도와 최근 3주 속도는 대개 다르고, 그 차이가 곧 불확실성입니다. 둘을 하나로 합쳐 신뢰도 점수를 붙이면 읽는 사람이 봐야 할 것이 사라집니다. 구간이 넓다는 것은 "아직 아무도 모른다"를 소리 내어 말하는 것입니다.
- **말하지 않는 쪽을 택합니다.** 2주치는 어떤 두 숫자에도 직선을 그을 수 있으므로 추정하지 않습니다. 진척이 늘지 않은 업무에는 완료 시점이 아예 없다고 답합니다. 1년이 넘는 추정은 주 단위 숫자를 내지 않습니다.
- 모든 추정은 **계산 근거인 두 속도와 사용한 주차 수를 함께** 표시합니다. 근거 없는 점수는 표시하지 않는다는 원칙을 형태로 강제한 것입니다.

새 발견 하나가 나옵니다. **"진행 중이지만 끝나는 시점이 보이지 않는 업무"** — 매주 1%씩 꾸준히 올라가는 업무는 정체도 아니고 이슈도 없어서 어느 현황판에서도 정상으로 보이지만, 자기 숫자로는 1년이 넘게 걸립니다. 기존 정체 규칙과 겹치지 않도록 정체 업무는 이 목록에서 제외합니다. 같은 업무를 두 제목 아래 두 번 적는 것이 위험 목록을 안 읽히게 만드는 방법입니다.

실측: 시드 데이터에 주당 1%씩 6주 진행한 업무를 넣자 정체·이슈 규칙은 모두 침묵했고, 이 규칙만 잡아냈습니다.

3.10.2 v0.49.0에서 완료 — 업무 마감일과 마감 대비 전망

v0.48은 "얼마나 더 걸리는가"까지만 답할 수 있었습니다. 늦는지 아닌지는 여전히 말할 수 없었는데, 비교할 마감일이 없었기 때문입니다.

- `work_items.due_date`에 값을 넣을 수 있게 했습니다. `PUT /api/v1/work-items/{id}/due-date`와 업무 추적 화면의 입력란입니다. 빈 문자열로 해제할 수 있습니다. 잘못 넣은 날짜를 다른 잘못된 날짜로만 바꿀 수 있다면 없느니만 못합니다.
- 마감일이 있으면 **그 날짜에 몇 %에 닿는지**를 두 속도로 각각 계산합니다. 둘 다 100%에 닿으면 `기한 내 예상`, 둘 다 못 닿으면 `기한 초과 예상`, 한쪽만 닿으면 `기한 불확실`입니다. 엇갈림 자체가 결론이므로 한쪽으로 합치지 않습니다.
- 추정의 기준점은 **마지막으로 보고된 주차**이지 오늘이 아닙니다. 한 달째 보고가 없는 업무를 오늘부터 계산하면 일하지 않은 4주를 실적으로 쳐주는 셈입니다.
- 마감일이 지났으면 아무것도 추정하지 않고 지났다고만 말합니다. 관측된 사실에 예측 문장을 붙일 이유가 없습니다.

이전에 제품이 가진 유일한 기한은 결정의 후속 조치 기한이었고, 그것은 **지난 뒤에만** 보고됐습니다. 16일에 15일을 놓쳤다고 알려 주는 것은 경고가 아니라 기록입니다.

발견한 것: 마감일을 저장한 직후 전망이 비어 있었습니다. 열려 있는 상세 대화상자에 `dueDate`만 덧씌우고 `dueOutlook`은 저장 전 값을 그대로 두었기 때문입니다. 마감일을 넣는 이유가 바로 그 답을 보려는 것이므로, 저장 후 목록을 다시 읽어 열려 있는 항목까지 교체합니다.

3.10.3 v0.50.0에서 완료 — 회의에서 합의된 기한을 마감일로

v0.49에서 마감일을 넣을 수 있게 만들었지만, 넣을 곳을 만들었다고 사람들이 채우지는 않습니다. `due_date`가 그동안 비어 있던 이유가 "입력할 곳이 없어서"였다면, 이제 남는 이유는 "채울 이유가 없어서"입니다.

그런데 기한은 원래 담당자가 정하지 않습니다. **회의에서 합의되고, 이 제품은 이미 그것을 기록하고 있습니다** — 결정의 후속 조치 기한입니다. 팀이 모여 "9월 21일까지"를 정하고 결정으로 적어도, 같은 업무의 마감

일 칸은 비어 있고 전망은 마감일이 없다고 답했습니다. 하나의 업무에 두 개의 날짜가 서로 다른 곳에 들어가 서로를 모르는 상태였습니다.

- 마감일이 없는 미완료 업무에 한해, 그 업무에 걸린 **열린 결정의 후속 기한 중 가장 이른 것**을 제안합니다. 가장 이른 것인 이유는 일정을 제약하는 것이 가장 가까운 약속이기 때문입니다.
- **복사하지 않고 제안합니다.** 후속 조치가 업무 전체와 같다는 보장이 없으므로, 조용히 마감일로 승격시키면 회의가 하지 않은 말을 하는 것이 됩니다. 날짜·결정자·후속 조치 문구를 그대로 보여 주고 한 번의 클릭으로 지정합니다.
- 이미 마감일이 있는 업무에는 제안하지 않습니다. 답이 있는 자리에 두 번째 날짜를 나란히 놓으면 답이 논쟁이 됩니다.

3.10.4 v0.51.0에서 완료 — 마감 위험을 경영 요약으로

v0.49·v0.50에서 만든 마감 위험은 업무 추적 화면에만 있었습니다. 그런데 기한을 실제로 처리하는 사람은 그 화면을 열지 않고 경영 요약을 봅니다.

경영 요약의 점수는 관측된 사실의 합이고 모든 항목이 근거를 되돌려 줍니다. 마감 위험을 여기 넣으면서 두 반쪽을 다르게 다뤘습니다.

- **기한 초과**는 관측입니다. 날짜가 지났고 완료되지 않았다는 것은 계산이 아니라 사실이므로, 얼마나 오래 그랬는지에 따라 점수가 올라갑니다.
- **기한 초과 예상**은 산술입니다. 그래서 고정 점수를 줍니다. 추정치의 크기가 순위를 끌게 두면 추정이 관측 위에 서게 됩니다.
- **기한 불확실**(두 속도가 엇갈리는 경우)은 넣지 않았습니다. 업무 추적 화면에서는 양쪽을 다 볼 수 있으니 유효한 발견이지만, 브리핑에서는 "아마도"이고 아마도로 가득한 브리핑은 읽히지 않게 됩니다.

실측으로 확인했습니다. 마감일이 없을 때 이 업무는 경영 요약에 아예 없다가(5건), 6주 뒤 기한을 넣으면 25점으로 맨 아래에 붙고(6건), 이미 지난 기한을 넣으면 40점으로 2위에 올라옵니다.

발견한 것: `summarizeWorkItem`이 멱등이 아니었습니다. 이슈·정체·계획 반복 카운터를 0으로 되돌리지 않고 `++`로 누적해서, 같은 뷰에 두 번 부르면 매주 진척이 오른 업무가 정체로 바뀝니다. 실제 경로에서는 한 번만 부르므로 드러나지 않았고, 이 회차의 테스트가 두 번 부르면서 걸렸습니다.

3.10.5 v0.52.0에서 발견 — 규모에서 드러난 것

작은 데이터로는 전부 빠릅니다. 300명·52주·109,200건으로 실제 인스턴스를 만들어 API 전체를 훑었더니 두 가지가 나왔습니다.

업무 목록이 23.6MB였습니다. 조직 전체 범위에서 업무 2,100건에 주차 스냅샷 109,200개가 실려 나왔고, 그중 19MB는 화면이 한 번도 그리지 않는 주차별 본문이었습니다. 목록 표는 제목·진척도·경과·상태만 읽고, 주차별 기록은 상세 대화상자를 열었을 때만 쓰입니다.

- 목록에서 `weeks`를 뺐습니다. 상세는 `GET /api/v1/work-items/{id}`로 한 건만 가져옵니다.
- 상한과 전체 건수를 함께 냅니다. v0.25에서 세운 규칙("목록을 반환하는 API는 상한과 전체 건수를 함께 갖습니다")을 이 경로만 지키지 않고 있었습니다. 화면도 "2,100건 중 200건"이라고 적습니다.
- 실측: **23,603,838B → 164,591B (143배)**. 상세 1건은 10,980B입니다.

나머지는 측정만 하고 남겨 둡니다. 경영 요약 1,183ms, 기간 보고(연간) 1,334ms, 업무 그래프 1,280ms입니다. 원인은 같습니다 — `loadWorkItems`가 109,200행을 전부 읽어 요약한 뒤 대부분을 버립니다. 고치려면 집계를 SQL로 내려야 하고, 그것은 이 회차의 범위를 넘습니다. 응답 크기와 달리 지연은 300명 규모에서 아직 견딜 만한 수준이라 순서를 뒤로 둡니다.

백필은 문제가 아니었습니다. 초기 로그의 속도(500건/3.7초)로 109,200건이면 13분으로 보였지만, 그것은 업무 2,100건을 새로 만드는 구간의 비용이었고 그 뒤로는 500건/0.25초로 올라가 전체 82초에 끝났습니다. 다만 진행 로그의 `pending`이 첫 배치의 총계에서 움직이지 않아, 지켜보는 사람에게는 남은 양이 얼어붙은 것처럼 보입니다.

3.10.6 v0.53.0에서 완료 — 새로고침마다 달라지던 협업 목록

v0.52에서 "지연은 나중에"라고 남긴 것을 재러 갔다가, 지연보다 큰 것을 먼저 찾았습니다.

같은 데이터·같은 바이너리로 업무 그래프를 두 번 부르면 협업 목록이 달라졌습니다. 팀 10의 짝이 한 번은 팀 9, 다음 번은 팀 2로 나왔습니다. 원인은 정렬의 동점 처리입니다. 행은 map에서 나오므로 도착 순서가 매번 다른데, 정렬 기준이 공유 업무 수와 왼쪽 조직명뿐이라 한 조직의 모든 짝이 동점이었고, `SliceStable`이 그 임의의 도착 순서를 그대로 보존했습니다. 오른쪽 조직명을 마지막 기준으로 넣어 순서를 전순서로 만들었습니다.

측정 방법이 틀렸던 것도 정정합니다. v0.52에서 기록한 경영 요약 1,183ms는 제가 만든 시드가 모든 제목을 "업무 N 처리"로 찍어 낸 탓이었습니다. 모든 쌍이 토큰을 공유하니 `linkWorkItems`의 2.2백만 번 비교가 전부 끝까지 갔습니다. 어휘를 흩뜨린 현실적인 제목으로 다시 재니 같은 규모에서 175ms입니다.

그 위에 최적화를 하나 얹었습니다. 의미 있는 단어를 하나도 공유하지 않는 쌍은 어차피 버려지는데 유사도를 먼저 계산하고 있었습니다. 단어 역색인으로 살아남을 수 있는 쌍만 비교합니다. **결과는 동일하고**(응답 바이트 단위로 대조), 업무 그래프 175–190ms → 121–154ms, 경영 요약 172–187ms → 101–115ms입니다.

3.10.7 v0.54.0에서 완료 — 결정성 전면 점검과 기간 보고 응답

v0.53에서 고친 비결정성이 한 곳뿐인지 확신할 근거가 없어서, 눈에 띄기를 기다리지 않고 기계적으로 훑었습니다.

결정성 점검은 깨끗했습니다. 300명·109,200건 인스턴스에서 주요 GET 28개를 각각 5회씩 불러 바이트 단위로 대조했습니다. 관리자와 팀장 두 역할로, 결정 700건과 선행·후행 링크 419건을 동점이 많이 생기도록 심은 뒤에도 어긋난 곳은 없었습니다. 기간 보고 세 개만 달랐는데 차이는 `generatedAt` 하나였습니다. v0.53의 `edges()`가 유일한 사례였습니다.

대신 훑는 중에 v0.52와 같은 구조를 하나 더 찾았습니다. 연간 조직 전체 기간 보고가 885KB인데 그중 522KB가 업무별 주차 계열이었습니다. 타임라인은 상위 20건만 그리고 아래 표는 주차 계열을 아예 읽지 않습니다. 나머지 236건의 주차별 본문은 내려받아 버려집니다.

- 앞의 `timelineItems` 건만 주차 계열을 싣습니다. 몇 건인지는 **서버가 정해서 응답에 담습니다** — 화면이 숫자를 따로 갖고 있으면 둘이 어긋나는 순간 빈 행을 그리게 됩니다.
- 항목 자체는 전부 보냅니다. 표는 그것을 다 씁니다.
- CSV·PPTX는 별도 경로이고 전체 뷰에서 만들므로 영향이 없습니다. CSV 257줄(헤더 포함 256건)을 확인했습니다.
- 실측: 연간 807KB → 332KB, 분기 512KB → 278KB, 월간 → 266KB.

3.10.8 v0.55.0에서 완료 — 규모 점검을 명령 한 줄로

v0.52·v0.53·v0.54의 발견은 셋 다 큰 데이터에서만 드러났습니다. 그런데 그 방법이 제 손안에만 있었습니다. 매번 시드 SQL을 새로 쓰고 훑는 스크립트를 새로 짜는 한, 다음 기능이 같은 함정을 다시 만들어도 아무도 모릅니다.

- `scripts/seed-scale.sql` — 조직 41개·사용자 300명·52주·보고 항목 109,200건. 빈 데이터베이스에만 실행되도록 막아 두었습니다.
- `scripts/scale-check.py` — 주요 조회 경로를 5회씩 불러 **크기·지연·결정성·상한 표기**를 봅니다. 지적 사항이 있으면 종료 코드 1입니다.

만들자마자 두 가지를 잡았습니다.

회의 모드의 변경점 섹션이 2,100건이었습니다. 상한도 전체 건수도 없이 전 조직 업무를 그대로 실었습니다. "회의에서 다뤄야 할 것만" 보여 준다는 화면이 전 조직 목록을 안건이라고 내놓고 있었습니다. 섹션마다 40건 상한과 전체 건수를 넣고, 잘라 낼 때 **덜 중요한 것이 잘리도록** 정렬했습니다 — 진척이 뒤로 간 업무가 먼저, 그다음 변화 폭이 큰 순서입니다. 예전에는 도착 순, 즉 업무 식별자가 작은 순이었는데 그건 논의할 이유가 아닙니다. 회의 응답 744,820B → 21,291B.

업무 인사이트의 반복 업무 목록이 934건이었습니다. 같은 응답 안의 유사 업무·중복 의심은 상한과 전체 건수를 갖고 있는데 이것만 없었습니다. 같은 상한(200)과 전체 건수를 붙였습니다.

체크 자체도 한 번 고쳤습니다. 처음에는 `duplicates`의 개수를 `duplicateTotal`(단수)로 세는 것을 못 알아보고 다섯 번 연속 오탐을 냈습니다. 우는 늑대가 되는 체크는 읽히지 않게 되므로, 개수 이름의 여러 표기를 인정하고 한 단계 아래 중첩된 개수도 찾도록 고쳤습니다.

시드도 두 번 고쳤습니다. 처음에는 마지막 주 진척이 전부 100%에 닿아 "지난주 대비 변화"가 0건이 됐고, 그다음에는 PostgreSQL이 UTC라 일요일 저녁에 시드의 마지막 주가 애플리케이션이 보는 주보다 한 주 앞이었습니다. 후자는 **보고 누락 1,866건 · 변경점 0건**으로 나타나 변화 감지 버그처럼 보였지만 시드 버그였습니다.

남은 것(체크가 지적한 5건): 기간 보고의 `contributors`(300건), `/team/members`, `/admin/users`가 전체 건수 없이 전량을 반환합니다. 잘리지는 않지만 사용자 1만 명이면 메가바이트 단위가 됩니다.

3.10.9 v0.56.0에서 완료 — 규모 점검 지적 사항 0건

v0.55의 체크가 남긴 5건을 처리했습니다. 처리하면서 셋 다 성격이 달랐다는 것이 드러났습니다.

`/admin/users`는 진짜 문제였습니다. 전량을 반환했고(300명에 63KB), 화면에는 검색이 아예 없어서 특정 계정을 찾는 방법이 브라우저의 페이지 내 찾기뿐이었습니다. 1만 명이면 메가바이트입니다.

- 상한(기본 100)과 전체 건수를 넣되 **검색을 함께** 넣었습니다. 찾을 방법 없는 상한은 무제한보다 나쁩니다 — 필요한 계정이 그냥 사라지기 때문입니다. `q`는 아이디·표시 이름·이메일을 함께 봅니다.
- **검토 책임자 선택기가 표의 목록을 재사용하고 있었습니다.** 상한을 걸면 조직이 커질수록 검토자가 조용히 빠지고, 보는 사람은 그 이름이 자격이 없어서 없는 건지 페이지 밖이라서 없는 건지 알 수 없습니다. 역할 필터(`role=`)로 따로 불러오게 했습니다. 검토자는 조직이 아무리 커도 작은 집합입니다.
- 실측: 63,354B → 21,195B. 검색 `u100` → 1건, 역할 필터 → 13명.

나머지 4건은 위반이 아니었습니다. 기간 보고의 `contributors`와 `/team/members`는 전량을 반환합니다 — 잘리지 않으니 "일부를 전부처럼 보여 준다"는 규칙을 어기지 않습니다.

그래서 체크가 **추측 대신 확인**하게 고쳤습니다. 개수가 없는 큰 목록을 만나면 같은 경로를 `limit=1`로 한 번 더 불러 봅니다. 응답이 바이트 단위로 같으면 그 경로는 페이징을 하지 않는다는 뜻이고, 전량 반환이므로 완전합니다. 짧아지면 말없이 페이징하고 있다는 뜻이고 그것이 진짜 위반입니다. 전자는 "상한 없이 전량 반환"으로 따로 보고하되 종료 코드에 반영하지 않습니다.

이제 `scripts/scale-check.py`가 **22개 경로, 지적 사항 0건**으로 끝납니다.

3.10.10 v0.57.0에서 발견 — 월요일 아침 로그인이 컨테이너를 죽입니다

지금까지의 측정은 전부 **요청 하나씩**이었습니다. 응답 크기도 질의 계획도 건강한데, 실제로 배포를 무너뜨리는 실패는 거기 없었습니다.

300명 규모에 동시 300 세션(쓰기 250·읽기 50)을 걸자 컨테이너가 **exit 137, OOM으로 죽었습니다.** 최대 메모리 12GB였습니다.

원인을 짚기 전에 제 귀인이 한 번 틀렸습니다. 경로별로 메모리를 재니 네 경로 모두 30 동시에 2GB였고 응답이 48KB인 경로까지 그랬습니다. 데이터 크기로 설명되지 않아 힙 프로파일을 떴더니 **98%가** `argon2.initBlocks`, 즉 로그인이었습니다. 조회 경로가 아니라 부하 스크립트가 매 스레드에서 로그인하고 있었던 것입니다.

Argon2id는 검증 한 번에 64MiB를 잡고 그 시간 동안 붙들고 있습니다. 로그인이 하나씩 올 때는 아무도 눈치채지 못합니다. **월요일 아홉 시에 250명이 동시에 들어오면 그들끼리 16GB를 잡습니다** — 그 주의 첫 요청이, 그 주에서 가장 바쁜 1분에.

- 검증과 해싱을 큐에 넣었습니다. 동시 실행은 코어 수를 따르되 **최소 2, 최대 8**입니다. 검증 1회가 61ms이므로 조직 전체가 몇 초 안에 로그인하고, 최대 메모리는 무엇이 오든 8 × 64MiB로 묶입니다.
- 상한이 8인 이유는 그 이상이 처리량을 사지 못하기 때문입니다(CPU·메모리 바운드이지 대기가 아닙니다). 반면 슬롯 하나는 컨테이너가 갖고 있어야 할 64MiB입니다.
- **비용을 낮추는 선택은 하지 않았습니다.** 그것은 스케줄링 문제를 풀자고 저장된 모든 비밀번호를 약하게 만드는 일입니다.

실측(동시 250 쓰기·50 읽기·4회전):

	이전	이후
최대 메모리	12.01 GiB	1.14 GiB (실사용 힙 512MB = 8×64MiB)
저장 중앙값	264ms	18ms
저장 p99	390ms	51ms
조회 p99	782ms	204ms
결과	OOM 종료	오류 0건

`scripts/load-check.py`로 남겼습니다. 다음에 같은 실패가 생기면 손으로 재현하지 않아도 됩니다.

3.10.11 v0.58.0에서 완료 — 방금 만든 상한이 배포 한도를 통째로 먹고 있었습니다

v0.57에서 argon2 동시 실행을 8개로 묶었습니다. 그러고 나서 배포 매니페스트를 열어 보니 메모리 한도가 **512Mi**였습니다. 8 × 64MiB = 512MiB. 상한이 한도 전부입니다.

매니페스트 그대로(cpu 1, memory 512Mi) 재현하니 v0.57 수정이 있어도 `OOMKilled=true`로 죽었습니다.

원인은 둘이었습니다.

첫째, 풀 크기가 컨테이너를 모릅니다. `runtime.NumCPU()`는 호스트의 코어 수를 봅니다. 32코어 노드에서 CPU 1개짜리 파드도 슬롯 8개를 열었습니다. OOM을 막으려고 만든 상한이 스스로 OOM을 냈습니다. 이제 cgroup v2의 `memory.max`를 읽어 **감당 가능한 만큼만** 엽니다. 메모리가 구속 조건이므로 메모리가 결정하고, CPU는 그보다 더 줄일 때만 관여합니다.

둘째, Go에게 한도를 알려 주지 않았습니다. 5슬롯 320MiB로 줄여도 여전히 죽었습니다. 런타임은 30GB가 남은 기계를 보고 회수를 미루므로, 다 쓴 argon2 블록이 쌓여 RSS가 실사용의 두 배에 닿습니다. `debug.SetMemoryLimit`으로 컨테이너 한도의 7/8을 알려 주자 멈췄습니다.

기동 시 `password hashing pool sized`로 슬롯 수·예약량·읽은 한도를 남깁니다. 운영자가 한도를 조정할 때 볼 숫자가 그것뿐이기 때문입니다.

실측(300명·109,200건, 동시 300 세션):

한도	슬롯	결과
512Mi / 1 CPU (수정 전)	8 (호스트 기준)	**OOM 종료**
512Mi / 1 CPU (수정 후)	5	오류 0건, 48.8초, 최대 86%
1Gi / 2 CPU (수정 후)	8	오류 0건, 31.3초

매니페스트 기본값을 1Gi / 2 CPU로 올리고 근거 숫자를 주석에 적었습니다. 512Mi도 동작하며 로그인이 느려질 뿐입니다 — 큐가 그것을 장애가 아니라 지연으로 만들어 줍니다.

3.10.12 v0.59.0에서 완료 — 정상 동작이 로그를 채우고 있었습니다

v0.57 부하 시험 중에 눈에 걸렸지만 지나쳤던 둘을 봤습니다. 하나는 진짜였고 하나는 제 짐작이 틀렸습니다.

로그 폭주는 진짜였습니다. 부하 3.6초 동안 `report list truncated`가 60줄 찍혔고, **그 시간의 유일한 로그가 그것이었습니다.** 그때 실제로 무언가 잘못됐다면 "모든 것이 정상"이라는 보고 속에 묻혔을 겁니다.

상한이 목록을 자르는 것은 **사건이 아니라 상태**입니다. 이 배포가 누군가 정한 한도보다 크다는 뜻이고, 그것은 요청마다 참입니다. 요청마다 쓰는 것은 상태를 사건처럼 기록하는 일입니다.

- `conditionLog`을 만들어 조건마다 **프로세스당 한 번만** 기록합니다. 재시작하면 다시 말하는데, 그때는 운영자가 어차피 보고 있습니다.
- 숫자를 잃지 않습니다. 잘린 응답은 이미 `total`과 `limit`을 담고 있고, 요청 지표 테이블이 호출 수를 셉니다.
- 같은 유형이던 `work graph truncated`도 함께 옮겼습니다.
- 실측: 같은 부하에서 **60줄 → 1줄**.

지표 행 경합은 제 짐작이 틀렸습니다. 요청마다 같은 `(시각·메서드·경로·상태)` 행을 upsert하므로 줄을 설 것이라 의심했는데, 재 보니 그 행 하나가 16-way 동시에서 **초당 9,411건**을 처리합니다. 앱의 최대치는 초당 38건이었습니다. 250배 여유입니다. 부하 중 `pg_stat_activity` 표본 60개에서 락 대기는 1건, 최대 동시 1이었습니다.

고칠 것이 없으므로 고치지 않았습니다. 숫자를 남기는 이유는 다음에 같은 의심이 들 때 다시 재지 않아도 되게 하기 위해서입니다.

3.10.13 v0.60.0에서 완료 — 마감일 없이 기간 말을 묻습니다

v0.48~v0.51은 네 회차에 걸쳐 완료 시점 추정, 마감일 필드, 회의 합의 기한 연결, 경영 요약 반영을 만들었습니다. 전부 하나의 질문에 답합니다 — 제때 끝나는가. 그리고 전부 `work_items.due_date`에 걸려 있는데, 실측이 **8건 중 0건**이었습니다. 아무도 입력할 이유가 없는 칸 하나에 네 회차가 묶여 있었습니다.

그런데 **기간 보고는 이미 그 질문을 하고 있습니다.** 분기 보고를 여는 사람은 "이번 분기에 뭐가 끝나나"를 묻고 있고, 그 경계는 사람이 아니라 달력이 정해서 요청 안에 이미 들어 있습니다. 같은 산술을 그 경계에 대고 돌리면 아무도 아무것도 입력하지 않아도 됩니다.

- 항목마다 `periodOutlook` — 두 속도로 기간 말 진척을 각각 계산해 구간으로 냅니다.
- **약속이 아니며 그런 척하지 않습니다.** 마감일은 누군가 약속했다는 뜻이고 기간 말은 보고가 여기서 끊긴다는 뜻입니다. 문구를 갈라 놓았고, 마감일이 있는 업무는 자기 전망을 따로 계속 갖습니다.
- **이미 끝난 기간에는 추정하지 않습니다.** 결과가 이미 있는데 내놓는 추정은 예보가 아니라 예보처럼 보이는 잡음입니다.
- 기간 업무보고에 `기간 내 미완` 필터와 경영 인사이트 항목이 붙습니다.

만들면서 두 번 고쳤습니다. 첫 판은 263건 중 **217건**을 지적했는데 그건 발견이 아니라 목록 전체입니다. 원인은 진척이 늘지 않은 업무까지 추정한 것이었고 — 25%에 서 있는 업무에 "기간 말 15%"라고 답했습니다. 늘릴 속도가 없으면 추정하지 않고 정체 규칙에 맡깁니다. 그리고 완료 시점 자체가 없는 업무는 이미 `완료 시`

`점 불명`이 보고하므로 이 제목에서 뺐습니다 — 같은 업무를 두 제목 아래 두 번 적는 것이 위험 목록을 안 읽히게 만드는 방법입니다.

실측: 13주에 걸쳐 주당 2.5%로 움직인 업무에 대해 마감일 입력 없이 **"이 속도로는 기간 말(2026-09-30)에 52%"**. 종료된 분기(2026-Q1)에서는 256건 전부 침묵합니다.

3.10.14 v0.61.0에서 완료 — 장애물이 어떻게 끝났는지 기록합니다

이슈는 주간보고에 나타났다가 몇 주 뒤 사라집니다. 지금까지 제품이 아는 것은 **중간**뿐입니다 — 이슈 3주 지속, 경영 요약에서 점수. 끝은 아무도 기록하지 않습니다. 그래서 **해결된 것과 포기한 것이 데이터에서 똑같아 보입니다.**

중간은 이미 주차별 행에서 유도할 수 있으니 여기서 다시 만들지 않습니다. 끝만 사람이 필요합니다. 이슈란이 비었다는 것이 문제가 사라졌다는 뜻인지 적기를 그만뒀다는 뜻인지는 그 주에 작성한 사람만 압니다.

- 마이그레이션 018 `work_item_issue_outcomes`. 끝난 주·시작한 주·이어진 주 수·해소 문구를 담습니다.
- 편집기가 **그 한 주에 한 번** 묻습니다. 지난주 이슈가 있고 이번 주가 비었을 때만, 그 항목에만. `해소됨`을 고르면 저장 시 기록하고, `아직 남음`을 고르면 지난주 문구를 이번 주 이슈란에 되돌려 넣습니다 — 후자는 조용히 빠진 장애물을 되살리는 교정이라 그 자체로 값이 있습니다.
- 이어진 주 수는 **연속된 주만** 셉니다. 3월에 막혔다가 조용해지고 8월에 다른 것에 막힌 업무는 다섯 달짜리 장애물 하나가 아니라 두 개입니다. 공백을 건너뛰면 이 표 위에 세우는 모든 숫자가 부풀려집니다.
- 같은 주를 두 번 저장해도 한 번만 기록합니다. 보고서를 다시 저장하는 일은 일상이고, 그때마다 해소가 하나씩 늘면 안 됩니다.
- 기간 보고에 **이슈 해소** 지표 — 건수와 중앙값·최장. 평균이 아니라 중앙값인 이유는 1년 끌던 장애물 하나가 평균을 실제 장애물이 서 있지 않은 곳으로 끌고 가기 때문입니다.
- 아무도 답한 적이 없으면 0이 아니라 `–`로 적습니다. 0은 "아무것도 시간이 안 걸린다"로 읽히고 그건 빈 표가 뜻하는 것의 반대입니다.

실측: 1주짜리와 4주짜리 에피소드를 각각 만들어 정확히 1과 4로 기록되는 것을, 그리고 4주짜리에서 그 앞 공백 이전의 이슈는 세지 않는 것을 확인했습니다. 기간 보고는 `2건 해소 · 중앙값 1주 · 최장 4주`를 표시합니다.

이 표는 3.10.15(조직을 가로지르는 공유 장애물)의 재료이기도 합니다. 같은 장애물을 묶을 수 있게 되면 "이 장애물은 보통 N주 걸립니다"를 붙일 수 있습니다.

3.10.15 v0.62.0에서 완료 — 막혔다고 쓰는 자리에서 무엇에 막혔는지 말하게

"조직을 가로지르는 공유 장애물"을 만들려고 이슈 텍스트 군집화를 제안했었습니다. **먼저 재 봤고, 만들지 않았습니다.** 군집을 평가할 실제 이슈 말뭉치가 어디에도 없습니다 — 시드는 2종(하나가 15,351회), 데모 데이터는 3종 12행입니다. 모델 품질을 잴 수 없는 기능을 만드는 것은 이 프로젝트가 Slack 알림과 문장 단위 근거에 대해 거부했던 것과 같은 일입니다.

대신 재는 중에 훨씬 확실한 것을 찾았습니다. **이슈 12건, 선언된 선행 링크 0건.**

v0.40~42에서 선행·후행 링크, 병목 분석, 회의 안건의 `타 조직 대기`, 품질 점검의 차단 사유를 만들었습니다. 전부 링크를 읽습니다. 그리고 링크는 **한 건도 만들어지지 않았습니다.** 마감일이 v0.60 전까지 그랬던 것과 정확히 같은 상태 — 완성돼 있고 구조적으로 도달 불가능합니다.

이유는 둘이었습니다.

- **선언하는 자리가 막혔다고 쓰는 자리와 다른 화면입니다.** 이슈는 주간보고 편집기에 쓰고, 선행 관계는 업무 추적 상세에서 등록합니다. 이슈를 쓰는 곳에 선언을 가져왔습니다. 이미 등록한 사람에게는 등록된 내용을 보여 주고 다시 묻지 않습니다.
- **일반 사용자는 남의 업무를 찾을 수 없었습니다.** 패널이 쓰던 `/work-items/search`는 기본이 SELF 범위이고 팀장 미만에게는 조직 범위를 거부합니다. 선행 관계란 본디 남의 업무를 가리키는 것인데, 가리킬 대상을 찾는 길이 정작 막힌 사람 거의 전부에게 닫혀 있었습니다.

`/work-items/lookup`을 새로 냈습니다. 배포 전체를 보되 **선언이 어차피 화면에 표시할 네 가지만** 돌려줍니다(식별자·제목·담당자·조직). 기존 검색이 싣는 이슈·해결 본문은 담지 않습니다. 노출이 넓어지는 변경이므로 코드와 문서에 그대로 적어 두었습니다.

실측: 일반 사용자(팀 6)가 편집기에서 **다른 조직(팀 9)의 업무**를 선행으로 등록했고, 그것이 팀장의 회의 자료 `타 조직 대기`에 나타나는 것까지 확인했습니다. 2글자 질의도 답합니다.

3.10.16 v0.63.0에서 완료 — 주간보고 제품에 주간 취합이 없었습니다

"팀장 보고서를 팀원 보고서에서 AI로 초안 생성"을 제안했었습니다. **만들기 전에 쟀고, 전제가 틀렸습니다.**

데모 데이터에서 팀장(choi)의 주간보고와 팀원 보고서의 **겹침이 0**입니다. 팀장은 팀원 것을 요약하지 않고 자기 업무("조직 KPI 정리")를 씁니다. 이 제품의 모델에서 팀 취합은 개인 보고서가 아니라 별도 화면의 일입니다.

그래서 진짜 빈 곳을 찾았습니다. `PeriodKind = MONTH | QUARTER | HALF | YEAR` — **주간이 없습니다.** 팀 주간보고 화면은 팀원 보고서를 한 건씩 나열하고, 중복을 제거해 한 장으로 만드는 취합은 월간부터 시작합니다. 주간보고 제품에서 주 단위 팀 취합이 빠져 있었습니다.

- `kind=WEEK`를 더했습니다. 식별자는 그 주의 시작일(`yyyy-mm-dd`)로, `weekly_reports.week_start`와 같은 형식입니다. ISO 주차 번호를 새로 만들면 한 주를 부르는 방식이 둘이 되고 시작 요일을 바꾸는

순간 어긋납니다.

- 시작일이 아닌 날짜를 주면 그 주로 맞춥니다. 수요일을 넘긴 호출자는 그 주를 뜻하지, 수요일부터 화요일까지를 뜻하지 않습니다.
- 주 시작 요일은 배포 설정이므로 경계도 그것을 따릅니다.
- 아래쪽은 전부 그대로 동작합니다 — 중복 제거, 인사이트, 하이라이트, CSV·PPTX 내보내기, 발표 모드.

실측: 팀장이 팀원 10명의 주간보고를 **업무 21건으로 정리한 한 장**으로 봅니다. CSV 56건, PPTX 15슬라이드.

3.10.17 v0.63.0에서 함께 고침 — UTC로 하루씩 밀리던 날짜 네 곳

주간 선택기를 브라우저에서 확인하다가 잡았습니다. 선택지는 `2026-08-23 (이번 주)`인데 서버는 `2026-08-17 주차`로 답했습니다. `Date.toISOString()`이 UTC로 답하므로, KST 자정은 전날이 됩니다.

같은 실수가 네 곳에 있었고 세 곳은 이미 배포돼 있었습니다.

- **회의 모드의 주 이동이 하루씩 어긋났습니다.** 08-24에서 `다음 주 ▶`를 누르면 08-31이 아니라 **08-30** — 어느 주의 시작도 아닌 날짜이고, 그 주 안건은 아무것도 찾지 못합니다.
- 결정 기록 화면이 오전 9시 이전에 기록하는 사람에게 **어제 날짜**를 기본값으로 내놓았습니다.
- 주간 선택기가 지난주를 "이번 주"라고 적었습니다.
- 타임라인 차트의 주차 키.

`localdate.ts` 하나로 모았습니다. 브라우저가 이미 지역 시간으로 갖고 있는 값을 읽지, 다른 시간대로 바꿔 문자열을 자르지 않습니다. Go 가드 테스트가 `toISOString().slice(0,10)`의 재등장을 막습니다 — 이 실수는 호출부에서 완벽히 합리적으로 보이고, 아무도 테스트하지 않는 시간대에서만 드러납니다.

3.10.18 v0.64.0에서 완료 — 만들어 두고 부르지 않은 것을 기계로 찾습니다

세 회차 연속 같은 모양이었습니다. 완성돼 있고, 테스트도 문서도 있고, **아무 화면도 부르지 않아** 쓸 수 없는 기능. 마감일(v0.60), 선행 링크(v0.62), 주간 취합(v0.63). 셋 다 코드를 읽어서가 아니라 실행 중인 배포를 재다가 빈 표를 보고 찾았습니다.

그래서 싸게 훑었습니다. 서버가 등록한 96개 경로를 프런트엔드 소스와 대조했습니다. 진짜 후보는 셋이었고, 그중 하나가 실물이었습니다.

보고서 코멘트가 한쪽으로만 흐르고 있었습니다. 검토자가 할 수 있는 일은 승인 또는 반려 둘뿐이라, 질문 하나를 하려면 반려해야 했습니다 — 상태를 바꾸고 남의 책상으로 일을 돌려보내는 일입니다. 대부분은 그래서 아무 말도 하지 않습니다. 작성자 화면은 `검토 의견`을 **보여주기만** 했고, 그 칸은 반려로만 채워질 수 있었으며, 답할 방법이 없었습니다.

`POST /reports/{id}/comments`는 권한 검사·길이 제한·감사 기록까지 갖춘 채 아무도 부르지 않고 있었습니다. 양쪽에 자리를 만들었습니다. **보고서 상태는 바뀌지 않습니다.**

나머지 둘은 이유를 적었습니다 — 후보 출처는 목록 응답이 이미 담고 있고, 미매핑 사용자는 `users/mappings`가 전체 사용자를 LEFT JOIN으로 담아 매핑 제안까지 주는 것의 필터판입니다.

가드를 남기면서 한 번 고쳤습니다. 첫 판은 접미사를 느슨하게 찾아, CSS 클래스 `"comments"`가 검사를 통과시켰습니다. 코멘트 호출을 지워도 테스트가 통과했습니다 — **짖지 못하는 감시견은 없느니만 못합니다. 믿게 되니까요.** URL 경로 조각(`/comments`)으로만 인정하게 좁히고, 경로를 `${id}/${action}`으로 조립하는 승인·반려는 이유를 적어 예외로 두었습니다.

3.10.19 v0.65.0에서 완료 — 반대 방향으로도 훑었습니다

v0.64가 "서버가 가진 것 중 화면이 안 부르는 경로"를 찾았습니다. 반대도 같은 값이 있습니다 — 데이터베이스와 응답이 가진 것 중 아무도 읽지 않는 것.

컬럼 쪽은 깨끗했습니다. 284개 중 Go 코드가 언급하지 않는 것은 둘뿐이고 (`schema_migrations.applied_at`은 기본값으로만 쓰이는 기록용, `work_items.parent_work_item_id`는 쓰이지 않는 계층 구조 자리), 전부 NULL인 컬럼 14개는 모두 데모가 결재 흐름을 거치지 않은 결과로 설명됩니다. 특히 `report_status_history.from_status`가 35행 전부 NULL이라 의심했지만, 데모에 초안 생성만 있고 **생성에는 이전 상태가 없는 것이 맞습니다.**

응답 필드 쪽에서 하나 나왔습니다. 371개 중 화면이 이름조차 언급하지 않는 것이 20개인데, 19개는 남의 전문입니다(Confluence·OpenAI·JSON-RPC·OIDC). 우리 것은 하나 — `movedItems`.

병합·분리 API는 **실제로 옮겨진 주차 기록 수**를 돌려주고, 서버 주석은 그것을 "정정이 의도대로 됐는지 알려주는 유일한 숫자"라고 적어 두었습니다. 화면은 그것을 버리고 **요청한 수**를 알렸습니다. 합치기는 아예 수를 말하지 않았습니다.

이제 실제 수를 말합니다. 분리는 요청 수와 다를 때 다르게 적고(서버가 어긋난 id를 400으로 막으므로 지금은 방어적입니다), 합치기는 옮겨진 기록 수를 알립니다 — 화면이 전에는 알 수 없던 숫자입니다.

가드를 하나 더 남겼습니다. 응답에 실려 나가는데 화면이 이름조차 언급하지 않는 필드가 생기면 실패합니다. 남의 전문은 파일 단위로 제외하고, 정당한 예외는 이유를 적어야 통과합니다. `movedItems`를 다시 버리게 만들어 실제로 실패하는 것을 확인했습니다.

3.10.20 v0.66.0에서 완료 — 데이터베이스가 끊겼을 때 화면이 하던 거짓말

지금까지 잰 것은 전부 정상 흐름이었습니다 — 크기, 지연, 동시성, 도달 가능성. 오프라인망 배포에서 실제로 사람을 곤란하게 만드는 것은 보통 **잘못됐을 때 화면이 뭐라고 하는가**입니다. 그래서 PostgreSQL 연결을 실제로 끊고 봤습니다.

가장 중요한 것은 이미 옳았습니다. 보고서를 쓰던 중에 데이터베이스가 사라져도 작성 중인 내용이 남고, 복구 후 그대로 저장됩니다.

그런데 **화면이 두 번 거짓말했습니다.**

- 저장을 누르면 **"로그인이 필요합니다"** 라고 했습니다. 세션 조회가 데이터베이스를 거치는데, "세션이 없다"와 "세션을 확인할 수 없다"를 같은 오류로 돌려주고 있었습니다. 게다가 화면은 401을 세션 만료로 다루므로 **새 탭에서 로그인하라고 권했고**, 그것을 따르면 쓰던 보고서가 사라집니다.
- 로그인을 누르면 **"아이디 또는 비밀번호가 올바르지 않습니다"** 라고 했습니다. 사용자 조회의 오류가 자격 증명 실패 분기로 흘러들었기 때문입니다. 더 나쁜 것은 그 시도가 로그인 차단 카운터에 기록된다는 점입니다 — **가장 열심히 재시도한 사람이 잠깁니다.** 한 번도 그들을 거부한 적 없는 서비스에서.

둘 다 원인이 같습니다. **모른다는 것과 아니라는 것을 구분하지 않았습니다.** `errNoSession` 표지를 두어 진짜로 로그인하지 않은 경우에만 401을 주고, 나머지는 503과 "로그인은 유효하고 작성 중인 내용도 남아 있으니 잠시 후 다시"를 말합니다.

그리고 매달렸습니다. 로그인 화면은 20초, 로그인 요청은 25초 넘게 응답이 없었습니다. 원인은 곱셈입니다 — 실패하는 질의 하나가 연결 시도 시간을 다 쓰는데 핸들러는 여러 번 질의합니다(로그인 화면만 설정 4개). 각 호출에 기한을 붙였습니다. 이 패턴은 이미 `readyz`가 자기 질의에 쓰고 있었고, 다른 경로에만 없었습니다.

	이전	이후
로그인 화면	20초 매달림	5.6초 · 200(기본값으로 렌더)
로그인 요청	25초 매달림	12초 · 503 · 정확한 원인
저장 중 장애	"로그인이 필요합니다" + 세션 만료 배너	원인을 말하고, 배너 없음
작성 중이던 내용	유지됨	유지됨

12초는 여전히 깁니다. 설정·차단 카운터·사용자 조회의 기한이 순차로 더해진 값이며, 한 번 실패한 뒤 잠시 즉시 실패시키는 장치를 두면 줄어듭니다. 지금은 **무한정 매달리지 않고 정확한 답을 준다**는 것까지가 확인된 범위입니다.

3.10.21 v0.67.0에서 완료 — 실패한 업로드가 디스크를 영구히 먹고 있었습니다

v0.66이 데이터베이스를 끊었으니 이번엔 디스크를 채웠습니다. 320KB짜리 볼륨에 이미지를 올려 봤습니다.

구조는 이미 옳았습니다. 업로드는 파일을 먼저 쓰고 행을 나중에 넣으므로, 쓰기가 실패하면 행이 생기지 않습니다. "목록에는 있는데 열면 404"가 되는 순서가 아닙니다. 백업 스크립트가 같은 이유로 그 순서를 지킨다고 적어 둔 것과 일치합니다.

그런데 실패한 쓰기가 조각 파일을 남겼습니다. `os.WriteFile`은 대상 파일을 만들고 그 안에 씁니다. 디스크가 도중에 차면 잘린 파일이 최종 이름으로 남습니다. 행은 없으니 그 파일을 가리키는 것도 없고, 이 제품의 어떤 정리 경로도 그것을 찾지 못합니다 — 부팅 시 점검은 행에서 파일을 찾지, 반대로는 보지 않습니다.

실측: **실패 한 번이 320KB 중 319KB를 먹고 그대로 잠갔습니다.** 이후 모든 업로드가 실패합니다.

고치는 방법은 이 코드베이스 안에 이미 있었습니다. PPTX 템플릿과 Import 저장소는 **임시 파일에 쓰고 이름을 바꾸며** 실패 시 `defer os.Remove`로 치웁니다. 첨부만 맨 `os.WriteFile`이었습니다. 같은 패턴으로 맞췄고, 덤으로 최종 이름이 절반만 쓰인 파일을 가리키는 순간이 사라집니다.

메시지도 고쳤습니다. "이미지를 저장할 수 없습니다"는 고칠 수 있는 사람(관리자)이 보지 않는 말이었습니다. 공간 부족일 때는 507과 함께 원인과 확인할 볼륨을 말합니다.

	이전	이후
실패 후 디스크	100% 사용, 영구	1% 사용, 회복
남은 조각 파일	319KB	없음
응답	500 "저장할 수 없습니다"	507 원인과 조치
DB 행	생기지 않음	생기지 않음

3.10.22 v0.68.0에서 완료 — 네 가지 고장이 한 문장으로 나오고 있었습니다

데이터베이스(v0.66)와 디스크(v0.67)를 끊어 봤으니, 실패 경로에서 남은 절반은 **AI 게이트웨이와 Confluence**였습니다. 응답하지 않는 주소, 401을 주는 서버, HTML을 돌려주는 엔드포인트를 실제로 붙여 놓고 화면이 뭐라고 하는지 읽었습니다.

AI 쪽은 네 가지 서로 다른 고장이 한 글자도 다르지 않은 문장으로 나왔습니다.

실제 고장	이전 화면 문구
API Key가 거부됨(401)	AI Gateway가 유효한 구조화 결과를 반환하지 못했습니다. 관리자 설정과 모델 지원 여부를 확인하세요.
게이트웨이 내부 오류(500)	(완전히 같은 문장)

실제 고장	이전 화면 문구
프록시가 로그인 페이지를 돌려줌	(완전히 같은 문장)
모델이 Structured Output 미지원	(완전히 같은 문장)

넷 중 셋에 대해 그 문장은 **틀린 곳을 가리킵니다.** 키가 거부된 관리자는 모델 지원 여부를 확인하러 가고, 프록시가 가로챈 관리자도 같은 곳으로 갑니다. 마지막 한 줄만 우연히 맞습니다.

이제 일곱 가지 고장이 각각 다른 말을 하고, 각자 **다음에 무엇을 할지**를 말합니다. 401/403은 API Key를, 404는 Endpoint가 `/v1/chat/completions`로 끝나는지를, 5xx는 "게이트웨이 쪽 문제이므로 이 설정을 바꿔도 해결되지 않습니다"를, 비-JSON 응답은 프록시나 로그인 페이지를, 연결 실패는 사내망 접근 경로를 가리킵니다.

Confluence 쪽은 반대 문제였습니다 — 말을 안 한 게 아니라 내부를 그대로 뱉었습니다.

```
decode Confluence response: invalid character '<' looking for beginning of value
Get "http://confluence.internal:8090/rest/api/content/search?cql=type+%3D+page+and+lastmod…
```

설정 화면은 내부 주소와 질의문이 처음 등장할 자리가 아닙니다. 원문은 로그로 보내고, 화면에는 401이면 연동 계정을, 404면 Base URL이 Confluence 루트인지를, 인증서 오류면 사내 CA를 확인하라고 적습니다.

가드가 실제로 무는지 확인했습니다. `aiUserMessage`를 원래의 한 문장으로 되돌리자 테스트가 실패했습니다. 그런데 Confluence 테스트는 원문 노출을 되돌려도 **통과했습니다** — 사례가 전부 앞쪽 분기에 걸려 마지막 폴백을 건드리지 않았기 때문입니다. 알아보지 못한 오류야말로 아무도 문구를 읽어 본 적 없는 오류이므로, 그 경로를 덮는 사례를 더해 다시 확인했습니다. 짖지 못하는 감시견은 없느니만 못하다는 v0.64의 교훈이 이번에는 절반만 짖는 형태로 나타났습니다.

원칙으로 남깁니다. **서로 다른 원인은 서로 다른 문장으로 말합니다.** 원인을 구분하지 못하는 하나의 문장은 넷 중 셋을 틀린 곳으로 보냅니다.

3.10.23 v0.69.0에서 완료 — 나오지 않은 것은 아무도 세지 않았습니다

v0.68까지 확인한 AI 실패는 전부 **답을 못 받는** 실패였습니다. 남은 것은 반대쪽입니다. 게이트웨이가 200을 주고, 스키마에 맞는 JSON을 주고, 신뢰도까지 0.9를 붙여 놓았는데 **내용이 모자란** 경우.

10장짜리 덱(그중 2장은 빈 간지)을 올리고, 8장의 내용 중 3장만 인용하는 완벽하게 유효한 응답을 돌려주는 게이트웨이를 붙였습니다.

	v0.68.0	v0.69.0
상태	READY	NEEDS_REVIEW
확정 화면	기본 선택됨	직접 선택해야 함
경고	0건	빠진 슬라이드 번호를 지목
사라진 업무	5장 분량, 흔적 없음	5, 6, 7, 9, 10번으로 명시

`validateAIResult`는 이미 꼼꼼합니다. 인용된 슬라이드가 실제 입력에 있는지, 진행률이 범위 안인지, 같은 업무가 중복되지 않는지 봅니다. 그런데 검사 대상이 전부 **나온 것**입니다. `pptxSourceSlides(input)`로 입력 슬라이드 전체 집합을 이미 계산해 두고도 반대 방향 — 아무도 인용하지 않은 슬라이드 — 은 세지 않았습니다.

빈 슬라이드는 세지 않습니다. 간지와 빈 표만 있는 슬라이드까지 지목하면 모든 Import가 경고를 달고, 그러면 아무도 경고를 읽지 않게 됩니다. 정규화 텍스트에서 `[SHAPE n]`·`[TABLE n]`·`[ROW n]`·`[COL n]`·`[EMPTY]` 같은 뼈대만 남은 슬라이드를 제외했습니다. 잘림 안내가 있는 슬라이드는 내용으로 셉니다 — 보낸 것보다 많았지 적지 않았습니다.

경고와 상태를 갈라 놓지 않았습니다. 처음엔 판정을 `validateAIResult` 안에 넣었는데, 그러면 경고는 뜨지만 파일은 READY로 남아 확정 화면에서 기본 선택됩니다. `finalizeImportedAIResult`의 다른 모든 경고가 `NEEDS_REVIEW`를 함께 세우는 것과 어긋납니다. 판정을 그쪽으로 옮겨 둘을 붙였습니다.

임계값은 지어내지 않았습니다. 실제 덱 말뭉치가 없으므로 "몇 퍼센트 이상 누락이면 검토"를 정할 근거가 없습니다(4.5의 원칙). 대신 이 코드베이스가 **이미 정해 둔 기준**을 씁니다 — 슬라이드 *일부*가 입력 한도로 잘린 것만으로도 `NEEDS_REVIEW`가 됩니다. 슬라이드 *전체*가 보고되지 않은 것은 그것의 무거운 쪽이므로 더 낮은 대우를 받을 수 없습니다. 대가는 "감사합니다" 장으로 끝나는 덱이 필요 없는 클릭 한 번을 요구한다는 것이고, 실제 덱이 모이면 그것이 다듬을 지점입니다.

원칙으로 남깁니다. **검증은 나온 것만 보면 절반입니다.** 200과 유효한 JSON과 높은 신뢰도를 동시에 갖춘 응답도 틀릴 수 있고, 그 틀림은 화면에 없는 것의 모양으로 나타납니다. 사람은 있는 것을 검토하지 없는 것을 검토하지 못합니다.

3.10.24 v0.70.0에서 완료 — 미매핑을 세고 있었지만 방향이 반대였습니다

v0.69에서 얻은 원칙 — 검증은 나온 것만 보면 절반이다 — 을 Confluence 동기화에 그대로 물었습니다. 답은 같았습니다.

가짜 Confluence를 붙여 12개 페이지를 내보냈습니다. 작성자는 `j.hkjang`, `k.lee`, `p.park` — Weekly 계정과 아이디가 다른 사람들입니다. 관리자 화면은 이렇게 나왔습니다.

	v0.69.0	v0.70.0
동기화 상태	SUCCESS	SUCCESS
조회 / 새 초안	12 / 0	12 / 0
실패	0건	0건
매핑 / 미매핑	1 / **0**	1 / 0
최근 진단	**0건**	아이디 3개를 지목
연결 안 된 Confluence 계정	**없는 칸**	3명
초안이 못 된 페이지	**없는 칸**	12개

미매핑 사용자 기능은 이미 있었습니다. 다만 세는 방향이 반대였습니다.

```
SELECT count(*) FROM users u WHERE u.active=true AND NOT EXISTS(
  SELECT 1 FROM user_external_accounts e WHERE e.user_id=u.id AND ... )
```

이것은 **우리 쪽 사람 중 연결 안 된 사람**을 셉니다. 그런데 업무가 사라지는 지점은 반대편입니다 — **스캔에 나타났는데 아무에게도 붙지 못한 Confluence 계정**. 두 숫자는 동시에 성립합니다. Weekly 사용자가 전원 연결돼 미매핑 0명인 상태에서, 12개 페이지가 전부 버려질 수 있습니다. 실제로 그렇게 나왔습니다.

`buildConfluenceActivities`는 `mappings[...]`가 돌려준 0을 보고 `creatorID > 0` 검사에서 조용히 넘어갑니다. 실패가 아니므로 `PagesFailed`도 오르지 않고, 만들어진 것이 없으므로 `CandidatesCreated`도 그대로입니다. 네 개 카운터 어디에도 이 사건이 들어갈 칸이 없었습니다.

진단은 이름을 말합니다. 개수만으로는 고칠 수 없습니다 — 관리자가 매핑 화면에 입력해야 하는 것은 아이디이기 때문입니다. 실제로 지목된 `j.hkjang`을 매핑하고 다시 돌리자 계정 3→2명, 페이지 12→8개, 초안 0→3건으로 움직였습니다.

기록 경로를 따로 만들었습니다. `recordConfluenceError`는 입력을 `safeConfluenceError`에 통과시킵니다(v0.68에서 원문 노출을 막으려고 그렇게 했습니다). 그 함수는 알아보지 못한 문자열을 "Confluence에 연결하지 못했습니다"로 바꿉니다. 즉 이 진단을 그 경로로 보내면 화면에서 사라집니다. `recordConfluenceNotice`를 따로 두고, 그 이유를 테스트로 고정했습니다 — 나중에 누가 둘을 합치려 할 때 왜 갈라져 있는지 읽을 수 있도록.

원칙으로 남깁니다. **"누락"을 세는 지표가 있다고 해서 누락이 보이는 것은 아닙니다.** 어느 쪽 집합을 세고 있는지가 전부입니다. 우리 쪽 명부를 세면 상대 쪽에서 들어온 것이 사라지는 것은 영원히 보이지 않습니다.

3.10.25 v0.71.0에서 완료 — 미제출자 253명 중 실제로 늦은 사람은 0명이었습니다

같은 질문을 세 번째로, 이번엔 주간보고 제출 현황에 물었습니다. 이 지표는 **방향이 맞았습니다** — `users`를 훑으며 보고서가 없는 주를 셉니다. 세는 집합이 옳습니다. 그런데 다른 곳이 틀렸습니다.

303명·15,600건이 든 규모 시험 데이터에서 실측한 "미제출 상위" 목록입니다.

```
부하 기준      누락 12주   마지막 제출 없음
신입 사원      누락 12주   마지막 제출 없음      ← 3일 전 입사
scaleadmin    누락 12주   마지막 제출 없음      ← 부트스트랩 계정
사용자 1       누락  1주   마지막 제출 2026-08-17
사용자 10      누락  1주   마지막 제출 2026-08-17
...
```

미제출자 총계 **253명**을 뜯어 보면 250명은 "이번 주(08-24) 미제출"입니다. 마감은 08-31입니다. **아무도 늦지 않았습니다.** 나머지 3명은 3일 전 입사자와 서비스 계정입니다. 관리자가 조치할 목록이 100% 소음이고, 하필 오탐이 정렬 상단을 차지합니다.

두 가지가 겹쳐 있었습니다. **마감 전인 주를 누락으로 셉니다** — 같은 함수의 추이 질의는 마감 규칙을 제대로 쓰는데 미제출 질의만 쓰지 않았습니다. **없던 사람에게 그 주를 묻습니다** — 존재하지 않던 주가 전부 누락으로 잡힙니다.

첫 수정이 틀렸고 실측이 잡았습니다. `u.created_at < 주차 시작일`로 막았더니 미제출자가 0명이 됐습니다. 시드 사용자가 전부 1~3일 전에 만들어졌기 때문인데, 이건 시드만의 문제가 아닙니다. **과거 보고서를 Import한 실제 배포에서 똑같이 일어납니다** — 전원의 `created_at`이 오픈 날짜이므로 모든 과거 주차가 그보다 앞서고, 지표가 영원히 0을 보고합니다. 이 제품에는 과거 PPTX를 적재하는 Import 파이프라인이 통째로 있으므로 가정이 아니라 기본 시나리오입니다.

고친 규칙은 **계정 생성일과 첫 제출 주차 중 이른 쪽**입니다. 이관된 사용자는 첫 제출 주차부터, 진짜 신입은 계정 생성일부터 답합니다. 어느 쪽도 지어낸 임계값이 아니라 "그때 있었다"는 증거입니다.

	v0.70.0	v0.71.0
미제출자 총계	253명	2명
그중 실제로 늦은 사람	0명	2명
목록 1위	3일 전 입사자	3주 누락자
진행 중인 주	완료된 주와 나란히 16%	"마감 전"으로 분리

두 질의를 한 문자열로 묶었습니다. 목록과 총계가 각자 조건을 갖고 있으면 한쪽에 나오고 다른 쪽에 없는 사람이 생깁니다. 그건 두 숫자 중 어느 하나가 틀린 것보다 나쁩니다.

덤으로 마감 문구의 애매함을 없앴습니다. "7일째 되는 날 자정까지"는 그날이 시작하는 자정과 끝나는 자정 두 가지로 읽히고 둘은 하루 차이입니다. 규칙은 그대로 두고 문장만 어느 쪽인지 말하게 했습니다.

원칙으로 남깁니다. **옳은 집합을 세는 것으로는 부족하고, 각자에게 언제부터 물을 수 있는지도 알아야 합니다.** 그리고 그 "언제부터"를 계정 생성일로 잡으면, 과거를 이관한 배포에서는 지표 전체가 조용히 0이 됩니다.

3.10.26 v0.72.0에서 완료 — 최신 2000행 안에서 매긴 순위를 전체 순위라고 불렀습니다

각도를 바꿔, 규모에서 답이 달라지는 곳을 찾았습니다. 검색이 그랬습니다.

15,594개 보고서·109,175개 항목이 든 데이터에 **제목이 정확히 질의어인 항목 1건**을 1년 전 주차에 심고, 같은 문구를 최근 주차 본문 2,600건에 넣었습니다.

```
v0.71.0  결과 30건  제목 일치 포함 0건
  2026-08-24  점수 20  [currentResult]
  2026-08-24  점수 20  [currentResult]   ← 점수 50짜리는 어디에도 없음
```

원인은 스캔이 `ORDER BY week_start DESC LIMIT 2000`이라는 것입니다. 순위는 그 2000행 **안에서만** 매겨지는데, 화면은 그것을 전체에 대한 순위로 보여줍니다. `truncated: true`가 뭔가 잘렸다고는 말하지만, 남은 것이 최선이라는 주장은 그대로입니다.

첫 시도는 옳았지만 비쌌습니다. 정렬을 SQL로 옮겨 `ORDER BY <점수> DESC`로 바꾸니 제목 일치가 1위로 올라왔습니다. 대신 흔한 단어에서 0.04s → 0.29s, **7배**가 됐습니다. 109,175행 전부에 ILIKE 6개를 평가해야 거의 전부가 같은 점수라는 것을 알게 되기 때문입니다.

대신 값싼 우선 패스를 앞에 뒀습니다. 문단보다 높은 점수를 받는 필드 — 제목과 구분 — 만 따로 묻습니다. 매칭되는 행이 적으므로 싸고, 끝나면 높은 점수는 이미 손에 있습니다.

질의	v0.71.0	v0.72.0
진행(109,175행 매칭)	0.031s	0.085s
구축	0.054s	0.128s
프로젝트	0.184s	0.189s
제목 일치 노출	**안 됨**	**1위(점수 50)**

우선 패스 예산을 줄여도 빨라지지 않습니다. 500 → 150으로 낮춰 재 봤는데 그대로였습니다. 날짜순 정렬이므로 최신 것을 고르려면 **이력 전체의 제목 일치를 먼저 다 찾아야** 하고, 그 비용은 상한과 무관합니다. 즉 늘어난 ~60ms는 낭비가 아니라 "전부를 본다"의 값이고, 옛 코드가 빠르면서 틀렸던 이유가 정확히 그것을 안 봤기 때문입니다. 주석에 그 사실을 적어 뒀습니다 — 다음 사람이 상한을 낮춰 최적화하려다 헛수고하지 않도록.

인덱스가 하나 빠져 있었습니다. 다른 검색 대상 컬럼은 모두 트라이그램 인덱스가 있는데 `category`만 없었습니다. 4글자 질의에서 47ms → 0.5ms, **94배**입니다(마이그레이션 020). 2글자 한글 질의는 여전히 느리고, 이건 트라이그램이 3글자 미만에서 선택도를 못 갖는 구조적 한계라 인덱스를 더 넣어도 해결되지 않습니다 — 확인하고 적어 둡니다.

가드가 처음엔 물지 못했습니다. 테스트가 핸들러가 아니라 테스트 자신이 쓴 SQL을 검사하고 있어서, 우선 패스를 통째로 지워도 통과했습니다. 스캔을 `searchScan`으로 뽑아내 테스트가 실제 코드를 부르게 고쳤고, 이제 패스를 지우거나 대상 필드를 바꾸면 실패합니다. v0.64·v0.68에 이어 세 번째로 같은 실수를 했으니, 가드를 쓴 뒤 **되돌려 보는 것**은 선택이 아니라 절차입니다.

원칙으로 남깁니다. **부분집합 위에서 매긴 순위를 전체 순위로 제시하지 않습니다.** 잘렸다고 표시하는 것과, 남은 것이 최선이라고 말하는 것은 다릅니다.

3.10.27 v0.73.0에서 완료 — 짓지 못하는 감시견을 기억이 아니라 절차로 잡습니다

세 릴리즈 연속으로 같은 실수를 했습니다. 특정 결함을 잡으라고 쓴 테스트가 통과했는데, **애초에 실패할 수 없는 것**이었습니다.

- v0.64 — 도달성 가드의 일치 조건이 느슨해 CSS 클래스 이름이 조건을 만족시켰습니다. 유일한 호출부를 지워도 통과했습니다.

- v0.68 — 메시지 테스트의 모든 사례가 앞쪽 분기에 걸려, 정작 그 테스트가 존재하는 이유인 폴백이 한 번도 실행되지 않았습니다.
- v0.72 — 순위 테스트가 제품 함수 대신 테스트 자신이 쓴 SQL을 검사했습니다. 그 함수를 통째로 지워도 통과했습니다.

뒤의 둘은 기계적으로 같은 신호를 냅니다. **테스트가 지키겠다는 코드를 실행하지 않았다.** 그건 퍼-테스트 커버리지가 재는 것이고, 따라서 기억해서 확인할 일이 아닙니다.

`scripts/guard-check.py`를 만들었습니다. 가드 테스트가 자기 대상을 선언합니다.

```
// guards: searchScan, appendSearchMatches
func TestATitleMatchOutranksAFloodOfRecentBodies(t *testing.T) {
```

각 테스트를 혼자 커버리지와 함께 돌려, 선언한 함수에 닿지 않으면 보고하고 종료 코드 1을 냅니다. 검사기 자신도 되돌려 확인했습니다 — v0.72의 실수를 재현하니 `go test`는 `ok`인데 검사기가 잡습니다.

함수에 닿는 것과 정작 그 분기에 닿는 것은 다릅니다. v0.68의 경우가 그랬습니다. 그래서 임계값을 선언할 수 있게 했습니다.

```
// guards: safeConfluenceError=100
```

"원인마다 다른 문장"이 주장의 전부인 가드에는 이걸 씁니다. 안 닿은 분기가 곧 아무도 확인하지 않은 사례이기 때문입니다.

27개 가드에 마커를 붙였고, 여섯 개의 진짜 구멍이 나왔습니다.

발견	내용
`TestOnlyGenuineSignOutIsMarkedAsNoSession`	`authenticate`를 **한 번도 부르지 않음**. `errors.Is`의 동작만 확인. `authenticate`가 래핑을 그만둬도 통과
`aiUserMessage`	예상 못 한 상태 코드의 폴백 문구 미검증
`safeConfluenceError`	같은 폴백 + **Confluence 타임아웃 문구가 한 번도 읽힌 적 없음**
`outlookForDueDate`	읽을 수 없는 주차, 최근이 더 느린 경우의 문장, 범위로 모자란 경우의 문장 — 3개 분기

발견	내용
`outlookForPeriodEnd`	기간 미설정, 읽을 수 없는 주차, 범위로 모자란 경우 — 3개 분기
CI	**PostgreSQL이 없어 통합 테스트가 전부 건너뛰어지고 있었음**

첫 번째가 가장 무겁습니다. v0.66에서 "DB 장애를 로그아웃으로 보고하던" 버그를 고치며 쓴 가드인데, 그 버그를 다시 넣어도 잡지 못했습니다. `authenticate`를 실제로 부르도록 고치니 이제 잡습니다.

마지막도 큽니다. CI에 데이터베이스가 없어 마이그레이션·제출 현황·검색 순위 검증이 전부 **누군가의 노트북에서만** 돌고 있었습니다. `pgvector/pgvector:pg16` 서비스를 붙였습니다.

이 검사기가 하지 않는 일도 적어 둡니다. 단언이 강한지는 보지 않습니다. 되돌려서 실패하는지 확인하는 것은 여전히 진짜 검사이고, 이건 *되돌릴 것이 애초에 없던* 경우를 잡습니다.

원칙으로 남깁니다. **세 번 반복한 실수는 규율의 문제가 아니라 절차의 부재입니다.** 사람에게 기억하라고 요구하는 대신, 기계가 물을 수 있는 형태로 바꿉니다.

3.10.28 v0.74.0에서 완료 — 아무도 닿지 않는 곳을 물었더니, 가장 위험한 두 함수가 나왔습니다

v0.73이 "이 가드가 물 수 있는가"를 기계화했으니, 이번엔 반대 질문을 던졌습니다. **아무 테스트도 닿지 않는 곳은 어디인가.**

전체 스위트를 커버리지와 함께 돌리니 41.5%, 0%인 함수가 251개입니다. 그중 196개는 `*App` 메서드 — HTTP 핸들러이고, 이 코드베이스에 HTTP 수준 시험 장치가 없다는 알려진 사실입니다. 남은 55개 순수 함수를 읽었고, 둘이 눈에 띄었습니다.

함수	하는 일	커버리지
`workScope.where`	**누가 누구의 업무를 볼 수 있는지** 정하는 술어	0%
`safeImportPath`	**경로 탈출**을 막는 검사	0%

둘 다 틀리면 조용히 위험하고, 둘 다 아무도 확인한 적이 없었습니다.

둘 다 맞았습니다. 테스트를 써서 확인했고, 그 자체가 결과입니다. `where`는 역할마다 올바르게 좁히고 비어 있지 않으며, `SelfOnly`가 관리자까지 이깁니다. `safeImportPath`는 `..` 탈출, 다른 작업 디렉터리, **번호가 겹치는 작업(5 대 51)**, 확장자 덧붙이기를 모두 거부합니다.

대신 다른 것이 나왔습니다. 술어는 `w`와 `u`라는 SQL 별칭에 의존하고, 네 호출부 중 세 곳에 이런 주석이 있습니다.

> 별칭을 바꾸면 이 코드는 **컴파일되고 필터만 조용히 사라집니다.**

같은 위험을 세 번 적어 뒀다는 것은 그것이 진짜라는 뜻입니다. 그런데 지키는 것은 사람의 주의력뿐이었습니다. 이제 모든 `scope.where()` 호출부의 질의문이 `w`와 `u`를 실제로 묶는지 기계가 확인합니다. 별칭을 바꾸거나, 호출부가 사라지거나, 가드가 볼 대상이 없어지는 세 경우 모두 실패하는 것을 확인했습니다.

Go가 이미 막아 주는 쪽도 적어 뒀습니다 — 술어를 질의에 붙이지 않으면 `where`가 미사용 변수가 되어 빌드가 깨집니다. 별칭만 미끄러질 수 있습니다.

검사기가 제가 방금 쓴 테스트를 잡았습니다. `scopeForPrincipal`을 지킨다고 선언해 놓고 부르지 않았습니다. v0.73에서 만든 절차가 만든 당일에 저를 잡은 셈이고, 이것이 절차가 기억보다 나은 이유입니다.

원칙으로 남깁니다. **주석으로 세 번 적은 위험은 가드로 옮길 때가 된 것입니다.**

3.10.29 v0.75.0에서 완료 — 핸들러 196개가 통째로 미검증이던 것

v0.74에서 커버리지 0%인 함수 251개 중 **196개가 `*App` 메서드 — HTTP 핸들러**라는 것을 확인하고 "이 코드베이스에 HTTP 수준 시험 장치가 없다는 알려진 사실"이라고 적었습니다. 알려진 사실은 고쳐도 되는 사실입니다.

이 제품이 반환하는 모든 권한 규칙, 오류 코드, 상태 코드가 그 안에 있습니다. 지금까지 그것을 확인한 방법은 **컨테이너를 띄우고 손으로 두드리는 것**이었고, 그건 누군가 기억할 때만, 그리고 그 순간 떠올린 경로에 대해서만 일어납니다.

바이너리가 쓰는 것과 같은 mux에 요청을 보내는 장치를 만들었습니다. 실제 `New`로 앱을 세우므로 마이그레이션·부트스트랩·미들웨어가 전부 진짜입니다.

두 번 돌리자 깨졌고, 그게 설계를 결정했습니다. 처음에는 계정 이름을 고정했다가 `USERNAME_EXISTS`를 만났습니다. 이름을 실행마다 유일하게 만들었더니 이번엔 전부 `INVALID_CREDENTIALS`였습니다 — **부트스트랩 관리자는 관리자가 하나도 없을 때만 생성되므로**, 이전 실행이 남긴 행 하나가 이후 모든 실행을 막습니다. 즉 스위트가 자기 정리에 의존하고 있었습니다.

정리를 더 잘하는 대신 의존을 없앴습니다. **테스트마다 자기 데이터베이스를 만들고 끝나면 지웁니다.** 두 번 연속 돌려 통과하고 임시 DB가 남지 않는 것을 확인했습니다.

첫 네 개의 테스트로 확인한 것 — 전부 읽었을 때는 맞았고, 그것을 강제하는 경로로는 한 번도 지나간 적이 없던 규칙들입니다.

- 남의 보고서는 읽을 수도, 고칠 수도, 지울 수도 없습니다
- `requireRole`은 계층이 아니라 허용 목록이므로 USER는 작은 ADMIN이 아닙니다
- 쿠키 없는 요청은 403이 아니라 401이고, 오류 코드를 함께 냅니다
- 다른 출처에서 온 쓰기는 유효한 세션을 갖고 있어도 `CSRF_REJECTED`

	v0.74.0	v0.75.0
구문 커버리지	41.5%	45.6%
커버리지 0%인 함수	251개	214개
가드	30개	34개

네 개의 테스트가 37개 함수를 새로 덮었습니다 — 인증, 로그인, 로그인 제한, CSRF 미들웨어 전체가 여기 들어 있습니다. 값은 이 네 테스트가 아니라 **장치**에 있습니다. 지금까지 이 반복에서 찾은 결함 상당수가 이것이 먼저 있었다면 애초에 잡혔을 것입니다.

원칙으로 남깁니다. **"그건 구조적으로 못 한다"는 말은 대개 "아직 안 했다"입니다.** 두 번은 다르게 취급해야 합니다.

3.10.30 v0.76.0에서 완료 — 검토 워크플로 전체가 한 번도 지나간 적 없었습니다

v0.75의 장치를 처음으로 제대로 써 봤습니다. 대상은 **검토 워크플로** — 제출·승인·반려·재수정입니다. 이 제품이 "주간보고 플랫폼"인 이유의 절반이 여기 있고, 커버리지는 0%였습니다.

전부 옳았습니다. 다섯 가지 규칙을 핸들러로 확인했고 모두 맞습니다.

규칙	결과
자기 보고서는 자기가 승인 못 함 — **관리자도**	403
USER는 누구 것도 검토 못 함	403
팀장은 자기 조직만, 다른 조직은 못 함	200 / 403
검토는 `SUBMITTED`에서만, 두 번은 안 됨	409 `INVALID_STATUS`
반려에는 사유가 필요하고, 없으면 상태도 안 움직임	400

가장 확인하고 싶었던 것은 **승인된 보고서를 작성자가 고칠 수 있는가**였습니다. 고칠 수 있고, 고치면 `DRAFT`로 되돌아가며 `reviewed_by`가 지워지고 이력에 *"작성자가 제출·승인 후 내용을 수정하여 재검토 상태로 전환"*이 남습니다. 승인이 내용보다 오래 살지 않습니다.

읽어서 맞는 것과 강제 경로로 확인한 것은 다르므로, 세 가지를 되돌려 전부 실패하는 것을 봤습니다 — 본인 검토 차단 제거, `AND status='SUBMITTED'` 제거, 승인 뒤 수정 시 상태 유지. 마지막 변이에서는 이력 행도 함께 사라져 두 단언이 동시에 터집니다.

검사기가 또 저를 잡았습니다. `canReviewReport=100`을 걸었더니 50%였습니다. 빠진 절반이 **팀장이 자기 조직원을 검토하는 정상 경로** — 이 제품에서 가장 흔한 검토 — 였습니다. 조직을 만들어 채웠습니다.

나머지 절반은 `p.Role == "USER"` 분기인데, 승인 라우트가 이미 `requireRole`로 USER를 막으므로 **요청으로는 닿을 수 없는 방어선**입니다. 100%를 요구하면 그 함수를 직접 부르는 테스트를 쓰게 되고, 그건 라우트가 아니라 함수를 시험하는 것입니다. 임계값을 내리고 그 이유를 주석으로 적었습니다 — 다음 사람이 "왜 여기만 100%가 아니지"를 다시 묻지 않도록.

	v0.75.0	v0.76.0
구문 커버리지	45.6%	46.7%
커버리지 0%인 함수	214개	207개
가드	34개	39개

원칙으로 남깁니다. **모든 규칙에 100%를 요구하면 규칙이 아니라 함수를 시험하게 됩니다.** 요구를 낮출 때는 낮춘 이유를 남깁니다.

3.10.31 v0.77.0에서 완료 — 이름을 언급하기만 하면 통과하던 가드

Import 확정 경로를 보러 갔다가 다른 것을 찾았습니다.

확정은 `WHERE user_id=$1`로 **자기 보고서만** 건드립니다. 남의 주간보고를 교체하거나 병합할 수 없습니다. 여기까지는 안전한데, 눈에 걸린 것이 하나 있었습니다 — REPLACE가 승인된 보고서의 내용을 통째로 바꾸면서 `reviewed_by`는 지우지 않습니다. v0.76에서 확인한 "승인이 내용보다 오래 살지 않는다"와 어긋납니다.

그래서 `reviewed_by`가 화면에 나오는지 봤습니다. **어디에도 안 나옵니다.** 그런데 이 프로젝트에는 v0.65부터 *"응답에 실려 나가지만 화면이 읽지 않는 필드"*를 잡는 가드가 있습니다. 왜 못 잡았는지 보니, 가드가 이렇게 되어 있었습니다.

```
if !strings.Contains(frontend, field) { … }
```

프런트엔드 어딘가에 그 이름이 들어 있으면 통과합니다. `types.ts`의 타입 선언만으로 충분합니다. v0.64에서 CSS 클래스 이름이 도달성 조건을 만족시킨 것과 같은 모양입니다.

세어 봤습니다. 응답 필드 335개 중 **26개가 `types.ts`에만 있고 화면 코드 어디에서도 쓰이지 않습니다.**

그런데 화면이 안 쓰는 것이 곧 죽은 것은 아닙니다. 이 제품은 개인 API 키와 MCP 서버로도 읽힙니다. 26개 중 7개는 `docs/openapi.yaml` 산문에 설명이 있었습니다 — 화면 대신 연동하는 쪽이 소비합니다. 즉 옳은 기준은 "화면이 읽는가"가 아니라 **"화면이 그리거나 문서가 적는가"**입니다.

그 기준으로 다시 세니 **19개가 어디에도 없습니다.** 계산해서 網선으로 내보내는데, 그리지도 적지도 않습니다. 연동하는 사람은 존재 자체를 알 방법이 없습니다.

무리	필드
수명 주기 시각	`submittedAt` `reviewedAt` `startedAt` `completedAt` `analyzedAt` `confirmedAt` `builtAt` `generatedAt`
기간 보고 인사이트	`continuingItems` `oneOffItems` `noLandingItems` `sourceReports` `sourceItems` `dedupRate`
그 밖	`periodEnd` `overlapWeeks` `latestIssue` `scannedCharacters` `roles`

쓰레기가 아니라 **문서화되지 않은 API 표면**입니다. 전부 `docs/openapi.yaml`에 뜻과 함께 적었고, 가드는 `types.ts`를 빼고 보되 문서를 함께 보도록 고쳤습니다.

되돌려서 확인했습니다. 문서에서 인사이트 여섯 줄을 지우면 새 가드가 실패합니다. 그리고 **옛 가드로 되돌리면 문서를 통째로 지워도 통과합니다** — 19개를 애초에 잡을 수 없었다는 증거입니다.

원칙으로 남깁니다. **"어딘가에 그 이름이 있다"는 사용의 증거가 아닙니다.** 그리고 소비자가 화면 하나뿐이라고 가정하면, API를 가진 제품에서는 살아 있는 것을 죽었다고 부르게 됩니다.

3.10.32 v0.78.0에서 완료 — 바뀐 본문에 남아 있던 승인 도장

v0.77에서 `reviewedAt`을 문서에 적으면서 이렇게 썼습니다.

> 검토자가 승인 또는 반려한 시각. **작성자가 내용을 고치면 다시 `null`이 됩니다.**

문서에 적고 나니 확인해야 할 것이 생겼습니다. PPTX Import로 내용을 바꾸는 것도 작성자가 내용을 고치는 것입니다. 그때도 그런가?

아니었습니다.

```
승인된 보고서를 PPTX로 통째로 교체한 뒤
  summary      = "PPTX에서 가져온 내용"   ← 본문은 완전히 바뀜
  reviewedAt   = 2026-08-24 17:02:30      ← 그대로
  reviewed_by  = 1                        ← 그대로
```

이전 문장을 승인한 사람이, 자기가 본 적 없는 문장의 검토자로 남습니다. 편집기로 고치면(`updateReport`) 둘 다 지워집니다. Import 경로만 빠져 있었습니다. 병합도 같습니다 — 아무도 검토하지 않은 문단을 승인된 보고서에 더하면서 도장은 유지했습니다.

두 곳 다 `reviewed_at=NULL, reviewed_by=NULL`을 넣었고, 각각 독립적으로 되돌려 두 테스트가 따로 실패하는 것을 확인했습니다.

같은 종류가 더 있는지 전수로 봤습니다. `UPDATE weekly_reports SET`은 여섯 곳입니다.

위치	하는 일	검토 도장
`reports.go` 제출	DRAFT → SUBMITTED	지움 ✓
`reports.go` 검토	SUBMITTED → APPROVED/반려	찍음 ✓
`reports.go` 수정	본문 변경	지움 ✓
`imports.go` REPLACE	본문 교체	**안 지움 → 고침**
`imports.go` MERGE	본문 추가	**안 지움 → 고침**
`confluence_handlers.go`	자동 초안 수락	`DRAFT` / `REVISION_REQUESTED`에서만 동작하고 본문은 편집기 저장으로 들어옴 — 해당 없음

문서를 적은 것이 결함을 찾아냈습니다. v0.77에서 19개 필드에 뜻을 붙인 것은 문서화 작업이었는데, 그중 한 줄이 검증 가능한 약속이 되면서 코드가 그 약속을 어기고 있다는 것이 드러났습니다.

원칙으로 남깁니다. **행동을 문장으로 적으면 그 문장이 시험이 됩니다.** 적을 수 없다면 아직 정하지 않은 것이고, 적었는데 다르다면 둘 중 하나가 틀린 것입니다.

3.10.33 v0.79.0에서 완료 — 한 문장, 두 가지 구현

v0.78이 통했던 이유 — 문서에 적은 행동이 시험이 된다 — 를 넓혀 봤습니다. `docs/openapi.yaml`에서 **행동으로 서술된 약속**을 추려 실제로 그런지 확인했습니다.

가장 여러 엔드포인트에 걸친 약속은 상한 규칙입니다.

> limit 기본값 200, 최대 500. **범위를 벗어나면 잘라 맞추고, 읽을 수 없는 값은 기본값으로 본다.**

네 개 목록 엔드포인트에 여섯 가지 입력(생략·상한 초과·0·음수·비숫자·상한 자체)을 넣어 봤습니다.

저부터 틀렸습니다. `limit=0`이 기본값을 줄 것이라 기대했는데 1을 줍니다. 다시 읽으니 계약은 "잘라 맞추고"이므로 1이 맞습니다 — 제 기대가 틀렸습니다. 그런데 그 실수 자체가 신호였습니다. 방금 그 문장을 다시 읽은 사람도 헷갈렸다면, 연동하는 사람은 더 헷갈립니다. 문장을 고쳤습니다.

> 범위를 벗어나면 가까운 끝으로 잘라 맞춘다 — 상한보다 크면 상한, **1보다 작으면 1이다(0이나 음수도 기본값이 아니라 1이다)**.

그리고 진짜가 하나 나왔습니다. `/admin/audit`만 다르게 답합니다.

	`limit=0`	`limit=-5`
`clampQueryInt` — 5개 엔드포인트	1	1
`auditLogs` — 직접 구현	50	50

같은 규칙을 문서에 한 번 적어 놓고 구현이 둘이었습니다. `/reports`에서 규칙을 배운 클라이언트가 `/admin/audit`에서 다른 답을 받습니다. `offset`도 마찬가지로 혼자 다른 형태였습니다. 둘 다 공용 헬퍼로 모았습니다.

되돌려 확인했습니다 — audit을 직접 구현으로 되돌리면 실패하고, 상한을 무시하게 하면 실패하고, 읽을 수 없는 값의 기본값 처리를 없애면 실패합니다. 음수 `offset`이 500이 되지 않는지도 함께 고정했습니다(PostgreSQL은 음수 OFFSET에 오류를 냅니다).

원칙으로 남깁니다. **규칙을 문장으로 한 번 적었다면 구현도 한 곳이어야 합니다.** 문장은 하나인데 구현이 둘이면, 둘 중 하나는 문서를 읽은 사람을 배신합니다.

3.10.34 v0.80.0에서 완료 — 문서의 숫자와 코드의 숫자를 묶었습니다

v0.79에서 상한 규칙 하나를 확인했으니, 이번엔 `docs/openapi.yaml`이 **숫자로 못 박은 약속 전부**를 훑었습니다. 열두 개가 나왔습니다 — 목록 상한, 검색 단어 수, 병목 20건, 관련 업무 5건, 다이제스트 10건, 유사·중복 상위 200건, 결정 50건, 검색 단계 전환 5건.

전부 맞았습니다. 열두 개 모두 코드의 상수와 일치합니다.

그러면 이 반복의 값은 무엇인가. 맞다는 것을 확인한 것 자체도 결과지만, 더 중요한 것은 **이 일치가 유지된다는 보장이 하나도 없었다**는 사실입니다. 누군가 병목 상한을 25로 올리면 문서는 20이라고 계속 말합니다. 오프라인망에서 연동하는 사람에게 이 파일은 유일한 근거이고, **자신 있게 틀린 문서는 아무 말도 안 하는 것보다 나쁩니다.**

그래서 기댓값을 상수에서 만들어 냅니다.

```
{"병목", fmt.Sprintf("최대 %d건 담는다", bottleneckLimit)},
```

상수를 바꾸면 찾는 문장이 바뀌고, 문서가 따라오지 않으면 실패합니다. 세 개를 실제로 바꿔 전부 실패하는 것을 확인했습니다.

매직 넘버 셋을 상수로 뽑았습니다. 문서가 말하는데 코드에는 이름이 없던 숫자들입니다 — Import 상한 200, 검색 단계 전환 기준 5. 이름 없는 숫자는 이 검사가 볼 수 없고, 볼 수 없으면 지킬 수도 없습니다.

원칙으로 남깁니다. **문서가 말하는 숫자에는 코드에 이름이 있어야 합니다.** 그래야 둘이 어긋나는 순간을 기계가 압니다.

3.10.35 v0.81.0에서 완료 — 아무것도 켜지 않은 상태, 그리고 물지 못한 제 테스트

이 제품이 실제로 설치되는 모습은 **모든 선택 기능이 꺼진 상태**입니다. AI 게이트웨이 없음, Confluence 없음, 검토 워크플로 없음. 문서는 그 상태에서 무엇을 답하기로 했는지 적어 두었지만, 그 약속을 확인한 적은 없었습니다.

전부 옳았습니다.

상황	답
AI 꺼짐 · 본문 분석 / 결정 제안 / PPTX 업로드	503 `AI_UNAVAILABLE`
워크플로 꺼짐 · 승인	409 `WORKFLOW_DISABLED`
Confluence 꺼짐 · 강제 동기화	409 `CONFLUENCE_DISABLED`

상황	답
아무것도 안 켠 채 읽기 7종	전부 200

500이 아니라 503·409라는 것이 요점입니다. 500은 "이 제품이 고장 났다"로 읽히고, 빈 200은 "찾을 게 없었다"로 읽힙니다. 둘 다 **누군가 바꿀 수 있는 설정**을 감춥니다.

확장이 하나도 없는 PostgreSQL도 흉내 냈습니다. `word_similarity`와 `<=>`를 쓰는 세 함수가 모두 자기 능력 플래그 뒤에 있는지 읽고, 실제로 꺼서 확인했습니다. (읽는 중에 `semanticWorkItems`가 pg_trgm 함수를 pgvector 플래그로 막는 것처럼 보였는데, `<=>`여서 맞았습니다.)

그리고 이번 반복의 진짜 수확은 여기서 나왔습니다. 제가 방금 쓴 테스트 세 개가 물지 못했습니다.

- 확장 테스트는 `존재하지않는말`로 검색했습니다. 아무것도 매치되지 않으니 뒤 단계도 빈손이고, 게이트를 통째로 제거해도 `fuzzy`는 `false`로 남습니다.
- AI 테스트는 게이트웨이를 **아예 설정하지 않은** 상태로 돌렸습니다. 활성 여부 검사를 느슨하게 해도 주소가 없어 어차피 실패하니 티가 나지 않습니다.

`guard-check.py`는 이걸 못 잡습니다 — 두 테스트 모두 대상 함수를 **실행하긴 했기** 때문입니다. v0.73에서 "단언이 강한지는 보지 않는다"고 적어 둔 한계가 그대로 나타났고, 되돌려 보는 절차가 여전히 진짜 검사인 이유입니다.

고쳤습니다. 검색은 **매치되는 오타**(`확장업는`)를 쓰고, 확장이 켜진 상태에서 `fuzzy`가 `true`인 것을 먼저 확인한 뒤 끕니다. AI는 **설정은 되어 있고 꺼져만 있는** 상태로 만듭니다. 이제 둘 다 되돌리면 실패합니다.

원칙으로 남깁니다. **없는 것을 확인하는 테스트는 있는 것으로 먼저 확인해야 합니다.** 꺼진 상태만 보면, 애초에 켜도 안 되는 것과 구분되지 않습니다.

3.10.36 v0.82.0에서 완료 — 틀린 비밀번호가 통과해도 아무도 몰랐습니다

v0.73이 *"테스트가 대상을 실행했는가"*를 기계화했고, v0.81이 그것으로 부족하다는 것을 보여 줬습니다. 실행하고도 물지 못하는 테스트는 커버리지로 구분되지 않습니다. **코드를 실제로 망가뜨려 보는 것**으로만 구분됩니다.

`scripts/mutation-check.py`를 만들었습니다. `// guards: F` 마커가 이미 짝을 알고 있으니, F 안에서만 비교 연산자를 뒤집고 `&&` ↔ `||`를 바꾸고 반환 상수를 돌린 뒤 그 테스트를 돌립니다.

첫 실행에서 도구 자체의 결함이 나왔습니다. 살아남은 변이를 그대로 보고했더니, `canReviewReport`의 어떤 변이는 자기 가드는 놓치지만 **다른 테스트가 잡습니다.** 둘을 섞으면 멀쩡한 곳을 구멍이라 부르게 됩니다. 살아남을 때마다 전체 스위트를 한 번 더 돌려 둘로 나누게 했습니다.

	건수
적용한 변이	204 (컴파일 불가 11건 제외)
지목된 가드가 잡음	97
다른 테스트가 잡음 — 마커가 엉뚱한 곳을 가리킴	43
아무도 못 잡음	**64**

64건 중 둘이 즉시 눈에 띄었고, 둘 다 인증입니다.

`login`의 `||`를 `&&`로 바꾸면 틀린 비밀번호로 로그인됩니다. 전체 스위트가 통과합니다. 이 제품에서 가장 중요한 단언 — *틀린 비밀번호는 통과하지 못한다* — 이 한 번도 검증된 적이 없었습니다. 기존 로그인 테스트는 전부 **올바른** 비밀번호로 들어가거나 저장소 장애를 다뤘습니다.

API 키 SQL의 `expires_at>now()`를 `<`로 뒤집으면 만료된 키가 통과하고 살아 있는 키가 거부됩니다. 개인 API 키 경로에 테스트가 하나도 없었습니다 — 이 제품이 MCP 서버와 연동을 노출하는 바로 그 경로입니다.

둘 다 테스트를 썼고, v0.81의 교훈대로 **긍정 대조를 먼저** 둡니다: 올바른 비밀번호가 통과하는 것, 갓 발급한 키가 동작하는 것을 확인한 뒤에 거부를 확인합니다. 덤으로 폐기된 키와 발급된 적 없는 토큰도 막습니다. 없는 계정과 틀린 비밀번호가 **같은 오류 코드**인 것도 고정했습니다 — 다르면 그 차이가 어떤 아이디가 실재하는지 알려 줍니다.

세 변이를 되돌려 전부 실패하는 것을 확인했습니다.

남은 62건은 목록으로 둡니다. 상당수는 의미상 무해할 것이고, 판별에는 사람의 판단이 필요합니다. `python3 -u scripts/mutation-check.py`가 언제든 다시 만들어 냅니다. 도구는 CI 단계가 아닙니다 — 한 번 도는 데 40분 넘게 걸리고, `scale-check.py`처럼 필요할 때 돌리는 물건입니다.

원칙으로 남깁니다. **테스트가 통과한다는 것은 코드가 옳다는 뜻이 아니라, 이 테스트가 이 코드를 틀리게 만들 방법을 모른다는 뜻입니다.**

3.10.37 v0.83.0에서 완료 — 남은 62건을 훑었더니, 제 주석이 틀렸습니다

v0.82의 변이 검사가 남긴 "아무도 못 잡는 변이 62건"을 훑었습니다. 함수별로 묶으니 `uploadImportPPTX` 4건, `deleteReport` 4건, `csrf` 1건이 눈에 띄었습니다.

업로드 상한 넷이 전부 뒤집혀도 아무도 몰랐습니다. 파일 개수, 파일 크기, 빈 파일, 요청 전체 크기 — 로그인한 누구나 파일을 보낼 수 있는 엔드포인트의 방어선입니다. 개수 3개로 제한해 놓고 4개를 보내면 400이

고, 1MB 제한에 1MB+1을 보내면 그 파일만 실패로 기록됩니다. **거부하면서 기록을 남기는지**까지 확인했습니다 — 기록 없는 거부는 올린 사람이 알 방법이 없습니다.

삭제의 버전 검사도 전부 미검증이었습니다. 두 사람이 같은 이름으로 서로 다른 것을 지우지 못하게 하는 장치인데, 버전 없음·0·음수·비숫자·낡은 버전 다섯 경우 모두 통과했습니다. 다섯 번의 거부 뒤에도 보고서가 그대로 있는 것, 그리고 **맞는 버전으로는 실제로 지워지는 것**(긍정 대조)까지 확인했습니다.

세 번째에서 제가 틀렸습니다. `csrf`의 `p.AuthType == "session"` 조건을 넓히는 변이가 살아남았고, v0.75에 제가 이렇게 적어 뒀습니다.

> API 키는 브라우저가 아니고 출처를 갖지 않으므로 이 검사가 적용되면 안 된다 — **그러지 않으면 모든 연동이 깨진다.**

확인해 보니 **개인 API 키는 조회 전용**입니다. 쓰기가 아예 없으니 깨질 연동도 없습니다. 그 조건은 `canReviewReport`의 USER 분기와 같은 **닿을 수 없는 방어선**이고, 제 주석은 과장이었습니다. 더 나쁜 것은 그 주석이 *검사한다고 약속하면서 검사하지 않았다*는 점입니다 — 읽는 사람은 그 규칙이 지켜지고 있다고 믿게 됩니다.

주석을 사실로 고치고, **실제로 성립하는 규칙**을 대신 고정했습니다: 조회 전용 키는 어떤 쓰기도 하지 못하며, 거부 코드가 `API_KEY_SCOPE_DENIED`여서 연동하는 쪽이 이유를 압니다. 그 검사를 제거하는 변이를 넣으니 **조회 전용 키가 보고서를 만들었습니다.**

원칙으로 남깁니다. **주석이 약속한 검사는 코드가 하고 있어야 합니다.** 하지 않는다면 그 주석은 설명이 아니라 잘못된 보증입니다.

3.10.38 v0.84.0에서 완료 — 정시와 지각이 같은 칸이어도 아무도 몰랐습니다

남은 변이를 계속 훑었습니다. 이번에 걸린 것은 v0.71이 절반만 끝낸 일입니다.

v0.71에서 마감 시각을 Go와 SQL이 똑같이 계산하는지는 고정했습니다. 그런데 **그 시각이 무엇을 위한 것인지** — 마감 전에 낸 보고서가 정시로, 뒤에 낸 것이 지각으로 세어지는지 — 는 확인한 적이 없습니다.

```
count(*) FILTER (WHERE r.submitted_at <  deadline.at)   -- 정시
count(*) FILTER (WHERE r.submitted_at >= deadline.at)   -- 지각
```

한 줄만 뒤집어도 전체 스위트가 통과합니다. 뒤집힌 상태의 출력이 그 무의미함을 보여 줍니다 — 보고서 **하나**가 `onTime=1 late=1`로 **양쪽에 동시에** 세어집니다. 관리자 화면의 "기한 내 제출"은 그 두 숫자로 만들어집니다.

마감 한 시간 전·한 시간 후·정확히 마감 시각 세 경우를 고정했습니다. 경계에서는 어느 쪽이 가져가든 **합이 1**이어야 한다는 것만 요구합니다 — 규칙이 어느 쪽인지보다, 한 보고서가 두 번 세어지지 않는 것이 먼저입니다.

테스트를 쓰다 제가 그 버그를 그대로 재현했습니다. 마감 시각을 `time.UTC`로 계산했더니 실제보다 아홉 시간 늦어져, 마감 한 시간 전에 낸 보고서가 지각으로 나왔습니다. v0.71이 고친 바로 그 실수 — 서비스 시간대가 아닌 곳에서 순간을 만드는 것 — 를 테스트 안에서 다시 저지른 셈이고, 주석에 그렇게 적어 뒀습니다.

업무 검색의 두 글자 하한도 고정했습니다. 한 글자 질의는 오류가 아니라 **빈 결과**입니다 — 화면이 타이핑할 때마다 요청을 보내기 때문입니다. 다만 검색이 *실행되어서는* 안 됩니다. 비교를 뒤집으면 진짜 질의가 전부 빈손이 되고 한 글자가 전수를 훑는데, 아무도 몰랐습니다. 처음엔 400을 기대했다가 실제 설계가 더 낫다는 것을 확인하고 기대를 고쳤습니다.

원칙으로 남깁니다. **계산이 맞는지 고정했다면, 그 계산이 무엇을 결정하는지도 고정해야 합니다.** 시각이 정확한 것과 그 시각으로 옳게 나누는 것은 다른 두 가지입니다.

3.10.39 v0.85.0에서 완료 — 위험 목록에 완료된 업무가 올라와도 아무도 몰랐습니다

남은 변이 중 예측 판정 계열 넷을 봤습니다. 전부 **경계와 제외 규칙**이고, 전부 관리자 화면에 틀린 문장을 내놓는 부류입니다.

`noLandingDate`의 제외 조건을 `||`에서 `&&`로 바꾸면 완료된 업무가 "끝날 주차를 못 잡는 업무" 목록에 오릅니다. 정체 업무를 빼는 규칙은 테스트가 있었는데, 바로 옆의 완료 업무 제외는 없었습니다. 주석이 그 이유를 이미 적어 두고 있었습니다 — *"같은 업무를 두 제목으로 부르는 것이 위험 목록이 안 읽히게 되는 방식"*. 완료된 업무를 위험이라 부르는 것은 그보다 나쁩니다.

마감일이 마지막 보고 주차에 걸리면 `weeksLeft`가 0입니다. `<= 0`을 `< 0`으로 바꾸면 그 경우가 추정 분기로 넘어가, **이미 도착한 날짜에 대해 예보를 내놓습니다.** 기간 보고 쪽도 같은 모양이었습니다.

넷 다 고정했고, 각각 되돌려 따로 실패하는 것을 확인했습니다. 긍정 대조를 먼저 둔 것도 같습니다 — `noLandingDate`가 *참*을 내야 하는 두 경우를 먼저 놓지 않으면, 어디서나 거짓을 내는 구현도 통과합니다.

원칙으로 남깁니다. **규칙에 제외가 있다면 제외마다 사례가 있어야 합니다.** 하나를 시험하고 그 옆을 비워 두면, 빠진 쪽이 정확히 그 규칙이 막으려던 것을 통과시킵니다.

3.10.40 v0.86.0에서 완료 — 조직도가 두 단계뿐이라 보이지 않던 것

테스트를 굳히는 갈래가 수확 체감에 들어서 제품 쪽으로 돌아왔습니다. `scripts/scale-check.py`를 오랜만에 돌렸더니 22개 경로 모두 깨끗했고, `/insights/work-graph`가 457KB인 것도 뜯어보니 화면이 실제로 그리는 세 목록(각 200건)이었습니다. 여기까지는 설계대로입니다.

그러다 시드를 봤습니다. 조직 41개가 **깊이 2**입니다 — 뿌리 하나와 그 자식들. 그런데 가시성 술어는 조직도를 **재귀로** 훑습니다. 즉 "조직장은 자기 아래 전부를 본다"가 **한 홉 이상 확인된 적이 없습니다.** 실제 기업 조직도는 회사 → 본부 → 실 → 팀입니다.

네 단계를 세우고 물었습니다. 본부장이 세 단계 아래 팀원의 보고서를 보는가, 그리고 옆 가지는 못 보는가. **둘 다 맞았습니다.** 단건 조회와 목록 양쪽에서 확인했습니다 — 목록에서 조용히 빠지는 것은 오류 없이 같은 실패이기 때문입니다.

재귀를 한 단계로 줄이는 변이를 넣으니 실패합니다: *"본부장이 세 단계 아래를 못 봅니다 — 재귀 조회가 도중에 멈춥니다."*

시드도 고쳤습니다. 이 사각지대를 만든 것이 시드였습니다. 이제 회사 → 본부 4 → 실 8 → 팀 32의 네 단계이고, 각 본부에 본부장을 둡니다. 규모 점검이 한 팀짜리 시야와 회사 4분의 1짜리 시야를 함께 재게 됩니다.

그리고 제 도구가 저를 두 번 속였습니다. 깊은 조직도로 규모 점검을 돌리자 `/work-items?scope=TEAM`이 "호출마다 다름"으로 나왔습니다. 이 프로젝트가 전에 고친 "새로고침마다 순서가 바뀌는 목록"과 같은 모양이라 파고들었는데, DB 질의는 결정적이고 정렬은 안정적이고 입력 순서도 지켜집니다. 한참 뒤에야 정렬 키의 `ageWeeks`가 26에서 52로 바뀐 것을 알아챘습니다 — **기동 시 백필이 도는 중에 잰 것**이었습니다. 안정된 뒤에는 완전히 동일합니다.

없는 결함을 보고하는 도구는 아무 말도 안 하는 것보다 나쁩니다. 오후를 씁니다. 그래서 점검 앞에 **안정 대기**를 넣었습니다: 대표 경로를 연속 두 번 같은 답이 나올 때까지 폴링하고, 시간 안에 안 멈추면 *"움직이는 대상을 재고 있다"*고 먼저 말합니다. 그 경고 분기가 실제로 도는 것도 확인했습니다.

원칙으로 남깁니다. **측정 도구는 자기가 언제 못 믿을지를 말해야 합니다.** 그러지 않으면 도구를 믿은 사람이 그 대가를 치릅니다.

3.10.41 v0.87.0에서 완료 — 76명을 관할하는데 99바이트가 왔습니다

v0.86이 만든 네 단계 시드로 **본부장 시야**를 처음 재 봤습니다. 관할 인원 76명, `/team/reports` 응답 **99바이트** — 빈 목록입니다.

원인은 제 손이었습니다. 그 계정에 비밀번호를 넣으려고 관리자 API로 수정하면서 `organizationId`를 빼고 보냈고, **보내지 않은 필드가 지워졌습니다.** PUT의 의미로는 옳습니다. 손대지 않은 hq2는 조직이 그대로였습니다.

문제는 그 다음입니다. 소속을 잃은 조직장이 팀 보고서를 열면 **오류 없이 빈 목록**이 옵니다. 코드에 그 경우를 위한 분기가 이미 있는데, 하는 일이 빈 배열을 돌려주는 것뿐이었습니다.

```
if p.OrganizationID == nil {
    writeData(w, 200, reportListView{Items: []reportListItem{}, ...})
```

```
    return
}
```

화면에는 "조회할 팀 보고서가 없습니다"가 뜹니다. **조직원이 아무도 안 쓴 것과 글자 하나 다르지 않습니다.** 하나는 기다리면 되는 일이고 다른 하나는 관리자에게 전화해야 하는 일인데, 읽는 사람은 구분할 수 없습니다.

v0.66(데이터베이스 장애를 "로그인이 필요합니다"로), v0.69(사라진 슬라이드), v0.70(연결 안 된 Confluence 계정)과 같은 부류입니다 — **고칠 수 있는 설정 문제가 정상적인 빈 결과로 위장하는 것.**

빈 목록이 왜 비었는지 말하게 했습니다. 오류가 아니라 `notice` 입니다 — 계정은 유효하고 요청도 허용되기 때문입니다. 화면은 그 문구가 있으면 기본 안내 대신 그것을 보여 줍니다.

발단이 된 동작도 문서에 적었습니다. `PUT /admin/users/{id}` 는 문서에 아예 없었습니다. 이제 *"보낸 것으로 바꾸는 것이 아니라 보낸 것이 전부가 된다"* 고 적혀 있고, 생략하면 소속과 검토 책임자가 비워진다는 것, 그리고 **비밀번호만은 보냈을 때만 바뀐다**는 예외도 함께 적혀 있습니다. 그 예외는 깨져도 사람들이 로그인하지 못하게 되기 전까지 아무도 모르므로 테스트로 고정했습니다.

원칙으로 남깁니다. **빈 결과에는 종류가 있습니다.** 아무 일도 없었던 빈 것과 뭔가 잘못돼서 빈 것을 같은 화면으로 보여 주면, 고칠 수 있는 쪽이 영원히 고쳐지지 않습니다.

3.10.42 v0.88.0에서 완료 — 같은 침묵이 세 화면에 있었습니다

v0.87이 팀 보고서 목록에 "소속이 없어서 비었다"를 붙였습니다. 같은 조건을 코드 전체에서 찾으니 **세 곳**이 더 있었습니다.

화면	소속 없는 팀장이 보는 것
팀 주간보고	빈 목록 (v0.87에서 설명 추가)
대시보드	전부 0
인수인계 명단	본인 한 명

전부 200이고 전부 정상처럼 보입니다. 화면마다 문구를 붙일 수도 있었지만, 그러면 **다음에 추가되는 조직 기반 화면은 또 조용해집니다.**

그래서 세션이 한 번 말하게 했습니다. `/me` 가 `accountNotice` 를 함께 돌려주고, 앱 껍데기가 그것을 띄웁니다. 읽는 사람이 어느 화면에 있든 보이고, 관리자가 소속을 지정하면 사라집니다.

서비스 공지(`notice`)와 **일부러 분리했습니다.** 하나는 모두에게 하는 말이고 다른 하나는 당신에 관한 말입니다. 섞으면 관리자가 공지를 지울 때 이것도 지워집니다.

조건은 좁게 뒀습니다 — **소속 없는 팀장·조직장**만입니다. 관리자는 어차피 전부 보고, 일반 사용자의 본인 보고서는 소속과 무관하며 조직 기반 화면은 애초에 열리지 않습니다. 넓히면 아무 문제 없는 사람에게 경고가 뜨고, 그러면 아무도 안 읽습니다. 일곱 경우를 표로 고정하고, 조건을 넓히는 변이와 뒤집는 변이 둘 다 실패하는 것을 확인했습니다.

원칙으로 남깁니다. **같은 침묵이 세 곳에 있으면 세 번 고치지 말고, 말하는 자리를 하나 만듭니다.** 그래야 네 번째가 생겨도 조용해지지 않습니다.

3.10.43 v0.89.0에서 완료 — "찾지 못했습니다"가 찾아본 적 없을 때도 나왔습니다

v0.87·v0.88이 "설정 때문에 빈 것"을 다뤘으니, 이번엔 그 이웃을 봤습니다. **기능은 켜져 있는데 아직 아무것도 만들지 않은** 상태입니다.

화면 전체의 빈 문구 27개를 훑었습니다. 대부분은 정직한 "찾아봤는데 없음"이었고, 둘이 걸렸습니다.

Confluence 자동 초안 — 화면이 스스로와 모순되고 있었습니다. 머리말 칩은 `첫 수집 대기`라고 하는데, 본문은 *"이번 주에 발견된 Confluence 업무 후보가 없습니다"* 라고 단정합니다. 사용자가 읽는 것은 본문이고, 그 문장은 수집이 한 번도 끝나지 않았을 때도 똑같이 나옵니다. 이제 `lastSuccessAt`이 없으면 첫 수집이 아직이라고 말합니다.

업무 유사 검색 — 의미 검색이 돌지 않은 이유를 말하지 않았습니다. 응답에 `semantic` 불리언이 있지만 *참일 때만* 쓰입니다. 거짓이면 pgvector가 없어서인지, 임베딩이 설정 안 됐는지, 실패했는지, 아니면 정말 없는지 구분할 수 없고, 0건 화면은 네 경우 모두 "비슷한 과거 업무를 찾지 못했습니다"입니다.

`semanticReason`을 더했습니다. 셋 다 서로 다른 문장이고, 각자 다음에 무엇을 할지 말합니다 — 확장이 없다, 임베딩을 켜라, 로그를 봐라. **새 배포는 전부 이 상태에서 시작합니다.** 임베딩은 기본이 꺼져 있으니, 지금까지 모든 신규 설치가 "글자로만 찾고 있다"는 사실을 말한 적이 없습니다.

v0.68에서 AI 실패 일곱 가지를 서로 다른 문장으로 가른 것과 같은 작업입니다. 그때는 답을 못 받는 경우였고, 이번엔 **아직 답할 준비가 안 된** 경우입니다.

원칙으로 남깁니다. **0건에는 최소한 두 종류가 있습니다** — 찾아봤는데 없는 것과, 찾아볼 수 없었던 것. 둘을 한 문장으로 쓰면 고칠 수 있는 쪽이 영원히 고쳐지지 않습니다.

3.10.44 v0.90.0에서 완료 — 회의에 들고 갈 파일을 아무도 열어 본 적이 없었습니다

각도를 바꿨습니다. 이 제품이 존재하는 이유의 절반은 **회의에 들고 갈 PPTX**인데, `exportReportPPTX`의 커버리지가 **0%**였습니다. 200을 돌려주는 것과 PowerPoint가 여는 것은 다른 이야기이고, 다르다는 사실은

회의실에서 알게 됩니다.

내보낸 파일을 zip으로 열어 봤습니다. 항목 하나짜리 보고서의 덱이 **4쪽**이고, 그중 **3쪽이 열 머리와 `- -` 두 줄만 있는 빈 페이지**였습니다. 템플릿이 4장이라 언제나 4장이 나갑니다.

`distributeReferenceItems`는 항목 수만큼만 나누고 있었습니다. 문제는 호출부가 **템플릿의 슬라이드를 전부 렌더링**한 것입니다.

빼는 것이 더하는 것보다 까다롭습니다. zip에서 슬라이드 파일만 지우면 `[Content_Types].xml`·`ppt/presentation.xml`·`presentation.xml.rels`가 없는 장을 계속 가리키고, 그런 파일은 **내려받아지고 나서 안 열립니다.** 세 곳을 함께 정리했고, 슬라이드 순서는 파일 이름이 아니라 **관계 ID**로 참조되므로 rels에서 ID를 먼저 뽑아 지웁니다.

테스트가 파일을 실제로 엽니다. 필수 파트가 있는지, 슬라이드 수가 일인지, 그리고 **세 목록이 실제 슬라이드 수와 정확히 일치하는지**를 각각 확인합니다. 셋 중 하나만 어긋나도 안 열리는 파일이므로 셋을 따로 셉니다.

변이 세 개로 확인했습니다 — 안 지우면, 파일만 지우고 목록을 두면, 관계만 지우고 순서를 두면 각각 다른 문장으로 실패합니다.

기존 테스트 둘이 깨졌고, 그게 맞습니다. 둘 다 1~2건짜리 보고서를 렌더링하고 "슬라이드 4장"을 기대하고 있었습니다 — 옛 동작을 고정한 것이지 옳음을 고정한 것이 아니었습니다. 기대를 "일한 만큼"으로 바꾸고 왜 바꿨는지 적었습니다.

아직 안 고친 것도 적어 둡니다. 편집기는 이슈를 받는데 이 템플릿에는 이슈 칸이 없어 **덱에 실리지 않습니다.** 회의 자료에 없는 이슈는 회의에서 제기되지 않습니다. 다만 이것은 템플릿의 형식을 바꾸는 문제라, 슬라이드 구조를 건드리는 변경과 같은 릴리즈에 섞지 않았습니다.

원칙으로 남깁니다. **결과물은 열어 봐야 합니다.** 200과 열리는 파일은 다른 것이고, 그 차이를 확인하지 않으면 사용자가 회의실에서 대신 확인합니다.

3.10.45 v0.91.0에서 완료 — 작성자가 쓴 이슈가 회의 자료에 없었습니다

v0.90에서 미뤄 둔 것을 마쳤습니다. 편집기는 업무마다 **이슈**를 받는데, 기본 템플릿으로 내보낸 덱에는 그 문장이 없습니다.

원인을 보니 절반은 이미 되어 있었습니다. `{{ISSUES}}` 토큰이 예전부터 있어 **직접 올린 템플릿에는 이슈가 들어갑니다.** 문제는 제품이 함께 배포하는 참조 템플릿입니다 — 추진실적과 추진계획 두 칸뿐이고 세 번째 자리가 없어, **기본 설치 전부에서** 작성자가 쓴 이슈가 데이터베이스까지만 가고 회의실에는 가지 않았습니다.

형식을 바꾸는 대신 장을 하나 더했습니다. 조직들이 표준으로 삼고 있을 두 칸 배치는 그대로 두고, 이슈가 있을 때만 「이슈 및 애로사항」 페이지가 뒤에 붙습니다. v0.90이 빈 장을 걷어냈으므로 이 장은 일하는 페이지들

바로 뒤에 옵니다.

한 장에 안 들어가면 그렇게 말합니다. 30건을 넣어 보니 페이지가 넘치는데, 넘친 것을 말없이 버리는 대신 *"이 페이지에 담지 못한 이슈가 N건 더 있습니다"* 를 적습니다. 이 프로젝트가 v0.25부터 지켜 온 규칙 — 잘랐으면 잘랐다고 말한다 — 이 화면뿐 아니라 종이에도 적용됩니다.

변이 셋으로 확인했습니다: 이슈가 없어도 장을 붙이면, 넘친 것을 말없이 버리면, 내보내기에서 장을 안 붙이면 각각 실패합니다.

원칙으로 남깁니다. **입력을 받는 자리와 그것이 나가는 자리는 함께 있어야 합니다.** 받아 놓고 내보내지 않으면, 사용자는 자기가 적은 것이 전달됐다고 믿습니다.

3.10.46 v0.92.0에서 완료 — 경영진에게 하는 요청이 경영진이 받는 자료에만 없었습니다

v0.91이 이슈를 덱에 실었으니, 같은 질문을 나머지 내보내기에 했습니다. CSV는 12개 열로 충분했고, **`ManagementAsk`가 걸렸습니다.**

이 필드의 주석이 스스로를 이렇게 설명합니다 — *"보고 라인이 결정하거나 제공해야 하는 것. 이슈와 일부러 분리한다: 이슈는 무엇이 잘못됐는지를 말하고, 요청은 위에서 무엇을 해야 하는지를 말한다."* 이 제품에서 가장 경영진을 향한 필드입니다.

닿는 곳을 세어 보니:

소비처	요청이 실리는가
CSV 내보내기	✓
회의 안건	✓
인수인계 노트	✓
보고 품질 점검	✓
주간보고 덱	✗
기간 보고 덱	✗

덱이 경영진이 읽는 물건입니다. 넷에 실리고 둘에 빠졌는데, 하필 빠진 둘이 그것입니다.

기간 보고 덱의 조립 주석은 이미 이렇게 적어 두고 있었습니다 — *"결정을 위험보다 먼저: 기간이 스스로를 설명한 다음 요청한다. 요청으로 끝나는 덱이라야 방에서 조치할 수 있다."* 그런데 그 덱이 끝나던 "요청"은

제품이 추론한 위험 목록이었지 누가 쓴 문장이 아니었습니다. 이제 「상위 조직 요청 · 결정 필요」 장이 세부 다음, 결정 앞에 옵니다.

주간보고 덱은 v0.91이 만든 장을 넓혔습니다. **한 장에 두 블록**이고 각각 내용이 있을 때만 나타나며, 제목이 그에 따라 바뀝니다(이슈만 / 요청만 / 둘 다). **합치지 않은 것이 요점입니다** — 편집기가 작성자에게 구분해서 쓰라고 요구하는 그 차이를, 내보내면서 뭉개면 요구한 의미가 없습니다.

양쪽 모두 넘치면 몇 건이 빠졌는지 적습니다. 변이 셋으로 확인했습니다 — 요청 블록 제거, 두 블록 합치기, 기간 덱의 장 제거가 각각 실패합니다.

원칙으로 남깁니다. **어떤 값이 어디에 닿는지는 세어 봐야 압니다.** 네 곳에 있으면 다 있는 것처럼 보이고, 빠진 두 곳이 하필 가장 중요한 곳일 수 있습니다.

3.10.47 v0.93.0에서 완료 — 두 번 연속 덱을 바꿨으니, 실물을 열어 확인했습니다

v0.90·v0.91·v0.92가 연달아 PPTX 생성을 건드렸습니다. 슬라이드를 빼고, 장을 붙이고, 또 붙였습니다. 그래서 이번 반복은 새 결함을 찾기 전에 **방금 낸 것들이 실제로 동작하는지**부터 확인했습니다.

릴리즈 이미지를 띄워 실제 보고서를 만들고 내려받았습니다.

확인	결과
주간보고 덱	2쪽(업무 + 이슈·요청), 세 목록 일치
기간 보고 덱	5쪽, 「상위 조직 요청」 장 포함
CSV	12열, 이슈와 요청 모두 포함

그러다 제가 v0.91에서 만든 위험을 알아챘습니다. 이제 `appendSlidesToPPTX`가 한 번의 내보내기에서 **두 번** 불립니다 — 이슈 장을 붙이고, 그다음 캡처 이미지를 붙입니다. 두 번째 호출은 **첫 번째 결과 위에서** 동작하는데, 관계 ID를 새로 매기지 않고 1부터 다시 세면 같은 ID가 두 번 나오고 그 파일은 열리지 않습니다.

시험한 적이 없었습니다. 시험해 보니 **올바르게 이어 붙입니다** — 기존 최대 ID 다음부터 셉니다. 다만 그것을 지키는 것이 아무것도 없었으므로 가드를 세웠습니다: 이슈와 캡처를 함께 가진 보고서를 내보내 세 목록이 일치하는지, 그리고 **관계 ID가 중복되지 않는지**를 확인합니다.

번호를 1부터 다시 세게 바꾸니 정확히 그 문장으로 실패합니다 — *"관계 ID rId1이 두 번 나옵니다. 두 번째 붙이기가 첫 번째의 번호를 재사용했습니다."*

결정 기록은 CSV에 없고, 그건 맞습니다. 업무와 열이 다른 별개의 표라 한 파일에 둘을 넣으면 어느 도구도 읽지 못합니다. 다만 적혀 있지 않아 내려받은 사람이 알 수 없었으므로, CSV가 무엇을 담고 무엇을 담지 않는

지, 결정은 어디에 있는지를 문서에 적었습니다.

원칙으로 남깁니다. **한 곳을 연달아 고쳤으면 멈추고 실물을 확인합니다.** 세 릴리즈가 각각 옳아도 함께 동작한다는 뜻은 아니고, 그 조합을 시험하는 것은 아무것도 없었습니다.

3.10.48 v0.94.0에서 완료 — 제품의 절반에 실행되는 검증이 하나도 없었습니다

아직 실물을 본 적 없는 산출물을 찾다가 **발표 모드**에 닿았습니다. 회의 안건은 이미 잘 덮여 있었고 (`buildMeeting` 98%), 발표 슬라이드가 이슈와 상위 조직 요청을 그리는 것도 확인했습니다. 여기까지는 괜찮습니다.

문제는 그것을 확인하는 것이 아무것도 없었다는 점입니다. 프런트엔드 전체에 실행되는 테스트가 하나도 없습니다. 제품의 절반이고, `presentSlides.ts`는 회의실에서 사람이 띄우는 화면을 결정합니다 — PPTX 내보내기와 같은 등급의 산출물인데 세 릴리즈를 들여 검증한 쪽과 달리 이쪽은 비어 있었습니다.

v0.75에서 핸들러 196개를 두고 "구조적으로 못 한다"가 대개 "아직 안 했다"라고 적었습니다. 같은 문장이 여기에도 적용됩니다.

vitest를 넣고 여덟 가지를 고정했습니다. 전부 **회의실에서 드러나는** 성질입니다.

고정한 것	왜
빈 항목은 블록을 만들지 않는다	라벨만 있고 내용 없는 줄은 읽고도 아무것도 못 얻습니다
길면 자르고 **잘랐다고 표시한다**	줄여서 맞추면 뒷줄에서 안 보입니다
발표자 화면에는 **자르지 않은 원문**	모든 단어가 필요한 사람은 발표자 한 명입니다
파일 없는 캡처는 띄우지 않는다	빈 화면은 없는 것보다 나쁩니다 — 방이 기다립니다
캡처는 표지 **다음**	이름표 없는 스크린샷으로 회의를 여는 것은 아무 말도 안 하는 것입니다
요청이 있으면 종료 화면이 결정을 확인하라고 말한다	v0.92가 덱에 실은 그 필드입니다

변이 넷으로 확인했습니다 — 빈 블록 넣기, 자름 표시 없애기, 없는 캡처 띄우기, 발표자 화면까지 자르기가 각각 실패합니다.

변이 하나가 처음엔 통과했습니다. 셸 안에 쓴 Python 문자열에서 `\n`이 진짜 줄바꿈이 되어 치환 대상이 안 맞았고, 적용되지도 않은 변이를 "가드가 못 잡았다"고 읽을 뻔했습니다. 이 세션에서 두 번째로 같은 함정을 밟았습니다 — 도구가 조용히 아무것도 안 했을 때 그것을 결과로 착각하는 것.

원칙으로 남깁니다. **검증이 없는 절반은 없는 절반입니다.** 한쪽을 세 릴리즈 동안 파고들면서 다른 쪽을 열어 보지 않으면, 고친 만큼 안심하게 되는데 그 안심의 근거는 절반뿐입니다.

3.10.49 v0.95.0에서 완료 — 같은 버그의 반대쪽 절반은 반대쪽 시간대에서만 보입니다

v0.94가 프런트엔드에 첫 테스트를 놓았으니, 다음은 **`localdate.ts`** 입니다. v0.65에서 화면 네 곳이 조용히 하루씩 밀렸던 바로 그 자리이고, 고친 뒤로 그것을 지켜보는 것이 하나도 없었습니다. `toISOString()`은 Date를 yyyy-mm-dd로 바꾸는 가장 자연스러워 보이는 방법이라, 다음 사람이 다시 손을 뻗기 쉬운 종류의 버그입니다.

여덟 가지를 고정했습니다 — 자정과 23:59가 같은 날인 것, 한 자리 월·일의 0 채움, 연말·윤년 경계, 주 이동이 **같은 요일에 머무는 것**.

그런데 변이 하나가 통과했습니다. 문자열을 `new Date(day)`로 UTC 파싱하도록 바꿨는데 한국 시간대에서 전부 통과합니다. UTC 자정이 현지 09:00이라 같은 날 안에 있기 때문입니다. 같은 변이를 `America/New_York`에서 돌리니 **세 개가 실패합니다** — 그곳에서는 UTC 자정이 전날 20:00입니다.

즉 이 버그에는 절반이 둘 있습니다.

	잘못	어디서 드러나는가
변환	`toISOString()`으로 출력	그리니치 **동**쪽
파싱	`new Date('yyyy-mm-dd')`로 입력	그리니치 **서**쪽

한 시간대에서만 돌리면 언제나 절반만 잡습니다. `npm test`가 이제 **두 시간대에서 연달아** 돕니다. 세 변이 모두 잡히는 것을 확인했습니다.

v0.65의 가드가 제 테스트 파일을 잡았습니다. 틀린 방식을 시연하려고 일부러 쓴 자리였고, 가드가 옳게 동작한 것입니다. 테스트는 제품 동작이 아니므로 `.test.ts`를 제외하되, 제외 뒤에도 **제품 코드에 같은 호출을 넣으면 여전히 잡히는지** 확인했습니다 — 잡습니다.

원칙으로 남깁니다. **환경에 따라 달라지는 코드는 한 환경에서만 시험하면 절반만 시험한 것입니다.** 어느 절반인지는 시험한 사람이 어디 앉아 있느냐가 정합니다.

3.10.50 v0.96.0에서 완료 — 방이 시간에 쫓길 때 무엇을 먼저 듣는가

프런트엔드의 남은 순수 로직을 마저 덮었습니다. `presentSlides.ts`의 기간 보고·회의 빌더인데, 여기 든 것은 겉모양이 아니라 **회의 결과**입니다.

코드가 스스로 그렇게 말하고 있었습니다 — *"시간이 모자란 방은 알파벳 순 첫 다섯 개가 아니라 위험을 들었어야 한다."* 그 문장을 지키는 것은 아무것도 없었습니다.

여덟 가지를 고정했습니다.

규칙	왜
위험·정체 업무가 먼저 온다	시간이 끊기는 지점이 목록 앞이라야 합니다
상위 조직 요청도 먼저 볼 업무다	v0.92가 덱에 실은 그 필드이고, 같은 이유로 순서에도 듭니다
공백뿐인 요청은 조치 대상이 아니다	공백을 세면 조치 목록이 조치가 아닌 것으로 찹니다
경영 인사이트가 업무 목록보다 앞선다	판단이 먼저, 근거가 나중
후속 조치가 남은 결정이 먼저	이미 이행된 것은 역사이고, 남은 것이 이 브리핑의 이유입니다
표지는 존재하는 안건 수, 구획은 보여 주는 수	2,100건짜리 제목에 40건이라 적는 덱은 앉아 있는 방에 거짓말을 합니다

변이 넷으로 확인했습니다 — 순서 뒤집기, 요청을 조치 대상에서 빼기, 결정 정렬 뒤집기, 구획이 표시 수만 말하게 하기가 각각 실패합니다.

프런트엔드 테스트는 이제 세 파일 24개이고, `npm test`가 두 시간대에서 각각 돕니다. v0.94에서 **하나도 없던** 것이 세 릴리즈 만에 제품의 판단이 든 자리를 덮었습니다.

원칙으로 남깁니다. **주석이 이유를 적어 둔 자리가 가장 먼저 시험할 자리입니다.** 누군가 왜 그렇게 했는지 적어 뒀다는 것은, 그것이 쉽게 반대로 될 수 있다는 뜻입니다.

3.10.51 v0.97.0에서 완료 — 새 탭으로 열 수 없는 도구를 하루 종일 씁니다

검증 갈래를 닫고 제품으로 돌아왔습니다. 실제 시드를 넣은 배포에 브라우저를 붙여 **화면 열세 개를 처음부터 끝까지 눌러 봤습니다.** 모두 정상이고 오류는 없었습니다.

그런데 자동화를 붙이다가 걸린 것이 있습니다. 탐색 항목을 링크로 찾으려 했더니 **하나도 없습니다.** 사이드바 전체가 `<button>`입니다.

버튼인 것은 **의도적이었습니다.** 저장하지 않은 편집이 있는 화면을 떠날 때 먼저 물어야 하고, 평범한 앵커는 묻기 전에 이동해 버립니다. 대가는 링크가 거저 주는 것 전부였습니다 — 가운데 클릭, Ctrl+클릭, "새 탭에서 열기", "링크 주소 복사".

하루 종일 열어 두는 사내 도구에서 **내 보고서와 팀 보고서를 두 탭에 놓고 비교하는 것**은 평범한 요구인데, 방법이 없었습니다.

앵커로 바꾸고 **평범한 왼쪽 클릭만 가로챕니다.** 보조 키가 눌린 클릭과 가운데·오른쪽 버튼은 브라우저에 그대로 넘깁니다 — 그것들은 "다른 데서 열어라"라는 뜻이고, 이 탭에서 버려지는 것은 없으니 물을 이유도 없습니다.

브라우저로 확인했습니다: Ctrl+클릭이 새 탭에 `#/team`을 열고 원래 탭은 `#/dashboard`에 남습니다. 평범한 클릭은 전체 새로고침 없이 앱 안에서 이동합니다. 사이드바의 모양(여백·둥근 모서리·활성 배경)도 그대로입니다 — 요소를 바꾸면서 CSS 선택자 일곱 개를 함께 옮겼습니다.

판단 규칙을 `appHandlesClick`으로 빼내 고정했습니다. 변이 셋으로 확인했습니다 — 보조 키를 무시하면(새 탭이 안 열림), 가운데 버튼까지 가로채면, 평범한 클릭을 브라우저에 넘기면(전체 새로고침) 각각 실패합니다.

제 도구가 또 저를 속일 뻔했습니다. 링크 텍스트를 `^\W+`로 다듬었는데 JavaScript에서 한글은 `\W`라 이름이 통째로 지워졌고, 열세 화면이 전부 대시보드로 보였습니다. 화면이 고장 난 줄 알았습니다.

원칙으로 남깁니다. **의도한 제약이 의도하지 않은 것까지 막고 있는지 가끔 세어 봐야 합니다.** 버튼을 고른 이유는 옳았고, 그 이유가 링크의 나머지 전부를 가져가야 할 이유는 아니었습니다.

3.11 v0.21 — Source-to-Report Agent

- 커밋·결재·Jira에서 사용자 동의 아래 근거를 수집해 WorkItem 후보와 출처 링크를 생성합니다. v0.17의 provenance 모델을 전제합니다.

3.12 v1.0 — Enterprise Work Intelligence Platform

- 조직 Knowledge Graph. 사람·업무·시스템·프로젝트·문서 관계를 질의합니다.
- 의미 기반 검색 자체는 v0.12에서 기반이 놓였습니다(4.4). 남은 과제는 검색 결과를 관계 질의로 확장하는 것입니다.

---

4. 선행 기술 결정 사항

로드맵 후반에서 발견하면 비용이 큰 결정입니다. 해당 단계 이전에 확정합니다.

4.1 ReportItem 저장 방식 변경 (v0.11에서 해결)

보고서 수정은 `report_items`를 전량 삭제하고 다시 삽입해 저장할 때마다 행 식별자가 바뀌었습니다. 그 상태에서는 `work_item_id`를 추가해도 다음 저장에서 연결이 사라집니다.

v0.11에서 1번(안정 식별자 기준 재조정)을 적용했습니다. 제출된 항목 중 기존 행을 가리키는 것은 갱신하고, 새 항목만 삽입하며, 사라진 항목만 삭제합니다. 세 번 연속 저장해도 행 식별자와 업무 연결이 유지되는 것을 확인했습니다.

4.2 상태 볼륨에 저장하는 데이터

비밀 설정 키(4.1의 사례)와 첨부 이미지 파일이 `/var/lib/weekly`에 있고, 나머지는 PostgreSQL에 있습니다. 두 저장소가 분리돼 있으므로 **한쪽만 유지한 업그레이드는 조용히 깨집니다.** 행은 남고 파일만 사라지므로 목록은 정상으로 보이고 조회만 404가 됩니다.

v0.14.2에서 기동 시 유실을 감지해 로그로 남기고, 목록 응답이 파일 존재 여부를 반환하도록 했습니다. 이후 상태 볼륨에 파일을 추가로 저장하는 기능은 **같은 방식의 무결성 확인을 함께 제공해야 합니다.**

v0.15.0에서 감지에 예방을 더했습니다. 백업 도구가 두 저장소를 같은 복구 지점으로 묶고, 참조된 첨부 파일 수와 실제 보관된 파일 수를 비교해 부족하면 0이 아닌 종료 코드를 냅니다. 순서도 임의가 아닙니다. 업로드가 파일을 먼저 쓰고 행을 나중에 넣으므로 **DB를 먼저 덤프하고 파일을 나중에 복사해야** 덤프에 있는 행의 파일이 반드시 존재합니다.

4.3 자동 판정의 위치

제목 유사도 병합, Decision 추출, 변화 분류는 모두 **제안**이어야 하며 사용자 확정 없이 확정 데이터로 승격하지 않습니다. v0.6.0의 병합 오판 사례(한 글자 차이 업무가 하나로 합쳐짐)가 근거입니다.

v0.18.0에서 마지막 예외를 없앴습니다. 기간 보고가 저장된 식별자를 무시하고 제목으로 다시 추측하고 있었고, 그 결과 사용자가 방금 나눈 업무를 한 화면이 도로 합칠 수 있었습니다. **자동 판정을 쓰는 곳은 저장된 정정이 있으면 그것을 먼저 봐야 합니다.**

v0.11의 WorkItem은 이 원칙을 지키지 못한 채 출발했습니다. 제목 정규화 결과가 곧 확정이었고 정정 경로가 없었습니다. v0.16.0의 병합·분리로 해소했습니다. 이후 자동 판정을 저장 데이터로 쓰는 기능은 **정정 경로를 같은 릴리즈에 함께 제공해야 하며, 그 정정이 다음 자동 판정에도 살아남아야 합니다.**

4.4 임베딩 저장 방식 (v0.12에서 해결)

의미 기반 검색은 임베딩이 필요합니다. 오프라인망에서 `pgvector` 설치를 전제할 수 없어 세 가지 안을 두고 2번(배열 컬럼 + 애플리케이션 코사인 계산)을 권장했습니다.

v0.12에서 **1번(`pgvector` 선택 설치)**을 채택했습니다. 권장을 바꾼 근거는 다음과 같습니다.

- 실제 운영 대상 PostgreSQL에 `pg_trgm`과 `pgvector`가 이미 설치돼 있음이 확인됐습니다. 전제할 수 없다는 판단의 근거가 사라졌습니다.
- 확장 부재가 장애가 되지 않도록 마이그레이션이 확장 생성 실패를 흡수하고, 기동 시 감지한 능력에 따라 해당 검색 단계만 비활성화합니다. 즉 1번을 택해도 3번의 안전성을 유지합니다.
- 2번은 같은 안전성을 얻는 대신 정렬과 상한 처리를 애플리케이션으로 가져와야 하고, 확장이 있는 환경에서도 그 비용을 계속 냅니다.

임베딩 생성은 AI Gateway를 사용하므로 AI 비활성 환경에서는 의미 검색이 비활성화되고 앞의 두 단계만 동작합니다.

4.5 유사도 판정에 쓸 함수와 임계값

측정 없이 고르면 안 되는 항목입니다. v0.12에서 실제 말뭉치로 측정해 결정했습니다.

- **검색에는 `word_similarity`** : `similarity`는 질의어가 문서 전체에서 차지하는 비중을 재므로 긴 제목 속의 짧은 검색어를 놓칩니다(`전표검증` → `월결산 자동화` 0.214). `word_similarity`는 0.400으로 찾아냅니다.
- **워드 클라우드 합산에는 유사도를 쓰지 않습니다**: 합쳐야 할 `전표 검증` · `전표검증`(0.375)이 합치면 안 되는 `서버 A 점검` · `서버 B 점검`(0.600)보다 낮아 임계값이 존재하지 않습니다. 공백 제거 후 정확 일치로 판정합니다.
- **의미 검색 임계값은 고정하지 않습니다**: 코사인 점수 범위가 임베딩 모델마다 다릅니다. 설정으로 노출하고 관리자 화면에 임베딩 현황을 함께 제공합니다.

---

5. 리소스 및 품질 관리 전략

- **테스트 자동화**: Go 유닛 테스트, PostgreSQL 격리 통합 테스트, React 타입 검사와 Production Build를 릴리즈마다 실행합니다.
- **무중단 마이그레이션**: 스키마는 전진 전용으로 적용하고 기존 데이터는 변경하지 않습니다. 자동 down migration은 제공하지 않습니다.
- **오프라인 우선**: 모든 신규 기능은 AI Gateway와 외부 연동이 없는 환경에서도 핵심 동작이 성립해야 합니다. AI는 품질을 높이는 선택 계층으로 둡니다.
- **근거 보존**: 자동 생성 결과는 출처를 함께 저장하고, 근거를 확인할 수 없는 결과는 저장 후보로 사용하지 않습니다.

- **회귀 검증**: 릴리즈마다 실제 컨테이너를 기동해 주요 흐름을 확인하고 릴리즈 본문에 검증 내역을 남깁니다.

3.10.52 v0.98.0에서 완료 — 물어보긴 하는데, 사람들이 실제로 누르는 버튼에서는 안 물어봅니다

v0.97에서 사이드바를 링크로 바꾼 것이 회귀를 냈는지 먼저 확인했습니다. 두 버전을 나란히 띄우고 같은 이동을 시켜 보니 주소도 화면도 똑같이 움직였습니다. **회귀는 없었습니다.** 앞선 추적이 "주소가 안 바뀐다"고 읽힌 것은, 그 컨테이너에 남아 있던 이전 실행의 제출 상태 보고서를 보고 있었기 때문입니다.

그래서 손으로 확인한 규칙을 시험으로 굳히는 데서 시작했습니다. 저장하지 않은 변경이 있을 때 묻고, "아니오"를 지키는 규칙입니다. `unsavedGuard.test.ts` 일곱 개를 붙이고, 가드를 세 가지로 망가뜨려 봤습니다 — 확인 창을 항상 통과시키기, 조건 뒤집기, 더러움 판정을 항상 거짓으로 하기. 셋 다 잡힙니다.

그러다 이 가드가 **한 화면만 지키고 있다**는 것을 봤습니다. 등록하는 곳은 이번 주 편집기 하나뿐인데, 과거 보고 화면에는 자기만의 수정 편집기가 따로 있습니다. 그쪽은 전역 가드에 등록하지 않습니다.

브라우저로 재현했습니다. 과거 보고를 열어 수정하고 본문을 고친 다음:

- 새로고침·탭 닫기 → **경고 없음.** 고친 내용이 조용히 사라집니다.
- 뒤로가기 → **묻지 않음.** 창이 닫히고 고친 내용이 사라집니다.

사이드바는 모달 배경이 막고 있어서 애초에 눌리지 않습니다. 막힌 길 하나를 근거로 나머지가 안전하다고 볼 수는 없었습니다. 같은 앱의 이번 주 편집기는 이 두 길을 모두 막습니다. **같은 제품 안에서 한쪽만 지키는 것이 아무 데도 안 지키는 것보다 나쁩니다.** 사용자는 경고가 뜬다는 것을 배우고, 그 뒤로는 경고가 없으면 안전하다고 믿게 되기 때문입니다.

과거 보고 편집기의 더러움 판정을 전역 가드에 등록해서 두 길을 막았습니다. 그리고 거짓 경고가 생기지 않는지 여섯 갈래로 확인했습니다 — 목록만 볼 때, 읽기만 할 때, 수정을 눌렀지만 아직 안 고쳤을 때는 묻지 않고, 실제로 고쳤을 때만 묻고, 닫은 뒤에는 다시 안 묻습니다.

그 확인 과정에서 **두 번째 결함**이 나왔습니다. 고친 상태에서 × 로 닫았는데 아무것도 묻지 않았습니다.

`Modal`은 `beforeClose`를 Escape와 배경 클릭에만 걸어 둡니다. 대화상자 머리의 × 버튼은 부르는 쪽이 직접 쓰는 것이라, 무엇을 부르든 자유입니다. 그래서 이렇게 되어 있었습니다.

```
<button onClick={close}>×</button>
```

`beforeClose`를 거치지 않습니다. **Escape는 묻고, × 는 안 묻습니다.** 그리고 × 는 사람들이 실제로 누르는 버튼입니다.

`beforeClose`를 쓰는 대화상자는 둘인데 — 과거 보고와 관리자 사용자 편집 — **둘 다** 이 결함을 갖고 있었습니다. 2곳 중 2곳이면 부르는 쪽이 잊어버릴 수 있는 종류의 실수입니다. 관리자 쪽은 `beforeClose`에 이름 없는 함수를 그대로 넣어 둬서, × 가 같은 검사를 재사용할 방법 자체가 없었습니다.

둘 다 이름 있는 함수로 빼고 × 를 그 함수에 태웠습니다. 그리고 재발을 막는 정적 검사를 만들었습니다. `scripts/modal-close-check.py`는 `beforeClose`를 쓰는 파일에서 두 가지를 봅니다.

- 가드가 **이름 있는 함수**인가. 이름 없는 함수는 재사용할 수 없고, × 가 자기 동작을 따로 갖게 된 이유가 정확히 그것입니다.
- 그 파일의 × 버튼이 **모두 그 이름을 거치는가.**

고치기 전 상태에 돌려 보니 세 건을 모두 잡습니다. CI에 넣었습니다.

한 가지 도구 함정을 적어 둡니다. 이 검사의 첫 정규식은 `<button([^>]*)>` 로 속성을 읽으려 했고, **0개를 찾았습니다.** 화살표 함수의 `=>` 안에 있는 `>` 를 태그 끝으로 읽기 때문입니다. 통과했다고 착각하기 딱 좋은 실패였습니다 — 검사가 아무것도 안 보고 "통과"를 찍었습니다. × 글자를 먼저 찾고 거기서 뒤로 걸어가도록 바꿔서 두 개를 찾았습니다. **검사가 몇 개를 봤는지 항상 같이 찍어야 합니다.**

3.10.53 v0.99.0에서 완료 — 알림이 버튼 위에 앉아서 안 비켜 줍니다

v0.98에서 "한쪽 출구만 지키는" 패턴을 두 번 봤으니, 같은 것이 더 있는지 세어 봤습니다. 긴 글을 받는 화면 여섯 곳 중 가드에 등록한 곳은 둘뿐이었고, 그중 `ImportPage`가 남아 있었습니다. 나머지 셋(댓글·결정·의존성)은 한 줄짜리 입력이라 경고를 붙이면 그게 거짓 경고가 됩니다.

`ImportPage`는 **편집 가능한 초안 전체**를 들고 있습니다. PPTX를 분석한 결과를 사람이 고쳐서 확정하는 화면이고, 그 고친 값은 확정 전까지 브라우저에만 있습니다. 그런데 코드는 이미 이것을 잃을 값으로 취급하고 있었습니다.

- 재분석할 때: `'현재 분석 편집값을 버리고...'` 라고 묻습니다.
- 파일을 창 아무 데나 흘렸을 때: 주석이 *"브라우저가 그 파일을 열며 페이지를 떠나 검토 중이던 내용을 버립니다"* 라고 적고 막아 뒀습니다.

두 출구는 막고 나머지는 열어 뒀습니다. 세 번째 같은 패턴입니다.

확인하려면 초안이 실제로 화면에 떠야 하는데, 가져오기는 AI Gateway가 있어야 돕니다. `ai.endpoint`가 설정값이라 OpenAI 호환 응답을 돌려주는 스텁 게이트웨이를 세우고, 제품 자신이 내보낸 PPTX를 다시 올려서 진짜 초안을 만들었습니다. 이 갈래는 이제 가져오기 화면 전체를 브라우저로 확인할 수 있게 해 줍니다.

고치기 전 v0.98.0에서 재현했습니다. 초안을 고친 다음:

- 새로고침·탭 닫기 → **경고 없음.**

- 사이드바로 이동 → **묻지 않음.** 화면이 바뀌고 고친 내용이 사라집니다.

기준값을 잡아 전역 가드에 등록했습니다. 여기서 조심할 것은 **거짓 경고**였습니다. 분석 결과가 곧 초안이므로, "초안이 있으면 저장 안 한 변경"으로 치면 가져오기 화면에 들를 때마다 나갈 때 경고가 뜹니다. 그래서 `draftFromFile`이 만든 값을 그대로 기준값으로 잡고, 그것과 달라졌을 때만 셉니다. 확정에 성공하면 기준값을 현재 값으로 옮겨서 저장한 뒤에도 묻지 않게 했습니다.

네 갈래로 확인했습니다 — 들어가자마자, 분석 결과만 있고 안 고쳤을 때, 고쳤을 때, 나갈 때. 그리고 확정 뒤에 다시 확인했습니다.

확정 버튼을 누를 수 없었습니다

그 확인 과정에서 두 번째 결함이 나왔습니다. 브라우저가 `선택 보고서 확정 저장` 버튼을 30초 동안 못 눌렀습니다. 가로막은 것은 **성공 알림 토스트**였습니다.

재 봤습니다.

```
버튼 한가운데를 토스트가 덮나: 예 · 그 자리에서 잡히는 것: toast success
20초 뒤 토스트: 그대로 있음
```

`.toast` 는 `position:fixed; right:25px; bottom:25px; z-index:100` 이고, 확정 바는 `position:sticky; bottom:12px; z-index:5` 에 버튼이 오른쪽 끝입니다. 정확히 겹칩니다. 그리고 **토스트에는 사라지는 타이머가 없었습니다.** `notify` 가 띄우면 사용자가 × 를 누르거나 다른 알림이 덮을 때까지 남습니다. 업로드는 항상 알림을 띄우므로, 가져오기의 주 동작이 알림에 막힙니다.

두 가지를 고쳤습니다.

- **성공 알림은 5초 뒤 스스로 사라집니다.** 오류는 남습니다 — 두 번 읽어야 할 수 있고, 읽는 사람이 닫을 일입니다. 알림에 `id` 를 붙여 다시 그리게 해서, 같은 문장이 연달아 뜰 때 두 번째가 첫 번째의 남은 시간을 물려받지 않도록 했습니다.
- **토스트가 클릭을 삼키지 않습니다.** `pointer-events:none` 을 주고 닫기 버튼만 `auto` 로 되돌렸습니다. 남아 있는 오류 알림도 아래에 있는 것을 막지 못합니다.

고친 뒤 같은 자리를 다시 재니 `그 자리에서 잡히는 것: button primary` 로 바뀌었고, 20초 뒤에는 토스트가 없습니다. 가져오기를 끝까지 돌려 `PPTX_IMPORT` 보고서가 제가 고친 문장을 담고 저장되는 것까지 확인했습니다.

두 가지를 적어 둡니다. **알림은 화면 위에 얹는 것이므로, 무엇을 덮는지가 알림 자신의 책임입니다.** 그리고 `docker build -q ... >/dev/null 2>&1` 로 빌드 실패를 삼킨 뒤 없는 태그로 컨테이너를 띄우려 한 일이 있었습니다. **조용히 만들면 실패도 조용합니다.**

3.10.54 v0.100.0에서 완료 — 사고 나는 날 처음 돌려 보는 코드

가져오기 화면을 더 파 보려다 두 가지를 의심했고, **둘 다 틀렸습니다.** 저장된 보고서의 주차가 스텁이 감지한 주차와 달랐는데, 파서가 덱에서 읽은 날짜가 모델이 말한 날짜보다 우선하기 때문이었습니다 — 맞는 순서입니다. 새 초안이 꺼진 채로 온 것도 `NEEDS_REVIEW` 상태여서였고, 화면이 그 이유를 이미 적어 두고 있었습니다. 추측을 코드로 확인하면 대개 이렇게 끝납니다.

그래서 한 번도 돌아 본 적 없는 것을 찾았습니다. `scripts/weekly-backup.sh` 는 **v0.15부터 모든 릴리즈 노트의 업그레이드 1단계**로 안내됩니다. 219줄이고, CI에도 테스트에도 없습니다. 복구 스크립트는 한 해 중 가장 나쁜 날에 처음 실행되는 종류의 소프트웨어입니다.

문서에 적힌 방식 그대로 돌렸습니다. 상태 볼륨을 붙인 배포를 새로 띄우고, 첨부 세 건을 올렸습니다 — 그중 둘은 바이트가 같아서 파일 하나로 합쳐집니다. 스크립트의 회계가 정확히 이 차이를 다루므로 꼭 필요한 조건이었습니다.

```
attachments: 2 files for 2 referenced paths — complete
attachment_rows=3
attachment_files_expected=2
```

백업·검증·복구가 모두 통과했고, 빈 데이터베이스와 빈 볼륨에 복구한 뒤 앱을 띄우니 `attachment files verified: 3` 이 뜨고 첨부 세 건이 모두 HTTP 200 으로 열렸으며 **바이트가 원본과 같았습니다.** 매니페스트는 비밀번호를 가립니다. 결함은 없었습니다.

없다는 것을 확인한 것으로 끝내면, 다음에 누가 이 스크립트를 고칠 때 다시 아무도 안 보는 상태로 돌아갑니다. `scripts/backup-check.sh` 를 만들어 CI에 넣었습니다. 임시 데이터베이스 둘을 만들고, 저장소의 마이그레이션을 psql 로 적용하고, 중복 업로드를 포함한 첨부를 심고, 백업 → 검증 → 복구를 왕복시킨 뒤 행 수와 **파일 바이트**를 비교합니다.

세 가지를 더 봅니다.

- **복구는 남은 파일을 지웁니다.** 백업에 없던 파일이 복구된 데이터인 척 남아 있으면 안 됩니다.
- **매니페스트가 비밀번호를 가립니다.** 복구하는 사람이 읽는 파일이고, 그 사람에게 자격 증명을 건네줄 이유는 없습니다.
- **빠진 첨부는 실패로 보고해야 합니다.** 이게 핵심입니다 — 깨진 복구 지점을 성공으로 적으면 백업이 없는 것보다 나쁩니다. 믿게 되기 때문입니다.

검사가 실제로 무는지 백업 스크립트를 세 가지로 훼손해서 확인했습니다.

훼손	검사가 한 말
복구 때 기존 파일 정리를 없앰	`restore left a stale file behind`
첨부 개수 비교를 항상 참으로	`backup reported success while an attachment file was missing`
상태 볼륨에서 첨부를 빼고 압축	`backup exited non-zero on a complete deployment`

셋 다 정확한 자리를 짚습니다.

원칙으로 남깁니다. **문서가 믿으라고 말하는 것은 CI가 돌려 봐야 합니다.** 릴리즈 노트 열 번이 넘게 "먼저 백업하고 verify 가 통과하는지 확인하십시오" 라고 적어 왔는데, 그 문장을 뒷받침하는 실행은 한 번도 없었습니다.

3.10.55 v0.101.0에서 완료 — 팀장은 하루 동안 호출이 없었다고 읽었습니다

앞선 회차에서 두 자리를 봤는데 둘 다 깨끗했습니다. 암호화 키를 잃은 배포는 기동을 거부하면서 볼륨 경로·환경변수·탈출구를 말하고, 틀린 키일 때는 **어떤 설정이** 안 풀리는지까지 말합니다. `crypto_keyloss_test.go` 가 이미 지키고 있었습니다. 프런트엔드의 되돌릴 수 없는 동작 열세 곳도 모두 확인을 거칩니다.

그래서 아직 안 해 본 각도로 옮겼습니다. 화면 순회는 v0.97에서 **한 사용자로만** 했습니다. 역할마다 보이는 화면이 다르므로 세 역할로 걸었습니다 — 작성자 8개, 팀장 12개, 관리자 13개. 33번의 방문 중 **한 곳**이 걸렸습니다.

팀장이 `#/analytics` 를 열면 `/api/v1/analytics/endpoints` 가 403 을 냅니다. 그 화면은 팀장에게 열려 있고, 안의 한 구획만 관리자 전용입니다.

문제는 그 다음이었습니다.

```
api<EndpointMetric[]>('/api/v1/analytics/endpoints').then(setEndpoints).catch(() => setEndpoints
```

403 이 빈 배열이 됩니다. 화면 설명은 "조직 제출 현황과 Weekly API 운영 상태를 함께 분석합니다" 라고 약속하고, `최근 24시간 API 분석` 표는 머리글만 남은 채 한 줄도 없이 그려집니다. 팀장이 읽는 것은 **하루 동안 이 서비스에 호출이 한 건도 없었다** 입니다. 그런 일은 일어나지 않았고, 화면만 봐서는 구별할 방법이 없습니다.

세 가지 서로 다른 답이 같은 그림이 되어 있었습니다 — 권한이 없다, 못 불러왔다, 기록이 없다. 각각 다르게 말하도록 고쳤습니다.

답	이제 보이는 것
권한 없음	구획을 아예 안 그리고, 화면 설명에서 API 약속을 뺍니다
못 불러옴	"API 분석을 불러오지 못했습니다. 잠시 후 다시 열어 보십시오."
기록 없음	"최근 24시간 동안 기록된 API 호출이 없습니다."
정상	표 (41행)

응답을 가로채 네 갈래를 모두 브라우저에서 확인했습니다.

그리고 요청 자체를 없앴습니다. 세션이 역할을 알고 있는데 거부될 것이 확실한 왕복을 할 이유가 없고, 그대로 두면 팀장이 그 화면을 열 때마다 콘솔에 403 이 남습니다. `catch` 는 남겨 뒀습니다 — 권한은 서버가 정하는 것이고, 이 변경은 답을 이미 아는 왕복 하나를 아낄 뿐입니다.

고친 뒤 33번의 방문이 모두 깨끗합니다 — 콘솔 오류도, 오류 알림도, 멈춘 로딩도 없습니다.

원칙으로 남깁니다. **한 사용자로 걸어 본 것은 한 역할로 걸어 본 것입니다.** 그리고 `catch(() => setState([]))` 는 실패를 데이터로 바꿉니다. 빈 결과가 의미 있는 답인 화면에서는, 그 한 줄이 거짓말이 됩니다.

3.10.56 v0.102.0에서 완료 — 실패를 데이터로 바꾸는 한 줄이 세 군데 더 있었습니다

v0.101에서 `catch(() => setEndpoints([]))` 하나를 고쳤습니다. 이런 줄은 대개 혼자 있지 않으므로 프런트엔드 전체를 훑었습니다. 실패를 빈 값이나 기본값으로 바꾸는 자리가 **열두 곳**이었습니다.

전부 고칠 일은 아닙니다. 선택 패널이 조용히 비는 것은 대개 옳고, 여기에 다 경고를 붙이면 v0.98에서 경계한 거짓 경고가 됩니다. 문제가 되는 것은 **빈 결과가 사용자가 행동에 옮길 주장인** 자리입니다. 그 기준으로 세 곳을 골랐습니다.

첨부. `AttachmentPanel` 은 실패를 빈 목록으로 바꿉니다. 읽기 전용으로 보는 사람에게는 `첨부한 이미지가 없습니다` 라고 단언하고, 작성자에게는 "끌어다 놓으세요" 안내를 보여 줍니다. 브라우저로 재 보니 **정상 빈 상태와 실패 상태가 글자까지 같았습니다.** 캡처는 서버에 그대로 있는데, 사람은 없어졌다고 읽습니다. 하필 백업 스크립트가 존재하는 이유가 바로 그 파일들입니다.

이제 이렇게 말합니다 — `첨부를 불러오지 못했습니다. 이 보고서의 이미지가 없어진 것은 아니며, 화면을 다시 열어 보십시오.` 잘못된 믿음을 직접 짚어야 합니다. "불러오지 못했습니다" 만으로는 사람이 최악을 상상합니다.

Import 이력. 실패가 `아직 Import 작업이 없습니다` 가 됩니다. 어제 올린 것을 찾으러 온 사람에게는 사라졌다는 말입니다.

인수인계 담당자. 실패하면 명단이 비고, 명단이 비면 담당자 선택 자체가 화면에서 사라집니다. 팀장은 인수인계할 사람이 없다고 읽습니다. 이제 왜 없는지 말합니다.

세 갈래를 모두 응답을 가로채 브라우저에서 확인했습니다. 정상 동작은 그대로입니다.

원칙으로 남깁니다. **빈 목록은 값이 아니라 문장입니다.** "없습니다" 라고 말할 자격은 물어봐서 없다는 답을 받았을 때만 생깁니다. 물어보지 못한 것과 없는 것은 다르고, 화면은 그 둘을 구별해서 말해야 합니다.

3.10.57 v0.103.0에서 완료 — 감사 로그가 조용히 비어도 아무도 모릅니다

프런트엔드에서 "실패를 데이터로 바꾸는" 유형을 두 번 고쳤으니, 서버에도 같은 것이 있는지 봤습니다. `_, _ = a.db.Exec(...)` 자리가 여럿인데 대부분은 의도된 최선노력 쓰기입니다 — `last_seen_at` 갱신이나 만료 세션 정리가 실패해도 사용자가 할 일은 없습니다.

하나가 달랐습니다.

```
_, _ = a.db.Exec(r.Context(), `INSERT INTO audit_logs(...)`)
```

감사 로그는 최선노력으로 써도 되는 값이 아닙니다. 관리자 화면에 `감사 로그` 탭이 있고, 사람들은 그것을 읽고 판단합니다.

실패를 강제해서 재현했습니다. 표를 잠시 치워 두고 관리자로 계정을 하나 만들었습니다.

```
사용자: 1 aadmin ADMIN / 2 newperson USER
감사 기록: 0건
새로 찍힌 로그: (없음)
```

계정은 만들어졌고, 기록은 없고, 로그는 한 줄도 없습니다. 감사 로그를 읽는 관리자는 빈 자리가 있다는 사실 자체를 알 수 없습니다.

두 가지를 고쳤습니다.

실패를 남깁니다. 여전히 최선노력입니다 — 이미 커밋된 동작을, 그 장부 기록이 실패했다는 이유로 되돌리는 것이 더 나쁩니다. 하지만 **아무도 볼 수 없는 구멍은 아무도 못 고칩니다.** 이제 `action` · `resource_type` · `resource_id` · `actor_id` · `trace` 를 담아 ERROR 로 남깁니다.

요청 컨텍스트에 태우지 않습니다. `audit` 는 응답을 쓰기 **전에** 호출되므로, 그 시점에 동작은 이미 커밋되어 있습니다. 클라이언트가 그 사이에 떠나면 `r.Context()` 가 취소되고, 배포는 변경을 간직한 채 그것을 누가 했는지에 대한 유일한 기록을 잃습니다. `context.WithoutCancel` 로 값(추적 ID)은 유지하고 취소만 떼어 냈습니다.

배경 작업용 `auditSystem` 도 같은 문제였습니다. 같이 고쳤습니다.

시험 둘을 붙였습니다. 하나는 표를 치우고 계정을 만들어 **동작은 성공하되 로그에 남는지**, 하나는 정상 배포에서 기록이 실제로 쓰이고 실패 로그가 없는지 봅니다. 고치기 전 코드에 돌리니 첫 번째가 `a lost audit record left no trace in the log` 로 떨어집니다.

시험 하네스도 손봤습니다. 로그를 `io.Discard` 로 버리고 있어서, **제품이 스스로 하는 일과 일부러 견디고 지나가는 실패는 시험에서 볼 방법이 없었습니다.** 이제 버퍼에 담고 `logged("...")` 로 확인합니다. 가드는 68개에서 70개가 되었습니다.

원칙으로 남깁니다. **최선노력으로 써도 되는 것과, 실패를 알려야 하는 것은 다릅니다.** `last_seen_at` 은 전자고 감사 로그는 후자입니다. 구별은 "이게 실패하면 누군가 할 일이 있는가" 로 갈립니다.

3.10.58 v0.104.0에서 완료 — 제출은 되는데 열 수 있는 사람이 없습니다

아직 안 해 본 각도로 걸었습니다. **아무 데이터도 없는 새 계정의 첫날.** 화면 여덟 개가 모두 구체적인 안내를 갖고 있었습니다 — "주간보고를 저장하면 업무가 자동으로 만들어집니다" 같은. 여기는 잘 만들어져 있습니다.

그런데 그 계정에 **소속 조직이 없었습니다.** 관리자가 사용자를 만들 때 조직을 안 넣으면 그렇게 됩니다. v0.98에서 본 것처럼 `PUT /admin/users/{id}` 는 빠진 필드를 지우므로, 평범한 관리자 편집 한 번으로도 이 상태가 됩니다.

무슨 일이 생기는지 실제 배포에서 쟀습니다. 조직 없는 작성자를 만들고, 보고서를 쓰고, 제출했습니다.

```
제출: {"status":"SUBMITTED"}
팀장의 검토 대기 목록: 0 건
관리자의 목록: 1 건 ['orphan']
팀장이 보는 팀 목록(상태 무관): ['writer']
그 사람이 받은 안내: ''
```

제출은 성공했고, 열 수 있는 검토자가 없습니다. 관리자가 전체 목록을 뒤지지 않는 한 그 보고서는 검토 대기로 남습니다. 그리고 그 사람은 아무 말도 듣지 못한 채 기다립니다.

`accountNotice` 가 이미 있습니다. 조직 없는 **팀장**에게 화면들이 왜 비어 보이는지 알려 주는 장치입니다. 그런데 시험이 일반 사용자는 경고하지 않는다고 명시적으로 단언하고 있었고, 근거가 주석에 적혀 있었습니

다.

> A plain user's own reports do not depend on an organisation, and the org-scoped screens are not theirs to open.

뒷문장은 맞습니다. **앞문장이 사실이 아닙니다.** 자기 보고서는 조직에 의존합니다 — 쓰는 데는 아니고, **읽히는 데** 의존합니다. 위 측정이 그것입니다.

판단을 뒤집은 것이 아니라 전제를 고쳤습니다. 시험 이름도 `TestAnAccountThatCannotBeReachedIsToldWhy` 로 바꾸고, 왜 바뀌었는지 주석에 측정값과 함께 남겼습니다.

이제 조직 없는 작성자는 결재가 켜져 있으면 "제출해도 검토 대기 상태로 남습니다" 를, 꺼져 있으면 "어느 팀장의 팀 주간보고에도 나타나지 않습니다" 를 듣습니다. 관리자는 예외입니다 — 어차피 전부 보이니 잘못된 것이 없습니다.

원칙으로 남깁니다. **주석에 적힌 근거도 측정 대상입니다.** 왜 그렇게 했는지 적어 둔 자리는 시험할 자리라고 v0.93에서 적었는데, 이번에는 그 근거 자체가 틀린 경우였습니다.

3.10.59 v0.105.0에서 완료 — "한 번 확인해 보십시오" 라고 적었는데 확인할 방법이 없었습니다

v0.105 릴리즈 노트에 운영자용 문단을 이렇게 썼습니다. *"이미 운영 중인 곳에서는 소속 조직이 비어 있는 계정이 있는지 한 번 확인해 보시기를 권합니다."*

그 다음 회차에서 확인할 방법이 있는지 봤습니다. 관리자 사용자 목록에 `조직` 열은 있습니다. 그런데 목록은 v0.86에서 **한 페이지 100명**으로 상한이 걸렸고, 검색은 `q` (아이디·이름·이메일)와 `role` 뿐입니다. **빈 칸으로 거를 방법이 없습니다.** 2,100명짜리 배포에서는 스물한 페이지를 눈으로 훑어야 합니다.

제가 만든 숙제인데 풀 도구를 안 준 셈입니다.

두 가지를 넣었습니다.

개수를 말합니다. 응답에 `unassigned` 를 넣었습니다. 검색 조건과 무관한 전체 개수입니다 — 그 상태는 이 페이지의 성질이 아니라 배포의 성질이기 때문입니다. 사용자 목록 위에 뜹니다. *"소속 조직이 지정되지 않은 계정이 N명 있습니다. 이 계정의 주간보고는 어느 팀장에게도 보이지 않아, 제출해도 검토 대기 상태로 남습니다."*

거를 수 있게 했습니다. `organization=none` 이면 그 계정만 반환합니다. 경고 옆의 `해당 계정만 보기` 로 바로 갑니다.

인식하지 못한 값은 **무시합니다.** `organization=nome` 같은 오타가 목록을 조용히 비우면 관리자는 배포에 계정이 없다고 읽습니다. v0.101·v0.102에서 반복해 본 것과 같은 실패라, 시험으로 못 박았습니다.

시험 둘을 붙이고 서버를 두 가지로 훼손해 확인했습니다 — 필터를 안 걸게 하면 첫 번째가, 오타를 필터로 받아들이게 하면 두 번째가 떨어집니다. 가드는 70개에서 72개가 되었습니다.

도구 함정 하나. 하네스의 `lastCreatedUsername(stem)` 은 `LIKE stem || '_%'` 로 찾는데, **SQL LIKE 에서 `_` 는 한 글자 와일드카드입니다.** `adrift` 로 찾으면 `adriftlead_...` 도 걸립니다. 마침 호출 순서 덕에 맞는 답이 나왔지만 우연이었습니다. 겹치지 않는 이름으로 바꿨습니다.

원칙으로 남깁니다. **문서가 하라고 시킨 일은 할 수 있어야 합니다.** 상한을 두면 찾을 방법을 같이 줘야 한다고 v0.86에서 적었는데, 이번에는 제가 만든 새 상태에 대해 같은 빚을 졌습니다.

3.10.60 v0.106.0에서 완료 — 물려받은 배포가 어느 모드인지 알 방법

v0.105 에서 "문서가 하라고 시킨 일은 할 수 있어야 한다" 를 적었으니, 그것이 한 건인지 유형인지 확인했습니다. README·릴리즈 노트가 운영자에게 시키는 문장들을 훑었고, 하나가 걸렸습니다.

> `/var/lib/weekly` 볼륨과 `WEEKLY_ENCRYPTION_KEY` 를 각각 백업하십시오. 환경 키를 설정하지 않은 하위 호환 모드에서는 볼륨의 `instance.key` 가 유일한 복호화 키이며…

이 문장은 **자기 배포가 어느 모드인지 안다**는 것을 전제합니다. 확인해 보니 제품은 그것을 알고 있었고, 한 군데에서만 말했습니다.

```
{"level":"INFO","msg":"secret encryption initialized","key_source":"state_volume","stored_secret
```

기동 로그 한 줄입니다. 관리자 API 셋을 뒤져도 이 상태를 내보내는 곳이 없었습니다. 배포를 물려받은 사람이 몇 달 뒤에 묻는 질문인데, 그때 그 줄은 없습니다. 그리고 `weekly-backup.sh` 의 복구 안내는 "관리자 비밀 설정이 안전하게 설정됨으로 보이는지" 확인하라고 하는데, 그 표시는 값이 **있다**는 뜻이지 키를 **복구할 수 있다**는 뜻이 아닙니다.

`GET /api/v1/admin/encryption` 을 추가했습니다. 기존 설정 응답은 배열이라 모양을 바꾸면 계약이 깨지므로 따로 뒀습니다.

```
{"keySource":"state_volume","storedSecrets":1,"recoverable":false,"stateDirectory":"/var/lib/wee
```

`storedSecrets` 는 **요청 시점에 셉니다.** 오늘 오후에 넣은 키가 바로 지금 위험한 것이라, 기동 시점 값을 들고 있으면 답이 틀립니다.

관리자 · 서비스 설정 맨 위에 세 가지로 나뉘어 뜹니다.

상태	보이는 것
환경변수 키	안내 — 볼륨을 잃어도 비밀 설정 N개를 복구할 수 있습니다
볼륨 키 · 비밀 없음	안내 — 아직 저장된 비밀 설정이 없어 지금은 잃을 것이 없습니다
볼륨 키 · 비밀 있음	**경고** — 이 볼륨을 잃으면 저장된 비밀 설정 N개를 복구할 수 없습니다

세 상태를 실제 배포 셋으로 확인했습니다. 같은 배포에 비밀을 하나 넣자 `storedSecrets` 가 0에서 1로, `recoverable` 이 true 에서 false 로 바뀌고 안내가 경고로 바뀝니다.

시험 둘을 붙였습니다 — 비밀을 저장하면 위험 판정이 뒤집히는지, 그리고 작성자가 배포의 키 모드를 읽지 못하는지. 서버를 두 가지로 훼손해 둘 다 무는 것을 확인했습니다. 가드 72 → 74.

원칙으로 남깁니다. **기동 로그는 지금 있는 사람에게만 하는 말입니다.** 나중에 물어볼 질문의 답은 나중에 볼 수 있는 곳에 있어야 합니다.

3.10.61 v0.106.1에서 완료 — 도구가 말한 것을 제가 읽고 넘어갔습니다

v0.106.0 의 CI 가 떨어졌습니다. `guard-check` 가 제 시험 하나를 잡았습니다.

```
!  TestTheEncryptionModeIsNotReadableWithoutBeingAnAdministrator — never executed adminEncryptio
A test that never runs its subject passes whatever the subject does.
```

맞는 지적입니다. 그 시험은 작성자가 403 을 받는지 보는데, 403 은 `requireRole("ADMIN")` 이 **핸들러에 닿기 전에** 냅니다. 그러니 `adminEncryption` 은 한 줄도 실행되지 않습니다. 표식을 뗐습니다 — 시험 자체는 남습니다. 그 엔드포인트가 관리자 전용이라는 것은 여전히 지켜야 할 규칙이고, 다만 그것을 지키는 것은 라우터의 문지기이지 핸들러가 아닙니다.

더 볼 것은 **왜 로컬에서 못 잡았는가** 입니다. 올리기 전에 `guard-check` 를 돌렸고 이렇게 나왔습니다.

```
40 guard(s) skipped and therefore unverified: ...
34 guard(s) reached what they name.
```

도구는 말했습니다. 제가 마지막 줄만 보고 넘어갔습니다. 이유는 단순합니다 — **Bash 호출 사이에 환경변수가 유지되지 않습니다.** 앞 호출에서 `export WEEKLY_TEST_POSTGRES_DSN` 을 했고 다음 호출에서는 사라져 있었으며, 데이터베이스가 필요한 가드 40개가 조용히 건너뛰어졌습니다. 새로 넣은 시험이 정확히 그 40개 안에 있었습니다.

마지막 줄을 고쳤습니다. 사람이 읽는 것은 마지막 줄이기 때문입니다.

```
34 guard(s) reached what they name — 39 were NOT checked. Set WEEKLY_TEST_POSTGRES_DSN to check
```

전부 검사했을 때만 예전처럼 짧게 끝납니다.

원칙으로 남깁니다. **"검증되지 않음" 은 각주가 아니라 조치할 일입니다.** 그리고 요약의 마지막 줄은 그 위에 무엇이 적혀 있든 혼자 읽힐 각오를 하고 써야 합니다.

3.10.62 v0.107.0에서 완료 — 알림은 닫히고 화면은 남습니다

제품이 알고 있으면서 로그에만 말하는 것이 더 있는지 `conditions` 를 봤습니다. 둘뿐이었고 **둘 다 이미 잘 되어 있었습니다.** 보고서 목록 잘림은 응답에 `total` 이 실려 화면이 "N건 중 M건" 을 말할 수 있고, 업무 그래프 잘림은 `Capped` 컴포넌트가 화면에 표시합니다. v0.86 의 "찾을 방법 없는 상한은 무제한보다 나쁘다" 가 지켜지고 있습니다.

대신 그것을 보다가 `InsightsPage` 에서 다른 것을 봤습니다.

```
.catch(error => {
  setView({ weeks, since: '', workItems: 0, similar: [], similarTotal: 0, duplicates: [], ... })
  notify(errorText(error, '업무 인사이트를 불러올 수 없습니다.'), 'error')
})
```

v0.102 와 같은 유형인데 한 가지가 다릅니다 — **여기는 알림을 띄웁니다.** 그래서 v0.102 때는 손대지 않았습니다. 그런데 재 보니 그것으로 충분하지 않았습니다.

```
정상:     화면이 단언하는 것: ['조직 간 중복으로 의심되는 업무가 없습니다.']
서버 오류: 화면이 단언하는 것: ['조직 간 중복으로 의심되는 업무가 없습니다.']
```

글자까지 같습니다. 다른 것은 사용자가 닫을 수 있는 알림 하나뿐입니다. 알림을 닫으면, 또는 다른 알림이 덮으면, 화면에는 다섯 개의 단언만 남습니다 — 중복 없음, 유사 없음, 협업 없음, 반복 없음, 병목 없음. 조직을 가로질러 같은 일이 벌어지고 있는지 보러 온 사람에게 **없다고 다섯 번 말합니다.**

실패를 채워 넣지 않고 실패로 두었습니다. 다섯 탭 모두 그 탭 이름을 넣어 말합니다.

> 업무 인사이트를 불러오지 못했습니다. '협업 지도' 항목이 비어 있다는 뜻은 아니며, 화면을 다시 열어 보십시오.

원칙으로 남깁니다. **알림은 닫히고 화면은 남습니다.** 실패를 알리는 것과 화면이 거짓을 말하지 않는 것은 다른 일이고, 앞의 것이 뒤의 것을 대신하지 못합니다. v0.102 에서 "알림이 있으니 됐다" 고 넘긴 판단을 이번 측정이 뒤집었습니다.

도구 함정도 하나 더 적어 둡니다. `cd ..` 로 `frontend/` 에 있는 채로 `docker build` 를 돌려 실패했는데, 그 다음 줄이 그대로 실행되어 **옛 이미지로 컨테이너가 떴고 옛 문구가 그대로 나왔습니다.** v0.99 에서 `-q` 로 빌드 실패를 삼킨 것과 같은 모양입니다. 이번에는 오류가 보였는데도 뒷줄이 멈추지 않았습니다.

3.10.63 v0.108.0에서 완료 — 이미 등록된 선행 업무를 또 등록하라고 권합니다

같은 유형을 눈이 아니라 목록으로 훑었습니다. 실패를 값으로 바꾸는 자리가 열네 곳 남아 있었고, 그중 **행동에 옮길 주장**을 하는 것을 골랐습니다. `IssueDependency` 였습니다.

선행 업무를 실제로 하나 등록해 두고 그 요청만 실패시켜 나란히 봤습니다.

```
정상:     가져오기 검토 화면 확인 · 플랫폼팀을 기다리는 중으로 등록돼 있습니다. [고치기]
서버 오류: 이 이슈가 다른 업무가 끝나야 풀리는 것이라면 선행 업무로 등록해 두십시오. [선행 업무 등록]
```

이미 등록된 것을 또 등록하라고 권합니다. 그대로 따르면 중복이 생기고, 보고서를 읽는 팀장에게는 이 이슈가 무엇을 기다리는지 아무 표시도 남지 않습니다.

고치는 방법은 그 컴포넌트 자신이 이미 적어 두고 있었습니다.

> Nothing is said until the answer is known. A prompt that flashes "waiting on nothing" while the request is in flight is worse than a moment of quiet.

로딩 중에는 그 규칙을 지키는데, **실패했을 때는 빈 답을 지어내고 있었습니다.** 실패도 "답을 모르는 상태" 입니다. 빈 뷰를 만들지 않고 그대로 두었습니다.

`DependencyPanel` 도 같은 자리였습니다. 이쪽은 알림을 띄우지만 v0.107 에서 배운 대로 그것으로는 부족합니다 — 화면은 `등록된 선행·후행 관계가 없습니다` 를 계속 말하고, 알림은 닫히면 사라집니다. 같이 고쳤습니다.

원칙으로 남깁니다. **로딩 중에 지키는 규칙은 실패했을 때도 지켜야 합니다.** 둘 다 "아직 모른다" 이고, 모를 때 하는 말은 같아야 합니다.

3.10.64 v0.109.0에서 완료 — 검사를 만들지 않기로 한 판단, 그리고 한 줄이 두 개여야 했던 이유

여섯 곳을 손으로 고쳤으니 정적 검사로 못 박을지 봤습니다. **만들지 않기로 했습니다.**

남은 자리를 늘어놓고 보니 `catch(() => setX([]))` 라는 모양만으로는 판단할 수 없습니다. `HandoverPage` 의 빈 명단은 v0.102 에서 의도적으로 알림과 함께 남겼고, 빠른 이동의 검색 결과가 비는 것은 아무 해가 없습니다. **어떤 빈 값이 단언인지는 `catch` 가 아니라 렌더링에 있습니다.** 모양만 보고 전부 잡는 검사는 v0.98 에서 경계한 "대개 틀리는 경고" 가 되고, 그런 경고는 사람들이 눌러 넘기는 법을 배웁니다.

대신 남은 것 중 실제로 문제인 하나를 봤습니다. 보고서 편집기의 **지난주 계획**과 **결정에 딸린 후속 조치** 입니다. 둘 다 내용이 있을 때만 그려지므로 실패해도 거짓을 말하지는 않습니다 — 그냥 **사라집니다.** 이어받을 것이 있었는지 없었는지 작성자는 알 수 없고, 주차를 넘어가던 일이 조용히 떨어집니다.

한 줄로 알리게 했습니다. 그런데 처음 구현은 **틀렸고 브라우저가 그것을 말해 줬습니다.**

```
지난주 실패     안내="지난주 계획과 후속 조치를 불러오지 못했습니다..."
후속 조치 실패  안내="(없음)"
```

두 요청이 표시 하나를 공유하고 있었습니다. 나중에 끝나는 쪽이 답을 정하고, 지난주 계획이 성공하면 방금 기록된 후속 조치 실패를 지워 버립니다. 각자 자기 것만 관리하도록 나누고, 무엇이 없는지에 따라 문장도 달라지게 했습니다.

도구 함정이 **세 번째로** 같은 모양이었습니다. `cd ..` 로 `frontend/` 에 있는 채 `docker build .` 을 돌려 실패했고, 그 다음 줄이 멈추지 않아 옛 이미지가 떴습니다. 세 번 겪었으면 습관을 바꿔야 합니다 — 이제 `docker build -t 이름 /절대/경로` 로 씁니다.

원칙으로 남깁니다. **검사로 잡을 수 없는 것은 검사로 잡는 척하지 않습니다.** 잘못 짚는 검사는 없는 것보다 나쁩니다.

3.10.65 v0.110.0에서 완료 — 규모 검사가 관리자만 볼 줄 압니다

렌즈를 바꿔 제품이 만들어 내보내는 것을 봤습니다. PPTX 덱은 빈 슬라이드 없이 업무 2장에 이슈 1장이고, 등록해 둔 선행 업무는 약속대로 회의 안건과 병목 분석이 읽습니다. 같은 조직 안의 의존이 `타 조직 대기` 에 오르지 않는 것도 설계대로입니다.

그 과정에서 **제 측정이 틀린 것을 잡았습니다.** 회의 안건이 전 구획 0건으로 나와 결함이라고 확신했는데, JSON 필드가 `items` 가 아니라 `entries` 였습니다. 실제로는 신규 이슈 2건·변경점 2건으로 정상입니다. 필드 이름 하나 때문에 없는 결함을 고칠 뻔했습니다.

그다음 `scale-check.py` 를 다시 돌렸습니다. v0.86 이후 스무 개 넘는 릴리즈가 나갔고 그 코드를 이 검사는 한 번도 본 적이 없습니다. **302명 · 41조직 · 15,594보고서 · 109,175항목** 배포에서 **지적 사항 없음** — 규모 성질이 유지되고 있습니다. 가장 큰 응답이 423KB·769ms 입니다.

그런데 **팀장으로 돌리자 지적 5건이 나왔습니다.**

```
403  /api/v1/admin/analytics/keywords  <== 403
403  /api/v1/admin/users               <== 403
...
지적 사항 5건
```

다섯 건 모두 관리자 전용 경로가 팀장을 **정상적으로 거부한 것**입니다. 도구가 역할을 모르니 200이 아닌 것을 전부 결함으로 셉니다. 그리고 v0.101 의 진짜 결함을 찾아낸 방법이 정확히 **역할별로 걸어 보는 것**이었습니다. 그 방법을 쓰려는 사람에게 도구가 거짓 지적 다섯 건을 먼저 내밀면, 그 사람은 도구를 그만 읽습니다.

로그인 뒤 `/api/v1/me` 로 역할을 물어보고, 그 역할이 열 수 없는 경로는 건너뜁니다. `--role` 같은 깃발은 하나 더 틀릴 거리라 두지 않았습니다. 건너뛴 것은 **이름까지 밝힙니다** — v0.106.1 에서 "검증되지 않음은 각주가 아니라 조치할 일" 을 배웠으므로, 절반만 본 실행이 전부 본 실행처럼 읽히면 안 됩니다.

```
확인 17개 경로, 지적 사항 없음 — 5개는 TEAM_LEADER 권한 밖이라 건너뜀: ...
확인 6개 경로,  지적 사항 없음 — 16개는 USER 권한 밖이라 건너뜀: ...
```

분류는 서버에 직접 물어 확인했습니다. 일반 작성자에게 여덟 경로가 모두 실제로 403 입니다 — 제가 방금 쓴 목록을 제가 믿지 않고 잰 것입니다.

검색에 대해 하나 기록해 둡니다. **결과가 적은 질의가 4배 느립니다** (0.68초 대 0.16초). 정확 검색이 적게 찾으면 근사 검색이 도는데, 그것은 109,175행에 대해 컬럼 여섯 개의 `word_similarity` 를 두 번 계산하는 전수 스캔입니다. 색인이 못 탑니다. **의도된 절충이고 이 규모에서는 감당 범위라 건드리지 않았습니다** — 순위 로직은 v0.86 에서 실제 말뭉치로 조정한 것이고, 성능을 위해 그것을 다시 여는 것은 지금 근거가 없습니다. 데이터가 열 배가 되면 다시 볼 자리입니다.

3.10.66 v0.111.0에서 완료 — 로그인하면 바로 나던 예외

지금까지 브라우저 확인은 전부 **데이터가 거의 없는 배포**에서 했습니다. 302명 · 41조직 · 15,594보고서 · 109,175항목 배포가 떠 있으니, 거기서 세 역할로 화면 서른셋을 걸었습니다.

한 줄이 세 역할 모두에서 나왔습니다.

```
#/current  예외: Confluence 설정을 읽을 수 없습니다.
```

`Promise.all([load(), loadCandidates(), loadPrevious(), loadFollowUps()])` 에서 `loadCandidates` 만 `.catch` 가 없었습니다. Confluence 는 선택 기능이고 쓰지 않는 배포는 이 호출에 **매번** 설정 오류를 돌려주므로, 사람들이 가장 많이 쓰는 화면에서 방문마다 잡히지 않은 예외가 났습니다.

그것을 고치다 더 나쁜 것을 봤습니다. 응답을 실패시켜 보니

```
현재 보고서 실패: 로딩 표시가 남아 있음 · 편집기 없음 · 알림 없음 · 예외 2건
```

영원히 로딩 표시에 멈춥니다. `report` 는 답이 올 때까지 `undefined` 이고 그 동안 화면은 스피너를 그리는데, 실패하면 영영 `undefined` 로 남습니다. 편집기도 없고 안내도 없고 다시 시도할 방법도 없습니다.

같은 모양이 **대시보드**에도 있었습니다. 로그인하면 바로 열리는 화면입니다.

셋 다 고쳤습니다.

- `loadCandidates` 는 자기 실패를 처리합니다. 후보 구획은 후보가 있을 때만 그려지므로 조용히 실패하면 아무것도 보이지 않고, 연동을 설정한 운영자는 관리자 화면에서 동기화 상태를 봅니다.
- 편집기와 대시보드 모두 불러오기 실패를 상태로 두고 무슨 일이 있었는지 말합니다. 편집기는 **"작성한 내용이 사라진 것은 아니며"** 를 함께 말합니다 — 그 화면에서 사람이 가장 먼저 겁내는 것이 그것이기 때문입니다.
- 세션이 `confluenceEnabled` 를 싣습니다. `workflowEnabled`·`aiEnabled` 가 이미 같은 이유로 있습니다. **거부될 것이 확실한 요청은 보내지 않습니다** — v0.101 에서 분석 화면에 쓴 것과 같은 원칙입니다.

확인 중에 하나를 배웠습니다. 연동을 끄기 전까지 콘솔 기록이 남아 있었는데, 세션을 보니 `confluenceEnabled: true` 였습니다. 이 배포는 Confluence 가 **켜져 있는데 설정이 깨진** 상태였습니다 (비밀 재설정으로 비밀번호가 지워짐). 플래그는 정직했고 제 게이트도 맞았습니다. 실제로 꺼 보니 **서른세 화면이 전부 깨끗합니다** — 콘솔 오류도, 예외도, 멈춘 로딩도 없습니다.

원칙으로 남깁니다. **작은 배포에서 깨끗한 것과 실제 배포에서 깨끗한 것은 다릅니다.** 이 세 결함은 전부 첫 화면과 주 화면에 있었는데, 아홉 개짜리 개발 데이터베이스에서는 한 번도 보이지 않았습니다 — 그쪽에는 Confluence 설정이 아예 없어 오류가 다르게 났기 때문입니다.

3.10.67 v0.112.0에서 완료 — 사람이 회의에서 띄우는 파일을 아무도 재지 않았습니다

규모 배포에서 **실제 작업 흐름**을 끝까지 걸었습니다. 15,594건이 쌓인 곳에서 처음 하는 일입니다. 작성자가 쓰고 제출하면 `검토 대기` 가 되고, 같은 팀 팀장의 팀 화면(468건 중 200건, `더 보기` 있음)에서 그 보고서를 찾아 승인하면 `승인했습니다` 가 뜹니다. **결함 없습니다.**

팀 화면의 200행이 상한으로 보여 확인했더니 응답이 `items 200 · total 468 · limit 200` 으로 정직하고, 화면도 468을 말하며 `더 보기` 를 줍니다. v0.86 의 규칙이 지켜지고 있습니다.

남은 것 하나가 실제 공백이었습니다. **규모 검사가 CSV 내보내기는 재는데 PPTX 내보내기는 안 잽니다.** 덱은 사람이 회의에서 일어나 띄우는 파일입니다.

재 보니 건전했습니다. 연간 덱이 50KB·24ms, 슬라이드 17장이고 손상이 없으며, **표지에 "주간보고 315건 취합", 모든 업무 슬라이드에 "주간보고 업무 2205건을 중복 제거해 63건으로 정리했습니다", 쪽 번호까지** 있습니다. 무엇을 어떻게 줄였는지 덱이 스스로 말합니다.

세 번 뽑아 SHA-256 이 같습니다. v0.54 의 결정성 점검이 지켜지고 있습니다.

그래서 검사에 넣었습니다. 넣으면서 **비교 방식의 결함**을 하나 고쳤습니다. `normalise` 는 JSON 이 아닌 응답을 `decode("utf-8", "replace")` 로 문자열화해서 비교했는데, PPTX 는 이진이라 읽지 못한 바이트가 **전부 같은 대체 문자**가 됩니다. 덱 한가운데 한 바이트를 뒤집어 확인했습니다.

```
옛 방식(손실 디코딩)으로 구별: 아니오 — 다른 덱이 같아 보임
새 방식(해시)로 구별: 예
```

이론이 아니라 실제로 통과시킬 수 있었던 변화입니다. 이진 응답은 이제 SHA-256 으로 비교합니다.

도구 함정을 하나 더 겪었습니다. `section input[type="text"]` 로 제목 칸을 세었더니 0개였습니다. `type` **속성이 없는 입력은 그 선택자에 걸리지 않습니다** — 이 세션에서 이미 적어 둔 함정인데 또 밟았습니다. 적어 두는 것과 몸에 붙는 것은 다릅니다.

3.10.68 v0.113.0에서 완료 — 에이전트가 들어오는 문에는 시험이 둘뿐이었습니다

이 제품에는 MCP 인터페이스가 있습니다. 문서가 있고 분석 화면에 `MCP 제공` 배지도 있는데, 시험은 **인자 파싱 두 개**뿐이었습니다. 도구 넷이 노출되는데 실제로 호출해 본 시험이 없습니다.

규모 배포에 대고 직접 걸었습니다. 네 도구 모두 동작하고, 검색은 이런 것을 붙여 돌려줍니다.

> 조건에 맞는 15594건 중 1번째부터 5건만 반환했습니다. 나머지는 offset=5으로 이어서 가져오세요. **이 목록을 전체로 보고 요약하지 마세요.**

상한을 사람이 아니라 **모델에게** 알리는 문장입니다. 잘 만들어져 있습니다.

권한도 확인했습니다. `weekly_endpoint_analysis` 는 REST 에서 관리자 전용인데(v0.101), MCP 라는 두 번째 문에서도 같습니다 — 팀장에게는 `isError: true, 관리자 권한이 필요합니다` 이고, 제출 현황은 302명이 아니라 자기 조직 9명으로 좁혀집니다. 개인 API 키로도 같고, 키로 쓰기를 시도하면 `개인 API 키는 조회 전용입니다` 로 막힙니다. **팀장 키의 `tools/list` 는 도구를 셋만 돌려줍니다** — 쓸 수 없는 도구를 목록에서 아예 빼는 쪽이 부르고 나서 거절하는 것보다 낫습니다. 에이전트는 목록을 "할 수 있는 일" 로 읽기 때문입니다.

결함은 없었고, 지키는 것도 없었습니다. 방금 손으로 확인한 성질이 전부 시험 밖에 있었습니다. 넷을 붙였습니다.

`mcp.go` 의 역할 관문 세 곳을 하나씩 훼손해 확인했습니다.

훼손한 자리	잡은 시험
157행 · 도구 목록	`TestMCPOffersOnlyTheToolsTheCallerCanUse`
222행 · 관리자 도구 호출	`TestMCPRefusesAnAdministratorToolAndSaysWhy`
310행 · 검색 범위	`TestMCPSearchReturnsOnlyTheCallersOwnOrganisation`

세 번째는 처음에 **안 잡혔습니다.** 시험 셋을 먼저 쓰고 훼손해 보니 310행만 통과했고, 그 자리를 읽어 보니 `weekly_reports_search` 의 조직 범위였습니다. 훼손하면 팀장이 서비스 전체 보고서를 봅니다. 그래서 네 번째 시험을 썼습니다.

원칙으로 남깁니다. **시험을 쓴 뒤에 훼손해 보지 않으면, 무엇을 안 썼는지 알 수 없습니다.** 세 개를 쓰고 만족했으면 가장 심각한 자리가 그대로 비어 있었을 것입니다. 가드는 74개에서 78개가 되었습니다.

3.10.69 v0.114.0에서 완료 — 모든 화면이 지나가는 층에 시험이 없었습니다

v0.113 에서 배운 것을 넓히려고 `mutation-check.py` 를 다시 돌렸습니다. v0.82 에서 "아무도 못 잡는 변이 64건" 이었고 그 뒤로 시험이 많이 늘었으니 다시 잴 값이 있습니다. 오래 걸리므로 배경에 두고, 그동안 **Go 시험 하네스를 건드리지 않는 자리**를 봤습니다.

`api.ts` 였습니다. 화면 전부가 이 층을 지나가는데 시험이 **하나도** 없습니다. 프런트엔드 시험 다섯 파일은 날짜·슬라이드·라우팅·저장 가드를 덮고 있고, 그 아래 요청 층은 비어 있었습니다.

여기에는 시험할 값이 있는 판단이 여럿 있습니다.

- **FormData 에는 Content-Type 을 붙이지 않습니다.** multipart 는 경계 문자열이 필요하고 그것은 브라우저가 씁니다. 누가 헤더 로직을 "정리" 하면서 이 조건을 지우면 **첨부와 PPTX 업로드가 전부** 엉뚱한 오류로 실패합니다.

- **봉투가 `success:false` 면 200 이어도 실패입니다.** 상태 줄이 아니라 봉투가 계약입니다.
- **401 은 던지기 전에 알립니다.** `.catch` 를 잊은 화면이 정확히 영원히 도는 스피너가 되는 자리이고, v0.111 에서 실제로 그것을 고쳤습니다.
- **추적 ID 는 5xx 에만 붙습니다.** 틀린 비밀번호에 추적 ID 를 붙여 봐야, 그것을 읽는 사람은 찾아볼 수 있는 사람이 아닙니다.

열다섯 개를 붙이고 다섯 가지로 훼손해 전부 잡히는 것을 확인했습니다 — FormData 조건 제거, 401 알림 제거, 봉투 검사 제거, 추적 ID 조건 완화, 쿠키 전송 제거. 프런트엔드 시험은 여섯 파일 52개가 되었습니다.

작업 순서에서 하나 배웠습니다. 변이 검사는 **소스를 제자리에서 고칩니다.** 도중에 `git status` 를 보니 `internal/app/auth.go` 가 수정 상태였습니다 — 그때 `git add -A` 를 했으면 변이가 커밋에 실려 갔을 것입니다. 이번 릴리즈는 경로를 명시해 커밋했고, Go 시험도 그동안 돌리지 않았습니다.

도구 함정도 또 나왔습니다. 훼손 대상 문자열에 작은따옴표가 들어가자 쉘에 박은 파이썬이 깨졌고, **적용되지 않은 변이가 "통과" 로 보였습니다.** 이 세션에서 세 번째로 같은 모양입니다. 이제 훼손 스크립트는 파일로 씁니다.

3.10.70 v0.115.0에서 완료 — 살아남은 변이가 다 구멍은 아닙니다

변이 검사가 아직 돌고 있어 Go 는 계속 건드리지 않고, 시험 없는 순수 모듈을 마저 봤습니다. `layers.ts` 였습니다. 주석이 왜 있는지 적어 두고 있습니다.

> One keypress therefore closed every open layer at once: leaving a deck also threw away the report the reviewer was reading underneath it.

이번 창 내내 고쳐 온 것과 같은 종류의 손해인데 — 사람이 보고 있던 것이 사라지는 것 — 시험이 없었습니다. 일곱 개를 붙였습니다. 겹쳐 열린 것 중 마지막만 맨 위이고, 위를 닫으면 아래가 맨 위가 되고, **가운데 것이 먼저 닫혀도 맨 위는 그대로**이며, 같은 층을 두 번 닫아도 아래를 건드리지 않습니다.

쓰다가 제 시험의 결함을 하나 잡았습니다. 스택을 비우는 `beforeEach` 를 넣었는데, 그 안에서 `isTopLayer(probe)` 로 "혼자인가" 를 물었습니다. **방금 넣은 것은 항상 맨 위입니다.** 그래서 그 조건은 언제나 참이고 첫 바퀴에 빠져나가며, 아무것도 비우지 않습니다. 격리하는 척하는 설정은 이번 창 내내 고쳐 온 그 유형이라 없앴습니다.

훼손해 확인했습니다. 맨 아래를 보게 하기, 닫을 때 전부 비우기, 모든 층에 같은 표식 주기 — 셋 다 잡힙니다. 네 번째는 **살아남았고, 그것이 옳습니다.** `isTopLayer` 의 `stack.length > 0 &&` 를 지워도 빈 배열의 `stack[-1]` 은 `undefined` 라 어떤 심볼과도 같지 않습니다. 동등한 변이이고, 답은 죽은 코드이지 빠진 시험이 아닙니다. 그 자리에 주석으로 적어 뒀습니다.

`session.ts` 는 따로 붙이지 않았습니다. 함수 둘뿐이고 `api.test.ts` 의 401 시험이 실제로 그것을 통과합니다. **간접적으로 실행되는 것과 지켜지는 것은 다르지만**, 여기서는 그 401 시험이 잃으면 안 되는 규칙을 이미 못 박고 있습니다.

프런트엔드 시험은 일곱 파일 59개가 되었습니다.

원칙으로 남깁니다. **살아남은 변이를 볼 때 먼저 물을 것은 "시험이 없나" 가 아니라 "이 변이가 정말 다른 동작인가" 입니다.** 동등한 변이를 잡겠다고 시험을 늘리면, 지키는 것 없이 읽을 것만 늘어납니다.

3.10.71 v0.116.0에서 완료 — 제가 매번 손으로 하던 일

변이 검사가 Go 파일을 제자리에서 고치는 중이라 계속 Go 를 피했습니다. 그래서 **제가 매 릴리즈마다 손으로 하는 일**을 봤습니다. 버전이 일곱 곳에 흩어져 있습니다 — `VERSION`, 프런트엔드 패키지, OpenAPI, 배포 파일 셋, README 의 `docker load` 줄. 매번 `sed` 여섯 줄로 맞추고 있었고, 하나 빠뜨려도 여기서는 아무 일도 안 일어납니다. 드러나는 곳은 배포하는 쪽입니다.

`scripts/version-check.sh` 를 만들어 CI 에 넣었습니다. 릴리즈 노트가 있는지도 함께 봅니다 — 노트 없는 태그는 운영자가 무엇이 바뀌었는지 알 수 없는 릴리즈입니다.

다섯 가지로 어긋내어 전부 잡히는 것을 확인했고, 그 과정에서 **첫 판이 사람을 잘못된 줄로 보내는 것**을 봤습니다. 어긋난 파일에서 첫 번째 `x.y.z` 를 찾아 "지금 값" 으로 보여 줬는데, README 가 답한 것은 **Confluence 6.9.1** 이었습니다. 이 릴리즈와 아무 상관 없는 숫자입니다. 추측 대신 **없는 것을 그대로** 말하게 고쳤습니다.

```
- README.md 에 'weekly-v0.116.0.tar.gz' 이(가) 없습니다
```

원칙으로 남깁니다. **검사가 틀린 단서를 주면, 없는 것보다 나쁩니다.** 사람이 그 단서를 따라 시간을 씁니다.

변이 검사는 계속 돌고 있고, `TestNobodyExportsSomebodyElsesDeck` 이 자기 함수의 변이를 여섯 개 중 하나도 못 잡는다는 중간 결과가 나왔습니다. 제품은 확인해 보니 옳습니다 — 남의 덱은 양방향 403 이고 자기 것은 200 입니다. 그러니 볼 자리는 시험입니다. 그 시험은 **상태 코드만** 보고 작성자가 실제로 덱을 받았는지, 그리고 타인의 거부가 거부인지 고장인지(500 도 통과합니다)를 보지 않습니다. 변이 검사가 끝나면 그것부터 손봅니다.

3.10.72 v0.117.0에서 완료 — 권한 검사를 통째로 지워도 통과하던 시험

변이 검사를 38/78 에서 멈췄습니다. Go 를 네 회차째 피하는 값이 남은 절반의 값보다 커졌고, 부분 결과가 이미 따라갈 단서를 줬기 때문입니다. 부분 결과는 남겨 뒀고, 필요하면 `--test` 로 좁혀 다시 돌릴 수 있습니다.

단서는 이것이었습니다.

```
TestNobodyExportsSomebodyElsesDeck — exportReportPPTX: caught 0, survived 6
```

제품부터 확인했습니다. 남의 덱은 양방향 403, 자기 것은 200 — **제품은 옳습니다.** 그러니 볼 자리는 시험이고, 그 시험은 **상태 코드만** 봅니다. 작성자가 받은 것이 실제 덱인지 보지 않고, 타인의 거부가 거부인지 고장인지도 보지 않습니다. 400 이든 500 이든 "200 이 아님" 이면 통과합니다.

둘 다 못 박고 확인했습니다.

```
옛 시험 + 거부가 500 으로 바뀜 → ok
새 시험 + 같은 훼손           → FAIL
```

그다음 같은 약점을 다른 곳에서 찾다가 **훨씬 나쁜 것**을 봤습니다.
`TestOnePersonsReportIsNotAnothersToRead` 는 읽기는 `403 또는 404` 로 제대로 확인하는데, 수정과 삭제는 "200 이 아님" 만 봅니다. 그래서 그 조건을 강화했더니 **제품에 대해 실패했습니다** — 400 이 돌아옵니다.

제품 결함인 줄 알고 물어봤습니다.

```
버전 없이 수정: 400 VERSION_REQUIRED 보고서 버전이 필요합니다
버전 넣고 수정: 403 FORBIDDEN 본인의 보고서만 수정할 수 있습니다
버전 넣고 삭제: 403 FORBIDDEN 본인의 보고서만 삭제할 수 있습니다
```

제품은 옳습니다. **시험이 헛돌고 있었습니다.** 버전 없는 요청은 권한을 따지기 전에 형식으로 거절되므로, 그 시험은 언제나 거절을 받았고 그 이유는 한 번도 권한이 아니었습니다.

증명했습니다.

```
새 시험 + 수정 소유권 검사 제거 → FAIL
옛 시험 + 같은 훼손             → ok
```

`if ownerID != p.ID` 를 통째로 지워도 옛 시험은 통과합니다. "남의 보고서는 남의 것" 이라는 이름을 달고 있는 시험이, 그 규칙의 쓰기 절반을 한 번도 시험하지 않았습니다.

원칙으로 남깁니다. **거절을 확인할 때는 거절당한 이유까지 확인해야 합니다.** "성공하지 않았다" 는 권한이 지켜졌다는 뜻이 아니라, 무슨 이유로든 막혔다는 뜻일 뿐입니다. 형식 오류로 막힌 요청은 권한을 시험하지 않

습니다 — 그리고 그런 시험은 초록색인 채로 규칙이 사라지는 것을 지켜봅니다.

3.10.73 v0.118.0에서 완료 — 눈으로 찾던 것을 기계에게 맡겼습니다

v0.117 의 발견은 손으로 찾은 것입니다. 시험 이름과 실제로 시험하는 것이 다를 수 있다는 것을 알았으니, 같은 질문을 **한 번에 전부** 물어야 합니다.

`scripts/authz-check.py` 를 만들었습니다. 권한 거부(`writeError(..., 403, ...)` 뒤에 `return` 이 오는 자리) 를 하나씩 통째로 들어내고 전체 시험을 돌립니다. **없애도 아무도 못 알아차리는 거부는, 아무도 지키지 않는 규칙입니다.** 모양이 다른 자리는 건드리지 않고 개수와 위치를 밝힙니다 — 절반만 본 실행이 전부 본 실행처럼 읽히면 안 됩니다.

전체 39곳 중 처음 둘만 돌려 봤는데 **둘 다 걸렸습니다.**

```
!  attachments.go:94   이 보고서의 첨부 이미지를 조회할 권한이 없습니다.
!  attachments.go:181  본인의 보고서에만 이미지를 첨부할 수 있습니다.
```

제품부터 확인했습니다. 실제 배포에서 남의 보고서 첨부는 조회도 업로드도 403 이고 자기 것은 200 입니다 — **제품은 옳습니다.** v0.117 과 같은 모양입니다. 규칙은 있고 지키는 것이 없습니다.

두 규칙에 시험을 붙이고 각 검사를 지워 확인했습니다. 둘 다 잡힙니다. 가드는 78개에서 80개가 되었습니다.

업로드 시험에는 한 가지를 더 넣었습니다. 거절이 **조용한 무시가 아니라 거절**인지 — 작성자의 목록에 `mine.png` 만 있고 `theirs.png` 는 없어야 합니다. 403 을 돌려주면서 파일은 저장하는 구현도 "거절" 로 보이기 때문입니다.

CI 에는 넣지 않았습니다. 거부 한 곳마다 전체 시험을 한 번 돌리므로 39곳이면 30분 가까이 걸립니다. `mutation-check` 와 같은 성격의 도구입니다 — 권한 코드를 손볼 때 사람이 돌립니다.

원칙으로 남깁니다. **손으로 한 번 찾은 종류의 결함은, 기계에게 전부 물어볼 수 있는지 먼저 생각합니다.** v0.117 은 운이 좋았습니다 — 마침 그 시험을 강화하다 옆에서 발견했습니다. 39곳 중 나머지 37곳은 그런 운을 기다릴 이유가 없습니다.

3.10.74 v0.119.0에서 완료 — 문서에 없는 기능은 밖에서는 없는 기능입니다

권한 검사를 배경에 돌려 두고 Go 를 건드리지 않는 자리를 봤습니다. README 가 "세부 규격은 OpenAPI 초안을 참고하십시오" 라고 안내하므로, 그 문서가 실제 경로를 따라가는지 세어 봤습니다.

```
코드의 경로: 97개
```

```
문서의 경로: 91개
```

여섯 개가 빠져 있었고 그중 넷은 진짜 API 표면입니다 — **개인 API 키 폐기, 로그아웃, 보고서 의견, OIDC 연결 시험.** 키는 발급과 회전이 문서에 있는데 폐기만 없었습니다. 연동을 만드는 사람이 키를 만들 수는 있고 없앨 수는 없는 문서를 읽고 있었던 셈입니다.

넷을 채우고 `scripts/openapi-check.py` 를 CI 에 넣었습니다. **양쪽으로 봅니다.**

- 문서에 없는 코드 경로 — 저장소 밖에서는 찾을 수 없는 기능입니다.
- 코드에 없는 문서 경로 — **이쪽이 더 나쁩니다.** 문서가 404 를 약속하고 있고, 그것을 믿고 코드를 쓴 사람은 자기 잘못을 찾느라 시간을 씁니다.

두 방향 모두 어긋내어 확인했습니다. 브라우저 리디렉션 진입점 둘은 예외로 뒀고, **예외마다 이유를 문자열로 붙이게** 했습니다. 이유를 안 적어도 되는 예외 목록은 어긋남이 숨는 자리가 됩니다. 예외에 적힌 경로가 코드에서 사라지면 그것도 지적합니다 — 오래된 예외는 오래된 사실입니다.

권한 검사는 계속 돌고 있고 세 번째 지적이 나왔습니다: `attachments.go:300` · `이 이미지를 조회할 권한이 없습니다` . 개별 이미지 조회도 지키는 것이 없습니다. 다음 회차에서 전체 결과를 봅니다.

3.10.75 v0.120.0에서 완료 — 서른일곱 중 스물일곱

`authz-check` 를 전부 돌렸습니다.

```
권한 거부 37곳 · 지적 27곳 · 지켜짐 9곳 · 모양이 달라 건너뜀 2곳
```

규칙 스물일곱 개를 지워도 시험 전체가 초록입니다. 다만 숫자를 그대로 읽으면 안 됩니다. 지적의 무게가 제각각입니다.

- `attachments.go:300·335·384` — 남의 캡처를 **열고 고치고 지우는** 것. 데이터 노출과 파괴입니다.
- `decisions.go:87·201·282·330` — 조회 범위 밖의 업무와 결정 기록.
- `handover.go:355` , `meeting.go:230` , `worksearch.go:142` — 팀장 이상 기능 게이트.
- `reports.go:323` — 같은 문구지만 **보고서가 없을 때** 나가는 가지입니다. v0.117 이 못 박은 것은 331행(`ownerID != p.ID`)이고, 323행은 오류 경로라 무게가 다릅니다.

제품은 확인해 봤습니다. 남의 캡처는 조회·수정·삭제가 모두 403 이고 주인은 200 입니다. **제품은 옳고 지키는 것이 없습니다** — v0.117·v0.118 과 같은 모양이 이제 스물일곱 번입니다.

손해가 가장 큰 셋부터 채웠습니다. 각 시험에는 **거절이 조용한 실행이 아닌지**도 넣었습니다 — 403 을 돌려주면서 설명은 바꿔 두거나 파일은 지워 버리는 구현도 겉으로는 거절로 보입니다. 셋 다 지워서 잡히는 것을

확인했습니다. 가드 80 → 83.

작업 중에 v0.116 의 버전 검사가 **제 빠뜨림을 잡았습니다.**

```
- .github/release-notes/v0.120.0.md 이 없습니다
버전 검사: 1 곳이 0.120.0 과 어긋납니다.
```

만든 지 네 릴리즈 만에 만든 사람에게 쓰였습니다.

원칙으로 남깁니다. **스물일곱이라는 숫자는 결론이 아니라 목록입니다.** 무게를 나누지 않고 한 번에 채우려 하면 급한 것과 급하지 않은 것이 같은 속도로 밀리고, 그 사이에 정말 위험한 자리가 계속 비어 있습니다.

3.10.76 v0.121.0에서 완료 — 무게순으로, 다음 넷

v0.120 에서 "스물일곱은 결론이 아니라 목록" 이라고 적었으니 무게순으로 이어 갑니다. 다음은 `decisions.go` 의 넷이었습니다.

결정 기록은 **회의에서 무엇을 정했는지 남기는 자리**입니다. 네 규칙이 그것을 지킵니다 — 범위 밖 업무의 결정을 읽는 것, 그런 업무에 결정을 남기는 것, 남의 결정을 고치는 것, 그리고 **기록한 사람과 관리자만 지울 수 있다**는 것. 마지막은 누가 기록을 없앨 수 있는가의 문제라 무게가 다릅니다.

실제 배포에서 넷 다 403 입니다. **제품은 옳고 지키는 것이 없었습니다.** 시험 셋(규칙 넷)을 붙이고 각 거부를 지워 전부 잡히는 것을 확인했습니다. 가드 83 → 86.

이번에도 각 시험에 **거절이 조용한 실행이 아닌지**를 넣었습니다. 이제 이 항목은 습관이 되었습니다 — 403 을 돌려주면서 결정을 기록해 두거나 제목을 바꿔 두거나 지워 버리는 구현은 상태 코드만 보는 시험에 완전히 투명합니다.

시험을 쓰다 하네스에서 한 가지를 배웠습니다. `refused(t, what, w)` 같은 작은 도우미를 만들 때 **거절의 정의를 한 곳에 두면** 다음 시험이 그것을 그냥 물려받습니다. v0.117 에서 손으로 판단하던 "200 이 아님 이 아니라 403 이나 404" 를 이제 함수 하나가 말합니다.

남은 목록(무게순): `duedate.go:62` 남의 업무 마감일, `reports.go:693` 볼 수 없는 보고서에 의견, `handover.go:218` 범위 밖 담당자, 그리고 기능 게이트 셋(`handover.go:355` , `meeting.go:230` , `worksearch.go:142`).

3.10.77 v0.122.0에서 완료 — 거절 뒤에 남는 상태

목록의 다음 셋입니다. 남의 업무 마감일, 볼 수 없는 보고서에 남기는 의견, 다른 조직 담당자의 인수인계.

실제 배포에서 셋 다 403 입니다. 제품은 옳고 지키는 것이 없었습니다. 시험 셋을 붙이고 각 거부를 지워 전부 잡히는 것을 확인했습니다. 가드 86 → 89.

이번 셋을 쓰면서 "거절이 조용한 실행이 아닌지" 를 한 걸음 더 밀었습니다. **거절 뒤에 무엇이 남아 있어야 하는가**를 각각 다르게 물었습니다.

- **마감일**은 주인이 정한 날짜 그대로여야 합니다. 데이터베이스에서 직접 읽어 확인합니다. 남이 조용히 옮겨 놓은 마감일은 거절보다 나쁩니다 — 주인은 자기가 고르지 않은 날짜에 맞춰 일하게 되고, 그것을 알아차릴 계기가 없습니다.
- **의견**은 저장되어 있지 않아야 합니다. 응답이 403 이어도 행이 남았다면 남긴 것입니다.
- **인수인계**는 거절 응답이 거절한 내용을 담고 있지 않아야 합니다. 403 과 함께 본문을 실어 보내는 구현은 상태 코드만 보는 시험에 완전히 투명합니다.

세 가지가 각각 다른 질문이라는 것이 요점입니다. "거절했나" 는 한 문장이지만 "거절이 무엇을 남기지 않았나" 는 그 자리마다 다릅니다.

남은 목록: 기능 게이트 셋(`handover.go:355` 업무 인사이트는 팀장 이상, `meeting.go:230` 조직 단위 회의, `worksearch.go:142` 조직 단위 조회)과 오류 경로 몇 곳. 게이트는 **권한이 아니라 기능 범위**라 무게가 다릅니다 — 뚫려도 남의 것이 보이는 게 아니라 자기 조직 것이 보입니다. 그래도 지키는 것이 없으면 조용히 사라집니다.

3.10.78 v0.123.0에서 완료 — 두 번째 층은 구멍이 아닙니다

목록의 마지막은 기능 게이트 셋이었습니다. 그런데 셋을 나란히 놓고 보니 하나가 다릅니다.

```
GET /api/v1/insights/work-graph   requireRole("TEAM_LEADER","ORG_MANAGER","ADMIN")
GET /api/v1/meeting               requireAuth
GET /api/v1/work-items/search     requireAuth
```

업무 인사이트는 라우터가 이미 막습니다. `handover.go:355` 의 검사는 두 번째 층이고, 지워도 평사원은 여전히 403 을 받습니다. 시험이 알아차리지 못하는 것이 **정상**입니다 — v0.115 의 `layers.ts` 와 같은 동등한 변이입니다.

나머지 둘은 다릅니다. 경로는 세션만 확인하므로 **핸들러 안의 검사가 유일한 층**입니다. 지우면 작성자가 동료들의 회의 안건과 업무 검색 결과를 봅니다. 이 둘에 시험을 붙이고 각각 지워 잡히는 것을 확인했습니다. 가드 89 → 91.

시험에는 거절만 넣지 않았습니다. **팀장에게는 실제로 열린다**는 것도 함께 봅니다 — 막기만 하고 열리지 않는 문은 규칙이 아니라 고장이고, 거절만 확인하는 시험은 그 고장을 통과시킵니다.

그리고 도구를 고쳤습니다. `authz-check` 가 `app.go` 의 `requireRole(...)` 을 읽어 **라우터가 이미 막는 핸들러**를 알아보고, 그 안의 거부를 지적이 아니라 `두 번째 층` 으로 표시합니다.

```
1곳은 라우터가 이미 막는 두 번째 층입니다
  두 번째 층 handover.go:355  workGraph  ...
```

이번 창에서 반복해 배운 것을 도구에 적용한 것입니다. **잘못 짚는 검사는 없는 것보다 나쁩니다** — 사람이 그 단서를 따라 시간을 쓰고, 다음부터는 그 도구를 읽지 않습니다. 그러면 진짜 지적도 함께 묻힙니다.

목록을 다 채웠으니 `authz-check` 를 다시 돌려 숫자가 실제로 줄었는지 재고 있습니다. **고쳤다고 말하기 전에 다시 재는 것**이 이 회차들의 방식이었습니다.

3.10.79 v0.124.0에서 완료 — 27에서 14로, 그리고 줄과 규칙의 차이

목록을 다 채웠다고 말하기 전에 다시 쟀습니다.

	v0.120 처음	v0.124 재측정
지워도 아무도 모르는 규칙	27곳	**14곳**
시험이 지키는 규칙	9곳	**21곳**

절반으로 줄었습니다. 이번 회차에는 둘을 더 채웠습니다 — 조직 단위 변화 요약(`weeklychange.go:236`), 그리고 **API 키의** `mcp:read` **범위**.

두 번째가 이 회차의 배움이었습니다. 시험을 쓰고 `mcp.go:56` 을 지웠는데 **잡히지 않았습니다.** 미들웨어가 이미 막고 있었기 때문입니다.

```
mcp.go:56 만 제거          → ok   (미들웨어가 막음)
미들웨어 검사만 제거       → ok   (mcp.go:56 이 막음)
두 층 모두 제거            → FAIL
```

시험이 지키는 것은 **줄이 아니라 규칙**입니다 — 좁은 키는 MCP 를 쓸 수 없다는 것. 두 층 중 하나만 남아도 규칙은 지켜지고, 사용자에게는 아무 차이가 없습니다. `authz-check` 가 지적한 것은 그 줄이지 그 규칙이 아닙니다.

그래서 가드 표식을 `mcp` 에서 `apiKeyRequestAllowed` 로 옮겼습니다. 시험이 실제로 실행하는 것이 그쪽이기 때문입니다. 그리고 도구 안에 이 판단을 적어 뒀습니다 — **살아남은 거부를 볼 때 먼저 물을 것은 "다른**

곳이 이미 막고 있지 않은가" 입니다. v0.115 의 동등한 변이와 같은 질문이고, 이번에는 미들웨어라는 다른 모양으로 나왔습니다.

남은 열넷은 성격이 갈립니다. `workitemedit.go` · `workitemlinks.go` · `workitems.go` 의 여섯은 진짜 데이터 경계이고, `reports.go:323·408` 은 오류 경로이며, `auth.go` 의 둘은 로그인 설정, `mcp.go:48` 은 요청 출처입니다. 다음 회차는 여섯부터입니다.

3.10.80 v0.125.0에서 완료 — 건너뛴 시험은 아무것도 지키지 않습니다

남은 열넷 중 진짜 데이터 경계인 여섯을 채웠습니다. 업무를 합치고 쪼개는 것, 선행 관계를 보고 등록하고 지우는 것, 그리고 조직 단위 업무 목록입니다. 여섯 모두 지워서 잡히는 것을 확인했습니다. 가드 93 → 98.

제품을 먼저 확인하다 하나를 배웠습니다. 남의 업무를 합치려 하니 **400** 이 나왔습니다.

```
u1 → u57 의 업무 57 을 u1 의 업무 1 로 합치기
  400 DIFFERENT_OWNER  담당자가 다른 업무끼리는 합칠 수 없습니다
```

권한이 아니라 **다른 검사가 먼저** 잡은 것입니다. 98행에 닿으려면 **남의 업무 둘을** 합치려 해야 합니다. 그렇게 하니 `403 본인 업무만 정리할 수 있습니다` 가 나왔습니다. v0.117 의 버전 없는 요청과 같은 모양입니다 — **잘못된 요청은 권한을 시험하지 않습니다.**

그리고 제 시험에서 같은 실수를 했습니다. 업무 둘을 만들려고 서로 다른 주차에 보고서 둘을 냈는데, 제품이 **같은 제목의 업무를 주차를 넘어 하나로 묶습니다.** 의도된 동작입니다. 그래서 업무가 하나만 나왔고 시험 셋이 조용히 건너뛰어졌습니다.

```
--- SKIP: TestOnlyTheOwnerFoldsTwoTasksIntoOne
    the reports produced 1 work items, two are needed
```

건너뛴 시험은 아무것도 지키지 않습니다. v0.106.1 에서 `guard-check` 의 마지막 줄을 고친 것과 같은 문제인데, 이번에는 제가 그 함정을 직접 만들었습니다. 한 보고서에 서로 다른 제목의 항목 둘을 넣는 것으로 바꾸니 다섯 시험이 모두 실제로 돕니다.

원칙으로 남깁니다. **`t.Skip` 은 통과가 아닙니다.** 준비물이 갖춰지지 않아 건너뛴 시험은 초록색으로 보이지만, 그 자리에 시험이 없는 것과 정확히 같습니다.

3.10.81 v0.126.0에서 완료 — 제품이 저를 막아섰습니다

남은 여덟 중 넷을 채웠습니다. 그중 하나는 **제가 쓰려던 시험이 틀렸다**는 것을 제품이 알려 준 경우입니다.

`auth.go:98` 은 로컬 로그인이 꺼졌을 때의 거부입니다. 그것을 시험하려고 설정을 껐더니 제품이 거절했습니다.

```
400 LOGIN_REQUIRED  검증된 OIDC 설정을 먼저 저장·활성화한 뒤 로컬 로그인을 끄세요.
```

아무도 못 들어오는 배포를 만들 수 없게 막고 있습니다. 설정을 바꾼 관리자 본인도 되돌릴 수 없는 상태이므로, 이것이 `auth.go:98` 보다 무거운 규칙입니다. 그래서 시험 대상을 그쪽으로 옮겼습니다 — 거절하는지, 이유를 말하는지, **무엇을 먼저 해야 하는지 알려 주는지**, 그리고 거절 뒤에도 문이 열려 있는지.

`auth.go:98` 은 여전히 지키는 것이 없습니다. 거기에 닿으려면 검증된 OIDC 공급자가 필요하고 이 하네스에는 없습니다. **못 지킨 것을 지켰다고 적지 않기 위해** 여기 남깁니다.

나머지 둘도 채웠습니다.

- **MCP 요청 출처(`mcp.go:48`)** — 브라우저에 열린 다른 페이지가 사용자의 세션으로 MCP 를 부리지 못하게 합니다. 프로토콜로 직접 붙는 도구는 `Origin` 을 보내지 않으므로 **그대로 동작해야** 하며, 시험이 그 점도 함께 봅니다. 막기만 하는 규칙은 도구를 죽입니다.
- **결정 제안 범위(`decisionsuggest.go:125`)** — 여기는 **순서**가 규칙입니다. 범위 확인이 AI 기능 확인보다 먼저 와야 하고, 그렇지 않으면 AI 를 켜는 순간 남의 업무가 열립니다. 시험에 그 이유를 적어 뒀습니다.

가드 98 → 101.

원칙으로 남깁니다. **시험을 쓰다 제품이 막아서면, 막은 그 규칙이 더 무거운 경우가 있습니다.** 원래 쓰려던 것을 억지로 통과시키기 전에 무엇이 막았는지 읽어야 합니다.

3.10.82 v0.127.0에서 완료 — 규칙은 "거절한다"가 아니라 "같은 방식으로 거절한다"

목록의 마지막 둘은 오류 경로로 분류해 뒀던 `reports.go:323·408` 입니다. 없는 보고서를 수정·삭제하려 할 때 나가는 거부인데, 읽어 보니 **오류 경로가 아니라 보안 규칙**이었습니다.

없는 보고서에 대해 `404 찾을 수 없습니다` 가 아니라 **남의 보고서와 똑같은** **`403 본인의 보고서만 …`** 을 돌려줍니다. 둘이 다르게 답하면 번호를 하나씩 넣어 보는 것만으로 어떤 보고서가 존재하는지, 대략 몇 건인지 알 수 있습니다.

그래서 시험을 "403 을 돌려주는가" 로 쓰지 않았습니다. **두 응답이 서로 같은가** 로 썼습니다.

```
if theirs.Code != missing.Code { ... "the difference names what exists" }
if theirs.Body.String() != missing.Body.String() { ... }
```

세 가지로 훼손해 확인했습니다. 두 거부를 각각 지우면 잡히고, 그리고 **없는 보고서를 `404 찾을 수 없습니다`로 바꾸는 변경** — 얼핏 더 친절해 보이는 쪽 — 도 잡힙니다. 그 세 번째가 이 시험의 값입니다. 앞의 둘은 다른 시험도 잡을 수 있지만, 존재를 흘리는 변경은 "같은가" 를 묻는 시험만 잡습니다.

코드에 이유를 적었습니다. 어디에도 없었습니다.

> A report that does not exist is refused exactly as somebody else's is, down to the wording. Told apart, the two replies let anyone walk the identifiers and learn which reports exist and how many there are.

이유가 적혀 있지 않으면 다음 사람이 "친절하게" 고칩니다. v0.104 에서 주석에 적힌 근거가 틀린 경우를 봤는데, 이번에는 근거가 아예 없어 옳은 코드가 위태로웠던 경우입니다.

가드 101 → 103. 목록을 다 채웠으니 마지막으로 다시 재고 있습니다.

3.10.83 v0.128.0에서 완료 — 기억 속 숫자와 비교하지 않기

권한 목록을 다 채우고 마지막 재측정을 돌리다가, 그 재측정이 오래 걸리는 **이유**를 봤습니다. `authz-check` 는 거부 한 곳마다 전체 시험을 한 번씩 돌리고, 전체 시험은 데이터베이스를 쓰는 시험 예순일곱 개가 각각 자기 데이터베이스를 **처음부터 마이그레이션** 하며 만듭니다.

재 봤습니다.

```
newTestServer 하나        : 약 2.1초
그중 CREATE DATABASE      : 0.15초
템플릿에서 복제           : 0.4초
```

나머지 2초가 마이그레이션입니다. 스키마만 담은 템플릿을 한 번 만들고 복제하도록 바꾸니 하네스 한 번이 **2.1초에서 0.28초**가 되었습니다.

그런데 전체 시험을 재니 66초로, 바꾸기 전과 **같아 보였습니다.** 여기서 그냥 넘어갈 뻔했습니다 — 제가 비교한 "바꾸기 전" 은 **기억 속의 숫자**였습니다. 되돌려서 나란히 쟀습니다.

	1	2	3	중앙값
템플릿 없이	134.8초	83.9초	85.7초	85.7초
템플릿 사용	48.7초	61.7초	61.8초	61.7초

편차가 크지만 **두 무리가 겹치지 않습니다.** 템플릿 쪽 최악(61.8)이 없는 쪽 최선(83.9)보다 낫습니다. 이 정도가 이 기계에서 할 수 있는 정직한 주장입니다.

그리고 **제가 만든 누수**를 찾았습니다. 템플릿을 지우는 곳이 없어 실행마다 하나씩 남습니다. 세어 보니 **63개**가 쌓여 있었습니다. CI 에서는 컨테이너가 버려지니 보이지 않고, 사람이 실제로 작업하는 기계에만 쌓입니다. `TestMain` 에서 지우게 하고, 실행 뒤 남은 수가 0인 것을 확인했습니다.

원칙으로 남깁니다. **성능을 말하려면 같은 조건에서 두 번 재야 합니다.** 한 번 재고 기억과 비교하면, 이 경우처럼 "차이 없음" 이라는 결론이 나오고 실제로는 절반이 줄어 있습니다. 그리고 **빠르게 만드는 변경은 무언가를 남기지 않는지 함께 봐야 합니다** — 이번에는 제가 그 함정에 바로 빠졌습니다.

3.10.84 v0.129.0에서 완료 — 27에서 3으로, 그리고 도구가 못 보던 모양

빨라진 시험으로 마지막 재측정을 돌렸습니다.

	v0.120 처음	v0.129
지워도 아무도 모르는 규칙	27곳	**3곳**
시험이 지키는 규칙	9곳	**32곳**

남은 셋은 각각 이유가 있고, **그 이유를 적어 두는 것이 채우는 것만큼 중요합니다.**

- `auth.go:98` · `auth.go:386` — 검증된 OIDC 공급자가 있어야 닿습니다. 하네스에 식별 공급자가 없어 이 환경에서는 확인할 수 없습니다.
- `mcp.go:56` — 미들웨어의 `apiKeyRequestAllowed` 가 이미 막는 두 번째 층입니다(v0.124). 지워도 동작이 달라지지 않으므로 시험이 알아차리지 못하는 것이 옳습니다.

이번 회차에는 **도구가 한 번도 보지 못한 두 곳**을 채웠습니다. `authz-check` 는 "거부 다음 줄이 바로 `return` " 인 모양을 찾는데, 보고서 복제는 `else` 안에 있고 기간 집계는 값 셋을 돌려주며 끝납니다. 그래서 36곳을 재는 동안 이 둘은 매번 `건너뜀` 으로만 지나갔습니다.

도구가 한 모양만 볼 줄 안다는 것은 나머지가 괜찮다는 뜻이 아닙니다. 건너뛴 것을 개수와 위치로 밝히게 해 둔 덕분에(v0.118) 이 둘이 눈에 남아 있었습니다 — 조용히 빼는 도구였다면 27에서 3으로 줄이고 "다 됐다" 고 적었을 것입니다.

기간 집계 시험에는 하나를 더 넣었습니다. 화면 경로뿐 아니라 **CSV 와 PPTX 내보내기로도 우회할 수 없는지** 봅니다. 같은 기간을 같은 게이트로 읽는 세 경로인데, 게이트가 한 곳에만 있으면 나머지 둘이 문입니다.

원칙으로 남깁니다. **측정 도구의 사각지대는 측정값 안에 보이지 않습니다.** 그래서 도구가 무엇을 보지 않았는지 스스로 말하게 해야 합니다.

3.10.85 v0.130.0에서 완료 — 규칙이 지켜지는가에서, 규칙이 옳은가로

권한 갈래를 마쳤으니 렌즈를 옮겼습니다. 이번 창의 도구들은 **규칙이 지켜지는가**를 물었습니다. 이제 **규칙이 옳은가**를 물을 자리를 찾았습니다 — 제품이 사람에게 내리는 판단 중 근거가 코드에만 있는 것.

임계값 셋(`merge_similarity`, `stall_weeks`, `persistent_issue_weeks`)을 먼저 봤습니다. 범위를 벗어나면 조용히 기본값으로 되돌리기에 관리자의 생각과 어긋날까 했는데, **검증기가 저장 시점에 같은 범위로 막고 있었습니다.** 추측이 틀렸습니다.

그다음 `StalledWeeks` 를 읽었습니다. 규칙은 이랬습니다.

> 가장 최근 보고부터 거꾸로, 진척도가 같은 **보고를 센다**

재 봤습니다.

상황	StalledWeeks	AgeWeeks
매주 보고하며 여섯 주 동안 50%	6	6
여섯 주 사이 두 번만 보고, 둘 다 50%	**2**	6
세 주 연속 50%	3	3

여섯 주째 그대로인 업무가 2주째로 나옵니다. 그리고 회의 안건은 `10*StalledWeeks` 로 순위를 매기므로, **한 달 반 방치된 업무가 매주 챙겨 보던 3주짜리보다 아래에** 놓입니다. 회의에서 가장 먼저 봐야 할 것이 가장 늦게 보입니다.

사람이 읽는 문장은 `진척이 N주째 멈춰 있습니다` 입니다. **거기서 주는 주입니다.** 진척도가 마지막으로 바뀐 주부터 달력상 주 수로 세도록 고쳤습니다. 매주 보고한 경우 결과는 그대로입니다.

기존 시험 하나가 깨졌고 **깨진 것이 옳았습니다.** 준비물이 정확히 이 경우였습니다 — 06-08 에 30% 가 된 뒤 **06-15 가 빠지고** 06-22·06-29 에 그대로. 옛 기대값 3은 보고 횟수였고 정답은 4입니다. 그리고 그 자리에는 **근거가 적혀 있지 않았습니다** — 같은 시험이 `AgeWeeks`·`SilentWeeks` 에는 "다섯 주 중 넷이 보고됨" 이라고 달아 두었는데도.

`IssueWeeks` 는 그대로 보고 기준으로 두었습니다. **멈춤은 시간이 흐르면 저절로 쌓이지만, 이슈는 누군가 적어야 알 수 있는 관찰입니다.** 그 구별을 시험 주석에 적었습니다.

원칙으로 남깁니다. **숫자 옆의 문장이 그 숫자의 정의입니다.** "N주째" 라고 쓰면서 보고 횟수를 세면, 코드는 일관되어도 제품은 거짓말을 합니다.

3.10.86 v0.131.0에서 완료 — 두 가지를 섞어 잰 질의, 그리고 잠재 결함을 다루는 법

v0.130 의 렌즈를 이어 파생 지표를 훑었습니다. `RepeatedPlan` 이 옛 `StalledWeeks` 와 같은 모양이었습니다 — 보고를 세는데 화면은 `계획 N주 반복` 이라고 씁니다. **다만 여기서는 숫자가 맞고 말이 틀립니다.** 계획을 다시 적는 것은 누군가 하는 행위라 횟수가 옳습니다(v0.130 에서 `IssueWeeks` 에 적용한 구별과 같습니다). 같은 표의 세 줄 위가 이미 `{reportedWeeks}회` 로 쓰고 있어서, 표현만 맞추면 되는 일이었습니다.

그다음 이슈 해소 지표를 봤습니다. 여기서 **제 측정이 두 가지를 섞었습니다.**

```
SELECT count(*) FILTER (WHERE weeks <> (ended_week - started_week)/7 + 1) ...
→ 2 / 2 건이 어긋남
```

이 숫자를 결함으로 읽을 뻔했습니다. 실제 행을 꺼내 보니 어긋남의 정체는 **설계상의 차이**였습니다 — `weeks` 는 이슈가 있던 주를 세고, 제 식은 이슈가 사라진 보고까지 포함했습니다. 간격과는 아무 상관이 없었습니다. 간격이 있는 종료 건을 따로 세니 **0건**이었습니다.

그래서 이것은 **잠재 결함**입니다. 관측되지 않았습니다. 그런 것을 고칠 때는 계산식을 고르는 방식이 달라야 합니다.

```
(ended_week - started_week)/7
```

실제 두 행에 대해 1과 4 — **저장값과 정확히 같습니다.** 매주 보고한 경우 결과가 바뀌지 않고, 거른 주가 있을 때만 달라집니다. 관측되지 않은 문제를 고치면서 관측된 값은 건드리지 않는 식입니다.

간격 있는 사례를 시험으로 만들어 차이를 보였습니다. 옛 계산은 다섯 주 버틴 장애물 둘의 중앙값을 **2주**라고 답합니다.

> `both obstacles stood five weeks, the median says 2`

누군가 자리를 비운 주는 아무것도 해소되지 않던 바로 그 주입니다. 그 주를 빼고 세면 조직이 실제보다 장애물을 빨리 치우는 것처럼 보이고, 그 착시는 개선이 필요한 곳을 정확히 가립니다.

원칙 둘을 남깁니다. **질의 하나가 두 가지를 섞으면, 그 숫자는 어느 쪽도 말하지 않습니다** — `2/2 어긋남` 은 결함처럼 보였지만 아무것도 보여 주지 않았습니다. 그리고 **관측되지 않은 결함을 고칠 때는, 관측된 값이 바뀌지 않는 고침을 고릅니다.**

3.10.87 v0.132.0에서 완료 — 보고를 덜 하면 빨라 보입니다

같은 렌즈를 예측 지표에 댔습니다. `forecast.go` 는 잘 쓰여 있습니다 — 두 속도가 엇갈리면 그 불일치 자체를 결과로 내놓고, 기울기가 음수면 "끝나는 시점이 없습니다" 라고 말합니다. 그런데 분모가 이랬습니다.

```
spans := float64(len(series) - 1)
```

보고 간격입니다. 출력은 `shiftISOWeek` 로 달력 날짜를 내놓는데 분모는 보고 수입니다. 재 봤습니다.

상황	실제	제품이 말한 속도	완료 예상
매주 보고 · 6주에 50→80	6%/주	6.0%/주	4주 뒤
같은 6주, 세 번만 보고	**6%/주**	**15.0%/주**	**2주 뒤**
3주 연속 50→80	15%/주	15.0%/주	2주 뒤

가운데 줄이 결함입니다. **같은 기간에 같은 만큼 나아갔는데 보고를 덜 했다는 이유로 2.5배 빠른 것이 되고 완료가 두 주 앞당겨집니다.** 그리고 아래 두 줄은 하나가 6주, 하나가 3주짜리인데 **제품이 구별하지 못합니다.**

이번 창에서 세 번째로 같은 뿌리입니다 — v0.130 의 멈춤, v0.131 의 이슈 해소, 그리고 여기. 다만 무게가 다릅니다. 앞의 둘은 지나간 일을 잘못 세었고, **이것은 앞으로의 날짜입니다.** 사람이 그 날짜에 맞춰 일정을 짭니다.

첫 보고와 마지막 보고 사이의 달력 주 수로 나누도록 고쳤습니다. 매주 보고한 경우 결과가 그대로여서 기존 시험이 하나도 깨지지 않았습니다 — **고침이 옳다는 신호이지, 시험이 약하다는 신호가 아닙니다.** 옛 계산으로 되돌리면 새 시험이 잡습니다.

`weekSpan` 은 최소 1을 돌려줍니다. 같은 주에 두 항목이 들어오거나 날짜를 읽지 못하면 0으로 나누게 되고, 그러면 어떤 진척이든 무한한 속도가 됩니다. 그 경우들을 따로 시험했습니다.

원칙으로 남깁니다. **분모가 무엇을 세는지가 그 비율의 뜻입니다.** 분자가 맞아도 분모가 다른 것을 세면, 결과는 아무 뜻이 없는 숫자가 아니라 **그럴듯하게 틀린 숫자**가 됩니다 — 그리고 그런 숫자만 사람의 일정에 들어갑니다.

3.10.88 v0.133.0에서 완료 — 엄격하게 잡을수록 더 놓치던 설정

같은 뿌리가 세 번 나왔으니 눈으로 찾는 것을 그만두고 **기계적으로 훑었습니다** — `len(...)` 을 주 수처럼 쓰는 자리를 전부 나열했습니다. v0.118 에서 배운 방식입니다.

두 곳이 남아 있었습니다.

`isStalled` — 기간 업무보고의 정체 판정이고, `summarizeWorkItem` 의 것과 **다른 자리**입니다. 주석이 정직하게 "consecutive reported weeks" 라고 적어 두었는데, 그 결과 설정이 거꾸로 동작했습니다.

상황	3주 기준 이전	지금
3주 연속 같은 진척도 (보고 3회)	정체	정체
6주 동안 같은 진척도 (보고 2회)	**정상**	**정체**

더 엄격하게 잡으려고 값을 올리면 가장 오래 방치된 업무부터 빠져나갑니다. 오래 방치된 업무일수록 보고가 드물기 때문입니다. 설정의 뜻이 "3주 동안 안 움직인 것을 표시하라" 인데, 실제로는 "같은 값으로 세 번 보고된 것을 표시하라" 였습니다.

여기서 **제 라벨 하나가 틀렸습니다.** 보고가 하나뿐인 경우를 "6주간 50%" 라고 적어 놓고 `false` 를 결함처럼 읽을 뻔했는데, 보고가 하나면 그 이전 주에 대해 아무것도 알 수 없습니다. **`false` 가 옳습니다.** 시험에도 그렇게 적었습니다 — 한 번의 보고로 정체를 말하는 것은 없는 역사를 지어내는 일입니다.

`reportquality` 는 반대 결론이 났습니다. `같은 이슈를 N주째 보고하고 있습니다` 의 N 은 **다시 적은 횟수**이고, 다시 적는 것은 사람이 하는 행위입니다. v0.130 에서 세운 구별 — 관찰은 보고 수, 상태는 주 수 — 을 그대로 적용하면 숫자가 맞고 단어만 틀립니다. `N번째` 로 바꿨습니다. 계획 반복도 같습니다.

이번 창에서 이 뿌리를 네 번 만났고 **결론이 매번 같지 않았습니다.** 멈춤·이슈 해소·완료 예상은 숫자를 고쳤고, 계획 반복·이슈 반복은 단어를 고쳤습니다. 가르는 기준은 하나입니다 — **그 일이 시간이 흐르면 저절로 쌓이는가, 아니면 누군가 해야 일어나는가.**

3.10.89 v0.134.0에서 완료 — 시험대가 재현하지 못하던 배포

v0.130 부터 v0.133 까지 네 번에 걸쳐 주 수 계산을 고쳤습니다. 고친 것이 실제 배포에서 무엇을 바꾸는지 확인하려고 **옛 빌드와 새 빌드를 같은 데이터베이스에 나란히 붙여** 쟀습니다. 302명 · 15,594건 · 109,175행짜리 규모 배포입니다.

결과는 **완전히 같았습니다.** 기간 업무보고 63건, 업무 목록 14건, 어느 쪽도 한 글자도 달라지지 않았습니다.

처음에는 고침이 무의미했다는 뜻으로 읽힐 수 있었지만, 데이터를 보니 이유가 달랐습니다.

시드가 만들던 것	값
보고 누락 주가 있는 업무	3,837건 중 15건
4주 이상 정체된 미완 업무	**0건**

시드가 만들던 것	값
최장 정체	6주

진척도가 2주마다 1%씩 꾸준히 올라서 **정체 구간이 아예 생기지 않았습니다.** 보고 누락은 있었지만 정체 구간 안에 있지 않았으니, 고친 계산이 닿을 자리가 하나도 없었습니다. 배포가 조용했던 것이 아니라 **시험대가 그 상황을 만들지 못한 것입니다.**

시드 주석에 이미 같은 교훈이 적혀 있었습니다 — 예전 판은 모든 업무를 100%까지 올려 회의 안건을 비게 만들었습니다. 이번에는 반대 방향으로 같은 실수를 하고 있었습니다. **모든 업무를 끊임없이 움직이게 만들어 정체를 없앴습니다.**

여섯 업무 중 하나가 도중에 멈추고 다시는 움직이지 않도록 바꿨습니다.

	이전	지금
4주 이상 정체	0건	**350건**
최장 정체	6주	**32주**
완료 업무	234건	234건

그런데 더 큰 구멍이 그 아래 있었습니다. **업무 식별자는 보고서를 앱으로 저장할 때만 만들어집니다** — `resolveWorkItem` 이 저장 트랜잭션 안에서 하는 일입니다. SQL 로 직접 넣은 행은 연결되지 않으니, 갓 시드한 배포는 `work_items` 가 비어 있습니다.

그 위에 서 있는 화면이 전부입니다 — 업무 목록, 정체 판정, 완료 예상, 의존 관계, 결정 사항. **시드는 가득 차 보이는데 그 화면들은 전부 백지였습니다.** 규모 배포에서 값이 보였던 것은 제가 창 내내 그 배포를 두드려 왔기 때문입니다.

`candidateTitleKey` 와 같은 규칙으로 정규화해 시드가 직접 업무를 만들고 연결하도록 했습니다.

	이전	지금
업무	0건	**2,100건**
미연결 보고 항목	109,200행	**0행**
시드 시간	—	5.7초

한 사용자의 업무 목록이 이렇게 나옵니다.

```
업무 7건 (정체 3건)
  세션 개선     진척 10% 정체32주 주당0.2 DISTANT
  결산 개선     진척 37% 정체 2주 주당0.5 DISTANT
  암호화 개선    진척 55% 정체 1주 주당1   PROJECTED
  게이트웨이 개선 진척100% 정체 0주 주당0   DONE
```

완료 예상의 세 갈래가 모두 나타나고, 기간 업무보고의 정체 판정이 7건 중 3건으로 동작합니다. 규모 점검은 24개 경로에서 지적 사항 0건을 유지합니다.

원칙으로 남깁니다. **시험대가 만들지 못하는 상황은 시험한 적이 없는 상황입니다.** 그리고 시험대의 결함은 실패로 나타나지 않고 **조용한 통과로 나타납니다** — 이번에는 옛 빌드와 새 빌드가 완벽하게 일치한다는, 가장 안심되는 모양을 하고 나타났습니다.

3.10.90 v0.135.0에서 완료 — 지난 절의 정정, 그리고 말하지 않은 것을 지우던 저장

먼저 **바로 앞 절을 정정합니다.** v0.134.0 에서 "갓 시드한 배포는 `work_items` 가 비어 있고 그 위의 화면이 전부 백지였다" 고 적었는데, 그 말은 과장입니다. 앱은 부팅할 때 `backfillWorkItems` 를 돌립니다. 연결을 전부 끊고 재시작해 재 봤습니다.

백필 대상	109,200행
완료까지	약 150초
남은 미연결	0행

즉 화면이 영구히 비는 것이 아니라 **부팅 후 2분 반 동안** 빕니다. 시드가 직접 연결하도록 한 변경은 그 창을 없애 준다는 뜻이지, 없던 기능을 살린 것이 아닙니다. v0.134.0 의 다른 절반 — 정체가 한 번도 생기지 않던 것 — 은 그대로 유효합니다.

기록해 둡니다. **제가 찾은 결함의 크기를 제가 부풀렸습니다.** 시드에 연결이 없다는 것까지는 맞게 봤는데, 제품이 스스로 고친다는 것을 확인하지 않고 결론까지 갔습니다. 확인은 20초면 됐습니다.

그 확인을 하다가 진짜 결함이 나왔습니다.

`PUT /api/v1/admin/users/{id}` 는 모든 칸을 다시 씁니다. 그래서 **바꾸려는 칸만 적어 보내면 나머지가 지워집니다.** 비밀번호만 재설정하는 스크립트 모양으로 보내 봤습니다.

	이전	지금
이메일	`a@b.com` → `""`	그대로
소속	조직 → **없음**	그대로
비활성 계정의 활성 상태	**다시 활성**	그대로 비활성

세 번째가 가장 무겁습니다. `active` 를 안 보내면 기본값이 `true` 였습니다. **퇴사자 계정의 비밀번호를 재설정하는 것만으로 그 계정이 다시 로그인할 수 있게 됩니다.** 나머지 칸은 빠뜨리면 비워지는데 — 안전한 방향입니다 — 이 칸만 빠뜨리면 **권한이 열리는 방향**으로 움직였습니다.

두 번째도 가볍지 않습니다. 소속이 없어진 계정은 어느 팀장에게도 보이지 않으니, 그 사람이 올린 보고가 아무도 열 수 없는 대기열에 쌓입니다. v0.104 에서 화면까지 만들어 다룬 상태입니다.

화면은 폼 전체를 보내므로 도달하지 않습니다. **스크립트와 다른 클라이언트에서만 닿습니다** — 그리고 비밀번호 재설정이 바로 그곳에서 쓰입니다.

빠뜨린 칸과 비우라고 적은 칸을 구별하도록 고쳤습니다. 구조체 해석만으로는 둘 다 영값으로 도착하니, 본문에 어떤 키가 실제로 있었는지를 함께 읽습니다(`decodeJSONFields`). Go 는 JSON 키를 대소문자 없이 맞추므로 그 지도도 같은 방식으로 답해야 합니다 — 안 그러면 둘의 판단이 어긋납니다.

`""` 이나 `null` 을 명시해 보내면 여전히 지워집니다. 화면이 그렇게 동작하고, 그것이 지우는 유일한 방법이어야 합니다.

원칙으로 남깁니다. **빠뜨린 칸의 기본값은 그 칸이 무엇을 뜻하는지에 따라 정해야 합니다.** 값을 비우는 쪽은 눈에 보이고 되돌릴 수 있지만, 접근을 여는 쪽은 둘 다 아닙니다.

3.10.91 v0.136.0에서 완료 — PATCH 라고 적혀 있지만 PATCH 가 아니던 두 곳

v0.135.0 을 고치고 나서 같은 모양이 다른 곳에도 있는지 **저장 경로를 전부 훑었습니다.** `PUT` · `PATCH` 로 등록된 여덟 개입니다.

경로	판정
`PUT /admin/settings`	이미 부분 갱신 — 저장값과 병합합니다
`PATCH /reports/{id}/attachments/{id}`	이미 부분 갱신 — 포인터로 받습니다
`PUT /work-items/{id}/due-date`	칸이 하나뿐입니다

경로	판정
`PUT /admin/confluence/users/{id}/mapping`	빠뜨리면 꺼지는 방향이고, 문서가 필수로 적어 두었습니다
`PUT /reports/{id}`	아래 설명 — 그대로 둡니다
`PATCH /decisions/{id}`	**모든 칸을 다시 씁니다**
`PATCH /report-candidates/{id}`	**모든 칸을 다시 씁니다**

두 곳이 `PATCH` 로 선언되어 있으면서 전체 교체로 동작했습니다. 계약이 거짓인 셈이고, 그 거짓을 믿은 클라이언트가 데이터를 잃습니다.

`PATCH /decisions/{id}` 가 더 무겁습니다. `validateDecision` 이 제목과 결정자를 요구하므로 **호출자는 반드시 몇 칸을 보내게 되고, 그래서 요청이 부분 갱신처럼 보입니다.** 상태만 `DONE` 으로 옮기려고 필수 칸 셋과 상태를 보내면 이렇게 됩니다.

	이전	지금
결정 근거	**지워짐**	그대로
후속 조치	**지워짐**	그대로
합의된 기한	**지워짐**	그대로

기한은 문장이 아닙니다. 자기 기한이 없는 업무에는 **회의가 합의한 이 기한이 제시됩니다**(`agreedDue`). 결정을 완료로 옮기는 것만으로 그 제시가 사라졌습니다.

두 곳 모두 **저장된 값을 먼저 읽어 구조체를 채운 뒤 그 위에 본문을 씌우도록** 고쳤습니다. 요청이 말하지 않은 칸은 원래 값이 남고, `""` 를 명시하면 여전히 지워집니다. 화면은 양쪽 다 객체 전체를 보내므로 동작이 달라지지 않습니다.

`PUT /reports/{id}` 는 그대로 두었습니다. 항목이 곧 보고서이므로 **"항목을 언급하지 않았다" 와 "항목이 없다" 가 같은 뜻**이고, 버전 토큰을 함께 요구하므로 실수로 지나가는 요청이 아닙니다. 판단 기준은 v0.135.0 과 같습니다 — 빠뜨림이 뜻하는 바가 하나뿐인가.

그리고 **여기서 제 고침을 엉뚱한 함수에 넣을 뻔했습니다.** `var input decisionInput` 다음에 `decodeJSON` 이 오는 모양이 `createDecision` 과 `updateDecision` 양쪽에 똑같이 있어서, 첫 번째 것이 바뀌었습니다. 컴파일러가 잡아 주었습니다 — 잡히지 않았다면 기록을 만드는 쪽이 조용히 달라졌을 것입니다. **같은 모양이 두 번 나오는 파일에서 첫 일치를 바꾸는 것은 고침이 아니라 추측입니다.**

원칙으로 남깁니다. **동사가 약속한 것과 처리기가 하는 일이 어긋나면, 어긋남을 아는 사람은 아무도 없습니다.** 화면은 늘 전체를 보내니 드러나지 않고, 드러나는 것은 계약을 읽고 그대로 믿은 쪽입니다.

3.10.92 v0.137.0에서 완료 — 이미 답이 나온 이슈로 채워져 있던 위험 등록부

v0.134.0 에서 시드를 고쳤으니 화면들이 처음 받아 보는 데이터 모양이 생겼습니다. 브라우저로 열한 개 화면을 돌아봤고, 화면은 멀쩡했습니다. 대신 **문장 하나가 걸렸습니다.**

`grep 주째` 로 남은 자리를 전부 세어 v0.130 의 기준 — 시간이 쌓이는가, 사람이 해야 일어나는가 — 으로 나눴습니다. 정체 쪽 네 자리는 맞았고, `IssueWeeks` 를 쓰는 자리가 어긋나 있었습니다. 그리고 `reportquality.go` 에 **v0.133 이 놓친 한 줄**이 있었습니다 — 같은 `planRun+1` 값인데 차단이 선언된 변형만 아직 `주째` 라고 불렀습니다.

`IssueWeeks` 는 이렇게 계산됩니다.

```
for _, week := range merged {
    if strings.TrimSpace(week.Issue) != "" {
        item.IssueWeeks++
    }
}
```

연속을 보지 않습니다. 이력 전체에서 이슈가 적힌 보고를 셀 뿐입니다. 그런데 그 값이 이런 자리에 쓰였습니다.

자리	문장
`meeting.go`	`이슈가 %d주째 지속되고 있습니다`
`meeting.go`	`%d주째 해소되지 않은 이슈입니다`
`handover.go`	`같은 이슈가 %d주간 이어졌습니다`
`pptx_rollup.go`	`이슈 %d주 지속`
`worksearch.go`	`아직 해결되지 않은 …(%d주 지속)`
`workitems.go` · `rollup.go`	`AtRisk` — 위험 등록부에 올릴지

설정 이름도 `rollup.persistent_issue_weeks` 입니다. **전부 "지금 열려 있음" 을 말하는데, 값은 "언젠가 있었음" 을 셉니다.**

시드 배포에서 쟀습니다.

	이전	지금
업무 63건 중 위험 표시	**56건 (89%)**	7건 (11%)
그중 최근 보고에 이슈가 **없는** 것	**47건 (83%)**	**0건**
기간 업무보고 56건 중 위험	50건	7건

`계약 표준화` 는 위험 표시가 붙어 있는데 최근 보고의 이슈 칸이 비어 있었습니다. 이슈가 흩어진 일곱 주에 언급됐고, 마지막 언급은 몇 달 전이었습니다. **위험 등록부의 83%가 이미 답이 나온 이슈였습니다.**

올바른 모형은 이미 코드 안에 있었습니다. `reportquality.go:154` 는 `item.Issue` 가 비어 있지 않을 때의 **연속 구간(** `issueRun` **)** 으로 판단합니다. 그 모형을 나머지에 맞췄습니다.

값을 다시 정의하는 대신 **나눴습니다.**

- `IssueWeeks` — 이력. 이슈가 적힌 주가 몇 번이었나. 화면의 `이슈 발생 N주` · `이슈 이력 N주` 가 읽는 값이고, 그 라벨은 원래 정직했습니다.
- `IssueRunWeeks` — 지금 열려 있는 이슈. 마지막 보고에서 끝나는 끊기지 않은 구간을 **달력 주**로 잽니다. 마지막 보고에 이슈가 없으면 0입니다.

지속을 말하는 자리는 전부 두 번째로 옮겼습니다. 한 업무가 이제 `지속=22주 / 이력=27주` 로 두 값을 따로 보여 줍니다.

v0.130 의 판단을 여기서 뒤집습니다. 그때 "이슈는 관찰이므로 보고 수로 센다" 고 적었는데, v0.133 이 세운 기준을 그대로 적용하면 반대입니다. **이슈가 해소되지 않은 상태는 아무도 쓰지 않아도 시간이 지나면 쌓입니다.** 보고를 거른 주에 이슈가 사라지지는 않습니다. 그때는 단위(보고냐 주냐)만 보고 연속성 문제를 못 봤습니다.

그리고 v0.134.0 과 **똑같은 일이 또 있었습니다.** 고침을 넣고 재니 위험이 56건에서 **0건**이 됐습니다. 시드의 이슈 규칙이 `(r.id + slot) % 7` 인데 같은 업무의 다음 주는 `r.id` 가 305 만큼 커지고 305 mod 7 = 4 이므로, **두 주 연속으로 이슈가 붙는 일이 아예 없습니다.** 0은 옳은 값이었고, 시드가 지속 이슈를 한 번도 만들지 못한다는 뜻이었습니다. 아홉 업무 중 하나가 30주차부터 끝까지 답이 안 나온 이슈를 안고 가도록 시드를 고쳤습니다.

원칙으로 남깁니다. **하나의 숫자가 두 뜻으로 읽히고 있으면, 둘 중 하나는 반드시 틀린 자리에서 쓰이고 있습니다.** 고치는 방법은 값을 다시 정의하는 것이 아니라 나누는 것입니다 — 두 뜻이 다 필요해서 그렇게 된 것이기 때문입니다.

3.10.93 v0.138.0에서 완료 — 통과하지만 아무것도 못 잡던 가드들

이번 창에서 가드를 여럿 새로 썼습니다. `guard-check.py` 는 전부 자기가 이름 붙인 함수에 닿는다고 답합니다. 그런데 그 도구는 **"닿았는가" 만 답하고 "잡을 수 있는가" 는 답하지 못합니다** — v0.81 이 만든 `mutation-check.py` 가 그 질문을 위해 있고, 이번 창에서 한 번도 돌리지 않았습니다.

새로 쓴 가드 다섯 개에 겨눠 돌렸더니 **아무 시험도 잡지 못하는 변이가 여섯 개** 나왔습니다.

자리	변이	뜻
`summarizeWorkItem`	`IssueRunWeeks >= 임계값` → `>`	임계값 자체가 고정되지 않음
`summarizeWorkItem`	`StalledWeeks >= 임계값` → `>`	같음
`summarizeWorkItem`	`index >= 0` → `index > 0`	첫 보고까지 같은 계획인 경우
`updateUser`	`DisplayName == "" \`	\

!validRole `→` &&`	이름·역할 검사
`	잠금 방지의 적용 범위
`→` &&`	검사 전체

전부 같은 모양입니다. **제 시험들이 경계에서 멀리 떨어진 자리만 골라 짚었습니다.** 이슈 지속 시험은 0주와 6주 이상을 확인했는데 기본 임계값은 2주입니다. `>=` 를 `>` 로 바꿔도 두 시험이 다 통과합니다. 즉 **관리자가 입력하는 숫자가 무엇을 뜻하는지 아무것도 고정하고 있지 않았습니다.**

경계와 범위를 짚는 가드를 넣었습니다.

- 이슈가 정확히 2주 열려 있으면 위험, 1주면 아직 아님
- 정체가 정확히 2주면 정체, 지난주에 움직였으면 아님
- 첫 보고부터 같은 계획이면 그 첫 보고까지 셈
- 표시 이름이 비었거나 없는 역할이면 거부하고, **거부한 요청은 아무것도 쓰지 않았음**
- 자기 잠금 방지는 자기 계정에만, 그리고 실제로 역할이나 활성 상태를 뺐을 때만 — 관리자가 자기 이름을 바꾸는 것은 평범한 작업이고, 남을 강등하는 것은 이 규칙의 소관이 아님

- `{"active": null}` 은 언급하지 않은 것과 같은 뜻
- 자동 초안도 보고서와 같은 길이 한계를 지키고, 거부된 수정은 초안을 바꾸지 않음

그리고 **시험 하나가 제품이 더는 따르지 않는 결정을 문서화하고 있었습니다.** `workitems_test.go` 의 주석이 v0.130 의 근거("이슈는 관찰이므로 보고 수로 센다")를 그대로 담고 있었는데, v0.137.0 에서 그 판단을 뒤집었습니다. 단언은 여전히 맞았습니다 — `IssueWeeks` 는 이력이니까요 — 그래서 **시험이 깨지지 않았고, 그래서 주석이 틀린 채로 남았습니다.** 주석을 고치고 `IssueRunWeeks` 단언을 함께 넣었습니다.

원칙으로 남깁니다. **시험이 통과하는 것과 시험이 무언가를 지키는 것은 다른 일이고, 통과는 그 차이를 보여 주지 않습니다.** 임계값을 다루는 코드에서 그 차이는 거의 항상 같은 자리에 있습니다 — 경계에서 멀리 떨어진 값만 골라 짚은 시험입니다.

3.10.94 v0.139.0에서 완료 — 문을 지키고 방 안은 보지 않던 시험들

v0.138.0 에서 제가 쓴 가드에 변이 점검을 돌렸으니, 이번에는 **v0.120~v0.129 에서 인가 거부에 붙인 가드들**에 돌렸습니다. 열 개를 골랐고, **아무 시험도 잡지 못하는 변이가 17개** 나왔습니다.

가장 무거운 것 둘입니다.

`deleteWorkItemLink` — 여섯 변이 중 넷이 살아남았습니다. 그중 하나는 인가 조건 자체입니다.

```
if !canEditWorkItem(p, blockedOwner) && !canEditWorkItem(p, blockerOwner) {
```

`&&` 를 `||` 로 바꾸면 **양쪽 끝을 모두 소유한 사람만** 의존 관계를 지울 수 있게 됩니다. 주석은 정반대를 말합니다 — "어느 쪽 끝이든 지울 수 있다. 막고 있는 쪽은 자기 업무에 대한 사실이 아닌 주장을 취소하는 데 상대의 동의가 필요하지 않기 때문이다." 그런데 시험은 통과했습니다.

이유는 시험을 열어 보니 분명했습니다. `TestOnlyTheTwoEndsRemoveAPredecessorLink` 는 **바깥 사람이 거부되는 것만** 확인합니다. 이름이 약속한 "두 끝" 은 한 번도 실행되지 않았고, 게다가 픽스처의 두 업무가 **같은 사람 것**이라 다른 쪽 끝이 존재하지도 않았습니다.

`handover` — 여섯 변이가 모두 살아남았습니다. `TestNobodyReadsSomebodyElsesHandover` 는 주인이 200을 받고 바깥 사람이 거부되는 것을 확인하고 끝납니다. **문은 지키고 방 안은 보지 않습니다.** 그래서 함수 본문 전체가 무방비였습니다.

그중 하나는 처음에 잘못 읽었습니다. `item.UserID == target` 을 뒤집어도 제가 새로 쓴 내용 시험이 통과했습니다. 파고 보니 그 줄이 만드는 목록은 **결정 사항을 가져올 때만** 쓰이고, 화면에 나가는 목록은 255줄에서 다시 걸러집니다. 즉 변이의 효과는 **인수인계에서 결정 사항이 사라지는 것**입니다 — 업무도 진척도 계획도 다 맞아 보이는데, 그 일이 왜 그렇게 갔는지만 없어집니다. 새 담당자가 가장 필요로 하는 부분입니다.

정렬도 마찬가지였습니다. 나이 비교를 뒤집어도 시험이 통과했는데, 제 픽스처가 **정체 여부에서 먼저 갈렸기** 때문입니다. 두 키는 따로 확인해야 합니다 — 정체 상태가 같은 두 업무를 놓아야 나이 정렬이 실제로 쓰입니다.

무해한 변이도 하나 있었습니다. `if target == p.ID { scope = 자기 것만 }` 을 뒤집어도 응답이 같습니다. 255줄이 어차피 다시 거르므로 읽는 행 수만 늘어납니다. 도구가 "변이 생존은 판정이 아니라 질문" 이라고 말한 그대로이고, **가드를 지어내지 않고 그렇게 기록합니다.**

나머지 네 곳은 규칙은 있는데 실행된 적이 없는 자리였습니다.

자리	지켜지지 않던 규칙
`addComment`	의견은 1~5000자
`meetingMode`	**타 조직 대기** 구획은 실제로 대기 중일 때만 생김
`mergeWorkItem`	합칠 대상을 지정해야 함
`splitWorkItem`	원래 업무와 같은 제목으로는 분리 불가

`meetingMode` 의 것이 가장 눈에 띕니다. 조건을 꺼 버려도 모든 회의 시험이 통과합니다 — 회의실은 **누구를 연결해야 하는지 영영 듣지 못하게** 되는데도요.

가드 열두 개를 새로 썼고 17개 변이가 전부 잡힙니다(무해한 하나 제외). 확인은 도구가 아니라 **직접 변이를 넣고 시험을 돌려** 했습니다 — 도구는 한 시험당 한 번씩 컴파일하느라 느리고, 확인해야 할 것은 열일곱 줄뿐이었습니다.

작업 중 **복원을 스크래치 사본으로 하다가 변이가 든 파일을 원본으로 덮어썼습니다.** `git diff` 가 잡았습니다. 원본은 스크래치가 아니라 git 에 있습니다.

원칙으로 남깁니다. **거부를 확인하는 시험은 허용을 확인하지 않습니다.** 두 문장은 서로를 함의하지 않는데, 이름은 대개 둘 다 약속합니다 — `OnlyTheTwoEnds` 는 "두 끝만" 과 "두 끝은" 을 함께 말하고, 시험은 앞의 절반만 지키고 있었습니다.

3.10.95 v0.140.0에서 완료 — 켜는 쪽을 아무도 확인하지 않던 설정들, 그리고 변이 점검을 CI로

세 번째 변이 점검 묶음입니다. 결재·MCP·API 키·설정 가드 열 개에 돌려 **아무 시험도 잡지 못하는 변이 일곱 개**가 나왔고, 전부 같은 한 가지 모양이었습니다.

자리	시험이 확인하던 것	확인하지 않던 것
`updateSettings`	AI를 설정 없이 켜면 거부 — **아무도 확인 안 함**	제대로 켜지는지
`updateSettings`	Confluence 를 주소 없이 켜면 거부 — **아무도 확인 안 함**	끄는 것은 언제나 되는지
`updateSettings`	로컬 로그인을 OIDC 없이 **끄면** 거부	**다시 켜지는지**
`apiKeyRequestAllowed`	범위 없는 키가 MCP 를 **못 쓰는지**	범위 가진 키가 **쓸 수 있는지**
`mcp`	—	JSON-RPC 2.0 이 아닌 요청을 거르는지
`mcp`	—	모르는 프로토콜 판을 합의한 척하지 않는지

v0.139.0 에 적은 원칙이 그대로 다시 나왔습니다 — **거부를 확인하는 시험은 허용을 확인하지 않습니다.** 다만 이번에는 방향이 반대인 경우가 섞였습니다. `auth.local_enabled` 는 끄는 쪽만 지켜져 있었고, 조건을 조금만 넓히면 **로컬 로그인을 다시 켤 수 없게 됩니다** — OIDC 가 없는 배포에서 유일한 출구가 막힙니다.

`updateSettings` 세 곳은 처음 쓴 가드가 잡지 못했습니다. 이미 설정을 채운 뒤에 껐기 때문입니다. **아무것도 설정되지 않은 상태에서 끄는 것을 맨 앞에 놓아야** 조건이 넓어진 것을 볼 수 있습니다.

도구를 CI 로

`mutation-check.py` 는 v0.81 부터 있었고 이번 창에서 세 묶음을 돌려 **무방비 지점 30개**를 찾았습니다. 그런데 CI 에 없습니다 — 가드 하나당 컴파일과 시험을 여섯 번 하니 전수 실행이 두 시간을 넘기 때문입니다.

전수로 돌 필요가 없습니다. **잡지 못하는 가드는 거의 항상 새 가드**입니다 — 대상이 분기를 새로 얻었거나, 가드 자체를 방금 썼거나. 둘 다 diff 안에 있습니다. `--changed BASE` 를 넣었습니다.

- 기준 커밋 이후 바뀐 **비시험 함수**를 이름으로 지목하는 가드
- 그 diff 가 바꾼 **가드 시험** 자체
- **추적되지 않은 새 파일**도 전부 바뀐 것으로 셉니다. `git diff` 는 새 파일을 보지 않는데, 갓 쓴 가드가 가장 확인하고 싶은 대상입니다. 이걸 빼놓으면 로컬에서 조용히 아무것도 확인하지 않습니다
- 기준 커밋을 해석할 수 없으면(브랜치 첫 푸시의 0으로 채워진 값) `HEAD~1` 로 물러섭니다

만드는 동안 두 가지가 더 나왔습니다.

중단된 도구가 소스를 고친 채로 둡니다. 9분 예산에 잘렸을 때 `middleware.go` 에 변이가 남아 있었습니다. `finally` 는 신호를 받으면 실행되지 않으므로, `atexit` 과 `SIGINT`·`SIGTERM` 처리기로 되돌립니다. 이번 창에서 저는 같은 이유로 **변이가 든 파일을 원본으로 덮어쓸 뻔했고**, `git diff` 가 잡았습니다.

시간 예산은 못 본 것을 말해야 합니다. `--budget` 을 넣되, 넘겼을 때 확인하지 못한 가드 이름을 찍습니다. 아무 말 없이 끝나면 "전부 지켜집니다" 로 읽히는데, 이 도구가 절대 암시해서는 안 되는 한 가지입니다.

CI 에는 `continue-on-error` 로 넣었습니다. **변이 생존은 판정이 아니라 질문**이고, 무해한 것이 실제로 있습니다 — 이번 창에서도 하나 나왔습니다. 질문을 병합 차단으로 바꾸면 사람은 질문을 없애는 쪽으로 답하게 됩니다.

원칙으로 남깁니다. **기능을 켜는 길과 끄는 길은 서로 다른 코드이고, 시험은 대개 무서운 쪽만 지킵니다.** 그런데 배포를 멈추는 것은 대개 반대쪽입니다 — 켜지지 않는 기능, 열리지 않는 키, 다시 들어갈 수 없는 로그인.

3.10.96 v0.141.0에서 완료 — 첫 번째 갈래만 밟고 지나간 시험

v0.140.0 에서 CI 에 올린 변이 점검이 **첫 실행에서 바로 무언가를 물었습니다.** 그것도 제가 바로 앞 릴리스에서 직접 손으로 확인하고 "잡힙니다" 라고 적은 자리입니다.

문제의 줄입니다.

```
case strings.HasPrefix(r.URL.Path, "/api/v1/reports") ||
     strings.HasPrefix(r.URL.Path, "/api/v1/team/reports") ||
     strings.HasPrefix(r.URL.Path, "/api/v1/rollups") ||
     strings.HasPrefix(r.URL.Path, "/api/v1/search"):
```

제가 쓴 시험은 `/api/v1/reports` 하나만 부릅니다. 그 경로는 **첫 갈래에서 단락**되므로 뒤의 세 갈래는 실행되지 않습니다. 손으로 확인할 때 저는 첫 번째 `||` 를 바꿨고 — 그건 잡힙니다 — 도구는 두 번째와 세 번째를 바꿉니다. **그 둘은 아무것도 바꾸지 않은 것처럼 보입니다.** 제 시험이 그 갈래에 닿지 않기 때문입니다.

즉 `reports:read` 를 가진 키가 **기간 업무보고와 검색과 팀 보고서를 열 수 있는지**는 여전히 아무도 확인하지 않고 있었습니다. 범위가 이름만 있고 아무것도 열지 못해도 시험은 통과합니다.

네 경로를 모두 부르도록 고쳤고, 이제 여섯 변이가 전부 잡힙니다.

기록해 둡니다. **제 손 확인이 도구보다 약했습니다.** 저는 제가 이미 덮은 갈래를 골라 바꿨습니다 — 사람이 직접 고를 때 자연스럽게 그렇게 됩니다. 도구는 고르지 않습니다.

원칙으로 남깁니다. **하나의 조건에 갈래가 여럿이면, 시험이 첫 갈래만 밟고도 그 줄 전체를 덮은 것처럼 보입니다.** 단락 평가는 나머지를 실행하지 않고, 실행되지 않은 코드는 시험된 적이 없습니다.

예산이 가드 하나에 다 쓰이던 것

CI 첫 실행이 480초 예산으로 가드 5개 중 3개만 확인하고 2개를 남겼습니다. 남긴 것을 이름으로 말해 주었으니 도구는 정직했지만, 검사가 **가드 사이에서만** 일어나서 느린 가드 하나가 예산을 통째로 쓸 수 있었습니다. 변이 사이에서도 확인하도록 고쳤습니다.

3.10.97 v0.142.0에서 완료 — 글자만 읽고 자리는 보지 않던 슬라이드 시험

네 번째 변이 점검 묶음입니다. 첨부·검색·예측·인증·내보내기 가드 열 개에 돌렸습니다.

비밀번호가 없는 계정. `login` 의 조건을 한 글자 넓히면 `nil` 해시가 비교까지 내려가 **서버가 죽습니다.** 그 경우를 시험한 적이 없습니다 — 로그인 시험은 전부 비밀번호가 있는 계정을 씁니다. 그런데 그런 계정은 흔합니다. 가져오기나 일괄 생성으로 만든 계정이 전부 그 상태이고, **이 저장소의 규모 시드가 만드는 300개 계정이 정확히 그렇습니다.** 거부되는지, 그리고 거부가 "비밀번호가 없다" 는 사실을 흘리지 않는지 함께 지킵니다.

닿는 속도를 지목하지 않던 문장. `outlookForDueDate` 의 SPLIT 판정은 두 속도가 엇갈릴 때 **닿는 쪽을 문장 앞에 놓습니다.** 시험은 `noteHas: "전체 평균"` 으로 확인하고 있었는데, **두 문장 모두 "전체 평균" 과 "최근 속도" 를 다 담고 있습니다.** 부분 문자열로는 구별이 안 됩니다.

	문장
최근 속도가 닿음	`최근 속도(…)로는 닿지만 전체 평균(…)으로는 …%에 그칩니다`
전체 평균이 닿음	`전체 평균(…)으로는 닿지만 최근 속도(…)로는 …%에 그칩니다`

시험 옆 주석은 이미 옳은 말을 적어 두었습니다 — "닿는 쪽 속도를 문장이 지목해야 한다". 단언만 그렇지 않았습니다. **어느 쪽이 먼저 오는지**로 바꿨습니다.

겹쳐 그려지던 슬라이드. `TestTheAskReachesTheDeckAndStaysApartFromTheIssue` — 이름이 "이슈와 떨어져 있다" 라고 말합니다. 실제로는 제목과 본문 글자가 있는지만 봅니다. 이슈와 요청이 함께 있을 때 이슈 블록이 절반 높이를 쓰는 이유는 요청 블록이 들어갈 자리를 만들기 위해서인데, 그 조건을 뒤집으면 **요청이 이슈 위에 겹쳐 그려집니다.** 파일 안의 글자는 하나도 안 바뀌므로 모든 단언이 그대로 통과합니다.

슬라이드는 기하학이고, 글자만 읽는 시험은 그것을 볼 수 없습니다. 도형 좌표를 읽어 요청 블록이 이슈 블록 아래에서 시작하는지, 슬라이드 밖으로 넘어가지 않는지, 그리고 요청이 없을 때는 이슈가 더 넓게 쓰는지를 확인합니다. 이 가드를 쓰다가 정규식이 `<a:gd name="adj">` 에 걸려 좌표를 한 도형씩 밀려 읽었습니다 — `cNvPr` 에 고정했습니다.

고칠 수 없어서 기록만 하는 셋

의미 검색 갈래(`searchReports`**)**는 벡터 확장과 AI 게이트웨이가 둘 다 있어야 실행됩니다. 시험 환경에는 없으므로 그 안의 조건은 어떻게 바꿔도 보이지 않습니다. v0.81 이 이 도구를 만들며 적어 둔 바로 그 범주입니다 — "꺼져 있고 설정도 안 된 게이트웨이".

닫기 실패 갈래(`writeAttachmentFile`**)**는 파일 시스템에 이음매를 넣어야 시험할 수 있습니다. 한 줄을 시험하려고 제품 코드에 추상을 넣는 것이 그 위험보다 큽니다. 다만 그 변이가 무엇을 하는지는 적어 둡니다 — **쓰기가 실패한 뒤 닫기가 성공하면 성공으로 보고합니다.**

아무도 읽지 않는 열(`authenticate`**).** `user_sessions.last_seen_at` 은 인증된 요청마다(5분 간격으로 눌러) 갱신되는데, **읽는 곳이 코드에도 화면에도 없습니다.** 그래서 그 줄의 조건을 뒤집어도 관찰 가능한 동작이 바뀌지 않습니다 — 무해한 변이입니다. 동시에 그것은 **비용만 있고 독자가 없는 쓰기**라는 뜻이기도 합니다.

원칙으로 남깁니다. **시험이 읽지 않는 차원은 시험되지 않습니다.** 글자를 읽는 시험은 자리를 보지 못하고, 부분 문자열을 찾는 시험은 순서를 보지 못합니다. 이름은 대개 둘 다 약속합니다.

3.10.98 v0.143.0에서 완료 — 같은 팀 동료가 시험에 없었습니다

다섯 번째 변이 점검 묶음입니다. 가장 무거운 것이 하나 나왔습니다.

`canViewReport` 의 조직 갈래입니다.

```
if (p.Role == "TEAM_LEADER" || p.Role == "ORG_MANAGER") && p.OrganizationID != nil && orgID != n
```

`&&` 를 `||` 로 바꾸면 **평사원이 같은 조직 동료의 보고서를 읽습니다.** 역할 검사가 무력화되고 조직이 둘 다 설정되어 있기만 하면 하위 조직 질의가 실행되며, 같은 조직이니 통과합니다.

시험은 통과했습니다. `TestOnePersonsReportIsNotAnothersToRead` 를 열어 보니 이유가 분명했습니다 — **두 사용자를 모두 조직 없이 만듭니다.** 조직 갈래가 한 번도 실행되지 않으니 그 안을 어떻게 바꾸든 보이지 않습니다.

그런데 그 갈래가 **제품에서 가장 흔한 경우**입니다. 조직 없는 계정 둘이 아니라, 한 팀에 앉은 두 사람. 이 저장소는 v0.104 에서 소속 없는 계정을 결함으로 다루기까지 했으면서, 사생활 시험만은 그 상태를 픽스처로 쓰고 있었습니다.

같은 조직 안에서 네 가지를 확인하도록 썼습니다 — 본인은 읽고, **동료는 못 읽고**(그리고 거부가 요약을 흘리지 않고), 그 조직의 팀장은 읽고, 다른 조직의 팀장은 못 읽습니다. 거부와 허용을 함께 두어야 갈래를 통째로 꺼도 알아챕니다.

감사 기록에서 둘 더 나왔습니다. `audit` 은 실패했을 때 로그를 남기는지, 클라이언트가 떠나도 기록이 남는지는 시험되고 있었는데 **기록을 읽어 보는 시험이 없었습니다.** 사고 뒤 운영자가 필요로 하는 두 칸 — 누가 했고 어디서 왔는지 — 이 비어 있어도 아무도 몰랐습니다. 행위자가 없는 기록(서명 전 감사)도 함수를 직접 불러 지킵니다.

`issueClearanceFor` 는 정렬이 무방비였습니다. 질의가 가장 긴 것부터 돌려주고 코드가 첫 행을 집는데, 기존 시험은 **두 장애물의 기간을 똑같이 5주로** 두었습니다 — 주 수와 언급 수를 가르려는 의도였고 그건 옳습니다. 다만 그래서 어느 행을 집든 답이 같습니다. 마지막 행을 집게 바꾸면 화면이 **가장 짧은 장애물을 가장 긴 것으로** 부릅니다. 기간이 다른 경우를 넣었습니다.

기록만 하는 둘

`instant` **의 297·301** 은 Go 로직이 아니라 **SQL 문자열 상수** 안의 경계입니다(`<= now()` , `>= 시작주`). `now()` 를 고정하지 않으면 그 경계를 시험할 수 없습니다. 참여도 집계에서 "이번 주가 아직 열려 있는가" 를 가르는 자리이므로 값은 큽니다.

`appendSearchMatches` **의 213·232** 는 검색 순위 경계입니다. 순위는 여러 신호의 합이라 한 경계만 움직여도 상위 결과가 그대로일 수 있어, 무해한 변이와 진짜 구멍을 가르려면 순위 자체를 다루는 별도의 작업이 필요합니다.

원칙으로 남깁니다. **픽스처가 고르는 것은 상황이 아니라 실행되는 코드입니다.** 조직 없는 계정으로 사생활을 시험하면 사생활 규칙의 절반은 실행되지 않고, 기간이 같은 두 행으로 정렬을 시험하면 정렬은 실행되지 않습니다. 시험이 무엇을 확인하는지는 단언이 아니라 **픽스처가 정합니다.**

3.10.99 v0.144.0에서 완료 — "시험할 수 없다" 고 적어 둔 것을 다시 봤습니다

바로 앞 절에서 참여도 집계의 SQL 경계 둘을 **"`now()` 를 고정해야 시험할 수 있다"** 는 이유로 기록만 하고 넘어갔습니다. 다시 보니 그 말이 반만 맞았습니다. `now()` 를 고정할 필요가 없습니다 — **데이터를 `now()` 기준으로 놓으면** 됩니다.

그리고 둘의 성격이 서로 달랐습니다.

`deadlinePassed` **(** `<= now()` **)** 는 정말로 시험할 수 없습니다. `<` 로 바꿔도 **마감 시각과** `now()` **가 마이크로초까지 같은 순간에만** 달라집니다. 이제 "시험 못 함" 이 아니라 **이유가 있는 무해한 변이**로 기록합니다. 둘은 다릅니다 — 앞의 것은 모르는 상태이고, 뒤의 것은 아는 상태입니다.

`weekIsOwed` **(** `week.day >= 기대 시작주` **)** 는 시험할 수 있고, 무방비였습니다. `>` 로 바꾸면 **사람마다 밀린 주가 정확히 한 주씩 줄어듭니다** — 그것도 **들어온 그 주**, 신입이 가장 놓치기 쉬운 주입니다. 기존 참여도 시험은 전부 창보다 훨씬 전에 들어온 사람을 쓰므로 `>` 와 `>=` 가 같은 답을 냅니다.

제가 그 함정에 그대로 빠졌습니다

첫 번째 시험을 이렇게 썼습니다 — 4주 전에 들어온 경우와 3주 전에 들어온 경우를 재고, **차이가 정확히 1주인지** 확인. 통과했습니다. 변이를 넣어도 통과했습니다.

당연합니다. **두 값이 함께 하나씩 줄면 차이는 그대로입니다.** 상대 비교가 경계를 상쇄해 버립니다.

이번 창 내내 남의 시험에서 찾아낸 바로 그 모양을 제가 썼습니다. 같은 응답이 알려 주는 마감된 주 수와 **절대값으로** 견주도록 고쳤고, 그러자 변이가 잡힙니다.

원칙으로 남깁니다. **차이를 확인하는 시험은 두 항에 똑같이 얹힌 오차를 볼 수 없습니다.** 경계는 절대값으로 재야 합니다 — 경계를 옮기면 양쪽이 함께 움직이기 때문에, 그것이 바로 경계라는 뜻입니다.

그리고 하나 더. **"지금은 못 고친다" 는 기록은 결론이 아니라 다음에 다시 볼 자리 표시입니다.** 이번에는 하루가 아니라 한 절 만에 돌아왔고, 둘 중 하나는 사실 고칠 수 있었습니다.

3.10.100 v0.145.0에서 완료 — CI에 올린 도구가 바로 다음 커밋을 물었습니다

v0.144.0 을 밀어넣자 CI 의 변이 점검이 **그 커밋이 새로 만든 가드에서 잡히지 않는 변이 둘**을 보고했습니다. 사람이 다시 돌릴 필요가 없었습니다 — 이것이 그 단계를 CI 에 올린 이유입니다.

둘 다 0으로 나누기를 막는 자리입니다.

```
if activeUsers > 0 { item.SubmissionRate = ... }
if item.Submitted > 0 { item.OnTimeRate = ... }
```

뒤집으면 조건이 영영 참이 되지 않아 **두 비율이 언제나 0** 입니다. 곁의 숫자는 "대상 3명 중 1명 제출" 이라고 말하는데 비율만 0%로 붙습니다. 분석 화면의 두 값을 **어떤 시험도 확인한 적이 없어서**, 참여율이 전면 붕괴로 보이더라도 시험은 통과했을 것입니다.

비율이 곁의 숫자와 맞는지 확인하도록 썼습니다.

여기서 또 한 번 픽스처가 갈래를 골랐습니다. 첫 판은 정시율 쪽 변이를 잡지 못했습니다. 보고서를 지금 제출했는데 대상 주가 3주 전이라 **지각**이고, 정시 건수가 0이면 정시율은 어느 쪽이든 0이기 때문입니다. 제출 시각을 마감 안으로 옮기고 나서야 그 갈래가 실행됩니다.

바로 앞 절에서 "차이를 재면 경계가 상쇄된다" 고 적었는데, 이번 것은 다른 모양의 같은 교훈입니다 — **분자가 0인 채로 비율을 시험하면 비율 계산이 실행되지 않습니다.**

원칙으로 남깁니다. **비율을 시험하려면 분자와 분모가 모두 0이 아니어야 합니다.** 당연해 보이지만, 픽스처를 만들 때 자연스럽게 나오는 값은 대개 0입니다 — 아무도 제출하지 않았고, 아무도 정시가 아니고, 아무도 완료하지 않았습니다.

3.10.101 v0.146.0에서 완료 — 자기 사용법대로 돌릴 수 없던 부하 점검

여덟 릴리스를 시험과 도구에 썼으니 제품 쪽을 봤습니다. 이번 창에서 한 번도 돌리지 않은 것이 하나 있었습니다 — `scripts/load-check.py`, 월요일 아침을 재현하는 부하 점검입니다.

돌릴 수 없었습니다. 스크립트의 사용법이 이렇게 적혀 있습니다.

> 로그인하는 계정은 `scripts/seed-scale.sql` 이 만드는 것들입니다 — `u1..u300`, **공유 비밀번호**로.

시드는 비밀번호를 설정하지 않습니다. 그래서 문서대로 돌리려면 누군가 300개 계정에 비밀번호를 하나씩 넣어야 했고, 저는 이번 창 내내 필요할 때마다 한두 개씩 그렇게 해 왔습니다. v0.134 와 같은 계열입니다 — **도구가 재려는 상태를 시험대가 만들지 못합니다.**

시드가 공유 해시를 직접 넣도록 했습니다. 평문은 이미 `load-check.py` 안에 그대로 있으므로 저장소에 없던 비밀이 생기지는 않고, 시드는 보고서가 든 데이터베이스에서 실행을 거부합니다 — 그것이 실제로 지키는 장치입니다.

그래서 실제로 재 봤습니다

250명이 동시에 로그인하고 저장하며 50명이 읽는 상황을, 컨테이너 크기 둘에서.

컨테이너	해시 작업자	소프트 한도	최대 사용	저장 p99	조회 p99	오류	OOM
1 GiB	8명 (512 MiB 예약)	896 MiB	861.6 MiB	77ms	522ms	0건	없음
512 MiB	**5명 (320 MiB)**	448 MiB	441.4 MiB	69ms	218ms	0건	없음

작업자 수가 한도를 보고 줄고, 최대 사용량이 두 경우 다 소프트 한도 아래에 머뭅니다. Argon2id 한 번이 64 MiB 를 잡으므로 250명이 한꺼번에 들어오면 16 GiB 가 필요해지는데, 그 계산이 컨테이너를 죽인 것이 이 방어가 만들어진 계기였습니다. 지금은 큐가 한도에 맞춰지고 아무도 실패하지 않습니다.

861 MiB / 1 GiB 는 처음에 아슬아슬해 보였는데, 소프트 한도가 896 MiB 이고 수거기가 그것을 목표로 삼습니다. **아슬아슬한 것이 아니라 겨냥한 자리입니다.**

원칙으로 남깁니다. **자기 사용법대로 돌아가지 않는 도구는 돌아가지 않는 도구입니다.** 그리고 그런 도구는 조용히 안 돌아갑니다 — 아무도 돌리지 않으니까요.

3.10.102 v0.147.0에서 완료 — 파일 해시는 그 파일이 온전한지만 답합니다

앞 절의 "자기 사용법대로 안 도는 도구" 를 나머지 스크립트에도 적용해 보다가, `scripts/export-offline.sh` 가 릴리스 워크플로와 **같은 일을 따로 적어 두었다**는 것을 봤습니다. 둘이 정말 같은 것을 만드는지 재 봤습니다.

달랐습니다. 공개된 파일과, 그것을 적재한 뒤 같은 스크립트로 다시 저장한 파일의 SHA-256 이 다릅니다.

원인을 가르는 데 세 번의 측정이 필요했습니다.

물음	답	
`docker save` 가 비결정적인가?	아니오 — 같은 이미지를 두 번 저장하면 바이트가 같습니다	
두 코드 경로가 다른가?	아니오 — 둘 다 `docker save … \	gzip -9 -n` 입니다
그럼 무엇이 다른가?	**적재→저장 왕복**에서 Docker 가 자기 포장 메타데이터를 다시 씁니다	

이미지 자체는 같았습니다. 설정 다이제스트 하나와 레이어 다이제스트 셋이 **모두 동일**합니다. 달라진 것은 `repositories` 파일과 매니페스트의 `LayerSources` 블록뿐입니다.

그래서 무엇이 문제인가

릴리스 본문은 파일 SHA-256 만 싣습니다. 그것은 **"이 파일이 온전히 도착했는가"** 에 답합니다. 그런데 이 제품은 외부망이 없는 망으로 반입됩니다 — 파일이 중간 매체를 거치고, 다시 묶이고, 다시 복사됩니다. 그 과정에서 **파일 해시는 정당하게 달라지고, 그러면 운영자에게는 확인할 방법이 남지 않습니다.**

이미지 ID 는 그 여정을 견딥니다. 직접 재 봤습니다.

	왕복 전	왕복 후
이미지 ID	`62a7bcaf…`	**`62a7bcaf…` 그대로**
압축 파일 해시	`40bfda50…`	`2dfa406b…`

릴리스 본문과 `export-offline.sh` 와 README 세 곳이 이미지 ID 를 함께 싣도록 했습니다. **두 확인은 서로 다른 질문에 답합니다** — 반입 전에는 파일이 온전한지를, 적재 뒤에는 그것이 빌드된 이미지가 맞는지를.

검증 절차를 문서에 싣기 전에 그 절차가 실제로 왕복을 견디는지 먼저 시험했습니다. **검증하지 않은 검증 절차를 싣는 것은 확인이 아니라 안심입니다.**

원칙으로 남깁니다. **무엇을 확인하느냐보다 그것이 무엇에 답하느냐가 먼저입니다.** 파일 해시는 전송을 확인하고 신원을 확인하지 않습니다. 둘을 같은 것으로 쓰면, 전송이 정상적으로 달라졌을 때 확인이 통째로 사라집니다.

3.10.103 v0.148.0에서 완료 — 바로 앞 릴리스가 실은 확인 명령이 틀렸습니다

v0.147.0 은 오프라인 반입에 이미지 신원을 함께 싣고, 운영자에게 이렇게 확인하라고 적었습니다.

```
docker image inspect weekly:v0.147.0 --format '{{.Id}}'
```

그 명령이 틀렸습니다. 릴리스를 내려받아 그대로 해 봤습니다.

	값
파일 SHA-256	릴리스가 적은 값과 **일치**
`docker image inspect .Id`	`fcf4eb5d…`
릴리스가 적은 값	**`da567938…`**

지시를 따른 운영자는 자기 이미지가 빌드된 것이 아니라고 결론지었을 것입니다. 맞는데도요.

왜 틀렸는가

제가 v0.147.0 에서 한 검증은 **양쪽 끝이 모두 "적재된 이미지"** 였습니다. 아카이브를 적재하고, 저장하고, 다시 적재해서 ID 가 그대로인 것을 확인했습니다. 그런데 실제 여정은 **CI에서 빌드 → 저장 → 반입 → 적재** 이고, Docker 는 **빌드한 이미지와 적재한 이미지에 다른 식별자를 매깁니다.**

즉 저는 여정의 한 구간만 재고 전체를 잰 것처럼 적었습니다. 바로 앞 절에 "검증하지 않은 검증 절차를 싣는 것은 확인이 아니라 안심" 이라고 써 놓고서.

무엇이 실제로 견디는가

아카이브 안의 **설정 다이제스트**입니다. 공개된 값 `da567938…` 이 바로 그것이고 — 즉 **값은 처음부터 옳았고 읽는 방법만 틀렸습니다** — 세 가지를 거쳐도 움직이지 않습니다.

	파일 해시	아카이브 다이제스트
공개된 파일	`2399169b…`	`da567938…`
적재 후 다시 저장	`aba6b907…`	**`da567938…`**
풀었다 다시 묶음	`a09fe7ff…`	**`da567938…`**

Docker 에게 묻지 않고 아카이브에서 직접 읽도록 세 곳을 고쳤습니다.

```
gzip -dc weekly-vX.tar.gz | tar -xO manifest.json | grep -o "blobs/sha256/[0-9a-f]\{64\}" | head
```

`docker image inspect --format '{{.Id}}'` 는 확인에 쓰지 말라고 명시했습니다. 값이 다른 것이 정상이기 때문입니다.

원칙으로 남깁니다. **여정의 한 구간을 재고 전체를 잰 것처럼 말하지 않습니다.** 왕복이 견딘다는 것과 편도가 견딘다는 것은 다른 주장이고, 이번 경우에는 앞의 것만 참이었습니다.

3.10.104 v0.149.0에서 완료 — 배포가 안 뜨는 순간에 읽는 유일한 문장

처음 배포하는 운영자가 겪는 경로를 밟아 봤습니다. 기동이 거부되는 경우는 셋입니다.

상황	이전 메시지
필수 변수 누락	`WEEKLY_POSTGRES_DSN, WEEKLY_BOOTSTRAP_ADMIN and WEEKLY_BOOTSTRAP_ADMIN_PASSWORD are required`
비밀번호 12자 미만	`WEEKLY_BOOTSTRAP_ADMIN_PASSWORD must be at least 12 characters`
DSN 오류	PostgreSQL 드라이버 메시지 (호스트·DB·SQLSTATE 포함)

세 번째는 드라이버의 것이고 실제로 유용합니다 — 그대로 둡니다. 앞의 둘은 **제품이 직접 쓰는 문장**입니다.

이 패키지의 다른 모든 오류는 영어여도 됩니다. 로그로만 남거나, HTTP 응답이 되면서 한국어 문장으로 감싸이기 때문입니다. **이 둘만 사람이 직접 읽습니다** — 설치가 안 되는 순간에, 콘솔에서, 운영자가. 제품의 나머지가 전부 한국어로 말하고 "다음에 무엇을 하라" 까지 말하는데 이 둘만 그렇지 않았습니다.

그리고 **없는 것만 말하지 않았습니다.** 하나가 빠져도 셋을 전부 나열하니, 운영자는 멀쩡한 둘까지 확인하러 갑니다.

```
WEEKLY_POSTGRES_DSN, WEEKLY_BOOTSTRAP_ADMIN_PASSWORD 환경변수가 없습니다.
deploy/.env.example 을 복사해 채운 뒤 --env-file 로 넘기십시오

WEEKLY_BOOTSTRAP_ADMIN_PASSWORD 는 12자 이상이어야 합니다.
첫 관리자 계정의 비밀번호이며, 기동한 뒤 화면에서 바꿀 수 있습니다
```

복제의 두 모드

여섯 번째 변이 점검 묶음에서 `cloneReport` 의 네 변이가 살아남았습니다. 복제에는 모드가 둘이고 그 차이가 기능의 전부입니다 — `STRUCTURE` 는 업무 이름만 다음 주로 옮기고, `FULL` 은 지난주 글까지 옮겨 고쳐 쓰게 합니다. 도착한 모드는 **아무것도 안 보냈을 때만** 기본값으로 채우는데, 그 비교를 뒤집으면 **모든 `FULL` 요청이 조용히 `STRUCTURE` 가 됩니다.** 작성자는 다음 주를 열어 자기 글이 없어진 것을 보고, 어디에도 이유가 적혀 있지 않습니다.

여기서 제 기대가 틀렸습니다

이미 보고서가 있는 주차로 복제하는 경우를 시험하며 `REPORT_EXISTS` 를 기대했습니다. 제가 보고 있던 갈래가 그 코드를 내기 때문입니다. **제품은 이미 더 나은 것을 하고 있었습니다** — 더 앞선 검사가 `REPORT_PERIOD_OVERLAPS` 로, 주차를 말하고 무엇을 할지("그 보고서를 여십시오")까지 답합니다.

즉 그 `23505` 갈래는 앞선 검사가 막아 주는 **두 번째 층**이고, 그래서 어떤 변이도 잡히지 않습니다. 시험을 **작성자가 실제로 읽는 문장**에 맞췄습니다 — 코드 경로가 아니라.

원칙으로 남깁니다. **시험이 기대해야 할 것은 코드가 내는 값이 아니라 사람이 받는 것입니다.** 제가 보고 있던 갈래를 기준으로 기대를 세우면, 제품이 더 나은 답을 하고 있어도 그것을 결함으로 읽게 됩니다.

아직 안 본 것

같은 묶음의 `adminUsers`(348·354), `queryReports`(827), `uploadImportPPTX`(158) 는 다음으로 넘깁니다. `authenticate`(92) 는 v0.142 에서 이미 무해한 변이로 판정한 자리입니다 — `last_seen_at` 은 아무도 읽지 않습니다.

3.10.105 v0.150.0에서 완료 — 제가 CI를 잘못 읽고 있었습니다

v0.149.0 을 밀어넣고 CI 를 확인했더니 "점검할 가드 0개" 였습니다. 그 커밋은 `config.go` 를 고치고 가드 시험 둘을 새로 추가했으니 0일 수 없습니다.

제가 이전 커밋의 실행을 읽고 있었습니다.

기다리는 방법이 이랬습니다 — "완료된 가장 최근 CI 실행을 가져와라". 새 실행이 아직 대기열에 들어가기 전이면 **그 자리에는 직전 커밋의 실행이 있습니다.** 완료 상태이고, 성공이고, 그럴듯합니다.

이번 창에서 같은 곳을 두 번 고쳤습니다. 처음에는 워크플로 이름이 없어 GitHub Pages 빌드를 CI 로 읽고 있었고, 이번에는 이름은 맞는데 **커밋이 틀렸습니다.** 첫 번째를 고칠 때 커밋까지 고정했어야 했습니다.

이제 `headSha` 로 이번 커밋의 실행을 지정합니다. 지난 릴리스들의 "CI 통과" 라는 결론 자체는 유효합니다 — 실행 목록을 커밋별로 다시 확인해 전부 `completed success` 임을 봤습니다. 다만 **그때그때 제가 읽은 것이 그 커밋의 실행이었다는 보장은 없었습니다.**

원칙으로 남깁니다. **"가장 최근" 은 식별자가 아닙니다.** 무엇을 확인했는지 말하려면 그것을 이름으로 붙들어야 하고, 시간 순서는 이름이 아닙니다.

그래서 실제 결과는

가드 4개 · 변이 16개 · **잡히지 않은 변이 6개** 였습니다. 셋은 같은 자리의 다른 형태이고, 그중 하나는 **이력에 남는 문구**를 고르는 줄입니다.

```
comment := "보고서 #… 에서 업무 구조 복제"
if input.Mode == "FULL" {
    comment = "보고서 #… 에서 전체 내용 복제"
}
```

뒤집으면 **보고서 이력이 자기가 어떻게 생겼는지 반대로 말합니다.** 작성자는 지난주 글이 넘어왔다고 읽고, 복사된 적 없는 글을 찾으러 갑니다. 아무 말도 없는 것보다 나쁩니다.

나머지 셋(`535`)은 v0.149.0 에 적은 대로 **앞선 검사가 막아 주는 두 번째 층**이라 도달할 수 없습니다.

3.10.106 v0.151.0에서 완료 — 중단된 실행이 남기고 간 409 MB

작업 공간을 정리하다 PostgreSQL 에 하네스가 만든 데이터베이스가 남아 있는 것을 봤습니다.

`weekly_tmpl_*`	36개
`weekly_h_*`	7개
합계	**409 MB**
붙어 있는 연결	**0개**

v0.128 이 이 문제를 이미 한 번 다뤘습니다. 그때는 템플릿을 아무도 지우지 않아 63개가 쌓였고, `TestMain` 에서 지우도록 고쳤습니다. **정상 종료 경로를요.**

중단된 실행은 거기 닿지 않습니다. 그리고 중단은 특별한 일이 아닙니다 — Ctrl-C, 편집기의 정지 단추, 시간 초과, 그리고 이번 창에서는 `scripts/mutation-check.py` 가 더 필요 없어진 시험 실행을 계속 죽였습니다. **한 번의 작업 세션에서 43개가 쌓였습니다.**

v0.140 에서 그 도구가 중단돼도 소스를 되돌리도록 `atexit` 과 신호 처리기를 붙이며 같은 것을 배웠습니다. **깨끗한 종료에서만 일어나는 정리는 일어나지 않습니다.** 그때는 도구 쪽만 고쳤고, 도구가 죽이는 쪽은 그대로였습니다.

이름이 나이를 말하게

정리하려면 무엇이 버려진 것인지 알아야 하는데, 기존 이름은 알려 주지 않았습니다. `harnessRun` 은 `UnixNano() % 1e9` — 나노초의 **하위 부분**이라 사실상 난수입니다.

연결 수로 판단할 수도 없습니다. `CREATE DATABASE … TEMPLATE x` 는 x 에 다른 연결이 없기를 요구하므로 **템플릿은 쓰이는 사이사이 비어 있습니다.** "아무도 안 붙어 있으면 버려진 것" 이라고 하면 옆에서 돌고 있는 다른 실행의 멀쩡한 템플릿을 지웁니다.

그래서 이름에 초 단위 시각을 넣고, 각 실행이 시작할 때 **두 시간보다 오래된 것**을 쓸어냅니다. 옆 실행이 아무리 느려도 건드리지 않을 만큼의 여유입니다.

심어 두고 확인했습니다 — 3시간 전 이름 둘은 사라지고 방금 것은 남았습니다. 실제 정리에서 43개 · 409 MB 가 0이 되었습니다.

이름을 읽는 함수에 가드를 달았습니다. **그 함수 하나가 버려진 데이터베이스와 살아 있는 것 사이에 서 있기 때문입니다** — 살아 있는 실행의 이름을 오래된 것으로 읽으면, 옆 실행이 막 복사하려던 템플릿을 지웁니다.

원칙으로 남깁니다. **자원을 만든 쪽이 지우게 하면, 그 쪽이 죽는 경우에 아무도 지우지 않습니다.** 다음 사람이 지울 수 있으려면, 남겨진 것이 자기가 언제 남겨졌는지 말할 수 있어야 합니다.

3.10.107 v0.152.0에서 완료 — 태그는 있는데 릴리스가 없었습니다

v0.151.0 을 올린 뒤 확인해 보니 **v0.150.0 은 태그와 푸시까지 끝났는데 릴리스가 없었습니다.** GitHub 이 그 커밋의 실행을 대기열에 넣은 채 한 시간 넘게 시작하지 않았고, 그 사이 뒤에 올린 v0.151.0 은 정상적으로 게시됐습니다. Actions 는 켜져 있었고 태그도 원격에 있었으니 제 쪽 문제가 아니었습니다.

릴리스 워크플로가 **태그 푸시로만** 돌기 때문에, 실행이 멈추면 복구 수단이 **태그를 지웠다 다시 쓰는 것**뿐이었습니다. 파이프라인을 고치려고 이미 공개된 참조를 다시 쓰는 것은 나쁜 거래입니다.

`workflow_dispatch` 로 태그 이름을 받아 같은 태그를 다시 만들 수 있게 했습니다. 만들면서 두 가지가 더 필요했습니다.

- **태그 이름을 job 수준에 한 번만 정의합니다.** dispatch 는 브랜치에서 돌므로 `github.ref_name` 을 읽는 단계는 브랜치 이름을 태그로 알고 엉뚱한 것을 만듭니다. 처음 고칠 때 한 단계를 놓쳐 `RELEASE_TAG` 가 빈 값이 되는 상태를 만들었고, diff 를 읽다 잡았습니다.
- **체크아웃도 그 태그를 봅니다.** 아니면 `VERSION` 과 릴리스 본문이 main 의 현재 상태에서 나옵니다.

그리고 실제로 v0.150.0 을 다시 만들어 확인했습니다 — 파일 해시와 아카이브 다이제스트가 릴리스에 적힌 값과 정확히 일치합니다. **복구 장치를 그것이 만들어진 바로 그 문제에서 시험했습니다.**

그 사이 CI 가 실패하고 있었습니다

v0.151.0 의 CI 가 `guard-check` 에서 떨어졌습니다. 제가 v0.151.0 에서 하네스 정리 함수에 `// guards: harnessNameStamp` 표시를 붙였는데, **그 표시는 제품 함수 전용**입니다. `guard-check.py` 는 커버리지 프로필로 실행 여부를 재고, Go 는 `_test.go` 안의 코드에 대한 커버리지를 보고하지 않습니다. 그래서 시험이 무엇을 하든 "한 번도 실행되지 않았다" 고 나옵니다.

표시를 걷어내고 왜 붙이면 안 되는지 그 자리에 적었습니다. 시험 자체는 그대로입니다 — 쓸어내기의 판단을 지키는 일은 변하지 않았습니다.

원칙으로 남깁니다. **밀어넣은 것과 릴리스된 것은 다른 일입니다.** 이번 창에서 저는 "밀어넣었습니다" 라고 보고하고 확인이 끝나기 전에 다음 반복으로 넘어간 적이 있고, 그동안 태그 하나가 산출물 없이 남아 있었습니다.

3.10.108 v0.153.0에서 완료 — 태그에 릴리스가 붙어 있는지 세는 검사

바로 앞 절에서 v0.150.0 이 태그만 남고 릴리스가 없었던 것을 고쳤습니다. 고치고 나서 남는 질문은 하나입니다 — **어떻게 알았나.**

우연히 알았습니다. 다른 것을 확인하다 릴리스 목록을 보고, 두 판올림이 지난 뒤에야 눈에 들어왔습니다. 그때까지 그 태그는 **완성된 것처럼 보이는 채로** 있었습니다.

`scripts/release-check.sh` 를 만들었습니다. 모든 `v*` 태그에 릴리스가 있는지 세고, 없으면 어느 태그인지와 다시 만드는 명령을 함께 말합니다.

```
릴리스 검사: 통과 — 태그 156개가 모두 릴리스를 가지고 있습니다
(최근 1시간 내 태그 2개는 아직 만들어지는 중일 수 있어 건너뜀).
```

한 시간의 유예가 필요합니다. 릴리스 워크플로는 CI 와 **나란히** 돌지 앞서 돌지 않으므로, 가장 최근 태그는 정당하게 아직 만들어지는 중입니다.

만들자마자 실패하는지 확인했습니다. 릴리스가 없는 가짜 태그를 로컬에 만들어 돌렸더니 종료 코드 1로 떨어지며 그 태그를 지목했습니다. 통과만 하는 검사는 이번 창에서 제가 계속 찾아낸 바로 그것입니다 — 자기가 만든 검사를 그렇게 두지 않는 것이 최소한입니다.

CI 에는 **차단하지 않는 단계**로 올렸습니다. 과거에 밀어넣은 태그에 대한 보고이지 이번 변경에 대한 판정이 아니고, GitHub API 를 읽으므로 일시적인 통신 실패가 빨간불이 되어서는 안 됩니다. 변이 점검과 같은 자리입니다.

전체를 한 번 훑었습니다 — **태그 158개, 릴리스 158개.** 지금은 빠진 것이 없습니다.

원칙으로 남깁니다. **한 번 우연히 발견한 것은 다음에도 우연에 기댑니다.** 고침과 함께 그것을 세는 방법을 남기지 않으면, 같은 일이 다시 생겼을 때 알아채는 것은 또 운입니다.

3.10.109 v0.154.0에서 완료 — 300명 중에서 사람을 찾는 방법

여섯 번째 변이 점검 묶음에서 기록만 하고 넘어갔던 세 자리를 처리했습니다.

관리자 사용자 목록의 두 조건입니다.

```
if query != "" { …이름·아이디·이메일로 좁힘… }
if len(roles) > 0 { …역할로 좁힘… }
```

어느 쪽이든 뒤집으면 **모든 조회가 전원을 돌려줍니다.** 200으로 답하고, 그럴듯한 목록을 담고, 아무것도 좁히지 않습니다. 300명 배포에서 검색 상자는 관리자가 사람을 찾는 유일한 수단이고, v0.129 가 그 이유로 넣은 것입니다. `TestAnUnknownOrganizationFilterIsIgnoredRatherThanObeyed` 는 같은 함수를 지키지만 **조직 필터만** 봅니다.

아무와도 맞지 않는 검색이 **아무도** 돌려주는지도 함께 확인합니다 — 전원을 돌려주는 것과 구별되는 유일한 지점입니다.

끝나지 않는 가져오기 작업이 두 번째입니다.

```
if queued == 0 { …작업을 FAILED/PARTIAL 로 닫음… } else { …작업자를 깨움… }
```

모든 파일이 거부되면 처리할 것이 없으므로 그 자리에서 닫아야 합니다. 뒤집으면 반대가 됩니다 — 할 일이 있는 작업은 시작하기도 전에 닫히고, **할 일이 없는 작업은 영영 열려 있습니다.** 올린 사람은 움직이지 않을

진행 표시를 바라보고, 왜인지 알려 주는 것은 아무 데도 없습니다.

이 갈래는 **AI 게이트웨이 스위치 뒤에** 있어서 그냥은 닿지 않습니다. 시험에서 설정을 켜야 실행되는데, 게이트웨이를 부르지는 않습니다 — 거부된 파일은 요청을 읽는 동안 이미 거부되기 때문입니다. v0.81 이 적어 둔 "꺼진 기능이 검사를 가린다" 는 경우이고, 이번에는 **켤 수 있었습니다.**

닿지 않아 기록만 하는 둘

`queryReports` 의 조건은 운영자 로그 한 줄만 좌우합니다. 응답은 `total` · `limit` · `offset` 을 어느 쪽이든 그대로 담습니다.

가져오기의 `PARTIAL` 갈래는 **모든 파일이 실패한 경우엔** `FAILED` **갈래가 먼저 맞습니다.** 거기 닿으려면 일부는 실패하고 일부는 중복이어야 합니다 — 중복은 실패로도 대기로도 세지 않는 세 번째 경로입니다. 갈래가 죽은 것은 아니고, 제 픽스처가 닿지 못한 것입니다.

원칙으로 남깁니다. **좁히는 코드는 넓어져도 답을 돌려줍니다.** 그래서 필터의 시험은 "맞는 것이 있는가" 로는 부족하고 **"맞지 않는 것이 없는가" 를 함께 물어야** 합니다.

3.10.110 v0.155.0에서 완료 — 관리자가 낮춘 한도를 올려 버리는 보호

먼저, 인가 표면은 그대로입니다

`authz-check.py` 를 다시 돌렸습니다. 이 도구가 v0.120~v0.129 에서 무방비 인가 거부 27곳을 찾아냈고, 그 뒤로 저는 `updateUser` · `updateDecision` · `canViewReport` · `apiKeyRequestAllowed` 를 손댔습니다. 도구 자신이 "인가 표면이 바뀌면 돌리라" 고 적어 두었으니 지금이 그때였습니다.

거부 36곳 중 무방비 3곳 — v0.129 에 기록한 그 셋과 같습니다. 둘은 검증된 OIDC 공급자가 있어야 닿고, 하나는 미들웨어가 이미 막은 뒤의 두 번째 층입니다. **새로 생긴 구멍은 없습니다.** 이것이 이 점검의 값입니다 — 고친 것을 보여 주는 게 아니라, 고치는 동안 무너진 것이 없음을 보여 줍니다.

그리고 CI가 제 시험을 바로 물었습니다

v0.154.0 을 올리자 CI 의 변이 점검이 **그 커밋이 만든 시험에서 잡히지 않는 변이 둘**을 보고했습니다.

하나는 제 단언이 얕았습니다. 모든 파일이 거부된 가져오기가 "닫혀 있는지" 만 봤는데, 닫히는 방식이 둘입니다.

```
CASE WHEN $2 >= total_files THEN 'FAILED'
     WHEN $2 > 0            THEN 'PARTIAL'
     ELSE                        'READY' END
```

`>=` 를 `>` 로 바꾸면 **한 건도 들어가지 않은 가져오기가 "부분 성공" 으로 남습니다.** 올린 사람이 화면에서 읽는 것이 바로 그 낱말이고, 제 시험은 PENDING 도 PROCESSING 도 아니라는 것까지만 확인하고 있었습니다.

다른 하나는 방향이 뒤집히는 보호입니다.

```
if maximumRequest > 500<<20 { maximumRequest = 500 << 20 }
```

관리자의 `파일 수 × 파일 크기` 설정에서 계산한 요청 상한을 **500 MB 아래로 누르는** 장치입니다. 비교를 뒤집으면 **올리는** 장치가 됩니다 — 가져오기를 작은 파일 하나로 묶어 둔 배포가 500 MB 짜리 본문을 받아들이고, 관리자가 일부러 낮춘 값이 아무 뜻도 없어집니다.

한도를 1 MB로 낮추고 8 MB를 보내 확인했습니다. 거부되고, 거부된 업로드가 파일 기록을 남기지 않는 것까지 봅니다.

원칙으로 남깁니다. **상한을 내리는 코드와 올리는 코드는 한 글자 차이이고, 둘 다 "한도를 적용한다" 처럼 보입니다.** 그래서 한도의 시험은 설정한 값보다 **큰 것을 보내 거부되는지**로 물어야 합니다 — 작은 것이 통과하는지로는 둘을 구별할 수 없습니다.

3.10.111 v0.156.0에서 완료 — 방금 자기가 쓴 기록과 어긋나던 응답

CI 의 변이 점검이 가져오기 응답의 한 줄을 계속 지목했습니다.

```
"status": map[bool]string{true: "PENDING", false: "READY"}[queued > 0]
```

무엇을 지키는 줄인지 보려고 실제 응답을 찍어 봤습니다. **모든 파일이 거부된 업로드에 대해 응답이 `READY` 라고 답합니다.** 같은 요청이 방금 저장한 상태는 `FAILED` 입니다.

`READY` 는 이 제품에서 가져오기가 **성공했을 때** 붙는 낱말입니다. 한 건도 들어가지 못한 사람이 성공 통보를 받고, 진실은 작업을 따로 열어야 보입니다.

원인은 **같은 사실을 두 번 계산한 것**입니다. 상태는 SQL 의 `CASE` 가 정하고, 응답은 "대기열에 넣은 것이 있었나" 로 다시 정합니다. 두 계산이 갈리는 지점이 하필 가장 중요한 경우였습니다.

`RETURNING status` 로 **방금 쓴 값을 그대로 돌려주도록** 고쳤습니다. 두 번째 계산이 없어졌으니 어긋날 자리도 없습니다.

닿을 수 없다고 적었던 갈래

v0.154.0 에서 `PARTIAL` 갈래를 "일부는 실패하고 일부는 중복이어야 닿는다" 며 기록만 했습니다. 응답이 저장된 상태를 그대로 말하게 되자 그 경우를 만들 수 있게 됐습니다 — 같은 파일을 두 번 올리고, 두 번째에 빈 파일을 함께 보냅니다. 하나는 중복이라 대기열에도 실패에도 세지 않고, 하나는 거부됩니다.

전부 실패도 아니고 전부 통과도 아닌 세 번째 결과이고, 그것이 그 갈래가 설명하는 상황입니다. 이미 있는 파일 때문에 "실패" 라고 들으면, 올린 사람은 아무 문제 없는 파일에서 문제를 찾게 됩니다.

그리고 제 손이 미끄러졌습니다

변이를 되돌리려고 `git checkout -- internal/app/imports.go` 를 했는데, **그 파일에는 아직 커밋하지 않은 제 고침도 들어 있었습니다.** 변이와 함께 사라졌고, `git diff --stat` 이 빈 줄을 내놓아서 알았습니다.

이번 창에서 같은 종류를 세 번째 만납니다 — 되돌리기는 **무엇을 되돌리는지** 알고 해야 합니다. 파일 전체를 되돌리는 명령은 그 파일에 있는 다른 모든 것도 되돌립니다.

원칙으로 남깁니다. **같은 사실을 두 곳에서 계산하면, 둘이 갈리는 경우가 반드시 생기고 그 경우는 대개 가장 중요한 것입니다.** 한 번 쓰고 그것을 읽는 편이 언제나 짧습니다.

3.10.112 v0.157.0에서 완료 — 빈 것을 못 보던 빈 검사

CI 의 변이 점검이 일곱 개를 한꺼번에 내놓았고, 전부 같은 시험 아래 있었습니다 — `TestWithNoAIGatewayEveryAIPathSaysSoRatherThanFailing`. 그 시험은 **게이트웨이가 꺼져 있을 때 모든 AI 경로가 실패 대신 사정을 말하는지**를 봅니다. 문입니다. 문 뒤는 한 번도 실행된 적이 없습니다.

v0.154.0 에서 배운 것을 여기에도 썼습니다 — **게이트웨이 없이 스위치만 켤 수 있습니다.** 연결을 즉시 거절하는 주소를 넣으면 입력 검사까지는 실행되고 그 앞에서 멈춥니다.

`parseAIText` 의 입력 검사가 그렇게 드러났습니다. 빈 글, 공백뿐인 글, 5만 자를 넘는 글이 거부되는지 — 그리고 **한도 안의 글은 거부되지 않는지**를 함께 봅니다. 뒤의 것이 없으면 전부 거부하는 검사도 통과합니다. 다섯 변이가 모두 잡힙니다.

빈 것을 볼 수 없던 검사

`suggestDecisions` 는 업무에 적힌 것이 없으면 모델을 부르지 않고 빈 결과와 안내를 돌려줍니다. 그런데 그 경우를 만들어 보니 **게이트웨이까지 갔습니다.**

조립하는 쪽을 보니 이유가 분명했습니다.

```
builder.WriteString("## " + week.Format("2006-01-02") + " 주차\n")
for _, field := range …{ if 값이 있으면 쓴다 }
```

주차 머리글을 내용과 무관하게 먼저 씁니다. 그래서 모든 칸이 비어 있어도 글자는 남고, "적힌 것이 없다" 는 검사가 참이 될 수 없습니다. 아무도 쓰지 않은 업무가 **날짜 목록을 읽히려고 호출 비용을 치릅니다.**

내용이 있는 주차만 머리글을 쓰도록 고쳤습니다. 검사가 이제 자기가 하려던 일을 합니다.

원칙으로 남깁니다. **비어 있는지 묻는 검사는, 묻기 전에 무언가를 써 넣는 코드보다 뒤에 있으면 아무것도 묻지 못합니다.** 검사가 통과한 적이 없다는 것은 그 검사가 옳다는 뜻도, 필요 없다는 뜻도 아닙니다 — 대개는 볼 수 없는 자리에 있다는 뜻입니다.

다음 자리

`suggestDecisions` 의 나머지 둘 — 빈 제목 후보 걸러내기, 후보 수 상한 — 은 **AI 가 실제로 후보를 돌려줘야** 닿습니다. 시험 안에 가짜 게이트웨이를 세우면 그 영역이 통째로 열립니다. 다음으로 둡니다.

3.10.113 v0.158.0에서 완료 — 답하는 게이트웨이를 세웠습니다

지난 절에서 "다음으로 둔다" 고 적은 것을 했습니다. 시험 안에 **답하는 AI 게이트웨이**를 세웠습니다. 엔드포인트가 POST 대상 그대로 쓰이므로 로컬 서버 하나면 됩니다.

이것이 열어 준 영역이 넓습니다. 지금까지 AI 관련 시험은 두 가지뿐이었습니다 — **꺼 두고 정중히 거절하는지 보거나, 켜 두고 닿지 않는 주소를 주거나.** 둘 다 문까지입니다. **모델이 답한 뒤부터 사람이 읽기까지 사이의 모든 처리**는 한 번도 실행된 적이 없었고, 마음대로 뒤집어도 아무도 몰랐습니다.

모델의 답을 다루는 네 가지

지키는 것	뒤집으면
제목이 빈 후보는 결정 제안에서 뺀다	빈 줄이 결정 후보로 화면에 오른다
후보는 상한(5)까지만	게이트웨이가 보낸 만큼 다 올라간다
한 칸에 사실 하나, 글머리표 없이	`- 첫째\n- 둘째` 가 보고서 한 줄이 된다
규약을 지킨 답은 그대로 보고서가 된다	검사가 전부 거부해도 통과처럼 보인다

마지막 것이 없으면 앞의 셋이 무의미합니다. **거부만 확인하는 시험은 전부 거부하는 코드와 구별되지 않습니다** — 이번 창에서 반복해 만난 모양입니다.

한 규칙의 두 반쪽

원자성 검사는 조건이 둘입니다.

```
if len(parts) != 1 || marked { …거부… }
```

처음 쓴 시험은 `- 첫째\n- 둘째` 를 보냈는데, 그것은 **두 조건을 동시에** 어깁니다. 그래서 `||` 를 `&&` 로 바꿔도 여전히 거부되고, 시험은 통과합니다.

한쪽씩만 어기는 두 경우를 넣었습니다 — **글머리표 하나에 사실 하나**(`- 큐를 둘로 나눴습니다`)와 **글머리표 없이 사실 둘**(줄바꿈으로만 나눔). 각각이 실제로 나쁜 답이기도 합니다: 앞의 것은 글머리표가 보고서에 그대로 박히고, 뒤의 것은 한 칸에 두 가지가 들어갑니다.

원칙으로 남깁니다. **두 조건을 한 번에 어기는 예시는 그 둘이 `그리고` 인지 `또는` 인지 말해 주지 않습니다.** 규칙에 반쪽이 둘이면 시험도 둘이어야 합니다.

3.10.114 v0.159.0에서 완료 — 배경 작업자와 경주하던 제 시험

v0.158.0 의 CI 가 실패했습니다. **로컬에서 통과한 시험이 CI 에서 떨어졌습니다** — v0.156.0 이 추가한 `PARTIAL` 시험입니다.

```
일부 실패·나머지 중복인데 상태가 "PROCESSING" 입니다
```

중복이어야 할 파일이 중복으로 잡히지 않고 대기열에 들어갔습니다. 중복 판정의 조건을 보면 이유가 나옵니다.

```
WHERE j.user_id=$1 AND f.file_hash=$2 AND f.status NOT IN ('FAILED','SKIPPED')
```

제 픽스처는 같은 파일을 **먼저 업로드해** 중복의 원본을 만들었습니다. 그런데 업로드된 작업은 곧바로 **배경 작업자**가 집어 갑니다. 그 환경에는 진짜 게이트웨이가 없으니 처리가 실패하고, 파일 상태가 `FAILED` 로 바뀝니다. **`FAILED` 인 파일은 무엇의 중복도 아닙니다.**

로컬에서는 작업자가 거기까지 가기 전에 두 번째 업로드가 끝났고, CI 에서는 아니었습니다. **타이밍에 기대는 픽스처가 정확히 이렇게 생겼습니다** — 한쪽에서 통과하고 한쪽에서 떨어지며, 어느 쪽도 제품에 대해서는 아무 말도 하지 않습니다.

원본을 **직접 등록**하도록 고쳤습니다. 작업자가 집어 가지 않는 완료 상태의 작업에 파일 한 줄을 넣습니다. 열 번 반복해도 흔들리지 않고, `PARTIAL` 갈래 변이는 여전히 잡힙니다.

원칙으로 남깁니다. **픽스처가 제품의 배경 동작을 거치면, 그 시험은 제품이 아니라 경주 결과를 잽니다.** 만들려는 상태가 있으면 그 상태를 만들어야지, 그 상태로 이어질 것 같은 행동을 하고 기다리면 안 됩니다.

그리고 하나 더. **릴리스는 게시됐는데 CI 는 빨간불이었습니다.** 두 워크플로가 나란히 돌기 때문이고, 그래서 릴리스가 있다는 것이 CI 가 통과했다는 뜻은 아닙니다 — v0.152.0 에서 배운 것의 반대 방향입니다. 그때는 태그가 있어도 릴리스가 없을 수 있었고, 이번에는 릴리스가 있어도 CI 가 떨어질 수 있습니다.

3.10.115 v0.160.0에서 완료 — CI는 게시를 막을 수 없습니다

v0.158.0 에서 **CI 는 빨간불인데 릴리스가 나갔습니다.** 처음에는 두 워크플로가 나란히 도는 탓이라고만 적었는데, 그 문장을 다시 읽으니 더 큰 것이 들어 있었습니다.

CI 와 릴리스는 같은 푸시에서 동시에 시작합니다. 그러므로 CI 의 어떤 검사도 게시를 막지 못합니다. 순서가 없으니 관문도 아닙니다.

그러면 릴리스가 스스로 확인하는 것은 무엇인가 — 보니 이랬습니다.

```
go test ./...
go vet ./...
npm --prefix frontend ci
npm --prefix frontend run lint
```

제품의 절반인 프런트엔드의 시험이 없습니다. 린터는 돌리고 시험은 돌리지 않습니다. CI 가 돌리니까요 — 그런데 CI 는 게시를 막지 못합니다.

그 시험들이 존재하는 이유가 CI 주석에 적혀 있습니다. *"제품의 절반이 프런트엔드인데 실행 가능한 검증이 하나도 없었다. `presentSlides` 는 회의에서 발표자가 화면에 무엇을 띄울지 정하고, 그것은 PPTX 내보내기와 같은 산출물이다."*

무엇이 게시 관문에 속하는가

산출물이 옳은지를 정하는 검사는 릴리스로 옮겼습니다.

검사	왜 게시 시점인가
프런트엔드 시험	게시되는 것의 절반
`version-check.sh`	배포 파일이 운영자가 받을 이미지 태그를 담고 있음
`openapi-check.py`	문서가 아카이브와 함께 나감
`modal-close-check.py`	같은 빌드의 화면 불변식

CI 에 남긴 것은 **시험을 판정하는** 검사들입니다 — `guard-check`(20분), `mutation-check`, `release-check`. 이들은 산출물이 아니라 우리가 그것을 얼마나 믿을 수 있는지를 말하고, 느리며, 빨간불이 병합을 막는 자리에 있는 편이 맞습니다.

원칙으로 남깁니다. **관문은 순서가 있을 때만 관문입니다.** 나란히 도는 두 흐름 중 하나가 다른 하나를 막는다고 생각하고 있었다면, 그것은 관문이 아니라 **같은 시각에 울리는 두 개의 알람**입니다.

3.10.116 v0.161.0에서 완료 — 두 대표 기능이 처음으로 만났습니다

이 제품에는 큰 기능이 둘 있습니다. 회의에 들고 갈 **덱을 내보내고**, 다시 타이핑하지 않으려고 **덱을 가져옵니다.** 둘은 한 번도 만난 적이 없습니다.

가져오기 쪽은 특히 그랬습니다. 업로드가 받아들여지는지, 거부가 기록되는지까지는 시험이 있었는데 — **모델이 답한 뒤부터 보고서가 생기기까지**는 전부 운영 환경에서만 돌았습니다. 배경 작업자, PPTX 추출, 주차 탐지, 항목으로의 투영. 아무것도 시험된 적이 없습니다.

v0.158.0 에서 세운 **답하는 게이트웨이**가 그 문을 열었습니다.

한 사람이 쓴 주간보고를 발표자가 하듯 내보내고, 그 파일을 그대로 업로드하고, 작업자가 끝낼 때까지 기다린 뒤, 나온 것이 처음에 쓴 그 업무인지 봅니다. **`READY` 로 끝나고, 업무명·실적·계획이 그대로 돌아옵니다.**

기다리는 방식도 정했습니다 — 짐작한 시간만큼 자는 대신 **끝난 상태를 기다립니다.** 작업자는 고루틴이고, 물어야 할 것은 "끝나는가" 이지 "얼마나 빠른가" 가 아닙니다. v0.159.0 에서 픽스처가 작업자와 경주하다 CI 에서 떨어진 바로 다음 절이라, 이번에는 처음부터 그렇게 썼습니다.

닿을 수 없는 하한 하나

`processImportFile` 의 변이 넷 중 셋이 잡히고 하나가 남았습니다 — 파서 예산의 하한 256입니다.

```
parserBudget := cfg.MaxInput - runeLength("원본 파일명: "+filename+"\n") - 160
if parserBudget < 256 { parserBudget = 256 }
```

`cfg.MaxInput` 은 1000 이상으로 검증되고 파일명은 255자로 제한되므로, 가장 작은 경우도 `1000 - 264 - 160 = 576` 입니다. **하한 아래로 내려갈 수 없습니다.** 방어로서 옳고, 도달할 수 없다는 것도 사실입니다.

원칙으로 남깁니다. **기능 하나를 시험하는 것과 기능 둘 사이를 시험하는 것은 다른 일입니다.** 내보내기에도 시험이 있었고 가져오기에도 있었지만, 내보낸 것을 가져올 수 있는지는 둘 중 어느 쪽의 질문도 아니었습니다.

3.10.117 v0.162.0에서 완료 — 누가 열쇠를 받는가

v0.129 부터 무방비 인가 거부 목록에 셋이 남아 있었고, 그중 둘은 **"검증된 OIDC 공급자가 필요하다"** 는 이유로 세 번 기록만 되었습니다. 둘 다 처리했습니다.

하나는 공급자가 필요 없었습니다

`oidcConfig` 를 읽어 보니 **설정만 확인하고 네트워크를 타지 않습니다.** 로컬 로그인을 끄는 데 필요한 "검증된 설정" 은 발급자·클라이언트 ID·비밀·`openid` 범위가 채워져 있다는 뜻이지, 그 주소에 실제로 무언가 있다는 뜻이 아닙니다.

즉 `LOCAL_LOGIN_DISABLED` 에 닿는 데 공급자가 필요했던 적이 없습니다. **제 기록이 과대평가였고, 그것을 세 번 반복해 적었습니다.** 읽어 보는 데 몇 분이면 됐습니다.

거부를 지우면 시험이 잡는 것까지 확인했고, **틀린 비밀번호에도 같은 답을 하는지**도 함께 봅니다 — 닫힌 문은 자격증명에 대한 사실이 아니라 배포에 대한 사실입니다.

다른 하나는 진짜로 필요했습니다

디스커버리 문서, JWKS, 토큰 종점, 그리고 그 열쇠로 서명한 ID 토큰까지 갖춘 **가짜 공급자**를 세웠습니다. `go-jose` 가 이미 간접 의존성에 있어 새로 들여온 것은 없습니다.

그 문 뒤가 이번 작업의 값입니다.

도착한 사람	되어야 하는 역할
지목된 무리에 속함	ADMIN
다른 무리에 속함	USER
무리가 없음	USER
관리자 무리를 지정하지 않음	USER

조건이 둘입니다 — **무리가 지정되어 있고, 그 무리에 속할 것.** 네 경우를 모두 두어야 `&&` 인지 `||` 인지가 드러납니다. v0.158.0 에서 배운 것을 그대로 적용했습니다.

SSO 배포에서 이 코드가 열쇠를 누구에게 줄지 정합니다. 한 번도 실행된 적이 없었습니다.

목록이 줄었습니다

거부 36곳 중 무방비가 **3곳 → 1곳**. 남은 하나는 미들웨어가 이미 막은 뒤의 두 번째 층이라 HTTP 로는 닿을 수 없습니다.

원칙으로 남깁니다. **"닿을 수 없다" 는 기록에는 유효기간이 있습니다.** 그때는 사실이었을 수 있고, 아니었을 수도 있으며, 어느 쪽이든 다시 읽기 전에는 알 수 없습니다. 이번 둘 중 하나는 처음부터 닿을 수 있었습니다.

3.10.118 v0.163.0에서 완료 — 남의 로그인을 끝내는 일

v0.162.0 이 세운 가짜 공급자 덕분에 CI 의 변이 점검이 곧바로 다음 것을 물었습니다. 새 시험이 잡지 못하는 변이 넷이고, 그중 셋이 한 줄에 있습니다.

```
if state == "" || code == "" || cookieErr != nil || cookie.Value != tokenHash(state) {
```

콜백은 낯선 이의 브라우저가 코드를 들고 와 "여기서 시작한 로그인의 것" 이라고 말하는 순간입니다. 그 말을 믿기 전에 넷을 확인하는데, 그중 어느 하나만 무너져도 **자기 로그인을 끝내는 것과 남의 로그인을 끝내는 것의 차이**가 사라집니다.

한 번도 실행된 적이 없었습니다 — 망가진 콜백을 보낸 적이 없기 때문입니다. 넷을 하나씩 어기는 네 경우를 넣었습니다.

보낸 것	
상태 없음	거부
코드 없음	거부
쿠키 없음	거부
다른 로그인의 쿠키	거부

그리고 두 가지를 더 봅니다 — **거부들이 아무 계정도 만들지 않았는지**, 그리고 **그 뒤에 정상 로그인이 여전히 되는지**. 뒤의 것이 없으면 "전부 거부" 하는 코드와 구별되지 않고, 거부가 시작된 로그인을 소모해 버렸는지도 알 수 없습니다.

되돌아올 주소를 스스로 정할 때

넷째 변이는 다른 줄입니다.

```
if r.TLS != nil || r.Header.Get("X-Forwarded-Proto") == "https" { scheme = "https" }
```

`oidc.redirect_url` 을 비워 둔 배포는 **지금 답하고 있는 요청에서** 돌아올 주소를 만들어 공급자에게 알립니다. 체계를 틀리면 그 주소를 검사하는 공급자가 토큰 교환을 거절하고, **로그인은 마지막 단계에서 실패하며**

화면에는 이유가 없습니다.

가짜 공급자가 `redirect_uri` 를 기록하게 해서 잽니다 — 평문으로 들어오면 `http://`, 프록시가 `X-Forwarded-Proto: https` 를 붙이면 `https://`.

원칙으로 남깁니다. **거부를 시험할 때는 거부 뒤에 무엇이 남았는지도 함께 봐야 합니다.** 아무것도 만들지 않았는지, 그리고 정당한 다음 시도가 여전히 가능한지 — 거부는 그 둘을 함께 지킬 때만 거부입니다.

3.10.119 v0.164.0에서 완료 — 제대로 막아 낸 것을 실패로 세던 표

시드가 v0.134 이후 크게 바뀌었으니 화면을 다시 걸어 봤습니다. 열한 개 화면에서 JS 오류도, 5xx 도, `NaN` 도 없었습니다. 대신 관리자 화면의 표에서 한 줄이 걸렸습니다.

```
GET /api/v1/me   8회   오류율 50%
```

로그인 화면이 세션을 확인하는 401 입니다. 설계상 정상이고, 이 저장소는 그것을 알고 있습니다 — `App.tsx` 에 *"첫 로그인 전에는 잃을 것이 없고, 기동 시의 401은 설계된 것"* 이라고 적혀 있습니다. 그런데 운영자 화면에서는 **가장 많이 불리는 인증 경로가 절반 실패한 것**으로 보입니다.

계산을 보니 `오류율 = 400 이상 / 전체` 였습니다. 즉 이것들이 전부 실패로 셉니다.

상태	실제로 일어난 일
401	틀린 비밀번호, 또는 로그인 전 세션 확인
403	인가가 제 일을 한 것
404	없는 자원을 물은 것
400	입력이 형식에 맞지 않다고 알려 준 것

전부 제품이 제대로 동작한 결과입니다. 그리고 5%를 넘으면 빨갛게 칠합니다 — 사람들이 비밀번호를 틀리고 권한 경계에 부딪히는 평범한 하루가 지나면, 화면은 빨갛습니다.

정작 쓸 수 있는 숫자는 옆에 있었습니다

질의는 `sum(request_count) FILTER(WHERE status>=500)` 을 **이미 세고 있었고**, 응답에 `serverErrors` 로 담아 보내고, 프런트엔드 타입에도 선언되어 있었습니다. **그리고 한 번도 그려지지 않았습니다.**

표에 `서버 오류` 열을 넣고, 빨간색을 그쪽으로 옮기고, 4xx 열의 이름을 `거부` 로 바꿨습니다. 카드 아래에 두 숫자가 무엇인지 한 줄로 적었습니다. OpenAPI 와 MCP 도구 설명도 같은 말을 하도록 고쳤습니다 — 기계가 읽는 쪽이 사람이 읽는 쪽과 다른 것을 말하면 안 됩니다.

실제 화면으로 확인했습니다. 일부러 비밀번호를 세 번 틀린 뒤, `서버 오류` 는 전부 0, `거부` 는 `/me` 50%·로그인 37.5%, **빨간 표시는 없습니다.**

원칙으로 남깁니다. **"오류" 는 계산이 아니라 판단입니다.** 400 이상을 더하는 것은 계산이고, 그중 무엇이 우리 잘못인지 정하는 것은 판단입니다. 계산만 하고 이름을 오류라고 붙이면, 그 숫자를 보는 사람이 대신 판단하게 되고 대개 틀립니다.

3.10.120 v0.165.0에서 완료 — 마지막 어두운 구석

화면 걸음에서 업무 인사이트의 두 숫자가 걸렸습니다 — **협업 지도 0, 병목 0.** 300명·업무 2,100건짜리 배포에서요.

재 보니 둘의 사정이 달랐습니다.

협업 지도는 범위 효과였습니다. 관리자로 보면 496입니다. 팀장은 자기 조직만 보는데 그 카드는 "조직을 가로질러" 를 말하므로, 하위 조직이 없는 팀장에게는 **구조적으로 빌 수밖에 없습니다.** 화면이 옳고, 다만 왜 비었는지는 말하지 않습니다.

병목은 진짜였습니다. 관리자로 봐도 0입니다. 이유는 하나였습니다 — **시드가 의존 관계를 하나도 만들지 않습니다.**

그래서 이 기능 전체가 어두웠습니다.

어두웠던 것	
병목 화면	데이터가 아무리 늘어도 0
회의의 **타 조직 대기** 구획	나타날 수 없음
의존 관계 화면	그릴 것이 없음

이 파일이 v0.134 에서 정체를, v0.137 에서 열린 이슈를 배운 것과 **같은 교훈이고 세 번째**입니다. 다만 이번에는 한 지표가 아니라 기능 하나가 통째로 걸려 있었습니다.

두 모양을 넣었습니다 — 한 조직 안에서 옆자리 업무를 기다리는 평범한 쌍과, **조직을 가로질러 여러 업무를 붙잡고 있는 소수의 선행 작업.** 뒤의 것이 회의가 존재하는 이유이고, 병목이 되는 유일한 모양입니다.

재 보니 이렇습니다.

	이전	지금
의존 관계	0건	**260건**
병목	0건	**5건** (막는 업무와 기다리는 업무 이름까지)
회의의 타 조직 대기	나타날 수 없음	해당 조직 팀장에게 **1건**

여기서 한 번 헛짚었습니다

시드를 고친 뒤 u25 의 회의 안건을 봤는데 여전히 **타 조직 대기가 없었습니다.** 고침이 안 먹었다고 결론지을 뻔했는데, 세어 보니 조직 간 의존을 가진 조직이 46개 중 5개였고 u25 의 조직은 그중이 아니었습니다. **화면이 옳았습니다.** 실제로 가진 조직의 팀장(u150)으로 들어가니 1건이 안내 문구와 함께 나옵니다.

원칙으로 남깁니다. **비어 있는 화면을 보면 두 가지를 구별해야 합니다** — 만들 수 없어서 빈 것인지, 이 사람의 범위에 없어서 빈 것인지. 앞의 것은 결함이고 뒤의 것은 사실이며, 둘 다 화면에는 똑같이 보입니다.

3.10.121 v0.166.0에서 완료 — 있을 수 없던 것의 부재

바로 앞 절에서 원칙 하나를 적었습니다 — **비어 있는 화면은 "만들 수 없어서" 와 "이 사람의 범위에 없어서" 를 구별해야 한다.** 그것을 제품에 적용했습니다.

업무 인사이트의 다섯 탭 중 둘이 **조직을 가로지르는** 비교입니다. 그런데 팀장의 범위는 자기 조직이고, 하위 조직이 없으면 보이는 조직이 하나입니다. 그 사람에게 화면은 이렇게 말했습니다.

> 조직 간 중복으로 의심되는 업무가 없습니다.

사실이지만, 있을 수 없던 것의 부재입니다. 읽는 사람은 "우리 회사에 조직 간 중복 투자가 없구나" 로 받아들이는데, 화면은 그것을 확인해 준 적이 없습니다.

이 화면은 이미 같은 원칙을 한 겹 알고 있었습니다. 소스에 이렇게 적혀 있습니다 — *"실패한 조회를 0으로 채워 넣으면 모든 탭이 진짜 아무것도 없을 때와 같은 말을 한다. 화면은 자기가 확인하지 않은 부재를 단언해서는 안 된다."* 이번 것은 그 한 겹 안쪽입니다.

응답이 **읽는 사람에게 보이는 조직 수**를 함께 싣게 하고, 둘 미만이면 두 탭이 이렇게 말하도록 했습니다.

> 보이는 조직이 하나뿐이라 조직을 가로지르는 비교가 성립하지 않습니다.

> 상위 조직 범위에서 열면 다른 팀의 업무와 견줄 수 있습니다.

실제 화면으로 확인했습니다 — 팀장은 위 문장을, 관리자는 5,655건 중 200건을 그대로 봅니다.

병목 탭은 이미 옳게 하고 있었습니다: *"여러 업무를 막고 있는 선행 업무가 없습니다. 다른 업무를 기다리고 있다면 업무 추적에서 선행 관계를 등록하세요."* 없다는 말 뒤에 **다음에 할 일**이 붙어 있습니다. 나머지 둘만 그러지 못하고 있었습니다.

원칙으로 남깁니다. **"없습니다" 는 두 가지 뜻입니다** — 찾아봤는데 없다는 뜻과, 여기서는 찾을 수 없다는 뜻. 화면이 그 둘을 같은 문장으로 말하면, 읽는 사람은 언제나 앞의 것으로 읽습니다.

3.10.122 v0.167.0에서 완료 — 걸어 다니지 말고 세어 보기

정체(v0.134), 열린 이슈(v0.137), 의존 관계(v0.165) — 시드가 만들지 못하는 것을 세 번 찾았고, 세 번 다 **화면을 걷다가 우연히** 만났습니다. 이번에는 세어 봤습니다.

씨를 뿌린 배포에서 모든 테이블의 행 수를 셌더니, 사용자 대면 기능 다섯이 0이었습니다.

비어 있던 것	어두웠던 화면
`decisions`	결정 기록, 회의의 결정 필요, **합의된 기한**, 결정 제안
`work_item_issue_outcomes`	이슈 해소 보고 (v0.131 이 만든 것)
`report_comments`	검토 의견
`report_attachments`	캡처 첨부 — 디스크의 파일이 필요해 남겨 둡니다
`report_status_history`	보고서 이력 — 제출 흐름을 거쳐야 생깁니다

앞의 셋을 넣었습니다. 재 보니 이렇습니다.

	이전	지금
결정	0건	233건 (미결 77 · 완료 156)
해소된 이슈	0건	161건
검토 의견	0건	380건
이슈 해소 보고	빈 창	**해소 5건 · 중앙값 4주 · 최장 7주(업무명까지)**
미결 후속 조치	0건	제목·기한·후속 조치와 함께 표시

	이전	지금
합의된 기한	제시된 적 없음	`캐시 구축 → 2026-09-07 · 캐시 구축 방향 결정`

제 픽스처가 제가 찾던 함정에 빠졌습니다

처음 쓴 결정 삽입은 `w.id % 9 = 0` 인 업무를 고르고, 그 안에서 `w.id % 3 = 0` 이면 OPEN 이라고 했습니다. **9의 배수는 언제나 3의 배수이므로 전부 OPEN 이 됐습니다.** 233건이 모두 같은 상태였고, 제품은 그 둘을 다르게 다룹니다.

이번 창 내내 남의 시험에서 찾아내던 "전부 같은 값" 을 제가 썼습니다. 나눈 뒤에 나머지를 보도록 고쳐 77 대 156 으로 갈렸습니다.

그리고 한 번 헛짚었습니다

`agreedDue` 가 0으로 나와 고침이 안 먹었다고 볼 뻔했는데, 그 값은 **자기 기한이 없고 아직 끝나지 않은** 업무에만 제시됩니다. 제가 처음 본 사용자의 업무는 완료된 것이었습니다. 조건에 맞는 사용자로 보니 정상입니다.

원칙으로 남깁니다. **없는 것은 화면을 걸어서 찾기 어렵습니다** — 빈 화면은 눈에 잘 띄지 않고, 대개 "아직 데이터가 없나 보다" 로 지나갑니다. 무엇이 비어 있는지는 **세면** 한 번에 나옵니다.

3.10.123 v0.168.0에서 완료 — 인수인계는 남이 읽는 화면입니다

변이 검사가 `handover.go` 의 비교 하나를 뒤집었고, **시험 전부가 통과했습니다.**

```
scope := scopeForPrincipal(p, false)
if target == p.ID {          // != 로 뒤집어도 아무도 몰랐습니다
    scope = scopeForPrincipal(p, true)
}
```

뒤집히면 이렇게 됩니다. 팀장이 떠나는 팀원의 인수인계를 열면 **자기 자신의 범위**로 업무를 읽어 오고, 뒤따르는 `item.UserID == target` 필터가 그 전부를 버립니다. 화면에는 **빈 목록과 그 사람의 올바른 이름**이 남습니다 — 이름은 업무가 없을 때 users 테이블에서 따로 채우기 때문입니다.

인계할 업무가 하나도 없는 동료와 구별되지 않습니다. 기능 전체가 조용히 사라집니다.

왜 아무도 못 봤나

인수인계 내용 시험이 넷 있고, **전부 자기 것을 읽습니다.**

시험	읽는 사람	대상
떠나는 사람의 업무만 담기는가	떠나는 사람	자기 자신
막힌 것 먼저, 그다음 오래된 것	소유자	자기 자신
결정이 딸려 오는가	떠나는 사람	자기 자신
남의 인수인계는 못 읽는가	외부인	**남 — 그러나 403 만 봅니다**

읽는 사람과 대상이 같으면 두 범위가 같은 결과를 냅니다. 문은 지켰고(200/403), 문 뒤는 자기 자신으로만 봤습니다.

인수인계는 정의상 남이 읽는 화면입니다. 누군가 떠날 때 여는 것이니 읽는 사람과 대상은 언제나 다릅니다. 정작 그 경로에 확인이 없었습니다.

이번 창에서 여섯 번째

"거절은 시험하고 허용은 안 한다" 가 또 나왔습니다. 권한, 설정, API 키 범위, AI 검증에 이어서입니다. 403을 확인하면 일이 끝난 것 같지만, 그 옆에는 언제나 **200이 옳은 것을 담고 있는가** 라는 질문이 남아 있습니다.

팀장이 떠나는 팀원의 인수인계를 읽으면 그 팀원의 업무가 오고 팀장 자신의 업무는 오지 않는지 확인하는 시험을 넣었습니다. 변이를 다시 넣으니 `got none of their work: []` 로 떨어집니다.

3.10.124 v0.169.0에서 완료 — 범위 전환기 다섯 곳이 전부 무방비였습니다

v0.168.0 에서 인수인계의 "자기 것인가 남의 것인가" 판단이 무방비였습니다. 같은 판단을 하는 자리를 전부 찾아 하나씩 뒤집어 봤습니다.

```
grep scopeForPrincipal → scope == scopeSelf 을 쓰는 곳 다섯
```

자리	화면	뒤집었을 때
`workitems.go:243`	업무 목록	★ 아무도 못 잡음
`worksearch.go:146`	업무 검색	★ 아무도 못 잡음
`weeklychange.go:253`	주간 변화	★ 아무도 못 잡음

자리	화면	뒤집었을 때
`meeting.go:248`	회의 자료	★ 아무도 못 잡음
`meeting.go:259`	타 조직 대기	★ 아무도 못 잡음

다섯 곳 전부입니다. `scope` 매개변수가 실제로 무슨 일을 하는지 확인하는 시험이 하나도 없었습니다.

왜 보이지 않았나

이 화면들에는 거절이 붙어 있고, 그 거절은 지켜집니다 — 평범한 작성자가 `scope=TEAM` 을 요구하면 403 이고, 시험이 그것을 확인합니다.

문제는 **평범한 구성원의 자리에서는 두 범위가 같은 행을 읽는다**는 것입니다. 넓힐 조직이 없으니 SELF 와 TEAM 이 구별되지 않습니다. 차이는 팀장에게만 존재하는데, 이 다섯 화면을 **팀장으로 읽어 본 시험이 없었습니다.**

뒤집히면 이렇게 됩니다.

- 팀장이 **자기 업무를 물으면 팀 전체가 나옵니다.** 화면에는 "내 업무" 라고 쓰여 있습니다.
- 팀장이 **팀을 물으면 자기 것만 나옵니다.**

앞의 것이 더 나쁩니다.

넣은 것

팀장과 동료를 한 조직에 두고 네 화면을 SELF·TEAM 두 번씩 읽습니다. SELF 에 동료의 업무가 있으면 실패, TEAM 에 없으면 실패, 그리고 **양쪽 모두에 팀장 자신의 업무가 있어야** 합니다 — 그냥 빈 응답이 앞의 두 조건을 우연히 통과하지 못하게 하는 세 번째 조건입니다.

`타 조직 대기` 는 같은 전환기를 두 번째로, 별도 질의에서 읽으므로 따로 지킵니다. 조직 둘, 사람 셋, 조직을 가로지르는 의존 링크 하나를 만들어 팀장의 회의 자료가 동료의 대기를 담고 자기 안건에는 담기지 않음을 확인합니다.

다섯 변이를 다시 넣으니 전부 떨어집니다.

이번 창에서 일곱 번째

"거절은 시험하고 허용은 안 한다" 입니다. 403 을 확인하고 나면 일이 끝난 것처럼 보입니다. 옆에는 언제나 **200 이 옳은 것을 담고 있는가** 가 남아 있고, 이번에는 그 질문이 한 번에 다섯 자리를 열었습니다.

3.10.125 v0.170.0에서 완료 — 보내고 버리던 490KB

씨앗이 채워진 배포에서 응답의 **무게**를 재 봤습니다. 화면을 보면 알 수 없고, 시간을 재도 알 수 없습니다 — 141ms 로 충분히 빨랐으니까요.

화면 (관리자 · 전사 **2,100**건)	시간	크기	담긴 줄
업무 목록	228ms	170KB	200 / 2,100
회의 자료	201ms	38KB	섹션당 40
업무 검색	284ms	14KB	25
주간 변화	**141ms**	**490KB**	**1,866 — 상한 없음**

진척 1,166줄, 정체 700줄. 한 응답에 담겨 나갑니다.

그런데 그 줄을 읽는 화면이 없습니다

대시보드는 변화 막대를 `count` 하나로 그립니다. `entries` 는 타입에는 있지만 **어느 컴포넌트도 건드리지 않습니다.** 1,866줄을 직렬화해 보내고 브라우저가 통째로 버립니다. 제품이 처음 여는 화면에서입니다.

회의 안건이 이미 받은 처방

`meetingSectionLimit = 40` 과 `orderMeetingEntries` 가 v0.147 무렵 같은 문제를 이렇게 풀었습니다. 같은 처방을 그대로 씁니다.

- `changeGroupLimit = 40`
- `orderChangeEntries` — 변화폭이 큰 것 먼저, 같으면 덜 끝난 것 먼저, 같으면 제목순(결정적)
- `count` 는 **진짜 개수를 그대로** 유지합니다. 대시보드 막대가 달라지면 안 됩니다.
- `limit` 을 새로 실어, 받은 쪽이 잘렸는지 알 수 있게 합니다.

	이전	지금
크기	490,157 B	**21,604 B** (23배)
진척	1,166줄	count 1,166 · entries 40
정체	700줄	count 700 · entries 40

자르는 것보다 순서가 중요합니다

상한만 넣으면 남는 40줄이 **업무 번호가 낮은 것들**입니다. 그래서 시험은 두 가지를 따로 봅니다. 가장 크게 움직인 업무를 **목록 맨 끝에** 만들어 두고, 상한을 없애면 "40줄을 넘었다" 로, 정렬을 없애면 "가장 크게 움직인 업무가 선두가 아니다" 로 떨어지는지 확인합니다. 둘 다 떨어집니다.

다음 걸음으로 남기는 것

규모 배포에서 정렬을 눈으로 확인하려니 **씨앗의 주간 변화폭이 전부 1~2%** 라 모든 항목이 동률이었습니다. 그리고 변화 7종 가운데 **다섯이 한 번도 나타나지 않습니다.**

완료	신규	재개	진척	역행	정체	누락
0	0	0	1,166	**0**	700	0

대시보드의 주 시각 요소가 일곱 칸 중 두 칸만 씁니다. 씨앗이 만들지 못하는 상태를 세는 일(v0.167)의 연장선이고, 다음 걸음입니다.

3.10.126 v0.171.0에서 완료 — 일곱 칸 중 두 칸, 그리고 그것을 못 보던 자

v0.170.0 에서 주간 변화의 상한을 넣으며 씨앗의 한계 둘을 적어 두었습니다. 이번에 둘 다 걷어냅니다.

하나 — 변화 일곱 종 가운데 다섯이 어두웠습니다

	완료	신규	재개	진척	역행	정체	누락
이전	0	0	0	1,166	**0**	700	0
지금	**78**	**43**	**83**	1,052	**72**	518	**72**

씨앗의 모든 업무는 매주 빠짐없이 보고되고 진척이 오르거나 멈추기만 했습니다. 그래서 다섯 종은 **정의상 나올 수 없었습니다.** 대시보드의 주 시각 요소가 일곱 칸 중 두 칸만 쓰고, 나머지 다섯을 만드는 분류 규칙은 데이터가 있는 채로 한 번도 실행된 적이 없었습니다.

마지막 두 주를 다르게 만들었습니다. 없음으로 정의되는 두 종은 **행을 빼서** 만듭니다 — 이번 주에서 빠지면 보고 누락, 지난주에서 빠지면 재개. 완료는 90 미만에서 100 으로, 역행은 15 낮게, 신규는 이번 주에만 존재하는 슬롯 하나로.

읽히는 문장도 함께 살아납니다. `26주 만에 완료됐습니다` · `1주 쉬었다가 다시 보고됐습니다` · `지난주에는 보고됐으나 이번 주 보고에 없습니다`.

둘 — 정렬을 볼 수 없는 픽스처

v0.170.0 이 넣은 `orderChangeEntries` 는 천 줄 중 어느 사십 줄을 보여 줄지 정합니다. 그런데 씨앗의 주간 변화폭이 **전부 1~2%** 라 모든 항목이 동률이었습니다. 상한은 옳아 보였지만, 그 상한이 옳게 자르는지는 픽스처가 보여 줄 수 없었습니다.

한 주에 실제로 무언가 끝나는 업무를 섞었습니다. 이제 진척 상위 40줄이 **17~21% 움직인 업무**입니다.

그리고 — 재는 자가 못 보고 있었습니다

`scale-check.py` 는 규모에서만 보이는 결함을 잡으라고 만든 도구입니다. 그런데 v0.170.0 의 490KB 를 **놓쳤습니다.**

```
200    663B    7ms   /api/v1/changes        ← 이 도구가 본 것
200 490,157B  141ms  /api/v1/changes?scope=TEAM   ← 실제
```

범위 없이 불렀고, 부트스트랩 관리자는 업무가 없습니다. 24개 경로 중 여섯이 **빈 응답을 재고 있었고**, 작은 숫자는 좋은 숫자처럼 보였습니다.

셋을 고쳤습니다.

- `?scope=TEAM` 을 주고, `?scope=SELF` 와 업무 검색을 따로 넣었습니다.
- **행이 0인 응답을 그렇다고 말합니다** — "잰 것이 없는 응답 N건 — 아래 숫자는 무엇도 증명하지 않습니다".
- 씨앗이 **본부장 넷에게도 보고서를 줍니다.** `u%` 계정만 보고서를 쓰고 있어, 가장 높은 자리에서는 내 보고서·인수인계·미결 후속 조치가 전부 비어 있었습니다.

바이트로 세려다 늑대 소년이 될 뻔했습니다

처음에는 "1KB 미만이면 빈 응답" 으로 잡았습니다. 그런데 `/api/v1/analytics/overview` 는 **221바이트에 76명과 미해결 126건**이 들어 있는 진짜 요약입니다. 스크립트 자신의 주석이 이미 이렇게 말하고 있었습니다 — *"늑대를 외치는 도구는 사람들이 안 읽게 됩니다."*

가장 깊은 목록의 행 수로 바꿨습니다. 주간 변화는 아무 일이 없어도 그룹 일곱이 늘 있으므로, 행은 그 한 층 아래에서 셉니다. 목록이 아예 없는 응답은 요약 객체이니 건드리지 않습니다.

계정	이전	지금
admin (업무 없음)	—	5건이 "잰 것 없음"
hq1 (본부장)	6건	**1건**

계정	이전	지금
u100 (팀장)	3건	**1건**

3.10.127 v0.172.0에서 완료 — 상태가 하나뿐이라 보이지 않던 것

씨앗의 모든 보고서가 `APPROVED` 였습니다. 다섯 상태 가운데 하나만 존재했습니다.

	작성 중	검토 대기	반려/수정	승인	확정
이전	0	0	0	305	0
지금	27	27	28	194	28

그래서 `보고서 상태 분포` 는 다섯 칸 중 넷이 0이었고, **제출률은 어느 배포에서나 정확히 100.0%** 였으며, "아직 안 낸 사람" 을 묻는 화면은 부를 이름이 없었습니다. 검토 워크플로는 이 제품이 하는 일의 삼분의 일인데 픽스처는 그것을 한 상태로 두고 있었습니다.

현재 주만 갈랐습니다. 이전 주는 전부 `APPROVED` 로 남겨 롤업과 예측이 읽는 1년 치 이력은 그대로입니다. `submitted_at` · `reviewed_at` · `reviewed_by` 도 상태와 앞뒤가 맞게 채웠습니다 — 셋 다 씨앗 전체에서 NULL 이었습니다.

반려된 보고서에는 사유를 붙였습니다. `reviewReport()` 가 사유를 상태 이력과 **`report_comments` 양쪽에** 쓰고, 작성자가 화면에서 읽는 것은 후자입니다. 사유 없는 반려는 제품이 만들지 않는 상태입니다.

그러자 팀 화면에서 빈틈이 드러났습니다

배지가 다섯 가지로 나오자마자 보였습니다. **팀장이 해야 할 단 하나의 일 — 검토 대기를 찾는 것 — 을 할 방법이 없습니다.**

- 목록은 3,952건 중 200건씩 옵니다.
- 승인·반려 버튼은 `SUBMITTED` 에만 나타납니다.
- 상태 열은 있지만 **거를 수가 없습니다.**

76명짜리 조직에서 검토할 것을 찾으려면 스무 페이지를 넘기며 배지를 읽어야 합니다. 그런데 API 는 처음부터 `status` 매개변수를 받았고 `docs/openapi.yaml` 에도 적혀 있었습니다. **화면만 그것을 쓰지 않았습니다.**

	이전	지금
검토 대기를 찾기	3,952건을 훑기	한 번 고르면 7건

필터를 바꾸면 첫 행부터 다시 읽습니다 — 다른 필터에서 넘어온 offset 은 다른 목록을 가리킵니다. 빈 결과일 때는 "이 상태인 보고서가 없습니다" 라고 말합니다. 무엇이 비었는지 모르는 빈 화면을 또 만들지 않기 위해서입니다.

손대지 않은 것

팀장이 동료의 `작성 중` 보고서 **내용까지** 열 수 있습니다. 상태 열이 작성 중을 일부러 보여 주고 열기 링크를 주는 것으로 보아 의도된 설계로 읽힙니다. 가시성 규칙은 배포된 조직의 기대가 걸린 문제라 혼자 바꾸지 않고 적어만 둡니다.

3.10.128 v0.173.0에서 완료 — 반려 사유가 네 화면 아래에 있었습니다

v0.172.0 이 씨앗의 보고서 상태를 갈랐습니다. 그러자 **한 번도 존재한 적 없던 자리**가 생겼습니다 — 보고서가 반려당한 작성자.

그 자리에서 화면을 열어 봤습니다. 상태 줄이 이렇게 말합니다.

> 반려 의견을 확인하고 수정해 주세요.

그리고 그게 전부입니다. 사유가 어디 있는지도, 무엇인지도 말하지 않습니다.

재 봤습니다.

	y 좌표
"반려 의견을 확인하고 수정해 주세요"	**182**
실제 반려 사유	**4,562**
문서 높이	4,780

3.8 화면 아래, 업무 항목 일곱 개와 보고 품질 점검을 지나 맨 밑입니다. 검토 워크플로에서 작성자가 무언가를 들어야 하는 단 하나의 순간에, 화면은 *어딘가에 읽을 것이 있다*는 사실만 알려 줍니다.

사유를 작성자가 서 있는 자리로 옮겼습니다.

	이전	지금
사유까지의 거리	y=4,562 (3.8 화면)	**y=250 (첫 화면)**

어느 의견이 사유인가

서버는 반려를 두 번 적습니다. 상태 이력에 한 번 — 아무도 읽지 않습니다 — 그리고 `report_comments` 에 한 번. 작성자가 보는 것은 후자입니다. 그래서 사유는 **필드가 아니라 규칙**입니다.

되돌려보낸 사람은 그 보고서의 작성자가 아니고, 여러 번 말했다면 마지막 말이 따라야 할 말입니다.

이 규칙을 `revisionReasonOf()` 로 빼서 시험을 붙였습니다. 특히 이것 하나:

> **작성자가 같은 자리에 답글을 답니다.** 그러면 가장 최근 의견은 작성자 자신의 말입니다. 최신 것을 집으면 작성자에게 자기 말을 검토 의견이라고 보여 주게 됩니다 — 아무것도 안 보여 주는 것보다 나쁩니다.

`!==` 를 `===` 로 뒤집으면 네 건이, 역순 순회를 정순으로 바꾸면 한 건이 떨어집니다.

이번이 세 번째입니다

v0.171 은 변화 일곱 종 중 다섯을, v0.172 는 보고서 상태 다섯 중 넷을 씨앗에 넣었습니다. 그때마다 **바로 다음에 제품 결함이 하나씩 나왔습니다** — 상한 없는 490KB, 필터 없는 검토 대기, 그리고 이번 것.

픽스처가 만들 수 없는 상태는 결함이 숨는 자리입니다. 화면을 아무리 걸어도 그 자리에는 닿지 못합니다.

3.10.129 v0.174.0에서 완료 — 모든 줄에 붙은 표시는 아무것도 가리키지 않습니다

업무 목록의 모든 행에 `계획 52회 반복` 배지가 붙어 있었습니다. 세어 봤습니다.

repeatedPlan	이전	지금
배지가 뜨는 비율 (≥3)	**100.0%**	**13.8%**

씨앗이 52주 내내 같은 차주 계획 문장을 썼기 때문입니다. 계획이 2주씩 이어지도록 바꾸고 — 배지 임계값 3보다 낮습니다 — **여덟에 하나는 일부러 계속 같은 계획을 되풀이하게** 두었습니다. 배지는 그 행들을 위한 것입니다.

규칙 넷 중 하나는 걸릴 수가 없었습니다

`보고 품질 점검` 은 네 가지를 봅니다. 씨앗이 매주 모든 줄에 실적을 채워 넣어, 그중 하나는 **한 번도 실행된 적이 없었습니다.**

규칙	이전	지금
진척도 역행	v0.171 부터	✓
같은 계획 반복	✓ (과다)	✓
계획한 일의 빈 실적	**0회**	✓
지속되는 이슈	✓	✓

현재 주에서 스물아홉에 하나씩 실적을 비웠습니다.

그리고 통과했다는 말을 하지 않았습니다

```
{quality && quality.findings.length > 0 && <Card title="보고 품질 점검" …>}
```

지적이 없으면 **카드가 통째로 사라집니다.** 깨끗한 보고서를 쓴 사람은 점검이 통과한 것인지 아예 돌지 않은 것인지 구별할 수 없습니다. 제출 전에 확신을 주라고 있는 화면이 정작 좋은 소식이 있을 때 입을 다뭅니다.

지금까지 이것이 보이지 않은 이유는 **모든 작성자가 지적을 받았기** 때문입니다. 계획 반복이 100% 였으니까요. 규칙이 성글어지자마자 지적 0건인 사람이 나왔고, 그 사람의 화면이 비어 있었습니다.

통과 상태를 넣었습니다. 무엇을 봤는지도 함께 말합니다 — 아무것도 못 찾았다는 말은 무엇을 찾았는지 알아야 뜻이 있습니다.

> 7개 업무 확인 · 규칙 점검에서 걸리는 것이 없습니다. 진척도 역행, 같은 계획 반복, 계획한 일의 빈 실적, 지속되는 이슈 네 가지를 봤습니다.

네 번째입니다

v0.171 변화 종류, v0.172 보고서 상태, v0.173 반려 사유, 그리고 이번 계획 문장. **픽스처를 성글게 만들 때마다 바로 다음에 제품 결함이 하나씩 나왔습니다.** 이번 것은 특히 그렇습니다 — 지적 0건인 사람이 한 명도 없는 동안에는, 지적 0건일 때 화면이 어떻게 되는지 아무도 볼 수 없었습니다.

3.10.130 v0.175.0에서 완료 — 제품 코드를 자기 안에 베껴 둔 시험

기간 보고에 변이 검사를 걸었더니 하나가 살아남았습니다.

```
for index := range view.Items {
    if index >= rollupTimelineItems {   // > 로 바꿔도 아무도 몰랐습니다
```

```
        view.Items[index].Weeks = nil
    }
}
```

뒤집으면 스물한 번째 항목이 주차 데이터를 달고 옵니다. 응답은 `timelineItems: 20` 이라고 말하는데 스물한 줄 치가 실려 오고, 화면은 스무 줄만 그립니다 — **이 코드가 애초에 막으려고 쓰인 낭비 그대로입니다** (885KB 중 522KB).

시험은 있었습니다

`TestRollupSendsTheWeeklySeriesOnlyForTheRowsThatAreDrawn` 이 바로 그것을 위해 있었습니다. 그런데 이렇게 생겼습니다.

```
view := rollupView{Items: items, TimelineItems: rollupTimelineItems}
for index := range view.Items {
    if index >= rollupTimelineItems {     // ← 핸들러의 코드를 그대로 베껴 옴
        view.Items[index].Weeks = nil
    }
}
// ... 그리고 자기가 방금 만든 것을 검사합니다
```

핸들러를 부르지 않습니다. 핸들러의 로직을 시험 안에 복사해 두고 그 사본이 사본대로 동작하는지 확인합니다. 핸들러가 어떻게 바뀌든 언제나 통과합니다.

자르는 일을 `trimTimelineSeries()` 로 빼서 핸들러와 시험이 **같은 것 하나**를 쓰게 했습니다. `>=` 를 `>` 로 바꾸면 이제 `21 rows carry a series, want 20` 으로 떨어집니다.

같은 모양이 더 있는지 훑었습니다

시험 파일에서 제품 코드와 글자까지 같은 구조 줄(`if` / `for` / `switch` / `return`)을 찾아 70건이 나왔지만, 대부분은 zip 을 읽거나 `io.EOF` 를 보는 관용구라 겹치는 게 자연스럽습니다. 판단을 베낀 것은 이 하나였습니다.

곁가지로 `TestDigestIsCappedAndOrderedByScore` 를 손봤습니다. 20건을 넣어 상한 10이 걸리는 것을 확인하는데, **상한이 20 위로 올라가면 조용히 아무것도 증명하지 않게 됩니다.** 회의 섹션 시험이 이미 쓰고 있는 방어를 같이 달았습니다 — 픽스처가 상한을 넘지 못하면 그 자리에서 실패합니다.

그리고 제 실수 하나

변이 검사를 두 개 동시에 돌렸습니다. 둘 다 소스를 고칩니다. 알아채고 즉시 둘 다 멈췄고, `git diff` 로 남은 변이가 없음을 확인한 뒤 하나씩 다시 돌렸습니다. **이번 창에서 두 번째로 같은 실수입니다.**

3.10.131 v0.176.0에서 완료 — 열 건이 전부인지 스물둘 중 열인지

`경영 요약` 을 규모에서 열어 봤습니다. 열 건이 나오는데 **점수가 190, 185, 185, 185, 185…** 로 아홉이 동점이었습니다. 그리고 화면은 이렇게만 말합니다.

> 핵심 **10건** · 업무 543건 검토

동점 무리를 가로질러 잘랐다는 말이 없습니다. 형제 화면은 전부 말합니다 — 회의 안건은 `N건 중 40건`, 주간 변화는 `count` · `limit`, 인사이트는 `208건 중 200건`. **가장 높은 자리가 읽는 화면만 말하지 않았습니다.**

어떤 수를 말해야 하는가

처음에는 "점수가 붙은 것 전부"를 세어 내보냈습니다. 재 보니 **543건 중 468건**이었습니다.

> 선정 기준을 넘은 468건 가운데 10건입니다.

이건 반대 방향으로 거짓말입니다. 468건 대부분은 중복 의심 `+25` 한 줄짜리이고, 본부장에게 "확인할 것이 468건" 이라고 말하는 셈입니다.

읽는 사람이 실제로 쓸 수 있는 수는 하나입니다. **마지막으로 실린 것과 같은 점수 이상이 몇 건인가** — 잘라내는 대신 넣었어도 똑같았을 것들. 그 수는 **22건**이었습니다.

	수	뜻
점수가 붙은 것 전부	468	대부분 +25 한 줄. 겁만 줍니다
특정 조합(이슈 8주+정체)	50	제가 처음 어림한 수. 근거 없는 잘라내기
마지막 실린 것과 동급	**22**	상한이 없었다면 열 자리를 두고 다퉜을 것들

> 아래 10건과 **같은 점수 이상인 업무가 22건** 있습니다. 여기 실린 것이 그중 전부는 아닙니다.

상한은 함수로

v0.175.0 에서 배운 대로, 자르는 일을 `capDigest()` 로 빼서 핸들러와 시험이 같은 것 하나를 씁니다.

그리고 제 픽스처가 절반만 보고 있었습니다

기존 시험은 **동점 20건**을 만들어 넣습니다. 그러면 "잘려 나간 열 건을 세는가" 는 볼 수 있지만 **"나머지 코퍼스를 안 세는가" 는 볼 수 없습니다.** 낮은 점수 여덟 건을 섞은 시험을 따로 넣었습니다. 이제 적게 세도 `equallyUrgent=10, want 20`, 많이 세도 `equallyUrgent=20, want 12` 로 떨어집니다.

곁가지 — 이슈 문장이 두 개뿐이었습니다

경영 요약 열 건이 전부 같은 문장이었습니다. 씨앗이 110,534줄에 이슈 문구를 **두 개**만 썼기 때문입니다. 지속 이슈 아홉 가지, 단발 이슈 다섯 가지로 갈랐습니다. 장애는 주간보고에서 사람이 실제로 읽는 부분입니다.

3.10.132 v0.177.0에서 완료 — 씨앗이 스스로 획일해졌는지 봅니다

이번 주기에 여섯 번, **모든 값이 같아서 아무것도 보이지 않는** 픽스처가 제품 결함을 가리고 있었습니다.

	무엇이 같았나	가려진 것
v0.171	주간 변화폭이 전부 1~2%	정렬이 뭘 남기는지 볼 수 없음
v0.171	변화 7종 중 5종 부재	대시보드 주 시각 요소가 두 칸만 씀
v0.172	보고서 상태 전부 `APPROVED`	검토 대기를 찾을 방법이 없던 것
v0.173	(상태가 갈리자) 반려당한 자리	사유가 네 화면 아래
v0.174	차주 계획이 52주 내내 한 문장	배지가 100%에 붙어 무의미
v0.176	이슈 문장 두 개	경영 요약 열 줄이 같은 문장

여섯 번 다 **세어서** 찾았습니다. 화면을 걸어서는 한 번도 못 찾았습니다.

그래서 씨앗이 스스로 말하게 했습니다.

```
     check     | report_statuses | issue_sentences | plan_sentences | blank_results | rows_this_we
---------------+-----------------+-----------------+----------------+---------------+-------------
 다양성 점검 |               5 |              13 |           2176 |            77 |           2099
```

찍기만 해서는 부족합니다

숫자 한 줄은 출력 더미에 묻힙니다 — 이번 주기가 계속 발견한, 사람이 그냥 지나치는 조용한 신호 그 자체입니다. 그래서 무너지면 **실패합니다.**

```
$ psql -f scripts/seed-scale.sql
ERROR:  차주 계획 문장이 1가지뿐입니다. 계획 반복 배지가 모든 줄에
        붙어 아무것도 가리키지 못합니다.
종료 코드 3
```

계획 문장을 v0.174 이전으로 되돌려 실제로 걸리는지 확인했고, 되돌린 뒤 종료 코드 0으로 돌아오는 것까지 봤습니다. 문턱값은 현재 수식이 만드는 값보다 한참 아래라, 진짜로 납작해졌을 때만 울립니다.

네 가지를 봅니다 — 현재 주 보고서 상태 가짓수, 이슈 문장 가짓수, 차주 계획 문장 가짓수, 실적이 빈 줄의 수. 각 메시지는 **무엇이 안 보이게 되는지**까지 말합니다.

3.10.133 v0.178.0에서 완료 — 주간보고 메일 발송

관리자가 SMTP를 설정하고, 각자가 개인 설정에 받을 주소를 저장해 두면, **주간보고를 제출할 때** 그 주소로 내용이 갑니다.

관리자	`관리자 설정 → 주간보고 메일 발송` — 호스트·포트·보안·계정·보내는 주소 + `메일 발송 시험`
개인	`개인 설정 → 주간보고 메일 발송` — 받을 주소, 제출할 때 보내기, **주차별 발송 결과**
발송	제출 트랜잭션 안에서 큐에 넣고, 작업자가 트랜잭션 밖에서 보냅니다

릴레이는 이 제품의 것이 아닙니다

그래서 **제출을 막지 않습니다.** 발송할 것을 기록하는 일과 보고서를 접수하는 일은 하나의 결정이라 한 트랜잭션이고, 남의 서버에 닿는 일은 아니라서 밖입니다. 릴레이가 죽어 있으면 메일이 늦어질 뿐, 한 주의 작업이 사라지지는 않습니다.

밀린 발송은 큐에 남아 다시 시도되고, **실패한 이유는 릴레이가 준 문장 그대로** 작성자 화면에 남습니다. 도착하지 않은 메일은 질문이고, 그 답은 이미 릴레이가 했습니다.

평문 릴레이가 기본값입니다

이 제품이 도는 망에서는 사내 릴레이가 포트 25에 인증 없이 열려 있는 것이 보통이라, 보안 `없음(평문)`이 대비책이 아니라 정식 선택지입니다. 다만 **계정을 쓰면 STARTTLS나 TLS를 요구합니다** — Go의 SMTP 클라이언트는 평문 연결에 비밀번호를 싣지 않으므로, 그 조합은 어떻게 설정해도 동작하지 않습니다. 발송 시점에 남의 서버가 주는 거절 대신, 어긋난 두 설정의 이름을 여기서 말합니다.

시험이 세 가지를 잡았습니다

하나 — 빈 문자열이 "암호화됨"이었습니다. `Security == "NONE"` 으로 물었는데, 설정된 적 없는 빈 값은 NONE이 아니라서 암호화된 것으로 읽혔습니다. `sendMail` 은 STARTTLS·TLS가 아닌 모든 것을 평문으로 다루므로 둘이 어긋나 있었고, 비밀번호가 걸린 유일한 경우였습니다. `encrypted()` 로 바꿔 한쪽에 맞췄습니다.

둘 — 헤더 주입 시험이 방어를 빼도 통과했습니다. 제목과 보낸사람은 Base64 encoded-word로 감싸이므로 무엇을 넣어도 안전합니다. 실제로 위험한 곳은 **인코딩하지 않는** `To:` 하나였고, 시험은 안전한 두 곳만 찌르고 있었습니다. 그리고 단언도 틀렸습니다 — `"bcc:"` 라는 글자가 있는지 물으면, 무력화되어 To 줄 안에 평범한 글자로 들어간 경우까지 잡습니다. **헤더 필드 이름의 집합**으로 봐야 합니다.

셋 — 시험이 백그라운드 작업자와 경쟁했습니다. 제출이 작업자를 깨우므로 메일은 이미 나갔고, 시험이 직접 부른 `sendNextQueuedMail` 은 빈 큐를 봤습니다. 실패 문구는 `nothing was queued` — **기능이 동작한 것을, 동작하지 않았다고** 보고했습니다. 상태를 기다리는 방식으로 바꿨습니다.

저장소의 가드가 저를 잡았습니다

`sqlparams_test.go` 가 제 SQL 두 곳에서 자리표시자 재사용을 지적했습니다. 예외 목록에 넣는 대신 재사용을 없앴습니다 — `SELECT $1, s.user_id, ...` 와 `ON CONFLICT ... SET address=EXCLUDED.address`.

3.10.134 v0.179.0에서 완료 — 재시작보다 짧은 재시도

v0.178.0으로 메일 발송을 낸 직후, 그 기능의 끝 상태를 재 봤습니다. 릴레이를 닫힌 포트로 돌리고 제출한 뒤 30초마다 지켜봤습니다.

```
03:19:05  QUEUED 시도2
03:19:33  QUEUED 시도3
03:20:01  QUEUED 시도4
03:20:28  FAILED 시도5      ← 2분 30초 만에 영구 실패
```

릴레이가 2분 30초 닿지 않으면 그 주의 보고는 영영 안 갑니다. 재시작, 인증서 재적재, 네트워크 끊김 — 전부 그보다 깁니다. 재시도 정책이 아니라 조금 늦춘 포기였습니다.

그리고 한 건이 뒤를 막고 있었습니다

작업자는 매 틱마다 **가장 오래된 대기 건**을 집었습니다. 릴레이가 거부하는 주소 하나가 큐 맨 앞에 있으면, 그 건이 시도를 다 쓸 때까지 뒤에 선 사람들의 메일이 전부 기다립니다.

고침

`next_attempt_at` 을 두고, 실패한 건은 그 시각을 미룹니다. 두 문제가 같은 한 줄로 풀립니다 — 미뤄진 건은 **아직 차례가 아니므로** 다음 틱이 그것을 건너뛰고 뒤를 봅니다.

간격은 4의 거듭제곱입니다: 1분 · 4분 · 16분 · 1시간 · 2시간(상한). 다섯 번이 **2분 30초에서 약 3시간**으로 늘어납니다.

배포에서 같은 장애를 재현해 확인했습니다.

	이전	지금
시도 1 → 2	30초	**1분**
시도 2 → 3	30초	**4분**
2분 30초 장애	**영구 실패**	대기 중
릴레이 복구 후	—	**시도 3회에 SENT**

픽스처가 절반만 보고 있었습니다

선두 막힘 시험을 처음 쓸 때 릴레이의 거부를 **통째로 껐다가** 두 번째 건을 넣었습니다. 그러면 작업자가 첫 건을 다시 시도해 성공해 버리므로, 미루기를 빼도 두 번째 건이 나갑니다 — 시험은 떨어졌지만 **엉뚱한 이유로** 떨어졌습니다.

가짜 릴레이를 **주소별 거부**로 고쳤습니다. 이제 미루기를 빼면 `the second writer's mail never went out — the refused one is still holding the queue` 로 정확히 떨어집니다.

기다리는 것과 멈춘 것

작성자 화면에 `다음 시도` 시각을 함께 보입니다. 그 값이 없으면 대기 중인 건과 멈춘 건이 같아 보입니다.

3.10.135 v0.180.0에서 완료 — 세 시간 뒤에나 닿는 끝, 그리고 씨앗이 못 만들던 화면

v0.179.0이 재시도 간격을 벌리면서 **포기 상태가 실제로는 약 3시간 뒤에나 도달**하게 됐습니다. 아무도 우연히 그 자리에 닿지 않는다는 뜻이고, 그래서 시험이 필요합니다.

예산을 다 쓴 발송은 `FAILED` 로 바뀌고 **릴레이가 마지막으로 한 말**을 지녀야 합니다. 그러지 않으면 영원히 `대기 중` 으로 남아, 작성자는 오지 않을 메일을 기다립니다.

- 포기 조건을 없애면 → `status=QUEUED after the budget ran out, want FAILED`
- 사유를 안 남기면 → `the last thing the relay said was lost: ""`
- 그리고 포기한 건을 다시 집지 않는지도 함께 봅니다

씨앗이 이 화면에 닿지 못했습니다

`user_mail_settings` 와 `report_mail_deliveries` 가 씨를 뿌린 배포에서 **둘 다 비어 있었습니다.** 개인 설정의 발송 카드는 언제나 빈 상태만 보여 줬습니다 — 저장된 주소도, 발송 이력도, 읽어 볼 실패 사유도 없이. 이번 주기가 여섯 번 캔 바로 그 자리입니다.

다섯에 한 명이 발송을 켜 두고, 최근 12주의 **제출된** 보고서에 발송 기록이 붙습니다.

발송됨	621건
대기 중	63건 — 사유와 다음 시도 시각을 지님
실패	43건 — 릴레이가 준 문장 그대로

처음 쓴 픽스처가 만들 수 없는 상태를 만들었습니다

화면을 열어 보고 둘을 고쳤습니다.

하나 — 보낸 시각이 내일이었습니다. 이번 주 발송을 그 주 금요일로 찍었는데, 금요일은 아직 오지 않았습니다. `least(..., now() - interval '3 hours')` 로 눌렀습니다.

둘 — 작성 중 보고서에 발송 기록이 붙었습니다. 발송은 제출이 큐에 넣는 것이므로, 제출되지 않은 보고서에 딸린 발송은 제품이 만들 수 없는 행입니다. `r.status <> 'DRAFT'` 를 걸었습니다.

다양성 검사도 함께

v0.177.0이 넣은 검사에 발송 상태 가짓수를 더했습니다. 세 가지 미만이면 씨앗이 실패합니다 — 보낸 것·기다리는 것·포기한 것을 구별할 수 없는 픽스처는 그 화면을 시험할 수 없기 때문입니다.

3.10.136 v0.181.0에서 완료 — 운영자에게 절반만 알려 준 메모리

관리자 화면의 캡처 첨부 설명이 이렇게 말하고 있었습니다.

> 요청 하나가 쓰는 메모리는 대략 **한 보고서의 첨부 총량**입니다. 즉 `보고서당 최대 캡처 수 × 캡처 파일당 최대 MB` 가 상한입니다.

씨앗은 `report_attachments` 를 만들지 못하므로 이 경로는 **한 번도 규모에서 밟힌 적이 없었습니다.** 캡처를 만들어 실제 API로 올리고 재 봤습니다.

첨부	캡처 20장 · 44.9MB
내보내기 전	78MiB
피크	**235MiB**
상승	**157MiB = 3.5배**

두 번 반복해 156MiB·157MiB로 재현했습니다.

기본값 20장 × 10MB 라면 요청 하나가 **약 700MB**입니다. 안내대로 200MB를 잡은 운영자는 내보내기 한 번에 OOM 으로 죽습니다. 폐쇄망에서 컨테이너 한도를 그 문장 보고 정합니다.

어디서 불어나는가

이미지는 이미 **하나씩** 읽습니다 — `slideMedia.Load` 가 함수이고, 주석이 그렇게 하는 이유까지 적어 두었습니다. 불어나는 것은 완성된 덱 자체입니다. `Content-Length` 를 세려면 파일을 다 만들어 들고 있어야 하고, zip 을 새로 쓰는 동안 입력과 출력이 함께 있습니다.

그래서 코드를 바꾸는 대신 **실측한 값을 말하게** 했습니다.

그런데 그 "하나씩"을 지키는 것이 없었습니다

주석은 "deferring the read is what keeps a whole export's worth of images from being resident at once" 라고 주장하는데, 그 주장을 검사하는 시험이 없었습니다. 누가 `Load` 를 바이트로 되돌려도 아무것도 실패하지 않습니다 — 그리고 그 순간 메모리는 3.5배에서 4.5배가 됩니다.

시험을 넣었습니다. 계획 단계에서 한 장이라도 읽으면 `6 capture(s) were read while the slides were only being planned`, 한 장을 두 번 읽으면 `read 2 times, want 1` 로 떨어집니다.

3.10.137 v0.182.0에서 완료 — 사용법을 물었더니 부하 시험이 시작됐습니다

이번 창에서 한 번도 쓰지 않은 도구가 하나 남아 있어 사용법을 물었습니다.

```
$ python3 scripts/load-check.py --help
Exception in thread Thread-5 (writer):
Exception in thread Thread-4 (writer):
Exception in thread Thread-7 (writer):
... (스레드 120개)
```

`sys.argv[1]` 을 파싱 없이 주소로 읽기 때문에 **`--help` 가 대상이 되고 120개 스레드가 그것을 두드립니다.** 형제 스크립트는 전부 argparse 를 씁니다. 이 하나만 아니었습니다.

그리고 이 도구는 읽기만 하지 않습니다

`writer` 는 보고서를 새로 만드는 것이 아니라 **이번 주 것을 덮어씁니다.** u1..uN 의 업무 항목이 전부 `부하 업무 0..6` 이 되고 원래 내용은 남지 않습니다.

이 위험은 파일 맨 위에 이미 적혀 있었습니다.

> Point it at a scale seed, never at a deployment holding real work.

문서에만 있고 지키는 장치가 없었습니다. 이번 창에서 계속 만난 그 모양입니다. `--yes` 를 요구하고, 거부할 때 무엇을 덮어쓸지 그 자리에서 말합니다.

```
거부했습니다. 이 도구는 읽기만 하지 않습니다.

  대상       http://127.0.0.1:19262
  덮어쓸 것  u1..u20 의 이번 주 보고서 — 업무 항목이 전부
             '부하 업무 0..6' 으로 바뀌고 원래 내용은 남지 않습니다.
```

고친 뒤 실제로 돌렸습니다

동시	쓰기 250명 · 읽기 50명 · 3회전
총 시간	7.4초
저장	중앙 22ms · p99 65ms
조회	중앙 142ms · p99 316ms
오류	**0건**
메모리 피크	1,264MiB

Argon2id 예약이 큐로 묶여 있어 300명이 한꺼번에 들어와도 버팁니다 — 도구가 기록하려던 그 동작이 그대로입니다.

배포 매니페스트가 절반만 말하고 있었습니다

`deploy/kubernetes.yaml` 의 메모리 설명이 이렇게 시작합니다.

> Memory is sized by what password hashing reserves, which is **the only part of this service that holds a large block for the length of a request**.

v0.181.0 에서 잰 캡처 첨부 내보내기가 그 문장을 뒤집습니다. 기본 설정에서 내보내기 하나가 약 700MB 를 요구하는데, 한도는 1Gi 입니다. **월요일 아침과 내보내기가 겹치면 죽습니다.**

둘 다 적도록 고쳤습니다 — 무엇이 크게 잡는지, 얼마인지, 그리고 1Gi 가 무엇까지 버티고 무엇은 못 버티는지.

3.10.138 v0.183.0에서 완료 — 같은 데이터에 문이 둘, 한쪽만 배웠습니다

씨앗에 개인 API 키가 없어 **MCP 표면은 데이터와 함께 불려 본 적이 없었습니다.** 키를 만들어 세 도구를 규모 배포에 대고 불렀습니다.

`period_report_rollup` 이 **2,563,406 바이트**를 돌려줬습니다.

문	항목	주차 데이터를 든 항목	크기
화면 (HTTP)	263	**20**	430KB
MCP	263	**263**	**2.5MB**

화면 쪽은 주차별 이력이 885KB 중 522KB를 차지한다는 사실을 알고 v0.164 무렵부터 잘라 왔습니다. MCP 쪽은 `loadRollup` 결과를 **그대로** 돌려줍니다. `trimTimelineSeries` 는 HTTP 핸들러에만 있었습니다.

그리고 `timelineItems: 0` 이라고 말하면서 263개가 전부 주차 데이터를 답니다 — 숫자와 데이터가 어긋납니다.

화면보다 더 잘라야 합니다

받는 쪽이 사람이 아니라 언어 모델입니다. 표를 스크롤해 중요한 세 줄을 찾는 것이 아니라, **이 payload 가 그 모델이 가진 전부**이고 모든 바이트를 컨텍스트로 지불합니다.

- 화면과 **같은** `trimTimelineSeries` 를 씁니다 — 한 곳에 하나 (v0.175의 교훈)
- 항목도 100건으로 자릅니다. 보고서 검색이 이미 정착시킨 수입니다
- 그리고 **문장으로** 말합니다. 검색 도구가 이미 그렇게 합니다

> 업무 263건 중 주차별 진척 이력(weeks)은 상위 20건에만 있습니다. **나머지 항목의 weeks 가 비어 있는 것은 진척이 없었다는 뜻이 아닙니다.** 업무 263건 중 100건만 반환했습니다. 이 목록을 전체로 보고 요약하지 마세요.

가운데 문장이 중요합니다. 자르고 나면 빈 `weeks` 가 "그 업무는 아무 일도 없었다" 로 읽힙니다.

그리고 같은 답을 두 번 보내고 있었습니다

규격이 `structuredContent` 와 텍스트 사본을 둘 다 요구합니다. 그런데 사본이 **들여쓰기**되어 있었습니다.

structuredContent	214KB
들여쓴 텍스트 사본	**316KB**

공백에 절반을 더 씁니다. 이걸 편집기로 여는 사람은 없습니다.

	크기
처음	2,563,406 B
자르기·상한 뒤	517,073 B
들여쓰기 제거 뒤	**421,379 B**

곁가지 — 기간 보고 계층 변이 검사 완료

가드 8개를 격리된 워크트리에서 전부 돌렸습니다. 살아남은 변이는 **모두 다른 시험이 잡았습니다.** 빈틈 없음입니다.

3.10.139 v0.184.0에서 완료 — 재는 자가 못 보던 두 번째 문

v0.183.0의 2.5MB는 제가 **손으로 불러 봐서** 찾았습니다. 그동안 `scale-check.py` 는 같은 배포를 "지적 사항 없음" 이라고 보고하고 있었습니다. MCP 를 아예 부르지 않기 때문입니다.

v0.171에서 같은 일을 겪었습니다 — 그때는 범위 매개변수를 안 줘서 663바이트를 재고 있었고, 이번에는 문 하나를 통째로 안 보고 있었습니다.

MCP 도구 호출을 화면과 **같은 잣대**로 잽니다. 세션으로 키를 발급받아 부르므로 새 설정이 필요 없습니다.

```
 상태            크기         지연  MCP 도구
200          599B       23ms  weekly_submission_overview
200       52,735B       10ms  weekly_reports_search
200      421,379B      116ms  period_report_rollup
```

도구 결과에는 스크롤이 없습니다

화면이 잘리면 사람은 "N건 중 M건" 을 보고 스크롤합니다. 도구 결과가 잘리면 모델은 **알 방법이 없습니다.** 그래서 크기·지연에 더해 하나를 더 봅니다 — 목록에 총계가 붙어 있고 그 총계가 실린 개수보다 큰데 `note` **가 없으면** 지적합니다.

고치기 전 빌드에 대고 확인했습니다

```
 200   2,563,406B     130ms  period_report_rollup  <== 2,563,406B > 상한 1,000,000B

지적 사항 1건 (확인 21개 경로)
  mcp:period_report_rollup
    - 2,563,406B > 상한 1,000,000B
```

v0.182.0 컨테이너를 띄워 실제로 잡히는 것을, v0.183.0 에서 종료 코드 0 으로 통과하는 것을 함께 봤습니다. 다음에 누가 이 문으로 큰 응답을 흘리면 손이 아니라 도구가 먼저 말합니다.

3.10.140 v0.185.0에서 완료 — 한 줄에 두 가지, 어느 쪽도 아닌 숫자

관리자가 서비스가 건강한지 물을 때 여는 화면에 이런 줄이 있었습니다.

```
GET /   ·  55건  ·  거부 78.18%
```

열두 건은 **사이트가 정상적으로 열린 것**이고 마흔세 건은 **존재하지 않는 API 경로**를 부른 것입니다. 없는 API 경로는 애플리케이션 껍데기를 내주는 캐치올에 걸리므로 둘 다 `/` 로 기록됩니다.

한 줄, 두 가지 사실, 그리고 **어느 쪽에도 참이 아닌 백분율**. 화면은 멀쩡한데 고장 난 것처럼 보이고, 잘못된 경로를 마흔세 번 부른 클라이언트는 이름조차 없습니다.

갈랐습니다.

	이전	지금
`/`	55건 · 거부 78.18%	**12건 · 거부 0%**

	이전	지금
`(unknown API path)`	—	**44건**

있는 경로가 낸 404 는 자기 이름을 지킵니다. `GET /api/v1/reports/{id}` 가 없는 보고서로 404 를 내는 것은 없는 경로를 부른 것과 다른 일이고, 관리자는 어느 엔드포인트가 없는 것을 요구받는지 알아야 합니다.

같은 질문을 두 곳에서 묻고 있었습니다

`serveSPA` 가 "이건 API 경로인가" 를 묻고, 지표 미들웨어도 같은 것을 물어야 했습니다. `isAPIPath` 하나로 합쳤습니다 — 답이 갈리면 화면의 이름과 실제 응답이 어긋납니다.

그리고 하네스가 미들웨어를 지나지 않습니다

시험을 쓰다 알았습니다. `testServer.request` 는 `a.mux` 를 직접 부릅니다. 즉 `requestMetrics` 도, `securityHeaders` 도, `recoverMiddleware` 도 **어느 하네스 시험도 지나가지 않습니다.** 이 시험은 `a.Handler()` 로 전체 체인을 지나갑니다.

곁가지 — 제 대기 루프가 틀려 있었습니다

이번 창 내내 배포마다 `curl -sf .../api/v1/health` 로 기다렸습니다. 그런 경로는 없습니다. 앱은 `/healthz` 와 `/readyz` 를 열고 매니페스트도 그것을 봅니다. 제 루프는 매번 80초를 헛되이 기다린 뒤 진행하고 있었습니다. 위 44건 중 상당수가 그 흔적입니다.

3.10.141 v0.186.0에서 완료 — 제가 방금 쓴 코드에 변이 검사를 걸었습니다

메일 발송(v0.178~0.179)을 낸 뒤, 이번 창 내내 남의 코드에 쓰던 잣대를 **제 코드에** 댔습니다. 여덟 개가 무방비였습니다.

하나 — 주소 거부 이유를 하나씩 시험하지 않았습니다

```
if value == "" || len(value) > 320 || strings.ContainsAny(value, "<>,;\r\n \t\"") {
```

시험의 거부 목록이 **한 항목당 두 가지 이상의 이유로** 걸립니다. `"이름 <a@b.test>"` 는 공백으로도 걸리고 꺾쇠로도 걸립니다. 그래서 어느 한 조항을 지워도 전부 통과합니다.

각 이유를 하나씩만 어기는 입력으로 갈랐습니다. 특히 327자 주소는 **길이 검사만이** 막습니다 — `net/mail` 은 그것을 정상으로 파싱하고, 컬럼은 320자입니다.

그리고 `parsed.Address == value` 와 `@` 개수 검사는 앞 조항에 가려 **관측되지 않습니다.** 없앨 수도 있었지만 남겼습니다 — `net/mail` 이 무엇을 받아 주는지는 이 패키지가 약속할 수 있는 것이 아니고, 틀렸을 때의 대가는 우리가 만든 헤더에 남의 주소가 실리는 것입니다. 주석으로 그렇게 적어, 다음 사람이 "시험이 없네" 하고 지우지 않게 했습니다.

둘 — 짧은 본문은 줄바꿈을 보여 주지 않습니다

본문은 base64 이고 76자마다 접어야 합니다(RFC 2045). 그런데 시험 본문이 한 줄짜리라 **접는 산술을 세 가지로 뒤집어도** 아무것도 실패하지 않았습니다.

실제 보고서 길이의 한글 본문으로 바꿨습니다. 줄 길이, 빈 줄 없음, 그리고 **되돌아오는지** 를 봅니다 — 길이는 legal 인데 내용이 어긋나는 접기가 가능하기 때문입니다.

셋 — 비밀번호가 걸린 길을 아무도 걸어 보지 않았습니다

가장 무거운 것입니다. 모든 시험이 `NONE` 릴레이를 씁니다. 그래서 **STARTTLS 분기도 인증 분기도** 통째로 뒤집을 수 있었습니다.

가짜 릴레이에 자체 서명 인증서와 `AUTH PLAIN` 을 붙였습니다. 그리고 제품 코드에 이음매를 하나 뒀습니다 — 사내 릴레이의 인증서는 시험 프로세스가 믿는 누구도 서명하지 않으므로, 그것 없이는 이 분기를 **아예 밟을 수 없습니다.** 주석에 그 이유를 적었습니다.

시험은 "도착했다" 로 만족하지 않습니다. 암호화된 길로 갔는지, 인증을 실제로 했는지를 **릴레이가 기록**해 확인합니다 — 메일은 평문으로 익명 발송해도 도착하니까요.

뒤집은 것	결과
STARTTLS 실패를 무시	잡힘
인증을 건너뜀	잡힘
인증 실패를 무시	잡힘

틀린 비밀번호가 **조용히 익명 발송되지 않는지** 도 따로 봅니다.

3.10.142 v0.187.0에서 완료 — 변이 검사가 문자열 속 부등호를 세고 있었습니다

`appendSlidesToPPTX` 에 남은 변이를 보러 갔다가, 변이 자리가 **문자열 리터럴 안**이라는 것을 알았습니다.

```
relations += fmt.Sprintf(`<Relationship Id="%s" .../>`, ...)
                         ↑ 이 < 를 > 로 바꾼 것이 "아무도 안 잡음" 으로 보고됐습니다
```

`mutate()` 는 **줄 첫머리가 `//` 인 줄만** 건너뜁니다. 문자열 안은 그대로 후보입니다. 재 봤습니다.

`internal/app` 의 변이 후보	4,163개
문자열·주석 안	**1,223개 (29.4%)**
`pptx.go` 한 파일	683개

OpenXML 을 손으로 만드는 패키지라 파일이 대부분 꺾쇠입니다. 그 결과가 "깨진 `<Relationship` 태그를 아무 시험도 못 잡았다" — 참이지만 쓸모없고, **진짜 발견을 덮습니다.** 이번 창 내내 이 도구의 출력을 신호로 읽어 왔습니다.

문자열·룬·주석 안을 건너뛰게 했습니다. Go 파서가 아니라 스캐너입니다 — 이 자리에 연산자가 들어갈 수 있는 문법은 그게 전부입니다.

고치면서 제가 한 실수

첫 판에서 마스크를 `"".join(lines)` 로 계산했습니다. 그 `lines` 는 `splitlines()` 결과라 줄바꿈이 없어, 오프셋이 **줄마다 1씩 밀렸습니다.** 결과는 "0개 적용" — 진짜 후보 19개까지 전부 걸러졌습니다.

"0개 적용, 모두 잡힘" 은 좋은 소식처럼 보입니다. 그대로 넘어갔으면 도구를 조용히 죽여 놓고 통과를 자축할 뻔했습니다. 마스크를 직접 검사해서 잡았습니다.

그래서 진짜 둘이 남았습니다

하나 — `highestSlide` 가 최대가 아니라 마지막이 될 수 있었습니다.

```
if number, convErr := strconv.Atoi(match[1]); convErr == nil && number > highestSlide {
```

`&&` 를 `||` 로 바꾸면 조건이 항상 참이 되어 `highestSlide` 는 **zip 에서 마지막으로 본 슬라이드 번호**가 됩니다. zip 은 순서를 약속하지 않습니다. 슬라이드가 내림차순으로 실린 패키지에서는 새 캡처가 `slide2` 를 다시 쓰고, **보고서의 한 페이지가 캡처로 덮입니다.**

슬라이드를 내림차순으로 다시 쓴 덱으로 시험을 넣었습니다. 변이를 넣으면 `the deck went from 4 slides to 4, want 5 — a slide was written over` 로 떨어집니다.

둘 — 관측될 수 없는 비교였습니다. `strings.Index(types, "<Override") >= 0` 의 `>=` 는 `>` 와 다를 수 없습니다. 그 부분은 XML 선언으로 시작하므로 오프셋 0 에 override 가 올 수 없습니다. 시험 대신 주석을 달았습니다 — 다음 사람이 같은 자리를 또 쫓지 않도록.

3.10.143 v0.188.0에서 완료 — 자기 존재 이유를 지키지 않던 코드

v0.187.0이 변이 검사에서 소음 29%를 걷어내자, 그 아래 있던 것이 나왔습니다.

```
// The template is four pages. A week with one task used to export four,
// three of them holding nothing but the column headers and a pair of
// dashes. That deck goes into a meeting, and nobody reads past the first
// blank page wondering whether the rest failed to load.
...
if slideIndex >= used {
```

`>=` 를 `>` 로 바꾸면 **빈 페이지가 한 장 돌아옵니다.** 아무 시험도 실패하지 않았습니다.

옆에 시험이 있었는데도

`TestTheDeckGrowsWithTheWorkAndStaysOpenable` 이 같은 코드를 덮고 있습니다. 그런데 그것이 확인하는 것은 **덱의 내부 정합성**입니다 — 슬라이드 수와 목록·콘텐츠 타입·관계의 수가 서로 맞는지. 빈 페이지가 하나 더 있어도 **네 숫자가 모두 사이좋게 하나씩 늘어납니다.**

즉 이 코드가 존재하는 이유 — 주석이 문단으로 적어 둔 그것 — 만 빠져 있었습니다.

한 건짜리 주를 내보내 **정확히 한 장**인지 봅니다. 변이를 넣으면 `one task exported 2 pages — the extra ones carry only headers and dashes` 로 떨어집니다.

이번 창에서 반복된 모양

릴리스	시험은 있었지만
v0.175	시험이 제품 로직을 자기 안에 베껴 두고 사본을 검사
v0.176	픽스처가 동점 20건뿐이라 절반만 보임
v0.186	거부 목록의 각 항목이 두 이유로 걸려 조항 하나를 지워도 통과
v0.188	시험이 **덱의 정합성**을 보고, **왜 그렇게 만드는지**는 안 봄

네 번 다 시험은 통과하고 있었습니다.

3.10.144 v0.189.0에서 완료 — 어느 서식으로 내보낼지 정하는 세 줄이 전부 무방비였습니다

내보내기가 **어느 파일 위에 보고서를 그릴지**는 세 조건이 줄줄이 정합니다. 셋 다 `&&` 를 `||` 로 바꿔도 아무 시험이 실패하지 않았습니다.

줄	뒤집으면
337	조직 전용 서식을 읽고도 **사내 공용으로 갈아탑니다**
345	방금 읽은 서식을 **내장 기본으로 덮어씁니다**
369	커스텀 서식을 **엉뚱한 렌더러로 그립니다** — 운영자가 넣은 `{{AUTHOR}}` 가 그대로 페이지에 남습니다

셋 다 결과가 PPTX 파일이라 **아래쪽 어느 검사도 눈치채지 못합니다.** 그리고 그 파일이 회의에 들어갑니다.

왜 아무도 안 밟았나

`pptx_templates` 가 씨앗에도 시험에도 비어 있고, `customPPTXPath` 는 있지도 않은 파일을 가리킵니다. 조직 서식이라는 상태를 **만들 수 있는 곳이 없었습니다** — 이번 창이 여섯 번 만난 그 모양입니다.

서식 파일을 실제로 디스크에 놓고, 조직 서식 행을 넣고, 표식을 심어 어느 파일에서 나왔는지 읽습니다.

픽스처가 두 번 저를 속였습니다

하나 — 사본으로는 두 렌더러를 구별할 수 없습니다. 처음 두 시험은 기준 덱의 사본에 표식만 심었습니다. 그러면 어느 렌더러를 타든 표식이 남으므로 369행 변이가 살아남습니다. **치환자**를 넣어야 갈립니다.

둘 — 치환자는 런 전체여야 합니다. `replacePPTXToken` 은 `<a:t>{{AUTHOR}}</a:t>` 를 통째로 찾습니다. 제 헬퍼가 기존 글자 **뒤에 붙여서** 정상 코드에서도 시험이 떨어졌습니다. 제품 규약이 맞고 제 픽스처가 틀렸습니다 — 서식을 만드는 사람도 치환자를 자기 텍스트 상자에 따로 둡니다.

이제 셋 다 잡힙니다.

3.10.145 v0.190.0에서 완료 — 계약서만 지키고 README는 안 지키고 있었습니다

저장소에는 **문서의 숫자를 코드 상수와 짝지어 지키는 시험**이 이미 있습니다.
`TestTheContractsNumbersAreTheOnesTheCodeUses` — 열두 쌍입니다.

그런데 그것이 보는 파일은 `docs/openapi.yaml` 하나입니다. **운영자가 읽는 README 는 무방비였고, 그래서 어긋나 있었습니다.**

무엇	상태
`경영 요약은 5~10건만 선정합니다`	**하한이 코드에 없습니다.** 조용한 주에는 두 건일 수도, 없을 수도
가중치 표	**여덟 줄 중 두 줄이 빠져 있었습니다** — 마감일 초과, 마감일 초과 예상

무엇	상태
동점으로 잘렸다는 표시(v0.176)	문서에 없음

가중치 표가 빠진 것이 특히 아픕니다. 그 표는 이 화면의 **논지 자체**입니다 — *"근거를 볼 수 없는 요약은 읽는 사람이 반박할 수 없고, 반박할 수 없는 요약은 판단에 쓸 수 없습니다."* 근거 목록이 불완전하면 그 문장이 스스로를 깎습니다.

짝지으려면 이름이 있어야 합니다

가중치는 `add(40, "DECISION", ...)` 처럼 호출 안에 박힌 숫자였습니다. 상수로 뽑았습니다 — 문서가 발표하는 숫자는 코드에 이름이 있어야 짝지을 수 있습니다.

그리고 같은 가드를 README 에 씌웠습니다. 열다섯 쌍입니다: 경영 요약 상한과 가중치 아홉, 중복·유사 임계, 반복 업무 세 조건, 검색 단계 전환.

양방향으로 확인했습니다. 코드에서 `digestWeightDuplicate` 를 25→35 로 바꾸면 떨어지고, README 에서 `마감일 초과 예상` 줄을 지워도 떨어집니다.

곁가지 — 인증 계층 훑기 완료

가드 일곱을 격리 워크트리에서 돌렸습니다. 무방비는 **하나**였고, 그것은 관측될 수 없는 자리였습니다.

`csrf` 의 `p != nil && p.AuthType == "session"` . `||` 로 바꿔도 오늘은 아무 차이가 없습니다 — 개인 API 키의 쓰기는 `apiKeyRequestAllowed` 가 먼저 거부하므로, 세션이 아닌 주체가 이 검사에 닿을 일이 없습니다.

시험 대신 주석을 달았습니다. 이 조건이 왜 그렇게 쓰였는지 — **키에 쓰기 범위가 생기는 날 비로소 값을 하고, 그때 발견하면 스크립트에 보낼 이유가 없는 Origin 헤더를 요구하게 됩니다.**

3.10.146 v0.191.0에서 완료 — 제가 붙인 가드 표시가 거짓말이었고, 저장소가 그것을 잡았습니다

v0.190.0 의 CI 가 빨간불이었습니다. 이번 창 첫 실패이고, 원인은 제가 넣은 시험입니다.

```
1 guard(s) cannot fail:

  internal/app/contractnumbers_test.go:58  TestTheREADMEsNumbersAreTheOnesTheCodeUses
    never executed buildDigest; never executed recurringWorkItems

A test that never runs its subject passes whatever the subject does.
```

`// guards: buildDigest, recurringWorkItems` 를 붙였는데, 그 시험은 **상수와 텍스트 파일만 읽습니다.** 두 함수를 부르지 않습니다. 표시는 그 시험이 할 수 없는 주장이었습니다.

형제 시험 `TestTheContractsNumbersAreTheOnesTheCodeUses` 에는 표시가 없습니다 — 이 종류는 함수를 실행하는 것이 아니라 **발표된 문장과 그 뒤의 상수를 짝짓기** 때문입니다. 같은 이유로 제 것도 표시가 없어야 합니다. 그 사실을 주석으로 적었습니다.

이번 창의 규율을 제가 어긴 것입니다

"시험이 자기 대상을 실행하지 않으면 대상이 무엇을 하든 통과한다" — 이 창에서 네 번 찾아낸 바로 그 모양입니다(v0.175·v0.176·v0.186·v0.188). 다섯 번째는 제가 만들었습니다.

왜 미리 못 잡았나

릴리스 전에 `go test` · `go vet` · `gofmt` · `version-check` · `openapi-check` · `npm test` 를 돌립니다. **`guard-check` 는 안 돌립니다** — 커버리지와 함께 전체 시험을 돌려 7분쯤 걸리기 때문입니다. CI 가 그것을 대신 봅니다.

그 분업 자체는 타당합니다. 잘못된 가드 표시는 **산출물을 망가뜨리지 않습니다** — 그래서 릴리스 워크플로가 아니라 CI 가 볼 일이고, v0.190.0 의 tarball 은 멀쩡합니다.

바꿔야 할 것은 규칙 하나입니다. **`// guards:` 표시를 새로 달거나 고쳤으면 릴리스 전에 `guard-check` 를 돌린다.** 그 경우에만 돌리면 되므로 매번 7분을 쓸 일도 없습니다.

3.10.147 v0.192.0에서 완료 — 필요한 순간에 못 돌리던 검사

v0.191.0 은 제가 잘못 붙인 가드 표시를 걷은 것입니다. 그때 남긴 규칙이 있었습니다 — *"`// guards:` 를 새로 달거나 고쳤으면 릴리스 전에 `guard-check` 를 돌린다."*

그런데 `guard-check` 는 **가드 하나당 `go test` 를 한 번씩** 돌립니다. 200개면 7분입니다. 릴리스 전마다 7분을 쓸 수는 없으니, 실제로는 안 돌리게 됩니다. **규칙이 지켜지지 않을 규칙이었습니다.**

범위를 좁히는 두 옵션을 붙였습니다.

`--test NAME`	그 가드 하나만
`--changed [BASE]`	시험이나 대상 이름이 diff 에 나오는 가드만 (기본 `HEAD~1`, 추적 안 된 파일 포함)

표시가 거짓이 되는 길은 둘뿐입니다 — 시험이 이름한 것에서 멀어지거나, 대상이 멀어지거나. 둘 다 **이름으로 diff 에 나타납니다.**

그 실패를 그대로 재현해 봤습니다

v0.190.0 을 깬 표시를 되살리고 새 옵션으로 검사했습니다.

```
$ python3 scripts/guard-check.py --test TestTheREADMEsNumbersAreTheOnesTheCodeUses
1 guard(s) cannot fail:
  ... never executed buildDigest; never executed recurringWorkItems

real 0m5.770s
```

5.8초. CI 가 7분 걸려 찾은 것과 같은 답입니다.

	시간
전체 (CI)	약 7분
`--changed HEAD~3` — 가드 12개	**49초**
`--test` — 가드 1개	**5.8초**

CI 는 그대로 전부 봅니다. 달라진 것은 **릴리스 전에 그 규칙을 지킬 수 있게 됐다**는 것뿐입니다.

3.10.148 v0.193.0에서 완료 — 릴레이를 가진 사람만 아무것도 못 보고 있었습니다

메일 발송(v0.178~0.186)에서 작성자는 자기 발송 이력을 봅니다. 주차별 상태, 시도 횟수, 실패했다면 릴레이가 준 사유까지.

릴레이를 소유한 관리자는 아무것도 못 봤습니다.

- 연결 시험 버튼은 **지금 이 순간**만 말합니다. 지난 월요일이 나갔는지는 말하지 않습니다.
- 릴레이가 전원을 거부하는 상태와 **아직 아무도 안 쓰는 상태**가 설정 화면에서 똑같아 보입니다.

관리자 설정의 메일 카드에 최근 14일을 붙였습니다.

```
최근 14일 · 발송 150건 · 대기 17건 · 실패 11건 · 사용자 61명
마지막 실패 사유: 받는 주소를 릴레이가 거부했습니다: 550 5.1.1 mailbox unavailable
```

개수가 아니라 문장입니다

`실패 11건` 만 있으면 운영자는 로그를 열러 갑니다. **릴레이가 준 마지막 문장**이 함께 있으면 갈 곳이 정해집니다 — 주소가 없는 것인지, 인증이 막힌 것인지, 연결이 안 되는 것인지.

한 건도 없을 때도 말합니다: *"아직 발송한 주간보고가 없습니다. 사용자가 개인 설정에서 켜야 시작됩니다."* — 0이 고장인지 미사용인지 구별되지 않으면 그 숫자는 아무 말도 하지 않습니다.

그리고 이것은 운영자의 것입니다

배포 전체의 발송 현황은 관리자만 봅니다. 시험이 작성자로 같은 경로를 눌러 403 인지도 함께 확인합니다.

곁가지 — v0.192.0 의 도구가 바로 값을 했습니다

이 릴리스에서 `// guards: adminMailHealth` 를 새로 달았습니다. v0.191.0 이 남긴 규칙대로 릴리스 전에 확인해야 하는데, 전체는 7분입니다. 어제 만든 `--changed` 로 **7개만 골라** 확인했습니다.

3.10.149 v0.194.0에서 완료 — 제출은 처음 보이게 되는 시점이 아닙니다

`canViewReport` 에는 **상태 조건이 없습니다.** 팀장과 관리자는 `작성 중` 인 보고서도 지금 열어 볼 수 있고, 팀 목록은 `작성 중` 배지 옆에 `열기 →` 를 나란히 둡니다.

작성자는 그것을 모릅니다. 초안은 제출 전까지 자기 것이라고 넘겨짚기 쉽고, 이 제품은 그 짐작을 바로잡아 주지 않았습니다.

규칙을 바꾸지 않았습니다

가시성은 배포된 조직의 기대가 걸린 정책입니다. 팀장이 주중에 진행을 보는 것이 정상인 조직도 있고, 초안은 사적인 메모인 조직도 있습니다. **혼자 바꿀 일이 아닙니다.**

바꾼 것은 놀라움입니다 — 글을 쓰는 그 화면에서 말합니다.

> 작성 중인 보고서도 **관리자와 상위 조직의 팀장은 지금 열어 볼 수 있습니다.** 제출은 검토를 요청하는 행위이지, 이 글이 처음 보이게 되는 시점이 아닙니다.

README 에도 가시성 표를 넣었습니다. 누가 무엇을 보는지, 그리고 **상태가 그것을 바꾸지 않는다**는 것.

문장과 코드를 못 박았습니다

`작성 중` 보고서를 팀장이 실제로 **내용까지** 읽는지, 같은 조직의 동료는 여전히 못 읽는지 시험이 확인합니다. 가시성을 좁히면 이렇게 떨어집니다.

```
the leader cannot read the draft the writer was told they can: 403
```

작성자에게 보인다고 말해 놓고 감추는 것은, 감춰 준다고 말해 놓고 보이는 것만큼이나 틀린 일입니다.

3.10.150 v0.195.0에서 완료 — 순서대로는 닿을 수 없는 분기

되돌릴 수 없는 작업들(병합·분리·의존 링크·삭제·복제) 가드 여덟에 변이 검사를 걸었습니다. 일곱은 깨끗했고, 마지막 하나에서 둘이 살아남았습니다.

```
if errors.As(err, &pgErr) && pgErr.Code == "23505" {
    writeError(w, 409, "REPORT_EXISTS", targetWeek + " 주차 보고서가 이미 있습니다.")
} else {
    writeError(w, 500, "DATABASE_ERROR", "복제 보고서를 만들 수 없습니다.")
}
```

같은 주차로 두 번 복제하는 시험이 **이미 있습니다.** 그런데 그 시험은 이 코드에 닿지 않습니다 — 삽입보다 먼저 `weekIsFree` 가 걸러 내기 때문입니다.

이 분기는 경쟁 상황에서만 열립니다. 두 요청이 나란히 검사를 통과하고, 하나가 유니크 인덱스에서 집니다. 한 사람이 기기 둘에서, 버튼을 두 번 눌러서, 탭 두 개에서.

진 쪽에게 무엇을 말하는가

여기가 요점입니다. 진 요청은 **아무 잘못도 하지 않았습니다.** 그런데 `!=` 로 뒤집히면 `복제 보고서를 만들 수 없습니다` 라는 500 을 받습니다 — 읽는 사람이 할 수 있는 일이 없는 문장입니다. 옳게는 *"2026-09-07 주차 보고서가 이미 있습니다"* 이고, 그건 열어 보면 되는 일입니다.

경쟁을 실제로 만들었습니다

여섯 요청을 같은 신호에 풀어 같은 주차로 복제합니다. 하나만 만들어지고, 나머지 다섯은 **409 이면서 어느 주차인지 말해야** 합니다. 500 은 하나도 없어야 합니다. 그리고 그 주차에 보고서가 정확히 하나 남아야 합니다.

되돌리면 이렇게 떨어집니다.

```
a concurrent clone answered 500, which tells the reader nothing to do
3 refusals for 5 attempts
```

시험이 그 분기에 **실제로 닿는다**는 것까지 확인한 셈입니다 — 닿지 않았다면 되돌려도 통과했을 테니까요.

3.10.151 v0.196.0에서 완료 — 제약 위반을 다루는 자리를 전부 세어 봤습니다

v0.195.0 에서 경쟁으로만 닿는 분기를 하나 찾았습니다. 하나 있었다면 다른 곳에도 있을 테니, **제약 위반을 다루는 자리를 전부** 찾았습니다.

```
$ grep -rn '23505\|23503\|pgconn.PgError' internal/app/*.go
  admin.go:624    23505  조직 코드 중복
  admin.go:628    23503  상위 조직 없음
  reports.go:535  23505  같은 주차 복제      ← v0.195.0
```

셋뿐이고, 앞의 둘은 **무방비였습니다.** 그리고 이 둘은 경쟁이 아닙니다 — 순서대로 바로 닿습니다. 같은 코드로 두 번 만들거나, 없는 상위 조직을 가리키면 됩니다.

이것이 관리자가 배포를 세울 때 처음 하는 일입니다

조직을 만드는 것은 새 배포의 첫 작업 중 하나이고, 그때 틀리는 방법은 딱 둘입니다 — **이미 쓰는 코드**와 **없는 상위 조직**. 둘 다 관리자 자신의 실수이고, 둘 다 고치는 데 몇 초면 됩니다.

중요한 것은 상태 코드가 아니라 **그 문장이 관리자를 어디로 보내느냐**입니다.

답	관리자가 하는 일
`이미 사용 중인 조직 코드입니다`	코드를 바꿔 다시 누릅니다
`조직을 만들 수 없습니다` + 500	**없는 장애를 찾으러 로그를 엽니다**

시험이 보는 것

코드와 상태만이 아니라, 거절이 **무엇이 문제인지 말하는지**, 그리고 거절된 생성이 **행을 남기지 않는지** 까지 봅니다.

세 가지로 되돌려 확인했습니다 — 중복을 500 으로, 없는 상위를 500 으로, 그리고 둘의 답을 뒤바꿔서. 셋 다 떨어집니다.

3.10.152 v0.197.0에서 완료 — 유니크 제약이 사람을 만나는 아홉 자리

v0.196.0 은 제약 위반을 다루는 **코드**를 셌습니다. 이번에는 **스키마**를 셌습니다 — 유니크 제약 아홉 개 중 사용자 입력이 닿는 것이 넷입니다.

제약	상태
`organizations(code)`	v0.196.0
`weekly_reports(user_id, week_start)`	사전 검사 + v0.195.0
`users(username)`	**문자열 검색으로 판정, 시험 없음**

제약	상태
`work_item_links(blocker_id, blocked_id)`	코드는 옳고 **시험 없음**

나머지 다섯은 토큰 해시나 동기화 경로라 사람이 부딪히지 않습니다.

문자열로 판정하고 있었습니다

```
if strings.Contains(err.Error(), "users_username_key") {
```

동작합니다 — **드라이버가 그 문장을 계속 그렇게 쓰는 동안만.** 형제 두 곳은 구조화된 `pgErr.Code` 를 씁니다.

다만 코드만으로는 부족합니다. `users` 에는 유니크 제약이 **둘** 있습니다(username, oidc_subject). 23505 만 보면 *관리자가 아이디를 재사용한 것* 과 *SSO 주체가 이미 물린 것* 이 한 답이 되고, 그 둘은 고치는 방법이 다릅니다. 그래서 코드와 **제약 이름을 함께** 봅니다.

시험이 그것을 붙듭니다. 인덱스 이름을 바꾸면 떨어집니다 — 조용히 500 이 되는 대신에.

그리고 새 도구가 릴리스 전에 저를 잡았습니다

`// guards: adminCreateUser` 라고 적었는데 그런 함수는 없습니다(`createUser` 입니다).

v0.190.0 에서는 이 종류를 **CI 가 7분 뒤에** 잡았고 빨간 태그가 하나 남았습니다. 이번에는 v0.192.0 이 만든 `--changed` 가 커밋 전에 잡았습니다.

```
never executed adminCreateUser
A test that never runs its subject passes whatever the subject does.
```

도구를 고친 두 걸음(v0.191·v0.192)이 세 걸음 만에 값을 했습니다.

3.10.153 v0.198.0에서 완료 — 아홉 개의 검사를 알려면 아홉 개의 소스를 읽어야 했습니다

이 저장소에는 `go test` 가 잡지 못하는 것을 잡는 검사가 아홉 개 있습니다. 목적도 값도 서로 다릅니다 — 즉시 끝나는 것, 7분 걸리는 것, 배포가 하나 필요한 것, **데이터를 덮어쓰는 것**.

README 는 그중 둘만 언급했습니다. 나머지를 알려면 소스를 읽어야 했고, **이번 창에서 제가 그렇게 알았습니다** — 그리고 그 과정에서 둘의 함정을 찾았습니다(v0.182 의 `load-check --help`, v0.192 의 `guard-check` 7분).

`docs/CHECKS.md` 로 모았습니다. 무엇을 잡는지, 얼마나 걸리는지, **언제 돌리는지**.

문서는 조용히 낡습니다

열 번째 검사가 생기고 이 페이지에 안 적히면, 그 페이지를 믿는 사람은 그 검사가 있는 줄도 모릅니다. 그래서 페이지를 **디렉터리와 대조**합니다.

- `scripts/*-check.*` 인데 페이지에 없으면 실패 — *"a check nobody knows about is one nobody runs"*
- 페이지가 가리키는 파일이 없으면 실패 — 지워진 것을 찾아 헤매지 않도록

둘 다 실제로 만들어 확인했습니다.

그리고 이 시험에는 가드 표시가 없습니다

문서와 파일 목록을 짝짓는 것이지 함수를 실행하는 것이 아니기 때문입니다 — v0.191.0 에서 배운 것입니다. 계약 숫자·README 숫자 시험과 같은 종류이고, 셋 다 표시가 없습니다.

3.10.154 v0.199.0에서 완료 — AI를 쓰는 세 화면은 아무도 걸어 본 적이 없습니다

이번 주기가 가장 많이 캔 자리는 하나입니다: **씨앗이 만들 수 없는 상태에는 결함이 숨습니다.** 변화 일곱 종, 보고서 상태 다섯, 발송 세 상태, 조직 서식, 캡처 첨부 — 매번 그 자리에 있었습니다.

남은 가장 큰 자리가 AI 였습니다. **초안 작성 · 결정 제안 · PPTX 가져오기** 셋이 게이트웨이를 필요로 하는데, 씨앗은 그것을 켤 수 없습니다. 씨를 뿌린 배포에서 그 화면들은 언제나 *"AI Gateway가 비활성화되어 있습니다"* 만 보여 줍니다.

스키마를 읽어 답하는 가짜 게이트웨이

모델이 아닙니다. 요청에 실린 **JSON Schema 를 읽어 그것을 만족하는 문서**를 돌려줍니다. 그러면 제품의 파싱·검증·신뢰도 처리·미리보기가 **실제 HTTP 왕복 위에서** 돕니다.

돌아오는 글은 일부러 가짜인 티가 납니다 — 이 배포의 초안을 누가 모델이 쓴 것으로 오해하면 안 됩니다.

걸어 봤습니다

경로	결과
`POST /admin/settings/ai/test`	`{"ok":true,"model":"fake-model","items":1}`
`POST /ai/reports/parse-text`	요약·항목·날짜 신뢰도 0.9·경고까지

경로	결과
`POST /work-items/{id}/decisions/suggest`	후보 2건과 *"확정 전까지 아무것도 저장되지 않으며..."*
`내 주간보고` 화면	미리보기, 신뢰도 배지, 항목별 편집, `1개 업무 항목을 구조화했습니다`

결함은 없었습니다. 세 화면 모두 정상입니다 — 이제 그것을 **아는** 상태가 됐습니다.

그래서 도구로 남깁니다

`scripts/fake-ai-gateway.py`. 다음에 이 화면들을 손대는 사람은 명령 한 줄로 데이터가 있는 상태에서 걸을 수 있습니다. `docs/CHECKS.md` 에 씨앗과 함께 "화면을 걷기 위한 도구" 로 적었습니다 — 검사는 아니지만 **같은 이유로 있습니다.**

3.10.155 v0.200.0에서 완료 — 승인이 묻지도 않고 사라졌고, 목록은 자기 자신을 누르면 멈췄습니다

v0.199.0의 가짜 게이트웨이로 **PPTX 가져오기**를 처음 끝까지 걸었습니다. 파싱·근거 검증·미인용 슬라이드 경고·충돌 감지는 모두 맞았습니다. 그런데 그 화면 위에서 두 가지가 무너졌습니다.

승인이 묻지도 않고 사라졌습니다

배포에서 승인된 보고서(`#364 APPROVED`)가 있는 주차로 PPTX 를 가져와 **화면이 기본으로 골라 둔 전략**으로 확정했습니다.

```
확정 전  364 | APPROVED | reviewed_at 있음 | version 1 | 항목 7
확정 후  364 | CLOSED   | reviewed_at 없음 | version 2 | 항목 8
```

규칙 자체는 옳습니다 — 다른 경로로 본문이 바뀌었으니 검토 도장은 그 본문의 것이 아닙니다. 문제는 **말하지 않았다**는 것입니다. 화면이 한 말은 이것뿐이었습니다.

> 동일 주차 보고서 #364 (APPROVED)가 있습니다.

한글 화면 한가운데의 날 것 그대로의 영문 열거값이고, 그나마도 **무슨 일이 벌어지는지는 한 글자도 없습니다.** 이제 전략마다 결과를 말하고, 검토 기록을 버리는 전략일 때만 붉게 표시합니다.

> 동일 주차 보고서 #156(승인)가 있습니다. 이 파일의 업무 항목을 기존 보고서에 더합니다. **저장하면 상태가 확정으로 돌아가고 승인 기록은 사라집니다.**

목록은 자기 자신을 누르면 멈췄습니다

이력 목록은 열릴 때 **가장 최근 작업을 스스로 골라 둡니다.** 그 작업을 누르면 — 화면에 들어와서 가장 자연스러운 동작입니다 — 클릭 처리기가 `detail` 을 비우고 `setSelectedID(같은 값)` 를 부릅니다. 값이 같으니 다시 불러오는 효과는 돌지 않습니다.

브라우저로 재현했습니다.

```
같은 작업 다시 클릭 → 파일 카드 0
오른쪽 화면: 불러오는 중…
```

탭을 새로 열기 전까지 풀리지 않습니다. 그동안 검토 중이던 수정값은 상태에 그대로 남아 있지만 **닿을 수 없고**, 나가려 하면 저장 안 됨 경고가 붙잡습니다. 상세를 못 불러왔을 때도 `catch` 가 없어 같은 자리에서 같은 모양으로 멈췄습니다.

인라인이라 아무도 시험할 수 없었습니다

두 결정 모두 JSX 안에 한 줄로 박혀 있었습니다. `frontend/src/importConflict.ts` 와 `importPanel.ts` 로 꺼내 시험 12개를 붙였습니다(전체 79개). 화면에서 다시 걸어 네 전략의 문장, 붉은 표시, 같은 작업 재클릭이 모두 맞는 것을 확인했습니다.

3.10.156 v0.201.0에서 완료 — 운영 표가 메서드를 두 번 찍고 있었습니다

v0.200.0 을 확인하며 열네 화면을 다시 훑다가, 관리자만 보는 **최근 24시간 API 분석** 표에서 걸렸습니다.

메서드	경로
GET	`/`
GET	`GET /api/v1/me`
POST	`POST /api/v1/auth/login`

메서드 열이 따로 있는데 경로 열이 메서드를 다시 달고 있습니다. Go 는 메서드를 지정해 등록한 패턴에는 메서드를 넣고 그렇지 않은 패턴에는 넣지 않으므로, `r.Pattern` 을 그대로 저장하면 **같은 열에 두 가지 모양**이 섞입니다 — 껍데기를 내주는 `/` 만 경로처럼 생기고 나머지는 전부 겹칩니다.

기록하는 자리에서 메서드를 떼어 냈습니다. 메서드는 자기 열이 있으니 그쪽 하나면 됩니다.

```
GET    /readyz
GET    /api/v1/reports/{id}
GET    (unknown API path)
POST   /api/v1/auth/login
GET    /api/v1/me
```

기존 시험은 *"있는 경로에서 난 404 는 자기 이름을 지킨다"* 를 지키려던 것이었고 메서드 접두는 곁다리였으므로 기대값만 옮겼습니다. 여기에 **어떤 행도 메서드를 두 번 달지 않는다**는 확인을 더했습니다. 사람이 못 보고 지나칠 종류의 어긋남이라, 다음에 누가 되돌려 놓으면 시험이 말합니다.

3.10.157 v0.202.0에서 완료 — 볼 것이 없는 작업이 영영 "검토 가능" 이었습니다

이미 올린 PPTX 를 다시 올리면 모든 파일이 `DUPLICATE` 로 표시되고 대기열에 들어가지 않습니다. 실패한 것도 없습니다. 그러면 작업 상태를 정하는 두 자리가 모두 그것을 **`READY` — 화면에서 "검토 가능"** 이라고 부릅니다.

들어가 보면 고를 수 있는 파일이 없고, **나가는 길이 전부 막혀 있습니다.**

```
확정         → IMPORT_FILE_NOT_READY    선택한 파일은 확정할 수 있는 상태가 아닙니다
빈 선택 확정 → IMPORT_SELECTION_REQUIRED 확정할 Import 파일을 선택하세요
재분석       → NO_RETRYABLE_IMPORT      재분석할 수 있는 파일이 없습니다
```

그 줄은 이력 목록에 **할 일인 척 영원히 남습니다.** 한 달에 두 번 같은 폴더를 끌어다 놓는 사람은 그런 줄을 모으게 되고, 그것들은 정말로 기다리는 작업들 사이에 섞입니다.

상태를 정하는 자리가 둘이었고 서로 달랐습니다

파일 구성	업로드 응답	작업자
하나 거부·하나 중복	`PARTIAL`	`FAILED`

업로드 응답은 *무엇이 실패했는지*로, 작업자는 *볼 것이 남았는지*로 상태를 읽었습니다. 참인 문장은 **일부 실패** 입니다 — 둘 중 하나가 실패했으니까요. 이제 두 자리가 같은 함수 하나를 씁니다.

어휘를 하나 늘렸습니다

`NOTHING_TO_IMPORT` — 화면에서 **"가져올 것 없음"**. 마이그레이션 023 이 제약을 넓히고, **이미 갇혀 있던 작업들을 그 자리에서 옮깁니다.**

```
1|READY                ← 정말로 검토를 기다리는 작업
2|NOTHING_TO_IMPORT    ← 갇혀 있던 작업. 이관이 풀어 주었습니다
업로드 응답: {"id":3,"status":"NOTHING_TO_IMPORT"}
```

기존 시험이 *"일부만 실패했으면 PARTIAL"* 을 이미 정해 두었으므로 그 결정을 따랐고, 두 경로가 어긋나던 부분만 없앴습니다.

3.10.158 v0.203.0에서 완료 — 같은 고장을 한 화면은 설명하고 다른 화면은 영어로 흘렸습니다

게이트웨이가 있으니 이제 **AI 가 고장 났을 때**도 걸을 수 있습니다. 일부러 망가진 게이트웨이 여덟 개를 세워 관리자 연결 시험에 물었습니다.

게이트웨이	화면이 한 말
HTTP 500	게이트웨이 쪽 문제이므로 이 설정을 바꿔도 해결되지 않습니다
HTTP 401	관리자 설정의 AI API Key를 확인하세요
HTTP 404	Endpoint가 `/v1/chat/completions` 로 끝나는지 확인하세요
HTTP 429	요청 한도를 초과했다고 답했습니다. 잠시 후 다시 시도하세요
로그인 HTML	프록시나 로그인 페이지가 앞에 있는지 확인하세요
JSON 아님	〃
스키마 불충족	모델이 Structured Output을 지원하는지 확인하세요
죽은 포트	주소와 사내망 접근 경로를 확인하세요

여덟 가지가 전부 다르고 전부 무엇을 하라고 말합니다. **좋은 제품입니다.**

그런데 가져오기는 같은 고장을 이렇게 말했습니다

```
AI endpoint returned HTTP 500
```

Go 오류 문자열 그대로, 영어로, 나머지 모든 실패가 한국어 문장인 화면 위에 놓입니다. `processImportFile` 만 `aiUserMessage(err)` 를 거치지 않고 `err.Error()` 를 그대로 적고 있었습니다. 한 줄입니다.

일괄 가져오기를 돌리는 사람은 관리자 화면을 보고 있지 않습니다. 파일 카드에 적힌 그 한 줄이 그 사람이 받는 전부입니다.

배포에서 다시 걸었습니다

```
500     → AI Gateway가 오류로 응답했습니다(HTTP 500). 게이트웨이 쪽 문제이므로…
401     → AI Gateway가 인증을 거부했습니다(HTTP 401). 관리자 설정의 AI API Key를…
404     → AI Gateway 주소에서 해당 경로를 찾지 못했습니다(HTTP 404). Endpoint가…
HTML    → AI Gateway 주소가 JSON이 아닌 응답을 돌려줬습니다. 프록시나 로그인…
죽은 포트 → AI Gateway 주소에 연결하지 못했습니다. 주소와 사내망 접근 경로를…
```

시험은 게이트웨이가 500 을 돌려주게 해 놓고 **파일에 남은 말이 관리자 화면이 하는 말과 글자까지 같은지**를 봅니다. 고치기 전 코드에 대고 돌려 실제로 실패하는 것을 확인했습니다.

3.10.159 v0.204.0에서 완료 — 의미 검색은 켜도 4일 19시간 뒤에나 켜졌습니다

v0.199.0 의 가짜 게이트웨이에 `/v1/embeddings` 를 붙였습니다. 돌려주는 벡터는 글자에서 결정적으로 만든 것이라 뜻을 판단하는 데는 쓸 수 없지만, **임베딩 작업자·저장·차원·유사도 질의·화면이 실제로 도는지**는 이것으로 확인할 수 있습니다.

씨를 뿌린 배포에서 켜 보니 관리자 화면이 이렇게 말했습니다.

```
항목 110,534 · 임베딩 0
```

그리고 2분 뒤 **32**. 또 2분 뒤 **32**. 그대로였습니다.

2분에 한 묶음, 4일 19시간

작업자는 2분마다 깨어나 **한 묶음(32건)만** 처리하고 다시 잤습니다. 실제 일은 밀리초입니다 — 배포에서 재니 32건 왕복이 **9 ms** 였습니다. 나머지 119.99초는 놀았습니다.

110,534 ÷ 32 = 3,455 묶음 × 2분 = **4일 19시간.** 그동안 의미 검색은 우연히 임베딩된 일부에서만 답하고, 나머지에 대해서는 **아무 말도 하지 않습니다.** 이제 묶음이 가득 찼으면 곧 다음 묶음을 가져옵니다. 사이의 짧은 쉼은 실제 게이트웨이를 루프가 낼 수 있는 속도로 때리지 않기 위한 것입니다.

그랬더니 질의가 제곱이라는 것이 드러났습니다

후보를 고르는 질의는 **언제나 가장 최근 항목부터** 훑으며 이미 임베딩된 것들을 하나씩 건너뛰어 아직 안 된 32건까지 내려갔습니다. 43,200건이 끝난 시점에 한 묶음이 **버리려고 읽은 행이 43,168개**, 104 ms 였습니다. 회차마다 더 깊어지므로 말뭉치를 채우는 비용이 **크기의 제곱**입니다.

한 회차 안에서는 아래로만 내려가게 커서를 붙였습니다. 같은 시점, 같은 데이터에서:

	버린 행	실행 시간
커서 없음 (옛 방식)	19,808	**40.546 ms**
커서 있음	0	**0.080 ms**

500배이고, 옛 쪽만 회차마다 더 느려집니다. 회차 도중에 수정된 항목은 그때 이미 커서 위에 있고, 2분 뒤 다음 회차가 가져갑니다.

"다시 만들기"는 6,400건에서 멈추고도 완료라고 적었습니다

요청 하나는 끝나야 하므로 200묶음이 상한입니다. 그 상한이 **말없이** 걸렸습니다 — 로그에 `embedding rebuild complete`, 화면에 `임베딩 6400건을 생성했습니다`, 그리고 **104,134건이 빠진 채**. 계약 문서도 `모두 임베딩한다` 고 적혀 있었고 README 는 `즉시 채울 수 있습니다` 라고 했습니다. 셋 다 사실이 아니었습니다.

이제 남은 건수를 함께 답하고, 화면이 그것을 말합니다.

```
{"embedded":6400,"remaining":104134,"model":"fake-embed"}
→ 임베딩 6,400건을 생성했습니다. 104,134건이 남았고 백그라운드 작업자가 이어서 처리합니다.
```

README 의 6,400 은 이제 `embeddingRebuildBatches × embeddingBatchSize` 와 시험으로 묶여 있습니다.

3.10.160 v0.205.0에서 완료 — 검색창이 있으나 마나 한 부가 기능 때문에 1분 반을 붙잡혔습니다

v0.204.0 으로 의미 검색이 실제로 도는 배포가 처음 생겼으니, **게이트웨이가 고장 났을 때** 검색이 어떻게 되는지도 처음 볼 수 있게 됐습니다.

임베딩 게이트웨이	검색
정상	200 · 0.95초
죽은 포트 (연결 거부)	200 · 0.89초 — **깔끔합니다**
연결은 받고 답이 없음	200 · **90.7초**

연결이 거부되면 즉시 포기하고 나머지 결과를 내줍니다. 좋습니다. 그런데 **연결을 받아 놓고 답하지 않는** 릴레이 — 멈춘 프록시, 소켓만 잡고 있는 게이트웨이 — 에서는 `ai.timeout_seconds` 를 그대로 기다립니다. 기본값이 **90초**, 상한이 300초입니다.

그 값은 **모델이 주간보고 한 편을 통째로 써 내는 데 필요한 시간**입니다. 여기서 하는 일은 세 단어짜리 질의 하나를 임베딩하는 것이고, 그것은 밀리초입니다.

게다가 기다려서 얻는 것이 없습니다

의미 단계는 **더 싼 단계들이 빈손으로 돌아왔을 때만** 도는 덤입니다. 얻는 것은 몇 건의 추가 결과뿐입니다. 검색창 앞의 사람은 아무것도 못 얻을 1분 30초를 기다립니다.

자기 예산을 갖게 했습니다

`searchSemanticBudget = 5초`. 넘기면 그 단계만 포기하고 나머지 결과를 내줍니다.

```
제한시간 90초 · 응답 없는 게이트웨이   90.7초  →  6.25초
정상 게이트웨이                          0.95초  →  0.98초
```

시험은 연결을 받아 놓고 답하지 않는 게이트웨이를 세우고 `ai.timeout_seconds` 를 90 으로 둔 채 검색합니다. 고치기 전 코드에서는 **30초**(핸들러 backstop) 로 실패하고, 고친 뒤에는 **5.011초** 로 답합니다.

3.10.161 v0.206.0에서 완료 — 동기화가 받은 것이 아니라 서버가 주장한 수를 믿었습니다

AI 다음으로 큰 어둠은 **Confluence** 였습니다. **2,516줄, 라우트 열 개**, 화면 여러 곳인데 사내 Confluence 없이는 어느 배포에서도 켤 수 없습니다. `scripts/fake-confluence.py` 로 그 자리를 세웠습니다 — 연동이 실제로 부르는 REST 두 개에만, 클라이언트가 읽는 모양 그대로 답합니다.

첫 동기화가 **실패했습니다.**

```
status FAILED · pagesScanned 10080 · candidatesCreated 0
errorMessage: Confluence search exceeded the 10000 page safety limit
가짜 서버가 받은 요청 87건 — start=0 부터 start=9960 까지
```

문서는 **120개**뿐이었습니다. `start=120` 부터는 빈 결과가 돌아갔는데도 동기화는 **84번을 더 물었습니다.**

원인은 제 가짜 서버였고, 그 다음이 제품이었습니다

Confluence Server 는 `size` 에 **그 페이지의 건수**를, `totalSize` 에 전체를 담습니다. 제 서버가 총계를 `size` 로 보냈습니다 — 고쳤습니다.

그런데 루프를 보니 **받은 것을 한 번도 보지 않습니다.**

```
counters.PagesScanned += result.Size          // 서버가 주장한 수
if result.Size == 0 || result.Size < result.Limit { break }
start += result.Size
```

`result.Pages` 가 비어 있어도 상관하지 않습니다. **결과가 없는 응답은 끝났다는 뜻**이고, 그것은 어느 서버가 무슨 숫자를 적어 보내든 참입니다. 프록시 한 대, 다른 세대의 API 하나면 같은 일이 사내에서 일어납니다 — 그리고 그때 지는 것은 페이지 하나가 아니라 **동기화 전체**입니다.

`PagesScanned` 도 마찬가지입니다. **훑었다고 기록한 10,080쪽 중 실제로 본 것은 120쪽**이었습니다.

받은 것을 셉니다

```
                          고치기 전              고친 뒤
size=총계 서버          FAILED · 훑음 10080    SUCCESS · 훑음 120 · 후보 128
size=페이지 건수 서버   SUCCESS · 훑음 120     SUCCESS · 훑음 120 · 후보 128
```

서버가 성실하면 값이 같으므로 잃는 것이 없고, 성실하지 않으면 걸음을 멈춥니다. 시험은 총계를 `size` 로 보내는 서버를 세우고 동기화가 **호출 횟수와 기록한 쪽수 둘 다** 맞는지 봅니다. 고치기 전 코드에서는 바로 그 안전장치 오류로 실패합니다.

3.10.162 v0.207.0에서 완료 — 동기화가 멀쩡한 시스템을 탓했습니다

v0.206.0 으로 Confluence 를 걸을 수 있게 되자 곧바로 다음이 보였습니다. 동기화는 **SUCCESS** 로 끝났고 120쪽을 훑었는데, 오류 목록에는 이런 줄이 225개 있었습니다.

```
AI_SUMMARY  (114건): Confluence에 연결하지 못했습니다. 서버 로그에서 자세한 원인을 확인하세요.
AI_CLASSIFY (111건): Confluence에 연결하지 못했습니다. 서버 로그에서 자세한 원인을 확인하세요.
```

Confluence 는 멀쩡했습니다. 방금 끝까지 걸었고 성공했습니다. 거절한 것은 AI Gateway 입니다.

이 줄을 읽는 운영자는 Confluence 로 갑니다. 망을 보고, 계정을 보고, Base URL 을 봅니다 — 그리고 **아무 이상도 찾지 못합니다.** 고장 난 것은 거기가 아니기 때문입니다.

모든 단계를 한 함수가 설명하고 있었습니다

`recordConfluenceError` 는 단계를 가리지 않고 `safeConfluenceError` 를 씌웠고, 그 함수는 **Confluence 에 대해서만** 말할 줄 압니다. AI Gateway 를 부르는 단계도, 자기 데이터베이스에 쓰는 단계

도 전부 그 문장을 받았습니다.

단계를 셋으로 나눴습니다.

단계	실패한 것
`SEARCH` · `BODY` · `BODY_PREVIEW` · `BODY_VERSION_CHANGED`	Confluence
`AI_CLASSIFY` · `AI_SUMMARY` · `AI_CONFIGURATION`	AI Gateway — `aiUserMessage` 가 여덟 가지를 구분해 말합니다
`METADATA_STORE` · `DEDUPLICATION` · `EXCLUSION` · `IDENTITY_MAPPING`	이 서비스의 데이터베이스

배포에서 다시 돌렸습니다

```
AI_SUMMARY  (114건): AI Gateway가 유효한 구조화 결과를 반환하지 못했습니다.
                     모델이 Structured Output(JSON Schema)을 지원하는지 확인하세요.
AI_CLASSIFY (111건): 〃
```

그리고 **그 진단이 맞습니다.** 그 자리에 세워 둔 가짜 게이트웨이는 스키마를 걸어 답을 만들 뿐, 분류기가 요구하는 *모든 문서에 대해 판단을 내놓아라* 를 지키지 못합니다. 제품이 정확히 그렇게 말했습니다.

시험은 세 무리를 모두 확인하고, **AI 단계의 문장이 관리자 AI 연결 시험의 문장과 글자까지 같은지**도 봅니다 — 한 고장이 화면에 따라 두 가지 문제로 읽히면 안 됩니다.

3.10.163 v0.208.0에서 완료 — 지어낸 식별자로는 만족시킬 수 없는 계약이 있습니다

v0.207.0 이 붙인 정직한 문장이 곧바로 일을 했습니다. 동기화가 성공으로 끝나면서도 이렇게 말했습니다.

```
AI_CLASSIFY (111건): AI Gateway가 유효한 구조화 결과를 반환하지 못했습니다.
                     모델이 Structured Output(JSON Schema)을 지원하는지 확인하세요.
```

맞는 진단이었습니다. 스키마를 걸어 답을 만드는 가짜 게이트웨이로는 이 두 계약을 지킬 수 없습니다.

계약	요구
분류기	넘겨받은 **모든** 문서에 대해 판단을 내놓을 것

계약	요구
요약기	각 사실이 **어느 문서**에서 나왔는지 댈 것

지어낸 식별자는 제품이 옳게 거절합니다. 그래서 동기화의 **AI 절반 — 분류·본문 미리보기·요약·근거 검증 — 은 한 번도 걷힌 적이 없었고**, 매 동기화마다 225건의 오류만 남겼습니다.

요청에 실린 것을 돌려줍니다

게이트웨이가 요청에서 `pageId` 를 거둬 `pageIds` · `evidencePageIds` 에 그대로 채웁니다. 그러자 오류가 **225건에서 0건**이 되고 후보 114건이 만들어졌습니다.

그런데 114건이 전부 같은 문장이었습니다

이 주기가 네 번 만난 그 함정입니다. 제목 하나를 111건이 나눠 가지면 화면이 묶는지 정렬하는지 중복을 걸러 내는지 알 수 없고, `kind` 가 늘 첫 번째 값이면 **금주 실적**만 채워져 그 옆의 두 열은 아무것도 증명하지 못합니다.

자리마다 값을 달리하고, `facts` 는 **세 종류를 모두** 내게 했습니다.

```
후보 114 · 실적 있음 114 · 계획 있음 114 · 이슈 있음 114
[가짜] 회선 이설 … ‖ [가짜] 계약 표준화 … ‖ [가짜] 교육 통합 …
```

걸어 본 결과: 결함 없음

`보고서에 반영` 까지 갔습니다. 제출된 보고서에는 **409 로 거절**하고, 작성 중 보고서에서는 후보를 `ACCEPTED` 로 표시합니다. 항목을 넣지 않는 것이 맞습니다 — 화면은 항목을 **먼저 저장한 뒤** 이 호출로 표시만 하며, 계약 문서도 *"자동 초안을 저장한 본인 보고서에 연결"* 이라고 정확히 적혀 있습니다.

세 갈래 모두 정상입니다. 이제 그것을 **아는** 상태가 됐고, 다음에 손대는 사람은 명령 두 줄로 같은 자리에 설 수 있습니다.

3.10.164 v0.209.0에서 완료 — 작성자 화면이 사내 릴레이 주소를 들고 있었습니다

v0.203.0 과 v0.207.0 이 같은 자리를 두 번 고쳤습니다. 가져오기는 AI 고장을 영문 원본으로 흘렸고, 동기화는 멀쩡한 시스템을 탐했습니다. 세 번째 자리는 **제가 만든 메일 기능**이었습니다.

릴레이를 닿지 않는 주소로 두고 제출하니, `개인 설정` 의 발송 상태에 이렇게 남았습니다.

```
연결할 수 없습니다: dial tcp 10.20.0.25:25: connect: connection refused
```

```
연결할 수 없습니다: dial tcp: lookup smtp.internal.example on 127.0.0.11:53: no such host
```

릴레이의 **주소와 포트**, 그리고 컨테이너의 **DNS 서버 주소**입니다. 이 배포에서 그런 줄을 가진 사람이 **61명**이었고, 그 중 누구도 그 정보로 할 수 있는 일이 없습니다.

이 제품의 다른 연동은 전부 이것을 거릅니다 — 게이트웨이에는 `aiUserMessage`, Confluence 에는 `safeConfluenceError`. 메일만 자기 필터가 없었습니다.

그런데 릴레이가 한 말은 지우면 안 됩니다

처음엔 전부 안전한 문장으로 바꿨더니 기존 시험 둘이 무너졌습니다.

> `TestARefusedDeliveryKeepsTheRelaysOwnReason` — 릴레이가 거부하는 것은 이 망에서 흔한 실패이고, 작성자는 **둘 중 무엇인지** 구별할 수 있어야 한다.

옳은 결정입니다. `받는 주소를 릴레이가 거부했습니다: 550 5.1.1 mailbox unavailable` 은 주소가 틀렸다는 것을 **작성자에게 알려 주는 말**입니다. 새는 것은 릴레이가 한 말이 아니라 **Go 가 우리 망에 대해 한 말**입니다.

그래서 **답이 오기 전에 끝난 실패만** 바꿉니다.

남기는 것	바꾸는 것
받는·보내는 주소 거부, 본문 거부 — 릴레이의 응답	dial · no such host · i/o timeout — 우리 망
이미 사람이 읽을 문장인 것	x509 · certificate — 인증서 상세
	계정 인증 실패 — 자격 주변 문구

이미 쌓인 줄도 치웁니다

발송 줄은 보고서만큼 오래 삽니다. 코드만 고치면 **이미 그 화면에 있는 것은 영영 그대로**입니다. 마이그레이션 024 가 옮깁니다.

```
이관 전 망 주소가 남은 줄 60 → 이관 후 0
릴레이가 한 말은 그대로: 받는 주소를 릴레이가 거부했습니다: 550 5.1.1 … (22건)
```

씨앗도 같은 모양을 담고 있었습니다 — 그래서 이 누출이 **정상처럼 보였습니다.** 함께 고쳤습니다.

3.10.165 v0.210.0에서 완료 — 화면에 나가는 문장을 전부 훑었습니다

v0.203.0 · v0.207.0 · v0.209.0 이 같은 부류를 세 번 고쳤습니다. 네 번째를 기다리는 대신 **전부 훑었습니다.** `writeError` 로 나가는 문장 중 한글이 한 글자도 없는 것을 세는 검사를 돌리니 **문자열 상수는 모두 한국어**였습니다. 새는 자리는 `err.Error()` 를 그대로 잇는 곳이었고, 둘이 남아 있었습니다.

관리자 화면이 `embedding is disabled` 라고 말했습니다

`검색` 설정 줄은 이렇게 보였습니다.

```
pgvector 사용 가능 · 임베딩 0/110534 · embedding is disabled
```

`임베딩 다시 생성` 을 누르면 `의미 검색이 활성화돼 있지 않습니다: embedding endpoint or model is not configured` . 게다가 **Endpoint 가 빈 것인지 모델이 빈 것인지 말하지 않습니다.** 네 상태를 네 문장으로 갈랐습니다.

```
의미 기반 검색 사용이 꺼져 있습니다.
임베딩 Endpoint와 모델이 비어 있습니다.
임베딩 Endpoint가 비어 있습니다. OpenAI 호환 /v1/embeddings 주소를 넣으세요.
임베딩 모델 이름이 비어 있습니다.
```

OIDC 만 자기 문장이 없었습니다

게이트웨이에 `aiUserMessage` , Confluence 에 `safeConfluenceError` , 메일에 v0.209.0 의 `mailUserMessage` 가 있습니다. **로그인을 결정하는 연동에만** 없었고, 고정 접두사 뒤에 원본 오류를 이어 붙였습니다.

```
Keycloak OIDC Discovery 연결에 실패했습니다: Get "http://…": dial tcp 192.168.65.254:18899: connect:
```

그리고 주소가 평범한 404 페이지를 돌려주면 **HTML 문서 전체가 알림 안에** 들어갔습니다.

아홉 가지를 갈랐습니다 — 연결 거부·이름 해석 실패·응답 없음·인증서·`.well-known` 없음·거부·제공자 오류·**issuer 불일치**·JSON 아님. 마지막 둘은 Keycloak 설정에서 가장 흔한 실수인데 이전에는 영문 꼬리에 묻혀 있었습니다.

```
죽은 포트 → Issuer 주소에 연결하지 못했습니다. 주소와 사내망 접근 경로를 확인하세요.
404 응답 → Issuer 주소에서 .well-known/openid-configuration 을 찾지 못했습니다.
```

```
Issuer URL이 realm 루트(예: https://sso.internal/realms/weekly)인지 확인하세요.
```

시험은 **알림에 넣기에 너무 긴 문장**과 흘러나온 망 주소를 함께 봅니다. 전체 오류는 traceId 와 함께 서버 로그에 그대로 남습니다.

3.10.166 v0.211.0에서 완료 — 매니페스트는 한 프로세스를 요구하고 기본 전략은 둘을 돌립니다

배포 파일이 `replicas: 1` 과 `ReadWriteOnce` 를 쓰면서 **`strategy` 를 정하지 않았습니다.** Kubernetes 의 기본값은 `RollingUpdate` 이고, 그것은 `replicas: 1` 에서도 **새 파드를 먼저 띄운 뒤** 옛 파드를 내립니다. 업그레이드하는 동안 두 프로세스가 겹칩니다.

읽어서 안 것이 아니라 **재현했습니다.** 컨테이너 두 대를 같은 데이터베이스에 붙였습니다.

기동 복구는 겹침을 전제하지 않습니다

메일과 Import 는 `FOR UPDATE SKIP LOCKED` 로 한 건씩 집어 가므로 겹쳐도 안전합니다. 그런데 **기동할 때 하는 복구**는 그렇지 않습니다.

```
UPDATE import_jobs SET status='PENDING' WHERE status='PROCESSING'
```

크래시가 남긴 일을 되살리는 문장이고, 한 번에 한 프로세스만 돌 때는 옳습니다. 겹쳐 있는 동안 그 `PROCESSING` 은 중단된 것이 아니라 **옛 프로세스가 붙들고 있는 것**입니다.

느린 AI 게이트웨이로 파일 하나를 오래 붙잡아 두고 두 번째를 띄웠습니다.

```
started_at 13:04:16.012   ← B 의 기동 시각. A 의 것을 덮었습니다
ESTAB 127.0.0.1:34604 → 127.0.0.1:18820
ESTAB 127.0.0.1:52688 → 127.0.0.1:18820   ← 같은 파일, 연결 두 개
```

B 는 되돌린 직후 **스스로 그것을 집어 갔고**, 같은 PPTX 를 둘이 동시에 AI 로 분석했습니다. 같은 `import_files` 행에 두 번 쓰고, `started_at` 은 일을 하지 않는 쪽의 시각이 됩니다. Confluence 동기화도 같은 모양의 기동 복구를 갖고 있습니다.

`ReadWriteOnce` 도 같은 답을 가리킵니다

새 파드가 다른 노드에 잡히면 옛 파드가 볼륨을 놓을 때까지 붙지 못합니다. 롤아웃이 거기서 멈추고, 그동안 옛 파드는 계속 서비스하므로 **누가 볼 때까지 아무 일도 없어 보입니다.**

`strategy: Recreate`

한 줄입니다. 시험은 매니페스트를 읽어 `replicas: 1` · `Recreate` · `ReadWriteOnce` 가 함께 있는지 보고, compose 가 두 벌을 띄우지 않는지도 확인합니다. 이유는 매니페스트와 README 양쪽에 적었습니다 — 다음에 누가 `RollingUpdate` 로 되돌리려 할 때 읽을 자리는 코드가 아니라 거기입니다.

3.10.167 v0.212.0에서 완료 — 복구 검사가 첨부만 보고 비밀은 보지 않았습니다

`backup-check.sh` 는 *"복구 스크립트가 실제로 되돌릴 수 있는지 증명한다"* 는 이유로 있습니다. 현재 스키마에 대고 돌리니 **통과합니다** — 덤프·아카이브·검증·복구가 왕복하고, 빠진 첨부는 실패로 보고합니다.

그런데 `weekly-backup.sh restore` 는 마지막에 이렇게 안내합니다.

> `If secrets read as 복호화할 수 없음, WEEKLY_ENCRYPTION_KEY does not match the one in use when this backup was taken.`

작성자는 그것이 중요하다는 것을 알고 있었습니다. **검사는 그것을 보지 않았습니다.**

비밀에는 쪽이 둘 있습니다

어디에	무엇이
상태 볼륨	`instance.key` — 이것 없이는 암호문이 잡음입니다
PostgreSQL	`enc:v1:…` 암호문 — 이것 없이는 키가 아무것도 풀지 않습니다

검사가 한 확인은 `[ -f "$restored/instance.key" ]` 하나였습니다. **빈 파일도 통과합니다.** 그리고 암호화된 설정 행은 아예 넣지도, 보지도 않았습니다.

첨부가 전부 돌아와도 비밀이 안 풀리면 서비스는 **기동조차 하지 않습니다** — 이 제품은 복호화할 수 없는 상태로 뜨느니 멈추는 쪽을 고릅니다. 오늘 제가 볼륨 없이 컨테이너를 갈아 끼웠을 때 직접 그것을 봤습니다.

두 쪽이 함께 돌아오는지 봅니다

키의 바이트를 백업 전에 재고 복구 뒤에 견주며, 암호문과 평문 설정을 하나씩 심어 그대로 돌아오는지 봅니다.

```
attachment files: identical to the originals
secrets: key bytes and ciphertext both intact
```

새 확인이 실제로 실패를 잡는지도 확인했습니다.

```
복구가 키를 비워 두면 → instance.key came back with different bytes — every secret setting is unread
암호문이 사라지면     → the encrypted setting came back as ''
```

3.10.168 v0.213.0에서 완료 — 권한 검사가 알지 못한 것까지 "알아차린다"고 셌습니다

권한 거부를 하나씩 지워 아무도 눈치채는지 보는 스윕을 **36곳 전부** 돌렸습니다. 결과는 좋았습니다 — 그런데 요약 한 줄이 실제보다 많이 말했습니다.

```
?  [18/36] middleware.go:165  writeError(w, http.StatusForbidden, "API_KEY_SCOPE_DENIED", reason
...
권한 검사: 36곳 모두, 없애면 시험이 알아차립니다.
```

`?` 는 스크립트 자신의 주석으로 *"지워도 컴파일되지 않았다; 아무것도 알지 못했다"* 입니다. 그런데 요약은 그 자리를 **알아차린 쪽에 넣어 셌습니다.**

이 검사가 존재하는 이유가 바로 그 구분입니다 — *"이름이 규칙을 말하는 시험이 있어도, 그 규칙을 시험하는 것은 아닙니다."* **아무도 시험할 수 없었던 자리는 누군가 지키고 있는 자리가 아닙니다.**

```
1곳은 지워도 컴파일되지 않아 이 방법으로는 알 수 없습니다:
  internal/app/middleware.go:165
  이 자리들은 다른 방법으로 확인해야 합니다.

권한 검사: 35곳은 없애면 시험이 알아차립니다. 나머지 1곳은 알 수 없습니다.
```

그 한 자리는 손으로 확인했습니다

API Key 범위 거부입니다. 세 방향에서 지켜지고 있었습니다 — 경로별 표 시험, **좁은 키와 넓은 키로 `/mcp` 를 실제로 두드리는 왕복 시험**, 그리고 설정 선행조건 시험. 검사가 "알 수 없다"고 말하게 만든 것이 옳았고, 답은 사람이 채웠습니다.

그리고 갈 수 없는 분기 하나

회의 모드 변이 스윕에서 **아무도 잡지 못한** 변이가 하나 나왔습니다.

```
if _, err := time.Parse("2006-01-02", week); err != nil { ... return }   // 254행
if parsed, err := time.Parse("2006-01-02", week); err == nil { ... }     // 261행
```

주차를 두 번 파싱합니다. 첫 번째가 이미 돌려보냈으므로 **두 번째의 `err == nil` 은 거짓이 될 수 없습니다.** 뒤집으면 26주 이력 하한이 통째로 사라지는데 아무도 알아차리지 못합니다 — 갈 수 없는 분기에는 아무도 시험을 쓰지 않기 때문입니다. 한 번만 파싱해 둘 다에 씁니다.

52주를 담은 배포에서 재니 **26주 창 밖으로 이어지는 이슈는 0건**이었습니다. 그 경계는 숫자를 바꾸지 않고 읽는 양만 줄입니다 — 상수 `meetingHistoryWeeks` 와 주석에 그 측정을 적었습니다.

나머지 세 변이는 *"다른 시험이 잡았다"* 였고, 확인해 보니 `TestTheMeetingsCrossOrgWaitFollowsTheScopeSwitch` 가 실제로 두 단언 모두로 터뜨립니다. **도구는 정확했습니다.**

3.10.169 v0.214.0에서 완료 — 반려가 아무 말 없이 아무것도 하지 않았습니다

제품의 중심 고리를 브라우저로 끝까지 걸었습니다. 팀장 검토를 켜고, 팀 주간보고에서 구성원의 보고서를 열고, 반려하고, 작성자로 로그인해 사유를 확인하고, 고쳐서 다시 올리고, 승인했습니다.

```
SUBMITTED → REVISION_REQUESTED   이슈에 담당자가 없습니다. 누가 맡는지 적어 주세요.
REVISION_REQUESTED → SUBMITTED
SUBMITTED → APPROVED             확인했습니다.
```

`reviewed_by` 가 팀장으로 남고 사유가 이력에 실리며, 작성자 화면에 **팀장이 쓴 문장이 그대로** 보입니다. **고리 전체가 맞습니다.**

그런데 첫 시도는 아무 일도 일어나지 않았습니다

반려를 누르고 사유 창에서 그냥 확인을 눌렀더니 화면이 **아무 말도 하지 않았고** 보고서는 그대로였습니다.

```
const comment = prompt('반려 사유를 입력하세요.') ?? ''
if (action === 'reject' && !comment.trim()) return   // 조용히 끝
```

`prompt()` 는 취소하면 `null`, 빈 채로 확인하면 `''` 를 돌려줍니다. 화면은 **둘을 같게** 다뤘습니다. 취소가 조용한 것은 옳습니다 — 마음을 바꾼 것이니까요. 확인은 **시도**이고, 아무것도 하지 않는 시도는 왜인지 말해야 합니다. 그러지 않으면 검토자는 반려를 몇 번이고 다시 누르며 화면 어디가 고장 났는지 찾습니다.

셋을 가릅니다

입력	결과
취소 (`null`)	조용히 끝

입력	결과
빈 채로 확인 (`''`)	**반려 사유를 입력해야 반려할 수 있습니다. 작성자가 무엇을 고쳐야 하는지 알 수 있게 적어 주세요.**
사유 있음	앞뒤 공백을 다듬어 보냄

승인 의견은 여전히 선택입니다. JSX 안에 한 줄로 박혀 있던 판단을 `frontend/src/reviewAction.ts` 로 꺼내 시험 넷을 붙였고(전체 83개), 배포에서 다시 걸어 빈 확인이 이유를 말하고 취소는 조용하며 보고서 상태가 바뀌지 않는 것을 확인했습니다.

3.10.170 v0.215.0에서 완료 — 흔들리는 실패 하나가 지키는 사람 행세를 했습니다

v0.213.0 이 권한 검사에 *"알 수 없는 자리"* 를 따로 세게 했습니다. 그 뒤 같은 검사를 두 번 돌렸더니 한 자리의 답이 **서로 달랐습니다.**

```
전체 36곳 실행:  .  [16/36] mcp.go:56   ← 시험이 알아차림
앞 18곳 실행:    1곳은 없애도 아무도 알아차리지 못합니다: mcp.go:56
```

손으로 확인했습니다. **그 줄을 지우고 시험 전체를 돌리면 통과합니다.**

두 가지가 참이었습니다

첫째, 그 분기는 닿을 수 없습니다. `apiKeyRequestAllowed` 가 `/mcp` 를 부르는 키에 이미 `mcp:read` 를 요구하므로, 범위 없는 키는 이 핸들러에 오기 전에 거절당합니다. 두 번째 방어선이고 오늘은 아무도 그것을 건드릴 수 없습니다.

지우지 않았습니다. 두 규칙은 **바뀔 이유가 다릅니다** — 미들웨어의 표는 *어느 경로를 만질 수 있는가*, 이쪽은 *이 엔드포인트가 무엇을 요구하는가* 입니다. 표에만 있는 범위 검사는 편집 한 번이면 사라집니다. 대신 **왜 스윕이 이 자리를 매번 무방비라고 보고하는지** 코드에 적었습니다. v0.201.0 이 CSRF 의 관측 불가능한 절반에 한 것과 같습니다.

둘째, 그리고 이쪽이 더 무겁습니다. 검사가 통과·실패만 보고 판단했습니다. 그러면 **어떤 실패든** "이 규칙을 지키는 시험이 있다" 로 읽힙니다. 스위트 어딘가의 흔들리는 시험 하나가 무방비한 거부를 지켜지는 것으로 바꿉니다 — **이 검사가 막으려는 바로 그 거짓 음성입니다.**

누가 지켰는지 적습니다

실패한 시험 이름을 거두어 각 자리 옆에 적고, 한 시험이 **서로 무관한 여러 자리의 유일한 가드**로 나오면 경고합니다.

```
⚠ TestX 하나가 12곳의 유일한 가드로 나옵니다.
  가드가 아니라 흔들리는 시험일 수 있으니 그 자리들을 손으로 확인하십시오.
```

한 핸들러의 거부가 무관한 시험의 성패를 가르는 일은 보통 없습니다. 그런 모양이 보이면 그것은 가드가 아니라 잡음입니다.

그리고 스윕이 손대지 않는 자리들

거부 뒤가 바로 `return` 이 아닌 자리는 모양이 달라 건너뜁니다. 둘 있었고, **읽어서 확인했습니다.**

자리	규칙	지키는 것
`rollup_handlers.go:58`	조직 단위 집계는 팀장 이상	작성자 거부·팀장 허용을 CSV·덱까지 확인하는 시험
`reports.go:497`	본인의 보고서만 복제	남남이 복제를 시도하는 시험

마지막 줄이 이제 그림 전체를 담습니다 — 증명된 것, 알 수 없는 것, 스윕이 아예 훑지 않은 것. **사람이 인용하는 줄이 그 줄이기 때문입니다.**

지키는 시험의 이름

각 자리 옆에 **무엇이 그것을 지켰는지** 적습니다. 관계있는 이름인지 사람이 바로 봅니다.

```
.  decisions.go:355     ← TestOnlyTheRecorderOrAnAdministratorChangesADecision
.  duedate.go:62        ← TestOnlyTheOwnerSetsAWorkItemsDueDate
.  handover.go:218      ← TestNobodyReadsSomebodyElsesHandover
.  mcp.go:48            ← TestMCPRefusesARequestFromAnotherSite
.  meeting.go:252       ← TestOrganisationWideMeetingMaterialIsForLeaders
```

그리고 닿지 않는 자리는 이유와 함께 목록에

`mcp.go` 의 `MCP_SCOPE_REQUIRED` 는 매번, 정확하게, 무방비로 보고됩니다 — 닿을 수 없으니 사라져도 아무도 모르는 것이 맞습니다. 목록이 없으면 **다음에 돌리는 사람이 같은 조사를 다시 해서 같은 답에 도착합니다.**

줄 번호가 아니라 파일과 오류 코드로 적었습니다. 줄 번호는 다음 편집이면 틀리고, **조용한 예외는 없는 것보다 나쁩니다.** 형태로 거르지 않고 하나씩 적은 이유는 `reachability_test.go` 가 자기 목록에 대해 말한 그대로입니다 — *"probably fine 이 지난 세 개가 들어온 방법입니다."*

3.10.171 v0.216.0에서 완료 — 캡처 첨부는 씨앗이 만들 수 없었습니다

씨를 뿌린 배포의 첨부는 **언제나 0건**입니다. 첨부는 절반이 PostgreSQL 의 행이고 절반이 상태 볼륨의 **파일**이라, SQL 씨앗이 행만 넣으면 기동 로그에 `attachment files are missing` 이 뜨고 캡처 패널이 전부 `파일 없음` 이 됩니다. 정직한 씨앗이라면 만들 수 없습니다.

그래서 **캡처 패널·PPTX 캡처 슬라이드·본문 앞뒤 배치·유실 파일 안내** 는 아무도 걷지 않았습니다.

제품의 업로드 경로로 올립니다

`scripts/seed-captures.py`. 로그인하고 멀티파트를 보냅니다 — 크기 상한, 개수 상한, 형식 검사, 저장 경로, 중복 해시 처리가 모두 실제로 돕니다. 파일을 볼륨에 직접 놓지 않는 이유가 그것입니다.

첫 시도에 **세 장이 파일 하나로 합쳐졌습니다.** 그림이 셋 다 같았기 때문입니다. 제품이 옳게 한 것이지만, 그러면 순서도 배치도 확인할 수 없습니다 — 이 주기가 다섯 번째로 만난 함정입니다. 장마다 색을 달리했습니다.

걸어 본 결과: 결함 없음

```
첨부 15 · 서로 다른 파일 15 · 배치 AFTER,BEFORE
```

발표 순서는 파일 이름이 아니라 `p:sldIdLst` 입니다. 그것으로 읽으니:

배치	덱 순서
`BEFORE`	**캡처 3장 → 본문 5장**
`AFTER`	본문 5장 → **캡처 3장**

화면도 맞습니다 — 그림 3장이 뜨고 버튼이 `PPTX 다운로드 (캡처 3장 포함)` 입니다.

그리고 아무도 만들 수 없던 상태

볼륨에서 파일 하나를 지우고 다시 띄웠습니다.

```
WARN attachment files are missing; captures will not display or export
     missing=1 attachments=15
     hint: /var/lib/weekly 를 영속 볼륨으로 마운트했는지…
```

패널은 남은 2장을 보여 주고 하나를 **파일 없음** 으로 표시합니다. 내보내기는 **막히지 않고** 남은 2장으로 정상 덱(슬라이드 7)을 만듭니다 — 코드 주석이 약속한 그대로입니다. *"이미지 문제로 쓴 보고서가 막히지는 않는다."*

세 갈래 모두 정상이고, 이제 그것을 **아는** 상태가 됐습니다.

3.10.172 v0.217.0에서 완료 — 같은 보고서를 두 번 내려받으면 다른 파일이었습니다

규모 검사가 **기간보고 내보내기는 재고 주간보고 내보내기는 재지 않았습니다.** 매주 모든 사람이 쓰는 경로이고, README 가 메모리 3.5배를 경고하는 바로 그 경로인데 **호출자가 가장 많은 길에 지켜보는 눈이 가장 적었습니다.**

목록에 넣자마자 지적이 나왔습니다.

```
200   18,982B   8ms  /api/v1/reports/52/export.pptx  <== 호출마다 다름(2가지)
```

스무 번 내려받아 넷이 달랐습니다

```
16 f0a35f8a…    3 b5345f44…    1 c93db9ec…    1 bd1f92d1…
항목 순서 종류 3가지
```

바뀐 것은 아무것도 없는 보고서입니다. 항목 서른 개의 **내용은 매번 완전히 같고**, 시각도 같고, 오직 세 부분의 **순서**만 움직였습니다.

맵을 순회하며 zip 을 썼습니다

```
for name, data := range deferred {   // Go 는 맵 순회 순서를 섞습니다
```

`[Content_Types].xml`, `ppt/presentation.xml`, `ppt/_rels/presentation.xml.rels` — 슬라이드 목록을 다시 쓰느라 마지막에 미뤄 둔 세 부분입니다. 이름 순으로 씁니다.

왜 중요한가

덱에는 아무 문제가 없었습니다. **그리고 그것을 확인할 방법도 없었습니다.**

- 지난주에 받아 둔 사본과 견주는 사람은 **다섯 번에 한 번쯤 거짓으로 "바뀌었다"** 를 봅니다.
- 내용 해시로 같은 내보내기를 합치려는 어떤 장치도 합치지 못합니다.
- 그리고 검사가 이 경로를 매번 불안정하다고 표시하면, 사람은 검사를 무시하게 됩니다.

```
고치기 전 20회 → 파일 4가지
고친 뒤   20회 → 파일 1가지
```

시험은 여덟 번 내보내 전부 같은 파일인지 봅니다. 세 부분의 순서는 여섯 가지라 한 번 반복으로는 우연히 맞을 수 있기 때문입니다. 고치기 전 코드에 대고 세 번 돌려 세 번 다 실패하는 것을 확인했습니다.

3.10.173 v0.218.0에서 완료 — 씨앗에 끝나는 일이 없어 기간 선택이 아무것도 가르지 못했습니다

업무 인사이트를 걸으며 화면이 제공하는 네 기간을 모두 재 봤습니다.

```
 4주: 업무 2171 · 중복 총 5692 · 협업 328
12주: 업무 2171 · 중복 총 5692 · 협업 328
26주: 업무 2171 · 중복 총 5692 · 협업 328
52주: 업무 2171 · 중복 총 5692 · 협업 328
```

완전히 같습니다. 달라지는 것은 응답 시간뿐이었습니다.

씨앗에는 끝나는 일이 없었습니다

```
업무 2,171건 · 마지막 보고가 4주 전보다 오래된 것 0건
보고 주차 수: 52주 1,963 · 51주 165 · 1주 43
```

모든 (사람, 슬롯) 쌍이 52주 내내 행을 만들었습니다. 시작하는 일은 있었지만(1주짜리 43건) **끝나는 일이 없었습니다.** 그러면 '최근 N주' 라는 창은 언제나 전부를 담습니다.

기간을 통째로 무시하는 회귀가 생겨도 화면은 똑같이 답합니다. 그리고 이 제품에는 기간 버튼이 있는 화면이 여럿입니다.

일은 끝납니다

열한 쌍에 하나가 18~43주차 어딘가에서 보고를 멈추게 했습니다. 끝난 일이 다음 주 보고서에 없는 것이 정상이고, **그 부재가 있어야 '최근 N주' 에 뜻이 생깁니다.**

처음에는 18~37주로 두었는데, 그러면 끝난 업무가 전부 15주 넘게 전이라 4주와 12주가 여전히 같았습니다. 폭을 넓혔습니다.

```
 4주: 업무 1977 · 중복 총 4690 · 협업 325 · 반복 총 1613
12주: 업무 2007 · 중복 총 4866 · 협업 327 · 반복 총  861
26주: 업무 2108 · 중복 총 5486 · 협업 328 · 반복 총  405
52주: 업무 2171 · 중복 총 5865 · 협업 328 · 반복 총  339
```

다시 무너지지 않게

씨앗의 자기 점검에 차원을 더했습니다 — 4주 넘게 보고가 끊긴 업무가 50건 미만이거나, 12주 넘게 없는 업무가 20건 미만이면 **씨앗이 스스로 실패합니다.** 지금 여유는 194건과 164건입니다.

걸어 본 결과: 제품은 정상

`보고 분석` 은 원래부터 기간마다 달랐습니다 — 문서 1,216 → 3,648 → 7,904 → 15,808 이고 상위 용어도 바뀝니다. 인사이트도 같은 씨앗을 주자 곧바로 갈렸습니다. **고칠 것은 제품이 아니라 씨앗이었습니다.**

3.10.174 v0.219.0에서 완료 — 가장 좁은 두 창까지 갈리게 했습니다

v0.218.0 이 씨앗에 끝나는 일을 넣어 기간 선택에 뜻을 주었습니다. 그 뒤 **경영 요약**을 같은 방법으로 재 보니 창을 정확히 좁히고 있었습니다 — 고려 1977 / 1977 / 2108 / 2171 로 인사이트와 같은 수입니다.

그런데 **4주와 8주가 여전히 같았습니다.**

마무리 시점을 18~43주차에 두었기 때문입니다. 그러면 끝난 업무가 최소 8주 전이라, 4주 창과 8주 창이 똑같이 걸러 냅니다. 화면은 **4·8·12·26·52주** 를 내주는데 그 중 둘이 구분되지 않았습니다.

18~47주차로 넓혔습니다. 이제 최근 4~7주 사이에 끝난 업무도 있습니다.

```
업무 인사이트   4주 1977 · 8주 2002 · 12주 2027 · 26주 2114 · 52주 2171
경영 요약       4주 1977 · 8주 2002 ·            26주 2114 · 52주 2171
```

다섯 창이 모두 다릅니다. 자기 점검 여유는 그대로입니다.

한 축을 열고 나면 그 축의 **가장 좁은 눈금까지** 열려 있는지 봐야 합니다. 절반만 열린 축은 열린 것처럼 보이면서 여전히 아무것도 가르지 못합니다.

3.10.175 v0.220.0에서 완료 — 한 주 빠진 것은 경고하고 일곱 달 사라진 것은 말하지 않았습니다

v0.218.0 이 씨앗에 **끝나는 일**을 넣었습니다. 그 새 상태를 화면에서 걸었더니 인수인계가 이렇게 답했습니다.

```
채용 통합   마지막 2026-08-24 · 32주째 진척이 없습니다…
계약 통합   마지막 2026-08-24 · 보고가 1주 누락됐습니다…
평가 통합   마지막 2026-01-26 · (없음)          ← 일곱 달 전
감사 통합   마지막 2026-08-24 · 완료로 보고된 업무입니다…
```

한 주 빠진 것은 말하고, 일곱 달 사라진 것은 아무 말도 하지 않습니다. 진척 49% 에서 멈춘 채로.

모든 잣대가 그 업무의 안쪽만 봤습니다

필드	끊긴 업무	이번 주 업무
`stalled`	**false**	true
`atRisk`	**false**	true
`silentWeeks`	**0**	0

`silentWeeks` 는 정의가 옳습니다 — **첫 보고와 마지막 보고 사이**의 빈 주입니다. 그냥 멈춘 업무는 자기 구간 안에 빈 곳이 없으니 0 이 맞습니다. `stalled` 도 `ageWeeks` 도 마찬가지로 구간 안에서 계산됩니다.

마지막 보고 이후 흐른 주를 세는 곳이 아무 데도 없었습니다. 그래서 버려진 업무가 살아 있는 업무보다 건강해 보였습니다.

지금과 견주는 잣대를 하나 넣었습니다

`weeksSince(lastWeek, currentWeek)` 를 더하고, 경고 사다리의 **맨 위**에 놓았습니다. 넉 달째 아무도 언급하지 않은 업무라면 그것이 먼저 할 말입니다. 넉 주를 문턱으로 삼은 것은 그보다 짧은 공백은 휴가나 조용한 2주일 수 있기 때문입니다.

```
평가 통합  마지막 2026-01-26 · 30주째 이 업무를 언급한 보고가 없습니다.
                              마지막 기록은 2026-01-26 주차이며, 그 뒤의 상황은 아무 데도 없습니다.
```

완료로 보고된 업무는 버려진 것이 아니라 **끝난 것**이므로 그대로 완료 안내를 받습니다. 나머지 경고도 그대로입니다.

인수인계는 *"이 사람이 아직 지고 있는 것이 무엇인가"* 를 묻는 화면입니다. 넘겨받는 사람이 **1월 이후 아무도 손대지 않았다는 사실**을 모른 채 그 업무를 안고 가는 것이 이 화면이 막아야 할 바로 그 일입니다.

3.10.176 v0.221.0에서 완료 — 업무 추적도 버려진 일을 "진행" 이라고 불렀습니다

v0.220.0 이 인수인계에 *"몇 주째 아무도 언급하지 않았다"* 를 넣었습니다. 같은 렌즈를 나머지 화면에 대봤습니다.

업무 추적이 같은 자리에서 무너졌습니다.

```
개발  캐시 점검  54% (+34)  24주  2025-09-01 ~  24회  진행
```

24회 보고하고 **28주째 아무 말이 없는** 업무가 상태 **진행** 입니다. 이 화면의 머리글은 이렇게 약속합니다.

> 얼마나 오래 진행됐고, **몇 주 보고가 빠졌고**, 언제부터 진척이 멈췄는지 확인합니다.

빠진 주는 셉니다 — 다만 **구간 안의** 빠진 주만입니다.

잣대를 업무 자체에 붙였습니다

v0.220.0 은 인수인계 안에서만 지금과 견줬습니다. 그 계산을 `workItemView.staleWeeks` 로 올려 **모든 화면이 같은 수를 봅니다.** 인수인계는 그 필드를 쓰고, 업무 추적은 넉 주 이상이면 상태 칸에 그대로 답니다.

```
전:  캐시 점검 … 진행
후:  캐시 점검 … 28주째 보고 없음
```

완료로 보고된 업무에는 붙이지 않습니다 — 끝난 일이 다음 주에 없는 것은 정상입니다.

경영 요약은 손대지 않았습니다

버려진 업무를 경영 요약에도 밀어 넣을 수 있었지만 하지 않았습니다. 이 제품에는 업무를 닫는 상태가 없어서 **사라짐이 곧 끝남**입니다 — 49%에서 사라진 업무는 버려진 것일 수도, 끝났는데 100으로 적지 않은 것일 수도 있습니다.

인수인계와 업무 추적은 *"이 사람이 무엇을 지고 있는가"* 를 묻기 때문에 **모른다는 사실 자체가 답**입니다. 경영 요약은 *"경영진이 무엇을 결정해야 하는가"* 를 묻고, 거기에 모호한 188건을 밀어 넣는 것은 v0.17 이 정한 규칙 — *이미 한 일을 하라고 말하는 것이 목록 전체를 무시하게 만드는 방법* — 을 어기는 일입니다.

3.10.177 v0.222.0에서 완료 — 마감일이 하나도 없어 전망 여섯 갈래가 전부 어두웠습니다

```
업무 2,171건 · 마감일 있는 것 0건
```

씨앗은 `work_items` 를 직접 만들면서 `due_date` 를 한 번도 채우지 않았습니다. 그래서 어느 배포에서도

- `dueOutlook` 은 언제나 **NONE**
- 마감 초과·위태·닿음 배지가 한 번도 그려지지 않고
- 경영 요약의 마감 관련 가중치 **셋**이 한 번도 더해지지 않으며
- `duedate.go` 의 전망 계산 전체가 단위 시험 말고는 실행되지 않습니다

갈래는 날짜가 아니라 속도가 정합니다

날짜를 `id % 11` 로 흩뿌렸더니 **셋**만 나왔습니다. 진척을 보고 놓았더니 **넷**이었습니다.

속도를 실측하고서야 이유를 알았습니다 — 이 씨앗은 **전체 평균 0.5~0.8%/주, 최근 8%/주** 입니다. 남은 주 `W` 에 대해 낮은 추정은 `진척 + 0.8W`, 높은 추정은 `진척 + 8W` 이므로:

갈래	조건
닿음	낮은 추정도 100 — 진척 60이면 **W ≥ 50**
갈림	높은 추정만 100 — **W 5~44**
위태	둘 다 못 닿음 — W 가 작을 때

그에 맞춰 놓으니 여섯이 다 나옵니다.

```
+385일 · u106 → ON_TRACK 2 · AT_RISK 1
 -35일 · u2   → FINISHED 1 · OVERDUE 1 · ON_TRACK 1 · AT_RISK 2
 +42일 · u6   → UNKNOWN 1 · ON_TRACK 3 · AT_RISK 1
```

목록이 400건에서 잘려 안 보였습니다

관리자 화면으로 세었을 때는 넷만 보였습니다. 팀 목록이 급한 순서로 400건까지만 싣기 때문입니다 — **가장 안 급한 갈래인 `ON_TRACK` 과 `FINISHED` 가 언제나 잘려 나간 쪽에 있습니다.** 소유자의 화면에서 봐야 보입니다. 도구가 볼 수 없으면 없는 것과 같다는 이 주기의 교훈이 그대로 반복됐습니다.

다시 무너지지 않게

자기 점검에 차원을 더했습니다 — 마감일이 다섯 자리 미만에만 있으면 **씨앗이 스스로 실패합니다.** 지금은 여섯 자리, 마감일 있는 업무 1,268건입니다.

3.10.178 v0.223.0에서 완료 — 업무를 합치면 결정이 사라지고 자기 자신에게 막혔습니다

씨앗에 병합된 업무가 **0건**이었습니다. `merged_into_id` 를 채우는 것은 사람의 행동이라 씨앗이 만들 수 없고, 그래서 병합 뒤의 상태는 어느 배포에도 존재한 적이 없습니다. 배포에서 직접 합쳐 봤습니다.

옮겨진 것은 주간 본문뿐이었습니다

```
병합 전  9번 결정 1 · 8번 결정 0 · 9↔8 링크 1
병합 후  9번 결정 1 · 8번 결정 0 · 9번 보고항목 0
```

보고 항목 52건은 8번으로 갔고, **결정은 9번에 남았습니다.** 9번은 보고 항목이 없어 어느 목록에도 나오지 않습니다. 그 결정은 이제 닿을 수 없습니다 — 8번의 결정 목록은 빈 배열입니다.

그리고 8번은 자기 자신에게 막혔습니다

`9 → 8` 이던 의존 관계가 그대로 남았습니다. 9가 8이 된 뒤에도 8의 화면은 이렇게 말합니다.

```
blockers: [{ workItemId 9 · title 교육 개선 · progress 0 · lastWeek "" · ready false }]
```

진척 0, 마지막 주차 없음 — 이력이 8번으로 옮겨 갔기 때문입니다. 회의 자리에서는 *"감사 개선은 교육 개선을 기다리고 있습니다"* 가 계속 뜨는데, 교육 개선은 **어느 목록에도 없어서 열 수도, 끝낼 수도 없습니다.**

가리키던 것을 모두 따라가게 했습니다

업무를 가리키는 것은 넷입니다 — 보고 항목, 결정, 이슈 결과, 그리고 선행 관계의 양쪽 끝. 옮기던 것은 하나였습니다.

두 끝을 잇던 관계는 삭제합니다. 병합은 *"이 둘은 하나의 일이다"* 라는 선언이고, 그 선언 뒤에 남은 자기 의존은 `work_item_links_not_self` 가 애초에 금지하는 모양입니다. 나머지는 옮기되 이미 있는 관계와 겹치면 버립니다.

```
병합 후  9번 결정 0 · 8번 결정 1 · 자기 링크 0 · 9번 가리키는 링크 0
        8번 화면: blockers 0 · blocking 0
```

자리표시자를 재사용하게 되어, 이 저장소의 규칙대로 PostgreSQL 에 물어 `{bigint, bigint}` 임을 확인하고 이유와 함께 허용 목록에 적었습니다.

3.10.179 v0.224.0에서 완료 — 뒤집힌 결정이 하나도 없었습니다

```
결정 241건 · 대체된 것 0건 · 상태 DONE, OPEN
```

씨앗은 결정을 만들면서 `supersedes_id` 를 한 번도 채우지 않았고 `SUPERSEDED` 상태도 만들지 않았습니다. 그래서 어느 배포에서도

- 대체된 결정이 후속 목록에서 **빠지는 규칙**
- 화면의 `이전 결정 #N 대체` 표시
- 인수인계가 **열린 것만 세는** 자리

셋 다 켜지지 않았습니다.

먼저 손으로 걸었습니다

```
다른 업무의 결정을 대체 시도 → 대체할 결정은 같은 업무의 결정이어야 합니다
같은 업무의 결정을 대체     → 1 처음 결정 SUPERSEDED · 2 바꾼 결정 OPEN 대체 1
후속 목록                    → 1건 '바꾼 결정'  (옛 것은 빠짐)
업무 화면                    → 둘 다, 각자의 상태와 함께
인수인계                     → openDecisions 1
```

전부 설계대로입니다. 교차 업무 대체를 막는 것도, 뒤집힌 기록을 지우지 않고 상태만 바꾸는 것도 — 주석이 적어 둔 이유 그대로입니다. *"열린 채 두면 목록에 영영 남고, 지우면 뒤집혔다는 사실을 잃습니다."*

그래서 씨앗이 그 상태를 만들게 했습니다

네 업무에 하나꼴로 앞선 결정을 뒤집습니다. 결정은 뒤집히는 것이고, 그때 옛 기록이 남아야 왜 바뀌었는지 읽을 수 있습니다.

```
결정 301 · DONE, OPEN, SUPERSEDED · 대체 관계 60
u6 인수인계 → openDecisions 1
  결산 구축 방향 재결정  OPEN        대체 5
  결산 구축 방향 결정    SUPERSEDED  대체 -
```

자기 점검에 차원을 더했습니다 — 결정 상태가 세 가지 미만이면 **씨앗이 스스로 실패합니다.**

이번에도 고칠 것은 제품이 아니었습니다

v0.218.0(끝나는 일)·v0.222.0(마감일)에 이어 세 번째로, 어두운 자리를 켜고 보니 제품은 옳았습니다. **어둡다는 사실 자체가 결함입니다** — 옳은지 아무도 알 수 없었으니까요.

3.10.180 v0.225.0에서 완료 — 라우트에 묻던 질문을 표에도 묻습니다

씨를 뿌린 배포에서 표마다 행 수를 세어 **비어 있는 표**를 찾았습니다. 대부분은 이미 도구가 채웁니다 — 첨부는 `seed-captures.py`, Confluence 넷은 `fake-confluence.py`, 임베딩은 가짜 게이트웨이, Import 셋은 PPTX 왕복. 세션·로그인 시도처럼 **실행 중에만 생기는 것**은 비어 있는 게 맞습니다.

그런데 `report_status_history` 는 다른 이유로 비어 있었습니다.

여섯이 쓰고 아무도 읽지 않습니다

```
INSERT 하는 곳 : reports.go 네 곳, imports.go 두 곳
SELECT 하는 곳 : Go 없음 · 계약 없음 · 화면 없음
```

시험 셋만 그것이 **쓰였는지**를 확인합니다. 읽히는지는 아무도 확인하지 않습니다. 표에는 `(report_id, id)` 색인이 따로 있고, README 는 *"모든 상태 전이는 `report_status_history`와 감사 로그에 기록됩니다"* 라고 적습니다.

`TestEveryRouteIsReachable` 이 라우트에 대해 묻는 것과 **똑같은 모양**입니다 — 그 시험의 서두가 말하듯 *"완성되고 시험되고 문서화됐는데 구조적으로 닿을 수 없는"* 것. 다만 한 층 아래입니다.

그래서 표에도 묻습니다

제품이 쓰는 표는 읽기도 해야 하고, 아니라면 **이유와 함께 목록에 적혀야 합니다.**
`report_status_history` 는 운영자가 데이터베이스에서 직접 읽는 감사용 흔적이라고 적었습니다 — README 가 그렇게 말하고, 색인이 그 조회를 위해 있습니다.

만들면서 두 번 배웠습니다

첫 판은 주석의 영어 산문을 SQL 로 읽어 `skip` 과 `that` 이라는 표를 찾아냈습니다. **마이그레이션이 실제로 만든 이름**으로만 보게 했습니다.

두 번째 판은 `oidc_login_states` 를 읽히지 않는다고 신고했습니다. 로그인 상태 검증이라 심각해 보였는데, 실제로는 이렇게 씁니다.

```
DELETE FROM oidc_login_states WHERE state_hash=$1 AND expires_at>now() RETURNING nonce,pkce_veri
```

한 번 쓰고 버리는 기록을 읽으면서 동시에 무효화하는 올바른 방식입니다. 검사가 `DELETE … RETURNING` 을 읽기로도 세게 고쳤습니다. 도구가 제품을 의심할 때는 도구가 틀렸을 수도 있습니다.

3.10.181 v0.226.0에서 완료 — 개인 기간 보고의 "이슈 해소"가 언제나 거짓말이었습니다

씨를 뿌린 배포에 v0.225.0 을 올리고 **열네 화면을 브라우저로 한 바퀴 돌며** 콘솔 오류와 실패한 요청을 모았습니다. 화면은 조용했는데 **서버 로그가 조용하지 않았습니다.**

```
WARN issue clearance error="ERROR: missing FROM-clause entry for table \"wi\" (SQLSTATE 42P01)"
```

별칭 하나

기간 보고는 "누구의 일인가"를 좁히는 조건을 **한 번만 쓰고 두 질의에 넘깁니다.** 그렇게 한 이유가 코드에 적혀 있습니다 — 둘로 쓰면 언젠가 서로 달라지고, *"한쪽 사람들을 집계하면서 다른 쪽의 결정을 나열하는 기간 보고는 결정이 아예 없는 것보다 나쁘다."*

그 조건은 `decisionsInPeriod` 의 별칭으로 쓰여 있습니다.

```
개인 범위 : AND wi.user_id=$3
결정 질의 : JOIN work_items wi ON wi.id=d.work_item_id   ← 있습니다
해소 질의 : JOIN work_items w  ON w.id=o.work_item_id    ← 없습니다
```

조건을 하나로 합치면서 별칭이 갈라졌습니다. 조직 범위 조건은 `u.organization_id` 라 양쪽에 다 있고, 그래서 **팀 화면만 멀쩡히 동작했습니다.**

실패가 답처럼 보였습니다

실패는 보고서 전체를 잃지 않도록 잡혀서 경고로만 남습니다. 판단 자체는 옳습니다 — 숫자 하나 때문에 기간 보고를 통째로 잃을 이유는 없습니다. 문제는 **그다음**입니다.

```
resolved = 0  →  화면: "해소로 기록된 이슈가 아직 없습니다"
```

`issueClearance` 의 주석은 *"0은 아직 아무도 답하지 않았다는 뜻이며, 화면은 그것을 측정값처럼 보여 주지 않는다"* 고 적어 두었습니다. 그런데 **질의가 죽어서 생긴 0** 과 **표가 비어서 생긴 0** 이 똑같이 생겼습니다. 화면은 **묻지도 못한 질문에 대해 답이 없다고 단언하고** 있었습니다.

실제 배포에서 확인한 모습입니다.

```
고침 전  u91 SELF : resolved 0        ← "아직 없습니다"
고침 후  u91 SELF : 7주, "대시보드 자동화"
데이터베이스      : 3분기 RESOLVED 1건, 7주
```

시험이 딱 하나 안 쓰이는 모양만 봤습니다

`issueClearanceFor` 를 부르는 시험이 셋 있고, **셋 다 조건을 빈 문자열로 넘깁니다.** 제품이 절대 넘기지 않는 그 한 가지 모양입니다. 그래서 새 시험은 **라우트를 통해** SELF 와 TEAM 을 둘 다 읽습니다. 그리고 표 이름을 바꿔 질의를 실제로 죽여 놓고, 보고서가 살아남으면서도 **읽지 못했다고 말하는지** 봅니다. 둘 다 고치기 전 코드에서 붉게 실패하는 것을 확인했습니다.

응답에 `unread` 가 생겼고, 화면은 그럴 때 "이 기간의 해소 기록을 읽지 못했습니다" 라고 적습니다.

3.10.182 v0.227.0에서 완료 — 말없이 줄어든 답 둘

지난 회차의 결함은 **오류를 잡아 경고로 남기고 화면엔 그럴듯한 답을 보여 주는** 모양이었습니다. 같은 모양이 더 있는지 세었습니다.

```
오류를 잡고 아무것도 돌려주지 않는 자리 : 21곳
```

대부분은 옳습니다 — 감사 로그 쓰기, 오래된 행 정리, 파일 삭제, 메일 워커의 장부. **끝난 일을 장부 실패 때문에 되돌리는 것이 더 나쁩니다.** 코드에 그 이유가 이미 적혀 있습니다. 그중 둘만 **사용자에게 답처럼 보였습니다.**

하나 — 짧은 검색 결과가 "그런 보고는 없다"로 읽힙니다

보고서 검색은 정확 일치가 5건 미만이면 두 단계를 더 밟습니다. 유사(pg_trgm), 의미(pgvector+임베딩). 두 단계 모두 **못 돌아도 아무 말도 하지 않았습니다.**

```
확장이 없음       → 조용
임베딩 미설정     → 조용 (searchSemantic 이 nil, nil 을 돌려줍니다)
그 단계가 실패    → 경고 로그로만
```

주간보고 도구에서 "누가 그 일을 보고한 적 있나"는 **핵심 질문**입니다. 짧은 목록은 읽는 사람에게 답으로 읽히는데, 넓히는 단계가 실제로 돌았을 때만 답입니다.

고칠 답은 바로 옆 화면에 이미 쓰여 있었습니다. 업무 검색(`worksearch.go`)은 같은 상황에서 세 문장을 구분해 화면에 적습니다 — 확장이 없다, 설정이 꺼져 있다, 실패했다. 설정 계층(`embeddingConfig`)은 v0.210에서 그 문장들을 **사람이 읽으라고** 한국어로 써 두었는데, 보고서 검색은 그것을 버리고 있었습니다. 같은 문장을 그대로 씁니다.

결과가 넉넉하면 `reason` 은 비웁니다. 넓힐 필요가 없던 검색에 어떤 확장이 없는지 알리는 것은 **묻지 않은 질문**입니다.

둘 — 캡처 다섯 장을 넣고 세 장이 나옵니다

파일이 사라진 첨부는 슬라이드에서 **그냥 빠졌습니다**. 배포는 그것을 기동할 때 한 번 경고합니다 — *"attachment files are missing; captures will not display or export"* — 그런데 그 글은 **운영자**에게 하는 말입니다. 파일을 내려받는 **작성자**는 다른 사람이고 다른 질문을 합니다.

원래 규칙은 옳았습니다. 깨진 그림 참조를 패키지에 넣으면 안 됩니다. 다만 그 규칙을 지키는 방법으로 **페이지를 지운 것**이 틀렸습니다. 이제 페이지는 제목과 순서를 그대로 지킨 채, 그림 자리에 무슨 일이 있었는지 적

습니다.

실제 배포에서 확인했습니다.

```
캡처 3장 · 파일 없음
고침 전 : 슬라이드 5장 (아무 표시 없음)
고침 후 : 슬라이드 8장 · "u1-캡처1.png · 이미지 파일을 찾을 수 없어 담지 못했습니다"
```

3.10.183 v0.228.0에서 완료 — 스크립트에게 "로그인이 필요합니다"라고 답하고 있었습니다

씨를 뿌린 배포의 표를 세었을 때 `personal_api_keys` 가 비어 있었습니다. 화면이 아니라 **스크립트와 MCP 클라이언트가 쓰는 표면**이고, 실제 배포에서 한 번도 걸어 본 적이 없었습니다. 걸어 봤습니다.

범위는 정확했습니다 — 읽기 셋은 200, 쓰기와 키 관리와 관리자 화면은 403, MCP 도구 셋도 정상. `last_used_at` 도 갱신됩니다. 그러다 키가 **거절되는** 쪽을 봤습니다.

```
글자가 틀린 키   → UNAUTHORIZED · 로그인이 필요합니다.
형식이 아닌 토큰 → UNAUTHORIZED · 로그인이 필요합니다.
만료된 키        → UNAUTHORIZED · 로그인이 필요합니다.
회수된 키        → UNAUTHORIZED · 로그인이 필요합니다.
회전에 걸린 키   → UNAUTHORIZED · 로그인이 필요합니다.
```

스크립트는 로그인을 할 수 없습니다. 그리고 이 다섯은 사람이 해야 할 일이 서로 다릅니다.

더 나쁜 건 **만료**입니다. 이 제품이 `expiresInDays` 와 관리자 상한을 제공하고, 스스로 그 상태를 만들어 두고, 정작 그 이름을 말하지 않습니다. 사내 스크립트를 이 API에 연결해 둔 사람은 어젯밤 키가 만료됐다는 것을 알 방법이 없었습니다.

인증 질의 하나가 다섯 조건을 묶고 있었습니다

```
WHERE k.token_hash=$1 AND k.revoked_at IS NULL
  AND (k.expires_at IS NULL OR k.expires_at>now())
  AND u.active=true AND k.key_version=u.key_version
```

`ErrNoRows` 하나에 다섯 사연이 들어갑니다. 그 질의가 **이미 빗나간 뒤에만** 해시로 한 번 더 찾아 무슨 일이 있었는지 봅니다 — 평상시 경로에는 비용이 없습니다.

키를 갖지 않은 사람에게는 아무것도 알려 주지 않습니다. 이 답은 키를 실제로 제시해야 보이고, 키는 무작위 32바이트입니다.

시험이 제 예상을 고쳤습니다

회전(rotate)에 걸린 키가 "일괄 회수 뒤 발급된 키가 아닙니다" 라고 답할 것으로 보고 시험을 썼는데, 실제로는 **"회수되었습니다"** 였습니다. `rotateKeys` 는 버전을 올리는 동시에 살아 있는 키 전부에 `revoked_at` 을 찍습니다 — 회전은 **정말로 일괄 회수**이고, 답이 정확했습니다. 시험을 사실에 맞췄습니다.

버전 대조는 그 아래 두 번째 층으로 남습니다. 도장이 못 잡는 키가 하나 있습니다 — **회전 트랜잭션과 겹친 발급 요청**은 옛 버전을 읽고, 쓸어 담기가 지나간 뒤에 삽입됩니다. 그 키의 주인은 잘못한 것이 없으므로 "존재하지 않는 키" 말고 다른 말을 들어야 합니다.

3.10.184 v0.229.0에서 완료 — 받은 파일 그대로는 설치가 시작되지 않았습니다

고객이 이 제품과 처음 만나는 순간을 그대로 해 봤습니다. tarball 을 적재하고, `.env.example` 을 복사하고, README 가 채우라는 값을 채우고, `docker compose up`.

```
error while interpolating services.weekly.environment.WEEKLY_ENCRYPTION_KEY:
required variable WEEKLY_ENCRYPTION_KEY is missing a value: required
```

첫 명령이 렌더조차 되지 않습니다. 컨테이너도, 로그도, 힌트도 없습니다.

같은 값을 두고 세 파일이 서로 다른 말을 했습니다

```
deploy/compose.yaml   : ${WEEKLY_ENCRYPTION_KEY:?required}   ← 없으면 거부
deploy/.env.example   : WEEKLY_ENCRYPTION_KEY=               ← 비워서 배포
README.md             : "필수 권장" · "환경 키를 설정하지 않은 하위 호환 모드"
바이너리              : 비어 있으면 기동하고 경고를 남깁니다
```

넷 중 셋이 "없어도 된다"고 말하고, 실제로 배포되는 파일 하나만 아니라고 말합니다. **그리고 이 셋을 함께 보는 것이 아무것도 없었습니다.**

`:?required` 를 뗐습니다. 하드 요구는 문서화된 하위 호환 모드를 막고, 무엇보다 **키가 볼륨에 있는 기존 배포가 배포된 compose 로 업그레이드할 수 없게** 만듭니다. 권고는 설명할 수 있는 자리에 이미 있습니다 — 기동 로그가 *"키가 상태 볼륨에만 있으니 볼륨을 잃어도 업그레이드가 살아남도록 이 변수를 설정하라"* 고 적습니다.

최초 설치에서 만날 수 있는 첫 문장이 영어였습니다

README 가 시키는 `openssl rand -base64 32` 를 잘못 붙여 넣으면 기동이 멈춥니다.

```
"error":"WEEKLY_ENCRYPTION_KEY must be a base64-encoded 32-byte key"
```

한국어 제품에서, 운영자가 **처음 만나는 문장**이, 값의 모양만 말하고 그 값을 만드는 명령은 말하지 않습니다. 바로 옆 두 기동 오류(app.go)는 이미 한국어로 정성껏 쓰여 있습니다. 이 하나만 남아 있었습니다.

`install-check.sh`

`scripts/` 의 다른 검사는 전부 **제품**을 잽니다. 이건 제품이 한 번도 돌기 전에 **고객이 복사하는 두 파일**을 잽니다.

- `.env.example` 을 문서대로만 채워 `docker compose config` 가 렌더되는가
- compose 가 가리키는 이미지가 `VERSION` 과 같은가
- 코드가 읽는 모든 `WEEKLY_*` 를 compose 와 kubernetes 가 전달하는가 (없이도 도는 값은 그렇다고 적혀 있어야 합니다)
- `.env.example` 이 그대로 써도 될 것처럼 보이는 비밀번호를 담고 있지 않은가

만들자마자 **자기 자신의 오탐**을 하나 잡았습니다 — 시험만 읽는 `WEEKLY_TEST_POSTGRES_DSN` 을 배포 파일이 전달하지 않는다고 신고했습니다. `_test.go` 를 빼게 했습니다.

3.10.185 v0.230.0에서 완료 — 사용자에게 읽어 준 추적 ID가 로그에 없었습니다

설치 다음에 오는 것을 봤습니다. 무언가 잘못됐을 때 운영자가 밟는 경로입니다. 사용자가 오류 화면의 번호를 불러 주면 운영자가 로그에서 찾습니다.

```
화면 : 업무를 조회할 수 없습니다. (추적 ID 69df5a452c4e4a2b)
로그 : (없음)
```

이건 우연한 누락이 아니라 **제품이 한 약속**입니다. `api.ts` 가 5xx 응답의 메시지 뒤에 `(추적 ID …)` 를 붙입니다. 불러 주라고 보여 주는 것입니다.

227곳 중 60곳

```
5xx 를 돌려주는 자리        : 227
  그중 로그가 있는 것        :  77
  그중 추적 ID 를 적는 것    :  60   ← 26%
  보여 준 ID 로 찾을 수 없음 : 167
```

150곳은 로그 자체가 없고, 17곳은 로그는 있는데 ID가 없습니다. 요청별 접근 로그도 없습니다 — 시간당 method·route·status 로 뭉친 지표 표만 있어서 "무언가 실패했다"는 알아도 **"이 요청"** 은 못 찾습니다.

167곳을 고치는 대신 **모든 응답이 지나가는 한 자리**에서 적습니다. `requestMetrics` 는 이미 route 와 status 를 알고 있고, 내년에 추가될 핸들러가 잊을 수 없습니다. 개별 사이트가 적던 *원인* 60줄은 그대로 둡니다 — 새 줄은 **어느 요청인지**를 말하고, 그 줄들은 **왜 실패했는지**를 말합니다.

4xx 는 남기지 않습니다. 화면이 추적 ID 를 보여 주지 않고, 요청마다 한 줄이면 정작 물어 온 요청이 묻힙니다.

왜 아무도 못 봤는지가 더 중요했습니다

```
s.app.mux.ServeHTTP(w, r)   // 시험 하니스
```

시험 전체가 미들웨어 체인을 건너뛰고 `mux` 를 직접 불렀습니다. 추적 ID를 심는 곳, panic 을 잡는 곳, 보안 헤더를 붙이는 곳 — 고객의 요청이 핸들러에 닿기까지 지나는 모든 것이 시험된 적이 없습니다. 추적 ID 가 어느 로그에도 없다는 사실이 보이지 않은 이유가 이것입니다.

`Handler()` 로 바꿨습니다. 시험 둘이 붉어졌고, 둘 다 **응답 본문을 통째로 비교**하고 있었습니다 — "남의 보고서 거절과 없는 보고서 거절이 똑같이 읽혀야 한다". 이제 추적 ID 가 붙으니 바이트로는 다릅니다. 그런데 **달라야 맞습니다.** 요청을 가리키는 값이니까요. 코드와 문장만 비교하도록 고쳤고, 시험의 뜻은 그대로면서 이제 진짜 체인을 지납니다.

3.10.186 v0.231.0에서 완료 — 행을 세어 크기를 막으려 했습니다

지난 회차에 만든 `request failed` 로그를 **도구로** 썼습니다. 씨를 뿌린 배포에 모든 읽기 경로를 두드리고, 5xx 가 스스로 이름을 대게 두었습니다. **하나도 없었습니다.** 대신 다른 숫자가 눈에 띄었습니다.

```
period_report_rollup  417,775B  ← 모델이 읽는 한 번의 도구 호출
```

근거는 크기인데 통제하는 것은 행 수였습니다

이 자리의 주석은 이미 옳게 적혀 있습니다.

> *"화면은 사람이 표를 스크롤하며 중요한 셋을 읽으니 전부 보낸다. 모델에게는 표가 없다 — **받은 것이 곧 전부**이고, 기간에 대한 질문에 답하려고 천 행을 사는 것은 나쁜 거래다."*

그리고 그 주석은 이 door 가 한때 **2,563,406바이트**를 돌려주었다고 기록합니다. 상한을 100행으로 걸어 그것을 잡았습니다. 그런데 **행은 고정 무게가 아닙니다.**

```
같은 100행 : MONTH 108,414B · QUARTER 119,820B · HALF 122,192B · YEAR 169,341B
```

행 하나가 자기가 등장한 **모든 주차의 문장**을 싣고, 주차 수는 기간마다 다릅니다. 개수는 표를 잡고, **문맥 창은 잡지 못합니다.** 그리고 이 호출자에게는 문맥 창밖에 없습니다.

무거운 것부터 덜어냅니다

64 KiB 상한을 함께 겁니다 — 같은 표면의 다른 도구(보고서 검색)가 자기 100행 상한으로 55,931B 였으므로, **이미 그 안에 있는 도구는 건드리지 않고 밖에 있는 것만 골어오는** 값입니다.

처음엔 뒤에서부터 행을 덜어냈습니다. YEAR 가 256건 중 **13건**이 되었습니다. 정직하지만 나쁜 거래였습니다 — 가장 무거운 것은 주차 이력이고, 그것은 이미 *"상위 N건에만 있다"* 고 설명된 부분입니다. 이력을 버리면 **없을 수도 있다고 이미 들은 것**을 잃고, 행을 버리면 **있었다는 사실조차 알 수 없는 것**을 잃습니다.

바닥은 5건입니다. 0으로 만들면 그 설명 문장이 아무것도 가리키지 않게 되고, 기간의 모양이 통째로 사라집니다.

```
        고침 전                       고침 후
MONTH   108,414B  100건  이력 20 → 51,663B  45건  이력 5
YEAR    169,341B  100건  이력 20 → 53,207B  24건  이력 5
```

한 호출이 **54–85k 토큰에서 25k 토큰**이 되었습니다.

덤으로 하나

`trimTimelineSeries` 는 행이 몇 개든 `TimelineItems` 를 20으로 적습니다. 업무가 셋뿐인 기간 보고도 *"상위 20건에 이력이 있다"* 고 말하고 있었습니다. 새 반복문이 이 숫자로 배열을 색인하므로 붙잡아야 했고, 붙잡으면서 그 문장도 사실이 되었습니다.

3.10.187 v0.232.0에서 완료 — 저장 실패가 "새로고침하세요"라고 답했습니다

쓰기 경로를 눌러 봤습니다. 먼저 **상한**입니다.

```
항목 100개 · 제목 240자 · 구분 80자 · 실적/계획/이슈 각 20000자 · 요청 5000자 · 몸통 2 MiB
```

촘촘하고, 넘기면 한국어로 정확히 거절합니다. 상한을 **다 채운** 보고서(797,772B)를 실제로 만들어 downstream 을 재 봤습니다 — 저장 90ms, 조회 8ms, PPTX 22ms, 기간 보고 18ms. 덱은 작성자가 쓴 글자의 **95%** 와 업무 100건 전부를 담았습니다. **여기엔 결함이 없습니다.**

다음으로 **동시 편집**입니다. 낙관적 잠금은 진짜 동시 요청에서도 정확히 한 쪽만 통과시킵니다. 문제는 진 쪽이 듣는 말이었습니다.

```
VERSION_CONFLICT · 다른 변경사항이 먼저 저장되었습니다. 새로고침 후 다시 시도하세요.
```

제품의 두 쪽이 서로 반대로 말하고 있었습니다

편집 화면은 저장이 실패해도 **쓴 것을 일부러 지킵니다.** 코드에 이유가 적혀 있습니다.

> *"the version and status have to catch up, but the content on screen must not be replaced by the server's copy. Overwriting it here would **destroy exactly the work the failed save was trying to keep**."*

그래서 실패하면 `refreshMeta()` 로 **버전만** 다시 읽습니다. 즉 이 시점에 사용자는 **저장을 한 번 더 누르면 되는** 상태입니다. 그런데 화면에 뜨는 문장은 서버 것이고, 그것이 **새로고침하라고** 말합니다 — 쓰던 글을 버리는 유일한 동작이고, 클라이언트가 이미 불필요하게 만들어 둔 동작입니다.

돌아가는 배포에서 확인했습니다.

```
낡은 버전으로 저장  → VERSION_CONFLICT
버전만 다시 읽고    → 같은 내용으로 다시 저장 → 성공 · v5
남은 내용           → "내가 지금 쓰고 있던 글"
```

문장을 사실로 바꿨습니다. 무엇이 일어났는지, 쓴 것은 그대로라는 것, 한 번 더 누르면 된다는 것, **그동안 저장된 내용을 덮어쓴다는 것**, 그리고 새로고침하면 사라진다는 것.

삭제는 반대입니다

삭제 충돌에서는 **다시 불러오라는 조언이 옳습니다** — 타이핑 중인 것이 없고, 지워질 것이 화면에 띄웠을 때의 그것이 아닐 수 있기 때문입니다. 그 자리는 조언을 유지하되 **왜 그런지**를 말하게 했습니다.

3.10.188 v0.233.0에서 완료 — 0바이트 파일에게 "크기 제한을 초과했을 수도"라고 답했습니다

이번 회차는 **업로드 표면**을 눌렀습니다. 사용자가 파일을 주는 두 문 — 과거 자료 PPTX Import 와 화면 캡처 첨부입니다. 결과의 대부분은 **음성**이었고, 그것 자체가 결과입니다.

적대적인 파일 넷을 던졌습니다

```
zero.pptx      0B  → 202 → 파일이 비어 있거나 관리자 크기 제한을 초과했습니다.
notzip.pptx  5KB  → 202 → 유효한 PPTX ZIP 구조가 아닙니다
empty.pptx   122B → 202 → PPTX 필수 프레젠테이션 또는 슬라이드가 없습니다
bomb.pptx    615KB → 202 → PPTX 슬라이드 XML이 너무 큽니다      (풀면 600MB)
```

압축 폭탄은 **막힙니다.** 파서가 항목별 압축 해제 크기를 합산하고(10MB/항목), XML 디코더에도 상한이 있으며, 1GB 메모리 컨테이너는 77.9MiB 에서 꿈쩍하지 않았습니다.

왕복도 됩니다

내보낸 덱을 다시 들여왔습니다.

```
원본 2026-08-24 주차  →  export.pptx 19,631B  →  import
감지: 2026-08-24 ~ 2026-08-30 · 확신도 0.98 · 근거 slide_text
```

두 문이 같은 데이터를 알아봅니다.

멈춘 작업도 결국 말합니다

일부러 500을 주는 게이트웨이를 향한 배포에서, 작업은 135초를 `PROCESSING` 으로 앉아 있다가 이렇게 끝났습니다.

> *"AI Gateway가 오류로 응답했습니다(HTTP 500). 게이트웨이 쪽 문제이므로 이 설정을 바꿔도 해결되지 않습니다."*

한 문장만 틀렸습니다

```
if readErr != nil || len(body) == 0 || len(body) > maximumFileBytes {
    ... "파일이 비어 있거나 관리자 크기 제한을 초과했습니다."
```

원인 셋, 문장 하나, 그중 둘만 이름이 있고 "또는"으로 묶여 있습니다. 0바이트 파일은 관리자 한도를 넘었을지도 모른다는 말을 들었습니다 — 정반대의 사실이고, 해야 할 일도 정반대입니다(파일을 다시 내보내기 / 관리자에게 한도를 올려 달라기).

바로 옆 첨부 업로드는 이미 제대로 합니다 — *"huge.png의 크기가 허용 한도(10MB)를 초과했습니다."* 같은 종류의 문장을 쓰게 했습니다. 크기를 넘은 파일은 상한이 몇 MB 인지와 **어느 설정을 올려야 하는지**까지

듣습니다.

첨부 쪽에도 같은 모양이 하나 있었습니다. `ParseMultipartForm` 실패가 "형식이 올바르지 않거나 허용 용량을 초과했습니다" 하나로 뭉쳐 있는데, Go 는 `*http.MaxBytesError` 로 **어느 쪽인지 말해 줍니다.** 요청 전체가 넘친 것과 몸통이 안 읽히는 것은 할 일이 다릅니다 — 나눠 올리기와 다시 시도하기.

3.10.189 v0.234.0에서 완료 — 누가 승인했는지를 아무도 볼 수 없었습니다

이 제품의 핵심 절차를 아직 끝까지 걸어 본 적이 없었습니다. 작성 → 제출 → 반려 → 수정 → 재제출 → 승인을 실제 계정 둘로 밟았습니다.

```
DRAFT v2 → SUBMITTED v3 → REVISION_REQUESTED v4 → (수정) v5 → SUBMITTED v6 → APPROVED v7
```

절차 자체는 **정확합니다.** 사유 없는 반려는 `COMMENT_REQUIRED` 로 막고, 지나온 순서는 `report_status_history` 에 사유와 함께 남으며, **팀장이 자기 보고서를 스스로 승인하려 하면 거절합니다.**

그런데 승인자의 이름이 어디에도 없습니다

승인된 보고서의 응답에는 `reviewedAt` 만 있습니다. 시각은 있고 **사람이 없습니다.** 데이터베이스에는 있습니다.

```
reviewed_by 를 쓰는 곳 : 다섯 (승인·반려에서 채우고, 재제출·Import 에서 지움)
reviewed_by 를 읽는 곳 : 시험 셋. 그것이 쓰였는지를 확인하는 시험들뿐
```

v0.225 의 `report_status_history` 와 **같은 모양**입니다. 다만 표가 아니라 **열**이라 그때 만든 도달성 시험이 못 잡습니다. `weekly_reports` 는 당연히 읽히니까요.

그리고 이번에는 화면이 그 사실을 소리 내어 말하고 있었습니다.

> *"사유가 남아 있지 않습니다. **검토자에게 직접 확인해 주세요.**"*

300명 배포에서 작성자에게 검토자를 찾아가라고 하면서, **그 사람이 누구인지는 알려 주지 않는** 열을 손에 쥐고 있었습니다. 승인은 그 주에 대해 누가 책임을 졌는지의 기록이고, 시각만으로는 답이 되지 않습니다.

이제 `reviewedBy` 가 응답에 오고, 반려 안내는 반려한 사람의 이름을 부르며, 승인된 보고서는 누가 언제 승인했는지를 적습니다. 재제출하면 검토 기록과 함께 지워집니다 — 아직 하지 않은 결정을 누구의 것으로 돌리지 않기 위해서입니다.

열 단위로 훑어 나온 나머지 둘은 그대로 둡니다

- `confluence_pages.body_hash` — 본문 해시를 저장하지만 읽지 않습니다. 목적이던 "바뀌지 않은 문서 건너뛰기"는 **Confluence 자신의** `page_version` 이 이미 하고 있고(`previousVersion != page.Version`), 그쪽이 더 정확합니다. 남은 잔재입니다.
- `user_sessions.last_seen_at` — 5분에 한 번으로 조절된 갱신이고 읽는 곳이 없습니다. 비용이 거의 없고, 세션이 마지막으로 살아 있던 시각은 운영자가 데이터베이스에서 직접 볼 값입니다.

둘 다 사용자에게 무언가를 약속하지 않습니다. `reviewed_by` 만 약속했고, 그것만 지키지 않고 있었습니다.

3.10.190 v0.235.0에서 완료 — 표에 묻던 질문을 열에도 묻습니다

지난 회차의 `reviewed_by` 는 v0.225 의 `report_status_history` 와 **같은 결함**이었습니다. 쓰는 곳은 여럿, 읽는 곳은 없음, 그리고 화면이 그 값을 전제로 말함. 그런데 v0.225 에서 만든 도달성 시험이 그것을 통과시켰습니다 — **표 단위**라서, `weekly_reports` 가 당연히 읽히기 때문입니다.

한 층 더 내려갑니다.

만들면서 세 번 틀렸습니다

첫 판은 `WHERE token_hash=$1` 을 쓰기로 셌습니다. `SET x=$1` 과 글자 모양이 같습니다. **SET 절만** 도려내 보게 했습니다.

두 번째 판은 통과했는데 — **잡아야 할 그것을 못 잡았습니다.** v0.234.0 이전 코드로 되돌려 보니 조용했습니다. 이유:

```
reviewed_by = CASE WHEN $4 THEN NULL ELSE reviewed_by END
```

오른쪽의 `reviewed_by` 를 읽기로 센 것입니다. **자기 자신의 대입 안에서만 등장하는 열은 읽히는 것이 아닙니다** — 값이 쓰기를 견뎌 낼 뿐, 아무도 소비하지 않습니다. 대입된 이름은 그 절 안에서 통째로 지우게 했습니다. `version = version + 1` 도 같은 이야기입니다.

세 번째 판이 `reviewed_by` 를 정확히 신고합니다. **도구는 자기가 만들어진 이유를 잡아야 도구입니다.**

그렇게 해서 나온 열 열여섯

전부 이유를 적었습니다. 몇 가지는 그 자체로 알아 둘 만합니다.

- `confluence_pages` 의 `creator_username` · `last_modifier_username` · `created_at_source` 는 **지역 사본**입니다. 수집은 API 응답을 그대로 들고 일하고(`page.CreatorUsername` 을 직접 읽습니다), 표는 무엇을 언제 봤는지의 기록입니다. 사본을 도로 읽는 경로가 없는 것이 설계의 결과입니다.

- `title_hash` 와 `body_hash` 는 둘 다 "바뀌었는지" 를 보려던 값인데, 그 판단을 **Confluence 자신의 `page_version`** 이 이미 더 정확하게 합니다.
- `last_error` 에 실제로 들어가는 값은 `'HTTP 404'` 하나뿐이고, 그때 같은 사실이 `status='DELETED'` 로도 기록됩니다. **이미 전달되고 있는 사실의 사본**입니다.
- `from_status` · `to_status` 는 `report_status_history` 의 내용이고, 그 표가 왜 읽히지 않아도 되는지는 이미 표 단위 목록에 적혀 있습니다.

`reviewed_by` 처럼 화면이 약속해 놓고 지키지 않은 것은 이 열여섯 중에 없습니다.

스스로 인정하는 한계

홑기는 **열 이름 단위**입니다. 표를 가리지 않으므로, 한 표에서 읽히는 이름은 다른 표에서 쓰기만 해도 통과합니다. 놓치는 쪽으로 틀리고 없는 것을 지어내지 않습니다 — 아무도 다투고 싶지 않은 검사가 해야 하는 선택입니다.

3.10.191 v0.236.0에서 완료 — 조직도에 고리가 생기면 그 아래 화면이 영영 안 열립니다

조직 계층을 봤습니다. API 는 고리를 만들 수 **없습니다** — 조직은 이미 있는 부모를 가리키며 생기고, 그게 전부입니다. **그리고 그게 전부인 것이 문제입니다.**

```
POST /api/v1/admin/organizations   ← 만들기
(끝)
```

이름을 고치거나, 부모를 바꾸거나, 없애는 길이 **없습니다.** 조직개편을 해야 하는 운영자는 표를 손으로 고칩니다. 고리는 바로 거기서 생깁니다.

만들어 보고 열어 봤습니다

`전사 → 본부 → 팀 → 전사` 로 엮고, 그 안의 조직장으로 기간 보고를 열었습니다.

```
응답 없음. 60초. 브라우저가 포기할 때까지.
PostgreSQL 은 아무도 읽지 않을 중간 결과를 계속 키우고 있었습니다.
```

`UNION ALL` 은 중복을 지우지 않으므로 고리를 만나면 끝나지 않습니다. **오류는 아무에게도 가지 않습니다.** 화면이 그냥 안 열립니다 — 어떤 사람들에게만.

(v0.230.0 에서 넣은 줄이 이때 제 몫을 했습니다: `period rollup error="context canceled"` 와 추적 ID 를 단 `request failed` .)

같은 저장소가 이미 두 번 답을 알고 있었습니다

```
pptx.go         위로 걷기   WHERE ancestry.depth < 16    ← 묶여 있음
workitemlinks   의존 걷기   walk(id, path, titles, depth) ← 묶여 있음
조직 아래로 걷기 · 15군데                                 ← 안 묶여 있음
```

같은 자료를 걷는 재귀 셋 중 **둘은 알고 있었고, 셋째의 사본 열다섯 개는 몰랐습니다.** 열다섯 벌의 같은 질의는 그중 하나가 나머지가 고친 버그를 계속 안고 있을 기회가 열다섯 번이라는 뜻이기도 합니다.

`orgSubtree(placeholder)` 하나로 모으고 `depth < 16` 을 달았습니다. 16은 위로 걷기가 이미 쓰던 값입니다.

고리를 그대로 둔 배포에서 다시 열었습니다.

```
기간 보고    200 · 0.015s
팀 보고서    200 · 0.006s
분석 개요    200 · 0.008s
검색          200 · 0.009s
범위 안 업무 ['고리 업무']   ← 묶었다고 답이 비지 않습니다
```

3.10.192 v0.237.0에서 완료 — 조직을 고칠 수 있게 합니다

지난 회차는 조직도에 고리가 있으면 그 아래 화면이 열리지 않는 것을 고쳤습니다. 걷기를 깊이 16으로 묶어 **견딜 수 있게** 했습니다. 그런데 릴리스 노트에 이렇게 적었습니다.

> *"API 는 조직 고리를 만들 수 없습니다 — 조직은 이미 있는 부모를 가리키며 생기고, 그게 전부입니다. **그리고 그게 전부인 것이 문제입니다.**"*

원인은 그대로 남아 있었습니다.

```
POST /api/v1/admin/organizations   ← 만들기
(끝)
```

이름의 오타 하나도 고칠 수 없습니다. 조직개편은 해마다 일어납니다. 고칠 방법이 없으면 운영자는 표를 직접 만지고, **표에는 검사가 없습니다.**

손으로는 할 수 없는 검사를 여기서 합니다

`PATCH /api/v1/admin/organizations/{id}` 가 이름·코드·상위 조직을 바꿉니다. 보낸 키만 바꿉니다 — `parentId` 를 **보내지 않는 것**과 `null` 로 **보내는 것**은 다른 지시이고(그대로 두기 / 최상위로 올리기), `decodeJSONFields` 가 그 차이를 압니다.

그리고 자기 자신이나 **자기 아래 조직**을 상위로 삼으려 하면 거부합니다. 방금 만든 `orgSubtree` 로 "그 조직이 내 아래에 있는가"를 물어 봅니다 — 고리를 견디게 만든 그 함수가 이제 고리를 막는 데 쓰입니다.

```
전사 → 본부 → 팀 인 상태에서
전사의 상위를 팀으로  → 409 ORGANIZATION_CYCLE
  "그 조직은 이 조직 아래에 있습니다. … 그 안의 사람들에게 팀 단위 화면이 열리지 않습니다."
팀의 상위를 전사로    → 200   (고리가 아닌 이동은 됩니다)
```

지우는 경로는 만들지 않았습니다

사람과 보고서와 PPTX 템플릿이 조직을 가리킵니다. `ON DELETE SET NULL` 이라 지워도 데이터는 남지만, **지운 조직을 가리키던 사람은 소속이 사라진 채 남습니다.** 그 상태를 만들어 놓고 화면에서 어떻게 보여 줄지는 별개의 결정입니다. 대부분은 이름을 바꾸는 것으로 끝나고, 정말 없애야 한다면 구성원을 먼저 옮겨야 합니다. 계약에 그렇게 적었습니다.

3.10.193 v0.238.0에서 완료 — 숫자는 맞고 낱말이 틀렸습니다

지금까지 응답의 **크기**와 **문장**은 쟀는데, **숫자가 맞는지**는 잰 적이 없었습니다. 씨를 뿌린 배포에서 기간 보고가 내놓는 값을 데이터베이스에서 따로 세어 맞춰 봤습니다.

```
제품 : sourceItems 35 · totalItems 7 · sourceReports 5 · dedupRate 80
DB   : 원본행 35 · 업무 7 · 보고서 5
```

전부 맞습니다. 그런데 같은 묶음에 이런 값이 함께 있었습니다.

```
duplicatesCut: 46
```

35건에서 46건을 뺄 수는 없습니다. 화면은 그 둘을 **한 장의 카드**에 나란히 적었습니다.

```
제목: 중복 46건 제거 · 동일 업무 0건 병합
본문: 주간보고 업무 35건을 7건으로 정리했습니다(중복률 80.0%).
```

두 숫자, 두 단위, 낱말은 하나

`DuplicatesCut` 은 `lineSet.dropped` 의 합입니다 — 한 업무의 여러 주치를 한 항목으로 접을 때 **똑같이 반복된 문장 줄**을 세어 버린 수입니다. `SourceItems → TotalItems` 는 **행**이 업무로 접힌 수입니다. 둘 다 참이고, 둘 다 "중복"이라 불렸습니다.

읽는 사람에게는 산수가 깨진 것으로 보입니다. 이 제품이 경영 보고에 쓰이는 물건이라는 점에서, **숫자가 틀린 것보다 틀려 보이는 것이 나을 것도 없습니다.**

고친 것은 산수가 아니라 낱말입니다.

```
제목: 업무 35건을 7건으로 정리 · 동일 업무 0건 병합
본문: 주간보고에 35번 적힌 업무가 실제로는 7건이었습니다(중복 80.0%).
      그 과정에서 여러 주에 걸쳐 같은 내용을 반복해 적은 문장 46줄도 하나로 합쳤습니다.
```

기간 보고 화면의 업무별 지표(`중복 제거 N건` → `반복 문장 N줄`)와 발표 슬라이드도 같은 낱말을 쓰고 있어 함께 고쳤습니다.

3.10.194 v0.239.0에서 완료 — 숫자가 참인지 묻는 검사가 없었습니다

지난 회차는 숫자 하나를 데이터베이스와 맞춰 보고, 산수는 맞고 낱말이 틀렸다는 것을 찾았습니다. 이번에는 **전부** 맞춰 봤습니다 — 사람 열 명, 기간 네 종류, 개인과 조직 범위, 조합마다 스무 가지 값.

```
40개 조합 · 어긋난 값 0개
```

`sourceItems` , `sourceReports` , `reportedWeeks` , `expectedWeeks` , 완료율, 커버리지, 분류별 지분의 합, 추이의 주별 보고서 합, 참여자별 보고서 합 — **전부 데이터베이스와 같습니다.** 이 제품의 집계는 건전합니다.

두 번 틀린 것은 검사였습니다

처음 돌렸을 때 60개가 어긋났습니다. 화면이 언제나 **하나 더** 셌습니다.

```
검사 : week_start BETWEEN start AND end      ← 그 기간에 '시작한' 주
제품 : 기간에 '걸친' 주                       ← 8월 보고에 7월 27일 주가 들어갑니다
```

두 번째로, 조직 범위의 `totalItems` 가 543 대 256으로 크게 달랐습니다. 제목으로 세어 봐도 267이었습니다.

```
543  서로 다른 work_item
267  정규화한 제목
256  제품                  ← 사람을 가로질러 '비슷한' 제목까지 병합한 뒤
```

한 사람 안에서는 저장된 업무 식별자가 정답이지만, **사람을 가로지르는 식별자는 없으므로 제목이 유일한 증거**입니다. SQL 로는 재현할 수 없고, 그래서 **등식이 아니라 상한**으로만 검사합니다.

`number-check.py`

`scripts/` 의 다른 검사는 전부 **모양**을 잽니다 — 응답이 얼마나 큰지, 얼마나 걸리는지, 가드가 도는지, 표가 읽히는지, 받은 파일로 설치가 되는지. **화면의 숫자가 데이터베이스의 숫자인지 묻는 것은 없었습니다.** 이 제품의 숫자는 누군가의 경영 보고로 들어갑니다.

검사가 실제로 잡는지 확인했습니다. 완료율에 1을 더하고 보고 주 수에서 1을 빼서 이미지를 만들었더니, **세 방향에서** 신고했습니다 — 데이터베이스와의 차이, 비율과 그 분자의 불일치, 그리고 추이 배열과 요약의 불일치.

배운 두 가지는 도구의 첫 문단에 적어 두었습니다. 다음 사람이 같은 자리에서 두 번 틀리지 않도록.

3.10.195 v0.240.0에서 완료 — 아무도 마감을 넘긴 적이 없고, 아무도 한 주를 빠뜨린 적이 없습니다

지난 회차의 숫자 검사를 다른 집계 표면에 겨눴습니다 — 관리자 참여 현황. 숫자는 전부 맞았습니다. 그런데 **어떤 숫자는 언제나 같은 값**이었습니다.

```
12주 내내 · 305명 · 지연 0건 · 정시율 100.0%
뒤처진 사람 0명
```

씨앗이 그 상태를 만든 적이 없습니다

```
submitted_at = wk.day + interval '4 days 15 hours'   -- 모든 보고서, 모든 주
```

기본 마감은 주차 시작 +7일의 자정입니다. **모든 배포에서, 52주 전부, 언제나 정시**였습니다. 참여 현황의 지연·정시율과, 운영자가 그것을 의미 있게 만들려고 조정하는 마감 설정이 **실물 한 줄과도 만난 적이 없습니다.**

이 파일 위쪽에는 같은 교훈이 이미 적혀 있습니다 — 보고서 **상태**가 전부 APPROVED 라 *"제출률이 모든 배포에서 정확히 100.0%"* 였다는 기록입니다. **상태는 고쳐졌고 시각은 그대로였습니다.**

두 번째로, 305개 계정이 **전원 52주를 다 냈습니다.** 보고 커버리지는 모두 100%였고, 누가 뒤처졌는지 알려 주는 화면은 부를 이름이 없었습니다. 주간보고 도구의 씨앗이 **빠뜨린 주를 한 번도 본 적이 없다면**, 그 도구가 존재하는 이유를 시험한 적이 없는 것입니다.

세 번째는 켜 보고 나서야 보였습니다

빠뜨린 주를 흩뿌리자 뒤처진 사람이 146명 나왔습니다. 그런데 목록의 처음도 끝도 **누락 1주**였습니다. 이 화면은 *"누락 주차 많은 순"* 으로 정렬해 25명만 보여 줍니다. **모두가 똑같이 한 주씩 빠뜨렸다면 그 정렬은 아무것도 가르지 못하고, 보여 준 스물다섯은 그냥 아무나입니다.**

긴 휴직이나 퇴사처럼 **크게 뒤처진 사람**을 더했습니다. 그 화면이 존재하는 이유입니다.

```
고침 전 : 지연 0 · 정시율 100.0% · 뒤처진 사람 0
고침 후 : 지연 18~21건/주 · 정시율 92.8~93.5% · 뒤처진 사람 150명
          목록 맨 위 누락 6주, 맨 아래 누락 1주
```

켜 보니 제품은 옳았습니다

150명 중 25명만 보여 주면서 화면은 **"150명 중 25명"** 이라고 적습니다. 정렬도 맞고, 마지막으로 제출한 주도 함께 보여 줍니다. 새 씨앗에 숫자 검사와 규모 검사를 다시 돌려 둘 다 통과했습니다.

세 가지를 씨앗의 자기 점검에 넣었습니다 — 마감을 넘긴 제출이 200건 미만이면, 52주 중 몇 주를 빠뜨린 사람이 30명 미만이면, 빠뜨린 주 수가 세 가지 미만이면 씨앗이 스스로 실패합니다.

3.10.196 v0.241.0에서 완료 — 경영 요약이 업무가 아니라 자기 창을 말하고 있었습니다

이 제품에서 가장 높은 자리로 올라가는 문장을 처음으로 검증했습니다. 주장은 **사실이었습니다** — 디지스트가 "이슈가 8주째 지속" 이라 한 업무를 데이터베이스에서 확인하니 그 문장은 그대로 참이었습니다.

그런데 창을 바꿔 봤습니다.

```
창  4주 → "이슈가  4주째 지속되고 있습니다."
창  8주 → "이슈가  8주째 지속되고 있습니다."
창 12주 → "이슈가 12주째 지속되고 있습니다."
창 20주 → "이슈가 20주째 지속되고 있습니다."
```

리더가 읽는 숫자가 업무가 아니라 자기가 고른 창이었습니다. 그 이슈는 실제로 25주치 보고에 적혀 있습니다.

같은 날 같은 업무, 두 화면이 다른 말을 했습니다

```
업무 추적 (창 없음) : 이슈 22주 · 정체 32주
경영 요약 (창 8주)  : 이슈가 8주째 · 진척이 8주째
```

`workitems.go` 는 `since=""` 로 전체를 읽고, `meeting.go` 만 창을 넘깁니다. 그리고 그 파일의 주석에는 이 문장을 한 번 고친 기록이 있습니다 — 진척 정체를 *보고 건수*가 아니라 *달력의 주*로 세도록. **단위는 고쳐졌고 크기는 그대로였습니다.**

점수도 함께 잘려 있었습니다

`digestWeightIssuePerWeek * IssueRunWeeks` 이므로, 창에 닿은 모든 업무가 같은 점수를 받습니다. **32주 정체와 8주 정체가 같은 자리에 섭니다.** 어느 창에서든 동점이 20건씩 나오던 이유입니다.

창은 후보를 고르는 값이지 기간을 재는 값이 아닙니다

전체를 읽고 창으로 **거른** 뒤 점수를 매깁니다. 후보 집합은 그대로입니다(창 1주 → 523건, 2주 이상 → 543건). 비용은 실측 0.075s → 0.12s 이고, 같은 범위에서 업무 추적 화면이 이미 그만큼을 읽고 있었습니다.

```
고침 후 · 창 4/8/12/20 전부 → "이슈가 23주째 지속되고 있습니다." · 1위 190점 → 575점
업무 1622 · 경영 요약 → 이슈 22주 · 정체 32주   (업무 추적과 같은 숫자)
```

회의 자료도 같은 문장(`%d주째 해소되지 않은 이슈입니다`)을 쓰고 창이 26주입니다. 더 오래 숨겨 줄 뿐 틀린 것은 같아서 함께 고쳤습니다.

3.10.197 v0.242.0에서 완료 — 같은 종류를 나머지 세 곳에서 마저 찾았습니다

지난 회차는 경영 요약의 지속 주 수가 창을 말한다는 것을 고쳤습니다. 같은 모양이 남은 창 사용처에도 있는지 마저 봤습니다. **셋 다 있었습니다.**

업무 인사이트 — 중복 투자를 판단하라는 그 숫자

```
같은 한 쌍 · 창   4주 → overlapWeeks  4
같은 한 쌍 · 창 104주 → overlapWeeks 52      (기본값은 12주)
```

문장은 이렇습니다. *"서로 다른 조직에서 **%d주 동안** 함께 진행 중이며 핵심 용어가 일치합니다. **중복 투자 여부를 확인하세요.**"* 1년을 나란히 달린 두 조직의 업무가 기본 화면에서 **12주**로 읽힙니다.

그리고 `linkRank = Similarity*1000 + min(OverlapWeeks, 999)` 입니다. 제목이 똑같아 관련도가 100인 쌍이 여럿일 때 **순서를 가르는 것이 바로 이 숫자**이고, 그 숫자가 창에 닿으면 전부 같아집니다.

주간 변화 — 이미 알아차렸던 자리

```
// The same 26 weeks of history the agenda loads: ageing figures such as
// stalled weeks and silent weeks ... cannot be read from two weeks of data.
```

누군가 이미 이 문제를 알아차렸고, 답이 "창을 26주로 넓힌다" 였습니다. 옳은 직감이고 틀린 모양입니다 — 32주 정체는 여전히 26으로 읽히고, "%d주째 진척이 없습니다" 가 다시 창을 말합니다.

셋 다 같은 고침

전체를 읽고 창으로 **거른** 뒤 계산합니다. 후보 집합은 그대로고 — 업무 인사이트의 쌍 수는 창 4주와 104주에서 똑같이 362쌍입니다 — 겹침만 사실이 됩니다.

```
고침 전 : 창 4/12/26/104 → 겹침 최대 4 / 12 / 26 / 52주
고침 후 : 창 4/12/26/104 → 겹침 최대 52주 (전부)
비용    : 0.17~0.23s
```

이 종류를 세 회차에 걸쳐 다섯 자리에서 찾았습니다. 공통점은 하나입니다 — **한 매개변수가 "무엇을 볼지"와 "얼마나 오래됐는지" 두 가지를 동시에 정하고 있었습니다.** 앞의 것은 사용자가 고르는 값이고, 뒤의 것은 데이터가 정하는 값입니다.

3.10.198 v0.243.0에서 완료 — 보관 기간 두 가지가 한 번도 지켜진 적이 없습니다

CSV 내보내기(수식 주입은 이미 막혀 있었습니다)와 PPTX(제어문자는 U+FFFD 로 치환되어 파일이 깨지지 않습니다)를 확인한 뒤, **시간이 지나면 무엇이 지워지는가**를 봤습니다.

보관 기간 설정은 넷이고 넷 다 집행하는 코드가 있습니다. 그런데 그 코드가 전부 **30분 티커의 몸통 안에** 있었습니다.

```
cleanupImportSources    시험에서 0번
cleanupAttachmentFiles  시험에서 0번
pruneLoginAttempts      시험에서 0번
```

부를 수 있는 함수로 떼어내고 시험을 붙였더니 **첫 실행에서 붉어졌습니다.**

`($1 || ' days')` 는 실행된 적이 없습니다

```
failed to encode args[0]: unable to encode 30 into text format for text (OID 25)
```

`||` 로 문자열을 이어 붙이면 PostgreSQL 이 `$1` 을 **text 로 추론**합니다. Go 는 `int` 를 보내고, pgx 는 그것을 text 로 인코딩할 방법이 없어 **문장이 데이터베이스에 닿지도 못합니다.**

두 자리가 그 모양이었고, 둘 다 실패를 버립니다.

```
_, _ = a.db.Exec(ctx, `DELETE FROM api_request_metrics ... ($1 || ' days')::interval`, retention
...
if err != nil { return }   // Import 원본 정리
```

그래서 `분석 데이터 보관일` 과 `Import 원본 보관일` **은 어떤 배포에서도 한 번도 무언가를 지운 적이 없습니다.** 관리자가 90일로 두든 30일로 두든 같습니다. 요청 지표 표와 업로드된 PPTX 원본은 영원히 쌓입니다.

바로 아래 줄의 감사 기록 정리는 처음부터 옳은 형태였습니다.

```
DELETE FROM audit_logs WHERE created_at < now() - make_interval(days => $1)
```

같은 파일, 세 줄 아래, 맞는 답. 둘을 그 형태로 맞췄고, 실패를 더는 버리지 않습니다.

스윕이 무엇을 가져가는지도 처음으로 시험합니다

경계마다 양쪽에 한 줄씩 넣습니다 — 60일 지난 지표와 5일 된 지표, 60일 지난 감사 기록과 5일 된 것, 3일 지난 로그인 시도와 방금 것, 만료된 세션과 살아 있는 세션. **너무 많이 가져가는 DELETE 는 아무도 신고하지 않습니다. 그것이 지운 것은 아무도 더 보지 않는 것이기 때문입니다.**

그리고 `보관일 0` 은 "전부 보관"이라는 뜻입니다. 10년 된 기록을 넣고 0으로 두어 살아남는지 봅니다.

3.10.199 v0.244.0에서 완료 — 주석이 있다고 말한 시험이 없었습니다

지난 회차의 두 결함은 **오류가 변수에 닿지도 않는** 문장이었습니다. 그런 자리를 마흔여덟 개 세고, 하나씩 효과로 확인했습니다 — 로그인 시각은 움직이고, 세션의 마지막 활동도, 요청 지표도 쌓이고, 일부러 `PROCESSING` 으로 두고 재시작한 Import 작업도 복구됩니다. **전부 제 일을 합니다.**

그러다 `sqlparams_test.go` 의 첫 문단을 다시 읽었습니다.

> *"The full statement set is also exercised against a real server with **PREPARE** in TestDatabaseMigrationsAndSecretRotation's environment"*

그런 시험은 없습니다. 그 PREPARE 들은 추론된 형을 주석에 적으려고 **손으로** 돌린 것이고, 그 사실이 시험이 있다는 문장으로 굳었습니다.

그래서 돌려 봤습니다

제품이 싣고 다니는 SQL 문자열을 전부 뽑아 실제 스키마에 PREPARE 합니다. **269개가 통과합니다.**

지금은 하나도 깨진 것이 없습니다. 그것 자체가 결과입니다 — 그리고 다음 번은 다릅니다. 마이그레이션이 열 이름을 바꾸고 한 자리를 놓치면, 그 문장을 처음 밟는 사람이 아니라 이 시험이 먼저 압니다. 확인했습니다.

```
열 이름 하나를 바꿔 두었더니
  rollup_handlers.go: ERROR: column "org_name" does not exist (SQLSTATE 42703)
```

조각은 조각이라고 말합니다

런타임에 이어 붙는 것은 문장이 아닙니다. 세 가지 모양이 있고 셋 다 알아봅니다 — 따옴표 밖에서 `+` 로 이어지는 것, `Sprintf` 로 자리번호를 채우는 것, 그리고 **변수에 담아 두고 아래에서 `+=` 로 키우는 것**(범위 조건, 정렬, 때로는 `GROUP BY`).

건너뛴 36개를 파일별로 세어 **소리 내어 적습니다.** 문장의 8분의 1을 조용히 지나가는 검사는 이름이 말하는 그 검사가 아닙니다.

3.10.200 v0.245.0에서 완료 — README 가 싣는 숫자를 코드와 묶습니다

지난 회차는 **있다고 적혀 있으나 없는 시험**을 찾았습니다. 같은 질문을 문서 전체에 던졌습니다 — README 가 하는 사실 주장이 지금도 참인가.

숫자 주장을 뽑아 상수와 하나씩 대조했습니다. **열여섯 개 전부 맞습니다.** 디지스트 가중치 아홉 줄, 반복 업무 판정 셋, 유사·중복 임계 둘, 경영 요약 상한, 검색 확장 임계, 메일 현황 기간.

그런데 그 표 위에 이런 주석이 있었습니다.

> *"Named rather than written into the calls, because README publishes this table as the product's reasoning … and a published number that nothing pairs with the code drifts. **Two rows of that table had already gone missing before this was noticed.**"*

이미 두 번 일어난 일이고, 막는 것은 아무것도 없었습니다. 상수에 이름을 붙인 것이 답의 절반이었고, 나머지 절반이 없었습니다.

세 방향으로 어긋납니다

README 는 산문이라 컴파일되지 않습니다. 어긋나는 방법이 셋인데 셋 다 조용합니다.

```
코드가 바뀌고 README 가 그대로  → "README 에 '| 조직 간 중복 의심 | +15 |' 가 없습니다. 코드는 15 입니다"
README 가 바뀌고 코드가 그대로  → "README 에 '| 진척 정체 | 주당 +10 |' 가 없습니다. 코드는 10 입니다"
그 줄이 통째로 사라짐            → 같은 방식으로 잡힙니다   ← 실제로 두 번 일어났던 일
```

주장을 **문장째** 비교합니다. 숫자만 찾으면 표현이 바뀌어 그 줄이 사라져도 통과하는데, 사라지는 것이 실제로 일어난 일이기 때문입니다.

이것이 이 제품에서 다른 종류의 문서인 이유

경영 요약은 **모든 항목에 선정 근거를 함께 표시합니다.** README 에 그 이유가 적혀 있습니다 — *"근거를 볼 수 없는 요약은 읽는 사람이 반박할 수 없고, 반박할 수 없는 요약은 판단에 쓸 수 없습니다."* 그 표는 읽는 사람이 순위를 반박할 수 있게 하려고 싣는 것입니다. **틀린 가중치표는 없는 것보다 나쁩니다.**

3.10.201 v0.246.0에서 완료 — 숫자 다음은 행동이었습니다

지난 회차는 README 의 **숫자**를 코드와 묶었습니다. 이번에는 **행동 주장** 차례입니다 — "이렇게 하면 저렇게 됩니다". 돌아가는 배포에 하나씩 물었습니다.

```
"파일만 사라져 이미지가 404로 응답합니다"
  → 404 · ATTACHMENT_FILE_MISSING · 목록에는 available:false 로 남습니다   참

"상태는 가시성을 바꾸지 않습니다"
  → 작성 중 보고서를 팀장이 열었고, 팀 목록에도 나옵니다                    참

"변화가 없는 업무는 표시하지 않습니다" (회의 모드)
  → 팀 업무 71건 중 55건만 자료에 오릅니다                                  참

"PPTX로 내보낼 수 있는 화면은 모두 그 자리에서 발표할 수 있습니다"
  → 내보내기는 둘(주간·기간), 발표는 셋(주간·기간·회의). 상위집합입니다    참
```

전부 참입니다. 이 제품의 문서는 자기가 하는 일을 정확히 적습니다.

다만 지난 회차의 검사가 놓친 종류가 있었습니다

```
"한 번에 최대 **6,400건**까지 채우고"
```

`6400` 이라는 상수는 **없습니다.** `embeddingBatchSize(32) × embeddingRebuildBatches(200)` 입니다. 지난 회차는 "발표된 숫자에 이름 붙은 상수를 짝지어 준다"는 방식이었고, **이름 붙일 상수가 없는 숫자**는 그 그물을 그대로 지나갑니다. 배치 크기를 64로 바꾸면 README 는 조용히 절반만 맞는 문장이 됩니다.

같은 문단의 *"배경 작업이 **2분마다** 깨어나"* 도 그랬습니다 — 그 값은 `time.NewTicker(2 * time.Minute)` 안에 리터럴로 박혀 있었습니다. 이름을 주고 짝지었습니다.

```
배치 크기를 32→64  → "README 는 6,400건, 코드는 64×200=12,800건 입니다"
주기를 2분→5분     → "README 는 2, 코드는 5 입니다"
```

곱한 값과 리터럴로 박힌 값이 문서에 실리는 두 가지 방식이고, 둘 다 이제 잡힙니다.

3.10.202 v0.247.0에서 완료 — 시험 하니스가 옮기는 디렉터리가 셋인데 넷이었습니다

표 조사에서 비어 있던 것 중 아직 안 건드린 곳으로 갔습니다 — **조직별 PPTX 템플릿.** 4단 계층(`회사 → 본부 3 → 실 6 → 팀 5`)에 서로 구분되는 템플릿을 올리고 실제로 내보내 봤습니다.

```
본부 3 에만 등록          → 팀 5 의 보고서가 본부 서식으로 나옵니다
실 6 에도 등록            → 더 가까운 실 서식이 이깁니다 (본부 서식은 섞이지 않음)
다른 갈래의 팀            → 남의 서식이 새어 들지 않습니다
조직을 지정해 조회        → source: "inherited" 로 물려받은 것까지 정직하게 답합니다
내용                        → 업무 7건이 실적·계획 양쪽에 모두 들어갑니다
```

전부 정확합니다. 이번 회차의 다섯 번째 음성 결과입니다.

그런데 그 걸음을 시험으로 옮기려니 올라가지 않았습니다

```
upload the template: 500 STORAGE_ERROR "템플릿 저장소를 만들 수 없습니다."
```

하니스에 이렇게 적혀 있습니다.

> *"**The three directories** are derived from stateDirectory at package init, so each has to be pointed somewhere writable on its own."*

넷입니다. `stateDirectory` , `stateDirectoryAttachments` , `importDirectory` , 그리고 `customPPTXPath` . 넷째가 목록에서 빠져 있어서, 어떤 시험이 템플릿을 올리려 해도 `/var/lib/weekly`

에 쓰려다 실패합니다. **조직 템플릿 기능 전체가, 계단을 포함해, 이 목록의 누락 하나로 시험에서 닿을 수 없었습니다.**

고르는 것과 그 결과는 다른 반쪽입니다

`TestOrganizationTemplateWalksUpToTheNearestOwner` 가 이미 있습니다 — `templatePathFor` 가 옳은 행을 고르는지 봅니다. 그것이 **고르는 쪽**이고, **세 단계 아래의 작성자가 주간보고를 받으면 본부 서식 위에 놓여 있는가**는 아무도 보지 않았습니다. 기능이 존재하는 이유는 두 번째 쪽입니다.

라우트를 지나 덱 안의 글자까지 확인하는 시험을 붙였고, 계단을 두 방향으로 깨뜨려 붉어지는 것을 봤습니다 — 가장 먼 조상을 고르게 했을 때, 그리고 옆 조직으로 새게 했을 때.

3.10.203 v0.248.0에서 완료 — 주간 요약 칸이 자기 이름을 말하지 못했습니다

아직 한 번도 시험되지 않은 **발표 모드**를 브라우저로 걸었습니다. README 의 주장은 참입니다.

```
요약 칸에 저장하지 않고 입력 → 발표 슬라이드 1 에 그대로
실적 칸에 저장하지 않고 입력 → 발표 슬라이드 2 에 그대로
```

편집 화면이 `{ ...report, summary, items: validItems() }` 를 넘기기 때문입니다 — 저장된 사본이 아니라 **지금 화면의 상태**입니다.

그런데 그 시험을 두 번 헛짚었습니다

`getByLabel(/요약/)` 이 아무것도 찾지 못했고, 그래서 첫 판은 **AI 초안 입력칸**에 글자를 넣고 "발표에 안 나온다"고 보고했습니다. 두 번째도 같았습니다.

자동 검사가 이름으로 찾지 못하는 칸은, 보조기술을 쓰는 사람도 이름으로 찾지 못합니다.

세어 봤습니다

```
일반 사용자 : 입력 요소 62개 중 7개에 접근 가능한 이름 없음
팀장·관리자 : 146개 중 12개
```

라벨은 거의 모든 자리에 있습니다 — 누군가 그 일을 해 두었습니다. 남은 것은 아무도 두 번 보지 않는 자리였고, **그게 사소한 쪽이 아니었습니다.**

- **주간 요약** — 이 제품이 모으려고 존재하는 바로 그 칸. 스크린리더는 *"편집 텍스트, 비어 있음"* 이라고 읽습니다
- AI 초안 입력, 캡처 파일 선택, 검토 의견

- 과거 보고의 범위 선택, 회의 주차, 업무 문장 검색
- 회의·경영요약·인사이트·인수인계의 조회 범위 선택 넷

플레이스홀더는 이름이 아닙니다. 내용이 들어가면 사라지고, 읽히는 방식이 일정하지 않습니다.

열둘 모두 이름을 주었고 **146개 중 0개**가 되었습니다. `a11y-check.py` 가 그 자리를 지킵니다 — 이름 하나를 떼어 확인했습니다.

3.10.204 v0.249.0에서 완료 — 상세로 들어가는 유일한 문이 키보드에 닫혀 있었습니다

지난 회차의 접근성 훑기를 한 차원 옆으로 옮겼습니다. 이름 다음은 **닿을 수 있는가** 입니다.

```
커서가 포인터인 비대화형 요소 738개 중 탭으로 닿지 않는 것 680개
  history 520 · work 79 · rollup 80
```

680은 부풀려진 수입니다 — `<td>` 가 부모 행의 커서를 물려받아 여러 번 세어집니다. 그래서 **행만** 다시 봤고, 거기서 진짜가 나왔습니다.

행이 유일한 문이었습니다

```
<tr key={item.id} onClick={() => openDetail(item)}>
```

네 화면 — `과거 보고` , `팀 주간보고` , `업무 추적` , `기간 업무보고` — 이 상세로 들어가는 길을 **행 자체**에 두었고, 행 안에는 같은 일을 하는 버튼이 없습니다. 마우스로는 열리고 **탭으로는 아무것도 닿지 않습니다.** 키보드만 쓰는 사람은 업무 하나의 상세도, 지난 주 보고서 하나도 열 수 없습니다.

행을 모양이 아니라 조작으로 만듭니다

`openable(open, label)` 하나를 네 곳이 함께 씁니다. 행이 탭으로 닿고, 버튼으로 읽히고, Enter 와 Space 에 답합니다. Space 는 페이지를 스크롤하므로 기본 동작을 막습니다 — 그 키가 곧 목적인 행에서는.

`event.target !== event.currentTarget` 이면 무시합니다. 행 안의 버튼에서 누른 Enter 가 행까지 올라와 상세를 함께 여는 일을 막기 위해서입니다.

```
업무 추적: 행 7개 · 초점 TR:button · Enter 로 열림 = true
과거 보고: 행 52개 · 초점 TR:button · Enter 로 열림 = true
기간 보고: 행 7개 · 초점 TR:button · Enter 로 열림 = true
```

`a11y-check.py` 가 이제 두 가지를 봅니다 — 이름이 없는 입력 칸과, **포인터 커서를 그리면서 탭으로 닿지 않는 표 행**. `tabIndex` 를 떼어 확인했습니다.

3.10.205 v0.250.0에서 완료 — 전체화면 발표에서 Tab 이 뒤쪽 화면으로 나갑니다

접근성 축의 세 번째 차원입니다. 이름(v0.248), 닿을 수 있는가(v0.249) 다음은 **열려 있는 동안 어떻게 되는가** 입니다.

공용 `Modal` 은 셋을 다 갖췄습니다 — 초점을 안으로 넣고, Tab 을 가두고, 닫을 때 열었던 자리로 돌려줍니다. 그 주석에 이유도 적혀 있습니다.

> *"Tab used to walk out of the dialog and into the page behind it, which leaves a keyboard user **editing something they cannot see**."*

자기를 직접 그리는 오버레이 둘은 그 교훈을 배우지 못했습니다.

```
공용 Modal   초점 안으로 ✓ · Tab 25회 중 밖으로 0 · 닫으면 tr 로 복귀 ✓
빠른 이동    초점 안으로 ✓ · Tab 14회 중 밖으로 0 · 닫으면 body          ✗
발표 모드    초점 안으로 ✓ · Tab 12회 중 밖으로 6 · 닫으면 엉뚱한 a      ✗
```

전체화면 발표에서 Tab 을 열두 번 누르면 **여섯 번이 덱을 벗어나 뒤쪽 화면으로** 갑니다. 발표자에게도, 화면을 보고 있는 사람에게도 보이지 않는 자리입니다. 그리고 빠른 이동을 실수로 열었다가 Escape 로 닫으면, 읽고 있던 목록에서의 자리를 잃고 문서 맨 앞부터 다시 Tab 해야 합니다.

한 번 배운 것을 나눠 씁니다

`keepFocusInside(panel, focusFirst?)` 하나를 세 곳이 함께 씁니다. 초점을 넣고, Tab 을 가두고, 사라질 때 열었던 것에 돌려줍니다. 패널 밖에 초점이 있으면 안으로 끌어옵니다 — 오버레이가 다른 곳이 초점을 쥔 채 열렸을 때입니다.

```
고침 후  빠른 이동 : 닫으면 tr 로 복귀
         발표 모드 : Tab 12회 중 밖으로 0 · 닫으면 열었던 button 으로 복귀
```

`a11y-check.py --overlays` 가 그 자리를 지킵니다 — 초점 복귀를 떼어 두 오버레이가 모두 신고되는 것을 확인했습니다.

3.10.206 v0.251.0에서 완료 — 사내망이 끊기면 "Failed to fetch"

접근성 세 회차를 마치고 다른 축으로 갔습니다. **망이 끊겼을 때.** 오프라인 사내망에 놓이는 제품에서 가장 흔한 실패입니다.

편집 중에 연결을 끊고 저장을 눌렀습니다.

```
사용자가 본 알림: "Failed to fetch"
```

그게 전부입니다. 한국어 제품에서, 이 배포가 놓이는 망에서 가장 자주 일어날 일이, **브라우저가 개발자에게 하는 말**로 나옵니다.

그 자리에는 봉투가 없습니다

```
const response = await fetch(url, ...)   // ← 여기서 거절되면
```

서버까지 닿지 못한 요청에는 `success` 도 `error.code` 도 `traceId` 도 없습니다. `APIError` 가 만들어지지 못하고, `errorText` 의 첫 줄이 그것을 그대로 통과시킵니다.

```
if (!(error instanceof APIError)) return error instanceof Error ? error.message : fallback
```

제품 밖에서 온 문장은 **제품 밖의 언어**입니다. 크롬은 `Failed to fetch`, 파이어폭스는 `NetworkError when attempting to fetch resource.`, 사파리는 `Load failed` 라고 합니다.

화면이 하는 일은 이미 옳았습니다

끊긴 채로 저장을 눌러도 **쓰던 글은 그대로 남고**, 연결이 돌아온 뒤 다시 누르면 저장됩니다. 실측했습니다. 그러니 문장이 그 사실을 말하면 됩니다.

> *"서버에 연결하지 못했습니다. 화면에 있는 내용은 그대로 있으니, 연결이 돌아오면 다시 시도하세요."*

브라우저 문자열을 하나씩 알아보는 대신 `TypeError` 로 가릅니다 — `fetch` 가 거절할 때 오는 것이 그것이고, 새 브라우저가 새 문구를 쓰더라도 종류는 같습니다. 그 밖의 외부 오류는 **한글이 없으면** 호출자의 한국어 대체 문장으로 바꿉니다.

두 화면은 그 함수를 아예 지나지 않았습니다

`과거 보고` 와 `팀 주간보고` 가 `notify(error.message)` 로 원문을 그대로 띄우고 있었습니다. 열 개 화면에서 연결을 끊고 훑어 지금은 **영어가 한 곳도 없습니다.**

3.10.207 v0.252.0에서 완료 — 답하지 않는 서버를 영원히 기다립니다

끊긴 망 다음은 **느린 망**입니다. 사내망에서 똑같이 흔합니다.

저장을 20초 늦춰 봤습니다. **화면은 잘 합니다** — 버튼이 잠기고 글자가 `저장 중…` 으로 바뀌고, 조급해진 사용자가 네 번 눌러도 요청은 한 번만 갑니다.

그다음 **연결은 살아 있고 응답만 오지 않는** 경우를 만들었습니다. 멈춘 프록시, 사내망의 평범한 고장입니다.

```
10초: 버튼 잠김 · "저장 중…" · 알림 없음
30초: 버튼 잠김 · "저장 중…" · 알림 없음
60초: 버튼 잠김 · "저장 중…" · 알림 없음
90초: 버튼 잠김 · "저장 중…" · 알림 없음
```

제한시간이 없습니다. 사용자는 취소할 수도 다시 시도할 수도 없고, 빠져나갈 길은 새로고침뿐인데 **그건 쓰던 글을 버립니다** — 저장이 지키려던 바로 그 글입니다.

얼마로 잡을지는 재고 정합니다

300명 배포에서 가장 느린 정당한 요청을 전부 쟀습니다.

```
연도 범위 기간 보고 0.40s · PPTX 내보내기 0.40s · 업무 인사이트(104주) 0.29s
경영 요약(52주) 0.28s · 회의 자료 0.23s · 임베딩 현황 0.09s
```

30초. 실측 최악의 75배입니다. 이것은 성능 예산이 아니라, **더 기다리는 것이 더 이상 정직하지 않은 지점**입니다.

두 가지는 본질적으로 깁니다. 업로드는 파일 크기와 회선이 정하고, 임베딩 재생성은 최대 6,400건을 게이트웨이에 보냅니다 — 둘은 5분입니다.

저장을 다시 보내는 것이 안전한 이유도 있습니다. 보고서에는 버전이 있어, 서버가 이미 받았다면 두 번째 시도는 조용히 덮어쓰는 대신 `VERSION_CONFLICT` 로 갑니다 — v0.232 에서 그 문장을 사실로 바꿔 두었습니다.

```
고침 후  29초: 버튼 잠김 · "저장 중…"
         33초: 버튼 풀림 · "수정 저장" · "서버가 시간 안에 답하지 않았습니다…"
         쓰던 글: 그대로
```

사용자가 스스로 그만둔 `AbortError` 는 서버 침묵과 구분합니다 — 화면이 정리하며 취소한 요청을 실패로 알릴 일은 아닙니다.

3.10.208 v0.253.0에서 완료 — 배너가 한 말을 토스트가 흐렸습니다

망 실패 갈래의 마지막 질문입니다. **글을 쓰는 중에 세션이 만료되면** 그 글은 어떻게 되는가. `auth.session_hours` 가 있는 한 이것은 언젠가가 아니라 **반드시** 일어납니다.

재현했습니다. 요약을 쓰다가 세션 행을 지우고 저장을 눌렀습니다.

```
화면        : 편집 화면 그대로
요약 칸 값 : 쓰던 글 그대로
배너        : "세션이 만료되었습니다. 작성 중인 내용이 있어 이 화면을 그대로 두었습니다.
               새 탭에서 다시 로그인한 뒤 이 화면으로 돌아와 저장하세요." [새 탭에서 로그인]
```

이미 잘 되어 있습니다. 이 자리의 주석이 이유까지 적어 둡니다 — *"Never throw away typed work to show a login form. The editor is the one screen that can be holding an hour of it."*

그런데 그 옆에 토스트가 하나 더 떴습니다

```
토스트: "로그인이 필요합니다."
```

서버의 일반 401 문장입니다. 배너가 **화면을 그대로 두었고 어디서 로그인하면 되는지**까지 말한 옆에서, 이 문장은 더 적게 말하면서 혼자 읽히면 *"새로고침하고 로그인해라"* 로 읽힙니다 — 쓰던 글을 버리는 바로 그 동작입니다.

같은 파일의 다른 주석이 이 위험을 이미 알고 있었습니다.

> *"A banner on the login screen rather than a toast: the screen that fails first also raises its own error toast, and **whichever lands last wins.** The reason the user is looking at a login form should not be a race."*

배너를 고른 이유가 그것이었고, 토스트는 그래도 왔습니다. 배너가 서 있는 동안에는 오류 토스트를 내지 않습니다.

배너가 문제보다 오래 살면 안 됩니다

성공한 요청 하나면 쿠키가 다시 좋아졌다는 뜻입니다 — 배너가 시킨 대로 다른 탭에서 로그인하고 돌아온 것입니다. 그때 배너를 걷고 토스트를 다시 켭니다. 이것이 없으면 배너가 남고 오류 토스트가 그 페이지가 사는

내내 삼켜집니다.

배너가 설명하는 길을 끝까지 걸어 확인했습니다.

```
1) 만료 뒤   배너 서 있음 · 토스트 없음
2) 다른 탭에서 로그인하고 돌아와 저장 → "보고서를 저장했습니다." · 배너 사라짐
3) 새로고침  → 쓰던 글이 저장되어 있음
```

3.10.209 v0.254.0에서 완료 — 주소창이 화면과 다른 말을 합니다

보고서 링크를 주고받는 도구에서 **깊은 링크**를 봤습니다. 대부분 잘 됩니다.

```
로그인 전에 연 #/history?report=104  → 로그인 뒤 그 보고서가 열립니다
#/rollup?kind=QUARTER&period=2026-Q3 → 그 기간으로 복원됩니다
없는 보고서 번호                      → "이 보고서를 조회할 권한이 없습니다."
```

없는 화면 이름만 다릅니다.

```
#/nonexistent  → 대시보드를 그리고, 주소는 #/nonexistent 그대로
#/reports      → 같음 (있을 법한 이름이라 더 그럴듯합니다)
#/             → 같음
```

코드에는 바로잡으려는 의도가 있었습니다

```
useEffect(() => {
  if (!parseRoute()) replaceRoute('dashboard')
  ...
}, [])
```

계측해 보니 **첫 적재에서만** 돕니다.

```
ROUTE-PROBE hash=  parsed= undefined
ROUTE-PROBE normalising
ROUTE-PROBE after= #/dashboard      ← 여기까지는 맞습니다
--- 없는 주소로 이동 ---
(아무 출력 없음)                     ← 효과가 다시 돌지 않습니다
최종 주소: #/nonexistent
```

해시만 바뀌는 이동은 재적재가 아닙니다. 브라우저는 문서를 다시 읽지 않고 `hashchange` 만 쏘므로, 앱은 다시 마운트되지 않고 그 줄은 두 번 다시 실행되지 않습니다. `syncPage` 는 화면을 대시보드로 되돌리면서 주소는 손대지 않습니다.

결과는 **주소창이 화면과 다른 말을 하는 것**입니다. 지금 보는 것을 즐겨찾기하거나 남에게 보내면, 존재하지 않는 화면을 가리키면서 그럴듯해 보이는 주소가 갑니다.

같은 파일에 답이 이미 있었습니다

권한 없는 화면으로 갔을 때는 두 쪽을 다 합니다 — `replaceRoute('dashboard')` 와 `setPage('dashboard')`. 없는 화면에도 같은 것을 합니다.

```
고침 후  #/nonexistent → #/dashboard · #/reports → #/dashboard · #/ → #/dashboard
         정상 이동·뒤로 가기·저장 안 한 글의 이동 확인 모두 그대로
```

3.10.210 v0.255.0에서 완료 — 잘렸다고 말하지만 나머지로 가는 길이 없습니다

305명이 있는 배포에서 관리자 화면을 열었습니다. 카드 제목은 정직했습니다.

```
사용자 305명 중 100명
```

그 뒤로 가는 단추가 없습니다. 서버는 처음부터 `offset` 을 받았고, 화면이 한 번도 보내지 않았습니다. 재보니 닿지 않는 계정이 205명이고, 그중에 팀장 8명·조직장 3명·관리자 1명이 있습니다.

코드에는 판단이 적혀 있었습니다 — "상한은 검색과 함께 온다. 찾을 길 없이 잘린 목록은 잘리지 않은 목록보다 나쁘다." 맞는 말이고, **한 계정을 찾는 일**만 덮습니다. 이 화면의 나머지 절반 — 누구에게 검토 책임자가 없는지, 누가 어떤 역할인지 훑어 내려가는 일 — 은 덮지 못합니다.

같은 파일 아래쪽 감사 로그에는 이전·다음이 이미 있습니다. 같은 것을 붙였습니다.

```
1쪽  100행 · 전체 305명 중 1~100
2쪽  100행 · 전체 305명 중 101~200     첫 행: 본부장 1 → 사용자 189
4쪽    5행 · 전체 305명 중 301~305     다음 단추 비활성
검색   1행 · 쪽 넘김 없음              ← 검색은 1쪽으로 돌아갑니다
```

같은 종류를 찾는 검사

`paging-check.py` 는 `total` 을 받는 목록 화면마다 둘 중 하나를 요구합니다: 요청에 `offset` 을 싣거나, 왜 필요 없는지 허용 목록에 적혀 있거나. **"잘렸다고 말한다"는 세 번째 답이 아닙니다** — 잘렸다는 사실

을 말하면서 나머지로 가는 길이 없으면, 화면은 자기가 불완전하다는 것을 알면서 독자를 거기 세워 둘 뿐입니다.

검사 자신이 두 번 틀렸고, 둘 다 도구 안에 적어 두었습니다.

- 처음엔 `api<T>(` 바로 뒤의 문자열만 봐서 **주소를 조립해 넘기는 네 곳**을 못 봤습니다. ReportsPage 는 `query(`offset=… `)`, 감사 탭은 URLSearchParams 로 만든 변수였습니다.
- 그래서 파일 전체를 보게 했더니 이번엔 반대로, **한 파일에 사는 두 화면 중 하나만 쪽을 넘겨도 둘 다 통과**했습니다. AdminPage 가 정확히 그런 파일입니다. 화면 하나를 한 덩어리로 잘랐고, 그 자르는 규칙이 `export default function` 을 빠뜨려 **ImportPage 의 목록 호출이 통째로 사라진 채 통과**가 나왔습니다. 검사에서 사라지는 것은 통과가 아닙니다.

고치기 전 파일에 대고 돌려 실패하는 것을 확인했습니다 — 같은 파일의 감사 탭은 그대로 통과합니다.

3.10.211 v0.256.0에서 완료 — 읽을 수 없는 비밀 하나가 세 곳을 조용히 무너뜨립니다

모든 화면을 브라우저로 훑으면서 콘솔 오류를 모았습니다. 한 곳에서 나왔습니다.

```
current     ⚠ 1   내 주간보고
     [error] Failed to load resource: 500 (Internal Server Error)
     GET /api/v1/reports/current/candidates
     CONFLUENCE_CONFIGURATION_ERROR "Confluence 설정을 읽을 수 없습니다."
```

화면에는 아무 말도 없습니다. 알림도, 오류 상자도, Confluence 라는 낱말조차 없습니다. 카드 하나가 그냥 없는 채로 그려집니다.

원인은 쓰지도 않는 비밀번호였습니다

이 경로는 이미 데이터베이스에 있는 초안을 나열할 뿐 Confluence 와 이야기하지 않습니다. 그런데 설정을 통째로 불러오고, 그 길에 Confluence 비밀번호를 복호화합니다. 그 복호화가 실패하면 **작성자가 매주 여는 화면**이 500 이 됩니다.

시작 가드는 이미 이것을 막고 있습니다 — 읽을 수 없는 비밀이 있으면 기동을 거부하고 한국어로 이유를 말합니다. 그래서 볼륨을 잃고 모르는 채로 여기 도달하지는 않습니다. 들어오는 길은 하나, `WEEKLY_ALLOW_SECRET_RESET=true` 로 "비밀을 전부 다시 입력하겠다"고 답한 경우입니다. **그 답과 실제 재입력 사이의 창**이 이 이야기의 전부입니다.

그 창 안에서 벌어지던 세 가지

	그때
작성 화면	`Confluence 자동 초안` 카드가 사라진 채 콘솔에만 500
메일 워커	`level=ERROR msg="mail settings"` 를 **30초마다 영원히** — 이 파일이 "발송이 꺼져 있을 때는 30초마다 적지 않는다"고 일부러 정해 둔 바로 그 줄
관리자 암호화 카드	비밀 3개를 **세기만** 하고 하나도 읽어 보지 않은 채, "이 볼륨을 잃으면 복구할 수 없습니다"라는 **가정**을 말함

세 번째가 가장 나쁩니다. 이미 벌어진 일 위에 앞으로 벌어질 수 있는 일에 대한 경고가 걸려 있었습니다.

고침 — 읽을 수 없는 비밀은 그것이 필요한 기능만 멈춥니다

복호화 실패를 알아볼 수 있는 오류(`errSecretUnreadable`)로 만들고, 세 곳이 각자 답하게 했습니다. 초안 목록은 쓰는 설정 두 개만 읽습니다. 메일은 이것을 전송 실패가 아니라 **설정 상태**로 들고 다니며 `unusable()` 이 이름을 말합니다. 암호화 카드는 세는 대신 **읽어 봅니다**.

```
열쇠를 새로 만든 뒤   저장된 비밀 1 · 못 읽는 것 ['mail.password']
                    메일 시험: "저장된 SMTP 비밀번호를 현재 암호화 키로 읽을 수 없습니다.
                                관리자 설정에서 비밀번호를 다시 입력하십시오."
                    내 주간보고 후보: HTTP 200
다시 입력한 뒤      저장된 비밀 1 · 못 읽는 것 []
워커 로그           45초 동안 0줄
정상 배포           저장된 비밀 0·1 · 못 읽는 것 [] · 후보 200
```

덤으로, 설정 행이 아예 없는 것도 오류였습니다. `SELECT` 가 행을 못 찾으면 그대로 오류로 올라가, "쓰지 않는 기능"이 "읽을 수 없는 설정"과 같은 답을 받았습니다. 행이 없는 것은 비밀이 없는 것입니다.

모든 화면을 다시 훑어 콘솔이 조용한 것과, `내 주간보고` 에 사라져 있던 **Confluence 자동 초안 카드가 돌아온 것**을 확인했습니다.

돌아가는 길에 본 것 — 설명하지 못한 실패 한 번

이 변경을 넣고 처음 돌린 전체 시험에서 셋이 함께 깨졌습니다. 셋 다 같은 모양입니다 — **관리자가 아닌 사람이 관리자 경로에서 200을 받았습니다.**

```
--- FAIL: TestRoleAndSessionGatesAnswerBeforeTheHandlerDoes
    a USER reached the administrator list: 200 {"items":[…], "total":2}
--- FAIL: TestTheEncryptionModeIsNotReadableWithoutBeingAnAdministrator
    a writer read the deployment's key mode: 200
```

```
--- FAIL: TestTheOperatorCanSeeWhatHappenedToTheMail
    a writer read the deployment's mail health: 200
```

재현되지 않습니다. 셋만 8번 반복, 전체 6번(변경 없는 HEAD 1번 포함) — 모두 통과입니다.

제 변경 때문은 아닙니다. 이번에 손댄 것 중 인증에 닿는 것은 `secretSetting` 하나이고, 그것이 빈 값을 돌려주면 `oidcConfig` 는 "설정 미완성"으로 **막습니다**. 역할은 요청마다 `authenticate` 가 세션과 사용자를 조인해 읽고, `requireRole` 은 그 역할을 정확히 대조합니다. 공유되는 것들도 읽었습니다 — 시험마다 자기 데이터베이스를 템플릿에서 뜨고, 템플릿에는 사용자가 없으며, `t.Parallel()` 은 이 꾸러미에 하나도 없고, 부트스트랩은 ADMIN 이 이미 있으면 아무것도 하지 않습니다.

그래서 **원인을 모릅니다.** 여기 적어 두는 것은 다음에 같은 세 줄이 나왔을 때 "간헐적"이라고 넘기지 않기 위해서입니다. 권한 문이 한 번이라도 열린 채 답했다면 그것은 재현될 때까지 열린 질문입니다.

> **v0.260.0에서 답을 찾았습니다 — 제품이 아니라 제 도구였습니다.**

> `authz-check.py` 는 권한 거부를 하나씩 **지웠다가 되돌리며** 시험을 돌립니다. 수십 분 걸리고, 그동안 작업 트리는 일부러 망가진 상태입니다. 그 옆에서 제가 전체 시험을 돌렸고, 그래서 권한 문이 거부하지 않았습니다. 두 번 다 그랬습니다 — 이 세 줄과, v0.260.0에서 본 "남의 업무 연결을 지울 수 있다"까지.

> 더 나쁜 일도 있었습니다. `git add -A` 가 그 변이를 쓸어 담아 **v0.259.0 이 권한 검사 하나가 빠진 채 나갔습니다.** 자세한 것은 3.10.215에 있습니다.

3.10.212 v0.257.0에서 완료 — 발표 자료가 제일 빈 장으로 시작합니다

화면은 조용했습니다. 그래서 이번엔 **제품이 실제로 내놓는 것**을 받아 열어 봤습니다. CSV 는 BOM 도 따옴표 처리도 파일 이름도 맞고, PPTX 는 zip 무결성·슬라이드 순서·한글·토큰 치환이 다 맞습니다. 한 가지만 이상합니다.

```
업무 7개짜리 보고서 → 장별 글상자  [10, 16, 16, 16, 6]
                                    ↑ 첫 장만 업무 하나
```

장 나누기는 **가장 높은 장을 최소화**하는 DP 이고, 동점일 때는 제곱합으로 고르게 폅니다. 그런데 두 점수는 자주 동점입니다 — 비슷한 크기의 업무가 늘어선 한 주는 매번 동점입니다. 동점일 때 먼저 만난 후보를 지켰고, 그것이 가장 이른 분기라서 **남는 자리가 앞에** 옵니다.

합성 자료로 재 보니 예외가 없습니다.

```
업무  3 · 장 2 → [1 2]        업무  5 · 장 4 → [1 1 1 2]
업무  5 · 장 2 → [2 3]        업무  7 · 장 4 → [1 2 2 2]
업무  7 · 장 2 → [3 4]        업무 11 · 장 4 → [2 3 3 3]
업무  9 · 장 2 → [4 5]
```

두 장짜리 덱은 **언제나 얇은 쪽이 먼저**입니다. 그 덱이 회의에 들어가고, 첫 장이 모두가 보는 장입니다. 남는 자리는 끝에 있어야 합니다 — 끝의 짧은 장은 끝난 것으로 읽히고, 앞의 짧은 장은 덜 된 것으로 읽힙니다.

고침은 부등호 하나

제곱합 비교를 `<` 에서 `<=` 로 바꿔, 진짜 동점이면 **나중 분기**를 고르게 했습니다. 가장 높은 장의 높이는 그대로입니다 — `[3 4]` 와 `[4 3]` 의 최대값은 같습니다.

```
전:  [10, 16, 16, 16, 6]
후:  [16, 16, 16, 10, 6]      같은 내용 · 같은 장 수
```

시험은 규칙이 아니라 모양을 봅니다

`TestADeckOfEqualTasksPutsItsSlackAtTheEnd` 는 동점 규칙을 확인하지 않습니다. 비용이 같은 업무들에서 **앞 장이 뒷 장보다 적으면 안 된다**만 봅니다. 나중에 비용 모형이 바뀌어도 어떤 방법으로 도달하든 상관없게 하기 위해서입니다. 고치기 전 코드에 대고 돌려 다섯 줄로 실패하는 것을 확인했습니다.

3.10.213 v0.258.0에서 완료 — 보낸 메일에 보낸 시각이 없습니다

메일 기능은 다섯 판째인데 **실제로 나간 메일을 아무도 열어 본 적이 없습니다.** 가짜 SMTP 릴레이를 세워 받아 봤습니다.

```
From: =?UTF-8?b?7KO86rCE67O06rOg?= <weekly@example.com>
To: probe@example.com
Subject: =?UTF-8?b?...?=
MIME-Version: 1.0
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: base64
Auto-Submitted: auto-generated
```

한글 인코딩도, 제목 encoded-word 도, 본문 base64 도 맞습니다. **`Date` 와 `Message-ID` 가 없습니다.**

RFC 5322 §3.6 은 필수 헤더로 `Date` 와 `From` 둘만 꼽습니다. 빠진 것을 채워 주는 것은 **제출 서버 (MSA)** 이고, 이 제품은 사내 릴레이(MTA)의 25번 포트로 넘깁니다 — MTA 는 채워 주지 않습니다. 받는 사람의 메일 클라이언트에는 **보낸 시각이 없고**, 정렬할 기준도, 묶거나 중복을 걸러낼 열쇠도 없습니다.

고침

`Date` 는 RFC 1123Z, `Message-ID` 는 `<나노초.무작위12바이트@보내는도메인>` 입니다. 지역 부분을 발송 건에서 끌어오지 않은 것은, 실패한 보고의 **재시도는 두 번째 메시지**이고 이 헤더로 중복을 거르는 수신자가 그것을 버리면 안 되기 때문입니다.

```
Date: Fri, 28 Aug 2026 16:59:57 +0000
Message-ID: <1787936397804516132.7b66c7f11c50f5e25c2a5eb8@example.com>
```

시험은 Go 의 `net/mail` 로 **파싱해서** 봅니다 — 문자열을 찾는 것이 아니라 `Header.Date()` 가 날짜를 돌려주는지, `Message-ID` 가 두 번 부르면 달라지는지, 도메인 없는 보내는 주소에서 `<…@>` 가 되지 않는지. 헤더 주입 시험의 허용 목록도 함께 넓혔습니다 — 그 목록은 일부러 빠짐없는 목록이고, 새 헤더가 늘 때마다 사람이 "이건 우리 것"이라고 말하게 되어 있습니다.

그리고 이번에 헛짚은 것

가는 길에 **워커는 보내는데 관리자 시험은 거절하는** 모순을 봤습니다. 같은 함수, 같은 데이터베이스, 몇 초 사이입니다. 코드를 계측해 두 경로의 값을 찍어 보니 **둘 다 "보낼 수 없음"** 이었고, 그런데도 큐는 비워졌습니다.

원인은 제품이 아니라 제 책상이었습니다. `docker ps` 에 **26시간째 떠 있던 옛 컨테이너**가 같은 데이터베이스에 붙어 자기 워커로 큐를 비우고 있었습니다. 치우자 같은 건이 `QUEUED · 시도 0` 으로 남습니다 — 가드는 처음부터 설계대로였습니다.

배운 것: **한 데이터베이스에 두 배포가 붙어 있으면 측정이 아니라 소설입니다.** 모순이 나왔을 때 코드를 세 번 다시 읽기 전에 `docker ps` 를 한 번 보는 편이 빠릅니다.

3.10.211 정정 — v0.256의 서술이 과했습니다

그 절에 "**나머지는 아무도 말하지 않는다**"고 썼습니다. 사실이 아닙니다. 관리자 설정 목록은 비밀 설정을 저장할 때부터 키마다 복호화를 시도해 `복호화할 수 없음 · 다시 입력 필요` 라고 붙여 왔고(`adminSettings` , 커밋 a5bbab0), 기동은 아예 거부하며 이유를 말합니다.

v0.256이 실제로 고친 것은 그대로 유효합니다 — 작성 화면의 500, 30초마다의 워커 오류, 그리고 **키를 다루는 카드에 요약이 없던 것**. 마지막은 "아무도 말하지 않았다"가 아니라 "긴 양식 중간의 행 라벨 말고, 그 카드에 한 줄"입니다. 차이가 작아 보여도 기록이 사실보다 세면 다음 사람이 잘못된 지도를 받습니다.

3.10.214 v0.259.0에서 완료 — 비밀 설정은 넣을 수는 있어도 지울 수 없습니다

위 정정을 하다가 같은 화면에서 나왔습니다. 비밀 설정을 **지우는 길이 없습니다.**

```
PUT {"confluence.password": ""}   → {"updated":["confluence.password"]}   ← 아무것도 안 바뀜
                                    설정됨 True · 읽힘 False              ← 그대로
PUT {"confluence.password": " "}  → 설정됨 True · 읽힘 True               ← 공백 한 칸이 저장됨
DELETE 경로                       → 없음
```

빈 값이 "유지"인 것은 의도입니다 — 가려진 양식이 제출되는 것만으로 비밀번호가 날아가면 안 됩니다. 그런데 그래서 **나가는 길이 하나도 없습니다.** 연동을 끈 조직, 폐기된 계정, 그리고 지금 쓰는 키로 읽히지 않는 값이 전부 표에 남습니다. 마지막 경우가 눈에 띕니다: v0.256이 카드에서 그 키들을 이름으로 부르며 다시 입력하라고 하는데, 그 연동을 더 쓰지 않는 관리자에게는 **경고를 없앨 방법이 "원치 않는 비밀번호를 저장한다" 하나뿐**입니다.

두 가지가 더 있었습니다.

- **응답이 거짓말을 합니다.** 건너뛴 키를 `updated` 에 넣어 돌려줬습니다. 필드를 비우고 저장한 관리자는 "저장했습니다"를 보고 지워졌다고 믿습니다.
- **공백 한 칸은 진짜로 저장됩니다.** 암호화되어 들어가고, 설정은 다시 "설정됨·읽힘"으로 보이며, 릴레이에는 공백 한 칸이 비밀번호로 건네집니다.

고침

지우기는 값이 아니라 **동사**입니다 — `DELETE /api/v1/admin/settings/{key}`, 비밀 설정만, 관리자만, 감사 기록과 함께. 화면에는 저장된 비밀 옆에 `지우기` 가 붙고 무엇이 멈추는지 물어봅니다. 저장 쪽은 값을 먼저 다듬어 공백만 있는 입력을 "비움"으로 읽고, 응답과 감사 기록은 **실제로 쓴 키만** 담습니다.

시험은 고치기 전 코드에서 두 줄로 실패합니다.

3.10.215 v0.260.0에서 완료 — 운영자가 하는 가장 위험한 일을 아무것도 확인하지 않습니다

백업·복구는 도구가 있고 CI가 왕복시킵니다. **업그레이드는 없습니다.** 마이그레이션 24개가 부팅 때 적용되는데, 옛 데이터베이스 위에서 그것이 실제로 되는지 본 적이 없습니다.

손으로 먼저 했습니다. 시험이 한 번도 본 적 없는 것을 흉내 내면 그것은 추측입니다.

```
weekly:v0.166.0  ← 빈 DB + 빈 볼륨에 기동 (스키마 20)
                   사용자·보고서·업무·비밀 하나씩 API 로 씀
weekly:v0.259.0  ← 같은 DB · 같은 볼륨으로 교체 (93판, 마이그레이션 4개 뒤)
```

```
기동 ready · 스키마 24 · 보고서 1 · 항목 1 · 업무 1
'업그레이드 전에 쓴 요약입니다' 그대로 · CLOSED 그대로 · 42% 그대로
옛 판이 저장한 비밀 1개 · 못 읽는 것 없음
service.name '업그레이드시험' 그대로 · report_mail_deliveries 새로 생김
```

결함은 없었습니다. 그래서 옛 바이너리 없이도 돌아가는 시험으로 굳혔습니다.

불변식은 "행이 남는다"보다 셉니다

올린 배포는 **오늘 새로 설치한 것과 같은 스키마**여야 합니다 — 같은 컬럼, 같은 형, 같은 널 허용. 아니면 같은 판을 도는 두 서버가 자기 표에 대해 서로 다른 말을 하고, 다음 마이그레이션은 저자가 마침 보고 있던 쪽에 맞춰 쓰이게 됩니다.

`TestAnUpgradedDeploymentEndsUpWithTheSchemaAFreshInstallHas` 는 001부터 023까지 **모든 중간 지점**에서 데이터베이스를 만들고, 옛 배포가 남겼을 행을 심고, 나머지를 올린 뒤, 새로 설치한 스키마와 비교합니다. 컬럼 순서는 비교하지 않습니다 — PostgreSQL 은 추가된 순서로 번호를 매기므로 올바른 업그레이드도 거기서 달라집니다.

검사가 못 잡는 것을 알아냈습니다

전형적인 실수를 넣어 봤습니다: 새 마이그레이션 없이 `001_initial.sql` 에 컬럼을 몰래 추가. **통과했습니다.** 이 시험은 오늘의 파일로 옛 상태를 재현하므로, 옛 파일을 고치면 **비교하는 양쪽이 똑같이** 고쳐집니다.

그 실수야말로 가장 조용한 피해입니다. 이미 001을 적용한 배포는 새 컬럼을 영영 못 받고, 고친 뒤 설치한 배포는 갖습니다.

그래서 지문 원장(`internal/app/migrations.sums`)을 따로 두었습니다. 나간 마이그레이션은 역사이고 역사는 바뀌지 않습니다. 원장은 세 가지를 잡습니다.

```
001 을 고침        → "나간 뒤에 바뀌었습니다. 새 마이그레이션으로 나누세요."
원장 없이 025 추가 → "원장에 없습니다. 이 줄을 넣으세요: <해시>  025_…"
024 를 지움        → "원장에는 있는데 파일이 없습니다. 나간 마이그레이션은 지울 수 없습니다."
```

업그레이드 시험 쪽도 자기가 잡아야 할 것을 잡는지 확인했습니다 — 행을 지우는 마이그레이션과 적용되지 않는 마이그레이션 둘 다 실패로 나옵니다.

같은 판에서 나온 더 나쁜 것 — v0.259.0 은 권한 검사 하나가 빠진 채 나갔습니다

업그레이드 시험을 마치고 전체 시험을 돌리니 엉뚱한 곳이 깨졌습니다: **남이 만든 업무 사이의 연결을 모르는 사람이 지울 수 있다.** 혼자 돌리면 통과합니다. v0.256에 "설명하지 못한 실패"로 적어 둔 것과 같은 모양입니다.

`git status` 를 보고서야 알았습니다. 제가 건드리지도 않은 두 파일이 바뀌어 있었습니다.

```
M internal/app/handover.go     ← 권한 거부가 더해져 있음
M internal/app/worksearch.go   ← 권한 거부가 지워져 있음
```

`authz-check.py` 입니다. 이 도구는 권한 거부를 **하나씩 지웠다가 되돌리며** 매번 전체 시험을 돌립니다. 수십 분 걸립니다. 그 옆에서 제가 시험을 돌리면, 저는 **일부러 망가뜨린 나무를 컴파일**하고 있는 것입니다. 두 번의 "설명하지 못한 권한 실패"가 전부 이것이었습니다.

그리고 더 나쁜 일이 있었습니다.

```
git log -S '조회 권한 범위 밖의 담당자입니다' -- internal/app/handover.go
  1d7d80a fix: 저장된 비밀 설정을 지울 수 있게 하고…   ← v0.259.0
```

`git add -A` 가 도구의 진행 중인 변이를 쓸어 담았습니다. v0.259.0 의 `handover` 는 다른 조직 사람의 인수인계를 조회해도 거부하지 않습니다. 되돌렸고, `TestNobodyReadsSomebodyElsesHandover` 가 다시 지킵니다.

도구가 다시 이럴 수 없게

- **말합니다.** 시작할 때 "이 검사가 도는 동안 작업 트리는 일부러 망가진 상태입니다. 같은 디렉터리에서 build 나 test 를 돌리지 마십시오" 를 찍습니다.
- **표시를 남깁니다.** `.authz-check-running` 에 지금 어느 파일의 몇 번째 줄을 지운 상태인지 씁니다. 이 파일은 자기 자신을 위한 것이 아니라, 옆에서 이상한 실패를 보고 어리둥절할 사람을 위한 것입니다.
- **둘이 동시에 못 돕니다.** 서로의 원본을 덮어쓰기 때문입니다.
- **죽어도 되돌립니다.** `finally` 는 예외를 덮지 죽임을 덮지 않습니다. 변이 구간 동안 SIGINT·SIGTERM 에도 같은 복원을 겁니다.

그리고 마지막 문은 `version-check.sh` 입니다 — 표시 파일이 있으면 **버전을 찍어 주지 않습니다.** 판을 내보내기 직전이 이것을 멈출 마지막 자리입니다.

배운 것: **도구가 작업 트리를 고친다면, 그 사실이 트리에 보여야 합니다.** 보이지 않으면 그 도구는 자기가 찾으라고 만들어진 바로 그 결함을 만들어 냅니다.

3.10.216 v0.261.0에서 완료 — 사라진 거부를 커밋 시점에 잡습니다

v0.260.0에서 되돌린 뒤 남은 질문은 하나였습니다. **다른 것도 쓸려 들어갔는가.**

역사를 훑었습니다. 최근 60개 커밋에서 403 거부가 사라진 커밋은 하나뿐이고, 이미 고친 그것입니다.

```
v0.250.0  39곳    v0.258.0  39곳
v0.255.0  39곳    v0.259.0  38곳   ←
                  v0.260.0  39곳
```

그런데 이 셈을 **손으로 했습니다.** 다음에 같은 일이 나도 누가 같은 명령을 다시 칠 이유가 없습니다.

두 도구는 다른 질문을 합니다

`authz-check.py` 는 **"이 거부가 무엇이든 지키고 있는가"** 를 묻습니다. 한 자리씩 지우고 매번 전체 시험을 돌리므로 수십 분 걸리고, 매 커밋에 돌 수 없습니다.

새 원장은 **"아직 있는가"** 만 묻습니다. 밀리초입니다. 거부를 없애는 일이 이제 원장에서 한 줄을 지우는 **의도적인 행위**가 되고, 지나가던 `git add` 가 대신 해 줄 수 있는 일이 아니게 됩니다.

원장은 줄 번호가 아니라 **파일·핸들러·코드·메시지**를 적습니다. 줄 번호는 아무에게도 알릴 필요 없는 이유로 움직입니다.

실제 사고를 그대로 재현해 확인했습니다

```
handover 의 거부를 지움  → "거부가 사라졌습니다: handover.go  handover  FORBIDDEN  조회 권한 범위 밖의…"
원장에 없는 거부를 더함  → "새로 더한 것이라면 이 줄을 refusals.sums 에 넣으세요"
두 줄로 나눠 써서 파서가 못 읽게 함
                         → "읽지 못합니다 … 읽을 수 없는 자리는 지켜지지 않는 자리입니다"
```

세 번째가 중요합니다. 원장을 만들 때 처음 센 것은 38곳이었고, 실제로는 39곳이었습니다. 놓친 하나는 메시지가 문자열이 아니라 **변수**인 자리(`API_KEY_SCOPE_DENIED`)였습니다. 조용히 건너뛰었다면 그 자리는 원장에 없는 채로 영영 안 지켜졌을 것입니다. 그래서 **읽지 못한 자리는 건너뛰지 않고 실패**합니다.

3.10.217 v0.262.0에서 완료 — 확정된 보고서가 조용히 바뀝니다

검토 흐름을 두 계정으로 끝까지 돌렸습니다. 상태 전이는 촘촘합니다 — 작성 중인 것은 승인할 수 없고, 남의 팀 팀장은 거부당하고, 반려된 것을 또 반려할 수 없고, 승인된 것을 또 승인할 수 없습니다. 한 자리만 다릅니다.

```
제출 → 확정(CLOSED) · 제출시각 2026-08-28 20:20
확정된 뒤 요약과 항목을 전부 바꿈 → ok
  상태 확정 그대로 · 제출시각 20:20 그대로 · 진척도 10% → 95%
  상태 이력 없음
```

빠진 한 줄

```
newStatus := previousStatus
if previousStatus == "SUBMITTED" || previousStatus == "APPROVED" {
    newStatus = "DRAFT"
}
```

`CLOSED` 가 없습니다. 그리고 **`workflow.enabled` 의 기본값은 거짓**이라, 검토 절차를 켜지 않은 배포에서는 **제출이 곧 확정**입니다 — 대부분의 배포가 매주 끝나는 그 상태입니다.

두 검토 상태는 이미 제대로 다루고 있다는 점이 이 누락을 드러냅니다. 규칙은 "끝난 뒤에 내용이 바뀌었으면 더는 끝난 것이 아니다"이고, **확정은 셋 중 가장 끝난 상태**입니다. 상태 이력에 남기는 문장도 이미 그 뜻으로 쓰여 있었습니다.

그동안 팀장이 팀 주간보고에서 본 것은 `확정` 배지와 **다른 글에 속한 제출시각**이었습니다.

고침

확정도 같은 규칙을 따릅니다 — 작성 중으로 돌아가고, 제출시각·검토 기록이 지워지고, 상태 이력에 이유가 남습니다. 화면은 저장하기 전에 미리 말합니다.

```
확정된 보고서를 수정하면 작성 중 상태로 돌아갑니다. 다시 제출해야 확정됩니다.
```

시험은 `workflow.enabled` 를 껐을 때와 켰을 때 **양쪽**을 봅니다. 고치기 전 코드에서 꺼진 쪽이 두 줄로 실패합니다 — 상태가 확정 그대로인 것과, 다른 글의 제출시각이 남아 있는 것.

3.10.218 v0.263.0에서 완료 — 같은 주의 제출률이 두 화면에서 다릅니다

지난 결함은 **열거가 한 값을 빠뜨린 것**이었습니다. 같은 모양을 찾다가, 상태를 열거하는 두 곳이 서로 다른 답을 하고 있는 것을 봤습니다.

```
analytics.go        if status != "DRAFT" && status != "REVISION_REQUESTED"
admin_analytics.go  WHERE r.status <> 'DRAFT'
```

305명 배포에서 같은 주를 물었습니다.

```
관리자 참여 분석  2026-08-24: 제출 278명 · 91.1%
분석 화면        2026-08-24: 제출 250명 · 82.0%
```

```
                                차이 28 = 반려 28건
```

두 화면 다 이 숫자를 `제출률` 이라고 부릅니다. 팀장은 9.1포인트 다른 두 숫자를 같은 이름으로 받고, 어느 쪽이 답인지 알 방법이 없습니다.

어느 정의가 이기는가

둘 다 말이 됩니다 — "냈는가"와 "끝났는가"는 다른 질문입니다. 그래도 한 배포 안에서 한 낱말은 한 뜻이어야 합니다.

관리자 쪽 정의로 맞췄습니다. 그 화면의 **미제출 명단이 그 정의 위에 서 있기 때문**입니다. 반려된 사람을 미제출로 세면, 쓰고 나서 고쳐 달라는 말을 들은 사람이 **아무것도 쓰지 않은 사람과 같은 목록에** 오릅니다. 아직 안 끝난 것은 같은 화면의 상태별 분포가 이미 따로 말합니다.

`submitted_at` 도 그 정의 편입니다 — 반려는 제출시각을 지우지 않고, 정시·지각 집계는 이미 그 열을 씁니다. 이 배포에서 `status<>'DRAFT'` 와 `submitted_at IS NOT NULL` 이 어긋나는 행은 0건입니다.

시험은 정의가 아니라 일치를 봅니다

`TestBothScreensCountTheSameSubmissions` 는 한 주에 초안·제출·반려·승인 보고서를 하나씩 만들고 두 끝점에 같은 주를 묻습니다. **어느 정의가 옳은지는 묻지 않습니다** — 나중에 정의가 바뀌어도 두 화면이 함께 바뀌기만 하면 됩니다. 독자가 기대는 것은 그 일치이고, 어긋난 것도 그것입니다.

고치기 전 정의로 되돌리면 `참여 분석 3명, 분석 화면 2명` 으로 실패합니다.

3.10.219 v0.264.0에서 완료 — 오늘 설치한 배포가 이미 12건 밀렸다고 합니다

빈 배포를 세우고 모든 화면을 읽었습니다. NaN 도, 0으로 나누기도 없습니다. 한 줄만 이상합니다.

```
관리자 조직 분석   기본 조직: 구성원 1 · 기대 12 · 제출 0 · 0%
관리자 참여 분석   missing: []          ← 밀린 사람 없음
```

같은 관리자 화면 두 탭이 같은 질문에 반대로 답합니다. 그리고 이것은 설치 직후 운영자가 처음 보는 화면입니다.

참여 분석에는 이미 규칙이 있었습니다 — *"어떤 주가 누군가에게 빚이 되려면, 그 사람이 그때 여기 있었어야 하고 그 주의 마감이 지났어야 한다."* SQL 조각(`expectedFromWeek` , `deadlinePassed`)까지 상수로 뽑혀 있습니다. 조직 분석만 그것을 쓰지 않고 **구성원 × 주** 를 곱하고 있었습니다.

규칙을 나눠 쓰자 나머지 절반이 드러났습니다

분모가 아직 안 끝난 이번 주를 빼기 시작하니, 분자는 여전히 그 주의 제출을 세고 있었습니다.

```
본부 1  기대 11 · 제출 12 · 109.1%
```

화면은 이 둘을 `제출 / 기대건` 으로 나란히 보여 줍니다. 분자와 분모가 다른 주를 세면 그것은 분수가 아닙니다. 제출 쪽도 같은 주만 세게 했습니다.

```
고침 후  본부 1  기대 11 · 제출 11 · 100%
         팀 1   기대 99 · 제출 99 · 100%
         100% 를 넘는 조직 0개
```

마감 조각은 이제 **열 이름을 받는 함수**입니다. 같은 질문을 생성된 주에게도, 보고서 자신의 주에게도 하기 때문입니다 — 두 번 적으면 갈라지고, 갈라진 결과가 109.1% 였습니다.

3.10.220 v0.265.0에서 완료 — 실패한 화면이 "없습니다"라고 말합니다

첨부 패널의 주석이 이미 규칙을 적어 두었습니다.

> *A failed load used to become an empty list, and an empty list here is a claim: readers were told the report has no captures, which is a different thing from not knowing.*

그 규칙이 **그 파일에만** 있었습니다. 각 화면의 주 요청을 브라우저에서 500으로 만들고, 알림이 사라진 뒤 남는 화면을 읽었습니다.

```
업무 추적     "아직 추적할 업무가 없습니다"                        ← 544건 있습니다
인수인계      "선택한 담당자에게는 아직 인수인계할 업무 기록이 없습니다"
경영 요약     "핵심 0건 · 업무 0건 검토" · "선정 기준을 넘는 항목이 없습니다"
회의 모드     "0명 · 0건의 안건" · "이번 주에는 회의에서 다룰 변화가 없습니다"
개인 설정     "발급된 API 키가 없습니다"                          ← 폐기 확인이 여기 걸립니다
```

네 곳은 실패했을 때 **빈 화면을 지어내고 있었습니다** — `setView({people: 0, workItems: 0, sections: []})`. 그리고 알림은 사라집니다.

뒤의 둘이 특히 나쁩니다. **숫자를 지어냅니다.** 실패에서 만들어진 0이 경영 요약의 규모와 회의 안건의 수로 읽힙니다.

고침

다섯 화면 모두 실패를 상태로 들고, 화면에 이유와 `다시 시도` 를 그립니다. 업무 추적은 실패 분기를 스피너보다 **앞에** 놓아야 했습니다 — 뒤에 두었더니 `items` 가 계속 `undefined` 라 무한 스피너가 됐고, 그것도 브라우저에서 보고 알았습니다.

검사

`failstate-check.py` 가 묻는 것은 하나입니다: **요청이 실패했는데 화면이 비어 있다고 말하는가.** "화면이 오류를 말하는가"는 묻지 않습니다 — 알림으로만 말하는 것도 설계일 수 있지만, 없는 것을 없다고 말하는 것은 어떤 설계로도 옳지 않습니다.

검사 자신이 **두 번 틀렸고** 둘 다 적어 두었습니다.

- 오류 문구를 찾는 정규식에 정작 `불러오지 못했습니다` 가 빠져 있어 **대시보드를 거짓 양성**으로 잡았습니다. 제품이 아니라 도구가 틀렸습니다.
- 고친 뒤에도 못 봤습니다 — 화면은 **서버가 준 문구를 그대로** 보여 주는데 검사는 자기 한국어 표현만 찾고 있었습니다. 그래서 지금은 직접 주입한 표시 문구를 찾습니다.

허용 목록은 비어 있습니다. 처음에 개인 설정을 적어 뒀는데 검사가 "그 예외는 더는 나오지 않습니다" 라고 알려 줘서 지웠습니다. 예외는 남기는 것이 아니라 갚는 것입니다.

3.10.221 v0.266.0에서 완료 — 실패한 화면이 영원히 "불러오는 중"입니다

지난 판에서 "빈 상태로 거짓말하는가"를 고쳤습니다. 그때 여러 화면이 `알림만` 으로 남았는데, 알림은 사라집니다. **그 뒤에 무엇이 남는지** 봤습니다.

```
과거 보고      스피너 2 · "불러오는 중" 남음
팀 주간보고     스피너 2 · "불러오는 중" 남음
기간 업무보고   스피너 2 · "불러오는 중" 남음
보고 분석      스피너 2 · "불러오는 중" 남음
```

네 화면이 **끝나지 않는 적재**로 남습니다. 독자는 오지 않을 것을 기다립니다.

둘은 `catch` 가 아예 없었습니다

```
// 팀 주간보고
const load = () => api<ReportListView>(query())
  .then(value => { setReports(...) })          // catch 없음

// 보고 분석
api<AnalyticsOverview>('/api/v1/analytics/overview').then(setOverview)   // catch 없음
```

```
  if (isAdmin) api<EndpointMetric[]>('/api/v1/analytics/endpoints')
    .then(setEndpoints).catch(...)                                  // 바로 옆줄에는 있음
```

분석 화면은 **같은 줄에** 올바른 모양을 이미 갖고 있습니다 — 두 번째 요청은 `failed` 상태와 그에 맞는 문장까지 있습니다. 첫 번째만 없었고, 그래서 처리되지 않은 거부까지 함께 남았습니다.

기간 업무보고는 잡기는 하는데 `setRollup(undefined)` 로 되돌려 놓습니다. 그 아래 스피너가 `!rollup` 을 기다리므로, 잡는 것이 곧 영원히 기다리는 것이었습니다.

검사가 이제 두 가지를 묻습니다

- 요청이 실패했는데 화면이 **비어 있다**고 말하는가
- 요청이 실패했는데 화면이 **아직 불러오는 중**이라고 말하는가

`화면이 오류를 말하는가` 는 여전히 묻지 않습니다 — 알림으로만 말하는 것도 설계일 수 있습니다. 그러나 **없는 것을 없다고 말하거나, 끝난 것을 진행 중이라고 말하는 것**은 어떤 설계로도 옳지 않습니다.

두 번째 질문은 첫 번째를 고친 **뒤에야** 보였습니다. 한 층을 벗기면 그 아래가 드러납니다.

3.10.222 v0.267.0에서 완료 — 검색 결과가 "같은 문제"라고 주장합니다

검색이 읽을 권한 밖의 글을 보여 주는지 재 봤습니다. 한 사람의 보고서에 표식 문구를 심고 다섯 계정으로 찾았습니다.

```
u57 (작성자, 팀 26)  자기 표식을 찾음
u18 (팀 19)          표식 없음 — 자기 조직 것만
u89 hq1 hq2          표식 없음
```

새지 않습니다. 대신 다른 것이 보였습니다. u18 은 그 표식이 자기 범위에 아예 없는데도 **20건**을 받았고, 각 건의 설명이 이렇게 말합니다.

```
matched=[]  ·  같은 문제를 겪고 해결한 사례입니다. 해결 경과: 리포트 자동화 진행 상황을…
matched=[]  ·  같은 문제를 겪고 해결한 사례입니다. 해결 경과: 알림 자동화 진행 상황을…
```

`matched` 가 비어 있습니다. 질의와 **겹치는 낱말이 하나도 없는데** "같은 문제"라고 말합니다.

옳은 문장이 이미 그 함수 안에 있었습니다

```
case hit.Resolution != "":              // ← 여기서 걸립니다
    return "같은 문제를 겪고 해결한 사례입니다. 해결 경과: …"
case hit.Issue != "" && !hit.Resolved:
    return "아직 해결되지 않은 같은 계열의 이슈입니다…"
case semantic && len(hit.Matched) == 0: // ← 도달할 수 없습니다
    return "입력한 표현과 직접 겹치는 단어는 없지만 내용이 가까운 업무입니다."
```

겹침 없음 분기가 **세 번째**입니다. 그런데 해결 경과가 있는 항목은 첫 분기에서 이미 걸리므로, 그 분기는 이슈도 해결도 없는 항목만 볼 수 있습니다 — 즉 정작 필요한 경우에 절대 오지 않습니다.

사실을 지우지 않고 자격만 붙였습니다

해결 경과는 독자가 찾으러 온 것이므로 남깁니다. 겹침이 없다는 것은 그 문장을 **대체하는 대신 한정**합니다.

```
matched=[]        입력한 표현과 직접 겹치는 단어는 없지만 내용이 가까운 업무입니다. 해결 경과: …
matched=[자동화]   같은 문제를 겪고 해결한 사례입니다. 해결 경과: …
```

화면 쪽은 이미 정직했습니다 — `의미 유사` 칩과 "표현이 달라도 내용이 가까운 업무를 함께 찾았습니다" 를 붙이고 있습니다. 어긋난 것은 그 아래 한 줄뿐이었고, 그 한 줄이 결과의 **근거**입니다.

3.10.223 v0.268.0에서 완료 — 보고 커버리지가 아직 오지 않은 주를 셉니다

제품이 만드는 설명 문장들을 데이터베이스와 대조했습니다. 경영 요약의 근거는 정확합니다 — 업무 1753번의 `이슈가 23주째 지속`, `진척이 32주째 멈춰`, 그리고 중복 상대인 팀 19의 '로그 통합' 이 모두 실재하고, 점수 230+320+25가 합계 575와 맞습니다.

기간 보고의 인사이트 한 줄만 다릅니다.

```
2026-Q3  보고 커버리지 64.3% — 기간 내 14개 보고 주차 중 9개 주차에 보고가 등록되었습니다.
                              커버리지가 낮으면 집계 결과의 신뢰도도 함께 낮아집니다.
```

오늘은 2026-08-29 입니다. **빠진 5개 주차는 9월입니다.** 일어난 모든 주는 보고됐습니다. 이 숫자는 자료 품질이 아니라 **달력이 얼마나 지나갔는지**를 재고 있었고, 그것을 신뢰도 경고에 얹어 본부장의 덱 1장에 실었습니다.

같은 함정에 제가 다시 빠졌습니다

v0.264에서 배운 규칙(마감이 지난 주만 센다)을 그대로 가져왔더니 이렇게 나왔습니다.

```
2026-08  기대 4주 · 보고 5주 · 125%
```

분자와 분모가 다른 주를 셉니다 — v0.264가 적어 둔 바로 그 문장입니다. 참여 분석이 마감을 쓰는 이유는 *늦은 사람을 지목하기 때문*이고, 열려 있는 주가 누구에게도 불리하면 안 됩니다. 커버리지는 다른 질문입니다 — *지금 보고 있는 집계가 몇 주 위에 서 있는가* — 이고 그 분자는 시작한 주의 보고를 셉니다.

그래서 자르는 기준은 **"이 주가 시작했는가"** 입니다. 두 규칙은 정확히 한 주 차이이고, 그 한 주가 분자에 있습니다.

```
고침 후  2026-08 5/5 100% · 2026-Q3 9/9 100% · 2026-06 5/5 100%
         2026-Q4 0/0 · 2026(연간) 35/35 100%
```

검사도 옛 정의였습니다

`number-check.py` 가 여섯 조합에서 어긋났습니다. 검사가 걸치는 모든 주를 세고 있었습니다 — 8월에 "올해 35/53주" 라고 말하는 그 정의입니다. **제품이 아니라 검사가 틀렸고**, 이 파일의 세 번째 교훈으로 적었습니다.

3.10.224 v0.269.0 — 여섯 곳을 확인했고, 여섯 곳 다 맞았습니다

이번 판에는 고친 결함이 없습니다. **확인한 것을 기록해 두는 것이 이 판의 내용**입니다. 적어 두지 않으면 다음 사람이 같은 여섯 곳을 다시 밟습니다.

무엇	어떻게	결과
경영 요약의 근거 문장	업무 1753번을 SQL 로 대조	`이슈 23주째` = 연속 23주 · `정체 32주` = 진척 34로 32주 · 중복 상대 팀 19의 '로그 통합' 실재 · 점수 230+320+25 = 575
중복 의심의 겹침 주 수	668·924 의 보고 주차 교집합	각 52주, 겹침 52주 — 문장 그대로
반복 업무의 진척 변화	236번의 첫 주와 마지막 주	0% → 10% = `+10%` · 52주 보고 · 이슈 7주
덱이 보고서를 다 담는가	항목 9개 · 실적 1,300자 · 이슈·요청 포함해 내보내고 XML 에서 표식 찾기	**하나도 빠지지 않음** (4장 + 이슈 장)

무엇	어떻게	결과
재시작 중이던 Import	작업을 PROCESSING 으로 만들고 컨테이너 재기동	되돌아가 처리됨 — 여러 문장을 한 `Exec` 로 보내는 그 줄은 실제로 돕니다
조직별 PPTX 서식 상속	본부에만 올림 → 팀이 상속 · 실에도 올림 → 더 가까운 쪽	`inherited · 본부 1` → `inherited · 실 4`, 덱의 표식도 함께 바뀜

기존 검사도 다시 돌렸습니다 — 접근성(조작 가능한 자리 335개), 닫기 가드, 실패 상태, 숫자 검사 모두 통과입니다.

그다음 판에서 이어 확인한 네 가지 (판을 올리지 않았습니다)

무엇	어떻게	결과
업무 병합	주차 기록 52건짜리 둘을 합침	52건이 모두 옮겨지고, 원본 행은 `merged_into_id` 로 남아 참조가 끊기지 않음. 같은 주에 기록이 둘이 돼도 상세는 **52주로 접어** 셈
업무 분리	최근 6주를 새 업무로	104 → 98 + 6, 주인 없는 기록 0건
병합·분리 권한	남의 업무를 내 것으로, 내 것을 남의 것으로, 남의 주차 기록을 내 분리에 끼워 넣기	각각 `DIFFERENT_OWNER`, `FORBIDDEN`, `ITEM_NOT_IN_WORK_ITEM` — 마지막 것까지 막습니다
결정·댓글	남이 조회·수정·삭제·댓글 시도	전부 거절. 기한이 결정일보다 이른 것도 거절. 먼 미래 날짜는 허용되지만 열린/지난 결정 수를 왜곡하지 않음

코드가 바뀌지 않아 판을 올리지 않았습니다. 적어 두는 이유는 3.10.224와 같습니다 — **확인하고 적지 않으면 다음 사람이 같은 곳을 다시 밟습니다.**

두 판 연속으로 아직 안 본 기능들이 깨끗했습니다. 다음에는 기능을 하나씩 더 표집하는 대신, **부하·시간·규모** 쪽으로 자리를 옮기는 편이 낫습니다.

고친 것 하나 — 파일이 자기에 대해 과장했습니다

`presentSlides.ts` 의 첫 문장이 *"화면에 보이는 덱은 내보낸 파일과 같은 내용을 담는다"* 였습니다. 같은 **자료**에서 나오는 것은 맞지만 같은 **글**은 아닙니다 — 화면은 700자·420자에서 자르고, 캡처 위치도 일부러 다릅니다. 같은 파일 아래쪽 주석들이 그 둘을 이미 설명하고 있어서, 첫 문장만 사실보다 셌습니다.

이 로프에서 계속 고쳐 온 것이 바로 그 종류이고, 제 자신의 기록에서도 한 번 고쳤습니다(3.10.211 정정). 코드 주석도 같은 잣대를 받아야 합니다.

3.10.225 v0.270.0에서 완료 — AI 클라이언트에게 소수 열여섯 자리를 건넸습니다

아직 안 본 곳으로 갔습니다 — **MCP 표면**입니다. AI 클라이언트가 쓰는 산출물이고 자체 범위 모델이 있습니다.

범위는 정확합니다.

```
범위 없는 키 → API_KEY_SCOPE_DENIED "이 API 키에 mcp:read 범위가 없습니다"
본부장  SELF 7건 · TEAM 265건 · ORG 거절
평사원  SELF 8건 · TEAM "조직 단위 집계는 팀장 이상만" · ORG 거절
```

한 가지만 눈에 띕니다.

```
weekly_submission_overview → submissionRate 90.78947368421052
                             averageProgress 63.882797731569
```

형제 파일들은 모두 반올림합니다

```
admin_analytics.go   round1( ... )  6곳
rollup.go            round1( ... ) 10곳
analytics.go         round1        0곳
```

같은 주, 같은 사람들을 두고 참여 분석은 `90.8` 을, 이 끝점은 `90.78947368421052` 를 답합니다. 화면은 `.toFixed(1)` 로 찍으니 **화면은 원래 옳았고**, 틀린 것은 API 와 그 위에 세운 MCP 도구입니다. 한 주를 요약하라고 부탁받은 AI 에게 인용할 백분율을 열여섯 자리로 건넨 셈입니다.

시험이 자기를 시험하지 못한 것을 두 번 고쳤습니다

- 표본을 4명으로 만들어 25% 가 나왔습니다. **딱 떨어지는 비율은 반올림을 시험하지 못합니다.** 고치기 전 코드로 되돌려도 통과했습니다.
- 관리자로 물었습니다. ADMIN 은 배포 전체가 모집단이라 시험이 만든 계정 말고도 전부 들어와 비율이 또 떨어졌습니다.

셋으로 줄이고 같은 조직의 팀장으로 물으니 `33.333333333333336` 이 나오고, 그제야 시험이 실패합니다. **표본이 그 상태를 만들지 못하면 그것은 그 상태의 시험이 아닙니다.**

3.10.226 v0.271.0에서 완료 — 3년 된 배포를 처음으로 만들어 봤습니다

기능을 하나씩 표집하는 것이 두 판 연속 깨끗했으니 자리를 옮겼습니다 — **부하·시간·규모**입니다.

`seed-scale.sql` 은 **넓은** 배포를 만듭니다: 305명, 1년. **오래된** 배포는 만들지 않고, 둘은 다른 결함을 숨깁니다. 1년짜리 역사는 "누가 어느 주를 빚졌는가"를 정하는 규칙을 한 번도 시험하지 않고, 3년짜리 창을 기간 보고 뒤에 놓아 보지도 않습니다.

기존 1년을 뒤로 복제해 3년으로 만들었습니다 — 보고서 47,424건, 항목 331,590건, 304 MB.

```
scale-check      23개 경로 지적 없음 (검사가 못 잰 2건은 스스로 밝힘)
가장 무거운 창    YEAR 집계 157ms · digest 52주 339ms · work-graph 104주 350ms
동시 사용        쓰기 120명 · 읽기 25명 · 3회전 → 오류 0 · 저장 p99 48ms · 조회 p99 517ms
컨테이너 메모리   66 MiB → 54 MiB
```

참여 규칙을 세 방향으로 시험했습니다

v0.264에서 손댄 규칙이 3년치에서도 맞는지 봤습니다. 처음에는 **제 SQL 이 순진해서** 제품과 어긋났습니다 — 52주 창에 126명이 빠졌다고 셌는데, 실제로는 `admin` 이 사흘 전에 만들어진 계정이고 나머지 125명은 **아직 마감 전인 이번 주**만 빠진 것이었습니다. 코드 주석이 경고하는 바로 그 순진한 셈이었습니다.

반대 방향으로 확인했습니다. u30 의 창 **안쪽** 두 주를 지우니:

```
12주 창(2026-06-08~)   미제출 0명            ← 지운 주가 창 밖
52주 창(2025-09-01~)   1명 · u30 누락 2주
104주 창(2024-09-02~)  29명 · u30 누락 2주
```

복제 때문에 계정의 `created_at` 은 사흘 전인데 보고서는 2023년까지 있습니다. 그것이 바로 `expectedFromWeek` 이 존재하는 이유 — 과거를 Import 한 배포 — 이고, 규칙은 `least(created_at, 첫 제출 주)` 로 옳게 처리합니다.

씨앗 하나를 남겼고, 그 씨앗이 저를 한 번 속였습니다

`seed-years.sql` 로 "3년으로 늘리기"를 한 줄로 만들었습니다. 첫 판은 **모든 주를 원본으로** 썼습니다. 그럴듯하고 틀립니다 — 복제본이 생기는 순간 그것도 원본이 되어, 서로 다른 (원본, n) 짝이 같은 월요일에 떨어집니다. `NOT EXISTS` 는 같은 문장이 넣고 있는 행을 볼 수 없으므로 두 번째 실행은 **중복 키로 죽고 롤백**됐습니다.

그런데 행 수는 그대로였습니다. 저는 그것을 보고 "멱등하다"고 적을 뻔했습니다. **롤백은 성공과 똑같이 생겼습니다.** 원본을 최신 1년으로 좁히고 `ON CONFLICT DO NOTHING` 을 달아, 이제 다시 돌리면 두 INSERT 모

두 0건입니다.

그다음 판에서 3년치 위에 이어 확인한 것 (판을 올리지 않았습니다)

무엇	결과
156주짜리 업무의 지표	나이 156 · 정체 32 · 이슈연속 22 · 계획반복 156 — 모두 자료와 일치. 넘침도 음수도 없음
완료 예측	과거 기간은 전부 `NONE` (남은 기간이 없음). `DISTANT` 는 **두 속도 중 빠른 쪽이 양수**일 때만 붙고, 그래서 "진척은 늘고 있으나" 가 성립합니다 — `시작 45 → 끝 8` 로 보이는 것은 해마다 되감기는 제 톱니 표본 탓이었습니다
창 상한	work-graph 104주 · digest 52주 · 참여 104주 — 셋 다 잘라 놓고 `weeks` 와 `since` 로 **잘랐다고 말합니다.** 화면이 고르게 하는 값(26·52주)은 모두 상한 안쪽입니다

그리고 `seed-years.sql` 에 경고를 한 줄 넣었습니다. 그것은 **있는 것을 그대로 불립니다** — 제가 제품을 찔러 보며 남긴 시험용 보고서가 두 번 더 복제됐고, 상위 조직 요청이 달린 탐침 업무가 **쓰인 적 없는 해의 경영 인사이트에 이름으로** 올라왔습니다. 늘리기 전에 치우거나, 시작하기 전에 늘리십시오.

3.10.227 v0.272.0에서 완료 — 자주 재기동하는 배포는 보관 정책이 영영 적용되지 않습니다

3년치 배포가 새로 열어 준 자리를 봤습니다. **보관 기간 정리**는 오래된 자료가 있어야 확인할 수 있고, 지금까지는 그런 자료가 없었습니다.

1,100일 된 감사 기록과 400일 된 요청 지표를 심고 컨테이너를 재기동했습니다.

```
audit.retention_days = 365     남은 것: 300, 400, 500, … 1100일
analytics.retention_days = 90  남은 것:  80, 120, 160, …  400일
```

하나도 지워지지 않습니다.

```
func (a *App) maintenance(ctx context.Context) {
        ticker := time.NewTicker(30 * time.Minute)
        for { select { case <-ticker.C: a.runMaintenance(ctx) } }   // 티커에서만
}
```

30분마다 도는 것이 전부입니다. **기동 때는 한 번도 돌지 않습니다.** 30분보다 자주 재기동하는 배포 — 롤아웃, 크래시 루프, 설정을 적용하려고 다시 켜는 운영자 — 는 **한 번도 쓸지 못합니다.** 감사 기록도, 요청 지표

도, Import 원본도, 로그인 시도도, 만료된 세션도 전부 쌓이기만 합니다.

이 함수의 주석은 이미 그 방향을 가리키고 있었습니다. `runMaintenance` 는 *"불릴 수 있는 한 곳"* 으로 뽑혀 있었는데, 부르는 곳이 여전히 티커 하나뿐이었습니다.

```
고침 후 재기동 → 남은 audit 300일 하나 · 남은 metrics 80일 하나
```

경계가 정확합니다. 덤으로, 보관일을 방금 바꾼 운영자에게 **적용할 방법**이 생겼습니다 — 30분을 아무 표시 없이 기다리는 대신 재기동하면 됩니다.

도구가 또 저를 속였습니다

처음 심은 감사 기록 9건이 다음 조회에서 사라져 있었습니다. 저는 "정리가 도는데 300일짜리까지 지운다"고 읽을 뻔했습니다. 사실은 같은 `psql -c` 안의 **뒷 문장이 실패해 트랜잭션이 통째로 롤백**된 것이었습니다 — 앞의 `INSERT 0 9` 는 이미 찍힌 뒤였습니다.

v0.271.0의 씨앗도 같은 모양으로 저를 속였습니다(롤백이 성공처럼 보임). **한 번에 여러 문장을 보내면, 앞의 성공 표시는 아무것도 보증하지 않습니다.**

3.10.228 v0.273.0에서 완료 — 안내서가 제품과 반대로 적혀 있습니다

새 각도로 갔습니다 — **문서가 약속하는 것과 제품이 하는 것**을 맞춰 봤습니다.

이름 대조는 깨끗합니다. 문서가 대는 설정 키 10개는 모두 제품이 아는 82개 안에 있고, 환경변수도 어긋나지 않습니다. `WEEKLY_STATE_DIR` 이 제품 소스에 없어 한 번 의심했는데, **백업 스크립트**가 읽는 것이었고 문장도 그 절 안에 있었습니다 — 제 대조가 좁았습니다.

사용자 안내서 두 줄이 제품과 **반대**입니다.

```
안내서: 충돌 시 새로고침 안내가 표시됩니다
제품:   "…저장을 한 번 더 누르면 됩니다. 새로고침하면 지금 쓰고 있는 내용이 사라집니다."

안내서: 확정(CLOSED) 보고서는 수정 저장 후에도 확정 상태를 유지합니다
제품:   확정본을 고치면 DRAFT 로 돌아갑니다 (v0.262.0)
```

첫 줄은 **하지 말라는 일을 하라고** 시킵니다. 새로고침은 쓰던 내용을 지우는 유일한 동작이고, 제품의 문구는 그것을 막으려고 일부러 그렇게 쓰였습니다. 둘째 줄은 제가 v0.262.0에서 바꾼 규칙이고, 그때 안내서를 고치지 않았습니다.

실제로 눌러 확인했습니다 — 확정본을 고치니 `DRAFT`, 낡은 버전으로 저장하니 그 문구 그대로 돌아옵니다.

"문서 버전"이 265판 뒤처져 있었습니다

```
docs/USER_GUIDE.md    문서 버전: v0.7.0-ENTERPRISE
docs/ADMIN_GUIDE.md   문서 버전: v0.7.0-ENTERPRISE
docs/ROADMAP_PLAN.md  문서 버전: v0.25.0 (현재) ~ v1.0-VISION
VERSION               0.272.0
```

안내서는 릴리스 안에 함께 실려 나갑니다. 그러니 그 버전은 곧 릴리스의 버전이고, 독자는 지금 보는 매뉴얼이 자기 배포를 설명하는지 알 수 있어야 합니다. 두 안내서를 `version-check.sh` 에 넣어 이제 **일곱 곳이 아니라 아홉 곳**을 봅니다. 하나를 되돌려 잡히는 것도 확인했습니다.

3.10.229 v0.274.0에서 완료 — 64 KiB 라고 적어 두고 144 KB 를 보냈습니다

문서 대조를 MCP 로 이어갔습니다. 프로토콜 버전 협상, 구현 메서드, 도구 넷, 키 회전은 모두 문장 그대로입니다 — `weekly_endpoint_analysis` 가 목록에 없어 한 번 의심했는데 **관리자 전용**이라 문서가 맞았고, `notifications/initialized` 는 알림(=id 없음)으로 보내면 **202** 로 올바르게 답합니다.

한 문장만 다릅니다.

```
문서: 업무는 최대 100건이고, 그 위에 64 KiB 상한이 함께 겁니다.
실측: 연간·팀 범위 응답이 144,501 B
      = 텍스트 사본 64,977 + structuredContent 71,471
```

예산이 한 벌만 세고 있었습니다

프로토콜은 같은 답을 두 번 요구합니다 — `structuredContent` 와 텍스트 사본. 코드는 그것을 **알고** 있었고(주석에 그렇게 적혀 있습니다) 사본을 들여쓰지 않는 최적화까지 했는데, `encodedSize` 는 한 벌만 쟀습니다. 부르는 쪽은 예산의 **2.2배**를 받습니다. 예산이 존재하는 이유가 "부르는 쪽에는 문맥 창밖에 없다"인데, 문맥 창은 **도착한 것**에 대해 값을 치릅니다.

재기 시작하니 상한에 닿을 수 없었습니다

전선을 재도록 고쳤더니 업무를 264건에서 **0건까지** 깎고도 92 KB 였습니다. 무게를 뜯어보니:

```
contributors  40,865B (304명) ← 79%
trend          6,088B
highlights     2,386B
나머지 전부     2,000B 남짓
```

기여자 목록에는 아무 상한도 없었습니다. 사람 수만큼 자라고, 줄이는 손잡이가 닿지 않습니다. 그래서 부르는 쪽은 *묻지도 않은 이름 목록*을 지키려고 답 전체를 잃고 있었습니다. 상위 20명으로 자르고 `contributorsTotal` 과 문장을 함께 담았습니다.

```
고침 후  연간 65,311B · 업무 5/264 · 기여자 20/304
        분기 64,014B · 업무 13/264
        월간 65,378B · 업무 16/264
```

실제 사례에서는 잃은 것이 없습니다. 연간은 고치기 전에도 5건이었고, 이제 같은 5건을 **절반의 문맥**으로 받습니다. 기여자 상한이 값을 다 치렀습니다.

있던 시험이 248바이트를 잡았습니다

여유가 2배일 때는 보이지 않던 것이 하나 더 드러났습니다. **노트는 마지막 측정 뒤에 붙습니다.** `mcp_test.go` 의 "the model's whole view" 시험이 65,784 > 65,536 으로 정확히 그것을 짚었습니다. 노트를 클로저로 뽑아 **잴 때마다 최신 노트가 실려 있게** 했습니다.

그리고 100행이 남기를 기대하던 시험은 이제 55행입니다. 행이 아니라 **셈이 바뀐 것**이고, 그렇게 적어 두었습니다.

v0.275.0 — 첫날 한 번 쓰이는 비밀번호를 영원히 두게 했습니다

문서 대조를 배포 문서로 옮겼습니다. 숫자는 전부 맞았습니다 — 미제출 최대 25명, 반복 업무 4주/60%/10%, 중복 70%·유사 50%, 첨부 20장/10MB, 임베딩 6,400건, 요약 최대 10건, `ai.timeout_seconds` 90, `import.retention_days` 365. 한 문장만 남았습니다.

```
ADMIN_GUIDE: "Weekly 컨테이너 프로세스는 3개의 필수 환경변수로 부팅하며"
```

문서는 정확했습니다 — 그것이 문제였습니다

`weekly:v0.274.0` 에 `WEEKLY_POSTGRES_DSN` 하나만 주고 두 번 세워 봤습니다.

```
빈 DB      → 거절: "WEEKLY_BOOTSTRAP_ADMIN, WEEKLY_BOOTSTRAP_ADMIN_PASSWORD 환경변수가 없습니다"
관리자 있는 DB → 거절: 같은 문장
```

두 번째가 틀렸습니다. `bootstrapAdmin` 은 `count(*) FROM users WHERE role='ADMIN' > 0` 이면 **아무것도 하지 않고 돌아옵니다.** 즉 설치 첫날 이후 그 두 값은 읽히고, 검사되고, 버려집니다. 그런데 없으면 뜨

지 않습니다.

그 결과가 운영에 남기는 것: **최초 관리자의 비밀번호가 `.env` 에, Compose 파일에, Kubernetes Secret 에, CI 변수에 영원히 남습니다.** 다시 쓰이지 않는 비밀을 지우면 서비스가 죽으니 지울 수 없습니다. 화면에서 그 비밀번호를 바꾼 뒤에도 파일 속 옛 값은 그대로 남습니다.

검사를 필요한 곳으로 옮겼습니다

요구는 `loadEnvironment()` 에 있었습니다 — 데이터베이스를 열기 **전**입니다. 그래서 "관리자가 있는가"를 물을 수가 없었습니다. 검사를 `bootstrapAdmin` 안, 셈을 이미 한 그 자리로 내렸습니다.

```
WEEKLY_POSTGRES_DSN         항상 필요        (아무것도 대신할 수 없음)
WEEKLY_BOOTSTRAP_ADMIN      관리자가 없을 때만
WEEKLY_BOOTSTRAP_ADMIN_PASSWORD  관리자가 없을 때만 · 주어졌다면 12자 규칙은 그대로
```

거절 문장은 이제 왜 필요한지까지 말합니다: `"관리자 계정이 없는 데이터베이스입니다. … 환경변수가 없으면 아무도 로그인할 수 없습니다."`

컨테이너 셋으로 확인했습니다

```
① 관리자 있는 weeklyimp · DSN 만        → readyz 200 · 기존 admin 로그인 성공
② 빈 weeklymin · DSN 만                  → 거절 "관리자 계정이 없는 데이터베이스입니다…"
③ 빈 weeklymin · 5자 비밀번호            → 거절 "12자 이상이어야 합니다"
```

Compose 가 두 값을 `:?required` 로 걸고 있었습니다

프로세스만 고쳤다면 안내대로 `.env` 에서 두 줄을 지운 운영자는 **컨테이너가 아니라 compose 렌더에서** 막혔을 것입니다. `${…:-}` 로 바꿨습니다. v0.253.0 에서 `WEEKLY_ENCRYPTION_KEY` 에 같은 판단을 한 자리 바로 옆입니다 — 걸 수 있는 곳이 아니라 **설명할 수 있는 곳**에서 막아야 합니다.

문서 다섯 곳(README·ADMIN_GUIDE·DESIGN·OPERATIONS·.env.example)과 Kubernetes Secret 주석이 "첫 기동 이후에는 지우십시오"라고 말하게 했습니다.

있던 시험이 옛 약속을 지키고 있었습니다

`TestARefusedStartUpNamesWhatIsMissingAndWhatToDo` 가 세 변수 모두를 `loadEnvironment` 에서 요구했습니다. 시험을 지운 것이 아니라 **약속이 사는 자리로 옮겼습니다** — DSN 은 그대로 `loadEnvironment`, 나머지 둘은 사용자를 지운 데이터베이스에 대고 `bootstrapAdmin`. 셋 다 여전히 "없는 것만 이름 대기"와 ".env 를 보라"를 지킵니다.

v0.276.0 — CI 가 열아홉 커밋 동안 빨간불이었고 릴리즈는 계속 나갔습니다

v0.275.0 을 밀어 올린 뒤 CI 를 봤습니다.

```
2026-08-29T05:58  guard-check  failure   ← v0.275.0
2026-08-29T05:11  guard-check  failure   ← v0.274.0
...
2026-08-28T18:18  go test      failure   ← 여기서 시작
2026-08-28T17:08  success                ← 마지막 초록불
```

열아홉 커밋입니다. 그동안 릴리즈는 매번 성공했습니다 — 워크플로가 둘이고, `Offline Docker Release` 는 계속 초록불이었고, 자산은 매번 나왔기 때문입니다. 끝난 것처럼 보이는 신호가 하나라도 있으면 나머지는 안 봅니다.

왜 릴리즈 의식이 이것을 놓쳤는가

의식은 `version-check` → 커밋 → 태그 → **자산이 나왔는지** 확인이었습니다. "시험이 통과했는지"는 어디에도 없었습니다. 자산이 나오는 것과 시험이 통과하는 것은 **다른 워크플로**입니다.

`release-check.sh` 에 물음을 더했습니다. 가장 최근 태그의 CI 결과가 실패면 막습니다. **가장 최근 것만** 봅니다 — 밀린 것까지 전부 막으면 열두 번의 릴리즈 동안 빨간불로 남고, 늘 빨간 검사는 읽지 않게 되어 이 검사가 막으려던 바로 그 일이 됩니다. 지난 것은 세어서 보여 주기만 합니다.

무엇이 빨갛게 있었는가 — 셋

```
never executed mcpRollupContributors   (v0.274.0 부터)
never executed migrations              (v0.262.0 무렵부터)
never executed bootstrapAdmin          (v0.275.0 에서 내가 더함)
```

앞의 둘은 **함수가 아닙니다.** `mcpRollupContributors` 는 상수, `migrations` 는… 없습니다. 그런 이름은 이 꾸러미에 **한 번도 존재한 적이 없습니다.** 변수 이름은 `migrationFiles` 입니다.

커버리지는 함수만 보고합니다. 상수와 원장 변수는 백분율을 가질 수 없으니 "never executed" 규칙은 무엇을 하든 유죄였습니다. 그래서 셋 중 둘은 **고칠 수 없는 빨간불**이었고, 고칠 수 없는 빨간불은 읽히지 않는 빨간불입니다.

셈이 안 되는 대상에는 다른 약속을 물었습니다

```
함수       → 그 시험이 실제로 실행했는가 (커버리지)
상수·원장 → 그 시험이 그 이름을 실제로 부르는가 (본문)
```

```
없는 이름 → "이 꾸러미가 선언하지 않습니다" 라고 말합니다
```

세 번째가 `migrations` 를 하루 만에 정확히 짚었을 문장입니다. 옛 문장은 "never executed" 라 **약한 시험**처럼 읽혔지, **틀린 이름**처럼 읽히지 않았습니다.

그리고 그 절반은 커버리지 없이도 물을 수 있습니다

"이 꾸러미가 그 이름을 선언하는가"는 정적입니다. `TestEveryGuardMarkerNamesSomethingThisPackageHas` 를 더했습니다 — `go test ./...` 안에서 1초에 답합니다. guard-check.py 와 **같은 규칙으로** 표시를 읽습니다: 시험 함수 바로 위 주석 덩어리에 있는 것만. `refusalledger_test.go` 의 산문 표시(`guards: every writeError(w, 403, ...)`)는 둘 다 무시합니다.

되돌려 확인했습니다 — `loadEnvironment` 를 `loadEnvironmentt` 로 한 글자 바꾸니 잡습니다.

```
전체 쓸기: 259개 guard 가 모두 이름 댄 것에 닿았습니다.
```

v0.277.0 — 지난주를 먼저 펼치고, 아직 쓰지 않은 사람에게 한 번만 알립니다

이번 판은 매주 사람이 되풀이하던 일을 개인 설정으로 옮겼습니다. 작성자는 지난주 전체를 이번 주 초안으로 미리 펼칠 수 있고, 팀장 이상은 아직 쓰지 않은 담당자에게 권고할 요일을 고릅니다. Keycloak에 이미 로그인한 사람은 Weekly 로그인 화면도 건너뜁니다.

셋 다 "자동"이지만 같은 종류는 아닙니다. 복제는 **한 주에 한 번 데이터를 만드는 일**, 권고는 **권한 범위 안의 사람에게 한 통을 보내는 일**, SSO는 **브라우저를 다른 사이트로 왕복시키는 일**입니다. 하나의 편리함으로 묶되 각각의 실패 경계는 따로 설계했습니다.

자동 복제는 매분 복제가 아닙니다

Worker는 기동할 때 한 번, 그 뒤 매분 돌고 설정을 저장하면 즉시 깨어납니다. 그 말만 놓고 구현하면 사용자가 지운 초안이 다음 분에 다시 생깁니다. `user_weekly_preferences` 에 `auto_clone_processed_week` 을 두고 설정 행을 잠근 뒤, **만들었든 건너뛰었든 그 주는 한 번 처리한 것으로** 기록했습니다.

읽는 것은 정확히 한 주 전 보고서 하나입니다. 더 오래된 "가장 최근 보고서"로 빈 주를 메우지 않습니다. 지난주에 업무 항목이 있을 때 요약과 업무의 구분·실적·계획·이슈·상위 조직 요청·진척도·WorkItem 연결을 새 `DRAFT` 로 옮깁니다. 댓글·승인 이력·첨부·외부 출처는 다른 생애주기이므로 복제하지 않습니다.

이번 주 보고서나 날짜가 겹치는 보고서가 있으면 아무것도 덮어쓰지 않습니다. 자동화를 껐다 켜는 것은 사용자의 새 선택이므로 처리 표식을 다시 열지만, 단순히 자동 초안을 삭제한 것만으로는 되살아나지 않습니다.

"한 주 한 통"의 키는 권고자가 아니라 받는 사람입니다

팀장 둘의 조직 범위는 흔히 겹칩니다. `(requested_by, recipient, week)`를 유일하게 만들면 같은 사람에게 팀장 수만큼 갑니다. 그래서 `team_reminder_deliveries`의 유일성은 **`recipient_user_id + week_start`** 입니다. 먼저 자격을 갖춘 권고가 한 자리를 차지하고 나머지는 조용히 합쳐집니다.

권한은 설정 화면의 표시를 믿지 않습니다.

- `TEAM_LEADER`와 `ORG_MANAGER`는 본인 조직과 전체 하위 조직, `ADMIN`은 전체 활성 사용자를 봅니다.
- 권고자 본인, 비활성 사용자, 범위 밖 사용자, 이미 제출한 사용자는 제외합니다. 보고서가 없거나 `DRAFT`인 사람만 남깁니다.
- 개인 메일 설정 주소를 먼저 쓰고 없으면 계정 이메일을 쓰며, 둘 다 유효하지 않으면 건너뜁니다.
- 큐를 집은 뒤에도 현재 주차, 제출 상태, 권고자의 활성 상태·역할·조직·설정을 다시 확인합니다. 예약 뒤에 사실이 바뀌었으면 보내지 않습니다.

요일은 개인 선택이지만 시각은 서비스 시간대 오전 9시로 하나입니다. 주차 시작일이 수요일이어도 "월요일"은 그 보고 주차 안의 다음 월요일입니다. SMTP가 그 시각에 준비되지 않았으면 과거 주차 backlog를 만들지 않고, 같은 주 안에 정상화됐을 때만 따라잡습니다. 발송·재시도는 기존 제출 메일의 SMTP 계약을 재사용합니다.

기존 Keycloak 세션 확인은 최초 401에서만 합니다

세션이 만료될 때마다 `prompt=none`으로 보내면 작성 중인 화면도 통째로 이동합니다. 그래서 자동 SSO의 출발점은 **앱을 처음 열며 `/me`가 정확히 401을 돌려준 경우** 하나뿐입니다. 5xx, timeout, 네트워크 단절은 "로그아웃"이 아니고, 앱 사용 중 새로 생긴 401도 자동 왕복의 조건이 아닙니다.

브라우저는 `sessionStorage`에 시도 여부와 명시적 로그아웃 표시를 둡니다. 첫 시도를 기록하지 못하거나 저장소가 쓰기를 무시하면 이동하지 않는 쪽으로 닫힙니다. `로그아웃`은 Weekly 세션만 끝내고 현재 탭의 자동 재로그인을 막습니다. Keycloak Realm 로그아웃은 하지 않으므로 수동 SSO 버튼과 새 탭은 남은 사내 세션을 다시 쓸 수 있습니다.

서버는 기존 state·nonce·PKCE 검증을 그대로 쓰고 OIDC 상태 행에 `silent`와 `return_to`만 더했습니다. `prompt=none`은 silent 시작에만 붙습니다. 복귀 경로는 `#/`로 시작하는 2,048자 이하 SPA hash만 받아 외부 URL·역슬래시·CRLF를 버립니다. 정상 공급자 결과는 상태를 한 번만 소비하고 재생을 거부하지만, `code`와 `error`가 모두 없는 깨진 callback은 진짜 응답이 뒤이어 올 수 있으므로 상태를 먼저 지우지 않습니다. Weekly는 이 흐름에서도 공급자의 ID·Access·Refresh Token을 저장하지 않습니다.

검증은 "성공했다"보다 "두 번째에는 무엇이 남는가"를 물었습니다

자동화 통합 시험은 실제 PostgreSQL에 사용자와 조직을 만들고 다섯 경계를 확인합니다.

경계	만든 상태	확인한 결과
설정 권한	일반 사용자·팀장·강등 뒤 복구	일반 사용자는 자동 복제만 저장, 권고 활성화는 403. 강등 중 권고는 꺼져 보이되 저장 선택은 지워지지 않음
요일 계산	수요일 시작 주차의 월요일	그 주차 안 2026-09-07 오전 9시로 계산
자동 복제 멱등성	같은 주 Worker 두 번, 뒤이어 자동 초안 삭제 후 한 번	보고서 한 건만 생성되고 삭제 뒤 0건 유지. 전체 작성 필드와 자동 복제 이력 보존
권고 대상	하위 조직·초안·제출·외부 조직·비활성·주소 없음·겹치는 관리자	하위 조직과 초안만 포함, 제출·외부·비활성·무주소·권고자 제외, 개인 주소 우선, 받는 사람별 한 건
실제 발송	가짜 SMTP 릴레이와 같은 주 재삽입	받는 주소·주차·권고자 문구 확인, `SENT`·시도 1회, 같은 주 두 번째 큐 0건

OIDC 쪽은 프런트 두 시간대와 실제 PostgreSQL callback 시험으로 나눴습니다. 프런트는 401만 무세션으로 보는지, 시도·로그아웃 표식이 반복을 막는지, 저장소 예외와 쓰기 무시가 fail-closed인지, query에서 `oidc_auto`만 지우고 원래 hash를 보존하는지를 확인했습니다. 서버는 silent 시작에만 `prompt=none`이 붙는지, 네 가지 상호작용 요구가 로그인 화면으로 돌아오는지, 예상 밖 공급자 오류는 구분되는지, 상태가 한 번만 소비되는지, 깨진 callback이 상태를 보존하는지, 안전하지 않은 복귀 경로가 거부되는지, 성공 뒤 원래 화면으로 돌아오는지를 확인했습니다.

남은 운영 한계도 숨기지 않습니다. 권고 메일 개별 결과는 아직 관리자 메일 현황과 개인 제출 메일 이력 화면에 합산되지 않아 서버 로그와 큐 표를 함께 봐야 합니다. 로그아웃 억제는 탭 단위라 새 탭에는 적용되지 않습니다. 둘 다 지금의 계약이고, 더 넓혀야 한다면 별도의 제품 선택이 필요합니다.

v0.278.0 — 선택한 팀원의 보고를 내 주간보고 옆에 둡니다

팀장 이상은 이제 `개인 설정 → 팀원 주간보고 자료`에서 담당 팀원을 직접 고릅니다. 선택한 사람의 같은 주차 보고는 내 보고서 화면과 발표·PPTX·제출 메일에 함께 나오며, 아직 쓰지 않은 사람도 사라지지 않고 미작성으로 보입니다.

이 기능에서 가장 중요한 결정은 **포함과 복사를 구분하는 것**이었습니다. 다른 사람의 업무를 내 `report_items`에 넣으면 저장하는 순간 소유자가 바뀌고, 기간 보고와 조직 분석에서 같은 업무를 여러 번 세게 됩니다. 그래서 데이터베이스에는 선택 관계만 두고, 보고서를 읽거나 출력하는 순간 같은 주차 원본을 조합합니다.

선택 관계는 조회 권한이 아닙니다

설정 화면과 저장 API는 다음 범위만 허용합니다.

- `TEAM_LEADER`, `ORG_MANAGER`: 본인 조직과 모든 하위 조직의 활성 사용자
- `ADMIN`: 전체 활성 사용자
- 공통: 본인은 제외하며 최대 500명까지 선택

역할·조직·활성 상태는 선택할 때뿐 아니라 자료를 읽을 때마다 다시 확인합니다. 저장 뒤 팀원이 다른 조직으로 이동하거나 비활성화되거나, 선택한 사람이 일반 사용자로 강등되면 예전 선택 행이 새 권한이 되지 않습니다. 다른 관리자가 보고서를 열 때도 바깥 보고서 조회 범위와 선택 자료 범위의 교집합만 반환해, 관리자 작성 보고서 안의 전사 자료가 팀장에게 새로 열리지 않게 했습니다.

설정 교체는 소유자 사용자 행을 잠근 트랜잭션에서 처리합니다. 두 탭이 동시에 저장해도 반쯤 섞인 집합이 남지 않으며, 요청 인증 뒤 역할이 바뀐 경쟁 상황에서도 잠근 현재 역할·조직으로 검증하고 응답합니다. 존재하지 않음·비활성·본인·범위 밖 ID는 모두 같은 403으로 답해 설정 API가 계정 탐색기가 되지 않게 했습니다.

같은 주차를 한 단계만 조합합니다

선택 자료는 보고서 상태를 거르지 않습니다. `DRAFT`, 제출, 반려, 승인, 확정 상태를 원래 상태와 함께 보여주고, 정확히 같은 `weekStart`의 보고서가 없으면 미작성 자료를 만듭니다. "가장 최근 보고서"로 빈 주를 채우지 않습니다.

선택된 팀장이 자기 보고에 또 다른 사람을 포함했더라도 그 안쪽 선택은 따라가지 않습니다. 한 단계만 읽으므로 순환 선택과 폭발적인 중첩이 없습니다. 선택 변경은 이후 과거 보고 상세와 새 출력에도 적용되지만 이미 내려받은 PPTX와 이미 보낸 메일은 바뀌지 않습니다.

본인 보고서가 아직 만들어지지 않은 현재 주도 별도 읽기 API로 자료를 보여 줍니다. 화면은 팀원 자료를 편집 배열 밖에 렌더링하고 저장 요청에는 본인 업무만 보냅니다. 과거 보고와 팀 보고 상세, 발표 모드도 같은 읽기 전용 구성요소를 사용합니다.

PPTX는 생성 요청 안에서만 팀원 이름·아이디·원래 분류가 붙은 임시 항목으로 펼쳐 기존 템플릿 토큰과 기본 슬라이드 배치에 전달합니다. 제출 메일은 본인 업무 뒤에 별도 구획을 만듭니다. 둘 다 원본 ID와 WorkItem 연결을 복사하거나 DB에 저장하지 않습니다.

데이터와 검증

마이그레이션 027은 `user_report_inclusions(owner_user_id, member_user_id)` 관계와 역방향 삭제용 인덱스만 추가합니다. 설정 변경은 `weekly.report_inclusions` 감사 이벤트로 남습니다. `GET/PUT /api/v1/me/report-inclusions`가 선택과 후보를, `GET /api/v1/reports/current/included-materials`가 보고서가 없는 현재 주의 자료를 제공합니다. 보고 상세에는 항상 배열인 `includedMaterials`가 추가됐습니다.

실제 PostgreSQL 통합 시험은 하위 조직·관리자 전사 범위·본인/비활성/외부 조직 거부, 중복 정규화, 동시 강등, 조직 이동, 조회자 범위 교차, 정확한 주차, 작성 중·미작성, 비재귀와 DB 비복제를 확인합니다. 프런트는 서울과 뉴욕 시간대에서 선택 검색·상한, 읽기 전용 표시, 발표의 작성자·상태·미작성·요청 사항을 검증했습니다. 메일과 PPTX도 본인 자료와 팀원 자료가 섞이지 않는지 별도 시험했습니다.

v0.279.0 — 자동 권고를 기다리지 않고 지금 보냅니다

자동 권고 요일은 반복 업무를 줄이지만, 회의 직전이나 제출 일정이 바뀐 날에는 팀장이 그 요일을 기다릴 수 없습니다. 팀장·조직장·관리자가 `개인 설정 → 팀원 작성 권고 메일`에서 이번 주 미제출자에게 권고 메일을 직접 요청할 수 있게 했습니다. 이 동작은 자동 발송 사용 여부와 요일을 바꾸지 않습니다.

새 발송 경로가 아니라 같은 주의 같은 약속입니다

수동 요청도 기존 자동 권고와 같은 `team_reminder_deliveries`를 사용합니다. 받는 사람과 주차의 유일 제약을 그대로 지키므로 자동과 수동이 겹치거나 여러 권고자가 동시에 눌러도 한 사람에게 한 주 한 통만 갑니다. 마이그레이션 028은 행의 출처만 `AUTO`와 `MANUAL`로 구분합니다.

- 이미 `SENT`이거나 `QUEUED`이면 새 행을 만들지 않고 `alreadyQueued`로 셉니다.
- 자동 대기 행을 수동으로 다시 요청하면 `MANUAL`로 승격하되 현재 작업 임대와 시도 횟수를 보존해 Worker가 같은 메일을 두 번 집지 않게 합니다.
- 이미 수동 요청된 행은 뒤의 권고자가 최초 요청자·주소·임대 정보를 바꾸지 못합니다.
- 최종 `FAILED` 행은 현재 유효한 주소가 있을 때만 시도 횟수를 초기화해 다시 엽니다.
- 자동 설정을 끄거나 요일을 바꿀 때는 `AUTO` 대기 행만 취소하고, 사람이 직접 요청한 `MANUAL` 행은 남깁니다.

주소 선택도 명시적으로 유지했습니다. 개인 메일 설정이 비었을 때만 계정 이메일로 대체하고, 개인 설정에 들어 있는 값 자체가 유효하지 않으면 계정 주소로 조용히 우회하지 않습니다. 화면과 `202` 응답은 대상, 새 큐, 기존 큐·발송, 주소 없음 건수를 나눠 보여 줍니다.

누른 시점과 보낸 시점에 두 번 권한을 확인합니다

`POST /api/v1/me/team-reminders`는 빈 JSON 객체만 받고, SMTP가 준비되지 않았으면 아무 행도 만들지 않은 채 409로 답합니다. 트랜잭션 안에서 요청자의 현재 활성 상태·역할·조직 범위를 다시 읽고, 현재 서비스 주차에서 보고서가 없거나 `DRAFT`인 활성 팀원만 고릅니다.

Worker도 발송 직전에 현재 주차·제출 상태와 요청자의 역할·조직 범위를 다시 확인합니다. 수동 행에는 자동 설정이 필요하지 않지만 자동 행에는 여전히 활성화와 선택 요일이 필요합니다. 자동 스케줄러가 설정을 읽은

직후 사용자가 요일을 바꾸는 경쟁 상황도 막기 위해, 큐 트랜잭션에서 설정 행을 공유 잠금으로 다시 읽고 처음 계산한 요일과 같은지 확인합니다.

감사 이력은 사람이 누른 `mail.team_reminders_manual`과 스케줄러의 `mail.team_reminders_queued`를 구분합니다. 메일 본문도 직접 요청과 예약 발송을 다르게 안내해 받는 사람이 왜 지금 메일을 받았는지 알 수 있습니다.

검증

실제 PostgreSQL과 시험 SMTP에서 역할·하위 조직·관리자 범위, 미작성·`DRAFT`·제출 완료 판정, 자동 행 승격, 실패 재시도, 겹치는 수동 요청의 최초 소유권, 발송 직전 강등·조직 이동·제출, 자동 요일 변경 경쟁을 확인했습니다. 전체 Go 테스트와 `go vet`, 변경 영향 권한 가드 20개가 통과했습니다.

프런트엔드 117개 테스트는 `Asia/Seoul`과 `America/New_York`에서 각각 통과했습니다. 수동 카드의 역할별 노출, SMTP 준비 상태, 확인·중복 클릭 차단, 응답 집계와 오류 안내를 포함하며 린트·프로덕션 빌드·OpenAPI·문서 생성물 검사도 함께 통과했습니다.

새 환경변수는 없습니다. 업그레이드하면 마이그레이션 028이 기존 권고 행을 `AUTO`로 유지한 채 출처 열을 더합니다. 운영자는 기존처럼 PostgreSQL을 먼저 백업하고 SMTP 시험과 `service.timezone`·주차 시작일을 확인하면 됩니다.

v0.280.0 — 주간보고를 입력창 안에서 다시 스크롤하지 않습니다

현재 보고서와 과거 본인 보고서를 수정할 때 주간 요약·실적·계획·이슈·상위 조직 요청 입력창이 실제 내용 높이만큼 자동으로 펼쳐집니다. 입력창마다 작은 스크롤을 움직이지 않고 페이지 흐름에서 전체 내용을 읽고 고칠 수 있습니다.

줄바꿈과 화면 폭을 실제 레이아웃으로 측정합니다

행 개수를 문자열의 개행 문자만으로 세지 않습니다. 같은 문장도 모바일 폭에서는 여러 줄로 감기므로, 기존 높이를 잠시 해제하고 브라우저의 `scrollHeight`를 읽어 필요한 높이를 정합니다. 저장된 값을 처음 불러올 때, 입력하거나 붙여넣을 때, 화면 폭이 달라질 때마다 다시 계산합니다. 내용을 지우거나 폭이 넓어지면 이전 높이에 갇히지 않고 다시 줄어듭니다.

전역 `border-box`에 맞춰 테두리 높이를 보정하고 내부 세로 스크롤과 수동 리사이즈를 끕니다. 폭 관찰 뒤 높이 갱신은 다음 프레임에서 실행해 `ResizeObserver` 알림 루프를 피하며, 관찰자를 지원하지 않는 브라우저는 창 크기 변경 이벤트로 보완합니다.

공통 컴포넌트는 보고서에 저장되는 다섯 종류의 본문 필드에만 적용했습니다. AI 원문, Confluence 후보, 댓글과 의존성 메모처럼 용도와 내용 상한이 다른 보조 입력창의 기존 동작은 바꾸지 않습니다.

검증

단위 시험은 `border-box`와 `content-box` 계산, 늘어난 뒤 다시 줄어드는 경우, 기존 클래스와 접근성 속성 보존을 확인합니다. 프런트엔드 121개 시험을 서울과 뉴욕 시간대에서 각각 실행했고 린트와 프로덕션 빌드를 통과했습니다. 실제 Chrome에서는 명시적 줄바꿈, 자동 줄바꿈, 내용 삭제, 좁은 폭과 넓은 폭 전환 뒤에도 내부 스크롤 없이 모든 줄이 보이며 브라우저 오류가 없음을 확인했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.281.0 — 주 격자가 옮겨진 주에 이미 낸 사람에게 권고하지 않습니다

주차 시작 요일을 바꾸면 격자만 옮겨지고 이미 그 위에 놓인 보고서는 따라오지 않습니다. 전환되는 한 주 동안 기존 격자로 제출한 보고서는 다른 `week_start`로 같은 7일을 덮습니다. `weekIsFree`는 그것을 알고 있어 같은 기간의 두 번째 보고서를 거절하는데, 팀원 작성 권고는 주차 시작일이 정확히 같은 행만 찾고 있었습니다. 그래서 이미 제출을 마친 팀원 전체에게 제품이 허락하지 않는 일을 하라는 메일이 나갔습니다.

한 질문을 두 곳이 나눠 쓰게 했습니다

`weekIsFree`와 기간 집계가 이미 쓰던 겹침 질문을 `weekCoveringDays`로 모으고, 권고 대상을 고르는 `queueTeamReminders`와 발송 직전에 다시 확인하는 `reminderStillWanted`가 둘 다 그것으로 묻게 했습니다. 같은 전환 주를 놓고 한쪽은 "쓸 수 없다", 다른 쪽은 "아직 안 썼으니 쓰라"고 답하던 불일치가 사라집니다. 초안은 제출이 아니므로 여전히 권고를 받고, 요일을 바꾼 적 없는 배포에서는 정확일치와 겹침이 같은 답을 내므로 동작이 달라지지 않습니다.

검증

실제 PostgreSQL에서 회귀 시험 두 개를 더했습니다. 옛 격자로 제출한 팀원이 대기열에 들어가는 것과, 대기 중이던 권고가 발송 직전 검사를 지나 그 사람에게 실제로 나가는 것을 각각 잡으며 둘 다 고치기 전에는 실패합니다.

재검증 시험은 자기가 대기 행을 집었는지를 묻고 있었습니다. 제출은 메일 워커를 깨우므로 워커가 먼저 집으면 같은 시험이 "아무것도 없다"를 읽고, 폐기가 일어나지 않은 것처럼 실패합니다. 지키려던 것은 행이 사라지고 메일이 나가지 않는다는 사실이지 누가 집었는가가 아니므로, 그것을 묻게 고쳤습니다. 제품 동작은 바뀌지 않습니다.

전체 Go 테스트와 `go vet`, 가드 검사 276개, 프런트엔드 121개 시험을 서울과 뉴욕 시간대에서 통과했습니다.

함께 저장소 뿌리의 미들웨어 옛 사본을 지웠습니다. 셸 명령의 일부를 파일 이름으로 달고 있어 빌드에는 잡히지 않았지만, 현재 `middleware.go`에 있는 인증 거절 문구와 추적 ID 기록이 빠진 낡은 코드였습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.282.0 — 주 격자가 옮겨진 주에 작성 화면이 기존 보고서를 엽니다

v0.281.0 이 권고 메일 쪽에서 고친 것과 같은 불일치가 작성 화면에도 남아 있었습니다. 주차 시작 요일을 바꾸면 격자만 옮겨지고 이미 그 위에 놓인 보고서는 따라오지 않으므로, 전환되는 한 주 동안 기존 격자로 쓰던 보고서는 다른 `week_start` 로 같은 7일을 덮습니다. `weekIsFree` 는 그것을 알고 있어 같은 기간의 두 번째 보고서를 거절하는데, 작성 화면이 여는 `GET /api/v1/reports/current` 는 날짜가 정확히 같은 행만 찾았습니다. 그래서 작성자의 보고서는 화면에서 사라지고, 그 자리의 빈 편집기는 저장할 때마다 409 `REPORT_PERIOD_OVERLAPS` 만 돌려줬습니다.

같은 질문을 두 곳이 나눠 쓰게 했습니다

`currentReport` 의 조회를 `weekCoveringDays` (기간 겹침) 로 통일했습니다. "이 기간에 새로 쓸 수 있는가"와 "이 기간에 내 보고서가 있는가"를 이제 한 문장이 답합니다. 화면이 보여 주는 주차는 보고서가 실제로 덮는 기간이므로 작성자는 어느 주를 이어 쓰는지 알 수 있고, 관리자가 요일을 바꿀 때 확인하는 "작성자는 기존 보고서를 열어 이어 써야 합니다"라는 경고문의 그 일을 화면이 대신 합니다. 요일을 바꾼 적 없는 배포에서는 정확일치와 겹침이 같은 답을 내므로 동작이 달라지지 않습니다.

검증

실제 PostgreSQL에서 회귀 시험 하나를 더했습니다. 옛 격자로 쓰던 보고서를 요일을 바꾼 뒤에도 작성 화면이 그대로 여는 것과, 같은 기간의 두 번째 보고서 저장이 여전히 409 로 막히는 것을 함께 잡으며 고치기 전에는 실패합니다.

전체 Go 테스트와 `go vet` , 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.283.0 — 주 격자가 옮겨진 주에 팀원 자료를 미작성으로 보지 않습니다

v0.281.0 이 권고 메일에서, v0.282.0 이 작성 화면에서 고친 것과 같은 불일치가 팀장의 발표 자료에도 남아 있었습니다. 주차 시작 요일을 바꾸면 격자만 옮겨지고 이미 그 위에 놓인 보고서는 따라오지 않으므로, 전환되는 한 주 동안 기존 격자로 쓴 팀원의 보고서는 다른 `week_start` 로 같은 7일을 덮습니다. `includedMaterialsFor` 는 그 보고서를 날짜 정확일치로만 붙이고 있었고, 그래서 이미 쓴(그리고 `weekIsFree` 때문에 새 날짜로 다시 쓸 수도 없는) 팀원이 자료 없이 돌아왔습니다. PPTX 는 그것을 이름 옆의 "주간보고 미작성 / 해당 주차 주간보고가 없습니다" 로 바꿔 경영 보고 자리에 올립니다.

세 번째 자리도 같은 질문을 쓰게 했습니다

팀원 보고서 조회를 `weekCoveringDays` 기준의 LATERAL 조인으로 바꿔, 겹치는 것 중 `week_start` 가 가장 늦은 하나를 붙입니다. `currentReport` 와 `weekIsFree` 가 하는 선택과 같으므로 세 곳이 이제 한 문장을 나눠 씁니다. 작성 화면의 `currentIncludedMaterials` 와 저장된 보고서를 여는 `loadReport` 가 같은 함수를 지나므로 두 경로가 함께 고쳐지고, 요일을 바꾼 적 없는 배포에서는 정확일치와 겹침이 같은 답을 내므로 동작이 달라지지 않습니다.

검증

실제 PostgreSQL에서 회귀 시험 하나를 더했습니다. 옛 격자로 쓴 팀원의 보고서가 요일을 바꾼 뒤에도 팀장의 자료에 요약과 업무까지 붙는 것과, 팀원이 쓰지 않은 주가 다른 주의 보고서로 채워지지 않는 것을 함께 잡으며 고치기 전에는 실패합니다.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.284.0 — 주 격자를 옮긴 다음 주에도 지난주 보고서를 자동 복제합니다

지난주 전체 자동 복제는 한 함수 안에서 두 가지를 묻습니다. 이번 주에 이미 보고서가 있는가, 그리고 지난주에 복제할 원본이 있는가. 앞의 질문은 이미 날짜 겹침으로 물어 주차 시작 요일이 바뀐 주를 견디고 있었는데 뒤의 질문은 `week_start` 정확일치로 남아 있었고, 그래서 요일을 바꾼 다음 주에는 작성자가 옛 격자 위에서 이어 쓰던 전환 주 보고서를 원본으로 찾지 못했습니다. `runAutomaticCloneForUser` 는 그 주를 `auto_clone_processed_week` 에 처리 완료로 적은 뒤 초안 없이 끝났고, 매주 오던 초안이 한 번 조용히 오지 않았습니다.

같은 함수의 두 반쪽이 같은 규칙을 쓰게 했습니다

원본 조회를 `weekCoveringDays` 기준의 기간 겹침에 `ORDER BY week_start DESC LIMIT 1` 로 바꿨습니다. 겹치는 것 중 가장 늦은 하나는 작성자가 마지막으로 쓰고 있던 그 보고서이고, 이것이 그들에게 "지난주"의 뜻입니다. v0.281.0 이 권고 메일에서, v0.282.0 이 작성 화면에서, v0.283.0 이 팀장의 발표 자료에서 고친 것과 같은 불일치였고, 자동 복제는 절반만 고쳐져 있던 네 번째 자리였습니다. 전환 주 자체에는 기간이 겹치는 보고서가 이미 있으므로 종전처럼 복제하지 않으며, 요일을 바꾼 적 없는 배포에서는 정확일치와 겹침이 같은 답을 내므로 동작이 달라지지 않습니다.

검증

실제 PostgreSQL에서 회귀 시험 하나를 더했습니다. 요일을 바꾼 전환 주에는 복제가 일어나지 않는 것과 그 다음 주에는 전환 주 보고서를 원본으로 초안이 만들어지는 것을 함께 잡으며, 고치기 전에는 실패합니다.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.285.0 — 주 격자를 옮긴 주에 지금 쓰는 보고서를 지난주 계획으로 돌려주지 않습니다

작성 화면 옆 "지난주 계획" 패널은 `week_start < $2`, 즉 날짜가 더 이른 가장 최근 보고서를 물었습니다. 주차 시작 요일을 바꾼 전환 주에는 작성자가 이어 쓰고 있는 그 보고서가 옛 격자의 이른 날짜를 달고 있으므로, 패널은 눈앞의 보고서를 지난주 계획으로 되돌려 주었습니다. 업무 항목이 자기 자신과 짝지어지고, 이미 100%로 보고한 것까지 아직 남은 일로 다시 제안되었으며, 한 주 전에 세운 계획은 한 줄 아래에서 닿지 않았습니다.

"이전"을 끝난 7일로 묻습니다

`weekCoveringDays`의 나머지 반쪽인 `weekEndedBefore` 를 `weekstart.go` 에 더하고 `previousWeekPlan` 이 그것을 쓰게 했습니다. "이전"은 날짜가 이른 것이 아니라 일곱 날이 이번 주가 시작하기 전에 끝난 것입니다. 전환 주의 열려 있는 보고서는 이번 주를 덮으므로 더는 이전에 들지 않습니다. v0.281.0 이 권고 메일에서, v0.282.0 이 작성 화면에서, v0.283.0 이 팀장의 발표 자료에서, v0.284.0 이 자동 복제에서 "이번 주"를 겹침으로 묻게 한 데 이어, 그 반대편인 "이전 주"를 같은 규칙으로 맞춘 자리입니다. 정확히 한 주 전이 아니라 그 앞의 보고서를 고르는 동작은 그대로여서 휴가에서 돌아온 사람은 여전히 두고 간 계획을 보며, 요일을 바꾼 적 없는 배포에서는 두 조건이 같은 답을 냅니다.

검증

실제 PostgreSQL에서 회귀 시험 하나를 더했습니다. 전환 주에 이 끝점이 열려 있는 보고서가 아니라 그 앞 주의 계획을 돌려주고 끝내지 못한 업무를 이어지는 일로 표시하는지를 함께 잡으며, 고치기 전에는 실패합니다. 이 끝점에는 그동안 통합 시험이 하나도 없었습니다.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.286.0 — 보내는 이름에 쉼표나 꺾쇠가 있어도 From 헤더가 깨지지 않습니다

메일의 보내는 이름은 인코딩된 낱말(encoded-word)로 감싸고 있었지만, `mime.BEncoding.Encode` 는 순수 ASCII 를 그대로 돌려줍니다. 옳은 동작이며 그래서 영문 이름은 관리자가 설정 칸에 적은 그대로 헤더에 실렸고, RFC 5322 는 거기 있는 쉼표를 한 주소의 끝으로 읽습니다. `Weekly, Inc.` 는 보낸사람이 `Weekly` 한 명인 주소 목록이 되고, 꺾쇠가 든 이름은 자기 angle-addr 를 스스로 달아 진짜 보내는 주소를

두 번째 주소로 남겼습니다. 이름을 한글로만 쓰던 배포에서는 전부 base64 안에 들어가 보이지 않던 결함입니다.

이름은 이름으로만 읽히게 씁니다

`mailDisplayName` 을 더해, 인코딩이 필요한 이름은 지금처럼 인코딩된 낱말로 감싸고 그렇지 않으면 RFC 5322 가 특수문자로 세는 글자가 든 이름만 따옴표 문자열(`\` · `"` 이스케이프)로 씁니다. 글자와 숫자와 공백뿐인 이름은 그대로 적으므로 실제로 설정된 이름들은 보이는 모습이 달라지지 않고, 한글 이름은 종전과 같은 인코딩된 낱말로 나갑니다. 빈칸만 남은 설정 칸은 지운 것과 같게 다뤄 주소만 씁니다. 보고서 제출 메일, 팀원 작성 권고 메일, 관리자 시험 메일이 모두 이 한 함수를 지납니다.

검증

회귀 시험 2개를 더했습니다. 이름 8종으로 만든 메시지를 실제로 `net/mail` 로 다시 읽어 보낸사람이 한 명인지·주소가 그대로인지·이름이 그대로 도착하는지를 함께 보며, 고치기 전에는 4가지 이름에서 실패합니다.

전체 Go 테스트와 `go vet` , 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.287.0 — 주 격자를 옮긴 주에도 분석 화면이 그 주의 보고서를 셉니다

주차 시작 요일을 바꾸면 주 격자만 옮겨지고 이미 저장된 보고서는 그 자리에 남습니다. 전환되는 한 주 동안 팀이 써 둔 보고서는 같은 7일을 다른 날짜로 덮고 `weekIsFree` 는 같은 기간의 두 번째 보고서를 막는데, 분석 화면은 그 주의 숫자를 `week_start` 정확일치로 물었습니다. 그래서 모두가 보고한 한 주가 제출률 0%, 상태별 분포 빈칸, 미해결 이슈 0건, 평균 진행률 0 으로 보였습니다.

네 숫자가 같은 보고서를 셉니다

제출률·상태별 분포와 미해결 이슈·평균 진행률 질의를 기간 겹침(`weekCoveringDays`)으로 바꾸고 사람마다 한 건만 세도록 묶었습니다. 어떤 7일을 덮는 보고서는 많아야 하나이므로 이 선택은 `weekIsFree` 가 생기기 전에 쓰인 행 사이에서만 갈리며, 그때는 `currentReport` 와 같이 가장 늦은 `week_start` 를 고릅니다. 같은 함수를 MCP 주간 요약도 읽으므로, 화면 없이 숫자만 받는 AI 클라이언트가 0% 를 그대로 인용하던 자리도 함께 닫힙니다. v0.282.0 의 작성 화면, v0.283.0 의 팀원 자료, v0.284.0 의 자동 복제, v0.285.0 의 지난주 계획과 같은 불일치의 여섯 번째 자리입니다.

검증

회귀 시험 1개를 더했습니다. 요일을 바꾼 직후 전환 주에 팀원이 옛 격자로 제출해 둔 보고서를 두고 조직장이 분석 화면을 열어, 제출 인원·제출률·미해결 이슈·평균 진행률이 모두 그 보고서를 가리키는지 봅니다. 고치

기 전에는 네 숫자 모두에서 실패합니다.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.288.0 — 제출 메일이 발표자료를 함께 보내고, 월요일 전에 직접 받아 볼 수 있습니다

주간보고 제출 메일은 본문만 실어 보냈습니다. 회의에 들어가는 것은 PPTX인데 그 파일은 첨부되지 않았으므로, 메일을 받은 사람은 결국 화면으로 돌아와 내려받아야 했습니다. 메일을 아예 보내지 않는 것과 크게 다르지 않은 상태였고, 메일을 만드는 `buildMailMessage` 에는 본문 한 조각짜리 모양 하나뿐이어서 첨부를 넣을 자리가 없었습니다.

같은 파일을 붙입니다

내보내기 처리기 안에 있던 렌더링을 `reportDeck` 으로 꺼내고 메일 작업자가 그것을 부릅니다. 조직 템플릿, 이슈 슬라이드, 캡처 슬라이드가 모두 같은 경로를 지나므로 화면의 `PPTX 내보내기` 가 주는 것과 **바이트 단위로 같은 파일**이 첨부됩니다. 한 보고서를 두 번 내보내면 같은 파일이 나온다는 v0.246.0 의 성질 덕분에 이것은 시험으로 확인할 수 있는 주장이며, 회귀 시험이 두 파일의 SHA-256 을 비교합니다.

첨부가 있는 메일만 `multipart/mixed` 가 되고, 첨부가 없는 권고 메일과 관리자 시험 메일은 예전 그대로 단일 `text/plain` 입니다. 모든 메일에 빈 클립이 붙으면 아무도 클립을 보지 않게 됩니다. 파일명은 RFC 2231(`filename*=UTF-8''…`)과 ASCII 대체 이름을 함께 적습니다 — 이름이 한글이고 세미콜론이 들어갈 수 있어서, `url.PathEscape` 로는 `;` `=` `,` 가 그대로 남아 매개변수가 중간에서 끊깁니다.

첨부 상한은 10MiB 입니다. 넘거나 렌더링에 실패하면 **메일은 그대로 보내고** 본문 첫 줄에 그 사실과 이유를 적습니다. 한 주의 보고가 첨부 때문에 통째로 사라지는 것보다 낫고, 첨부가 조용히 빠지는 것은 받는 사람이 알 수 없기 때문입니다.

설정한 자리에서 확인합니다

관리자 시험 메일은 릴레이가 메시지를 받는지만 말합니다. 작성자가 알고 싶은 것 — 내가 적은 주소로 오는가, 본문이 생각한 대로인가, 첨부가 오는가 — 은 다음 제출 때 한 번 일어나고, 잘못 설정한 사람은 일주일 뒤 "아무것도 오지 않았다" 로 알게 됩니다. 그것은 릴레이가 죽은 것과 구별되지 않습니다.

`개인 설정 → 주간보고 메일 발송` 에 `시험 발송` 을 두었습니다. 저장된 주소로 가장 최근 주간보고를 지금 한 통 보내며, 제목의 `[시험]` 말고는 제출 때 나가는 본문·첨부와 같습니다. 받는 사람은 언제나 요청한 본인입니다 — 요청 본문으로 주소를 정할 수 있는 시험은 이 서버를 남의 메일함으로 가는 통로로 만듭니다. 시험은 발송 이력에 남지 않고(이력은 실제 제출만 셉니다), 한 사람당 30초에 한 번입니다. 한 번 누를 때마다 발표자

료를 만들고 릴레이 제한시간만큼 요청을 붙잡으므로, 월요일 아침 300명이 함께 쓰는 프로세스에서는 이것이 예의가 아니라 용량 문제입니다.

검증

회귀 시험 7개를 더했습니다. 실제 SMTP 릴레이가 받은 바이트를 `net/mail` 과 `mime/multipart` 로 다시 읽어, 클라이언트가 파일을 찾을 수 있는지를 구조로 확인합니다 — 첨부가 zip 으로 열리고 슬라이드가 들어 있는지, 내려받은 파일과 해시가 같은지, 한글·세미콜론·따옴표가 든 이름이 그대로 도착하는지, 첨부가 없을 때 메시지 모양이 그대로인지, 시험 발송이 저장한 주소로만 가고 이력을 만들지 않는지, 두 번째 요청이 429 인지, 한 사람의 대기가 다른 사람을 막지 않는지.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 121개 시험·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API 는 `POST /api/v1/me/mail/test` 하나가 늘었습니다. 데이터 모델과 환경변수 변경은 없습니다.

v0.289.0 — 주 격자를 옮긴 주에도 회의 안건과 변화 요약이 그 주의 보고서를 읽습니다

주차 시작 요일을 바꾸면 주 격자만 옮겨지고 이미 저장된 보고서는 그 자리에 남습니다. 전환되는 한 주 동안 팀이 써 둔 보고서는 같은 7일을 다른 날짜로 덮는데, 회의 모드와 주간 변화 요약은 스냅샷을 `week_start` 정확일치로 골랐습니다. 그래서 전환 주의 회의 자료가 통째로 비었고 — 업무 0건, 인원 0명, 이슈 0건, 결정 요청 0건 — 그다음 주에는 몇 달째 이어지던 업무가 `신규` 나 `재개` 로 올라왔습니다.

두 화면이 같은 7일을 읽습니다

스냅샷 조회를 기간 겹침으로 바꿨습니다. SQL 쪽의 `weekCoveringDays` · `weekEndedBefore` 와 같은 규칙을 Go 안에서 묻는 `snapshotFor` · `snapshotPrior` 이며, 두 화면이 판정을 공유하므로 (`classifyWeeklyChange`) 한 자리를 고치면 둘이 함께 맞습니다. 비교 대상은 이번 7일 바로 앞의 7일로 한정합니다 — 여기를 "더 이른 것 중 가장 늦은 것" 으로 열면 다섯 주 전에 마지막으로 보고된 업무가 `재개` 가 아니라 평범한 `진척` 이 되고, 그것은 이 화면이 있는 이유를 지웁니다. `신규` 판정도 업무의 첫 주차를 요청한 날짜가 아니라 찾아낸 스냅샷의 날짜와 견줍니다. v0.282.0 의 작성 화면, v0.283.0 의 팀원 자료, v0.284.0 의 자동 복제, v0.285.0 의 지난주 계획, v0.287.0 의 분석 화면과 같은 불일치의 일곱 번째 자리입니다.

검증

회귀 시험 2개를 더했습니다. 전환 주의 회의 안건이 그 주의 이슈와 진척을 담는지, 그리고 판정 여덟 가지가 각각 맞는지 봅니다. 고치기 전에는 전환 주 두 가지와 그다음 주 두 가지가 `ABSENT` · `ABSENT` · `RESUMED` · `ABSENT` 로 실패하고, 다섯 주 전 두 가지는 창을 한 주로 묶어 두었다는 것을, 겹치는 두 가지는 같은 7일에 두 보고서가 있을 때 늦은 것을 고른다는 것을 지킵니다. 판정의 모든 분기를

이미 걸어 보던 기존 시험에는 가드 표시가 없어 변이 검사가 세지 않았으므로 표시를 달고, 정체 판정의 경계를 짚는 사례를 하나 더했습니다.

전체 Go 테스트와 `go vet`, `gofmt`, 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.290.0 — 주 격자를 옮겨도 참여 분석이 이미 낸 보고서를 셉니다

참여 분석의 두 반쪽이 모두 `week_start` 정확일치였습니다. 제출률 추이는 보고서를 자기 `week_start` 로 묶고 화면은 그 행을 격자에서 찾으므로, 주차 시작 요일을 바꾼 뒤에는 그 전에 쓰인 어떤 보고서도 어느 행에도 실리지 않습니다 — 전환 주 하나가 아니라 창에 담긴 12주 전부가, 매주 보고한 팀에게 제출률 0% 로 보였습니다. 미제출자 명단은 같은 질문을 뒤집은 것(`NOT EXISTS ... week_start=week.day`)이라 같은 답을 냈습니다: 전원이, 자기가 보고한 모든 주에 대해 밀린 사람으로 이름이 오르고 그 명단은 독촉의 근거입니다. 전환 주는 `weekIsFree` 가 두 번째 보고서를 막으니 다시 써서 지울 수도 없는 미제출이었습니다.

격자가 세고, 한 사람은 한 번만 세어집니다

추세를 `generate_series` 격자에서 만들고 각 행이 그 7일을 덮는 보고서를 세도록 `weekCoveringDaysOf`(placeholder 대신 날짜 식을 받는 `weekCoveringDays` 의 형제)로 바꿨습니다. 옮긴 격자의 한 주는 옛 격자의 두 주와 겹치므로 사람마다 `DISTINCT ON` 으로 한 건만 고릅니다 — 그러지 않으면 매주 보고한 사람이 한 행에서 두 번 세어져 제출률이 100% 를 넘습니다. 동점은 실제로 제출된 것을, 그다음 늦은 주를 고르며 `snapshotFor` 가 Go 쪽에서 쓰는 규칙과 같습니다. `weekIsOwed` 의 `NOT EXISTS` 도 같은 겹침으로 바꿔 미제출 판정이 `weekIsFree` 의 거절과 같은 말을 합니다. 정시·지각은 보고서 자신의 주차 마감으로 재는데, 그것이 작성자가 실제로 지켜야 했던 마감이기 때문입니다. v0.281.0 의 권고 메일, v0.282.0 의 작성 화면, v0.283.0 의 팀원 자료, v0.284.0 의 자동 복제, v0.285.0 의 지난주 계획, v0.287.0 의 분석 화면, v0.289.0 의 회의 안건과 변화 요약과 같은 불일치의 여덟 번째 자리입니다.

검증

회귀 시험 1개를 더했습니다. 세 주의 제출 건수와 미제출자 명단, 미제출자 수를 보며 고치기 전에는 다섯 곳이 실패합니다. 넓히는 것이 곧 비우는 것이 되지 않았는지도 같이 봅니다 — 같은 날부터 한 건도 내지 않은 사람이 한 명 있고, 그 사람의 밀린 주 수가 응답이 스스로 마감되었다고 말한 주 수와 다르면 시험이 실패합니다.

전체 Go 테스트와 `go vet`, `gofmt`, 가드 검사, 프런트엔드 121개 시험·린트·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다.

v0.291.0 — 부서의 한 달을 한 화면에: 체크박스가 있는 월간 업무 상황판과 ITSM 연동

주간보고는 지나간 주를 기록합니다. 일정이 있는 부서는 그 전에 다가올 달을 계획하고, 그 계획은 지금까지 이 제품 밖 어딘가의 스프레드시트에 있었습니다. `work_items.due_date` 로는 대신할 수 없습니다 — 그것은 보고서에서 파생된 업무 하나에 붙는 마감 하나이고, 부서의 달력은 한 업무 아래 여러 날짜, 아직 아무도 보고하지 않은 일, 팀장이 먼저 깔아 둔 일로 이루어집니다.

상황판

`schedule_tasks` 한 표를 더했습니다. 담당자와 등록자를 나눠 팀장이 팀원의 달을 짤 수 있게 하고, 완료는 boolean 이 아니라 `done_at` · `done_by` 입니다 — 상황판은 "언제 누가 했는가"를 곧바로 되묻는 화면이고 boolean 은 둘 다 답하지 못합니다. 시작일과 종료일이 있어 `9/7~9/11 감사 준비` 가 걸친 다섯 칸에 모두 나오고, 조회가 기간 겹침이므로 달을 넘어가는 일은 양쪽 달에 나옵니다.

보기는 넷입니다 — 월 격자, 주, 목록, 담당자별 레인. 같은 한 달을 서로 다른 질문으로 읽기 때문입니다. 전체화면은 사무실 벽 화면용이고, 그때만 60초마다 스스로 다시 읽습니다. 걸려 있는 화면이 오전의 계획인 채로 남지 않게 하기 위해서입니다.

중요도는 네 단계이고 색이 본체입니다: 긴급 빨강, 중요 초록, 필요 파랑, 일반 검정. 색은 줄 왼쪽 테두리에 두고 글자를 함께 적었습니다 — 세 걸음 떨어져 읽는 화면에서 색이 유일한 신호가 되어서는 안 됩니다. 하루 안에서는 긴급이 먼저 옵니다.

보는 것과 쓰는 것을 나눴습니다

업무 추적의 범위 규칙을 그대로 쓰지 않았습니다. 그것은 "누구의 보고 이력을 읽을 수 있는가" 이고 팀원은 본인 것만 읽습니다. 상황판은 공유 계획이므로 팀원도 본인 조직과 하위 조직 전체를 읽습니다. 대신 쓰기는 종전 그대로이고(담당자 본인·등록자·상위 조직장·관리자), 서버가 행마다 `canEdit` 을 계산해 내려줍니다. 거부당할 체크박스를 그리지 않기 위해서입니다.

ITSM 연동

부서가 계획하는 일은 대개 SR 로 들어옵니다. 번호를 입력하면 제목을 가져와 그대로 상황판에 올립니다. 주소는 **두 개**입니다 — 기계가 제목을 묻는 API 와 사람이 여는 포털은 이 회사들에서 대개 서로 다른 시스템이고, 하나로 가정하면 상황판의 모든 링크가 브라우저에서 JSON 을 여는 주소가 됩니다. `{id}` 자리표가 있으면 그 자리에, 없으면 `?sr_id=…` 로 붙습니다(파라미터 이름도 설정). 제목은 JSON 경로로 찾고, API 가 없고 HTML 만 있는 시스템을 위해 정규식도 받습니다.

저장하는 것은 **번호뿐**입니다. 링크는 읽을 때마다 현재 설정으로 다시 만들기 때문에, 포털이 이사하면 설정 한 줄로 기존 일정의 링크가 모두 따라갑니다. 조회는 서버가 대신합니다 — 사내 ITSM 에 브라우저가 닿지

못하는 망이 흔하고, 인증 토큰을 브라우저로 내려보내지 않기 위해서입니다. 번호는 주소에 들어가기 전에 형식 정규식으로 검사하고 인코딩하며, 주소는 http·https 만, 응답은 1MB 까지만 읽습니다.

검증

Go 회귀 시험 7개와 프런트엔드 격자 시험 13개를 더했습니다. 체크박스가 누가 언제를 남기고 해제하면 지우는지, 달을 넘는 일이 양쪽 달에 나오고 그다음 달에는 안 나오는지, 팀원이 부서 판을 읽지만 남의 줄은 체크하지 못하는지, 팀장이 조직 밖 사람에게 일을 배정하지 못하는지, SR 조회가 실제 HTTP 서버에서 제목을 가져오고 링크는 포털을 가리키는지, `../../etc/passwd` 같은 번호가 주소에 닿지 못하는지, 컴파일되지 않는 정규식이 저장 시점에 거부되는지를 봅니다.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 134개 시험·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API 는 `/api/v1/schedule` 다섯 개와 `/api/v1/itsm/lookup`, 관리자 시험 하나가 늘었습니다. 마이그레이션 `029_schedule_tasks.sql` 이 추가되고, 관리자 설정에 `itsm.*` 열한 개가 생깁니다. 기존 데이터와 화면은 그대로입니다.

v0.292.0 — 한 번도 저장된 적 없는 설정은 화면에 없었습니다

v0.291.0 은 ITSM 연동 설정 열한 개를 코드에 정의하고 README 와 두 안내서에 적었습니다. 관리자 화면에는 없었습니다.

설정 목록 API 가 `app_settings` 표의 **행**을 돌려주고 화면이 받은 행마다 상자를 그리기 때문입니다. 한 번도 저장된 적 없는 키는 행이 없고, 행이 없으면 상자가 없습니다. 그래서 "관리자 설정에서 켜십시오" 라는 안내를 따라간 운영자 앞에는 켤 것이 없었습니다. API 로는 넣을 수 있었지만 그것은 답이 아닙니다.

정의가 목록입니다

이제 `adminSettings` 는 코드가 아는 설정 전부를 돌려주고 저장된 행이 그 값을 채웁니다. 표에는 있는데 코드에서 사라진 키도 함께 돌려줍니다 — 누군가의 데이터이고, 감추는 것은 같은 실수를 반대로 하는 일입니다.

같은 검사를 화면 쪽에도 두었습니다. API 가 주더라도 어떤 묶음에도 들어 있지 않으면 여전히 보이지 않기 때문입니다. 그 검사가 **이미 보이지 않던 설정 다섯 개**를 찾아냈습니다 — 기간 보고의 병합 유사도·정체 주차·이슈 지속 주차·최대 주차 수와 감사 로그 보관일. 정의되어 있고 검증되며 어디에서도 입력할 수 없던 값들입니다. `기간 업무보고 집계` 묶음을 만들고 감사 보관일은 `보안 · 분석` 에 넣었습니다.

상황판 화면 두 가지

일정 추가 창이 흰 상자 가장자리에 붙어 있었습니다. 제품의 다른 대화상자와 같은 모양 — 머리글, 여백 있는 폼, 바닥글 — 으로 바꾸고, ITSM 조회는 성격이 다른 동작이므로 따로 상자에 담았습니다. 중요도 옆에는 고르는 동안 색이 보이도록 점을 두었습니다.

사이드바 메뉴에는 스크롤이 없었습니다. 관리자는 묶음 셋에 항목 열넷을 보는데, 창이 목록보다 짧으면 맨 아래가 잘린 채 닿을 방법이 없었습니다. 메뉴가 남은 공간을 차지하고 그 안에서 스크롤하며, 스크롤 막대는 사이드바의 색으로 그려 패널의 일부로 보이게 했습니다. 브랜드와 바닥글은 제자리에 있습니다.

검증

회귀 시험 2개를 더했습니다. 새 설치의 설정 API 가 정의된 키를 하나도 빠뜨리지 않는지(비밀 여부까지), 그리고 관리자 화면 파일이 그 키를 모두 담고 있는지를 봅니다. 뒤쪽 시험은 처음 실행에서 위의 다섯 개를 찾아냈습니다.

실제 컨테이너에서 빈 데이터베이스로 기동해 관리자 화면에 ITSM 카드와 `ITSM 연동 사용` 토글, 주소 두 칸이 그려지는 것을 확인했고, 660px 높이 창에서 사이드바 맨 아래 항목에 스크롤로 닿는 것과 900px 에서는 스크롤이 생기지 않는 것을 확인했습니다.

전체 Go 테스트와 `go vet`, 가드 검사, 프런트엔드 134개 시험·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API 응답 한 곳이 넓어집니다 — `GET /api/v1/admin/settings` 가 저장 여부와 무관하게 정의된 설정을 모두 돌려줍니다. 데이터 모델과 환경변수 변경은 없습니다.

v0.293.0 — 상황판이 마감 닷새 전부터 담당자에게 말을 겁니다

상황판은 부서가 이번 달에 무엇을 하는지 보여 줍니다. 그 앞에 매일 서 있는 사람은 없고, 삼 주 전에 적어 둔 줄은 그 뒤로 아무도 생각하지 않은 줄입니다. 놓치는 것은 정확히 그 줄입니다.

하루 한 통, 남은 일수별로

담당자가 켜 두면 서비스 시간대 오전 9시 이후에 자기 몫의 미완료 일정 중 마감이 닷새 안으로 다가온 것을 묶어 받습니다. 날짜순 나열이 아니라 `[D-1 · 2026-10-06 마감] 2건` 처럼 **남은 일수로 묶습니다** — 사람이 움직이는 문장은 "이틀 남았습니다" 이고, 날짜는 읽는 사람이 계산해야 하는 값입니다. 메일에는 색이 없으므로 중요도는 글자로 씁니다.

두 가지를 일부러 넣지 않았고, 둘 다 말해 둘 값어치가 있습니다. **시작일은 보지 않습니다** — 아직 시작하지 않았는데 금요일이 마감인 일이야말로 이 알림이 있는 이유이고, 시작일로 거르면 그것이 가장 먼저 사라집니다. **오늘 마감과 지연은 넣지 않습니다** — 경고는 닥치기 전에 오는 것이고, 상황판 자신이 이미 빨간색으로 보여 줍니다.

기본은 꺼짐, 그리고 보낼 것이 없으면 보내지 않음

`user_mail_settings` 에 스위치 하나를 더했습니다. 주간보고 메일을 위해 저장해 둔 주소가 이 알림에 대한 동의는 아니기 때문에 기본은 꺼짐이고, 주소 없이 켜는 것은 거절합니다 — 켜져 있는데 아무것도 오지 않는 상태가 가장 나쁩니다.

큐(`schedule_reminder_deliveries`)의 `UNIQUE(user_id, reminder_on)` 가 하루 한 통을 보장합니다. Worker 는 매분 돌고 서비스는 재시작·복제되므로, "오늘 것을 보낸다" 는 이것이 없으면 "오늘 것을 또 보낸다" 가 됩니다. 발송 직전에 수신 여부·주소·계정·남은 일감을 다시 읽어, 그사이 알림을 끄거나 일을 다 체크했다면 행을 지우고 아무것도 보내지 않습니다.

검증

회귀 시험 4개를 더했습니다. 창 안팎의 여덟 가지 경우(D-1·D-2·D-5 안, D-6·오늘·지연·완료·남의 일 밖)를 한 사람의 상황판에 깔고 실제 SMTP 릴레이가 받은 메일에서 묶음 순서와 건수·중요도·분류를 읽습니다. 매분 세 번 돌려도 한 통인지, 다음 날은 새로 한 통인지, 켜지 않은 사람에게는 큐조차 생기지 않는지, 보낼 것이 사라진 다이제스트가 발송 대신 삭제되는지도 봅니다.

전체 Go 테스트와 `go vet` , 가드 검사, 프런트엔드 134개 시험·프로덕션 빌드, 버전·OpenAPI·모달 검사를 통과했습니다.

API 는 `/api/v1/me/mail` 의 `scheduleReminder` 한 칸이 늘었습니다. 마이그레이션 `030_schedule_reminders.sql` 이 추가되고 새 환경변수는 없습니다.

v0.294.0 — 읽지 못한 상황판을 "이번 달 계획 없음" 으로 그리지 않습니다

업무 상황판의 네 보기가 `board` 없이도 그려졌습니다. 첫 읽기가 실패하면 벽에는 42칸짜리 빈 달력이 걸리고, 목록·담당자 보기는 "이 기간에 등록된 업무 일정이 없습니다", 주 보기는 일곱 칸 모두 "일정 없음" 이라고 말합니다. 이 판은 사무실 벽에 거는 것이므로 지나가며 보는 사람에게 그것은 부서가 이번 달에 아무것도 계획하지 않았다는 문장이고, 벽에 걸린 것을 의심할 자리는 없습니다.

버리는 실패와 지키는 실패

더 나쁜 것은 창을 옮기다 실패했을 때입니다. 실패한 읽기가 앞의 판을 그대로 두므로, 다음 달로 넘어가다 실패하면 지난달 줄이 이번 달 제목 아래 남고 `부서 전체` 를 끄다 실패하면 남의 줄이 본인 것으로 남습니다. 오래된 것이 아니라 화면이 묻지 않은 질문에 답하는 것입니다.

`scheduleGrid.ts` 의 `boardAfterFailure` 가 그 둘을 가릅니다. 창(`from` · `to` · `scope`)이 다른 판은 버리고, 같은 창이면 벽에 걸린 것을 지킵니다 — 전체화면은 1분마다 스스로 다시 읽으므로 한 번 실패했다고 벽을 비우는 것은 1분 된 판보다 나쁩니다. 네 보기는 읽힌 판에만 그립니다.

검사 목록에서 빠져 있던 화면

`failstate-check.py` 의 화면 목록에도 `schedule` 을 더했습니다. `a11y-check.py` 의 `PAGES` 에는 있었는데 이쪽에는 빠져 있어, 이번 버그를 잡았어야 할 검사가 이 화면을 보고 있지 않았습니다. 손으로 관리되는 목록이라 새 화면이 조용히 빠집니다.

검증

시험 7개를 더했습니다 — 순수 함수 4개(`scheduleGrid.test.ts`)와 화면 3개(`SchedulePage.test.tsx`). 고치기 전 코드에서 새 화면 시험이 `month-cell` 42칸을 그리며 실패하는 것을 먼저 확인했습니다.

프런트엔드 141개 시험과 `tsc -b` ·프로덕션 빌드를 두 시간대(`Asia/Seoul` , `America/New_York`)에서 통과했고, 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` , 버전·OpenAPI·모달·페이징 검사를 통과했습니다.

API·데이터 모델·환경변수 변경은 없습니다. 프런트엔드만 바뀝니다.

v0.295.0 — 상황판에서 제목의 오타를 고쳐도 업무 링크가 끊기지 않습니다

상황판의 수정은 `PUT` 이고 `PUT` 은 줄 전체를 보낸 것으로 갈아 끼웁니다. 편집 창은 읽어 온 줄에서 `workItemId` 를 담지 않았고 저장도 보내지 않았으므로, 제목의 오타 하나를 고치면 그 줄이 가리키던 업무가 사라졌습니다. 링크는 상황판의 어느 보기에도 그려지지 않아 고치는 사람에게는 그런 것이 있었다는 단서조차 없고, 판은 고치기 전과 똑같이 보입니다 — 끊긴 것은 한참 뒤 업무 추적 쪽에서 발견됩니다.

창이 값을 들고 다닙니다

편집 창이 드는 줄(`ScheduleDraft`)이 `workItemId` 를 들게 해서 고치지 않은 것은 고쳐지지 않게 했습니다. 담당자를 바꿀 때만 링크를 함께 내려놓습니다: 한 줄의 링크는 담당자 본인의 업무만 가리킬 수 있어 서버가 `WORK_ITEM_NOT_OWNED` 로 거절하므로, 보이지도 않는 값 때문에 저장이 통째로 실패하는 것보다 넘길 때 놓는 쪽이 남은 하나입니다. 편집 창이 드는 줄과 저장이 보내는 것(`scheduleBody` · `draftOf` · `withAssignee`)을 `scheduleGrid.ts` 의 순수 함수로 옮겨 시험이 화면을 거치지 않고 직접 물을 수 있게 했습니다.

시험이 하나도 없던 쓰기

`updateScheduleTask` 는 쓰기 넷 가운데 전체 치환 계약을 가진 하나이고 담당자를 남에게 옮기는 권한 판정도 여기서만 걸리는데, 시험이 하나도 없었습니다. 두 계약을 적었습니다(`scheduleedit_test.go`): 보낸 것은 남고 보내지 않은 것은 지워진다(업무 링크 포함, 남의 업무·없는 업무는 400), 그리고 체크할 수 있는 사람이면 고쳐 쓰되 조직 범위로 재는 것은 '옮기는 것' 뿐이다 — 등록한 사람은 그 줄이 남에게 넘어간 뒤에도

고칠 수 있습니다. `openapi.yaml` 의 `PUT /schedule/{id}` 에도 전체 치환 계약을 적었습니다. 이 API 를 직접 부르는 쪽도 같은 함정을 밟기 때문입니다.

검증

프런트엔드는 이전 동작으로 되돌려 새 시험 3개가 모두 실패하는 것을 먼저 확인했고, Go 는 `UPDATE` 의 `input.WorkItemID` 를 nil 로 바꾼 변이가 시험에 잡히는 것을 확인했습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt`, 가드·OpenAPI·모달·페이징·버전 검사를 통과했고, 프런트엔드 144개 시험과 `tsc -b`·프로덕션 빌드를 두 시간대(`Asia/Seoul` , `America/New_York`)에서 통과했습니다. 변이 검사는 첫 회차에서 `&&` → `||` 변이 하나가 아무 시험에도 잡히지 않아 사례를 하나 더해 닫았습니다.

마이그레이션도 새 환경변수도 없고 응답 모양도 그대로입니다. `PUT /schedule/{id}` 의 계약은 원래부터 전체 치환이었고 문서에만 없었습니다.

v0.296.0 — 사용자·관리자 가이드에 실제 화면이 들어갑니다

이번 판에는 고친 결함이 없습니다. 두 안내서를 실제 화면 캡처가 든 완성본으로 다시 쓴 것이 내용입니다. 사용자 가이드와 관리자 가이드는 있었지만 그림이 한 장도 없었고, 구성도 처음 5분 → 화면별 → 자주 하는 작업 → 막혔을 때 / 구성 요소 → 설치 → 설정 → 계정과 권한 → 운영 → 장애 대응 → 보안의 표준 순서가 아니었습니다.

그림은 전부 실제로 띄운 화면입니다

새 `scripts/guide-captures.py` 가 씨앗 데이터(`seed-scale.sql`)를 넣은 배포에 팀원·팀장·관리자로 로그인해 headless Chrome 1440×900 으로 27장을 찍어 `docs/assets/guide/` 에 놓습니다. 상황판은 팀장이 API 로 줄 몇 개를 만들어 빈 달력을 피하고 끝나면 지웁니다. 대상 주소와 계정은 `WEEKLY_GUIDE_*` 전용 변수로만 받고 루프백이 아니면 멈추며, 전역 설정은 읽기만 합니다. 화면의 이름·조직·보고 내용은 씨앗 데이터이며 실제 사람이 아닙니다. 캡처에 찍히던 제품 UI 의 예시 이메일도 `hong@example.com` 으로 바꿨습니다(관리자 화면의 Confluence 안내문, README, `docs/CONFLUENCE.md`).

두 안내서

사용자 가이드는 처음 5분(로그인 → 작성 → 제출) → 화면별 사용법(14개 화면, 그림 16장) → 자주 하는 작업(옛 문단 이관) → 막혔을 때(화면이 실제로 내는 문구 열다섯 줄) → 용어입니다. 관리자 가이드는 구성 요소 → 설치(`docker load` → compose → `readyz`) → 설정(`config.go` 의 환경 변수 다섯 개 전부와 관리자 화면 카드 열 개) → 계정과 권한 → 운영 → 장애 대응(로그 문구 그대로) → 보안이며, 문서에 적은 API 11개는 `app.go` 에서 메서드까지 확인했습니다. 옛 문서가 주장하던 "관리자 전체 키 즉시 폐기" 는 그런 라우트가 없어 뺐습니다 — 있는 것은 사용자 개인 설정의 `모든 키 회전` 입니다.

정본이 둘이 되지 않게

두 안내서의 PDF 는 프로젝트 공통 도구 `md2pdf.mjs` 가 표지·쪽번호·그림 배치를 통일해 굽고, `render-docs.py` 는 두 안내서에 대해 그림을 지원하는 HTML 만 만들고 PDF 는 건너뜁니다. 27장이 모두 문서에 쓰이고 스크립트에는 비밀번호 글자가 없습니다.

검증

실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet`, 버전·OpenAPI·모달 검사를 통과했고, 프런트엔드 시험과 `tsc -b` 를 통과했습니다. 세 안내서(사용자·관리자·로드맵)의 HTML 과 PDF 를 다시 만들었습니다.

제품 코드는 관리자 화면의 예시 이메일 한 줄만 바뀝니다. 마이그레이션도 새 환경변수도 API 변경도 없습니다.

v0.297.0 — MCP 가 제품이 아는 것을 답합니다

MCP 표면을 사람이 부르듯 불러 보고 돌아온 것을 읽었습니다. 세 가지가 나왔습니다: 답이 틀렸는데 틀린 줄 알 수 없는 것, 프로토콜을 프로토콜 밖에서 답하는 것, 그리고 제품이 아는데 도구가 없는 것.

조용한 0건과, 서비스 탓

`weekly_reports_search` 를 `weekStart: "지난주"` 로 부르면 — 형식을 듣지 못한 모델이 정확히 그렇게 씁니다 — 그 글자가 PostgreSQL 에 날짜로 넘어가 `분석 중 오류가 발생했습니다` 가 돌아왔고, 그 옆에 데이터베이스를 지목하는 ERROR 로그가 남았습니다. 호출자는 무엇을 고쳐야 할지 알 수 없고, 로그를 보는 사람은 서비스가 죽었다고 듣습니다. `status: "제출됨"` 은 더 나빴습니다. 스키마에 허용값을 적어 두고 검사는 하지 않아 `{"total":0}` 이 돌아왔고, 건수의 권위를 그대로 두른 거짓 음성은 "그런 보고는 없습니다"로 보고됩니다.

인자는 이제 데이터베이스에 닿기 전에 검사합니다. 날짜 형식, 보고서 상태, 조회 범위, 상황판 상태 필터가 규격을 벗어나면 무엇이 틀렸고 무엇이 허용되는지 문장으로 답하고(`isError`), 호출자가 만든 실수는 장애로 기록하지 않습니다(`mcpRefusal`). `period_report_rollup` 의 `kind` 는 `WEEK` 를 받으면서도 스키마에 적지 않아 아무도 주 단위를 물을 수 없었고, 빈 기간의 `note` 는 `timelineItems: 0` 옆에서 "상위 20건에만 있습니다"라고 말했습니다 — 숫자와 그것을 설명하는 문장이 같은 답 안에서 어긋났고, 모델이 읽는 쪽은 문장입니다.

프로토콜은 프로토콜 안에서

본문이 깨진 요청은 제품의 REST 봉투와 HTTP 400 으로 돌아왔습니다 (`{"success":false,…,"INVALID_REQUEST"}`). MCP 클라이언트가 읽을 수 없는 모양이라, 호출자에게는 서버가 죽은 것과 구분되지 않습니다. 이제 HTTP 200 에 `-32700` 을 실어 답합니다. 같은 이유로 JSON-

RPC **배치**를 받습니다(한 번에 20건까지, 알림만 든 배치는 본문 없이 202): 서버는 `2025-03-26` 을 지원한다고 광고하면서 그 규격의 프레임을 거절하고 있었고, 그런 구멍은 그 클라이언트가 깨질 때에만 발견됩니다. 최상위에 `_meta` 같은 필드가 실려 와도 더는 거절하지 않습니다 — 알 수 없는 필드를 거부하는 디코더를 쓰고 있었기 때문입니다.

네 개의 도구

제품이 아는데 물을 수 없던 것들입니다.

- `weekly_report_detail`: 검색은 보고서를 찾아 주고 요약 한 줄을 돌려줄 뿐, 그다음 단계가 없었습니다. 무엇을 했느냐는 질문에 모델은 그 한 줄을 한 주의 일로 제시했습니다. 이제 업무·실적·계획·이슈·건의·검토 의견 본문을 읽습니다. 권한은 화면과 같고, **볼 수 없는 보고서와 없는 보고서는 같은 문장으로 거부합니다**. 팀원 보고서를 재료로 쓴 보고서는 재료의 본문 대신 이름과 `reportId` 만 싣습니다.
- `weekly_missing_submitters`: 제출 현황은 몇 명인지 알려 주지만 누구인지는 아무도 물을 수 없었습니다. 주간보고 제품에 가장 자주 오는 질문입니다. 임시저장은 제출로 세지 않고, 보고서는 날짜가 아니라 그 7일을 덮는지로 찾습니다. **팀장 이상만 부를 수 있고 그 밖의 계정에게는 목록에 아예 보이지 않습니다** — 몇 명인지와 누구인지는 다른 일입니다.
- `weekly_reports_text_search`: 검색은 주차와 상태로만 걸렀습니다. 어느 주인지 모르는 질문 — "그 장애 얘기가 어느 보고서에 있었나" — 에는 주를 넘겨 가며 요약을 읽는 것 말고 방법이 없었고, 제품에는 권한 범위와 스니펫을 갖춘 세 패스짜리 내용 검색이 처음부터 있었습니다. 화면과 같은 함수를 부르도록 `searchReportsFor` 로 꺼냈습니다. 근사 일치(`fuzzy`)나 의미 검색(`semantic`)이 섞이면 **원문 인용처럼 다루지 말라고** 문장으로 적고, 넓히는 패스가 아예 돌 수 없었으면 그 이유를 적습니다 — 이유 없는 빈 결과는 "그런 보고서는 없다"로 읽힙니다.
- `schedule_board_tasks`: 업무 상황판은 이 표면보다 네 판 뒤에 났고 아무도 이어 붙이지 않았습니다. 이번 달에 무엇이 있느냐는 질문에 모델은 주간보고로 답했습니다 — 지난주에 **한 일**이지, 금요일에 **마감할 일**이 아닙니다. 기간·범위·상태(`OVERDUE` , `URGENT` 등)로 묻고, `summary` 는 조건에 맞는 전체를 세며 목록은 100건까지입니다. 화면과 같은 행을 읽도록 상황판 조회를 `scheduleTasksBetween` 하나로 합쳤습니다.

목록이 약속이 되게

두 가지가 반복해서 틀렸습니다. 도구가 기본 인자만으로는 답하지 못하는 것(`<nil>` 사건), 그리고 개인 설정의 MCP 카드가 손으로 유지되다 뒤처지는 것 — 카드는 도구 둘을 적고 있었고 `period_report_rollup` 은 난 날부터 빠져 있었습니다. 이제 시험이 **목록에 오른 모든 도구를 세 역할로 각각 불러** 기본 인자만으로 답하는지 확인하고, 카드가 서버가 내놓는 모든 도구 이름을 담고 있는지 확인합니다.

검증

실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet`·`gofmt`, 가드 검사(새 시험 11개가 모두 자기가 지킨다고 적은 함수에 닿는 것을 확인), 새로 쓴 함수에 대한 변이 검사(살아남은 변이 없음), OpenAPI·모달·쪽넘김·버전 검사를 통과했고, 프런트엔드 144개 시험과 `tsc -b`·프로덕션 빌드를 통과했습니다. 표면의 동작은 시험 전에 손으로 눌러 확인했습니다 — 위에 적은 증상은 모두 그렇게 받아 본 응답입니다.

마이그레이션도 새 환경변수도 없습니다. 기존 도구 넷의 응답 모양은 그대로이고, 달라진 것은 잘못된 인자에 대한 답과 프로토콜 오류의 모양입니다.

v0.298.0 — 목록이 잘렸다는 사실과, 나머지로 가는 길

씨앗을 넣은 배포(305명·15,086건 보고·101,999개 항목)에 붙여 규모 점검을 돌리는 데서 시작했습니다. 먼저 계측기 자체가 문을 못 보고 있었습니다.

계측기가 못 보던 문들

규모 점검의 MCP 도구 목록은 손으로 적은 셋이었고 표면은 여덟이 되어 있었습니다 — 보고서 본문을 통째로 돌려주는 문을 포함해 넷이 한 번도 측정되지 않았습니다. 이제 `tools/list` 로 물어 전부 재고, 인자가 필요한데 표에 없어 거부당한 도구는 크기를 찍는 대신 지적합니다. `/api/v1/schedule` 은 목록에 아예 없었습니다: 부서 한 달을 그리는 화면이자 상한도 총계도 없이 찾은 것을 전부 돌려주는 유일한 목록인데, 어떤 규모에서도 잰 적이 없었습니다. `openapi.yaml` 의 GET 경로와 대조해 한 번도 부르지 않는 문을 매 실행마다 적게 했고, 그 목록을 보고 배포 크기에 따라 자라는 넷을 더 넣었습니다. 목록 상한 점검이 아는 총계 이름이 셋뿐이라 `missingTotal` 로 세는 미제출자 명단의 상한은 보이지 않았던 것도 함께 고쳤습니다. `seed-scale.sql` 은 상황판에 한 줄도 쓴 적이 없어서, 300명 한 달 900줄을 중요도 네 가지·지연 66건·여러 날 600건·SR 180건으로 씁니다.

상황판이 몇 건인지 말합니다

재 보니 300명 한 달이 **900행 274 KB** 였고, 기간은 한 해까지 넓힐 수 있습니다. 한 번에 2,000행까지 보내고 `total` 로 기간 전체 건수를 말합니다. 요약(지연·오늘·긴급)은 **잘리기 전 전체**를 센 값이라 판이 덜 그려져도 숫자는 부서 전체를 말하며, 화면은 몇 건 중 몇 건인지와 무엇을 하면 다 보이는지를 적습니다. 같은 줄에 있던 완료율이 그려진 줄만 세고 있던 것도 요약과 같은 모집단으로 맞췄습니다. 범위를 생략한 링크가 부서 판을 여는 기본값은 시험이 하나도 없었고, 이번에 생겼습니다.

"잘렸다" 는 답이 아닙니다

쪽 넘김 검사는 `items` 라는 이름의 배열만 목록으로 알아봤습니다. 이름이 다르면 검사에 존재하지 않았고, 상황판(`tasks`)이 정확히 그랬습니다. 배열 한 칸과 `total` 을 함께 가진 형이면 목록으로 보게 고치자 두 곳이 더 나왔습니다. 근거 사용처 패널은 "50건 중 12건만 보여 줍니다" 라고 말하고 거기서 끝나 있었는데 — 이 검사의 첫 문단이 그것을 답이 아니라고 적어 둔 바로 그 형태입니다 — 자기 페이지를 고치기 전에 영향

받는 곳을 빠짐없이 찾으려고 여는 패널이므로 `offset` 과 `더 보기` 를 붙였습니다. 자동완성(선행 관계 찾기)은 뒤로 가는 것이 아니라 더 적어 좁히는 자리라 이유를 적었습니다. 시험이 하나도 없던 `evidenceUses` 에 조회 범위와 쪽 넘김 시험이 생겼습니다.

검증

실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드 검사, 새 함수의 변이 검사, OpenAPI·모달·쪽넘김·버전 검사를 통과했고, 프런트엔드 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다. 규모 점검은 씨앗을 넣은 배포에서 34개 경로·MCP 도구 여덟 개 모두 지적 사항 없음입니다.

마이그레이션도 새 환경변수도 없습니다. `/api/v1/schedule` 응답에 `total` 이, `/api/v1/evidence/uses` 에 `offset` 이 생깁니다.

v0.299.0 — 달력이 다시 달력이 됩니다

부서 규모의 씨앗을 넣은 배포에 팀장·조직장으로 로그인해 화면을 하나씩 열어 보는 데서 나왔습니다. 75명 조직의 9월 상황판은 **225건**이고, 그 화면은 이렇게 보였습니다: 첫 주 한 줄이 화면을 다 차지하고, 나머지 넉 주는 스크롤 아래에 있습니다.

한 칸은 네 줄까지

달력 칸이 내용만큼 늘어났기 때문입니다. 바쁜 하루가 스무 줄이면 그 주의 행 전체가 그만큼 커지고, "부서의 한 달을 한 화면에 두고" 라고 적어 둔 화면에서 한 달이 보이지 않습니다. 벽에 걸고 3미터 뒤에서 읽는 판이라면 스무 줄은 읽히지도 않습니다.

이제 한 칸은 **네 줄까지** 그리고 나머지는 `+N건 더` 로 말합니다. 누르면 그 날이 든 **주 보기**로 갑니다 — 주 보기는 하루를 한 칸이 아니라 한 열로 그리므로 마흔 줄도 읽힙니다. 다섯 주가 한 화면에 들어오고, 행의 키가 모두 같아집니다.

끝난 줄은 뒤로

칸에 상한이 생기자 순서가 문제가 됐습니다. 정렬은 중요도만 보고 있어서 **끝난 긴급 네 줄이 남은 일을 전부 `+N건 더` 뒤로 밀어냈습니다.** 늘어나는 칸에서는 아무것도 숨기지 않던 순서가, 상한이 생긴 순간 가장 중요한 것을 숨기는 순서가 된 것입니다. 벽에 걸린 판에서 읽어야 하는 것은 지운 줄이 아니라 남은 줄이고 — 화면에 `완료 숨기기` 가 따로 있는 것도 같은 이유입니다 — 그래서 완료는 언제나 뒤로 갑니다. 월·주·담당자 보기가 모두 같은 함수를 쓰므로 세 곳이 함께 바뀝니다.

"못 한다" 를 "없다" 로 세지 않습니다

실패 상태 검사가 멀쩡한 화면 둘을 지적하고 있었습니다. 개인 설정의 "메일 서버를 준비하기 전에는 지금 발송할 수 없습니다" 와 Import 의 "AI Gateway가 비활성화되어 있어 새 파일을 분석할 수 없습니다" 입니다.

한국어에서 "…할 수 없습니다" 는 못 한다는 말이지 없다는 말이 아닌데, "없습니다" 로 끝난다는 이유로 없다는 주장으로 세였습니다. 지적받은 사람이 할 수 있는 일은 옳은 동작을 허용 목록에 적는 것뿐이었고, 그것은 그 파일이 예외에 대해 적어 둔 것과 정반대입니다.

검증

씨앗을 넣은 배포(305명·15,086건 보고·상황판 900줄)에서 접근성 검사(조작 지점 537개 전부 이름·키보드), 실패 상태 검사(화면 14곳), 규모 점검을 통과했고 화면을 직접 열어 눈으로 확인했습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드·OpenAPI·모달·쪽넘김·버전 검사, 프런트엔드 153개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 API 변경도 없습니다.

v0.300.0 — 달력에서 끌어다 놓고, 자기 기록을 봅니다

이번 판은 같은 종류의 제품들이 무엇을 갖추고 있는지 찾아보고 그중 이 제품에 맞는 것을 고른 결과입니다. 참고한 것은 상태보고 도구들의 기능 비교(Range·Geekbot·ClickUp·Taskade), 달력 UI 관례(드래그 이동, 뷰 전환 단축키, 사이드 패널), 그리고 스트릭(연속 기록) 설계에 관한 글들입니다.

끌어다 놓기

달력에서 가장 자주 꼽히는 조작인데 상황판에는 없었습니다 — 하루를 미루려면 창을 열고 날짜 두 칸을 고쳐야 했습니다. 월 보기에서 줄을 끌어 다른 날에 놓으면 옮겨집니다. **기간은 보존합니다**: 9/7~9/11 짜리 닷새 업무를 9/9 에 놓으면 9/9~9/13 이지 9/9 하루짜리가 되지 않습니다. 체크박스와 같은 방식으로 먼저 그린 뒤 서버가 거절하면 되돌립니다. 보내는 것은 편집 창이 보내는 것과 **같습니다** — PUT 이 줄 전체를 갈아 끼우므로, 화면에 그려지지 않는 업무 링크를 빠뜨리면 끌어 놓기 한 번에 그 연결이 조용히 끊깁니다(v0.295 가 편집 창에서 고친 것과 같은 함정). 마우스를 쓰지 않는 사람에게는 편집 창이 그대로 남아 있고, 접근성 검사는 조작 지점 456개가 모두 이름을 말하고 키보드로 닿는 것을 확인합니다.

단축키와 기억

`M` 월 · `W` 주 · `L` 목록 · `P` 담당자 · `T` 오늘 · `←` `→` 앞뒤. Google Calendar 를 쓰는 사람이 이미 손에 익힌 키이고, 벽 앞에서 노트북 하나로 판을 넘기는 사람에게는 이것이 가장 빠릅니다. 글을 쓰는 중이거나 창이 열려 있을 때는 듣지 않습니다 — 제목에 `w` 를 적다가 보기가 바뀌면 그 편이 더 나쁩니다. 마지막으로 고른 보기·범위·완료 숨기기는 이 브라우저에 기억됩니다(발표 모드의 테마와 같은 방식).

나의 제출 기록

습관이 제품입니다. 누가 밀렸는지는 관리자가 늘 볼 수 있었고, 정작 이번 주에 무언가 할 수 있는 **본인**은 자기 기록을 볼 자리가 없었습니다. 대시보드에 연속 제출 주차·최고 기록·최근 52주 중 낸 주가 나옵니다.

세는 주는 미제출 명단이 세는 주와 **같습니다** — 마감이 지난 주만, 계정이 생긴 뒤부터, 보고서는 날짜가 아니라 그 7일을 덮는지로. 그리고 **아직 열려 있는 이번 주는 기록에 넣지 않습니다**: 넣으면 월요일 아침마다 모두의 기록이 끊기고 금요일에 되살아나며, 그것은 사람에 대한 거짓말입니다. 이번 주는 "이미 냈습니다 / 아직입니다" 로 따로 말하고, 마지막으로 빠뜨린 주를 적어 어디부터 이어 쓰면 되는지 알려 줍니다.

고르지 않은 것

찾아본 것 가운데 넣지 않은 것도 적어 둡니다. 기분·사기 체크(mood pulse)는 사내 보고 체계에서 수집 주체와 용도가 분명하지 않으면 감시로 읽히기 쉬워 제품이 먼저 제안할 것이 아니라고 보았고, 칭찬(kudos)은 새 데이터 모델과 권한 규칙이 필요해 한 판에 끼워 넣을 크기가 아닙니다. 경영진 정기 다이제스트 메일은 이미 있는 메일 흐름(제출·권고·마감 임박) 위에 올리는 것이 맞아 다음 판의 후보로 둡니다.

검증

씨앗을 넣은 배포(305명·15,086건 보고·상황판 900줄)에서 브라우저로 직접 확인했습니다: 단축키로 주·월 전환과 앞뒤 이동, 9/1~9/10 짜리 아흐레 업무를 23일 칸에 끌어 놓아 **9/23~10/2** 로 옮겨지는 것, 대시보드의 22주 연속·최근 52주 중 49주. 접근성 검사와 실패 상태 검사(화면 14곳)를 통과했고, 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드·OpenAPI·모달·쪽넘김·버전 검사, 프런트엔드 159개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 없습니다. `GET /api/v1/me/participation` 이 생깁니다.

v0.301.0 — Keycloak 자동 로그인을 관리자가 켜고 끕니다

v0.277.0 의 `prompt=none` 자동 로그인은 스위치가 없었습니다. OIDC 를 켜면 곧 켜졌고, 익명 방문자마다 로그인 화면을 보기 전에 한 번 Keycloak 에 다녀왔습니다. 그리고 시작 주소의 `silent=1` 을 서버가 그대로 믿었으므로, 누구든 그 글자를 붙여 오류 응답을 SPA 로 되돌아가는 리다이렉트로 바꿀 수 있었습니다.

설정 하나, 기본은 꺼짐

silent-sso 표준대로 `oidc.auto_login` 을 둡니다. 관리자 화면 `인증 · Keycloak OIDC` 의 `Keycloak 세션 자동 로그인 (prompt=none)` 이며 **기본값은 꺼짐**입니다 — 꺼진 설치에서는 자동 확인이 일어나지 않고 로그인 화면의 `Keycloak SSO로 로그인` 버튼만 남습니다. `Keycloak OIDC 사용` 을 끄면 저장된 값과 무관하게 꺼집니다. `/auth/providers` 가 `autoLogin` 을 공개하고 화면은 그 값이 true 일 때만 시도하며, 값을 주지 않는 서버는 꺼진 것으로 읽습니다.

서버가 지킵니다

시작 주소의 `silent=1` 은 누구나 붙일 수 있는 글자이므로, 그것이 `prompt=none` 이 되는지와 실패가 오류 대신 SPA 로 돌아가는 리다이렉트가 되는지는 관리자의 설정이 정합니다. 꺼져 있으면 `silent=1` 은 평

범한 로그인이 되어 `prompt=none` 을 보내지 않고 state 행도 silent 로 적지 않으며, 실패는 다른 호출자와 같이 오류로 답합니다. 루프를 막는 세 겹(탭 세션에 한 번, 로그아웃 뒤 억제, 콜백 거절을 주소에 남김)과 저장소를 읽지 못하면 시도하지 않는 것, 최상위 이동, `#/` 로만 돌아가는 것은 이미 있었으므로 손대지 않았습니다.

검증

`oidcStart` 의 남은 변이 넷 — 프록시가 https 라고 알릴 때의 Secure 쿠키와 `redirect_url` 이 비었을 때 요청에서 만드는 콜백 주소 — 을 콜백 쪽과 같은 모양의 시험으로 닫았습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet`, 가드·OpenAPI·모달·버전 검사, 프런트엔드 시험과 `tsc -b`·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 없습니다. 관리자 설정 `oidc.auto_login` 이 늘고 `/auth/providers` 응답에 `autoLogin` 이 생깁니다. 이미 자동 로그인에 기대던 배포는 업그레이드 뒤 이 설정을 켜야 합니다 — 기본이 꺼짐이기 때문입니다.

v0.302.0 — MCP 를 개인 키 없이 Keycloak 로그인으로

MCP 인가 규격(2025-06-18 이후)은 OAuth 2.1 입니다. 지금까지 이 제품의 MCP 는 개인 키(`wky_…`)로만 열렸고, 규격이 말하는 "URL 만 주면 로그인 창이 열리는" 연결은 없었습니다. 이번 판에서 Weekly 는 **리소스 서버**가 되고, 로그인은 이미 있는 Keycloak 이 합니다. 토큰을 발급하는 쪽은 여기가 아니며, 이 서버가 답해야 하는 것은 둘뿐입니다 — 인증 서버가 어디인지, 그리고 이 토큰이 그 서버가 우리를 위해 발급한 것인지.

어디로 가면 되는지

`GET /.well-known/oauth-protected-resource` 와 `…/mcp` (RFC 9728)가 리소스 식별자(기본 `https://<host>/mcp`), 인증 서버(`oidc.issuer_url`), 지원 범위를 **봉투 없는 문서**로 돌려줍니다 — 읽는 쪽은 OAuth 라이브러리이지 이 제품의 SPA 가 아닙니다. 토큰 없이 `/mcp` 를 부르면 `401` 에 `WWW-Authenticate: Bearer resource_metadata="…"` 가 실립니다. 이 헤더가 없으면 거부는 막다른 길이고, 있으면 클라이언트가 스스로 길을 찾습니다. 꺼져 있으면 문서는 404 이고 헤더도 없습니다 — 설정되지 않은 인증 서버를 가리키는 것은 없는 것보다 나쁩니다.

이 토큰이 우리 것인지

`wky_` 로 시작하지 않는 Bearer 는 JWT 로 봅니다. 서명·발급자·만료는 Keycloak 의 JWKS 로 검사하고(발급자 정보는 한 번 읽어 두고 키가 바뀌면 다시 읽음), 토큰이 **이 서버를 위해 발급된 것인지**는 `aud` 또는 발급받은 클라이언트(`azp`)로 봅니다 — 리소스 식별자·웹 로그인의 `Client ID` ·관리자가 적은 허용 대상 가운데 하나여야 합니다. 실제 Keycloak 26 을 띄워 재 보니 액세스 토큰의 `aud` 는 `account` 뿐이고 클라이

언트는 `azp` 에 있었습니다(ID 토큰과 다릅니다). 그래서 **허용 대상에 MCP 클라이언트 ID 를 적으면 매퍼 없이 연결**되고, Audience 매퍼를 더한 규격의 길도 같은 Keycloak 에서 확인했습니다. 거부는 이유를 말합니다 — 대상이 다르면 어느 값을 더해야 하는지, 계정이 없으면 웹으로 먼저 로그인하라고, 꺼져 있으면 관리자가 켜야 한다고, JWT 도 키도 아니면 그렇다고.

같은 방, 두 번째 문

토큰은 **이미 등록된** 계정만 인증합니다. 웹 로그인은 계정을 만들지만, 프로그램이 토큰을 내미는 순간은 누군가를 등록할 자리가 아닙니다. 범위는 토큰이 아니라 관리자 설정(`SSO 토큰에 주는 범위`)이 정합니다 — Keycloak 에 Weekly 의 범위 어휘를 가르치지 않아도 되게 하기 위해서이고, 그 어휘는 키 발급 화면과 **한 목록**(`keyScopes`)을 씁니다. 그리고 토큰은 개인 키와 같은 읽기 전용 규칙을 따릅니다: 사람의 신원이지만 프로그램이 들고 다니는 것이고, 규칙은 프로그램에 대한 것입니다.

설정과 안내

관리자 화면에 `MCP SSO (OAuth)` 카드가 생깁니다(스위치, 리소스 식별자, 추가 허용 대상, 토큰 범위). 관리자 안내서에 Keycloak 쪽 할 일 — Audience 매퍼, 동적 등록(DCR) 정책 또는 미리 만든 공개 클라이언트, 사용자 등록 — 과 확인하는 법을 적었고, 개인 설정의 MCP 카드는 이 문이 열려 있을 때 그렇다고 말합니다.

검증

가짜 인증 서버(디스커버리·JWKS·서명)로 규격의 리소스 서버 절반을 시험합니다: 메타데이터와 `WWW-Authenticate` , 리소스 대상·클라이언트 대상·추가 대상 각각의 허용, 다른 대상·모르는 계정·만료·다른 키의 서명·JWT 아님·스위치 꺼짐 각각의 거부와 그 문장, 그리고 토큰으로 쓰기가 403 인 것. 변이 검사에서 `oauthPrincipal` 의 변이 여섯이 모두 잡힙니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드·OpenAPI·모달·쪽넘김·버전 검사, 프런트엔드 160개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 없습니다. 관리자 설정 `mcp.oauth.*` 넷이 늘고, `/.well-known/oauth-protected-resource` 가 생기며, `/auth/providers` 에 `mcpOAuth` 가 실립니다. 기본은 꺼짐이라 기존 배포의 동작은 그대로입니다.

v0.303.0 — 마이그레이션 시험이 데이터베이스의 과거가 아니라 이 빌드를 봅니다

제품의 코드는 v0.302.0 과 같습니다. 이번 판이 고치는 것은 릴리즈를 두 번 멈춘 시험 하나입니다. `TestDatabaseMigrationsAndSecretRotation` 은 설정된 데이터베이스의 `schema_migrations` 에서 `max(version)` 을 읽어 내장 마이그레이션 파일 수(30)와 견줬습니다. 그 표는 **그 데이터베이스에 지금까지 적용된 모든 것의 역사**라, 여러 브랜치가 한 시험 DB 를 나눠 쓰는 기계에서는 이 트리에 없는 다른 브랜치의

031~033 이 남아 `got=33 want=30` 으로 실패했습니다 — 이 빌드의 서른 개는 모두 적용돼 있었고 맞았는데도.

자기 데이터베이스에서 묻습니다

시험이 묻는 것은 "이 빌드의 마이그레이션이 빈 데이터베이스에 모두 적용되고 기록되는가" 이므로, 하네스가 이미 만들어 주는 자기만의 스크래치 DB(`createScratchDatabase`)에서 묻습니다. 가장 큰 번호 하나가 아니라 **기록된 번호의 집합 전체**를 내장 파일의 번호와 맞춥니다(`migrationVersions` 와 `slices.Equal`) — 마지막 파일만 기록되고 가운데 하나가 빠져도 최댓값은 같기 때문입니다. 같은 DB 를 되풀이해 쓰던 때의 `clone-test` 정리는 더는 할 일이 없어 지웠습니다. 워크플로와 검증 명령은 느슨하게 하지 않았습니다.

검증

고치기 전 파일을 되돌려 공유 DB 에서 `got=33 want=30` 실패를 먼저 확인했고, 고친 뒤 통과합니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt`, DB 없는 조건의 전체 Go 테스트, 가드·OpenAPI·모달·버전 검사를 통과했습니다.

마이그레이션도 새 환경변수도 없습니다. 바뀐 파일은 시험 하나뿐이라 제품의 동작은 v0.302.0 과 같습니다.

v0.304.0 — MCP 로 주간보고를 쓰고, 도구가 자기 스키마대로 답합니다

스키마가 답의 절반을 숨기고 있었습니다

모든 도구를 불러 `structuredContent` 를 자기 `outputSchema` 와 대조했습니다. 필수 필드가 빠지거나 타입이 틀린 곳은 없었지만, **넷의 도구가 스키마에 없는 필드를 보내고 있었습니다** — 기간 집계의 `itemsTotal` · `contributorsTotal` · `note`, 내용 검색의 `note`(근사 일치 경고와 넓히지 못한 이유), 상황판의 `today` · `scope` · `state`, 보고서 본문의 `version`. 잘렸다는 사실, 근사 일치라는 사실, 고칠 때 넣어야 하는 값 — 정확히 모델이 알아야 하는 것들이 스키마에서만 보이지 않았습니다. 모두 선언했고, 이제 시험이 목록의 모든 도구를 불러 답의 모든 키가 선언되어 있는지·필수 키가 있는지·타입이 맞는지 대조합니다.

본인 주간보고 만들기와 고치기

`weekly_report_create` 와 `weekly_report_update` 가 생깁니다. `mcp:write` 범위가 있는 연결에만 목록에 나타나고(개인 키 발급 때 따로 켜는 칸, SSO 는 관리자의 토큰 범위), 둘 다 호출한 계정 **본인의** 보고서만 다룹니다. REST API 는 키에 대해 여전히 읽기 전용입니다 — 이 범위가 여는 것은 이 두 도구뿐입니다.

화면의 편집기가 쓰는 함수를 그대로 부릅니다. 만들기와 고치기의 알맹이를 핸들러에서 `createReportFor` · `updateReportFor` 로 꺼내, 소유권·버전 검사·제출 뒤 수정 시 작성 중으로 되돌리는 규칙·업무 식별자·상태 이력·감사 기록이 두 문에서 같습니다. 거부는 `httpFailure` 하나로 결정되어 REST

는 예전 그대로의 상태 코드로, MCP 는 문장으로 답합니다. 거부 장부(`refusals.sums`)와 `authz-check` 가 이 두 번째 철자(`fail(403, …)`)를 읽도록 고쳤습니다 — 쓰기 경로를 공유한 날 거부 둘이 조용히 장부에서 사라졌기 때문입니다.

모델을 위한 규칙 둘. **읽지 않은 것을 덮어쓰지 못합니다**: `version` 이 필수이고 다른 곳에서 먼저 저장되면 거부됩니다. **보내지 않은 것은 그대로 둡니다**: 화면의 PUT 은 전체 치환이지만, 요약만 보낸 모델 때문에 항목 여덟이 사라지는 일은 없어야 하므로 현재 값을 먼저 읽고 보낸 것만 바꿉니다. `items` 는 기본 `REPLACE` 이고 뒤에 더하기만 하는 `APPEND` 가 있으며, detail 이 준 `id` 를 들고 오면 항목의 정체성과 업무 추적 링크가 유지됩니다. 제출은 하지 않습니다 — 그것은 사람이 화면에서 합니다.

`weekly_report_detail` 은 이제 `reportId` 없이도 **본인의 이번 주(또는 `weekStart` 의 주) 보고서**를 찾습니다. 쓰기 전에 모델이 묻는 "이번 주 내 보고서에 뭐가 있나" 에 id 가 있을 리 없고, 없으면 그렇다고 답하며 만들기를 안내합니다.

검증

쓰기 도구 시험: 주 중간 날짜가 그 주의 시작으로 맞춰지는 것, 요약만 고쳐도 항목이 남는 것, `APPEND` , `id` 를 든 `REPLACE` 가 정체성을 지키는 것, 옛 `version` 과 `version` 없음의 거부 문장, 같은 주 두 번째 만들기의 안내, 읽기 키에는 도구가 보이지 않고 부르면 범위를 말하는 것, 팀장이 팀원 보고서를 못 고치는 것, 볼 수 없는 것과 없는 것이 같은 문장인 것, REST 가 키에 여전히 403 인 것. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드 검사, OpenAPI·모달·쪽넘김·버전 검사, 프런트엔드 160개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 없습니다. 키 범위 어휘에 `mcp:write` 가 늘고, 기본 발급 키에는 들어가지 않습니다.

v0.305.0 — 내 기록 시험이 달력이 아니라 마감 규칙이 닫은 주를 셉니다

제품의 코드는 v0.304.0 과 같습니다. 이번 판이 고치는 것은 월요일마다 실패하던 시험 하나입니다. `TestMyRecordCountsTheWeeksTheArrearsListCounts` 는 "지난주" 를 달력으로 세었지만, 기록에 드는 주는 **마감이 지난 주**뿐이고 마감은 규칙이 정합니다. 기본 규칙(시작일로부터 7일 뒤, 그날이 끝나는 자정)에서 지난주의 마감은 이번 주 화요일 0시라 월요일에는 지난주가 온종일 아직 열려 있고, `myParticipation` 은 열린 주를 기록에 넣지 않으므로 낸 세 주 가운데 하나가 빠져 `연속 2주, want 3` 이 났습니다 — 코드가 옳고 시험이 틀린 경우라 제품 코드는 손대지 않았습니다.

규칙이 닫은 주부터 셉니다

시험은 `closedWeekStart(now, rule)` 로 서버의 `deadlineRule` 을 읽어 마감이 지난 가장 최근 주를 찾고 거기서부터 주를 셉니다(SQL 의 `deadlinePassedFor` 와 같은 셈, 서비스 시간대). 같은 시나리오를

`workflow.deadline_days=13` 으로 한 번 더 돌려 오늘이 무슨 요일이든 "지난주가 아직 열린" 경로가 시험에 듭니다 — 월요일이 오기를 기다리지 않아도 됩니다.

검증

옛 앵커로 되돌리면 금요일에도 같은 문장으로 실패하고 고친 앵커로는 두 규칙 모두 통과합니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드·OpenAPI·모달·버전 검사를 통과했고, 변이 검사가 적용한 변이 셋을 모두 잡았습니다.

마이그레이션도 새 환경변수도 없습니다. 바뀐 파일은 시험 하나뿐이라 제품의 동작은 v0.304.0 과 같습니다.

v0.306.0 — 관리자 메일 카드가 세 큐를 모두 세고 최근 발송 기록을 보여 줍니다

실패 0건이 거짓말이었습니다

관리자 화면의 `주간보고 메일 발송` 카드는 `report_mail_deliveries` 하나만 세었습니다. 릴레이는 세 큐를 지납니다 — 제출 메일, 팀원 작성 권고(`team_reminder_deliveries`), 상황판 마감 알림(`schedule_reminder_deliveries`). 권고와 알림이 전부 거부돼도 카드는 "실패 0건" 이라고 답했고, 관리자 가이드 3.7 이 "권고·알림 장애는 서버 로그와 큐 테이블에서" 라고 자백하고 있었습니다. 이제 세 표를 하나의 `UNION ALL` 서브쿼리(`mailDeliveriesUnion`) 위에서 세고, 집계·마지막 실패 사유·목록이 **같은 14일 창의 같은 행**을 읽습니다.

종류별 건수와 최근 발송 기록

`GET /api/v1/admin/mail/health` 는 기존 필드를 그대로 두고 `byKind{report, teamReminder, scheduleReminder}` 만 더했습니다 — `writers` 는 세 큐의 수신자를 합친 사람 수, `lastError` 는 세 큐 가운데 가장 최근의 실패 사유입니다. 새 `GET /api/v1/admin/mail/deliveries` (ADMIN 전용) 가 같은 창의 기록을 `createdAt` 내림차순으로 최근 50건까지 `{days, items}` 로 돌려줍니다. 각 행은 종류·수신자·주소·제목·상태·시도·사유·시각이 전부이며 보고서 본문과 제목은 싣지 않습니다 — `subject` 는 주 시작일 또는 알림 날짜로 만든 한 줄입니다. 관리자 화면의 메일 카드는 현황 문단 아래에 그 표를 보여 주고, 관리자 가이드 3.7 과 5.1 이 이 사실을 말합니다.

검증

새 시험 `TestTheOperatorSeesEveryQueueTheRelayTouches` 가 세 큐에 한 건씩 넣고 고치기 전 코드에서 `failed=0` · `queued=0` · `writers=1` ·deliveries 404 로 실패하는 것을 먼저 확인한 뒤 통과했습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드·OpenAPI·쪽넘김·모달·버전 검사, 프런트엔드 160개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 설정 키의 변경도 없습니다. health 응답은 필드가 하나 늘 뿐 기존 필드의 뜻이 같습니다.

v0.307.0 — 관리자 메일 기록을 종류·상태로 거르고, 최근 50건 밖의 실패도 찾습니다

좋은 한 주가 카드가 세고 있는 실패를 덮고 있었습니다

v0.306.0 은 카드의 숫자와 그 아래 표가 같은 14일 창의 같은 행을 읽게 했고, `mailDeliveriesUnion` 의 주석은 그래서 둘이 어긋날 수 없다고 적어 두었습니다. 그런데 목록은 `LIMIT 50` 이고 창 안의 행은 그보다 많을 수 있습니다. 아침마다 마감 알림이 수십 건씩 나가는 배포에서는 최근 50건이 전부 성공이라, 카드가 `실패 3건` 이라고 말하는데 표에는 하나도 없습니다 — 실패는 이미 바닥 밖으로 밀려나 있고 운영자는 사유를 보려고 큐 테이블을 직접 뒤져야 했습니다.

자르기 전에 거릅니다

`GET /api/v1/admin/mail/deliveries` 가 `status` (`QUEUED` · `SENT` · `FAILED`)와 `kind` (`REPORT` · `TEAM_REMINDER` · `SCHEDULE_REMINDER`)를 받아 `LIMIT 50` **앞의** `WHERE` 에 넣습니다. 잘린 뒤에 거르면 그 50건 안에 우연히 남은 실패만 돌아오고, 이 기능이 필요한 바로 그 주에는 0건입니다. 둘 다 대소문자를 가리지 않고(주소창에 직접 치는 일이 select 만큼 잦습니다), 이 배포가 저장하지 않는 값은 거르지 않은 쪽수가 아니라 400 `INVALID_FILTER` 로 답하며, 적용한 조건은 응답의 `status` · `kind` 로 되돌려 줍니다. `total` 은 더하지 않았습니다 — 전체 건수를 돌려주면 쪽 넘김을 약속하는 셈이고 이 표는 아직 한 쪽짜리입니다.

관리자 카드에는 표 위 종류·상태 select 두 개가 생겼고, 좁힌 결과가 비면 빈 표 대신 `이 조건에 맞는 발송 기록이 최근 n일 안에 없습니다` 를 적습니다 — 빈 표는 '발송이 없다' 로 읽히지만 뜻이 다릅니다. OpenAPI 에 두 파라미터와 400 을, 관리자 가이드 3.7·5.1 에 그 사실을 더했습니다.

검증

새 시험 `TestTheOperatorCanPickTheFailuresOutOfAGoodWeek` 가 성공 55건으로 한 쪽을 채운 뒤 그 밖에 FAILED 권고 1건·QUEUED 알림 1건을 넣고, 고치기 전 코드에서 `status=FAILED items=50, want the 1 the card counts` 로 실패하는 것을 먼저 확인한 뒤 통과했습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` · `go build` , 가드·OpenAPI·쪽넘김·모달·버전 검사, 프런트엔드 160개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 설정 키의 변경도 없습니다. 두 파라미터는 선택이고 비우면 v0.306.0 과 똑같이 최근 50건 전부가 돌아옵니다.

v0.308.0 — 발송 기록 표가 50건에서 잘렸다는 사실을 말합니다

가득 찬 표는 "최근 14일이 이것뿐" 으로 읽힙니다

v0.307.0 은 목록을 자르기 전에 종류·상태로 거르게 해서 좋은 한 주가 덮고 있던 실패를 표 위로 끌어올렸습니다. 그런데 응답에는 잘렸다는 표시가 없었습니다 — 목록은 `LIMIT 50` 이고 14일 창 안의 행은 그보다 훨씬 많을 수 있는데, 운영자가 보는 것은 꽉 찬 표 하나입니다. 꽉 찬 표와 "창 안에 딱 50건이 있는 표" 는 화면에서 구별되지 않고, 앞의 것을 뒤의 것으로 읽으면 더 좁혀야 한다는 것을 알 길이 없습니다. 종류·상태로 좁힌 뒤에도 그 조건의 행이 50건을 넘으면 같은 오독이 그대로 남습니다.

51번째 행 하나로 답합니다

응답에 `truncated: boolean` 이 생겼습니다 — 조건을 적용한 뒤에도 창 안의 행이 50건을 넘으면 `true` 입니다. 질의는 여전히 한 번입니다: `LIMIT` 에 `50+1` 을 보내고 51번째 행은 목록에 넣지 않은 채 존재만 읽습니다. 그 행이 있느냐가 답의 전부이므로 COUNT 질의를 따로 더하지 않았습니다. 총건수(`total`)는 싣지 않았습니다 — 몇 건이 더 있는지는 운영자가 쓸 수 있는 숫자가 아니고(할 수 있는 일은 좁히는 것뿐입니다), 전체 건수를 돌려주면 이 목록에 없는 쪽 넘김을 약속하는 셈입니다. 잘리지 않았으면 이 칸은 `omitempty` 로 생략되므로 화면의 문장이 상시 문구가 되지 않습니다. `ORDER BY`·14일 창· `UNION` 집계 · `INVALID_FILTER` 계약은 건드리지 않았습니다.

관리자 카드는 `truncated` 일 때만 표 아래에 한 문장을 답니다 — `최근 50건만 보여 줍니다 — 이보다 오래된 발송이 최근 14일 안에 더 있으니 종류·상태로 좁혀 보세요` . 50 을 화면에 박지 않고 받은 행 수를 읽으므로 서버가 쪽 크기를 바꾸면 문장도 따라갑니다. OpenAPI 에 새 칸과 그것이 왜 생략되는지를, 관리자 가이드 3.7 에 한 줄을 더했습니다.

검증

새 시험을 만들지 않고, 이미 창 안에 57건을 넣고 있던 기존 시험 `TestTheOperatorCanPickTheFailuresOutOfAGoodWeek` 를 늘렸습니다 — 고치기 전 코드에서 `truncated=<nil>, want true` 로 실패하는 것을 먼저 확인한 뒤 통과했고, 1건으로 좁힌 `?status=FAILED` 응답에 `truncated` 가 없음까지 같은 시험이 양쪽 방향으로 증명합니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` · `go build` , 가드·OpenAPI·쪽넘김·모달·버전 검사, 프런트엔드 160개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다. 표 아래 문장 자체는 관리자 설정 화면을 렌더하는 프런트 시험이 없어 타입 검사와 빌드까지만 확인했습니다.

마이그레이션도 새 환경변수도 설정 키의 변경도 없습니다. `truncated` 는 늘어난 칸 하나이고 잘리지 않으면 아예 없으므로, 이 칸을 모르는 옛 화면도 v0.307.0 과 똑같이 읽습니다.

v0.309.0 — 아직 마감 전인 지난주를 내 제출 기록이 이름으로 말합니다

월요일에 아직 살릴 수 있는 주가 응답 어디에도 없었습니다

`GET /api/v1/me/participation` 의 격자 질의는 마감 여부를 `WHERE … AND deadlinePassed` 로 걸러냈고, 마감이 지나지 않은 주는 셈에서 빠지는 것이 아니라 행 자체가 사라졌습니다. 기본 규칙(`deadline_days=7`, `deadline_hour=24`)에서 지난주의 마감은 이번 주 화요일 0시라 지난주는 월요일 하루 내내 아직 열려 있는데, 그 주는 `owed` · `filed` · `streak` · `lastMissed` (마감이 지난 주만 읽습니다) 에도 `thisWeekStart` · `thisWeekFiled` (이번 주만 읽습니다) 에도 들지 못했습니다. 그래서 지난주를 안 낸 사람이 월요일에 보는 문장은 "이번 주는 아직입니다" 하나뿐이고, 오늘 안에 내면 기록이 끊기지 않는 주가 있다는 사실을 볼 길이 어디에도 없었습니다 — 가장 급한 날에 카드가 가장 조용했습니다.

걸러내는 대신 이름을 붙입니다

응답에 `openArrears` 가 생겼습니다 — 이번 주보다 앞서면서 아직 마감이 지나지 않은 미제출 주 가운데 가장 오래된 주의 시작일이고, 해당하는 주가 없으면 생략됩니다. 마감 여부를 `WHERE` 에서 빼고 `SELECT` 의 세 번째 컬럼으로 뽑아 `closed` 로 읽으므로 질의는 여전히 한 번이고 새 질의도 새 플레이스홀더도 없습니다. 열린 행은 이름을 적은 뒤 `Owed` · `running` · `counting` · `LastMissed` 를 건드리기 전에 `continue` 하므로 숫자는 이 변경 전후로 완전히 같습니다 — 열린 주는 여전히 세지 않습니다. 세면 월요일 아침마다 서비스 전체의 기록이 끊기고 금요일에 되살아납니다. `openArrears` 는 점수가 아니라 이름입니다.

행은 `ORDER BY week.day DESC` 로 최신부터 오므로 마지막에 남는 값이 가장 먼저 시간이 떨어지는 가장 오래된 열린 주입니다. 이번 주는 `thisWeekFiled` 가 이미 답하고 있어 제외했습니다 — 같은 주를 두 필드가 서로 다르게 말하는 것은 말하지 않는 것보다 나쁩니다. 마감 판정은 관리자 화면이 미제출 명단을 세는 `deadlinePassedFor` 조각을 그대로 재사용해 이 카드가 열렸다고 부르는 주와 관리자 명단이 열렸다고 부르는 주를 같게 맞췄고, 조각 자체는 고치지 않았습니다(관리자 제출률이 같은 조각을 읽습니다). 날짜 비교는 `time.Equal` 이 아니라 형식화한 날짜 문자열로 합니다 — `date` 컬럼은 UTC 자정으로 스캔되고 이번 주 시작일은 서비스 시간대의 자정이라 `Equal` 로는 같은 날이 어긋나 이번 주가 `openArrears` 로 새어 나갑니다.

화면과 문서

대시보드의 `나의 제출 기록` 카드는 `openArrears` 가 있을 때만 그 주를 짚는 문장을 한 줄 더합니다 — 없으면 문구가 늘지 않아 상시 문구가 되지 않습니다. 카드 노출 조건(`record.owed > 0`)은 그대로 두었습니다. `docs/openapi.yaml` 에 새 칸과 그것이 왜 기록에 세지지 않는지를, 사용자 가이드 3.5.1 에 한 문장을 적었습니다.

검증

새 시험 `TestMyRecordCountsTheWeeksTheArrearsListCounts` 의 서브테스트가 고치기 전 코드에서 `열린 미제출 주가 "", want "2026-09-14"` 로 실패하는 것을 먼저 확인한 뒤 통과했습니다 — 오늘이 일요일이라 기본 마감 서브테스트는 열린 지난주가 없어 통과하고, `deadline_days=13` 서브테스트가 요일과 무관하게 그 경로를 밟습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` · `go build`, 가드

·OpenAPI·쪽넘김·버전 검사, 프런트엔드 160개 시험과 `tsc -b`·프로덕션 빌드를 통과했습니다. 카드의 문장 자체는 이 저장소에 대시보드 화면을 렌더하는 프런트 시험이 없어 타입 검사와 빌드까지만 확인했습니다.

마이그레이션도 새 환경변수도 설정 키의 변경도 없습니다. `openArrears` 는 늘어난 칸 하나이고 해당하는 주가 없으면 아예 생략되므로, 이 칸을 모르는 옛 화면도 v0.308.0 과 똑같이 읽습니다.

v0.310.0 — Confluence 진단 표가 20건에서 잘렸다는 사실을 말하고, 실패 수를 화면에 보여 줍니다

가득 찬 진단 표는 "진단이 이것뿐" 으로 읽힙니다

`confluence_sync_errors` 는 최신 500건까지 남는데(`recordConfluenceError`) `GET /api/v1/admin/confluence/sync/status` 는 그중 `LIMIT 20` 만 읽었고, 응답에는 잘렸다는 표시가 없었습니다. 그래서 실패가 40건 난 동기화에서 관리자가 보는 것은 꽉 찬 20행짜리 표 하나였습니다 — 스무 개를 읽고 고쳤는데 사유가 그대로인 실패가 스무 개 더 남아 있다는 것을 알 길이 없었고, 표가 가장 필요한 동기화일 때 표가 가장 조용했습니다. 같은 카드의 `실패` 수는 화면에 아예 없었습니다: 관리자 가이드 3.6 은 이 탭이 실패 수를 보여 준다고 이미 적어 두었지만 `pagesFailed` 는 응답에만 있고 `AdminPage.tsx` 에는 한 번도 등장하지 않았습니다.

21번째 행 하나로 답합니다

응답에 `recentErrorsTruncated` 가 생겼습니다 — 표에 담지 못한 진단이 더 남아 있을 때만 `true` 이고, 잘리지 않으면 생략됩니다. 질의는 여전히 한 번입니다: `LIMIT` 에 `confluenceRecentErrorLimit+1`(=21)을 보내고 21번째 행은 목록에 넣지 않은 채 존재만 읽습니다 — 그 행이 있느냐가 답의 전부이므로 COUNT 질의를 따로 더하지 않았고, 이것은 v0.308.0 이 발송 기록에서 쓴 것과 같은 방식입니다. 스캔 루프가 머리에서 `len(errorsView)==limit` 을 보고 멈추므로 `recentErrors` 는 여전히 20건이고 목록은 v0.309.0 과 같습니다. 총건수는 싣지 않았습니다 — 몇 건이 더 있는지는 관리자가 쓸 수 있는 숫자가 아니고, 돌려주면 이 목록에 없는 쪽 넘김을 약속하는 셈이 됩니다.

`ORDER BY created_at DESC` 에 `, id DESC` 를 더해 같은 순간에 적힌 진단들이 20번째와 21번째 자리를 번갈아 차지하지 않게 했습니다 — 같은 상태를 두 번 읽었을 때 빠지는 행이 달라지면 안 됩니다. `20` 은 `confluenceRecentErrorLimit` 상수 하나가 되어 쪽 크기와 `+1` , 멈추는 조건이 같은 숫자를 읽습니다. 잘리지 않았을 때 이 칸을 생략하는 것은 형식이 아니라 계약입니다: 항상 있는 칸은 화면이 항상 찍는 문장이 되고, 그러면 아무 말도 하지 않게 됩니다.

화면과 문서

`자동 수집 상태` 카드에 `실패` 칸(`pagesFailed`)이 생겨 가이드가 약속한 숫자가 처음으로 화면에 있습니다. `최근 동기화 진단` 카드는 `recentErrorsTruncated` 일 때만 표 아래에 한 문장을 더하고, 카드 노출 조건

(`recentErrors.length>0`)은 그대로입니다. 관리자 가이드 3.6 은 이 표가 오류만이 아니라 매핑 누락 같은 알림도 함께 담으므로 문구를 `오류` 에서 `진단` 으로 바꾸고, `실패` 칸은 마지막 동기화 1회의 Page 실패 수라서 누적되는 진단 건수와 다른 숫자라고 적었습니다 — 두 값을 같은 것으로 읽으면 표가 비어 있는데 실패가 3인 화면을 고장으로 오해합니다.

검증

새 시험 `TestTheOperatorLearnsTheDiagnosisTableIsCutOff` 가 고치기 전 코드에서 `recentErrorsTruncated=<nil>, want true` 로 실패하는 것을 먼저 확인한 뒤 통과했고, 같은 시험이 정확히 20건인 표의 응답에는 이 칸이 없음까지 양쪽 방향으로 증명합니다. 진단 행은 직접 INSERT 하지 않고 프로덕션 쓰기 경로인 `recordConfluenceNotice`·`recordConfluenceError` 로 만들었습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet`·`gofmt`·`go build`, 가드·OpenAPI·쪽넘김·모달·버전 검사, 프런트엔드 160개 시험과 `tsc -b`·프로덕션 빌드를 통과했습니다. 표 아래 문장과 `실패` 칸 자체는 관리자 설정 화면을 렌더하는 프런트 시험이 없어 타입 검사와 빌드까지만 확인했습니다.

마이그레이션도 새 환경변수도 설정 키의 변경도 없습니다. `recentErrorsTruncated` 는 늘어난 칸 하나이고 잘리지 않으면 아예 없으므로, 이 칸을 모르는 옛 화면도 v0.309.0 과 똑같이 읽습니다.

v0.311.0 — 조직 계정이 로그인의 첫 번째 길이 되고, 자동 로그인이 사람을 가두지 않습니다

사일런트 SSO 는 동작합니다 — 실제 Keycloak 에서 확인했습니다

Keycloak 26 을 띄우고 브라우저로 세 경우를 끝까지 따라갔습니다. Keycloak 에 이미 로그인한 브라우저는 Weekly 를 열자 로그인 화면 없이 대시보드에 섰고, 세션이 없는 브라우저는 한 번 물은 뒤 로그인 화면으로 돌아와 새로고침해도 다시 묻지 않았으며, 로그아웃한 탭은 곧바로 다시 들어가지 않았습니다. 자동 로그인은 관리자 설정 `Keycloak 세션 자동 로그인 (prompt=none)` 이며 기본은 꺼짐입니다.

그런데 두 곳이 사람을 두고 떠났습니다

세션이 없어 돌아온 로그인 화면은 **아무 말도 하지 않았습니다.** 자동으로 들어가질 줄 알았던 사람은 왜 아니었는지 알 방법이 없었습니다. 이제 "조직 계정 세션이 없어 자동으로 로그인하지 않았습니다" 라고 — 오류가 아니므로 경고가 아니라 안내로 — 말합니다.

더 나쁜 쪽은 **브라우저가 Keycloak 에 닿지 못할 때**였습니다. 자동 로그인은 탭 전체를 Keycloak 으로 보내므로, VPN 이 끊겼거나 IdP 가 재시작 중이면 사람은 브라우저의 "사이트에 연결할 수 없음" 화면에 갇히고 Weekly 는 사라집니다 — 검사 스크립트의 브라우저가 발급자 주소를 풀지 못해 정확히 그렇게 멈추는 것으로 발견했습니다. 이제 화면은 보내기 전에 발급자의 디스커버리 문서에 한 번(`no-cors`, 2.5초) 물어 답이 없으면 보내지 않고, "이 브라우저에서 Keycloak 에 연결할 수 없어 자동 로그인을 건너뛰었습니다. 사내망(VPN) 연결을 확인하거나…" 를 띄운 뒤 그 탭에서는 다시 묻지 않습니다. 그 한 번의 요청을 위해 **자동 로그**

인이 켜져 있을 때만, SPA 문서에만 CSP `connect-src` 에 발급자 origin 하나가 더해집니다 — fetch 는 그것을 실행하는 문서의 CSP 를 따르므로, API 와 정적 파일까지 넓히면 요청마다 설정을 읽는 비용만 늘고 얻는 것은 없습니다.

조직 계정이 첫 번째 길

로그인 화면은 아이디·비밀번호 칸을 맨 위에 파란 버튼과 함께 두고 SSO 를 "또는" 아래 흰 버튼으로 밀어 두었습니다 — 조직 계정을 쓰는 배포에서 가장 자주 쓰는 길이 가장 덜 눈에 띄었습니다. appstore 의 로그인처럼 `조직 계정으로 로그인` 을 주 버튼으로 올리고, 아이디·비밀번호는 `아이디·비밀번호로 로그인` 을 펼쳐야 보이게 했습니다(한 번 펼치면 그 탭에서는 펼친 채). 로컬 계정은 사라지지 않습니다 — SSO 가 없는 계정과 Keycloak 이 멈췄을 때의 복구 경로이며, 펼치면 그 사실을 적어 둡니다. SSO 를 쓰지 않는 배포에서는 처음부터 아이디·비밀번호 칸이 보입니다. 서비스 이름이 두 번 보이던 머리글도 appstore 처럼 마크와 제목 하나로 정리했습니다.

함께 고친 것

- 관리자로 접근성 검사를 돌리자 **보고 분석의 API 지표 표 27행**이 키보드로 닿지 않는 조작으로 잡혔습니다. 전역 CSS 가 모든 표의 행에 손가락 커서를 주어, 눌러도 아무 일 없는 행이 누를 수 있는 것처럼 보였기 때문입니다. 행을 여는 곳은 모두 `openable()` (`role=button`)을 쓰므로 커서를 그 표시에 맞췄습니다.
- 실패 상태 검사는 늘 대시보드를 거쳐 다음 화면으로 들어갔으므로, 대시보드 자신은 같은 주소로 "이동"만 하고 다시 그려지지 않아 **한 번도 대시보드의 실패를 본 적이 없었습니다.** 다른 화면을 거치게 고쳤습니다.
- 브라우저 검사 세 개(접근성·실패 상태·안내서 캡처)가 접힌 로그인 칸을 펼치고, 자동 로그인이 한 번 다녀오는 동안 기다리도록 고쳤습니다. 안내서의 로그인 그림 두 장을 새로 찍었습니다.

검증

실제 Keycloak 26 과 빌드한 이미지로: 세션 있음 → 자동 로그인, 세션 없음 → 안내와 함께 로그인 화면, Keycloak 에 닿지 않는 브라우저 → 0.7초 안에 안내와 함께 로그인 화면(이전: 브라우저 오류 화면), 로그아웃 → 다시 들어가지 않음, `조직 계정으로 로그인` → Keycloak 로그인 → 복귀, 펼친 칸으로 로컬 로그인, CSP 위반 0. 접근성 검사(관리자, 132곳) 통과. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` , 가드·OpenAPI·모달·쪽넘김·버전 검사, 프런트엔드 173개 시험과 `tsc -b` ·프로덕션 빌드를 통과했습니다.

마이그레이션도 새 환경변수도 새 설정도 없습니다. `/api/v1/auth/providers` 에 `oidcIssuer` 가 생기고(자동 로그인이 켜졌을 때만 값이 있음), 그 조건에서 SPA 문서의 CSP 가 발급자 origin 하나만큼 넓어집니다.

v0.312.0 — Confluence 자동 수집 상태가 실패한 질의를 답으로 내놓지 않고, 탭이 그 실패를 문구로 말합니다

조회에 실패한 카드가 "아무 문제 없습니다" 라고 말했습니다

`GET /api/v1/admin/confluence/sync/status` 는 네 번 읽는데 바로 위의 동기화 상태 질의 하나만 오류를 검사했고, 아래 세 읽기 — 진단 목록과 매핑·미매핑 사용자 수 — 는 오류를 통째로 버렸습니다 (`errorRows, _ := a.db.Query` , `_ = a.db.QueryRow(count …)` 두 번). 버려진 자리에 남는 값이 하필 화면이 낼 수 있는 가장 안심되는 답이었습니다: 진단 0건(`진단 없음`), 매핑 0명, 미매핑 0명. 그래서 진단 표를 읽지 못하는 바로 그때 화면은 "실패가 없다" 고 말했고, 표가 사라진 것과 표가 비어 있는 것을 관리자가 구분할 방법이 없었습니다 — v0.310.0 이 "가득 찬 표는 '진단이 이것뿐' 으로 읽힌다" 고 고친 그 카드에서, 이번에는 읽지 못한 표가 "진단이 없다" 로 읽혔습니다.

읽지 못했으면 읽지 못했다고 답합니다

세 읽기 모두 위의 질의와 같이 `500 QUERY_FAILED` 로 답합니다 — 카드가 말할 수 있는 것은 조회에 성공한 상태뿐이고, 실패한 조회에는 돌려줄 숫자가 없습니다. 조용히 건너뛰던 `errorRows.Scan(...) == nil` 과 한 번도 보지 않던 `rows.Err()` 도 검사해 행 하나를 못 읽는 것과 순회 도중 끊기는 것이 답이 되지 않게 했습니다. 단 잘림 판정으로 루프를 `break` 한 경우에는 `rows.Err()` 를 보지 않습니다: 그 중단은 v0.310.0 이 21번째 행을 읽고 일부러 멈추는 것이며, 의도한 중단을 오류로 읽으면 잘린 표가 통째로 500 이 됩니다. 응답 필드·질의· `LIMIT limit+1` 잘림 계약·OpenAPI 는 그대로여서 성공하는 조회의 답은 v0.311.0 과 같습니다.

거절이 영원히 도는 스피너로 끝나지 않습니다

이 거절이 그대로 도착하면 `ConfluenceTab` 의 `load()` 가 거부되고 `status` 가 `undefined` 로 남아 탭은 아무 말도 하지 않는 스피너가 됩니다 — 500 을 정직하게 돌려준 대가가 화면에서는 무한 로딩입니다. 같은 파일의 감사 로그 탭이 이미 쓰는 관례(`errorText` + 실패 문구 + `다시 시도`)를 그대로 따라, 관리자가 무엇이 실패했는지 읽고 다시 누를 수 있게 했습니다. 10초 폴링의 `.catch(()=>undefined)` 는 일부러 그대로 뒀습니다 — 한 번의 일시적 실패로 멀쩡한 카드를 오류 화면으로 바꾸는 쪽이 더 나쁩니다. 멈춘 카드가 오래된 숫자를 나이 표시 없이 계속 보여 주는 문제는 남아 있으며, `연속 실패` 와 `마지막 갱신 시각` 중 무엇을 말할지 먼저 정하는 다음 회차의 몫입니다. 관리자 가이드 3.6 이 이 탭의 실패 문구를 함께 적습니다.

검증

새 시험 `TestTheConfluenceCardRefusesToAnswerWithAFailedQuery` 가 고치기 전 코드에서 `status with confluence_sync_errors unreachable = 200 {… "recentErrors":[], "mappedUsers":0 …}, want 500 QUERY_FAILED` 로 실패하는 것을 먼저 확인한 뒤 통과했고, `user_external_accounts` 하위도 `200 … "mappedUsers":0,"unmappedUsers":0` 으로 함께 실패했습니다(표에는 진단이 실제로 1건 들어

있었습니다). 재현은 기존 관례(`audittrail_test.go` 의 `ALTER TABLE … RENAME TO …_hidden`)대로 표를 잠시 치우고 되돌린 뒤 카드가 다시 답하는 것까지 확인하며, 진단 행은 직접 INSERT 하지 않고 프로덕션 쓰기 경로 `recordConfluenceError` 로 만들었습니다. 실제 PostgreSQL 위에서 전체 Go 테스트와 `go vet` · `gofmt` · `go build`, 가드·OpenAPI·쪽넘김·모달·버전 검사, 프런트엔드 173개 시험과 `tsc -b`·프로덕션 빌드를 통과했습니다. 탭의 실패 문구 자체는 이 저장소에 `AdminPage` 를 렌더하는 프런트 시험이 없어 타입 검사와 빌드까지만 확인했습니다.

마이그레이션도 새 환경변수도 설정 키의 변경도 없습니다. 응답에 늘어난 칸도 사라진 칸도 없고, 달라지는 것은 조회가 실패할 때뿐입니다 — 그때 200 대신 `500 QUERY_FAILED` 가 갑니다. 0 을 정상값으로 읽던 클라이언트가 이제 오류를 받으며, 그것이 이 릴리즈가 고치는 것입니다.

v0.313.0 — Confluence 상태를 읽지 못하면 수동 동기화를 접수하지 않습니다

수동 동기화 POST 의 상태 Scan 오류 무시를 고쳤습니다. 테이블 조회 실패와 상태 행 없음은 worker 깨우기·요청 감사 기록 전에 안전한 한국어 메시지의 `500 QUERY_FAILED` 로 반환합니다. RUNNING·IDLE 성공 계약은 유지하며 POST OpenAPI 에 500 을 추가했습니다.

실제 PostgreSQL·HTTP 시험으로 오류/행 없음/정상 상태와 감사 증감을 검증했고, 구 코드의 202 접수와 감사 +1 을 재현했습니다. 마이그레이션·환경변수·설정 변경은 없습니다. 프런트 렌더링과 wake 호출 부재의 동적 계측은 검증 범위 밖입니다.

v0.314.0 — 거절된 첨부 업로드가 앞선 이미지를 남기지 않습니다

`uploadAttachments` 가 파일을 하나씩 저장하며 진행해, 중간 파일이 거절되면 `400 UNSUPPORTED_IMAGE` 를 답하면서도 앞선 이미지는 디스크와 `report_attachments` 에 남겼습니다. 화면은 실패라고 말하므로 작성자가 성한 파일만 다시 올려 같은 캡처가 두 벌이 됐습니다. 이제 `readUploadedImage` 로 모든 파일의 크기·디코딩·형식을 먼저 검사해 `checkedAttachment` 로 모아 둔 뒤, 그제서야 디스크 쓰기와 INSERT 를 한 트랜잭션에서 수행합니다. 검사 단계는 10MB×20장이 힘에 올라오는 회귀를 피하려고 본문을 들고 있지 않고 저장 단계에서 다시 읽습니다.

실제 PostgreSQL 위의 새 시험이 고치기 전 코드에서 `stored 1 image(s)` 와 재업로드 후 `3 rows` 로 실패하는 것을 확인한 뒤 통과했고, 별도 트리에서 독립 재현했습니다. 응답 코드·문구·필드와 OpenAPI 는 그대로이며 프런트엔드는 무변경입니다. 마이그레이션·환경변수·설정 변경은 없습니다. `tx.Commit` 실패와 `STORAGE_FULL` 경로는 시험으로 밟지 않아 검증 범위 밖입니다.

v0.315.0 — IPv6 로 접속한 사용자가 로그인하고, 그 로그인 실패가 속도 제한에 세어집니다

`remoteHost` 가 `strings.Split(r.RemoteAddr, ":")[0]` 로 주소를 읽어, net/http 가 넘기는 IPv6 형태 `"[::1]:54321"` 에서 `"["` 를 주소로 돌려줬습니다. 이 값을 받는 곳 셋이 모두 `inet` 컬럼(`audit_logs` · `user_sessions` · `login_attempts`)이라 세 쓰기가 실패하고, 세 실패가 각각 조용했습니다 — 감사 기록은 버려지고, 비밀번호가 맞으면 `issueSession` 의 INSERT 가 실패해 `500 SESSION_ERROR`, 틀리면 `login_attempts` 행이 들어가지 않아 `auth.max_login_attempts` 가 영원히 도달하지 않았습니다. 바이너리는 `":8080"` 에 붙고 Linux 에서 그것은 IPv6 도 받으므로 듀얼 스택 망의 기본 동작에서 재현됩니다. 이제 `net.SplitHostPort` + `net.ParseIP` 로 읽고 주소로 읽히지 않으면 `""` 를 돌려줍니다(영역이 붙은 `fe80::1%eth0` 를 넘기는 `netip` 대신 `ParseIP`). `X-Forwarded-For` 는 그것이 주소일 때만 이기고 아니면 연결 주소가 남아, 헤더에 쓰레기값을 넣어 자기 실패 횟수를 지우는 두 번째 입구도 막았습니다.

새 시험 3개를 실제 IPv6 리스너(`net.Listen("tcp","[::1]:0")`)에 프로덕션 `app.Handler()` 를 올리고 실제 HTTP 클라이언트로 POST 해서 돌렸습니다 — `RemoteAddr` 을 손으로 넣지 않고 net/http 가 채우게 했습니다. 고치기 전 셋 모두 `SESSION_ERROR` 500 과 `counts 0 failures` 로 실패하고, 고친 뒤 통과하며, 옛 한 줄을 되돌려 같은 실패가 다시 나는 것까지 확인한 뒤 별도 트리에서 독립 재현했습니다. 응답 코드·문구·필드와 OpenAPI 는 그대로이며 프런트엔드는 무변경입니다. 마이그레이션·환경변수·설정 변경은 없습니다. 새 시험은 DB 와 IPv6 루프백이 둘 다 있어야 돌고 없으면 건너뛰며, `parseClientHost` 의 대괄호 처리를 실제 프록시 출력으로 확인한 것과 기존 저장 행과의 표기 비교는 검증 범위 밖입니다.

v0.316.0 — 주소 제한으로 차단된 로그인이 실제 남은 차단 시간을 말합니다

로그인을 막는 계수기는 계정당 실패와 주소당 실패(`auth.max_login_attempts_per_ip`) 둘인데, `loginThrottleFor` 는 남은 시간을 계정 기준 `min(created_at)` 하나로만 계산했습니다. 주소 제한으로 차단된 호출자는 자기 실패 기록이 없으므로(주소 계수기를 채운 것은 같은 주소로 보이는 다른 사람의 오타입니다) 그 값이 NULL 이고, `RetryAfter` 가 0 이 되어 응답이 `Retry-After: 1` 과 "1분 후에 다시 시도하세요" 를 내보냈습니다. 주소 차단은 `auth.lockout_minutes` 동안 유지되므로 안내를 따른 사용자는 비밀번호가 처음부터 맞았는데도 1분마다 같은 거부를 다시 만나며 15분을 보냅니다. 차단 기록도 계정 실패 횟수만 남겨, 주소 계수기가 거부한 모든 건이 운영자에게 `failures: 0` 으로 차단됨으로 읽혔습니다. 이제 한 질의에서 두 계수기의 `min(created_at)` 을 각각 읽어 실제로 차단한 쪽의 남은 시간을 쓰고, 둘 다 걸렸으면 늦게 풀리는 쪽을 따릅니다(짧은 쪽을 알려 주면 그 시간 뒤에 같은 거부를 다시 만납니다). 남은 시간 계산은 `windowRemaining` 한 곳으로 모았고, 차단 기록에는 `addressFailures` 를 `failures` 와 함께 남깁니다.

새 시험 2개를 실제 PostgreSQL 위에서 프로덕션 `Handler()` 를 지나는 실제 로그인 요청으로 돌렸습니다 — 한 시험 안의 모든 요청이 같은 클라이언트 주소에서 오는 것이 요점이며, 사무실 NAT 하나와 Reverse Proxy 하나가 주소 계수기가 존재하는 이유입니다. 남의 이름으로 주소 계수기를 채운 뒤 실패 기록이 없는 계

정이 맞는 비밀번호로 로그인해, `Retry-After` 가 14분 이상이고 문구가 `15분` 을 말하는지와 `auth.login_blocked` 행의 `failures` 가 0·`addressFailures` 가 3 인지를 봅니다. 고치기 전 코드에서 `Retry-After says 60s, but the address stays blocked for 15 minutes` 와 `reports <nil> failures for the address` 로 먼저 실패하는 것을 확인한 뒤 통과했습니다. 응답 코드·문구 형식·필드와 OpenAPI 는 그대로이며 프런트엔드는 무변경입니다. 마이그레이션·환경변수·설정 변경은 없고, 주소 제한은 기본 비활성이므로 켜지 않은 배치에서는 달라지는 것이 없습니다. 두 제한이 동시에 걸려 늦게 풀리는 쪽을 고르는 경로와 창을 실제로 기다려 차단이 풀리는 것은 시험으로 밟지 않아 검증 범위 밖입니다.

v0.317.0 — 첨부 두 곳: 옮긴 이미지가 목적지의 끝에 서고, 정리가 저장 중인 업로드를 지우지 않습니다

`updateAttachment` (`internal/app/attachments.go`)은 `placement` 만 바뀐 PATCH 에서 `sort_order` 를 그대로 뒀는데, 업로드 경로는 새 이미지를 그 `placement` **안에서만** `max(sort_order)+1` 로 셉니다. 그래서 한 그룹을 빠져나간 번호가 그 안에서 다시 쓰이고, 그 번호를 들고 돌아온 이미지가 목적지에 이미 있는 행과 **같은** `sort_order` 를 갖습니다. 그러면 목록과 내보낸 덱의 순서가 사용자가 고른 값이 아니라 `id` 로 정해지고, `AttachmentPanel` 의 위/아래 이동은 같은 값끼리 맞바꾸므로 사용자가 그 순서를 고칠 수도 없습니다. 이제 `placement` 가 현재 값과 다를 때만 목적지 그룹의 `coalesce(max(sort_order),-1)+1` 로 다시 세도록 같은 `UPDATE` 안에 `CASE` 를 넣었습니다 — 업로드 경로와 **같은 식**이라 두 입구가 한 그룹의 끝을 같은 값으로 읽고, `SET` 의 하위질의는 변경 전 스냅숏을 읽으므로 옮기는 행이 자기 자신을 세지 않아 추가 왕복도 경합도 없습니다. 보낸 `sortOrder` 는 그대로 이기고 (맞바꾸기 PATCH 의 동작은 변하지 않습니다), 같은 `placement` 를 다시 보내는 것은 이동이 아닙니다.

`cleanupAttachmentFiles` 는 `report_attachments` 의 행 수로 디렉터리가 고아인지 판단했습니다. 첨부 행은 모든 파일 검사가 끝난 뒤 한 트랜잭션에서 들어가므로, **첫 업로드가 디스크에 쓰이는 동안 그 보고서의 행 수는 0** 입니다. 기동 후와 30분 유지보수 주기에 도는 정리가 그 틈에 겹치면 저장 중인 이미지를 디렉터리째 지웠습니다. 이제 `weekly_reports` 에 그 보고서가 **있는지**를 묻고, 살아 있으면 첨부 행이 없거나 디렉터리가 비어 있어도 보존합니다 — 정리의 대상은 처음부터 삭제된 보고서의 디렉터리였고, 조회가 실패하면 역시 보존합니다. 실패한 업로드의 잔여 파일은 보고서가 삭제된 뒤 정리됩니다. 이 동작을 `docs/OPERATIONS.md` 의 보존 절에 적었습니다.

이동 쪽은 새 시험 3개, 정리 쪽은 새 시험 4개를 실제 PostgreSQL 위에서 돌렸습니다. 첨부는 모두 프로덕션 `Handler()` 를 지나는 실제 HTTP 업로드로 만들고, 정리는 그 위에서 `cleanupAttachmentFiles` 를 직접 불러 봅니다 — 행이 아직 보이지 않는 틈은 `LOCK TABLE report_attachments IN SHARE MODE` 로, 조회 실패는 취소된 컨텍스트로 만듭니다. 이동 쪽은 프런트가 실제로 내는 요청(업로드는 항상 `AFTER` , select 은 `placement` 만)을 그대로 따라 2장 업로드·B 이동·1장 업로드·C 이동의 네 단계로 중복 `sort_order` 를 재현했습니다. 고치기 전 `BEFORE holds b.png and c.png on the same sortOrder`

`1` 로 실패하고 고친 뒤 통과하며, `CASE` 를 잠시 되돌려 같은 실패가 다시 나는 것까지 확인했습니다. 회귀 시험 2개(보낸 `sortOrder` 가 그대로 저장되는지, 같은 `placement` 재전송이 순서를 건드리지 않는지)는 의도대로 고치기 전에도 통과합니다. 재사용 매개변수는 `PREPARE` 로 형을 확인해 `sqlparams_test.go` 의 허용 목록에 사유와 함께 적었습니다. 응답 본문·OpenAPI·소유권 검사·`caption` / `placement` 의 `coalesce` 동작은 그대로이며 프런트엔드와 `migrations/` 는 한 줄도 바뀌지 않았습니다. 마이그레이션 중 겹치는 정리와 운영 DB 에 이미 들어간 중복 `sort_order` 의 사후 정정은 범위 밖입니다 — 이 릴리즈는 새로 생기는 중복을 막습니다.

v0.318.0 — 임베딩 현황이 읽지 못한 집계를 "0건" 으로 답하지 않습니다

`embeddingStatus` (`internal/app/semantic.go`)는 `items` · `embedded` · `stale` 세 count 를 한 번에 읽는 `Scan` 의 오류를 `_ =` 로 버려, 질의가 실패하면 세 값이 제로값 그대로 `200` 으로 나갔습니다. 관리자 화면에서 그 응답은 "임베딩 0/0건" 이고, 그 한 문장이 **"임베딩할 항목이 하나도 없다"(=할 일 없음)와 "집계를 읽지 못했다"(=가서 고쳐야 함)를 글자 그대로 같게** 만들었습니다. 같은 파일의 `pendingEmbeddingCount` 도 같은 모양으로 0 을 돌려줬고, 그 값은 `rebuildEmbeddings` 가 화면에 "남은 건수" 로 쓰는 숫자라서 backlog 가 10만 건이어도 집계가 실패하면 "임베딩 4건을 생성했습니다. 남은 항목이 없습니다." 로 보고했습니다 — 운영자가 버튼을 다시 누르지 않게 만드는 한 문장입니다. 이제 `status` 에 `countsUnread` (`omitempty`)를 더하고 `Warn` 을 남기며, 부분 `Scan` 이 남긴 값까지 0 으로 되돌립니다(읽은 한 값이 읽지 못한 두 값 옆에 서면 같은 거짓말이 작은 글씨로 남습니다). `vectorAvailable` · `enabled` · `model` 은 그대로 보냅니다 — `500` 으로 바꾸면 그 세 값이 같이 사라져 읽지 못한 집계와 pgvector 없는 배포가 다시 같아지고, 둘은 가서 고칠 것이 다릅니다. `pendingEmbeddingCount` 는 `(int, error)` 가 되어 실패를 호출자에게 넘기고, `rebuildEmbeddings` 는 실패 시 `remaining` 을 키째로 빼고 `remainingUnread: true` 를 담으며 `200` 을 유지합니다(방금 생성한 건수는 사실입니다). 이름과 모양은 `rollup_handlers.go:227` 의 기존 `Unread` 관례를 따랐습니다.

새 시험 2개를 손으로 만든 대역 없이 프로덕션 `app.Handler()` 와 실제 PostgreSQL(pgvector) 위에서 돌렸습니다. 현황 쪽은 `report_item_embeddings` 를 rename 해 집계만 실패시킵니다. 다시 만들기 쪽은 두 질의가 같은 테이블을 읽어 그냥 rename 하면 배치가 먼저 실패해 `502` 가 되므로, **배치가 임베딩 게이트웨이에 가 있는 동안** `report_items` 를 rename 해 "배치는 성공하고 뒤따르는 집계만 실패하는" 모양을 만들었습니다 — 응답의 `embedded: 4` 가 배치의 성공을 증명하므로 실패한 단계가 집계임이 고정됩니다. 둘 다 `t.Cleanup` 으로 복원하며 `t.Parallel()` 을 붙이지 않습니다. 고치기 전 두 시험이 각각 `the counts could not be read and the card does not say so` 와 `the backlog count failed and the answer reports remaining=0 as a fact` 로 실패하고, 고친 뒤 플래그 한 줄씩을 되돌려 같은 실패가 다시 나는 것까지 확인했습니다. 정상 경로의 응답 본문은 그대로이며 시험이 정상 카드의 키 집합을 직접 단정해 고정합니다.

이 응답을 읽는 자리가 저장소 안에 실제로 두 곳(`AdminPage.tsx:44` 카드, `:47` 다시 만들기 알림) 있어 둘 다 분기를 더하고 `types.ts` 에 선택 필드를 더했습니다. 프런트엔드는 타입 검사·프로덕션 빌드·기존 vitest 까지만 확인했고 `AdminPage` 렌더 시험이 없어 화면 문구를 UI 행동으로 증명하지는 않았습니다 — 그 보강은 다음 회차의 몫입니다. `remaining` 을 키째로 빼는 설계이므로 이 키를 필수로 읽는 저장소 밖 클라이언트는 `undefined` 를 보게 되며, 외부 통합은 확인할 길이 없어 OpenAPI 설명에 적었습니다. `migrations/` ·환경변수·설정은 무변경이고, 같은 파일의 다른 `_ =` (`searchSemantic` 등)는 범위 밖으로 두었습니다.

v0.319.0 — 첨부 순서를 읽지 못한 업로드가 이미지를 저장하지 않습니다

`uploadAttachments` 가 다음 순서의 `Scan` 오류를 버려, 순서를 읽지 못해도 0번 이미지로 저장하고 `201` 을 반환했습니다. 공개 PATCH 로 `sortOrder=2147483647` 을 저장하면 PostgreSQL 의 `max(sort_order)+1` 이 넘쳐 이 경로를 재현할 수 있습니다. 이제 조회가 실패하면 트랜잭션과 이미지 파일 쓰기 전에 `500 QUERY_FAILED` 와 "첨부 이미지의 순서를 조회할 수 없습니다." 를 반환합니다.

실제 PostgreSQL 과 `newTestServer` / `App.Handler()` 를 거치는 새 시험 두 개로 거절 전후 ID·순서·건수·디스크 파일 목록·기존 이미지 GET 200 및 바이트 보존, PATCH 복구 뒤 같은 이미지의 단일 저장·그룹 끝 배치, BEFORE/AFTER 독립 max+1 과 201 응답을 검증했습니다. 고치기 전 실패·고친 뒤 통과·원본 복원 시 재실패를 확인했습니다. 프로덕션 변경은 `attachments.go` 한 파일이며 마이그레이션·환경변수·설정 변경은 없습니다. 기존 중복 순서 정리, PATCH 상한 제한, 동시 업로드 개수 상한은 이번 범위 밖이고 UI 클릭·새 오류 시나리오의 PPTX 출력은 별도로 검증하지 않았습니다.

v0.320.0 — 참조 수를 읽지 못한 첨부 삭제가 살아 있는 이미지 파일을 지우지 않습니다

`deleteAttachment` (`internal/app/attachments.go:498`)는 `SELECT count(*) FROM report_attachments WHERE stored_path=$1` 의 `Scan` 오류를 `_ =` 로 버렸습니다. 질의가 실패하면 `remaining` 이 제로값 0 으로 남고, 0 은 이 분기에서 곧 "아무도 이 파일을 참조하지 않는다" 라서 바로 아래 `os.Remove` 가 실행됩니다. 저장 경로가 `reportID/sha256.ext` 내용 주소(같은 파일 255-259행)이라 같은 보고서에 같은 이미지를 두 번 올리면 **두 행이 한 파일을 공유**하므로, 그 중 하나를 지우는 요청에서 집계가 실패하면 살아 있는 다른 행의 이미지가 사라집니다 — 시험이 증명한 피해는 파일 소실과 이미지 GET 의 `ATTACHMENT_FILE_MISSING` 404, 목록의 `available:false` 까지이고, 내보낸 덱에서 그 캡처가 빠지는 것은 코드를 읽어 따라간 추론이며 측정하지 않았습니다. 앞선 다섯 릴리즈가 고친 같은 계열의 `Scan` 오류 무시와 달리 이것은 표시 오류가 아니라 **되돌릴 수 없는 파일 소실**이며, 응답은 `204` 이고 로그가 한 줄도 없어 운영자가 원인을 알 길이 없었습니다.

행 `DELETE` 는 이 지점에서 이미 커밋돼 되돌릴 수 없으므로 응답은 `204` 그대로 두고, 집계 실패 시 `os.Remove` 를 건너뛰어 파일을 보존하면서 `attachmentId` · `error` · `trace` 만 `Warn` 으로 남깁니다

(`stored_path` 원문은 로그에 넘기지 않으며 시험이 파일 이름이 로그에 나오지 않는 것을 단정합니다). `500` 으로 바꾸는 선택은 하지 않았습니다 — 행은 실제로 지워졌으므로 실패를 답하면 거짓말이 됩니다. 시험은 `BEFORE DELETE ... FOR EACH ROW` 트리거 안에서 테이블 이름을 바꿔 `DELETE ... RETURNING` 은 성공하고 뒤따르는 집계만 실패하는 실제 모양을 만들고(PG16 에서 트리거 안의 `ALTER TABLE ... RENAME` 이 허용되는 것을 임시 테이블로 먼저 실측했습니다), 손으로 만든 대역 없이 `newTestServer` / `app.Handler()` 와 실제 PostgreSQL 로 HTTP 를 거칩니다. 정상 3경로(참조가 남음·마지막 참조·파일이 이미 없음)도 각각 단정해 바꾸지 말아야 할 동작을 고정했으며, 트리거·함수·테이블 이름은 `t.Cleanup` 과 명시적 `restore()` 로 되돌립니다. 프로덕션 변경은 `attachments.go` 한 파일의 한 줄이고 `migrations/`·환경변수·설정·프런트엔드는 무변경입니다. 남은 우려는 이 교환이 만드는 반대쪽입니다 — 집계 실패로 건너뛴 파일이 실제로 마지막 참조였다면 `cleanupAttachmentFiles`(같은 파일 637-664 행)가 보고서 행이 사라진 디렉터리만 회수하므로 영구 고아가 됩니다. 쓰이지 않는 파일이 남는 것은 되돌릴 수 있고 쓰이는 파일이 사라지는 것은 되돌릴 수 없어 이 쪽을 택했지만, 회수 경로의 보강은 다음 회차의 몫입니다. 이미 사라진 파일의 복구, 질의 오류를 버리는 나머지 `Scan` 전수 조사, `serveAttachment` 가 아직 `storedPath` 원문을 로그에 쓰는 별건, 동시 업로드 개수 상한도 이번 범위 밖입니다.